用于决策的算法

用于决策的算法

Mykel J. Kochenderfer
Tim A. Wheeler Kyle H.
Wray

The MIT Press
Cambridge, Massachusetts
London, England

© 2022 麻省理工学院

T他的作品受知识共享署名-非商业性使用-禁止演绎（CC-BY-NC-ND）许可协议的约束。在此许可下，所有权利保留。



MIT 出版社感谢对本书草稿提出意见的匿名同行评审专家。学术专家的慷慨工作对于确立我们出版物的权威性和质量至关重要。我们感激这些未署名的读者所做出的贡献。

本书由作者使用 LATEX 中的 TEX Gyre Pagella 字体排版。在美国印刷和装订。

美国国会图书馆在版编目数据

Names：Kochenderfer, Mykel J., 1980– 作者。| Wheeler, Tim A. (Tim Allan)，作者。| Wray, Kyle H.，作者。题名：决策算法 / Mykel J. Kochenderfer, Tim A. Wheeler, Kyle H. Wray。描述：剑桥 : 麻省理工学院, [2022] |

包含参考书目和索引。标识符：LCCN 2021038701 | ISBN 9780262047012（精装本）主题：LCSH：决策支持系统——数学。| 算法。分类：LCC T58.62 .K666 2022 | DDC 658.4/03—dc23 LC记录可访问 https://lccn.loc.gov/2021038701

10 9 8 7 6 5 4 3 2 1

To our families

Contents

Preface xix
Acknowledgments xxi

1 *Introduction* 1 1.1 决策 1 1.2 应用 2 1.3 方法 5 1.4 历史 7 1.5 社会影响 12 1.6 概述 14

部分概率推理

2 *Representation* 19 2.1 信念度与概率 19 2.2 概率分布 20 2.3 联合分布 24 2.4 条件分布 29 2.5 贝叶斯网络 32 2.6 条件独立 35 2.7 总结 36 2.8 练习 38

3 Inference 43 3.1 贝叶斯网络中的推理 43 3.2 朴素贝叶斯模型中的推理 48 3.3 和-积变量消元 49 3.4 信念传播 53 3.5 计算复杂性 53 3.6 直接采样 54 3.7 似然加权采样 57 3.8 吉布斯采样 60 3.9 高斯模型中的推理 63 3.10 总结 65 3.11 练习 66 *4 Parameter Learning* 71 4.1 最大似然参数学习 71 4.2 贝叶斯参数学习 75 4.3 非参数学习 82 4.4 缺失数据学习 82 4.5 总结 89 4.6 练习 89 *5 Structure Learning* 97 5.1 贝叶斯网络评分 97 5.2 有向图搜索 99 5.3 马尔可夫等价类 103 5.4 部分有向图搜索 104 5.5 总结 106 5.6 练习 107 *6 Simple Decisions* 111 6.1 理性偏好约束 111 6.2 效用函数 112 6.3 效用引出 114 6.4 最大期望效用原理 116 6.5 决策网络 116 6.6 信息价值 119 6.7 非理性 122 6.8 总结 125 6.9 练习 125

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

PART II SEQUENTIAL PROBLEMS

7 *Exact Solution Methods* 133 7.1 马尔可夫决策过程 133 7.2 策略评估 136 7.3 值函数策略 139 7.4 策略迭代 140 7.5 值迭代 141 7.6 异步值迭代 145 7.7 线性规划形式 147 7.8 具有二次奖励的线性系统 147 7.9 总结 150 7.10 练习 151

8 *Approximate Value Functions* 161 8.1 参数表示 161 8.2 最近邻 163 8.3 核平滑 164 8.4 线性插值 167 8.5 单纯形插值 168 8.6 线性回归 172 8.7 神经网络回归 174 8.8 总结 175 8.9 练习 177

9 *Online Planning* 181 9.1 滚动时域规划 181 9.2 前瞻与模拟推演 183 9.3 前向搜索 183 9.4 分支定界 185 9.5 稀疏采样 187 9.6 蒙特卡洛树搜索 187 9.7 启发式搜索 197 9.8 标记启发式搜索 197 9.9 开环规划 200 9.10 总结 208 9.11 练习 209

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

10 Policy Search 213 10.1 近似策略评估 213 10.2 局部搜索 215 10.3 遗传算法 215 10.4 交叉熵方法 218 10.5 进化策略 219 10.6 各向同性进化策略 224 10.7 总结 226 10.8 练习 226
11 Policy Gradient Estimation 231 11.1 有限差分 231 11.2 回归梯度 234 11.3 似然比 234 11.4 奖励至目标 237 11.5 基线减法 241 11.6 总结 245 11.7 练习 246

12 Policy Gradient Optimization 249 12.1 梯度上升更新 249 12.2 限制梯度更新 251 12.3 自然梯度更新 253 12.4 信赖域更新 254 12.5 截断代理目标 257 12.6 总结 263 12.7 练习 264

13 Actor-Critic Methods 267 13.1 演员-评论家算法 267 13.2 广义优势估计 269 13.3 确定性策略梯度 272 13.4 蒙特卡洛树搜索的演员-评论家算法

13.5 总结 277 13.6 练习 277

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

14 Policy Validation 281 14.1 性能指标评估 281 14.2 罕见事件模拟 285 14.3 鲁棒性分析 288 14.4 权衡分析 289 14.5 对抗分析 291 14.6 小结 295 14.7 练习 295

第三部分 模型不确定性

15 Exploration and Exploitation 299

15.1 赌博机问题 299 15.2 贝叶斯模型估计 301 15.3 无向探索策略 301 15.4 有向探索策略 303 15.5 最优探索策略 306 15.6 多状态探索 309 15.7 小结 309 15.8 练习 311

16 Model-Based Methods 317 16.1 最大似然模型 317 16.2 更新方案 318 16.3 探索 321 16.4 贝叶斯方法 326 16.5 贝叶斯自适应马尔可夫决策过程 329 16.6 后验采样 330 16.7 总结 332 16.8 练习 332

17 Model-Free Methods 335 17.1 均值的增量估计 335 17.2 Q学习 336 17.3 Sarsa 338 17.4 资格迹 341

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

17.5 奖励塑造 343 17.6 动作值函数近似 343 17.7 经验回放 345 17.8 总结 348 17.9 练习 351

18 Imitation Learning 355 18.1 行为克隆 355 18.2 数据集聚合 358 18.3 随机混合迭代学习 358 18.4 最大边际逆强化学习 361 18.5 最大熵逆强化学习 365 18.6 生成对抗模仿学习 369 18.7 总结 371 18.8 练习 372

part iv 状态不确定性

19 Beliefs 379 19.1 信念初始化 379 19.2 离散状态滤波器 380 19.3 卡尔曼滤波器 383 19.4 扩展卡尔曼滤波器 385 19.5 无迹卡尔曼滤波器 387 19.6 粒子滤波器 390 19.7 粒子注入 394 19.8 总结 395 19.9 练习 397

20 Exact Belief State Planning 407 20.1 信念状态马尔可夫决策过程 407 20.2 条件计划 408 20.3 Alpha向量 411 20.4 剪枝 412 20.5 价值迭代 416 20.6 线性策略 419 20.7 总结 419 20.8 练习 422

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

21 *Offline Belief State Planning* 427

21.1 全observable值近似 427 21.2 快速知情界 429 21.3 快速下界 430 21.4 基于点的值迭代 431 21.5 随机化基于点的值迭代 433 21.6 锯齿上界 436 21.7 点选择 440 21.8 锯齿启发式搜索 442 21.9 三角化值函数 445 21.10 总结 447 21.11 练习 448

22 *Online Belief State Planning* 453 22.1 使用推演的前瞻 453 22.2 前向搜索 453 22.3 分支定界 456 22.4 稀疏采样 456 22.5 蒙特卡洛树搜索 457 22.6 确定化稀疏树搜索 459 22.7 间隙启发式搜索 460 22.8 总结 464 22.9 练习 467

23 *Controller Abstractions* 471 23.1 控制器 471 23.2 策略迭代 475 23.3 非线性规划 478 23.4 梯度上升 481 23.5 总结 486 23.6 练习 486

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

第五部分 多智能体系统

24 Multiagent Reasoning 493 24.1 简单游戏 493 24.2 响应模型 494 24.3 占优策略均衡 497 24.4 纳什均衡 498 24.5 相关均衡 498 24.6 迭代最佳响应 503 24.7 分层softmax 504 24.8 虚拟对弈 505 24.9 梯度上升 509 24.10 总结 509 24.11 练习 511

25 Sequential Problems 517 25.1 马尔可夫博弈 517 25.2 响应模型 519 25.3 纳什均衡 520 25.4 虚拟博弈 521 25.5 梯度上升 526 25.6 纳什Q学习 526 25.7 总结 528 25.8 练习 530

26 State Uncertainty 533 26.1 部分可观察马尔可夫博弈 533 26.2 策略评估 535 26.3 纳什均衡 537 26.4 动态规划 540 26.5 总结 542 26.6 练习 542

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

27 Collaborative Agents 545 27.1 分布式部分可观测马尔可夫决策过程 545 27.2 子类 546 27.3 动态规划 549 27.4 迭代最佳响应 550 27.5 启发式搜索 550 27.6 非线性规划 551 27.7 总结 554 27.8 练习 556

附录

A Mathematical Concepts 561 A.1 测度空间 561 A.2 概率空间 562 A.3 度量空间 562 A.4 赋范向量空间 562 A.5 正定性 564 A.6 凸性 564 A.7 信息量 565 A.8 熵 566 A.9 交叉熵 566 A.10 相对熵 567 A.11 梯度上升 567 A.12 泰勒展开 568 A.13 蒙特卡洛估计 569 A.14 重要性采样 570 A.15 收缩映射 570 A.16 图 572 *B Probability Distributions* 573 *C Computational Complexity* 575 C.1 渐近符号 575 C.2 时间复杂度类 577 C.3 空间复杂度类 577 C.4 可判定性 579

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

D Neural Representations 581 D.1 神经网络 581 D.2 前馈网络 582 D.3 参数正则化 585 D.4 卷积神经网络 587 D.5 循环网络 588 D.6 自编码器网络 592 D.7 对抗网络 594 *E Search Algorithms* 599 E.1 搜索问题 599 E.2 搜索图 600 E.3 前向搜索 600 E.4 分支定界 601 E.5 动态规划 604 E.6 启发式搜索 604 *F Problems* 609 F.1 六边形世界 609 F.2 2048 610 F.3 倒立摆 611 F.4 山地车 612 F.5 简单调节器 613 F.6 飞机防撞 614 F.7 哭闹婴儿 615 F.8 机器更换 617 F.9 接球 619 F.10 囚徒困境 621 F.11 石头剪刀布 621 F.12 旅行者困境 622 F.13 捕食者-猎物六边形世界 623 F.14 多照顾者哭闹婴儿 624 F.15 协作捕食者-猎物六边形世界 625

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

G Julia 627 G.1 类型 627 G.2 函数 64
0 G.3 控制流 643 G.4 包 645 G.5 便捷函数
648 *References* 651 *Index* 671

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Preface

本书广泛介绍了不确定性下决策的算法。我们涵盖了与决策相关的广泛主题，介绍了底层的数学问题表述及其求解算法。书中提供了图表、示例和练习，以传达各种方法背后的直觉。

本书面向高年级本科生、研究生以及专业人士。它要求读者具备一定的数学成熟度，并假定读者此前接触过多变量微积分、线性代数和概率概念。附录中提供了一些复习材料。这本书尤其适用的学科包括数学、统计学、计算机科学、航空航天、电气工程和运筹学。

本教材的核心在于算法，所有算法均使用Julia编程语言实现。我们发现，这种语言非常适合以人类可读的形式描述算法。在算法实现的设计中，我们优先考虑的是可解释性而非效率。例如，工业应用可能会从其他实现方式中获益。本书相关的代码片段被授权免费使用，但需注明代码来源。

Mykel J. Kochenderfer
Tim A. Wheeler
Kyle H. Wray
Stanford, California
February 28, 2022

Acknowledgments

本教材源于斯坦福大学开设的一门关于不确定性条件下决策的课程。我们感谢过去六年中帮助塑造该课程的学生和助教们。

作者希望感谢许多为我们的手稿早期草稿提供了宝贵反馈的人，包括Dylan Asmar、Drew Bagnell、Safa Bakhshi、Edward Balaban、Jean Betterton、Raunak Bhattacharyya、Kelsey Bing、Maxime Bouton、Hugo Buurmeijer、Austin Chan、Simon Chauvin、Shushman Choudhury、Jon Cox、Matthew Daly、Victoria Dax、Harrison Delecki、Richard Dewey、Paul Diederichs、Apoorva Dixit、Dea Dressel、Ben Duprey、Torstein Eliassen、Johannes Fischer、Rushil Goradia、Jayesh Gupta、Griffin Holt、Arec Jamgochian、Rohan Kapre、Mark Koren、Liam Kruse、Jonathan Larkin、Tor Lattimore、Bernard Lange、Ritchie Lee、Sheng Li、Michael Littman、Robert Moss、Joshua Ott、Emma Passmore、Oriana Peltzer、Francesco Piccoli、Nikhil Raghuraman、Jeffrey Sarnoff、Marc Schlichting、Ransalu Senanayake、Chelsea Sidrane、Michael Sheehan、Chris Strong、Zach Sunberg、Abiy Teshome、Alexandros Tzikas、Kemal Ure、Ziyu Wang、Josh Wolff、Zhen Wu、An l Y ld z、Shengtong Zhang和Zongzhang Zhang。我们还要感谢Sydney Katz、Kunal Menda和Ayan Mukhopadhyay对第1章讨论的贡献。Ross Alexander制作了本书中的许多练习。与MIT Press的Elizabeth Swayze合作准备本手稿的出版是一件愉快的事。

本书的风格灵感源自爱德华·塔夫特。在众多风格元素中，我们采用了他那宽阔的页边距以及小多图的使用。本书的排版基于Kevin Godby、Bil Kleb和Bill Wood开发的Tufte-LaTeX宏包。书籍的颜色方案改编自Sublime Text（sublimetext.com）的Jon Skinner设计的Monokai主题及一个更佳的调色板。

适应色盲个体。[1] 对于图表，我们使用viridis。
由Stéfan van der Walt和Nathaniel Smith定义的色彩映射。

我们也受益于本教科书所依赖的各种开源包（参见附录G）。代码的排版得到了pythontex的帮助，该工具由Geoffrey Poore维护。算法所使用的字体是JuliaMono（github.com/cormullion/juliamono）。绘图工作由pgfplots处理，它是由Christian Feuersänger维护的。

1 B. Wong，“观点：色盲”，*Nature Methods*，第8卷，第6期，第441-442页，2011年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

1 Introduction

许多重要问题都涉及不确定性下的决策，包括飞机防撞、野火管理和灾害响应。在设计自动化决策系统或决策支持系统时，考虑各种不确定性来源并仔细平衡多个目标非常重要。我们将从计算的角度讨论这些挑战，旨在提供决策模型和计算方法背后的理论。本章介绍不确定性下的决策问题，提供一些应用示例，并概述计算方法的范围。然后总结各个学科如何促进我们对智能决策的理解，并强调具有潜在社会影响的领域。最后给出本书后续内容的概要。

1.1 Decision Making

智能体是一种基于对其环境的观察而行动的实体。智能体可以是物理实体，如人或机器人，也可以是非物理实体，如完全以软件实现的决策支持系统。如图1.1所示，智能体与环境之间的交互遵循*observe-act cycle*或*loop*。

在 t 时刻，智能体接收环境的 *observation*，记为 o_t。观测可以通过例如生物感知过程（如人类）进行，也可以通过传感器系统（如空中交通管制系统中的雷达）进行。观测往往不完整或带有噪声；人类可能看不到正在接近的飞机，或者雷达系统可能因电磁干扰而漏掉一次检测。随后，智能体通过某种决策过程选择一个动作 a_t。

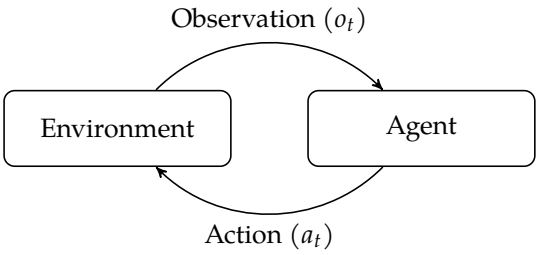


图1.1. 代理与其环境之间的交互。

此操作（例如发出警报）可能对环境产生不确定的影响。

我们的关注点在于那些能够智能交互以随时间实现其目标的代理。给定过去的观察序列$o_1, \ldots, o_t$，以及对环境的了解，代理必须在存在各种不确定性来源的情况下选择最能实现其目标的行动a_t，这些不确定性来源包括：

- *outcome uncertainty*，其中我们行动的影响是不确定的，
- *model uncertainty*，其中我们对问题的模型不确定，
- *state uncertainty*，其中环境的真实状态是不确定的，并且
- *interaction uncertainty*，其中环境中其他智能体的行为是不确定的。

本书围绕这四种不确定性来源组织内容。在不确定性存在的情况下做出决策是*artificial intelligence*、[2]领域的核心，因为
以及其他许多领域，如第1.4节所述。我们将讨论多种算法，即计算过程的描述
，用于制定对不确定性具有鲁棒性的决策。

1 我们这里关注离散时间问题。连续时间问题在*control theory*领域中进行研究。参见D. E. Kirk, *Optimal Control Theory: An Introduction*。Prentice-Hall，1970年。

2 人工智能的全面介绍由S. Russell和P. Norvig提供，*Artificial Intelligence: A Modern Approach*，第4版。Pearson，2021年。

1.2 *Applications*

上一节提出的决策框架可应用于多种领域。本节讨论几个具有实际应用的概念性示例。附录F概述了贯穿本文用于演示我们所讨论算法的其他示例问题。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

1.2.1 *Aircraft Collision Avoidance*

为了帮助防止飞机之间的空中相撞，我们希望设计一个系统，能够提醒飞行员潜在的威胁，并指导他们如何机动以避开威胁。[3] 该系统与其他飞机的应答器通信以一定程度的准确性确定其位置。决定向飞行员提供何种指导具有挑战性。飞行员响应的速度以及他们遵守指导的积极程度存在不确定性。此外，其他航空器的行为也存在不确定性。我们希望我们的系统能够足够早地发出警报，为飞行员提供足够的时间来操纵航空器以避免碰撞，但我们不希望系统过早发出警报，这将导致许多不必要的操纵。由于该系统将在全球范围内持续使用，我们需要系统提供卓越的安全水平。

3 此应用在M. J. Kochenderfer所著的《碰撞避免》一章中有所讨论，*De-*。麻省理工学院出版社，2015 *cision Making Under Uncertainty: Theory and Application*。

1.2.2 *Automated Driving*

我们希望构建一辆能够在城市环境中安全行驶的自动驾驶汽车。[4]该车辆必须依靠一套传感器来感知其环境，在为了做出安全的决策。一种传感器是激光雷达，它涉及测量环境中的激光反射以确定与障碍物的距离。另一种传感器是摄像头，通过计算机视觉算法，可以检测行人和其它车辆。这两种类型的传感器都不完美，易受噪声和遮挡的影响。例如，一辆停着的卡车可能会遮挡一个可能正试图在人行横道过马路的行人。我们的系统必须根据其它车辆、行人和其他道路使用者的可观察行为来预测他们的意图和未来路径，以便安全地导航到我们的目的地。

4 类似的应 用由 M. Bouton、A. Nakhaei、K. Fujimura 和 M. J. Kochenderfer 在“Safe Reinforcement Learning with Scene Decomposition for Navigating Complex Urban Environments,” 于 *IEEE Intelligent Vehicles Symposium*, 2019 中进行了探索。

1.2.3 *Breast Cancer Screening*

在全球范围内，乳腺癌是女性中最常见的癌症。早期发现乳腺癌有助于挽救生命，而乳房X光检查是最有效的筛查工具。然而，乳房X光检查也伴随着潜在风险，包括假阳性，这可能导致不必要且具侵入性的诊断后续。多年来的研究已基于年龄制定了多种基于人群的筛查时间表，以平衡检测的益处与风险。开发一个能够根据个人风险提出建议的系统

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

特征和筛查史有可能带来更好的健康结果。[5] 这种系统的成功可以与全人群

筛查方案在总预期质量调整生命年、乳腺X线摄影检查次数、假阳性发生率以及未检测到的浸润性癌症风险方面的效果。

5 这个概念由T. Ayer、O. Alagoz和N. K. Stout提出，“个性化乳腺X线摄影筛查决策的POMDP方法”，*Operations Research*，第60卷，第5期，第1019–1034页，2012年。

1.2.4 *Financial Consumption and Portfolio Allocation*

假设我们想要构建一个系统，推荐个人当年应消费其财富的多少以及应投资多少。[6] 投资组合可能包括股票和债券，具有不同的

风险水平和预期回报。财富的演变因收入和投资收入的不确定性而具有随机性，通常在投资者接近退休前增加，然后稳步减少。一年内消费一单位财富所带来的享受通常随着消费量的增加而减少，从而导致个人希望在其一生中平滑消费。

6 一个相关问题由 R. C. Merton 在“连续时间模型中的最优消费与投资组合规则”中进行了研究，*Journal*，第3卷，第4期，第373–413页，1971年。

1.2.5 *Distributed Wildfire Surveillance*

扑救野火时，态势感知是一项重大挑战。火势随时间演变，受风力和环境中燃料分布等因素影响。许多野火跨越广阔地理区域。监测野火的一个构想是使用配备传感器的无人机团队在火场上空飞行。[7] 单个无人机的感知范围有限，但

来自团队的信息可以融合，以提供态势的统一快照，从而驱动资源分配决策。我们希望团队成员能够自主决定如何相互协作，以提供对火灾的最佳覆盖。有效的监测需要决定如何机动以覆盖那些新的传感器信息可能有用的区域；将时间花在我们确定火灾是否正在燃烧的区域将是浪费的。识别需要探索的重要区域需要仅根据对火灾当前状态的不完全了解，对其随机演化进行推理。

7 该应用由K. D. Julian和M. J. Kochenderfer在“利用深度强化学习的自主飞行器分布式野火监控”中探讨，*AIAA Journal of Guidance, Control, and Dynamics*，第42卷，第8期，第1768–1778页，2019年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

1.2.6 Mars Science Exploration

火星车已经在火星上取得了重要发现，并增进了我们对火星的了解。然而，科学探索中的一个主要瓶颈一直是火星车与地球上操作团队之间的通信链路。传感器信息从火星发送到地球，以及命令从地球发送到火星，可能需要长达半个小时。此外，由于作为两颗行星之间信息中继的轨道器位置所限，与火星之间的上传和下载窗口有限，因此需要提前规划对火星车的引导。近期研究表明，通过引入更高程度的自主性，科学探索任务的效率可以提高五倍。[8] 人类操作员仍将提供高层

关于任务目标的指导，但探测器将具有灵活性，能够利用最新信息选择自己的科学目标。此外，理想情况下，探测器应能对各种危险和系统故障做出适当反应，无需人工干预。

8 这一概念由 D. Gaines、G. Doran、M. Paton、B. Rothrock、J. Russino、R. Mackey、R. Anderson、R. Francis、C. Joswig、H. Justice、K. Kolcio、G. Rabideau、S. Schaffer、J. Sawoniewicz、A. Vasavada、V. Wong、K. Yu 和 A.-a. Agha-mohammadi 提出并评估，“自给自足漫游车以提高任务生产率”，*Journal of Field Robotics*，第 37 卷，第 7 期，第 1171–1196 页，2020 年。

1.3 Methods

设计决策智能体有多种方法。根据具体应用，某些方法可能比其他方法更合适。这些方法在设计者的职责和留给自动化的任务方面有所不同。本节简要概述这些方法的集合。本书将主要关注规划和强化学习，但其中一些技术将涉及监督学习和优化的要素。

1.3.1 Explicit Programming

设计决策智能体最直接的方法是预想智能体可能遇到的所有场景，并明确编写智能体应对每种场景的行动程序。显式编程方法对于简单问题可能效果良好，但它给设计者带来了提供完整策略的沉重负担。为了使智能体编程更容易，人们已经提出了各种智能体编程语言和框架。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

1.3.2 *Supervised Learning*

对于一些问题，向智能体展示该做什么可能比编写一个让智能体遵循的程序更容易。设计者提供一组训练示例，自动化学习算法必须从这些示例中进行泛化。这种方法被称为 *supervised learning*，并已广泛应用于分类问题。当应用于学习从观察到动作的映射时，这种技术有时被称为 *behavioral cloning*。当专家设计者确实知道对一组具有代表性情境的最佳行动方案时，行为克隆效果很好。尽管存在各种各样的不同学习算法，但在新情境中，它们通常无法比人类设计者表现得更好。

1.3.3 *Optimization*

另一种方法是让设计者指定可能的决策策略空间以及一个需要最大化的性能指标。评估决策策略的性能通常涉及运行一批模拟。随后，优化算法在这个空间中搜索最优策略。如果空间相对较小且性能指标没有太多局部最优解，那么各种局部或全局搜索方法可能都适用。尽管通常假设需要一个动态模型来运行模拟，但除此之外，它并不用于指导搜索，这一点对于复杂问题来说可能很重要。

1.3.4 *Planning*

Planning 是一种利用问题动态模型来指导搜索的优化形式。大量文献探讨了各种规划问题，其中很多都集中在确定性问题。对于某些问题，用确定性模型近似动态可能是可以接受的。假设有确定性模型，我们就可以使用更容易扩展到高维问题的方法。对于其他问题，考虑未来不确定性则至关重要。本书完全聚焦于考虑不确定性很重要的问题。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

1.3.5 *Reinforcement Learning*

Reinforcement learning 放松了规划中关于模型提前已知的假设。相反，决策策略是在智能体与环境交互的过程中学习的。设计者只需提供一个性能度量；学习算法负责优化智能体的行为。强化学习中一个有趣的复杂性在于，动作的选择不仅影响智能体实现目标的即时成功，还影响智能体学习环境并识别可利用问题特征的能力。

1.4 *History*

决策自动化过程的理论根植于早期哲学家、科学家、数学家和作家的梦想。早在公元前800年，古希腊人就开始将自动化融入神话和故事中。*automaton*一词最早出现在荷马的*Iliad*中，其中提到了自动机器的概念，包括用于招待晚餐客人的机械三脚架。[9] 在17世纪，哲学家们提出了使用逻辑

自动解决分歧的规则。他们的想法为机械化推理奠定了基础。

从十八世纪末开始，发明家们开始创造出能够执行劳动的自动机器。特别是在纺织工业中，一系列创新导致了自动织布机的发展，这进而为第一批工厂机器人奠定了基础。[10] 在十九世纪初，

智能机器自动化劳动开始进入科幻小说。*robot*一词起源于捷克作家卡雷尔·恰佩克的戏剧《*R.U.R.*》，是*Rossum's Universal Robots*的缩写，讲述能够执行人类不愿做的工作的机器。这部戏剧启发了其他科幻作家将机器人融入他们的作品。二十世纪中叶，著名作家兼教授艾萨克·阿西莫夫在其著名的《*Robot*》系列中阐述了他对机器人技术的愿景。

在实际实施自动化决策时，一个主要挑战在于处理不确定性。即便到了二十世纪末，因开发单纯形算法而最为人熟知的乔治·丹齐格，在1991年仍指出：

9 S. Vasileiadou、D. Kalligeropou-los 和 N. Karcanias，“古希腊的系统、建模与控制：第一部分：神话中的自动机，”*Measurement and Control*，第 36 卷，第 3 期，第 76–80 页，2003 年。

10 N. J. Nilsson，*The Quest for Artifi-cial Intelligence*。剑桥大学出版社，2009年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

> 回顾起来，有趣的是，最初激发我研究的问题至今仍然悬而未决——即随时间动态进行规划或调度的问题，特别是在不确定性下的动态规划。如果这样的问题能够成功解决，它（最终通过更好的规划）可能有助于世界的福祉与稳定。[11]

11 G. B. Dantzig, “线性规划”, *Operations Research*, 第50卷, 第1期, 第42–47页, 2002年。

尽管不确定性下的决策仍然是一个活跃的研究领域，但过去几个世纪以来，研究人员和工程师们已越来越接近于实现这些早期梦想家提出的概念。当前最先进的决策算法依赖于多个学科领域发展出的概念的融合，包括经济学、心理学、神经科学、计算机科学、工程学、数学和运筹学。本节重点介绍了这些学科的一些主要贡献。学科之间的交叉融合促成了许多近期的进展，并且很可能在未来继续支持增长。

1.4.1 *Economics*

经济学需要人类决策模型。构建此类模型的一种方法涉及效用理论，该理论最早于十八世纪末被提出。[12]效用理论提供了一种建模和比较

12 G. J. Stigler, “效用理论的发展。I”, *Journal of Political Economy*, 第58卷, 第4期, 第307–327页, 1950年。

各种结果的可取性。例如，效用可用于比较货币量的可取性。在*Theory of Legislation*中，杰里米·边沁总结了货币效用的非线性：

> *1st.* 每一份财富都有相应的幸福。*2nd.* 在两个财富不等的人中，拥有最多财富的人拥有最多的幸福。*3rd.* 较富者的幸福之 excess 不会像他的财富之 excess 那样大。[13]

13 J. Bentham, *Theory of Legislation*. Trübner & Company, 1887.

通过将效用概念与理性决策理念相结合，二十世纪中叶的经济学家为最大预期效用原则奠定了基础。这一原则是自主决策代理创建背后的关键概念。效用理论还促进了博弈论的发展，博弈论试图理解多个代理在彼此存在的情况下为实现自身利益最大化而行动的行为。[14]

14 O. Morgenstern 和 J. von Neumann, *Theory of Games and Economic Behavior*。普林斯顿大学出版社，1953年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

1.4.2 *Psychology*

心理学家还研究人类决策，通常是从人类行为的角度出发。通过研究动物对刺激的反应，心理学家自19世纪以来一直在发展试错学习理论。研究人员注意到，动物倾向于根据它们在以往类似情境中所体验到的满足或不适来做出决定。俄罗斯心理学家伊万·巴甫洛夫在观察狗被喂食时的唾液分泌模式后，将这一思想与强化概念结合起来。心理学家发现，通过对特定刺激进行持续强化，可以增强或削弱某种行为模式。20世纪中期，数学家和计算机科学家艾伦·图灵提出了允许机器以同样方式学习的可能性：

> 将机器组织成通用机器，如果干扰的布置涉及很少的输入，那将是最令人印象深刻的。人类儿童的教育在很大程度上依赖于奖励和惩罚系统，这表明应该可能仅通过两个干扰输入来完成组织工作，一个用于“快乐”或“奖励”（R），另一个用于“痛苦”或“惩罚”（P）。[15]

15 A. M. Turing，《智能机器》，国家物理实验室，报告，1948年。

心理学家的工作为强化学习领域奠定了基础，这是一种用于教导智能体在不确定环境中做出决策的关键技术。[16]

16 R. S. Sutton和A. G. Barto，*Reinforcement Learning: An Introduction*，第2版，MIT出版社，2018年。

1.4.3 *Neuroscience*

当心理学家研究人类行为的发生过程时，神经科学家则专注于产生这些行为的生物过程。十九世纪末，科学家发现大脑由相互连接的神经元网络组成，这一网络赋予了大脑感知世界和进行推理的能力。人工智能先驱尼尔森·尼尔森（Nils Nilsson）将这些发现应用于决策过程的描述如下：

> 因为正是动物的 *brain* 负责将感觉信息转化为行动，所以可以预期，在研究大脑及其基本组成部分神经元的过程中，神经生理学家和神经解剖学家的工作中会涌现出许多好想法。[17]

17 N. J. Nilsson，*The Quest for Artifi-cial Intelligence*。剑桥大学出版社，2009年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

20世纪40年代，研究人员首次提出，神经元可以被视为单独的“逻辑单元”，当它们组合成网络时能够执行计算操作。这项工作为神经网络奠定了基础，而神经网络在人工智能领域中被用于执行各种复杂任务。

1.4.4 Computer Science

二十世纪中叶，计算机科学家们开始将智能决策问题构想为通过形式逻辑进行符号操作的问题。二十世纪中叶编写的用于执行自动推理的计算机程序“逻辑理论家”运用这种思维方式来证明数学定理。其发明者之一赫伯特·西蒙通过将程序的符号性质与人类思维相联系，阐述了这一特性：

> 我们发明了一个能够进行非数值思维的计算机程序，从而解决了古老的心身问题，解释了由物质构成的系统如何能具有心灵的特性。[18]

18 由 J. Agar 引用，*Science in the 20th Century and Beyond*，2012 年。

这些符号系统严重依赖人类专业知识。一种被称为*connectionism*的替代智能方法，部分受到神经科学发展的启发，侧重于使用人工神经网络作为智能的基础。了解到神经网络可以训练用于模式识别后，连接主义者试图从数据或经验中学习智能行为，而非依赖专家的硬编码知识。连接主义范式支撑了AlphaGo的成功——这个自主程序在围棋比赛中击败了人类职业选手，也支撑了自动驾驶汽车的大量发展。结合符号主义和连接主义范式的算法至今仍是一个活跃的研究领域。

1.4.5 Engineering

工程领域一直致力于让物理系统（如机器人）做出智能决策。世界知名机器人专家 Sebastian Thrun 将这些系统的组件描述如下：

> 机器人系统的共同特点是它们位于物理世界中，通过传感器感知其环境，并通过移动的事物操纵其环境。[19]

19 S. Thrun，“概率机器人学”，*Communications of the ACM*，第45卷，第3期，第52–57页，2002年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

为了设计这些系统，工程师必须解决感知、规划和行动的问题。物理系统通过使用其传感器来创建对其环境显著特征的表示，从而感知世界。状态估计领域一直专注于利用传感器测量来构建关于世界状态的信念。规划需要对执行其设计任务的方式进行推理。规划过程得益于半导体行业数十年来的进步。[20] 一旦制定了计划，

自主智能体必须在现实世界中对其采取行动。这项任务既需要硬件（以执行器的形式），也需要算法来控制执行器并抑制扰动。控制理论领域一直专注于通过反馈控制实现机械系统的稳定。[21] 自动控制系统

从烤箱温度的调节到航空航天系统的导航，它们在工业中广泛应用。

20 G. E. Moore，“将更多元件集成到集成电路上”，*Electronics*，第38卷，第8期，第114–117页，1965年。

21 D. A. Mindell, *Between Human and Machine: Feedback, Control, and Computing Before Cybernetics*. JHU Press, 2002.

1.4.6 Mathematics

智能体必须能够量化其不确定性，以便在不确定环境中做出明智决策。决策领域在很大程度上依赖概率论来完成这项任务。特别是，贝叶斯统计在本文中扮演着重要角色。1763年，托马斯·贝叶斯的一篇论文在他去世后发表，其中包含了后来被称为贝叶斯法则的内容。他的概率推断方法时而被接受，时而被冷落，直到二十世纪中叶，研究人员开始发现贝叶斯方法在许多场景中很有用。[22] 数学家伯纳德

库普曼在二战期间发现了该理论的实际用途：

> 搜索中涉及的每一项操作都充满了不确定性；只有从[...]概率的角度，才能定量地理解它。如今这或许被视为不言而喻的道理，但似乎正是第二次世界大战期间运筹学的发展，才使其实际意义深入人心。[23]

基于抽样的方法（有时被称为*Monte Carlo methods*）是在20世纪初作为曼哈顿计划的一部分为大规模计算而开发的，它们使一些以前难以处理的推断技术成为可能。这些基础为贝叶斯网络奠定了基础，后者在20世纪后期在人工智能领域越来越受欢迎。

22 W. M. Bolstad 和 J. M. Curran, *Introduction to Bayesian Statistics*。Wiley，2016。

23 B. O. Koopman，*Search and Screening*。Pergamon，1980。*Search and Screening: General Principles with Historical Applications*

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

1.4.7 Operations Research

Operations research 关注于为资源分配、资产投资和维护调度等决策问题寻找最优解。在十九世纪末，研究者们开始探索将数学和科学分析应用于商品和服务的生产。这一领域在工业革命期间加速发展，当时公司开始将其管理层细分为负责整体决策不同方面的部门。二战期间，决策优化被应用于为军队分配资源。战争结束后，企业开始注意到，先前用于军事决策的运筹学概念同样可以帮助他们优化商业决策。这一认识促成了管理科学的发展，正如组织理论家哈罗德·孔茨所描述的那样：

> 这一群体的坚定信念是，如果管理、组织、规划或决策制定是一个逻辑过程，那么它就可以用数学符号和关系来表达。这一学派的核心方法是模型，因为正是通过这些工具，问题才能以其基本关系和选定的目标或目的的形式得以表述。[24]

这种希望能够更好地建模和理解商业决策的愿望，激发了当今所使用的许多概念的发展，如线性规划、动态规划和排队论。[25]

Introduction

24 H. Koontz，《管理理论丛林》，*Academy of Manage-*，第4卷，第3期，第174-188页，1961年。

25 F. S. Hillier，*Introduction to Opera-*。McGraw-Hill *tions Research*

1.5 Societal Impact

算法决策方法已经改变了社会，并且未来很可能继续如此。本节简要介绍决策算法为社会做出贡献的几种方式，并介绍在试图确保广泛受益时仍然存在的挑战。[26]

算法方法为环境可持续性做出了贡献。例如，在能源管理领域，贝叶斯优化已被应用于家庭能源管理自动化系统。多智能体系统领域的算法被用于预测智能电网的运行、设计能源交易市场，以及预测屋顶太阳能的采用。算法也已被开发用于保护生物多样性。例如，神经网络被用于自动化野生动物普查，博弈论方法被用于

26 更深入的讨论由Z. R. Shi、C. Wang和F. Fang提供，“人工智能为社会公益：综述”，2020年。arXiv: 2001.01818v1。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

在森林中打击偷猎，并采用优化技术来分配栖息地管理的资源。

决策算法在医学领域已成功应用数十年。此类算法已被用于将住院医师匹配到医院，以及将器官捐赠者匹配给有需要的患者。贝叶斯网络的早期应用之一是疾病诊断，我们将在本书第一部分中介绍这一内容。此后，贝叶斯网络在医学中被广泛用于疾病的诊断和预后。医学图像处理领域已被深度学习所变革，而算法思想近来在理解疾病传播方面也发挥了重要作用。

算法使我们能够理解城市区域的发展并促进其设计。数据驱动算法已被广泛用于改善公共基础设施。例如，随机过程已被用于预测水管故障，深度学习改善了交通管理，马尔可夫决策过程和蒙特卡洛方法已被用于改善应急响应。来自去中心化多智能体系统的思想优化了出行路线，路径规划技术已被用于优化货物配送。决策算法已被用于自动驾驶汽车和提高飞机安全性。

优化决策的算法能够放大其用户的影响力，无论其意图如何。例如，若这些算法的使用者旨在政治选举期间散布虚假信息，优化过程则可助长这一行为。然而，相似的算法也能被用于监测和抵制虚假信息的传播。有时，这些决策算法的应用可能导致其用户未曾预料到的后续后果。[27]

尽管算法有可能带来显著的好处，但在社会中实施算法也伴随着挑战。数据驱动的算法往往因数据收集方式而存在固有的偏见和盲点。随着算法成为我们生活的一部分，了解如何减少偏见风险以及如何以公平公正的方式分配算法进步的益处变得非常重要。算法还可能容易受到对抗性操纵，设计能够抵御此类攻击的鲁棒算法至关重要。同样重要的是，扩展道德和法律框架，以防止意外后果并明确责任归属。

27 关于一般性讨论，见 B. Christian，*The Alignment Problem*。Norton & Company，2020年。另见 D. Amodei、C. Olah、J. Steinhardt、P. Christiano、J. Schulman 和 D. Mané 的讨论，“Con-crete Problems in AI Safety”，2016年。arXiv: 1606.06565v2。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

1.6 Overview

本书分为五个部分。第一部分探讨在单一时间点上的简单决策中关于不确定性和目标的推理问题。第二部分将决策扩展到序贯问题，在这类问题中，我们必须随着行动推进，根据有关行动结果的信息作出一系列决策。第三部分讨论模型不确定性，即我们并非从一个已知模型出发，而必须通过与环境的交互来学习如何行动。第四部分讨论状态不确定性，即不完善的感知信息使我们无法了解完整的环境状态。最后一部分讨论涉及多个智能体的决策情境。

1.6.1 Probabilistic Reasoning

理性的决策制定需要对我们所面临的不确定性和目标进行推理。本书的这一部分首先讨论如何将不确定性表示为概率分布。现实世界中的问题需要关于众多变量的分布进行推理。我们将讨论如何构建这些模型，如何利用它们进行推断，以及如何从数据中学习它们的参数和结构。接着，我们引入*utility theory*的基础，并展示它如何通过最大期望效用原则为不确定性下的理性决策奠定基础。随后，我们将讨论如何将效用理论的概念融入本章早些时候介绍的概率图模型中，形成所谓的*decision networks*。

1.6.2 Sequential Problems

许多重要问题要求我们做出一系列决策。最大期望效用原则仍然适用，但在序列情境下进行最优决策需要对未来动作和观察序列进行推理。本书的这一部分将讨论随机环境中的序列决策问题，其中我们行动的结果是不确定的。我们将重点关注在模型已知且环境完全可观测假设下序列决策问题的一般形式。本书稍后将放宽这两个假设。我们的讨论将从介绍 *Markov decision process* (*MDP*) 开始，这是标准的

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

sequential decision problems 的 mathematical model。我们将讨论几种寻找这类问题 exact solutions 的方法。由于 large problems 有时无法高效地找到 exact solutions，我们将讨论一组 both offline and online approximate solution methods，以及一种涉及直接搜索 parameterized decision policies 空间的方法。最后，我们将讨论用于验证我们的 decision strategies 在部署到 real world 时是否能够按预期表现的方法。

1.6.3 Model Uncertainty

在我们此前对序列决策问题的讨论中，我们假定了转移和奖励模型是已知的。然而，在许多问题中，动力学和奖励并非完全已知，智能体必须通过经验学习行动。通过观察其行动的结果，即状态转移和奖励，智能体需选择能最大化其长期奖励累积的行动。解决这类存在模型不确定性的问题，是*reinforcement learning*领域的主题，也是本书这一部分的重点。我们将讨论应对模型不确定性时的若干挑战。首先，智能体必须谨慎平衡对环境的探索与对通过经验获得的知识的利用。其次，奖励可能在重要决策做出很久之后才收到，因此后期奖励的功劳必须归因于早期的决策。第三，智能体必须从有限的经验中进行泛化。我们将回顾应对这些挑战的理论及一些关键算法。

1.6.4 State Uncertainty

在这一部分中，我们将不确定性扩展到包含状态。我们不再精确地观察状态，而是接收到与状态仅具有概率关系的观测。此类问题可以建模为*partially observable Markov decision process* (*POMDP*)。解决POMDP的一种常见方法涉及推断当前时间步下底层状态的信念分布，然后应用将信念映射到动作的策略。本部分首先讨论如何在给定过去一系列观测和动作的情况下更新我们的信念分布。然后讨论各种精确和近似方法以求解POMDP。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

1.6.5 *Multiagent Systems*

到目前为止，环境中只有一个代理在做决策。本部分将前四部分扩展到多个代理，讨论由交互不确定性带来的挑战。我们首先讨论简单博弈，其中一组代理同时各自选择一个动作。结果是基于联合动作组合的每个代理的个体奖励。

Markov game (*MG*)代表了简单博弈到多个状态的推广，以及MDP到多个代理的推广。因此，代理选择能够随机改变共享环境状态的动作。由于对其他代理策略的不确定性，MGs的算法依赖于强化学习。

partially observable Markov game (*POMG*)引入了状态不确定性，进一步推广了MGs和POMDPs，因为代理现在只接收到嘈杂的局部观测。

decentralized partially observable Markovdecision process (*Dec-POMDP*)将POMG聚焦于一个协作的多代理团队，其中代理之间共享奖励。本书的这一部分介绍了这四类问题，并讨论了解决它们的精确和近似算法。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

部分 i

概率推理

理性的决策制定需要对我们的不确定性和目标进行推理。不确定性源于我们预测未来事件能力的实践和理论限制。例如，要精确预测人类操作员如何响应决策支持系统的建议，除其他事项外，还需要一个详细的人类大脑工作模型。即使是卫星的轨迹也可能难以预测。尽管牛顿物理学允许对卫星轨迹进行高度精确的预测，姿态推进器的自发故障可能导致与标称路径的大幅偏离，甚至微小的不精确也可能随时间累积。为了实现其目标，一个稳健的决策制定系统必须考虑世界当前状态和未来事件中的各种不确定性来源。本书的这一部分首先讨论如何使用概率分布来表示不确定性。现实世界的问题需要对多个变量上的分布进行推理。我们将讨论如何构建这些模型，使用它们进行推断，并从数据中学习它们的参数和结构。然后我们介绍效用理论的基础，并展示它如何构成不确定性下理性决策制定的基础。效用理论可以整合到前面介绍的概率图模型中，形成所谓的决策网络。我们专注于单步决策，将序列决策问题的讨论留到本书的下一部分。

2 *Representation*

计算上考虑不确定性需要一种形式化的表示。本章讨论如何表示不确定性。[1]我们首先介绍

信念度的概念，并展示一组公理如何使我们能够使用概率分布来量化不确定性。[2] 我们讨论了几个

离散和连续变量上分布的有用形式。因为许多重要问题涉及大量变量上的概率分布，我们讨论了一种利用变量间的条件独立性来高效表示联合分布的方法。

1 关于表示不确定性的各种方法的详细讨论由 F. Cuzzolin, *The Geometry of Uncertainty* 提供。Springer, 2021。

2 关于更全面的阐述，请参见 E. T. Jaynes, *Probability Theory: The Logic of Science*。剑桥大学出版社，2003年。

2.1 *Degrees of Belief and Probability*

在涉及不确定性的问题中，能够比较不同陈述的合理性至关重要。我们希望能够表示，例如命题A比命题B更为合理。如果A代表“我的执行器故障”，而B代表“我的传感器故障”，那么我们表达为$A \succ B$。利用这一基本关系$\succ$，我们可以定义其他几种关系：

$$A \prec B \text{ if and only if } B \succ A \tag{2.1}$$

$$A \sim B \text{ if and only if neither } A \succ B \text{ nor } B \succ A \tag{2.2}$$

$$A \succeq B \text{ if and only if } A \succ B \text{ or } A \sim B \tag{2.3}$$

$$A \preceq B \text{ if and only if } B \succ A \text{ or } A \sim B \tag{2.4}$$

我们希望对这些运算符 $\succ$、$\sim$ 和 $\prec$ 所诱导的关系做出某些假设。对 *universal comparability* 的假设要求以下恰好一个成立：$A \succ B$、$A \sim B$ 或 $A \prec B$。对 *transitivity* 的假设要求，如果 $A \succeq B$ 和 $B \succeq C$，那么 $A \succeq C$。普遍可比性

并且传递性假设使得能够通过一个实值函数 P 来表示合理性，该函数具有以下两个性质：[3]

$$P(A) > P(B) \text{ if and only if } A \succ B \tag{2.5}$$

$$P(A) = P(B) \text{ if and only if } A \sim B \tag{2.6}$$

如果我们对 P 的形式做出一组额外的假设[4]，那么我们可以证明 P 必须满足基本的 *axioms of probability*（见附录 A.2）。如果我们确信 A，那么 $P(A) = 1$。如果我们认为 A 不可能，那么 $P(A) = 0$。A 真实性的不确定性由两个极值之间的值表示。因此，概率质量必须介于 0 和 1 之间，其中 $0 \leq P(A) \leq 1$。

Analysis

2.2 Probability Distributions

一个 *probability distribution* 为不同结果分配概率。[5] 存在根据结果是离散还是连续的，表示概率分布的不同方式。

2.2.1 Discrete Probability Distributions

一个 *discrete probability distribution* 是定义在离散取值集合上的分布。我们可以将这样的分布表示为一个 *probability mass function*，它为输入变量的每一种可能取值分配一个概率。例如，假设我们有一个变量 X，它可以取 n 个值之一：$1, \ldots, n$，或者使用 *colon notation*，$1:n$.[6] 与 X 相关的分布指定了

n 将该变量取各种值的概率，特别是 $P(X = 1), \ldots, P(X = n)$。图2.1展示了一个离散分布的例子。

离散分布的概率质量存在约束条件。这些质量之和必须等于1：

$$\sum_{i=1}^{n} P(X = i) = 1 \tag{2.7}$$

并且对所有 i，$0 \leq P(X = i) \leq 1$。

为方便表示，我们在讨论变量赋值时将使用小写字母和上标作为简写。例如，$P(x^3)$ 是 $P(X = 3)$ 的简写。如果 X 是一个 *binary variable*，它可以取真或假的值。[7] 我们将用0表示假，1表示真。例如，

我们使用$P(x^0)$来表示X为假的概率。

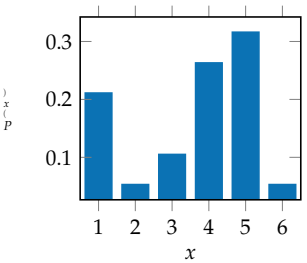


图2.1. 分布1至6上的概率质量函数。

3 参见 E. T. Jaynes, *Probability Theory: The Logic of Science*，第2章的讨论。剑桥大学出版社，2003年。

4 主观概率的公理化由P. C. Fishburn在“主观概率的公理”中给出，*Statistical Science*，第1卷，第3期，第335-345页，1986年。更近期的公理化包含在M. J. Dupré和F. J. Tipler的“严格贝叶斯概率的新公理”中，*Bayesian*，第4卷，第3期，第599-606页，2009年。

5 关于概率论的介绍，参见 D. P. Bertsekas 和 J. N. Tsitsiklis, *Introduction to Probability*。Athena Scientific，2002年。

6 我们经常会使用这种冒号表示法以求简洁。其他文本有时会使用$[1 \,..\, n]$的表示法来表示从1到n的整数区间。我们也会用这种冒号表示法来索引向量和矩阵。例如，$x_{1:n}$代表$x_1, \ldots, x_n$。这种冒号表示法有时会在编程语言中使用，比如Julia和MATLAB。

7 与许多其他编程语言一样，Julia在数值运算中同样将布尔值视为 0 和 1。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

分布的 *parameters* 决定了与不同赋值相关的概率。例如，如果我们用 X 表示掷一枚六面骰子的结果，那么我们将有 $P(x^1) = \theta_1$、$\cdots$、$P(x^6) = \theta_6$，其中 $\theta_{1:6}$ 是该分布的六个参数。然而，我们只需要五个 *independent pa- rameters* 就能唯一地确定掷骰子结果的分布，因为我们知道分布的总和必须为 1。

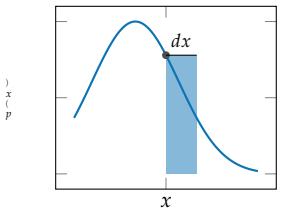


图 2.2。概率密度函数用于表示连续概率分布。如果 $p(x)$ 是概率密度，那么由蓝色矩形的面积所表示的 $p(x)dx$ 就是随机变量的样本落在区间 $(x, x + dx)$ 内的概率，当 $dx \to$ 趋于 0 时。

2.2.2 *Continuous Probability Distributions*

一个 *continuous probability distribution* 是连续值集合上的分布。表示连续变量上的分布比离散变量稍微不那么直接。例如，在许多连续分布中，一个变量取某个特定值的概率是无限小的。表示连续概率分布的一种方法是使用 *probability density function* (见图2.2)，用小写字母表示。如果 $p(x)$ 是 X 上的概率密度函数，那么 $p(x)dx$ 是 X 落在区间 $(x, x + dx)$ 内的概率，当 $dx \to 0$时。与离散分布相关的概率质量必须总和为1类似，概率密度函数 $p(x)$ 必须积分为 1：

$$\int_{-\infty}^{\infty} p(x)\,\mathrm{d}x = 1 \tag{2.8}$$

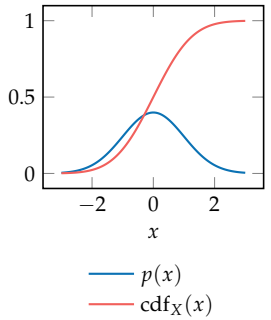


图2.3. 标准高斯分布的概率密度函数和累积分布函数。

另一种表示连续分布的方法是使用*cumulative distri- bution function* (见图2.3)，它指定了与低于某个阈值的值关联的概率质量。如果我们有一个与变量X相关的累积分布函数P，那么$P(x)$表示与X取值小于或等于x相关的概率质量。累积分布函数可以根据概率密度函数p定义如下：

$$\mathrm{cdf}_X(x) = P(X \leq x) = \int_{-\infty}^{x} p(x')\,\mathrm{d}x' \tag{2.9}$$

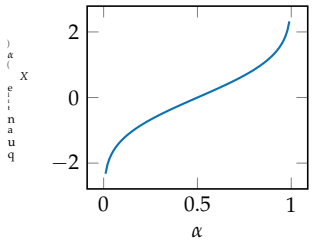


图2.4. 标准高斯分布的分位函数。

与累积分布函数相关的是*quantile function*，也称为 *inverse cumulative distribution function* (，见图2.4)。分位数$_X(\alpha)$的值是使得 $P(X \leq x) = \alpha$成立的值x。换句话说，分位数函数返回累积分布值大于或等于α的最小值x。当然，我们有$0 \leq \alpha \leq 1$。

有许多不同的参数化分布族。我们在附录B中概述了几种。一个简单的分布族是*uniform distribution*

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

$\mathcal{U}(a, b)$，它在 a 和 b 之间均匀分配概率密度，其他地方为零。因此，概率密度函数为 $p(x) = 1/(b-a)$，其中 x 在区间 $[a, b]$ 内。我们可以使用 $\mathcal{U}(x \mid a、b)$ 来表示 x 处的密度。[8] 所述 *support*

分布的支撑集是被赋予非零密度的值的集合。对于$\mathcal{U}(a、b)$而言，支撑集是区间$[a, b]$。参见例2.1。

8 有些文本使用分号来分隔分布的参数。例如，也可以写作$\mathcal{U}(x; a, b)$。

例2.1：下界为0、上界为10的均匀分布示例。

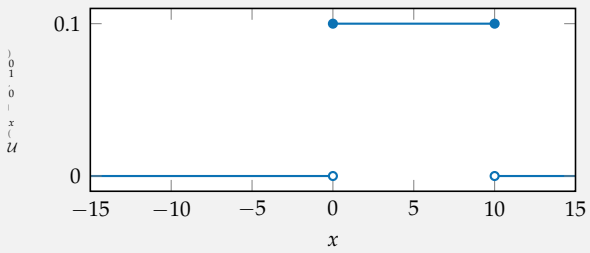
均匀分布 $\mathcal{U}(0, 10)$ 对范围 $[0, 10]$ 内的所有值赋予相等的概率，其概率密度函数为：

$$\mathcal{U}(x \mid 0, 10) = \begin{cases} 1/10 & \text{if } 0 \leq x \leq 10 \\ 0 & \text{otherwise} \end{cases} \tag{2.10}$$

从该分布中随机抽取的样本恰好等于常数 π 的概率基本为零。然而，我们可以为样本落在某个区间内（例如 $[3, 5]$）定义非零概率。例如，给定此处绘制的分布，样本位于 3 和 5 之间的概率为：

$$\int_3^5 \mathcal{U}(x \mid 0, 10)\, \mathrm{d}x = \frac{5-3}{10} = \frac{1}{5} \tag{2.11}$$

这个分布的支撑集是区间 $[0, 10]$。



连续变量的另一个常见分布是 *Gaussian distribu-*，也称为 *normal distribution*）。高斯分布由均值 μ 和方差 σ^2 参数化：

$$p(x) = \mathcal{N}(x \mid \mu, \sigma^2) \tag{2.12}$$

这里，σ 是 *standard deviation*，即方差的平方根。方差通常也用 ν 表示。我们用 $\mathcal{N}(\mu、\sigma^2)$ 来表示一个高斯

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

具有参数 μ 和 σ^2 以及 $\mathcal{N}(x \mid \mu, \sigma^2)$ 的 sian 分布，以表示在 x 处的概率密度，如下所示：

$$\mathcal{N}(x \mid \mu, \sigma^2) = \frac{1}{\sigma}\phi\left(\frac{x-\mu}{\sigma}\right) \tag{2.13}$$

其中 ϕ 是 *standard normal density function*：

$$\phi(x) = \frac{1}{\sqrt{2\pi}}\exp\left(-\frac{x^2}{2}\right) \tag{2.14}$$

附录B展示了不同参数下的高斯密度函数图。

虽然高斯分布通常很方便，因为它仅由两个参数定义，使得计算和推导变得容易，但它也有一些局限性。它为大的正值和负值分配非零概率，这可能不适合我们试图建模的量。例如，我们可能不希望为飞机在地面以下或不可行高度飞行分配非零概率。我们可以使用*truncated Gaussian distribution* (见图2.5)来限制可能值的*support*；即分配非零概率的值范围。密度函数由下式给出

$$\mathcal{N}(x \mid \mu, \sigma^2, a, b) = \frac{\frac{1}{\sigma}\phi\left(\frac{x-\mu}{\sigma}\right)}{\Phi\left(\frac{b-\mu}{\sigma}\right) - \Phi\left(\frac{a-\mu}{\sigma}\right)} \tag{2.15}$$

当 x 位于区间 (a, b) 内时。

函数Φ是*standard normal cumulative distribution function*，由下式给出

$$\Phi(x) = \int_{-\infty}^{x} \phi(x')\,\mathrm{d}x' \tag{2.16}$$

高斯分布是*unimodal*，这意味着分布中存在一个点，在该点一侧密度增加，另一侧减少。表示*multimodal*的连续分布有多种方法。一种方法是使用*mixture model*，它是多个分布的混合。我们将一组单峰分布混合在一起，以获得多峰分布。*Gaussian mixture model*是一种混合模型，它只是各种高斯分布的加权平均。高斯混合模型的参数包括高斯分布分量的参数$\mu_{1:n}$、$\sigma^2_{1:n}$，以及它们的权重$\rho_{1:n}$。密度由下式给出

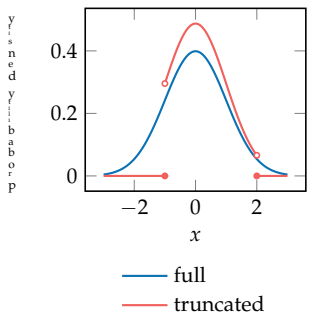


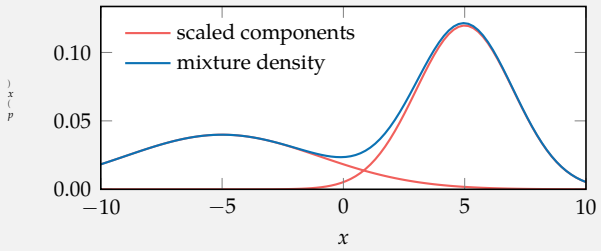
图2.5. 单位高斯分布的概率密度函数及相同分布在−1和2之间截断后的概率密度函数。

$$p(x \mid \mu_{1:n}, \sigma^2_{1:n}, \rho_{1:n}) = \sum_{i=1}^{n} \rho_i \mathcal{N}(x \mid \mu_i, \sigma_i^2) \tag{2.17}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

其中权重之和必须为1。示例2.2展示了一个包含两个组件的高斯混合模型。

我们可以创建一个高斯混合模型，其组成部分为 $\mu_1 = 5$、$\sigma_1 = 2$ 和 $\mu_2 = -5$、$\sigma_2 = 4$，权重分别为 $\rho_1 = 0.6$ 和 $\rho_2 = 0.4$。这里我们绘制了两个按各自权重缩放后的组成部分的密度：



示例 2.2. 高斯混合模型的一个例子。

另一种表示多模态连续分布的方法是通过离散化。例如，我们可以将连续变量上的分布表示为 *piecewise-uniform density*。密度由箱边缘指定，并且与每个箱关联一个概率质量。这种分段均匀分布是一种混合模型，其中分量是均匀分布。

2.3 *Joint Distributions*

一个 *joint distribution* 是多个变量上的概率分布。单个变量上的分布称为 *univariate distribution*，而多个变量上的分布称为 *multivariate distribution*。如果我们有两个离散变量 X 和 Y 的联合分布，那么 $P(x, y)$ 表示 $X = x$ 和 $Y = y$ 两者的概率。

从联合分布中，我们可以通过利用所谓的*law of total probability*来对其他所有变量求和，从而计算出某个变量或一组变量的*marginal*分布：[9]

$$P(x) = \sum_y P(x, y) \tag{2.18}$$

此性质贯穿本书始终。

9 如果我们的分布是连续的，那么在边缘化时，我们会将其他变量积分掉。例如：

$$p(x) = \int p(x, y)\, \mathrm{d}y$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

现实世界的决策往往需要对涉及许多变量的联合分布进行推理。有时变量之间存在复杂的关系，这些关系对表示很重要。根据变量是离散值还是连续值，我们可以使用不同的策略来表示联合分布。

2.3.1 Discrete Joint Distributions

如果变量是离散的，联合分布可以用表2.1所示的表格来表示。该表列出了三个二元变量所有可能的值分配。每个变量只能取0或1，导致2^3 =8种可能的分配。与其他离散分布一样，表中的概率总和必须为1。因此，尽管表中有八项，但只有七项是*independent*。如果θ_i表示表中第i行的概率，那么我们只需要参数θ_1，...，θ_7来表示分布，因为我们知道$\theta_8 = 1 - (\theta_1 + ... + \theta_7)$。

表2.1. 一个涉及二元变量X、Y和Z的联合分布示例。

X	Y	Z	$P(X, Y, Z)$
0	0	0	0.08
0	0	1	0.31
0	1	0	0.09
0	1	1	0.37
1	0	0	0.01
1	0	1	0.05
1	1	0	0.02
1	1	1	0.07

如果我们有 n 个二值变量，那么我们需要多达 $2^n - 1$ 个独立参数来指定联合分布。参数数量的这种指数增长使得在内存中存储分布变得困难。在某些情况下，我们可以假设变量是 *independent* 的，这意味着一个变量的实现不影响另一个变量的概率分布。如果 X 和 Y 独立，有时写作 $X \perp Y$，那么我们知道对于所有 x 和 y，有 $P(x, y) = P(x)P(y)$。假设我们有二值变量 X_1、...、X_n，它们彼此全部独立，结果为 $P(x_{1:n}) = \prod_i P(x_i)$。这种因式分解允许我们仅用 n 个独立参数来表示这个联合分布，而不是在不能假设独立时所需的 $2^n - 1$ 个（见表 2.2）。独立性可以在表示复杂性方面带来巨大的节省，但这通常是一个不好的假设。

表2.2。如果我们知道表2.1中的变量是独立的，我们可以用乘积$P(x)P(y)P(z)$来表示$P(x, y, z)$。这种表示方法对于三个单变量分布中的每一个都只需要一个参数。

X	$P(X)$
0	0.85
1	0.15

Y	$P(Y)$
0	0.45
1	0.55

Z	$P(Z)$
0	0.20
1	0.80

我们可以用因子来表示联合分布。一个定义在一组变量上的 *factor* ϕ 是从这些变量的赋值到实数的函数。为了表示概率分布，因子中的实数必须是非负的。具有非负值的因子可以被归一化，使其表示一个概率分布。算法2.1给出了离散因子的实现，例2.3演示了它们如何工作。

另一种减少表示具有重复值的联合分布所需存储量的方法是使用*decision tree*。一棵涉及三个离散变量的决策树

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct Variable
    name::Symbol
    r::Int # number of possible values
end

const Assignment = Dict{Symbol,Int}
const FactorTable = Dict{Assignment,Float64}

struct Factor
    vars::Vector{Variable}
    table::FactorTable
end

variablenames(ϕ::Factor) = [var.name for var in ϕ.vars]

select(a::Assignment, varnames::Vector{Symbol}) =
    Assignment(n⇒a[n] for n in varnames)

function assignments(vars::AbstractVector{Variable})
    names = [var.name for var in vars]
    return vec([Assignment(n⇒v for (n,v) in zip(names, values))
                for values in product((1:v.r for v in vars)...)])
end

function normalize!(ϕ::Factor)
    z = sum(p for (a,p) in ϕ.table)
    for (a,p) in ϕ.table
        ϕ.table[a] = p/z
    end
    return ϕ
end
```

算法2.1. 处理定义在离散变量集合上的因子时涉及的类型和函数。每个变量有一个名称（用符号表示），并且可以取1到m之间的整数值。一个赋值是从变量名称到用整数表示的值的映射。一个因子由因子表定义，该表为一组变量的不同赋值分配值，并且是从赋值到实数值的映射。该映射由字典表示。任何未包含在字典中的赋值都设为0。此算法块中还包括一些实用函数，用于返回与因子相关的变量名称、选择赋值的一个子集、枚举可能的赋值，以及归一化因子。如附录G.3.3所讨论，`product`生成一组集合的笛卡尔积。它是从`Base.Iterators`导入的。

我们可以使用以下代码，通过 `Factor` 类型从表 2.1 实例化这个表：

```
# requires convenience functions from appendix G.5
X = Variable(:x, 2)
Y = Variable(:y, 2)
Z = Variable(:z, 2)
ϕ = Factor([X, Y, Z], FactorTable(
    (x=1, y=1, z=1) ⇒ 0.08, (x=1, y=1, z=2) ⇒ 0.31,
    (x=1, y=2, z=1) ⇒ 0.09, (x=1, y=2, z=2) ⇒ 0.37,
    (x=2, y=1, z=1) ⇒ 0.01, (x=2, y=1, z=2) ⇒ 0.05,
    (x=2, y=2, z=1) ⇒ 0.02, (x=2, y=2, z=2) ⇒ 0.07,
))
```

例2.3. 构造离散因子。*The construction of the factor table using named tuples takes advantage of the utility functions defined in appendix G.5*。

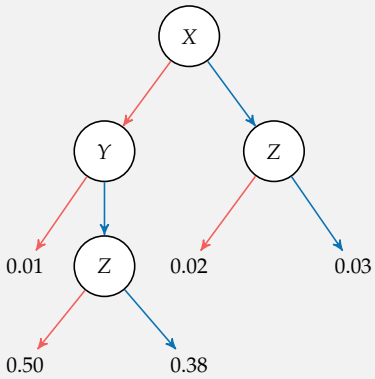
© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

变量在示例2.4中展示。尽管此示例中在参数数量上的节省可能并不显著，但当存在大量变量和许多重复值时，节省量可以变得相当可观。

示例2.4。决策树可以比表格更有效地表示联合分布。

假设我们有下表表示一个联合概率分布。我们可以使用其右侧的决策树来更紧凑地表示表中的值。当变量为0时，沿红色箭头行进；当变量为1时，沿蓝色箭头行进。我们只需存储五个概率值以及树的表示，而无需存储八个概率。

X	Y	Z	$P(X, Y, Z)$
0	0	0	0.01
0	0	1	0.01
0	1	0	0.50
0	1	1	0.38
1	0	0	0.02
1	0	1	0.03
1	1	0	0.02
1	1	1	0.03



2.3.2 *Continuous Joint Distributions*

我们也可以定义连续变量上的联合分布。一个相当简单的分布是 *multivariate uniform distribution*，它在有支撑的每个地方赋予恒定的概率密度。我们可以用 $\mathcal{U}(\mathrm{a}, \mathrm{b})$ 来表示一个 *box* 上的均匀分布，它是一个区间的笛卡尔积，其中第 i 个区间为 $[a_i, b_i]$。这个均匀分布族是 *multivariate product distribution* 的一种特殊类型，它是根据单变量分布的乘积定义的分布。在这种情况下，

$$\mathcal{U}(\mathbf{x} \mid \mathbf{a}, \mathbf{b}) = \prod_i \mathcal{U}(x_i \mid a_i, b_i) \tag{2.19}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们可以从多元均匀分布的加权集合创建混合模型，就像我们可以对单变量分布所做的那样。如果我们有一个关于 n 个变量和 k 个混合组分的联合分布，我们需要定义 $k(2n+1)-1$ 个独立参数。对于 k 个组分中的每一个，我们需要定义每个变量的上下界以及它们的权重。我们可以减去1，因为权重之和必须为1。图2.6展示了一个可以用五个组分表示的示例。

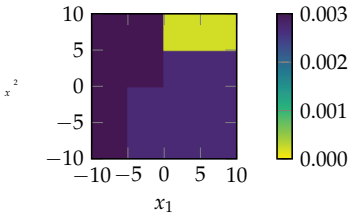


图2.6. 多变量均匀分布混合的密度函数。

用独立离散化每个变量的方法来表示分段常数密度函数也很常见。离散化通过每个变量的一组箱边缘来表示。这些箱边缘定义了变量上的网格。然后，我们将一个恒定的概率密度与每个网格单元关联起来。箱边缘不必均匀分隔。在某些情况下，可能希望在特定值附近增加分辨率。不同的变量可能关联有不同的箱边缘。如果有n个变量，每个变量有m个箱，那么我们需要m^n −个独立参数来定义分布——此外还有定义箱边缘的值。

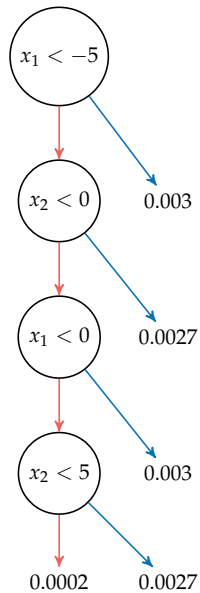


图2.7. 一个决策树示例，表示在区间$[-10, 10]^2$上定义的、基于x_1和x_2的分段常数联合概率密度。

在某些情况下，将连续联合分布表示为决策树可能更为内存高效，类似于我们对离散联合分布所讨论的方法。内部节点将变量与阈值进行比较，而叶节点则是密度值。图2.7展示了一个表示图2.6中密度函数的决策树。

另一个有用的分布是 *multivariate Gaussian distribution*，其密度函数为

$$\mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{n/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right) \tag{2.20}$$

其中 x 在 $\mathbb{R}^n$ 中，$\boldsymbol{\mu}$ 是 *mean vector*，而 $\boldsymbol{\Sigma}$ 是 *covariance matrix*。这里给出的密度函数要求 $\boldsymbol{\Sigma}$ 为 *positive definite*。[10] 独立的
凹痕参数等于 $n + (n+1)n/2$，即 $\boldsymbol{\mu}$ 中组件数量加上矩阵 $\boldsymbol{\Sigma}$ 上三角中组件数量之和。[11] 附录 B
展示了不同多变量高斯密度函数的图。我们也可以定义
multivariate Gaussian mixture models。图2.8展示了一个包含三个分量的例子。

如果我们有一个所有变量都独立的多变量高斯分布，那么协方差矩阵 $\boldsymbol{\Sigma}$ 是对角矩阵，仅包含 n 个独立参数。实际上，

10 此定义在附录A.5中进行了回顾。

11 如果我们知道$\boldsymbol{\Sigma}$上三角的参数，那么我们也知道下三角的参数，因为$\boldsymbol{\Sigma}$是对称的。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

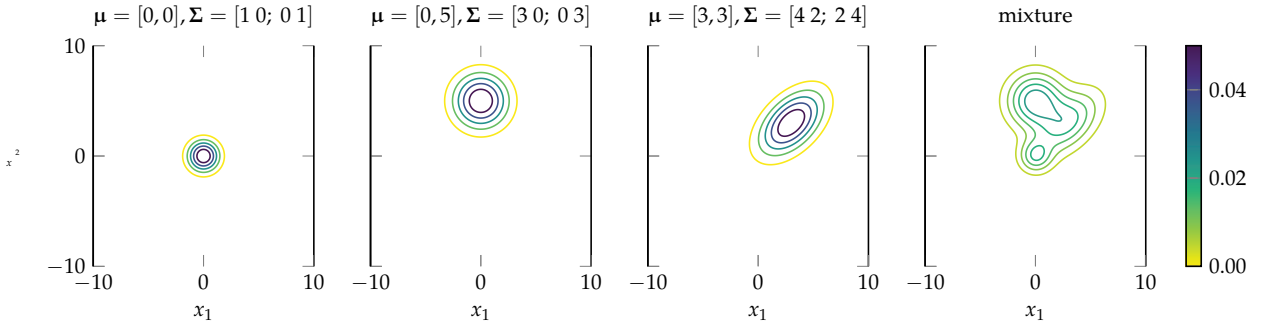


图2.8. 包含三个组分的多变量高斯混合模型。这些组分分别以0.1、0.5和0.4的权重混合在一起。

我们可以将密度函数写成单变量高斯密度的乘积：

$$\mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \prod_i \mathcal{N}(x_i \mid \mu_i, \Sigma_{ii}) \tag{2.21}$$

2.4 Conditional Distributions

前一节介绍了独立性的概念，这有助于减少定义联合分布所需的参数数量。然而，正如所提到的，独立性可能是一个过于强的假设。本节将介绍条件独立性的概念，它能在不做出如此强假设的情况下，帮助减少独立参数的数量。在讨论条件独立性之前，我们首先引入{v3*}　　注：由于原文中“*conditional distribution*”后似乎缺失了部分内容，如“的概念，即在给定一个或多个其他变量值的情况下，某一变量的分布。”请根据实际情况补全。

*conditional probability*的定义指出

$$P(x \mid y) = \frac{P(x, y)}{P(y)} \tag{2.22}$$

其中 $P(x \mid y)$ 读作“在 y 条件下 x 的概率”。在某些上下文中，通常将 y 称为 *evidence*。

由于条件概率分布是在给定某些证据的情况下，一个或多个变量的概率分布，我们知道

$$\sum_x P(x \mid y) = 1 \tag{2.23}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

对于离散的 X。如果 X 是连续的，其积分等于 1。

我们可以将条件概率的定义代入方程（2.18）中，得到全概率定律的一个略有不同的形式：

$$P(x) = \sum_{y} P(x \mid y)P(y) \tag{2.24}$$

对于离散分布。

从条件概率定义中得出的另一个有用的关系是 *Bayes' rule*：[12]

$$P(x \mid y) = \frac{P(y \mid x)P(x)}{P(y)} \tag{2.25}$$

如果我们有一个条件分布 $P(y \mid x)$ 的表示，我们可以应用贝叶斯规则来交换 y 和 x，从而获得条件分布 $P(x \mid y)$。

12 以英国统计学家和长老会牧师托马斯·贝叶斯（约1701–1761年）命名，他给出了这一定理的一种表述。相关历史参见 S. B. McGrayne, *The Theory That Would*。耶鲁大学出版社，2011年。

我们现在将讨论在离散变量和连续变量上表示条件概率分布的各种方法。

2.4.1 Discrete Conditional Models

离散变量上的条件概率分布可以用表格表示。事实上，我们可以使用与2.3.1节中用于联合分布相同的离散因子表示法。表2.3展示了一个表示$P(X \mid Y、Z)$且所有变量均为二值变量的表格示例。与联合表（例如表2.1）不同，包含概率的列不需要求和为1。然而，如果我们对与我们所条件化的事物一致的概率求和，我们必须得到1。例如，以y^0和z^0 (为条件，证据为)，我们有

$$P(x^0 \mid y^0, z^0) + P(x^1 \mid y^0, z^0) = 0.08 + 0.92 = 1 \tag{2.26}$$

表2.3. 涉及二值变量X、Y和Z的条件分布示例。

X	Y	Z	$P(X \mid Y, Z)$
0	0	0	0.08
0	0	1	0.15
0	1	0	0.05
0	1	1	0.10
1	0	0	0.92
1	0	1	0.85
1	1	0	0.95
1	1	1	0.90

条件概率表可能会变得相当大。如果我们创建一个像表2.3那样的表格，在其中所有变量都可以取m个值，并且我们以n个变量为条件，那么将会有m^{n+1}行。然而，由于我们不以之为条件的变量的m个值之和必须为1，因此只有（$m-1$）m^n个独立参数。我们以之为条件的变量数量仍然呈指数增长。当条件概率表中有许多重复值时，决策树（在第2.3.1节介绍）可能是一种更高效的表示方法。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

2.4.2 Conditional Gaussian Models

可以使用 *conditional Gaussian* 模型来表示在给定一个或多个离散变量的情况下连续变量上的分布。例如，如果我们有一个连续变量 X 和一个取值为 $1{:}\, n$ 的离散变量 Y，我们可以如下定义条件高斯模型：[13]

$$p(x \mid y) = \begin{cases} \mathcal{N}(x \mid \mu_1, \sigma_1^2) & \text{if } y^1 \\ \vdots & \\ \mathcal{N}(x \mid \mu_n, \sigma_n^2) & \text{if } y^n \end{cases} \tag{2.27}$$

使用参数向量 $\theta = [\mu_{1:n}, \sigma_{1:n}]$。所有这 $2n$ 个参数都可以独立变化。如果我们想要以多个离散变量为条件，我们只需添加更多的情况和相关的参数。

13 这个定义是针对单变量高斯混合模型，但这一概念可以很容易地推广到多维高斯混合模型。

2.4.3 Linear Gaussian Models

$P(X \mid Y)$的*linear Gaussian*模型将连续变量X上的分布表示为高斯分布，其均值为连续变量Y值的线性函数。条件密度函数为

$$p(x \mid y) = \mathcal{N}(x \mid my + b, \sigma^2) \tag{2.28}$$

其参数为 $\theta = [m、b、\sigma]$。均值是由参数 m 和 b 定义的 y 的线性函数。方差是常数。图 2.9 展示了一个示例。

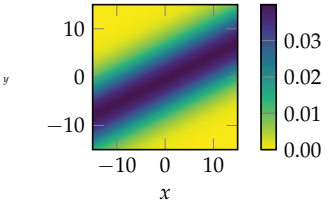


图2.9. 一个线性高斯模型，其中

$$p(x \mid y) = \mathcal{N}(x \mid 2y + 1, 10^2)$$

2.4.4 Conditional Linear Gaussian Models

conditional linear Gaussian 模型结合了条件高斯模型和线性高斯模型的思想，能够将连续变量以离散变量和连续变量为条件。假设我们想要表示 $p(X \mid Y、Z)$，其中 X 和 Y 是连续变量，Z 是取值为 $1{:}\, n$ 的离散变量。条件密度函数则为

$$p(x \mid y, z) = \begin{cases} \mathcal{N}(x \mid m_1 y + b_1, \sigma_1^2) & \text{if } z^1 \\ \vdots & \\ \mathcal{N}(x \mid m_n y + b_n, \sigma_n^2) & \text{if } z^n \end{cases} \tag{2.29}$$

这里，参数向量 $\theta = [m_{1:n}, b_{1:n}, \sigma_{1:n}]$ 有 $3n$ 个分量。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

2.4.5 Sigmoid Models

我们可以使用一个 {v*} 模型来表示二元变量上的分布
以连续变量为条件。例如，我们可能想要表示$P(x^1 \mid y)$，其中x是二元的，而y是连续的。当然，我们可以简单地设定一个阈值θ，并规定当$y < \theta$时$P(x^1 \mid y) =$为0，否则$P(x^1 \mid y) =$为1。然而，在许多应用中，我们可能不希望有这样一个硬阈值，导致对y的某些值将零概率分配给x^1。

14 S形曲线是一种S形曲线。数学上有多种定义此类曲线的方法，但我们将重点讨论logit模型。

我们可以使用 *soft threshold* 来代替硬阈值，当低于阈值时分配低概率，当高于阈值时分配高概率。表示软阈值的一种方法是使用 *logit model*，它会产生一条 S 形曲线:

$$P(x^1 \mid y) = \frac{1}{1 + \exp\left(-2\frac{y-\theta_1}{\theta_2}\right)} \tag{2.30}$$

参数 θ_1 控制阈值的位置，而 θ_2 则控制概率的“软度”或分布范围。图 2.10 展示了使用 logit 模型的 $P(x^1 \mid y)$ 的图表。

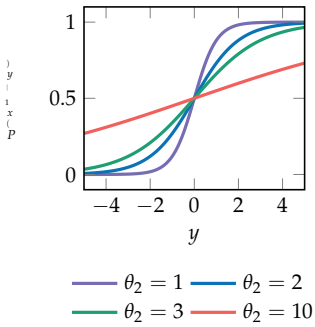


图2.10. θ_1 =为0且θ_2取不同值时的logit模型。

2.4.6 Deterministic Variables

一些问题可能涉及一个*deterministic variable*，其值在给定证据下是固定的。换句话说，我们给一个作为其证据的确定性函数的值赋予概率1。使用条件概率表来表示离散确定性变量是可能的，但这样做很浪费。对于每个父变量实例化，单个变量实例化将以概率1出现，而其余条目将为0。我们的实现可以利用这种稀疏性来实现更紧凑的表示。本文中使用离散因子的算法将因子表中缺失的任何赋值视为值0，从而使我们只需存储具有非零概率的赋值。

15 关于贝叶斯网络及其他形式的概率图模型的深入讨论，参见D. Koller和N. Friedman, *Probabilistic Graphical Models: Principles and Techniques*, MIT Press, 2009[4]。

2.5 Bayesian Networks

可以使用 *Bayesian network* 表示联合概率分布。[15] 该
贝叶斯网络的结构由一个由节点和有向边组成的*directed acyclic graph*定义。[16] 每个节点对应一个变量。有向边
连接成对的节点，禁止图中出现循环。有向

16 附录A.16回顾了常见的图论术语。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

边表示直接的概率关系。[17] 与每个节点 X_i 相关联的是一个条件分布 $P(X_i \mid \text{Pa}(X_i))$，其中 $\text{Pa}(X_i)$ 表示图中 X_i 的父节点。算法 2.2 提供了贝叶斯网络数据结构的实现。例 2.5 说明了贝叶斯网络在卫星监测问题中的应用。

17 在*causal networks*中，边的方向表示变量之间的因果 关系。然 而，在一般的贝叶斯网络中，并不要求因果关系。J. Pearl, *Causality: Models, Reasoning, and Inference*，第2版. 剑桥大学 出版社，2009年。

```
struct BayesianNetwork
    vars::Vector{Variable}
    factors::Vector{Factor}
    graph::SimpleDiGraph{Int64}
end
```

算法 2.2。基于一组变量、因子和图表示的离散贝叶斯网络。图数据结构由 `Graphs.jl` 提供。

贝叶斯网络的 *chain rule* 指定了如何从局部条件概率分布构造联合分布。假设我们有变量 $X_{1:n}$，并想要计算所有这些变量对值 $P(x_{1:n})$ 的特定赋值的概率。链式法则表明

$$P(x_{1:n}) = \prod_{i=1}^{n} P(x_i \mid \text{pa}(x_i)) \tag{2.31}$$

其中 $\text{pa}(x_i)$ 是 X_i 的父节点到其值的特定赋值。算法 2.3 提供了一种针对具有以离散因子表示的条件概率分布的贝叶斯网络的实现。

```
function probability(bn::BayesianNetwork, assignment)
    subassignment(ϕ) = select(assignment, variablenames(ϕ))
    probability(ϕ) = get(ϕ.table, subassignment(ϕ), 0.0)
    return prod(probability(ϕ) for ϕ in bn.factors)
end
```

算法2.3。给定贝叶斯网络`bn`，计算一个赋值的概率的函数。例如，如果`bn`如例2.5所定义，则

```
a = (b=1,s=1,e=1,d=2,c=1)
probability(bn, assignment(a))
```

返回0.034228055999999996。

在卫星示例中，假设我们要计算没有任何故障的概率；即，$P(b^0, s^0, e^0, d^0, c^0)$。根据链式法则，

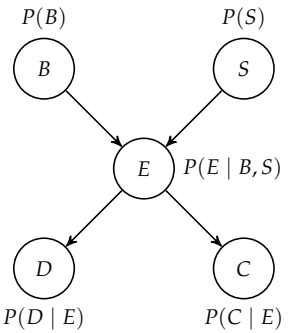
$$P(b^0, s^0, e^0, d^0, c^0) = P(b^0)P(s^0)P(e^0 \mid b^0, s^0)P(d^0 \mid e^0)P(c^0 \mid e^0) \tag{2.32}$$

如果我们完全指定了五个变量 B、S、E、D 和 C 的联合分布，那么我们将需要 $2^5 - 1 = 31$ 个独立参数。我们的贝叶斯网络中假设的结构允许我们仅使用 $1 + 1 + 4 + 2 + 2 = 10$ 个独立参数来指定联合分布。两者之间的差异

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

页边展示了一个涉及五个二元变量的卫星监测问题的贝叶斯网络。幸运的是，电池故障和太阳能板故障都较为罕见，尽管太阳能板故障比电池故障略为常见。任一故障都可能导致电气系统故障。除了电池或太阳能板故障外，电气系统故障还可能由其他原因引起，比如电源管理单元的问题。电气系统故障可导致轨迹偏离，这可通过地球上的望远镜观察到，同时也会造成通信中断，影响遥测数据和任务数据向下传输至各个地面站。不涉及电气系统的其他异常也可能导致轨迹偏离和通信中断。

例2.5. 一个表示卫星监控问题的贝叶斯网络。此处展示的是该网络作为有向无环图的结构。每个节点都关联着一个条件概率分布。



B 电池故障 *S* 太阳能板故障 *E* 电力系统故障 *D* 轨迹偏差 *C* 通信中断

与这五个变量中的每一个相关联的是五个条件概率分布。因为 B 和 S 没有父节点，我们只需要指定 $P(B)$ 和 $P(S)$。此处的代码创建了一个贝叶斯网络结构，并为相关因子表的元素提供了示例值。因子表中的元组索引变量的域，对于所有变量而言，该域为 $\{0, 1\}$。例如，`(e=2,b=1,s=1)` 对应于 (e^1, b^0, s^0)。

```
# requires convenience functions from appendix G.5
B = Variable(:b, 2); S = Variable(:s, 2)
E = Variable(:e, 2)
D = Variable(:d, 2); C = Variable(:c, 2)
vars = [B, S, E, D, C]
factors = [
    Factor([B], FactorTable((b=1,) ⇒ 0.99, (b=2,) ⇒ 0.01)),
    Factor([S], FactorTable((s=1,) ⇒ 0.98, (s=2,) ⇒ 0.02)),
    Factor([E,B,S], FactorTable(
        (e=1,b=1,s=1) ⇒ 0.90, (e=1,b=1,s=2) ⇒ 0.04,
        (e=1,b=2,s=1) ⇒ 0.05, (e=1,b=2,s=2) ⇒ 0.01,
        (e=2,b=1,s=1) ⇒ 0.10, (e=2,b=1,s=2) ⇒ 0.96,
        (e=2,b=2,s=1) ⇒ 0.95, (e=2,b=2,s=2) ⇒ 0.99)),
    Factor([D, E], FactorTable(
        (d=1,e=1) ⇒ 0.96, (d=1,e=2) ⇒ 0.03,
        (d=2,e=1) ⇒ 0.04, (d=2,e=2) ⇒ 0.97)),
    Factor([C, E], FactorTable(
        (c=1,e=1) ⇒ 0.98, (c=1,e=2) ⇒ 0.01,
        (c=2,e=1) ⇒ 0.02, (c=2,e=2) ⇒ 0.99))
]
graph = SimpleDiGraph(5)
add_edge!(graph, 1, 3); add_edge!(graph, 2, 3)
add_edge!(graph, 3, 4); add_edge!(graph, 3, 5)
bn = BayesianNetwork(vars, factors, graph)
```

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

10 和 31 并不代表在参数数量上的特别显著节省，但在更大的贝叶斯网络中，节省的量可以变得非常巨大。贝叶斯网络的力量在于它们能够减少指定联合概率分布所需的参数数量。

2.6 Conditional Independence

贝叶斯网络能够用比通常所需更少的独立参数来表示联合分布的原因，是其图形结构中编码的*conditional in-dependence*假设。[18]条件独立

依赖性是对2.3.1节中引入的独立性概念的推广。变量X和Y在给定Z的条件下条件独立，当且仅当$P(X, Y \mid Z) = P(X \mid Z)P(Y \mid Z)$。关于$X$和$Y$在给定$Z$条件下条件独立的断言写作$(X \perp Y \mid Z)$。根据此定义可以证明，$(X \perp Y \mid Z)$当且仅当$P(X \mid Z) = P(X \mid Y, Z)$。给定$Z$时，关于$Y$的信息不会提供关于$X$的额外信息，反之亦然。例2.6展示了这种情况的一个实例。

18 如果贝叶斯网络所做的条件独立假设无效，那么我们就有可能无法正确地对联合分布建模，这将在第5章中讨论。

假设在已知是否存在电力系统故障（E）的条件下，卫星轨迹偏差（D）的出现与是否发生通信丢失（C）条件独立。我们将其表示为（$D \perp C \mid E$）。如果我们知道发生了电力系统故障，那么观察到通信丢失这一事实不会影响我们对存在轨迹偏差的信念。我们可能对存在轨迹偏差有较高的预期，但这仅仅是因为我们知道发生了电力系统故障。

示例 2.6. 卫星跟踪问题中的条件独立性。

我们可以使用一组规则来确定贝叶斯网络的结构是否意味着在给定一组其他证据变量的条件下，两个变量必须条件独立。[19] 假设我们想要检查是否（$A \perp B \mid \mathcal{C}$）由网络结构隐含，其中$\mathcal{C}$是一组证据变量。我们必须检查从$A$到$B$的所有可能的无向路径，以确定所谓的*d-separation*。如果以下任一条件成立，则A与B之间的路径被$\mathcal{C}$ d-分离：

19 即使网络结构不意味着条件独立，由于条件概率分布的选择，仍可能存在条件独立。参见练习2.10。

1. 路径包含一个节点*chain*，$X \rightarrow Y \rightarrow Z$，使得$Y$在$\mathcal{C}$中。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

2. 路径包含一个*fork*，$X \leftarrow Y \rightarrow Z$，使得$Y$位于$\mathcal{C}$中。

3. 路径包含一个*inverted fork* (，也称为*v-structure*)、$X \rightarrow Y \leftarrow Z$，使得$Y$在$\mathcal{C}$中为*not*，且$Y$的任何后代都不在$\mathcal{C}$中。例2.7为这一规则提供了一些直观理解。

如果A和B之间的所有路径都被$\mathcal{C}$ d-分离，那么我们就说A和B被$\mathcal{C}$ d-分离。这种d-分离意味着$(A \perp B \mid \mathcal{C})$。[20] 例2.8
演示了检查图是否蕴含特定条件独立性假设的过程。

20 一种高效确定 d-分离的算法有点复杂。参见D. Koller和N. Friedman，*Probabilis*-中的算法3.1。麻省理工学院出版社，2009年。

tic Graphical Models: Principles and Techniques

例2.7. 链、叉和倒叉中隐含（及不隐含）的条件独立假设的直观理解。

如果我们有一个$X \rightarrow Y \rightarrow Z$ (链)或$X \leftarrow Y \rightarrow Z$ (叉)，且在Y处有证据，那么X和Z是条件独立的，意味着$P(X \mid Y, Z) = P(X \mid Y)$。有趣的是，如果箭头方向稍有不同，即$X \rightarrow Y \leftarrow Z$ (倒叉)，那么给定Y时，X和Z可能不再是条件独立的。换句话说，可能存在这样的情况：
$P(B \mid E) \neq P(B \mid S, E)$。为了提供一些直观理解，考虑从电池故障$B$经由电气系统故障$E$到太阳能板故障$S$的倒叉路径。假设我们知道存在电气故障。如果我们知道没有电池故障，那么我们更倾向于相信存在太阳能板故障，因为它是电气故障的另一个可能原因。相反，如果我们发现确实存在电池故障，那么我们对存在太阳能板故障的信念就会减弱。这种效应被称为*explaining away*。观察到太阳能板故障解释了电气系统故障的原因。

有时节点 X 的术语 *Markov blanket*[21] 用来指代最小的
如果已知其值的节点集合，使得X条件独立于所有其他节点。特定节点的马尔可夫毯最终由其父节点、子节点以及其子节点的其他父节点组成。

21 以俄罗斯数学家安德烈·安德烈耶维奇·马尔可夫（1856–1922）命名。J. Pearl，*Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference* 。Morgan Kaufmann，1988。

2.7 Summary

- 将不确定性表示为概率分布是由一组与比较不同陈述的合理性相关的公理驱动的。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

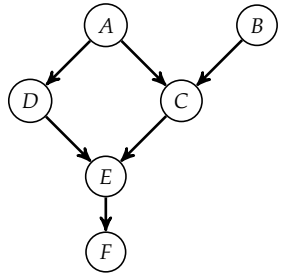
假设我们想要确定边缘显示的网络是否蕴含 $(D \perp B \mid F)$。从 D 到 B 有两条无向路径。我们需要检查这两条路径的 d-分离性。

路径 $D \leftarrow A \rightarrow C \leftarrow B$ 涉及分叉 $D \leftarrow A \rightarrow C$，随后是一个倒置分叉 $A \rightarrow C \leftarrow B$。在 A 处没有证据，因此从分叉处不存在 d-分离。由于 F 是 C 的后代，沿倒置分叉不存在 d-分离。因此，沿此路径不存在 d-分离。

第二条路径 $D \rightarrow E \leftarrow C \leftarrow B$ 涉及倒叉 $D \rightarrow E \leftarrow C$ 和一条链 $E \leftarrow C \leftarrow B$。由于 F 是 E 的后代，沿倒叉不存在 d-分离。同样地，由于沿该路径的链部分也不存在 d-分离，因此从 D 到 B 沿这条路径不存在 d-分离。

为了使 D 和 B 在给定 F 的条件下条件独立，从 D 到 B 的所有无向路径上必须存在 d-分离。在这种情况下，两条路径上都没有 d-分离。因此，条件独立性并不由网络结构所蕴含。

示例 2.8. 以下图形结构所隐含的条件独立假设。



- 存在许多离散和连续概率分布族。
- 连续概率分布可以用密度函数来表示。
- 概率分布族可以通过混合组合，创建更灵活的分布。
- 联合分布是多个变量上的分布。
- 条件分布是在给定证据变量取值的情况下，对一个或多个变量的分布。
- 贝叶斯网络由图结构和一组条件分布定义。
- 根据贝叶斯网络的结构，由于条件独立性假设，我们可以用更少的参数来表示联合分布。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

2.8 Exercises

练习2.1。考虑一个连续随机变量X，它服从由λ参数化的*exponential distri- bution*，密度函数为$p(x \mid \lambda) = \lambda \exp(-\lambda x)$，支持非负值。计算$X$的累积分布函数。

Solution: 我们从累积分布函数的定义开始。由于分布的下界由$x = 0$支撑，区间$(-\infty, 0)$内没有概率质量，允许我们将积分的下限调整为0。计算积分后，我们得到$\text{cdf}_X(x)$：

$$\begin{aligned}
\text{cdf}_X(x) &= \int_{-\infty}^{x} p(x')\,\mathrm{d}x' \\
\text{cdf}_X(x) &= \int_{0}^{x} \lambda e^{-\lambda x'}\,\mathrm{d}x' \\
\text{cdf}_X(x) &= -e^{-\lambda x'}\Big|_0^x \\
\text{cdf}_X(x) &= 1 - e^{-\lambda x}
\end{aligned}$$

练习 2.2。对于图 2.6 中的密度函数，混合模型的五个组成部分是什么？（存在多种有效解法。）

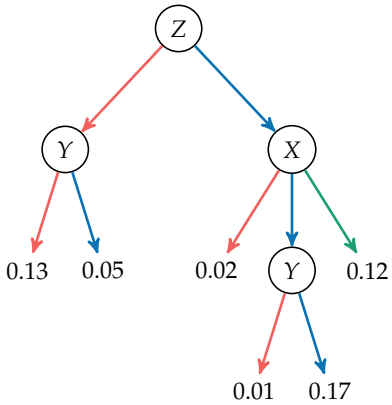
Solution: 一种解决方案是$\mathcal{U}([-10, -10], [-5, 10])$，$\mathcal{U}([-5, 0], [0, 10])$，$\mathcal{U}([-5, -10], [0, 0])$，$\mathcal{U}([0, -10], [10, 5])$，以及$\mathcal{U}([0, 5], [10, 10])$。

练习2.3。给定以下表格表示$P(X、Y、Z)$，生成一个等价且紧凑的决策树表示：

X	Y	Z	$P(X, Y, Z)$
0	0	0	0.13
0	0	1	0.02
0	1	0	0.05
0	1	1	0.02
1	0	0	0.13
1	0	1	0.01
1	1	0	0.05
1	1	1	0.17
2	0	0	0.13
2	0	1	0.12
2	1	0	0.05
2	1	1	0.12

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: 我们从最常见的概率开始：0.13，当Z =为0且Y =为0时出现，以及0.05，当Z =为0且Y =为1时出现。我们选择将Z作为决策树的根节点，当Z =为0时，我们继续到一个Y节点。根据Y的值，我们分支到0.13或0.05。接下来，我们继续处理Z =为1的情况。最常见的概率是0.02，当Z =为1且X =为0时出现，以及0.12，当Z =为1且X =为2时出现。因此，当Z =为1时，我们选择继续到一个X节点。根据X是0、1还是2，我们分别继续到0.02、一个Y节点或0.12。最后，根据Y的值，我们分支到0.01或0.17。



练习 2.4。假设我们想要指定一个定义在四个变量上的三组件多元高斯混合模型。我们要求三个高斯分布中的两个假设四个变量之间独立，而另一个高斯分布则不做任何独立假设。指定这个混合模型需要多少个独立参数?

Solution: 对于四个变量（$n = 4$）上的高斯分布，在独立性假设下，我们需要指定$n + n = 2n = 8$个独立参数；均值向量有四个参数，协方差矩阵有四个参数（这相当于四个独立的一元高斯分布的均值和方差参数）。对于四个变量上的高斯分布，在不假设独立性的情况下，我们需要指定$n + n(n + 1)/2 = 14$个独立参数；均值向量有4个参数，协方差矩阵有10个参数。此外，对于我们的三个混合分量（$k = 3$），我们需要为权重指定$k - 1 = 2$个独立参数。因此，我们需要$2(8) + 1(14) + 2 = 32$个独立参数来指定这个混合分布。

练习2.5。我们有三个独立变量$X_{1:3}$，它们分别由具有4、7和3个箱边缘的分段常数密度定义。指定它们的联合分布需要多少个独立参数?

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

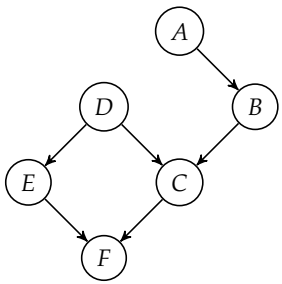
Solution: 如果我们的密度是分段常数且具有 m 个箱边缘，则有 $m-1$ 个箱和 $m-2$ 个独立参数。对于此问题，将会有 $(4-2)+(7-2)+(3-2)=8$ 个独立参数。

练习2.6。假设我们有四个连续随机变量，X_1、X_2、Y_1和Y_2，我们想要构建一个关于 $X = X_{1:2}$给定$Y = Y_{1:2}$的线性高斯模型；即，$p(X \mid Y)$。这个模型需要多少个独立参数?

Solution: 在这种情况下，高斯分布的均值向量是二维的，需要四个独立参数用于变换矩阵 M，以及两个独立参数用于偏置向量 b。我们还需要三个独立参数用于协方差矩阵 $\mathbf{\Sigma}$。总共，我们需要 $4+2+3=9$ 个独立参数来指定这个模型：

$$p(\mathbf{x} \mid \mathbf{y}) = \mathcal{N}(\mathbf{x} \mid \mathbf{M}\mathbf{y} + \mathbf{b}, \mathbf{\Sigma})$$

练习2.7。给定以下贝叶斯网络，其中每个节点可以取四个值之一，有多少个独立参数? 与使用完整的联合概率表相比，使用以下贝叶斯网络时所需独立参数数量的减少百分比是多少?

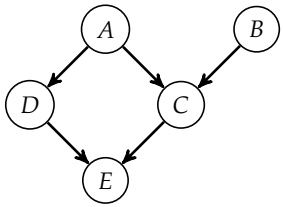


Solution: 每个节点的独立参数数量等于 $(k-1)k^m$，其中 k 是该节点可以取值的数量，m 是该节点的父节点数量。变量 A 有 3 个独立参数，B 有 12 个，C 有 48 个，D 有 3 个，E 有 12 个，F 有 48 个独立参数。这个贝叶斯网络总共有 126 个独立参数。

指定一张关于 n 个变量、每个变量可取 k 个值的联合概率表所需的独立参数数量等于 $k^n - 1$。因此，指定一张联合概率表将需要 $4^6 - 1 = 4096 - 1 = 4095$ 个独立参数。所需独立参数数量的减少百分比为 $(4095 - 126)/4095 \approx 96.9\%$。

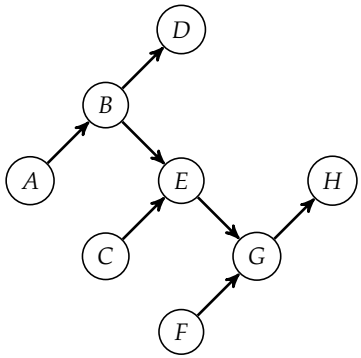
© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

练习2.8。给定以下贝叶斯网络，A在给定C的条件下与E是否d-分离？



Solution: 从A到E有两条路径：$A \rightarrow D \rightarrow E$和$A \rightarrow C \rightarrow E$。第二条路径存在d-分离，但第一条路径不存在。因此，给定C时，A与E不是d-分离的。

练习2.9。给定以下贝叶斯网络，确定B的马尔可夫毯：



Solution: 从B到A的路径只能在给定A时被d-分离。从B到D的路径只能在给定D时被d-分离。从B到E以及同时F、G和H的路径，在给定E时可以被高效地d-分离。从B到C的路径由于v-结构而自然d-分离；然而，由于E必须包含在我们的马尔可夫毯中，从B到C在给定E时的路径只能在给定C时被d-分离。因此，B的马尔可夫毯是$\{A、C、D、E\}$。

练习2.10。在结构为$A \rightarrow B$的贝叶斯网络中，A是否可能与B独立？

Solution: 从 A 到 B 有一条直接的箭头，这表明独立性并未被隐含。然而，这并不意味着它们不独立。A 和 B 是否独立取决于条件概率表的选择。我们可以选择使得存在独立性的表。例如，假设两个变量都是二元的，且 $P(a) = 0.5$ 是均匀的，$P(b \mid a) = 0.5$。显然，$P(A)P(B \mid A) = P(A)P(B)$，这意味着它们是独立的。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

3 *Inference*

上一章解释了如何表示概率分布。本章将展示如何使用这些概率表示进行 *inference*，这涉及在给定一组观测变量相关值的情况下，确定一个或多个未观测变量的分布。本章首先介绍精确推断方法。由于精确推断可能因网络结构而计算上不可行，我们还将讨论几种近似推断算法。

3.1 *Inference in Bayesian Networks*

在推断问题中，给定一些观测到的 *evidence variables*，我们希望推断出关于 *query variables* 的分布。其他节点被称为 *hidden variables*。我们通常将给定证据时查询变量上的分布称为 *posterior distribution*。

为了说明推理中涉及的计算，回顾例2.5中的贝叶斯网络，其结构在图3.1中再现。假设我们有一个查询变量 B 以及证据 $D = 1$ 和 $C = 1$。推理任务是计算 $P(b^1 \mid d^1, c^1)$，这对应于计算在观察到轨迹偏差和通信丢失的情况下，电池发生故障的概率。

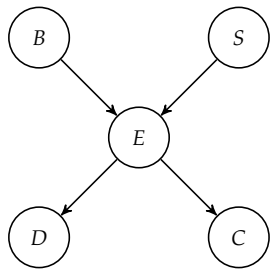


图3.1. 例2.5中的贝叶斯网络结构。

从公式(2.22)中引入的条件概率定义出发，我们了解到

$$P(b^1 \mid d^1, c^1) = \frac{P(b^1, d^1, c^1)}{P(d^1, c^1)} \tag{3.1}$$

为了计算分子，我们必须使用一个称为*marginalization*的过程，在此过程中，我们对不涉及的变量（在本例中为S和E）进行求和消除：

$$P(b^1, d^1, c^1) = \sum_s \sum_e P(b^1, s, e, d^1, c^1) \tag{3.2}$$

我们从公式（2.31）中引入的贝叶斯网络链式法则得知

$$P(b^1, s, e, d^1, c^1) = P(b^1)P(s)P(e \mid b^1, s)P(d^1 \mid e)P(c^1 \mid e) \tag{3.3}$$

右侧的所有组件都在与贝叶斯网络中节点相关的条件概率分布中指定。我们可以使用相同的方法计算公式（3.1）中的分母，但需对B的值进行额外求和。

利用条件概率的定义、边缘化以及应用链式法则的过程，可以用于在任何贝叶斯网络中进行精确推理。我们能够通过因子来实现精确推理。回忆一下，因子代表离散多变量分布。为了达到这一目的，我们对因子执行以下三种操作：

- 我们使用*factor product*（算法3.1）来合并两个因子，生成一个更大的因子，其作用域是输入因子的联合作用域。若我们有$\phi(X、Y)$和$\psi(Y、Z)$，则$\phi \cdot \psi$将覆盖X、Y和Z，其中$(\phi \cdot \psi)(x、y、z) = \phi(x、y)\psi(y、z)$。因子乘积在例3.1中演示。

- 我们使用*factor marginalization*（算法3.2）从整个因子表中对特定变量进行求和消除，将其从结果作用域中移除。例3.2展示了这一过程。

- 我们使用关于某些证据的*factor conditioning*（算法3.3）来删除表中与该证据不一致的任何行。例3.3演示了因子条件化。

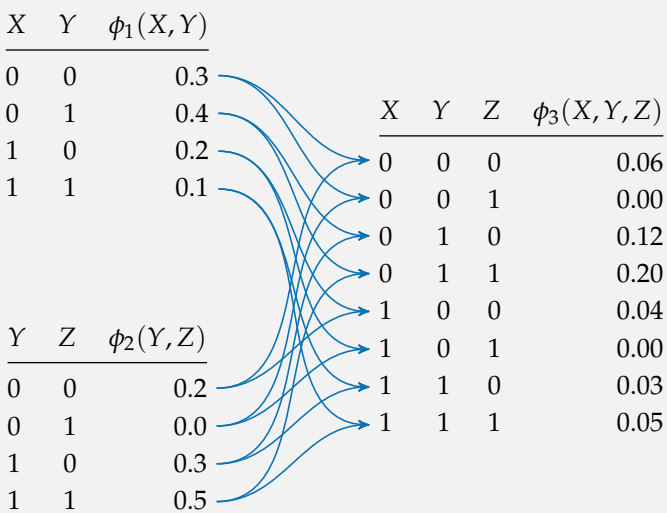
这三种因子运算在算法3.4中协同使用，以执行精确推理。该算法首先计算所有因子的乘积，基于证据进行条件化，边缘化隐藏变量，并进行归一化。这种方法的一个潜在问题是所有因子乘积的大小。因子乘积的大小等于每个变量可能取值的数量的乘积。对于卫星示例问题，仅有$2^5 = 32$种可能的赋值，但许多有趣的问题其因子乘积会大到实际上无法枚举。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
function Base.:*(ϕ::Factor, ψ::Factor)
    ϕnames = variablenames(ϕ)
    ψnames = variablenames(ψ)
    ψonly = setdiff(ψ.vars, ϕ.vars)
    table = FactorTable()
    for (ϕa,ϕp) in ϕ.table
        for a in assignments(ψonly)
            a = merge(ϕa, a)
            ψa = select(a, ψnames)
            table[a] = ϕp * get(ψ.table, ψa, 0.0)
        end
    end
    vars = vcat(ϕ.vars, ψonly)
    return Factor(vars, table)
end
```

算法3.1。因子乘积的一种实现，该算法构建表示两个较小因子ϕ和ψ联合分布的因子。若要计算ϕ和ψ的因子乘积，我们只需写出ϕ*ψ。

两个因子的因子乘积在它们变量的并集上产生一个新的因子。这里，我们从共享一个变量的两个因子中产生一个新的因子：



例3.1. 一个因子乘积的示例，该乘积结合了代表$\phi_1(X、Y)$与$\phi_2(Y、Z)$的两个因子，生成一个代表$\phi_3(X、Y、Z)$的因子。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
function marginalize(ϕ::Factor, name)
    table = FactorTable()
    for (a, p) in ϕ.table
        a′ = delete!(copy(a), name)
        table[a′] = get(table, a′, 0.0) + p
    end
    vars = filter(v -> v.name != name, ϕ.vars)
    return Factor(vars, table)
end
```

算法3.2. 一种从因子ϕ中边缘化名为name的变量的方法。

回忆表2.1中的联合概率分布$P(X、Y、Z)$。我们可以通过将每一行中对于X和Z具有匹配赋值的概率相加，来边缘化掉Y:

X	Y	Z	$\phi(X,Y,Z)$
0	0	0	0.08
0	0	1	0.31
0	1	0	0.09
0	1	1	0.37
1	0	0	0.01
1	0	1	0.05
1	1	0	0.02
1	1	1	0.07

X	Z	$\phi(X,Z)$
0	0	0.17
0	1	0.68
1	0	0.03
1	1	0.12

示例 3.2. 因子边缘化的演示。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
in_scope(name, ϕ) = any(name == v.name for v in ϕ.vars)

function condition(ϕ::Factor, name, value)
    if !in_scope(name, ϕ)
        return ϕ
    end
    table = FactorTable()
    for (a, p) in ϕ.table
        if a[name] == value
            table[delete!(copy(a), name)] = p
        end
    end
    vars = filter(v -> v.name != name, ϕ.vars)
    return Factor(vars, table)
end

function condition(ϕ::Factor, evidence)
    for (name, value) in pairs(evidence)
        ϕ = condition(ϕ, name, value)
    end
    return ϕ
end
```

算法 3.3。给定一些证据时对因子进行条件化的两种方法。第一种方法接受一个因子 `ϕ`，并返回一个新的因子，其表中的条目与名为 `name` 的变量取值为 `value` 相一致。第二种方法接受一个因子 `ϕ`，并以命名元组的形式应用证据。`in_scope` 方法在名为 `name` 的变量位于因子 `ϕ` 的范围内时返回真。

因子条件化涉及删除任何与证据不一致的行。这里是表2.1中的因子，并且我们以$Y = 1$为条件。所有$Y \neq 1$的行都被移除：

X	Y	Z	$\phi(X,Y,Z)$
0	0	0	0.08
0	0	1	0.31
0	1	0	0.09
0	1	1	0.37
1	0	0	0.01
1	0	1	0.05
1	1	0	0.02
1	1	1	0.07

$Y = 1$

X	Z	$\phi(X,Z)$
0	0	0.09
0	1	0.37
1	0	0.02
1	1	0.07

例 3.3. 因子中设置证据的一个示例，本例中针对 Y。得到的结果值必须重新归一化。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct ExactInference end

function infer(M::ExactInference, bn, query, evidence)
    ϕ = prod(bn.factors)
    ϕ = condition(ϕ, evidence)
    for name in setdiff(variablenames(ϕ), query)
        ϕ = marginalize(ϕ, name)
    end
    return normalize!(ϕ)
end
```

算法3.4。一个针对离散贝叶斯网络`bn`的朴素精确推理算法，其输入是一组查询变量名`query`以及与观察变量相关联的值`evidence`。该算法以因子的形式计算查询变量上的联合分布。我们引入`ExactInference`类型，以便允许使用不同的推理方法调用`infer`，这一点将在本章后续部分中看到。

3.2 *Inference in Naive Bayes Models*

前一节介绍了一种在任何贝叶斯网络中进行精确推断的通用方法。本节讨论如何用同样的方法求解一种特殊贝叶斯网络结构（称为*naive Bayes*模型）的*classification*问题。该结构如图3.2所示。图3.3使用*plate*给出了一个等价但更紧凑的表示，这里将其显示为一个圆角框。框底部的$i = 1:n$指定变量名下标中的i从1重复到n。

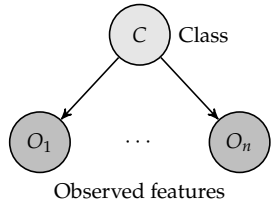


图3.2. 朴素贝叶斯模型。

在朴素贝叶斯模型中，类别 C 是查询变量，而观察到的特征 $O_{1:n}$ 是证据变量。朴素贝叶斯模型之所以被称为“朴素”，是因为它假设在给定类别的条件下，证据变量之间相互独立。使用第2.6节引入的符号，我们可以说对于所有 $i \neq j$，有 $(O_i \perp O_j \mid C)$。当然，如果这些条件独立假设不成立，那么我们可以在观察到的特征之间添加必要的有向边。

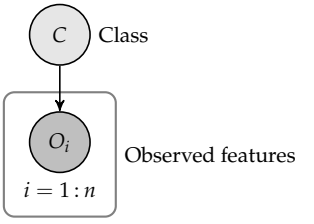


图3.3. 朴素贝叶斯模型的板块表示。

我们必须指定 *prior* $P(C)$ 和 *class-conditional distributions* $P(O_i \mid C)$。如前一节所述，我们可以应用链式法则来计算联合分布：

$$P(c, o_{1:n}) = P(c) \prod_{i=1}^{n} P(o_i \mid c) \tag{3.4}$$

我们的分类任务涉及计算条件概率 $P(c \mid o_{1:n})$。根据条件概率的定义，我们有

$$P(c \mid o_{1:n}) = \frac{P(c, o_{1:n})}{P(o_{1:n})} \tag{3.5}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们可以通过边缘化联合分布来计算分母：

$$P(o_{1:n}) = \sum_c P(c, o_{1:n}) \tag{3.6}$$

方程（3.5）中的分母不是 C 的函数，因此可以被视为常数。于是，我们可以写出

$$P(c \mid o_{1:n}) = \kappa P(c, o_{1:n}) \tag{3.7}$$

其中 κ 是一个 *normalization constant*，使得 $\sum_c P(c \mid o_{1:n}) = 1$。我们经常省略 κ，并写作

$$P(c \mid o_{1:n}) \propto P(c, o_{1:n}) \tag{3.8}$$

其中符号 *proportional to* 表示左侧与右侧成正比。例3.4说明了推理如何应用于雷达轨迹分类。

我们可以使用这种方法来推断类别上的分布，但对于许多应用而言，我们必须确定一个特定的类别。通常的做法是根据具有最高后验概率的类别进行分类，即 $\arg\max_c P(c \mid o_{1:n})$。然而，选择类别实际上是一个决策问题，通常应考虑误分类的后果。例如，如果我们有兴趣使用分类器来滤除不是飞机的目标，以实现空中交通管制的目的，那么我们可以承受偶尔让一些鸟类和其他杂波轨迹通过我们的滤波器。但是，我们希望避免滤除任何真实的飞机，因为这可能导致碰撞。在这种情况下，我们可能只想在后验概率接近1时，才将轨迹分类为鸟类。决策问题将在第6章讨论。

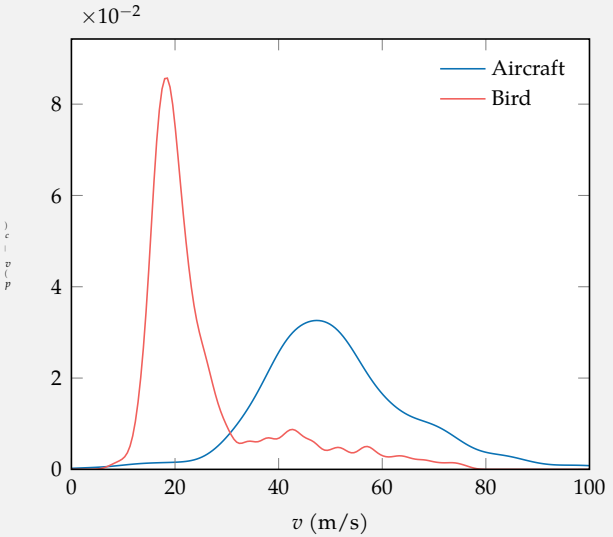
3.3 Sum-Product Variable Elimination

多种方法可用于在更复杂的贝叶斯网络中进行高效推理。一种方法称为 *sum-product variable elimination*，它将消除隐藏变量（求和）与链式法则的应用（乘积）交替进行。尽早将变量边缘化以避免产生大的因子会更高效。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

例3.4. 雷达目标分类中，我们需要判断雷达轨迹对应的是鸟类还是飞机。

假设我们有一个雷达航迹，并希望判断它是由鸟类还是飞机生成的。我们的推断基于空速和航向波动的程度。前者代表了我们在没有航迹任何信息的情况下，对目标是鸟类还是飞机的信念。以下是基于雷达数据估计的空速v的类别条件分布示例：



假设根据链式法则，我们确定：

$$P(\text{鸟，慢速，小航向波动}) = 0.03\ P(\text{飞机，慢速，小航向波动}) = 0.01$$

当然，这些概率之和不等于1。如果我们想要根据证据确定目标为鸟的概率，那么我们将进行以下计算：

$$P(\text{鸟} \mid \text{慢，少量航向波动}) = \frac{0.03}{0.03 + 0.01} = 0.75$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们将通过计算图3.1中贝叶斯网络的分布 $P(B \mid d^1, c^1)$ 来说明变量消除算法。网络中节点相关的条件概率分布可以用以下因子表示：

$$\phi_1(B), \phi_2(S), \phi_3(E, B, S), \phi_4(D, E), \phi_5(C, E) \tag{3.9}$$

因为 D 和 C 是观测变量，所以通过设置证据 $D = 1$ 和 $C = 1$，最后两个因子可以替换为 $\phi_6(E)$ 和 $\phi_7(E)$。

然后我们依次消除隐藏变量。可以选择不同的策略来确定顺序，但在本例中，我们随意选择顺序E，然后是S。为了消除E，我们取所有涉及E的因子的乘积，然后对E进行边缘化，得到一个新的因子：

$$\phi_8(B, S) = \sum_e \phi_3(e, B, S)\phi_6(e)\phi_7(e) \tag{3.10}$$

我们现在可以丢弃 ϕ_3、ϕ_6 和 ϕ_7，因为它们所提供的所需信息都已包含在 ϕ_8 中。

接下来，我们消除S。再次，我们收集所有涉及S的剩余因子，并从这些因子的乘积中边缘化掉S：

$$\phi_9(B) = \sum_s \phi_2(s)\phi_8(B, s) \tag{3.11}$$

我们丢弃 ϕ_2 和 ϕ_8，剩下 $\phi_1(B)$ 和 $\phi_9(B)$。最后，我们将这两个因子相乘并对结果进行归一化，得到一个代表 $P(B \mid d^1、c^1)$ 的因子。

此过程等效于计算以下内容：

$$P(B \mid d^1, c^1) \propto \phi_1(B) \sum_s \left(\phi_2(s) \sum_e \left(\phi_3(e \mid B, s)\phi_4(d^1 \mid e)\phi_5(c^1 \mid e) \right) \right) \tag{3.12}$$

这产生的结果与取所有因子的乘积然后进行边缘化的朴素过程相同，但比该过程更高效：

$$P(B \mid d^1, c^1) \propto \sum_s \sum_e \phi_1(B)\phi_2(s)\phi_3(e \mid B, s)\phi_4(d^1 \mid e)\phi_5(c^1 \mid e) \tag{3.13}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

和积变量消元算法在算法3.5中实现。它以贝叶斯网络、一组查询变量、一个观测值列表和变量顺序作为输入。我们首先设置所有观测值。然后，对于每个变量，我们将包含它的所有因子相乘，然后将该变量边缘化。这个新因子替换消耗的因子，我们对下一个变量重复该过程。

对于许多网络而言，变量消除使得推理所需的时间与网络规模成线性关系，但在最坏情况下其时间复杂度为指数级。影响计算量的是变量消除的顺序。选择最优消除顺序属于*NP-hard*、[1]，这意味着无法在多项式时间内完成。

最坏情况（第3.5节）。即使我们找到了最优的消元顺序，变量消元仍然可能需要指数级的计算次数。变量消元启发式方法通常试图尽量减少算法生成的中间因子所涉及的变量数量。

1 S. Arnborg、D. G. Corneil 和 A. Proskurowski，“在*k*-树中寻找嵌入的复杂性”，*SIAM Journal on Algebraic Discrete Methods*，第8卷，第2期，第277-284页，1987年。

```
struct VariableElimination
    ordering # array of variable indices
end

function infer(M::VariableElimination, bn, query, evidence)
    Φ = [condition(ϕ, evidence) for ϕ in bn.factors]
    for i in M.ordering
        name = bn.vars[i].name
        if name ∉ query
            inds = findall(ϕ->in_scope(name, ϕ), Φ)
            if !isempty(inds)
                ϕ = prod(Φ[inds])
                deleteat!(Φ, inds)
                ϕ = marginalize(ϕ, name)
                push!(Φ, ϕ)
            end
        end
    end
    return normalize!(prod(Φ))
end
```

算法3.5。和积变量消除算法的一种实现，该算法接收贝叶斯网络`bn`、查询变量列表`query`以及证据`evidence`。变量按照`ordering`给定的顺序进行处理。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

3.4 Belief Propagation

一种称为*belief propagation*的推理方法通过使用*sum-product algorithm*在网络中传播“消息”来计算查询变量的边际分布。[2]信念传播需要

线性时间，但仅当网络中没有无向环时才能提供精确答案。如果网络中存在无向环，则可以通过使用所谓的*junction tree algorithm*将多个变量合并为单个节点，从而将其转换为树。如果结果网络中任一节点所需合并的变量数量较少，则推断可以高效进行。一种称为*loopy belief propagation*的信念传播变体可以在具有无向环的网络中提供近似解。尽管这种方法不提供任何保证，且可能不收敛，但在实践中可以很好地工作。[3]

2 关于和积算法的教程，并讨论了其与不同社区中开发的许多其他算法的联系，由 F. Kschischang、B. Frey 和 H.-A. Loeliger 提供，“因子图与和积算法”，*IEEE Transactions on Information Theory*，第 47 卷，第 2 期，第 498–519 页，2001 年。

3 信念传播及相关算法由D. Barber详细阐述，*Bayesian Reasoning and Machine Learning*，剑桥大学出版社，2012年。

3.5 Computational Complexity

我们可以证明贝叶斯网络中的推断是NP难的，通过使用一个称为3SAT的NP完全问题。[4]从
一个任意的3SAT问题。例如，考虑以下3SAT公式：[5]

$$F(x_1, x_2, x_3, x_4) = \begin{pmatrix} x_1 & \vee & x_2 & \vee & x_3 \end{pmatrix} \wedge \begin{pmatrix} \neg x_1 & \vee & \neg x_2 & \vee & x_3 \end{pmatrix} \wedge \begin{pmatrix} x_2 & \vee & \neg x_3 & \vee & x_4 \end{pmatrix} \tag{3.14}$$

其中 $\neg$ 表示 *logical negation*（“非”），$\wedge$ 表示 *logical conjunction*（“与”）而 $\vee$ 表示 *logical disjunction*（“或”）。该公式由*clauses*的合取构成，这些是所谓*literals*的析取。一个文字就是一个变量或其否定。

4 G. F. Cooper，“使用贝叶斯信念网络进行概率推理的计算复杂性”，*Artificial Intelligence*，第42卷，第2–3期，第393–405页，1990年。本节中的贝叶斯网络构建遵循该文献。关于复杂性类别的简要回顾，请参见附录C。

5 这个公式也出现在附录C的示例C.3中。

图3.4显示了相应的贝叶斯网络表示。变量由$X_{1:4}$表示，子句由$C_{1:3}$表示。变量上的分布是均匀的。表示子句的节点以参与的变量为父节点。由于这是一个3SAT问题，每个子句节点恰好有三个父节点。每个子句节点对于不满足该子句的赋值分配概率0，对于所有满足的赋值分配概率1。其余节点如果其所有父节点

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

是真的。原问题可满足当且仅当$P(y^1)$ >=0。因此，贝叶斯网络中的推断至少和3SAT问题一样困难。

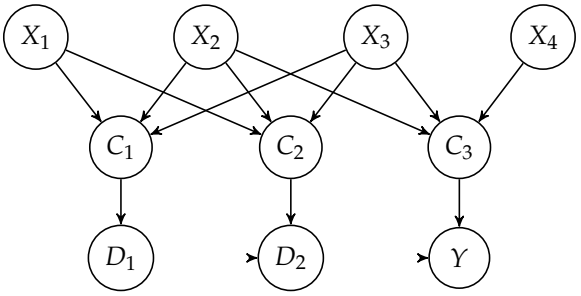


图3.4. 表示3SAT问题的贝叶斯网络。

我们之所以费心证明贝叶斯网络中的推断是NP难的，是为了避免浪费时间寻找适用于所有贝叶斯网络的高效精确推断算法。因此，过去几十年里的研究集中在近似推断方法上，接下来将对此进行讨论。

3.6 Direct Sampling

由于精确推断在计算上是不可行的，因此开发了许多近似方法。最简单的推断方法之一是基于 *direct sampling*，其中利用联合分布的随机样本得出概率估计。[6] 为了说明这一点，

假设我们想要从联合分布 $P(b、s、e、d、c)$ 中的一组 n 个样本推断 $P(b^1 \mid d^1、c^1)$。我们使用括号上标来表示样本的索引，其中我们将第 i 个样本写为 ($b^{(i)}、s^{(i)}、e^{(i)}、d^{(i)}、c^{(i)}$)。直接样本估计是

ized Algorithms

$$P(b^1 \mid d^1, c^1) \approx \frac{\sum_i (b^{(i)} = 1 \wedge d^{(i)} = 1 \wedge c^{(i)} = 1)}{\sum_i (d^{(i)} = 1 \wedge c^{(i)} = 1)} \tag{3.15}$$

我们约定，括号内的逻辑陈述在数值上视为真时为1，假时为0。分子是与b、d和c均设为1一致的样本数，分母是与d和c均设为1一致的样本数。

6 有时涉及随机抽样的方法被称为 *Monte Carlo methods*。这个名字来源于摩纳哥的蒙特卡洛赌场。R. Motwani 和 P. Raghavan 在 *Random-* 中介绍了随机算法及其在各种问题领域的应用。剑桥大学出版社，1995年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

从贝叶斯网络所表示的联合分布中进行采样是直接了当的。第一步涉及找到贝叶斯网络中节点的一个*topological sort*。有向无环图中节点的拓扑排序是一个有序列表，如果存在一条边$A \to B$，那么在该列表中A位于B之前。[7] 例如，图3.1中网络的一个拓扑排序是B、S、E、D、C。

拓扑排序总是存在，但可能不唯一。该网络的另一个拓扑排序是S, B, E, C, D。

一旦我们有了拓扑排序，就可以开始从条件概率分布中采样。算法3.6展示了如何在给定一个表示拓扑排序的序列$X_{1:n}$的情况下，从贝叶斯网络中采样。我们根据已经赋值的父节点的值，从与X_i相关的条件分布中抽取一个样本。由于$X_{1:n}$是一个拓扑排序，我们知道X_i的所有父节点都已经被实例化，这使得采样得以进行。直接采样在算法3.7中实现，并在例3.5中进行了演示。

7 A. B. Kahn，“大型网络的拓扑排序”，*Communications of the ACM*，第 5 卷，第 11 期，第 558–562 页，1962 年。拓扑排序的实现由 `Graphs.jl` 包提供。

```
function Base.rand(ϕ::Factor)
    tot, p, w = 0.0, rand(), sum(values(ϕ.table))
    for (a,v) in ϕ.table
        tot += v/w
        if tot >= p
            return a
        end
    end
    return Assignment()
end

function Base.rand(bn::BayesianNetwork)
    a = Assignment()
    for i in topological_sort(bn.graph)
        name, ϕ = bn.vars[i].name, bn.factors[i]
        a[name] = rand(condition(ϕ, a))[name]
    end
    return a
end
```

算法3.6. 一种从贝叶斯网络`bn`中采样赋值的方法。我们也提供了一种从因子`ϕ`中采样赋值的方法。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

假设我们从图3.1的网络中随机抽取10个样本。我们感兴趣的是推断$P(b^1 \mid d^1$、$c^1)$。在这10个样本中（表中已指出），只有2个样本与观测值d^1和c^1一致。一个样本中b =为1，另一个样本中b =为0。根据这些样本，我们推断$P(b^1 \mid d^1$、$c^1)$ =为0.5。当然，为了准确估计$P(b^1 \mid d^1$、$c^1)$，我们希望使用超过2个样本。

B	S	E	D	C	
0	0	1	1	0	
0	0	0	0	0	
1	0	1	0	0	
1	0	1	1	1	←
0	0	0	0	0	
0	0	0	1	0	
0	0	0	0	1	
0	1	1	1	1	←
0	0	0	0	0	
0	0	0	1	0	

例3.5. 一个示例，说明如何利用来自贝叶斯网络的直接样本进行推断。

```
struct DirectSampling
    m # number of samples
end

function infer(M::DirectSampling, bn, query, evidence)
    table = FactorTable()
    for i in 1:(M.m)
        a = rand(bn)
        if all(a[k] == v for (k,v) in pairs(evidence))
            b = select(a, query)
            table[b] = get(table, b, 0) + 1
        end
    end
    vars = filter(v->v.name ∈ query, bn.vars)
    return normalize!(Factor(vars, table))
end
```

算法3.7。直接采样推理方法，其输入为贝叶斯网络`bn`、查询变量列表`query`以及证据`evidence`。该方法从贝叶斯网络中抽取`m`个样本，并保留与证据一致的样本。返回一个关于查询变量的因子。如果没有找到满足证据的样本，该方法可能会失败。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

3.7 Likelihood Weighted Sampling

直接采样的问题在于，我们可能浪费时间生成与观测不符的样本，尤其是当观测不太可能发生时。另一种方法称为*likelihood weighted sampling*，它涉及生成与观测一致的加权样本。

为了说明这一点，我们将再次尝试推断 $P(b^1 \mid d^1, c^1)$。我们有一组 n 个样本，其中第 i 个样本仍记为 $(b^{(i)}, s^{(i)}, e^{(i)}, d^{(i)}, c^{(i)})$。第 i 个样本的权重为 w_i。加权估计为

$$P(b^1 \mid d^1, c^1) \approx \frac{\sum_i w_i(b^{(i)} = 1 \land d^{(i)} = 1 \land c^{(i)} = 1)}{\sum_i w_i(d^{(i)} = 1 \land c^{(i)} = 1)} \tag{3.16}$$

$$= \frac{\sum_i w_i(b^{(i)} = 1)}{\sum_i w_i} \tag{3.17}$$

为了生成这些加权样本，我们首先进行拓扑排序，并按顺序从条件分布中抽样。似然加权中的唯一区别在于我们如何处理观测变量。我们不再从条件分布中抽样它们的值，而是将变量赋予其观测值，并相应地调整样本的权重。样本的权重就是观测节点处条件概率的乘积。似然加权抽样在算法3.8中实现。例3.6演示了使用似然加权抽样的推理。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct LikelihoodWeightedSampling
    m # number of samples
end

function infer(M::LikelihoodWeightedSampling, bn, query, evidence)
    table = FactorTable()
    ordering = topological_sort(bn.graph)
    for i in 1:(M.m)
        a, w = Assignment(), 1.0
        for j in ordering
            name, ϕ = bn.vars[j].name, bn.factors[j]
            if haskey(evidence, name)
                a[name] = evidence[name]
                w *= ϕ.table[select(a, variablenames(ϕ))]
            else
                a[name] = rand(condition(ϕ, a))[name]
            end
        end
        b = select(a, query)
        table[b] = get(table, b, 0) + w
    end
    vars = filter(v->v.name ∈ query, bn.vars)
    return normalize!(Factor(vars, table))
end
```

算法3.8。似然加权采样推理方法，该方法接受贝叶斯网络 `bn`、查询变量列表 `query` 和证据 `evidence`。该方法从贝叶斯网络中抽取 `m` 个样本，但在可能的情况下根据证据设置值，并在此过程中记录条件概率。这些概率用于对样本进行加权，以使最终的推理估计准确。返回一个关于查询变量的因子。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

这里的表格展示了来自图3.1网络中的五个似然加权样本。我们从$P(B)$、$P(S)$以及$P(E \mid B, S)$中采样，正如直接采样所做的那样。当涉及到D和C时，我们给D =赋值为1，C =赋值为1。如果样本中E =为1，则权重为$P(d^1 \mid e^1)P(c^1 \mid e^1)$；否则，权重为$P(d^1 \mid e^0)P(c^1 \mid e^0)$。如果我们假设

$$P(d^1 \mid e^1)P(c^1 \mid e^1) = 0.95$$
$$P(d^1 \mid e^0)P(c^1 \mid e^0) = 0.01$$

然后我们可以从表中的样本近似得到：

$$P(b^1 \mid d^1, c^1) = \frac{0.95}{0.95 + 0.95 + 0.01 + 0.01 + 0.95} \approx 0.331$$

B	S	E	D	C	Weight
1	0	1	1	1	$P(d^1 \mid e^1)P(c^1 \mid e^1)$
0	1	1	1	1	$P(d^1 \mid e^1)P(c^1 \mid e^1)$
0	0	0	1	1	$P(d^1 \mid e^0)P(c^1 \mid e^0)$
0	0	0	1	1	$P(d^1 \mid e^0)P(c^1 \mid e^0)$
0	0	1	1	1	$P(d^1 \mid e^1)P(c^1 \mid e^1)$

示例3.6。来自贝叶斯网络的似然加权样本。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

虽然似然加权使得所有样本都与观测结果一致，但它仍可能造成浪费。考虑图3.5所示的简单化学检测贝叶斯网络，并假设我们检测到了某种感兴趣的化学物质。我们想要推断 $P(c^1 \mid d^1)$。由于这个网络很小，我们可以很容易地使用贝叶斯规则精确计算这个概率：

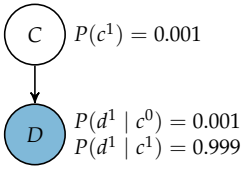


图3.5. 化学检测贝叶斯网络，其中C表示化学物质是否存在，D表示化学物质是否被检测到。

$$P(c^1 \mid d^1) = \frac{P(d^1 \mid c^1)P(c^1)}{P(d^1 \mid c^1)P(c^1) + P(d^1 \mid c^0)P(c^0)} \tag{3.18}$$

$$= \frac{0.999 \times 0.001}{0.999 \times 0.001 + 0.001 \times 0.999} \tag{3.19}$$

$$= 0.5 \tag{3.20}$$

如果我们使用似然加权，那么99.9%的样本将具有$C = 0$，权重为0.001。直到我们得到一个$C = 1$的样本，其关联权重为0.999，我们对$P(c^1 \mid d^1)$的估计将是0。

3.8 Gibbs Sampling

另一种推理方法是使用*Gibbs sampling*、[8]，这是一种*Markov chain Monte Carlo* 技术。吉布斯采样通过一种不涉及加权的方式抽取与证据一致的样本。从这些样本中，我们可以推断查询变量的分布。

8 以美国科学家乔赛亚·威拉德·吉布斯（1839–1903）命名，他与詹姆斯·克拉克·麦克斯韦和路德维希·玻尔兹曼共同创立了统计力学领域。

吉布斯采样涉及生成一系列样本，从一个初始样本$x_{1:n}^{(1)}$开始，该样本是在证据变量设为其观测值的情况下随机生成的。第k个样本$x_{1:n}^{(k)}$在概率上依赖于前一个样本$x_{1:n}^{(k-1)}$。我们如下所示就地修改$x_{1:n}^{(k-1)}$以获得$x_{1:n}^{(k)}$。使用未观测变量的任意顺序，这不必是拓扑排序，$x_i^{(k)}$从$P(X_i \mid x_{-i}^{(k)})$所表示的分布中采样。这里，$x_{-i}^{(k)}$表示样本k中除X_i之外所有其他变量的值。从$P(X_i \mid x_{-i}^{(k)})$中采样可以高效完成，因为我们只需要考虑变量X_i (的马尔可夫毯，见第2.6节)。

与目前讨论的其他采样方法不同，此方法产生的样本并非独立。然而，可以证明，在极限情况下，样本是从未观察变量的联合分布中精确抽取的。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

给定观测值下的变量。算法3.9展示了如何计算$P(X_i \mid x_{-i})$的因子。吉布斯采样在算法3.10中实现。

```
function blanket(bn, a, i)
    name = bn.vars[i].name
    val = a[name]
    a = delete!(copy(a), name)
    Φ = filter(ϕ -> in_scope(name, ϕ), bn.factors)
    ϕ = prod(condition(ϕ, a) for ϕ in Φ)
    return normalize!(ϕ)
end
```

算法3.9. 一种在给定当前赋值a的情况下，为贝叶斯网络bn获取$P(X_i \mid x_{-i})$的方法。

吉布斯采样可以应用于我们正在运行的示例。我们可以使用我们的 {v*} 样本来估计

$$P(b^1 \mid d^1, c^1) \approx \frac{1}{m} \sum_i (b^{(i)} = 1) \tag{3.21}$$

图3.6比较了化学检测网络中$P(c^1 \mid d^1)$估计的收敛性，分别使用了直接采样、似然加权采样和Gibbs采样。直接采样收敛所需时间最长。直接采样曲线有较长时间段估计值保持不变，因为样本与观测不一致。在这个例子中，似然加权采样收敛更快。当生成一个C =为1的样本时，会出现尖峰，随后逐渐减小。在这个例子中，Gibbs采样迅速收敛到真实值0.5。

如前所述，吉布斯采样与其他马尔可夫链蒙特卡洛方法一样，在极限情况下从期望分布中产生样本。在实践中，我们必须运行吉布斯采样一段时间，称为*burn-in period*，才能收敛到稳态分布。在预热期间产生的样本通常会被丢弃。如果要从单个吉布斯采样序列中使用许多样本，由于样本之间可能存在相关性，通常会通过仅保留每第h个样本来*thin*样本。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
function update_gibbs_sample!(a, bn, evidence, ordering)
    for i in ordering
        name = bn.vars[i].name
        if !haskey(evidence, name)
            b = blanket(bn, a, i)
            a[name] = rand(b)[name]
        end
    end
end

function gibbs_sample!(a, bn, evidence, ordering, m)
    for j in 1:m
        update_gibbs_sample!(a, bn, evidence, ordering)
    end
end

struct GibbsSampling
    m_samples # number of samples to use
    m_burnin  # number of samples to discard during burn-in
    m_skip    # number of samples to skip for thinning
    ordering  # array of variable indices
end

function infer(M::GibbsSampling, bn, query, evidence)
    table = FactorTable()
    a = merge(rand(bn), evidence)
    gibbs_sample!(a, bn, evidence, M.ordering, M.m_burnin)
    for i in 1:(M.m_samples)
        gibbs_sample!(a, bn, evidence, M.ordering, M.m_skip)
        b = select(a, query)
        table[b] = get(table, b, 0) + 1
    end
    vars = filter(v->v.name ∈ query, bn.vars)
    return normalize!(Factor(vars, table))
end
```

算法3.10。针对具有证据`evidence`和排序`ordering`的贝叶斯网络`bn`，实施Gibbs采样。该方法迭代更新赋值`a`，共进行`m`次迭代。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

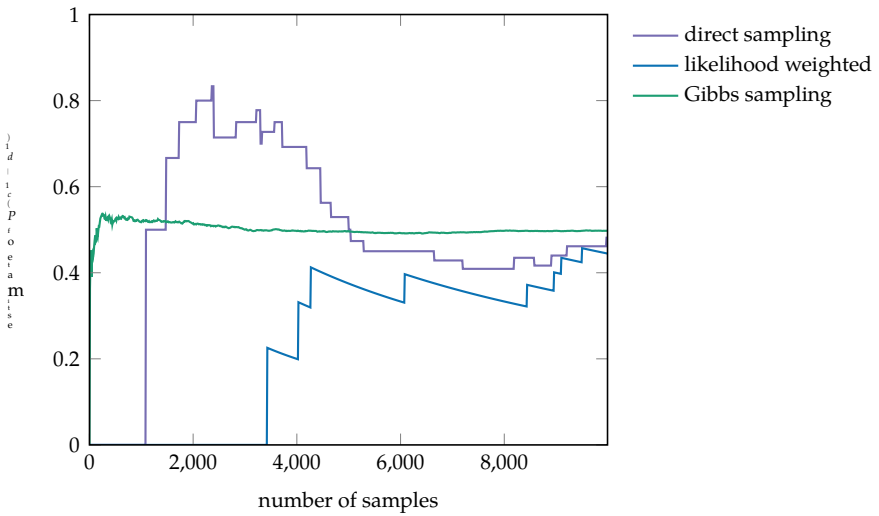


图3.6. 在化学检测网络上基于采样的推断方法比较。由于事件罕见，似然加权和直接采样均收敛不佳，而吉布斯采样即使没有预热期或稀释也能高效收敛至真实值。

3.9 *Inference in Gaussian Models*

如果联合分布是高斯分布，我们可以通过解析方法进行精确推断。两个联合高斯随机变量 a 和 b 可以表示为

$$\begin{bmatrix}\mathbf{a}\\ \mathbf{b}\end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix}\boldsymbol{\mu}_\mathbf{a}\\ \boldsymbol{\mu}_\mathbf{b}\end{bmatrix}, \begin{bmatrix}\mathbf{A} & \mathbf{C}\\ \mathbf{C}^\top & \mathbf{B}\end{bmatrix}\right) \tag{3.22}$$

多元高斯分布的边缘分布也是高斯分布：

$$\mathbf{a} \sim \mathcal{N}(\boldsymbol{\mu}_\mathbf{a}, \mathbf{A}) \qquad \mathbf{b} \sim \mathcal{N}(\boldsymbol{\mu}_\mathbf{b}, \mathbf{B}) \tag{3.23}$$

多元高斯分布的条件分布也是高斯分布，且有一个便捷的闭式解：

$$p(\mathbf{a} \mid \mathbf{b}) = \mathcal{N}\left(\mathbf{a} \mid \boldsymbol{\mu}_{\mathbf{a}|\mathbf{b}}, \boldsymbol{\Sigma}_{\mathbf{a}|\mathbf{b}}\right) \tag{3.24}$$

$$\boldsymbol{\mu}_{\mathbf{a}|\mathbf{b}} = \boldsymbol{\mu}_\mathbf{a} + \mathbf{C}\mathbf{B}^{-1}(\mathbf{b} - \boldsymbol{\mu}_\mathbf{b}) \tag{3.25}$$

$$\boldsymbol{\Sigma}_{\mathbf{a}|\mathbf{b}} = \mathbf{A} - \mathbf{C}\mathbf{B}^{-1}\mathbf{C}^\top \tag{3.26}$$

算法3.11展示了如何利用这些方程，在给定证据的情况下推断出一组查询变量的分布。例3.7说明了如何从多变量高斯分布中提取边缘分布和条件分布。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
function infer(D::MvNormal, query, evidencevars, evidence)
    μ, Σ = D.μ, D.Σ.mat
    b, μa, μb = evidence, μ[query], μ[evidencevars]
    A = Σ[query,query]
    B = Σ[evidencevars,evidencevars]
    C = Σ[query,evidencevars]
    μ = μa + C * (B\(b - μb))
    Σ = A - C * (B \ C')
    return MvNormal(μ, Σ)
end
```

算法3.11。多元高斯分布`D`中的推理。一个整数向量在`query`参数中指定查询变量，一个整数向量在`evidencevars`参数中指定证据变量。证据变量的值包含在向量`evidence`中。`Distributions.jl`包定义了`MvNormal`分布。

考虑

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix} \right)$$

x_1的边缘分布为$\mathcal{N}(0、3)$，x_2的边缘分布为$\mathcal{N}(1、2)$。

x_1 在给定 $x_2 = 2$ 的条件分布为

$$\begin{aligned} \boldsymbol{\mu}_{x_1 \mid x_2=2} &= 0 + 1 \cdot 2^{-1} \cdot (2-1) = 0.5 \\ \boldsymbol{\Sigma}_{x_1 \mid x_2=2} &= 3 - 1 \cdot 2^{-1} \cdot 1 = 2.5 \\ x_1 \mid (x_2 = 2) &\sim \mathcal{N}(0.5, 2.5) \end{aligned}$$

我们可以使用算法3.11执行此推断计算，通过构建联合分布`D = MvNormal([0.0,1.0],[3.0 1.0; 1.0 2.0])`，然后调用`infer(D, [1], [2], [2.0])`。

例3.7. 多元高斯分布的边缘分布与条件分布。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

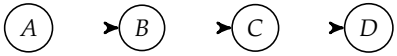
3.10 Summary

- 推理涉及在给定某些证据的情况下确定查询变量的概率。
- 精确推断可以通过计算变量的联合分布、设置证据并对任何隐藏变量进行边缘化来完成。
- 在朴素贝叶斯模型中，推理可以高效地进行，其中单个父变量影响许多条件独立的子变量。
- 变量消除算法可以通过按顺序边缘化变量来提高精确推理的效率。
- 信念传播代表了另一种推理方法，其中信息在因子之间迭代传递以得出结果。
- 贝叶斯网络中的推断问题通过归约到3SAT问题可以被证明是NP难的，这促使了近似推断方法的发展。
- 近似推断可以通过直接从贝叶斯网络表示的联合分布中采样来完成，但这可能涉及丢弃许多与证据不一致的样本。
- 似然加权采样可以通过仅生成与证据一致的样本并相应地加权每个样本，来减少近似推断所需的计算量。
- Gibbs 采样生成一系列与证据一致的无权重样本，可以极大地加速近似推断。
- 当联合分布为高斯分布时，可以通过矩阵运算高效地进行精确推断。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

3.11 Exercises

练习3.1。给定以下贝叶斯网络及其相关的条件概率分布，写出执行查询$P(a^1 \mid d^1)$精确推断所需的方程。



Solution: 我们首先使用条件概率的定义来展开推断表达式。

$$P(a^1 \mid d^1) = \frac{P(a^1, d^1)}{P(d^1)}$$

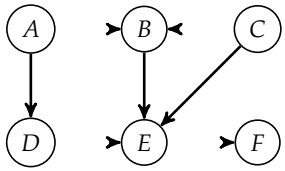
我们可以将分子重写为对隐变量的边缘化，并将分母重写为对隐变量和查询变量的边缘化：

$$P(a^1 \mid d^1) = \frac{\sum_b \sum_c P(a^1, b, c, d^1)}{\sum_a \sum_b \sum_c P(a, b, c, d^1)}$$

分子和分母中联合概率的定义都可以使用贝叶斯网络中的链式法则重写，并且通过从求和中移除常数，可以简化所得方程：

$$\begin{aligned} P(a^1 \mid d^1) &= \frac{\sum_b \sum_c P(a^1) P(b \mid a^1) P(c \mid b) P(d^1 \mid c)}{\sum_a \sum_b \sum_c P(a) P(b \mid a) P(c \mid b) P(d^1 \mid c)} \\ &= \frac{P(a^1) \sum_b \sum_c P(b \mid a^1) P(c \mid b) P(d^1 \mid c)}{\sum_a \sum_b \sum_c P(a) P(b \mid a) P(c \mid b) P(d^1 \mid c)} \\ &= \frac{P(a^1) \sum_b P(b \mid a^1) \sum_c P(c \mid b) P(d^1 \mid c)}{\sum_a P(a) \sum_b P(b \mid a) \sum_c P(c \mid b) P(d^1 \mid c)} \end{aligned}$$

练习3.2。给定以下贝叶斯网络及其相关的条件概率分布，写出执行查询$P(c^1, d^1 \mid a^0, f^1)$的精确推理所需的方程。



Solution: 我们首先使用条件概率的定义来展开推断表达式：

$$P(c^1, d^1 \mid a^0, f^1) = \frac{P(a^0, c^1, d^1, f^1)}{P(a^0, f^1)}$$

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

我们可以将分子重写为对隐变量的边缘化，并且我们可以将分母重写为对隐变量和查询变量的边缘化：

$$P(c^1, d^1 \mid a^0, f^1) = \frac{\sum_b \sum_e P(a^0, b, c^1, d^1, e, f^1)}{\sum_b \sum_c \sum_d \sum_e P(a^0, b, c, d, e, f^1)}$$

联合概率在分子和分母中的定义均可使用贝叶斯网络的链式法则重写，所得方程可通过从求和中移除常数来简化。请注意，最终方程中求和符号可能有多种排列顺序：

$$\begin{aligned} P(c^1, d^1 \mid a^0, f^1) &= \frac{\sum_b \sum_e P(a^0) P(b \mid a^0, c^1) P(c^1) P(d^1 \mid a^0) P(e \mid b, c^1, d^1) P(f^1 \mid e)}{\sum_b \sum_c \sum_d \sum_e P(a^0) P(b \mid a^0, c) P(c) P(d \mid a^0) P(e \mid b, c, d) P(f^1 \mid e)} \\ &= \frac{P(a^0) P(c^1) P(d^1 \mid a^0) \sum_b \sum_e P(b \mid a^0, c^1) P(e \mid b, c^1, d^1) P(f^1 \mid e)}{P(a^0) \sum_b \sum_c \sum_d \sum_e P(b \mid a^0, c) P(c) P(d \mid a^0) P(e \mid b, c, d) P(f^1 \mid e)} \\ &= \frac{P(c^1) P(d^1 \mid a^0) \sum_b P(b \mid a^0, c^1) \sum_e P(e \mid b, c^1, d^1) P(f^1 \mid e)}{\sum_c P(c) \sum_b P(b \mid a^0, c) \sum_d P(d \mid a^0) \sum_e P(e \mid b, c, d) P(f^1 \mid e)} \end{aligned}$$

练习3.3。假设我们正在为在城市中行驶的自动驾驶汽车开发一个物体检测系统。我们车辆的感知系统报告物体的尺寸S (要么小、要么中、要么大)，以及速度V (要么慢、要么适中、要么快)。我们想要设计一个模型，根据对物体尺寸和速度的观察，确定物体的类别C——要么是车辆、行人、要么是球。假设使用具有以下类别先验和类别条件分布的朴素贝叶斯模型，给定观察S =中等和V =慢，检测到的类别是什么？

C	$P(C)$
vehicle	0.80
pedestrian	0.19
ball	0.01

C	S	$P(S \mid C)$
vehicle	small	0.001
vehicle	medium	0.009
vehicle	large	0.990
pedestrian	small	0.200
pedestrian	medium	0.750
pedestrian	large	0.050
ball	small	0.800
ball	medium	0.199
ball	large	0.001

C	V	$P(V \mid C)$
vehicle	slow	0.2
vehicle	moderate	0.2
vehicle	fast	0.6
pedestrian	slow	0.5
pedestrian	moderate	0.4
pedestrian	fast	0.1
ball	slow	0.4
ball	moderate	0.4
ball	fast	0.2

Solution: 为了计算后验分布 {v*}，我们使用式（3.4）中朴素贝叶斯模型的联合分布定义：

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

$$P(c \mid o_{1:n}) \propto P(c) \prod_{i=1}^{n} P(o_i \mid c)$$

$$P(\text{vehicle} \mid \text{medium}, \text{slow}) \propto P(\text{vehicle}) P(S = \text{medium} \mid \text{vehicle}) P(V = \text{slow} \mid \text{vehicle})$$

$$P(\text{vehicle} \mid \text{medium}, \text{slow}) \propto (0.80)(0.009)(0.2) = 0.00144$$

$$P(\text{pedestrian} \mid \text{medium}, \text{slow}) \propto P(\text{pedestrian}) P(S = \text{medium} \mid \text{pedestrian}) P(V = \text{slow} \mid \text{pedestrian})$$

$$P(\text{pedestrian} \mid \text{medium}, \text{slow}) \propto (0.19)(0.75)(0.5) = 0.07125$$

$$P(\text{ball} \mid \text{medium}, \text{slow}) \propto P(\text{ball}) P(S = \text{medium} \mid \text{ball}) P(V = \text{slow} \mid \text{ball})$$

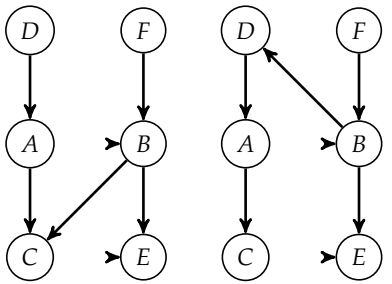
$$P(\text{ball} \mid \text{medium}, \text{slow}) \propto (0.01)(0.199)(0.4) = 0.000796$$

由于P(行人|中速、慢速)的概率最大，因此该对象被归类为行人。

练习3.4。给定方程（3.14）中的3SAT公式和图3.4中的贝叶斯网络结构，$P(c_3^1 \mid x_2^1、x_3^0、x_4^1)$以及$P(y^1 \mid d_2^1、c_3^0)$的值分别是多少?

Solution: 我们有 $P(c_3^1 \mid x_2^1、x_3^0、x_4^1) =$ 为1，因为 $x_2^1、x_3^0、x_4^1$ 使第三个子句为真，而 $P(y^1 \mid d_2^1、c_3^0) =$ 为0，因为 $Y =$ 为1要求 D_2 和 C_3 都为真。

练习3.5。为以下每个有向图给出一个拓扑排序：

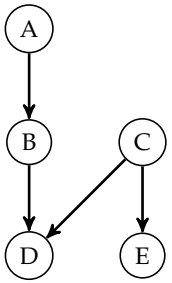


(1) (2)

Solution: 第一个有向图（贝叶斯网络）存在三种有效的拓扑排序：(F, D, A, B, C, E)、(D, A, F, B, C, E) 以及 (D, F, A, B, C, E)。由于第二个有向图存在环，因此不存在有效的拓扑排序。

练习 3.6. 假设我们有如下的贝叶斯网络，并且我们感兴趣的是使用似然加权采样来生成推理查询 $P(e^1 \mid b^0, d^1)$ 的近似值。给定以下样本，写出每个样本权重的表达式。此外，写出用样本权重 w_i 估计 $P(e^1 \mid b^0, d^1)$ 的方程。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com



A	B	C	D	E
0	0	0	1	0
1	0	0	1	0
0	0	0	1	1
1	0	1	1	1
0	0	1	1	0
1	0	1	1	1

Solution: 对于似然加权采样，样本权重是证据变量在其父节点取值条件下的分布乘积。因此，我们的权重的一般形式是 $P(b^0 \mid a)P(d^1 \mid b^0、c)$。然后，我们将联合分布中每个样本的每个值进行匹配：

A	B	C	D	E	Weight
0	0	0	1	0	$P(b^0 \mid a^0)P(d^1 \mid b^0, c^0)$
1	0	0	1	0	$P(b^0 \mid a^1)P(d^1 \mid b^0, c^0)$
0	0	0	1	1	$P(b^0 \mid a^0)P(d^1 \mid b^0, c^0)$
1	0	1	1	1	$P(b^0 \mid a^1)P(d^1 \mid b^0, c^1)$
0	0	1	1	0	$P(b^0 \mid a^0)P(d^1 \mid b^0, c^1)$
1	0	1	1	1	$P(b^0 \mid a^1)P(d^1 \mid b^0, c^1)$

为了估计$P(e^1 \mid b^0、d^1)$，我们只需将与该查询变量一致的样本的权重求和，再除以所有样本权重之和即可：

$$P(e^1 \mid b^0, d^1) = \frac{\sum_i w_i(e^{(i)} = 1)}{\sum_i w_i} = \frac{w_3 + w_4 + w_6}{w_1 + w_2 + w_3 + w_4 + w_5 + w_6}$$

练习3.7。每年，我们都会收到学生在标准化数学M、阅读R和写作W考试中的成绩。利用前几年的数据，我们构建了以下分布：

$$\begin{bmatrix} M \\ R \\ W \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} 81 \\ 82 \\ 80 \end{bmatrix}, \begin{bmatrix} 25 & -9 & -16 \\ -9 & 36 & 16 \\ -16 & 16 & 36 \end{bmatrix} \right)$$

计算学生的数学和阅读测试成绩的条件分布参数，给定写作成绩为90。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: 如果我们让 a 表示数学和阅读成绩的向量，b 表示写作成绩，则联合分布和条件分布如下：

$$\begin{bmatrix}\mathbf{a}\\ \mathbf{b}\end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix}\boldsymbol{\mu}_a\\ \boldsymbol{\mu}_b\end{bmatrix}, \begin{bmatrix}\mathbf{A} & \mathbf{C}\\ \mathbf{C}^\top & \mathbf{B}\end{bmatrix}\right)$$
$$p(\mathbf{a} \mid \mathbf{b}) = \mathcal{N}\left(\mathbf{a} \mid \boldsymbol{\mu}_{a|b}, \boldsymbol{\Sigma}_{a|b}\right)$$
$$\boldsymbol{\mu}_{a|b} = \boldsymbol{\mu}_a + \mathbf{C}\mathbf{B}^{-1}(\mathbf{b} - \boldsymbol{\mu}_b)$$
$$\boldsymbol{\Sigma}_{a|b} = \mathbf{A} - \mathbf{C}\mathbf{B}^{-1}\mathbf{C}^\top$$

在示例中，我们有以下定义：

$$\boldsymbol{\mu}_a = \begin{bmatrix}81\\ 82\end{bmatrix} \qquad \boldsymbol{\mu}_b = \begin{bmatrix}80\end{bmatrix} \qquad \mathbf{A} = \begin{bmatrix}25 & -9\\ -9 & 36\end{bmatrix} \qquad \mathbf{B} = \begin{bmatrix}36\end{bmatrix} \qquad \mathbf{C} = \begin{bmatrix}-16\\ 16\end{bmatrix}$$

因此，条件分布的参数，给定 b $= W =$ 90 是

$$\boldsymbol{\mu}_{M,R|W=90} = \begin{bmatrix}81\\ 82\end{bmatrix} + \begin{bmatrix}-16\\ 16\end{bmatrix}\frac{1}{36}(90 - 80) \approx \begin{bmatrix}76.5\\ 86.4\end{bmatrix}$$
$$\boldsymbol{\Sigma}_{M,R|W=90} = \begin{bmatrix}25 & -9\\ -9 & 36\end{bmatrix} - \begin{bmatrix}-16\\ 16\end{bmatrix}\frac{1}{36}\begin{bmatrix}-16 & 16\end{bmatrix} \approx \begin{bmatrix}25 & -9\\ -9 & 36\end{bmatrix} - \begin{bmatrix}7.1 & -7.1\\ -7.1 & 7.1\end{bmatrix} = \begin{bmatrix}17.9 & -1.9\\ -1.9 & 28.9\end{bmatrix}$$

条件分布下，学生写作测试得90分，
我们预计该学生在数学考试中会得到76.5分，标准差为$\sqrt{17.9}$，
以及阅读测试中的86.4分，标准差为$\sqrt{28.9}$。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

4 Parameter Learning

到目前为止，我们假设概率模型的参数和结构是已知的。本章讨论从数据中*learning*或*fitting*模型参数的问题。[1]我们首先介绍一种方法，其中我们识别使观测数据似然最大化的模型参数。在讨论这种方法的局限性之后，我们引入另一种贝叶斯方法，其中我们从未知参数的概率分布开始，然后根据观测数据使用概率定律更新该分布。接着，我们讨论避免固定参数数量的概率模型。

1 本章重点关注从数据中学习模型参数，这是 *machine learning* 领域的一个重要组成部分。该领域的广泛介绍由 K. P. Murphy，*Probabilistic Machine Learning: An Introduction*。MIT Press，2022。提供。

4.1 Maximum Likelihood Parameter Learning

在 *maximum likelihood parameter learning* 中，我们尝试找到使观测数据似然最大化的分布参数。如果 θ 表示一个分布的参数，那么 *maximum likelihood estimate* 是

$$\hat{\theta} = \arg\max_{\theta} P(D \mid \theta) \tag{4.1}$$

其中 $P(D \mid \theta)$ 是当模型参数设置为 θ 时，我们的概率模型赋予数据 D 出现的似然概率。[2] 我们通常使用“帽子”重音（“$\hat{}$”）表示参数的估计值。

最大似然参数学习面临两个挑战。一是选择一个合适的概率模型来定义 $P(D \mid \theta)$。我们通常假设数据D中的样本是*independently and identically distributed*，这意味着我们的样本$D = o_{1:m}$是从一个

2 在这里，我们将 $P(D \mid \theta)$ 写作仿佛是离散分布相关的概率质量。然而，我们的概率模型可能是连续的，在这种情况下，我们处理的是密度。

分布 $o_i \sim P(\cdot \mid \theta)$ 与

$$P(D \mid \theta) = \prod_i P(o_i \mid \theta) \tag{4.2}$$

概率模型可以包括，例如，前面章节提到的分类分布或高斯分布。

另一个挑战是执行方程(4.1)中的最大化。对于许多常见的概率模型，我们可以解析地进行这种优化。其他情况可能比较困难。一种常见的方法是最大化*log-likelihood*，通常表示为$\ell(\theta)$。由于对数变换是单调递增的，最大化对数似然等价于最大化似然：[3]

$$\hat{\theta} = \arg\max_\theta \sum_i \log P(o_i \mid \theta) \tag{4.3}$$

计算对数似然之和通常比计算许多小概率质量或密度的乘积在数值上更为稳定。本节余下部分将展示如何针对不同类型的分布来优化方程(4.1)。

3 尽管在这个方程中，最大化自然对数（底数为e）或常用对数（底数为10）都无关紧要，但在整本书中，我们将使用$\log(x)$来表示以e为底的x的对数。

4.1.1 Maximum Likelihood Estimates for Categorical Distributions

假设随机变量C表示一次飞行是否会导致空中相撞，而我们感兴趣的是估计分布$P(C)$。由于C要么是0要么是1，因此只需估计参数$\theta = P(c^1)$即可。我们要做的是从数据D推断θ。我们有一个跨越十年的历史数据库，包含m次飞行和n次空中相撞。当然，我们的直觉告诉我们，给定数据D，对θ的一个良好估计是n/m。在飞行间结果独立的假设下，在D中具有n次空中相撞的m个结果的序列的概率是：

$$P(D \mid \theta) = \theta^n (1-\theta)^{m-n} \tag{4.4}$$

最大似然估计$\hat{\theta}$是使方程 (4.4) 最大化的 θ 的值，这等价于最大化似然函数的对数：

$$\ell(\theta) = \log\left(\theta^n (1-\theta)^{m-n}\right) \tag{4.5}$$

$$= n \log \theta + (m-n) \log(1-\theta) \tag{4.6}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们可以使用求函数最大值的标准方法，即令 ℓ 的一阶导数为0，然后解出 θ。该导数为

$$\frac{\partial}{\partial\theta}\ell(\theta) = \frac{n}{\theta} - \frac{m-n}{1-\theta} \tag{4.7}$$

我们可以通过将导数设为 0 来求解 $\hat{\theta}$：

$$\frac{n}{\hat{\theta}} - \frac{m-n}{1-\hat{\theta}} = 0 \tag{4.8}$$

经过几个代数步骤，我们发现，确实，$\hat{\theta} = n/m$。

计算一个可以取k个值的变量X的最大似然估计也是直接了当的。如果$n_{1:k}$是k个不同值的观测计数，那么$P(x^i \mid n_{1:k})$的最大似然估计由下式给出

$$\hat{\theta}_i = \frac{n_i}{\sum_{j=1}^{k} n_j} \tag{4.9}$$

4.1.2 *Maximum Likelihood Estimates for Gaussian Distributions*

在高斯分布中，均值 μ 和方差 σ^2 在 m 个样本下的对数似然为

$$\ell(\mu, \sigma^2) \propto -m\log\sigma - \frac{\sum_i (o_i - \mu)^2}{2\sigma^2} \tag{4.10}$$

同样，我们可以将关于参数的导数设为 0，并求解最大似然估计：

$$\frac{\partial}{\partial\mu}\ell(\mu, \sigma^2) = \frac{\sum_i (o_i - \hat{\mu})}{\hat{\sigma}^2} = 0 \tag{4.11}$$

$$\frac{\partial}{\partial\sigma}\ell(\mu, \sigma^2) = -\frac{m}{\hat{\sigma}} + \frac{\sum_i (o_i - \hat{\mu})^2}{\hat{\sigma}^3} = 0 \tag{4.12}$$

经过一些代数运算，我们得到

$$\hat{\mu} = \frac{\sum_i o_i}{m} \qquad \hat{\sigma}^2 = \frac{\sum_i (o_i - \hat{\mu})^2}{m} \tag{4.13}$$

图4.1提供了对数据进行高斯拟合的示例。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

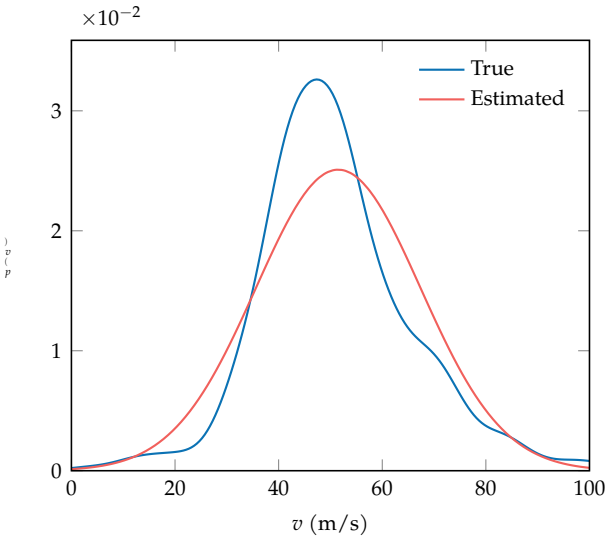


图4.1。假设我们有来自m条飞机轨迹的空速测量值$o_{1:m}$，并且想要拟合一个高斯模型。该图展示了一个高斯分布，其最大似然估计为$\hat{\mu} = 5$ 1.5 m/s和$\hat{\sigma} = 15.9$ m/s。为了便于比较，同时展示了“真实”分布。在这种情况下，高斯分布是对真实分布相当合理的近似。

4.1.3 *Maximum Likelihood Estimates for Bayesian Networks*

我们可以对贝叶斯网络应用最大似然参数学习。在这里，我们将假设我们的网络由一组n离散变量组成，我们将其记为$X_{1:n}$。我们的数据$D = \{o_1, \ldots, o_m\}$由这些变量的观测样本组成。在结构为G的网络中，r_i是X_i的实例化数量，而q_i是X_i的父节点的实例化数量。如果X_i没有父节点，那么$q_i = 1$。X_i的父节点的第j个实例化记为$\boldsymbol{\pi}_{ij}$。

X_i 的因子表因此有 $r_i q_i$ 个条目，总共在我们的贝叶斯网络中产生 $\sum_{i=1}^{n} r_i q_i$ 个参数。每个参数写成 θ_{ijk} 并确定

$$P(X_i = k \mid \boldsymbol{\pi}_{ij}) = \theta_{ijk} \tag{4.14}$$

虽然有 $\sum_{i=1}^{n} r_i q_i$ 个参数，但只有 $\sum_{i=1}^{n}(r_i - \text{个 } 1)q_i$ 是独立的。我们用 $\boldsymbol{\theta}$ 表示所有参数的集合。

我们使用 m_{ijk} 来表示在数据集中给定父代实例化 j 时 $X_i = k$ 出现的次数。算法 4.1 提供了一个从数据集中提取这些计数或统计量的函数实现。似然函数由下式给出：

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

m_{ijk} 的条款：

$$P(D \mid \boldsymbol{\theta}, G) = \prod_{i=1}^{n} \prod_{j=1}^{q_i} \prod_{k=1}^{r_i} \theta_{ijk}^{m_{ijk}} \tag{4.15}$$

与公式（4.9）中单变量分布的最大似然估计相似，我们离散贝叶斯网络模型中的最大似然估计为

$$\hat{\theta}_{ijk} = \frac{m_{ijk}}{\sum_{k'} m_{ijk'}} \tag{4.16}$$

示例4.1说明了这个过程。

```
function sub2ind(siz, x)
    k = vcat(1, cumprod(siz[1:end-1]))
    return dot(k, x .- 1) + 1
end

function statistics(vars, G, D::Matrix{Int})
    n = size(D, 1)
    r = [vars[i].r for i in 1:n]
    q = [prod([r[j] for j in inneighbors(G,i)]) for i in 1:n]
    M = [zeros(q[i], r[i]) for i in 1:n]
    for o in eachcol(D)
        for i in 1:n
            k = o[i]
            parents = inneighbors(G,i)
            j = 1
            if !isempty(parents)
                j = sub2ind(r[parents], o[parents])
            end
            M[i][j,k] += 1.0
        end
    end
    return M
end
```

算法4.1. 一个从离散数据集`D`中提取统计量或计数的函数，假设有一个包含变量`vars`和结构`G`的贝叶斯网络。数据集是一个$n \times m$矩阵，其中n是变量的数量，m是数据点的数量。该函数返回一个长度为n的数组`M`。第i个分量由一个计数矩阵$q_i \times r_i$组成。`sub2ind(siz, x)`函数返回一个线性索引，该索引指向一个维度由`siz`给定坐标`x`指定的数组。它用于识别哪个父实例化与特定的数据点和变量相关。

4.2 *Bayesian Parameter Learning*

Bayesian parameter learning 解决了最大似然估计的一些缺点，特别是在数据量有限的情况下。例如，假设我们的航空安全数据库仅限于过去一周的事件，并且我们没有发现任何记录在案的空中相撞事故。如果θ是飞行导致空中相撞的概率，那么最大似然估计将会

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

假设我们有一个小型网络 $A \rightarrow B \leftarrow C$，并且我们想要从数据矩阵 `D` 中提取统计信息。我们可以使用以下代码：

```
G = SimpleDiGraph(3)
add_edge!(G, 1, 2)
add_edge!(G, 3, 2)
vars = [Variable(:A,2), Variable(:B,2), Variable(:C,2)]
D = [1 2 2 1; 1 2 2 1; 2 2 2 2]
M = statistics(vars, G, D)
```

输出是一个数组 `M`，由这三个计数矩阵组成，每个矩阵的大小为 $q_i \times r_i$：

$$\begin{bmatrix} 2 & 2 \end{bmatrix} \quad \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 2 & 0 \\ 0 & 2 \end{bmatrix} \quad \begin{bmatrix} 0 & 4 \end{bmatrix}$$

我们可以通过归一化`M`中矩阵的行来计算最大似然估计：

```
θ = [mapslices(x->normalize(x,1), Mi, dims=2) for Mi in M]
```

从而产生

$$\begin{bmatrix} 0.5 & 0.5 \end{bmatrix} \quad \begin{bmatrix} \text{NAN} & \text{NAN} \\ \text{NAN} & \text{NAN} \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \quad \begin{bmatrix} 0 & 1 \end{bmatrix}$$

正如我们所见，第二个变量B的第一和第二父实例化导致nan（“非数字”）估计。由于数据中没有这两个父实例化的观测值，方程(4.16)的分母等于零，使得参数估计未定义。其他大多数参数不是nan。例如，参数θ_{112} =的0.5意味着$P(a^2)$的最大似然估计为0.5。

示例 4.1。使用 `statistics` 函数从数据集中提取统计数据。贝叶斯参数学习可用于避免 nan 值，但我们必须指定一个先验。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

是$\hat{\theta} = 0$。认为空中相撞的概率为零并不是一个合理的结论，除非我们的先验假设是，例如，所有航班都是完全安全的。

参数学习的贝叶斯方法涉及估计 $p(\theta \mid D)$，即在给定数据 D 的条件下 θ 的后验分布。与最大似然估计中得到点估计 $\hat{\theta}$ 不同，我们得到的是一个分布。这个分布可以帮助我们量化对 θ 真实值的不确定性。我们可以通过计算期望将这个分布转换为点估计：

$$\hat{\theta} = \mathbb{E}_{\theta \sim p(\cdot \mid D)}[\theta] = \int \theta p(\theta \mid D) \, \mathrm{d}\theta \tag{4.17}$$

然而，在某些情况下，如figure 4.2所示，期望可能不是一个可接受的估计值。另一种方法是使用*maximum a posteriori*估计：

$$\hat{\theta} = \arg\max_{\theta} p(\theta \mid D) \tag{4.18}$$

该估计对应于被赋予最大密度的值θ。这通常被称为分布的*mode*。如图4.2所示，众数可能不是唯一的。

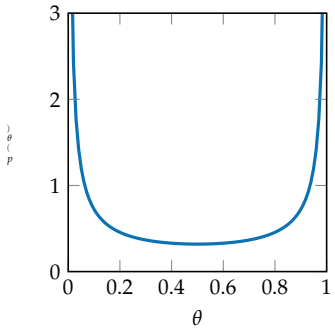


图4.2. 一个分布示例，其中θ的期望值不是一个好的估计。期望值0.5的密度低于在极端值0或1处出现的密度。这个分布恰好是贝塔分布，一种我们即将讨论的分布类型，参数为(0.2, 0.2)。

贝叶斯参数学习可以被视为在图4.3所示结构的贝叶斯网络中进行推断，该结构假设观测变量之间条件独立。与任何贝叶斯网络一样，我们必须指定 $p(\theta)$ 和 $P(O_i \mid \theta)$。我们经常使用均匀先验 $p(\theta)$。本节剩余部分讨论如何将贝叶斯参数学习应用于不同的 $P(O_i \mid \theta)$ 模型。

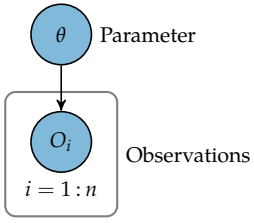


图4.3. 表示参数学习的贝叶斯网络。

4.2.1 *Bayesian Learning for Binary Distributions*

假设我们想要学习一个二值分布的参数。在这里，我们将使用$P(o^1 \mid \theta) = \theta$。为了在贝叶斯网络图4.3中推断关于$\theta$的分布，我们可以采用执行推断的标准方法。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

在第3章中讨论过。这里，我们将假设一个均匀先验：

$$
\begin{aligned}
p(\theta \mid o_{1:m}) &\propto p(\theta, o_{1:m}) && (4.19)\\
&= p(\theta) \prod_{i=1}^{m} P(o_i \mid \theta) && (4.20)\\
&= \prod_{i=1}^{m} P(o_i \mid \theta) && (4.21)\\
&= \prod_{i=1}^{m} \theta^{o_i} (1-\theta)^{1-o_i} && (4.22)\\
&= \theta^n (1-\theta)^{m-n} && (4.23)
\end{aligned}
$$

后验概率与 $\theta^n(1-\theta)^{m-n}$ 成正比，其中 n 是 $O_i = 1$ 的次数。为了找到归一化常数，我们积分

$$
\int_0^1 \theta^n (1-\theta)^{m-n} \, \mathrm{d}\theta = \frac{\Gamma(n+1)\Gamma(m-n+1)}{\Gamma(m+2)} \tag{4.24}
$$

其中 Γ 是 *gamma function*。伽玛函数是阶乘在实数域上的推广。若 m 为整数，则 $\Gamma(m) = (m-1)!$。考虑归一化，我们有

$$
\begin{aligned}
p(\theta \mid o_{1:m}) &= \frac{\Gamma(m+2)}{\Gamma(n+1)\Gamma(m-n+1)} \theta^n (1-\theta)^{m-n} && (4.25)\\
&= \text{Beta}(\theta \mid n+1, m-n+1) && (4.26)
\end{aligned}
$$

beta distribution $\text{Beta}(\alpha, \beta)$ 由参数 α 和 β 定义，该分布的曲线如图4.4所示。Beta(1, 1) 分布对应于从0到1的均匀分布。

分布 $\text{Beta}(\alpha, \beta)$ 的均值为

$$
\frac{\alpha}{\alpha + \beta} \tag{4.27}
$$

当 α 和 β 均大于1时，模式为

$$
\frac{\alpha - 1}{\alpha + \beta - 2} \tag{4.28}
$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

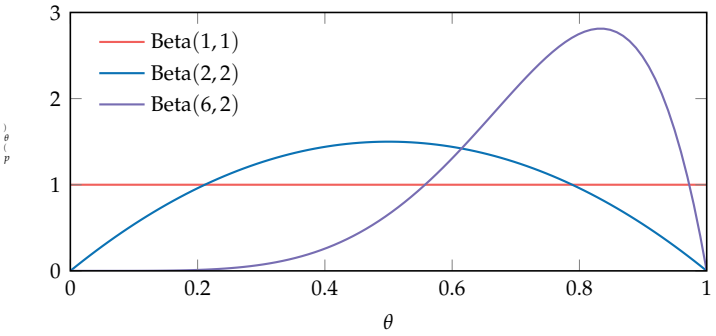


图4.4. 几种贝塔概率密度的叠加图。

方便的是，如果使用贝塔分布作为二元分布参数的先验，那么后验也是贝塔分布。特别地，如果先验由 Beta(α, β) 给出，并且我们进行了一次观测 o_i，那么当 o_i ==1 时，后验为 Beta($\alpha + 1, \beta$)，而当 o_i ==0 时，后验为 Beta($\alpha, \beta +$ 1)。因此，如果我们从由 Beta(α, β) 给出的先验开始，并且我们的数据显示在 m 次飞行中有 n 次碰撞，那么后验将由 Beta($\alpha + n, \beta + m - n$) 给出。先验中的 α 和 β 参数有时被称为 *pseudocounts*，因为它们在后验中被类似地视为两个结果类别的观测计数，尽管这些伪计数不一定是整数。

原则上，选择先验时不应知晓用于计算后验的数据。均匀先验在实践中往往效果良好，然而，若具备专家知识，则可将其融入先验之中。例如，假设我们有一枚略微弯曲的硬币，想要估计 θ，即硬币正面朝上的概率。在通过掷币收集任何数据之前，我们会基于一个大致围绕0.5的信念 θ 开始。与其采用均匀先验 Beta(1, 1)，我们或许会使用 Beta(2, 2)（如图4.4所示），它给予接近0.5的值更多权重。如果我们对接近0.5的估计更有信心，那么可以通过增加伪计数来减小先验的方差。先验 Beta(10, 10) 比 Beta(2, 2) 更为尖锐。然而，总体而言，先验的重要性随着用于计算后验的数据量增加而减弱。如果我们观察到 m 次投掷中有 n 次正面，那么在观测到成千上万次投掷的情况下，Beta($1 + n, 1 + m - n$) 与 Beta($10 + n, 10 + m - n$) 之间的差异可以忽略不计。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

4.2.2 *Bayesian Learning for Categorical Distributions*

Dirichlet distribution[4] 是贝塔分布的推广，并且可以用于估计类别分布的参数。假设 X 是一个离散随机变量，其取值范围为从 1 到 n 的整数值。我们定义分布的参数为 $\theta_{1:n}$，其中 $P(x^i) = \theta_i$。当然，这些参数之和必须为1，因此只有前 $n-1$ 个参数是独立的。狄利克雷分布可用于表示先验分布和后验分布，并由 $\alpha_{1:n}$ 参数化。其密度函数为

4 这个分布是以德国数学家约翰·彼得·古斯塔夫·勒热纳·狄利克雷（1805–1859）的名字命名的。

$$\text{Dir}(\theta_{1:n} \mid \alpha_{1:n}) = \frac{\Gamma(\alpha_0)}{\prod_{i=1}^{n} \Gamma(\alpha_i)} \prod_{i=1}^{n} \theta_i^{\alpha_i - 1} \tag{4.29}$$

其中 α_0 用于表示参数 $\alpha_{1:n}$ 的求和。[5] 如果 $n = 2$，则很容易看出，方程(4.29)等价于贝塔分布。

5 不同参数的狄利克雷分布密度图见附录B。

通常使用均匀先验，其中所有狄利克雷参数 $\alpha_{1:n}$ 都设为 1。与贝塔分布一样，狄利克雷分布中的参数通常被称为 *pseudocounts*。如果 $\theta_{1:n}$ 上的先验由 Dir($\alpha_{1:n}$) 给出，并且有 m_i 个 $X = i$ 的观测值，那么后验由下式给出：

$$p(\theta_{1:n} \mid \alpha_{1:n}, m_{1:n}) = \text{Dir}(\theta_{1:n} \mid \alpha_1 + m_1, \ldots, \alpha_n + m_n) \tag{4.30}$$

分布Dir($\alpha_{1:n}$)的均值向量的第i个分量为

$$\frac{\alpha_i}{\sum_{j=1}^{n} \alpha_j} \tag{4.31}$$

当 $\alpha_i >$ 为 1 时，该模式的第 i 个分量为

$$\frac{\alpha_i - 1}{\sum_{j=1}^{n} \alpha_j - n} \tag{4.32}$$

正如我们所见，贝叶斯参数估计对于二元和离散随机变量来说很直接，因为它只涉及简单地计数数据中的各种结果。贝叶斯法则可以用来推断其他参数分布的参数分布。根据先验的选择和参数分布的形式，计算参数空间上的后验也可以分析性地完成。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

4.2.3 *Bayesian Learning for Bayesian Networks*

我们可以将贝叶斯参数学习应用于离散贝叶斯网络。贝叶斯网络参数 θ 的先验可以分解如下：

$$p(\theta \mid G) = \prod_{i=1}^{n} \prod_{j=1}^{q_i} p(\theta_{ij}) \tag{4.33}$$

其中 $\theta_{ij} = (\theta_{ij1}, \ldots, \theta_{ijr_i})$。在某些弱假设下，先验 $p(\theta_{ij})$ 可以证明遵循 Dirichlet 分布 $\text{Dir}(\alpha_{ij1}, \ldots, \alpha_{ijr_i})$。算法 4.2 提供了一种实现，用于创建保存 α_{ijk} 的数据结构，其中所有条目均为 1，对应于均匀先验。

在观察到形式为 m_{ijk} 计数的数据（如第4.1.3节所介绍）后，后验分布为

$$p(\theta_{ij} \mid \alpha_{ij}, m_{ij}) = \text{Dir}(\theta_{ij} \mid \alpha_{ij1} + m_{ij1}, \ldots, \alpha_{ijr_i} + m_{ijr_i}) \tag{4.34}$$

与公式 (4.30) 类似。例4.2展示了这一过程。

```
function prior(vars, G)
    n = length(vars)
    r = [vars[i].r for i in 1:n]
    q = [prod([r[j] for j in inneighbors(G,i)]) for i in 1:n]
    return [ones(q[i], r[i]) for i in 1:n]
end
```

算法 4.2. 一个生成先验 α_{ijk} 的函数，其中所有条目均为 1。该函数返回的矩阵数组与算法 4.1 生成的统计量具有相同的形式。为了确定适当的维度，该函数以变量列表 `vars` 和结构 `G` 作为输入。

我们可以通过简单地将先验参数和计数相加来计算与贝叶斯网络相关的后验参数（方程（4.34））。如果我们使用在例4.1中获得的计数矩阵`M`，我们可以将其与先验参数矩阵`α = prior(vars, G)`相加，得到后验参数集`M + α`：

$$\begin{bmatrix} 3 & 3 \end{bmatrix} \quad \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 3 & 1 \\ 1 & 3 \end{bmatrix} \quad \begin{bmatrix} 1 & 5 \end{bmatrix}$$

示例4.2。计算贝叶斯网络中的后验参数。注意，与示例4.1不同，这里我们没有nan值。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

4.3 Nonparametric Learning

前两节假设概率模型具有固定形式，并且需要从数据中学习一组固定参数。另一种方法基于 *nonparametric* 方法，其中参数的数量随数据量而扩展。一种常见的非参数方法是*kernel density estimation* (算法 4.3)。给定观测值 $o_{1:m}$，核密度估计将密度表示如下：

$$p(x) = \frac{1}{m}\sum_{i=1}^{m} \phi(x - o_i) \tag{4.35}$$

其中 ϕ 是一个 *kernel function*，其积分等于 1。核函数用于为接近观测数据点的值分配更大的密度。核函数通常是对称的，这意味着 $\phi(x) = \phi(-x)$。一种常见的核函数是零均值高斯分布。当使用这种核函数时，标准差通常被称为 *bandwidth*，可以通过调整它来控制密度函数的平滑度。较大的带宽通常会导致更平滑的密度。贝叶斯方法可以基于数据用于选择合适的带宽。带宽选择的效果如图 4.5 所示。

```
gaussian_kernel(b) = x->pdf(Normal(0,b), x)

function kernel_density_estimate(ϕ, O)
    return x -> sum([ϕ(x - o) for o in O])/length(O)
end
```

算法4.3。方法`gaussian_kernel`返回一个零均值高斯核$\phi(x)$，带宽为`b`。核密度估计也针对核`ϕ`和观测值列表`O`实现。

4.4 Learning with Missing Data

在学习概率模型的参数时，我们的数据中可能会有*missing*个条目。[6]例如，如果我们正在进行一项调查，一些受访者

可能会决定跳过一个问题。表4.1展示了一个涉及三个二元变量A、B和C且含有缺失项的数据集示例。处理缺失数据的一种方法是丢弃所有*incomplete*的实例，其中存在一个或多个缺失项。根据缺失数据的多少，我们可能不得不丢弃大部分数据。在表4.1中，我们将不得不丢弃除一行之外的所有行，这可能造成浪费。

6 缺失数据下的学习是大量文献的主题。G. Molenberghs、G. Fitzmaurice、M. G. Kenward、A. Tsiatis和G. Verbeke编著的 *Handbook of Missing Data Methodology* 提供了全面的介绍和综述。CRC出版社，2014年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

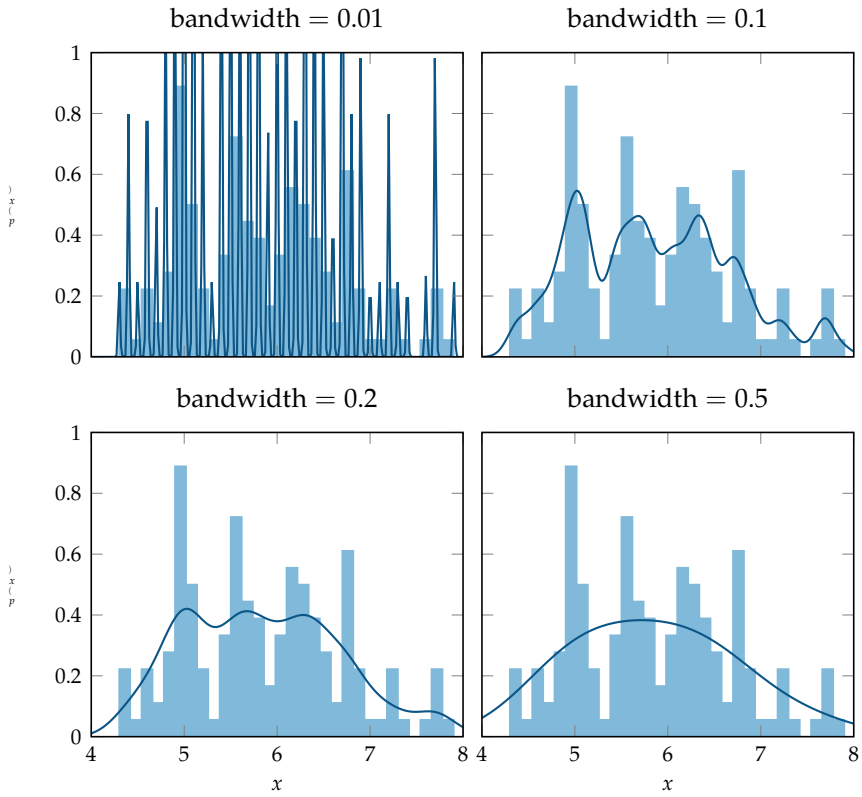


图4.5. 使用不同带宽的零均值高斯核对同一数据集进行的核密度估计。蓝色直方图显示了底层数据集的频率，黑色线条表示核密度估计得出的概率密度。较大的带宽使估计更平滑，而较小的带宽则可能过度拟合特定样本。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们可以使用最大似然或贝叶斯方法从缺失数据中学习模型参数。如果采用贝叶斯最大后验方法，我们希望找到估计

$$\hat{\theta} = \arg\max_{\theta} p(\theta \mid D_{\text{obs}}) \tag{4.36}$$

$$= \arg\max_{\theta} \sum_{D_{\text{mis}}} p(\theta \mid D_{\text{obs}}, D_{\text{mis}}) P(D_{\text{mis}} \mid D_{\text{obs}}) \tag{4.37}$$

其中 D_{obs} 和 D_{mis} 分别包含所有观测数据和缺失数据。如果数据是连续的，那么求和将被积分所替代。对缺失数据的边缘化处理可能会带来高昂的计算成本。同样的边缘化处理也会影响贝叶斯方法的计算可行性。

A	B	C
1	1	0
?	1	1
1	?	?
?	?	?

表4.1. 数据示例包含四个实例，其中有六个缺失项。

本节讨论两种在缺失数据情况下进行学习的通用方法，无需枚举所有可能的缺失值组合。第一种方法涉及使用缺失条目的预测值来学习分布参数。第二种方法涉及一种迭代方法来改进我们的参数估计。

我们将重点关注数据为*missing at random*的上下文，这意味着在给定观测变量值的情况下，某条目缺失的概率与其值条件独立。一个不符合这一假设的情况示例可能包括：雷达数据包含对目标距离的测量，但测量可能因噪声或目标超出感应范围而缺失。条目缺失的事实表明该值更可能是高的。处理这种形式的缺失需要使用不同于我们在此讨论的模型和算法。[7]

7 不同的 *missingness mechanisms* 及相关的推断技术由 R. J. A. Little 和 D. B. Rubin 在 *Statistical Analysis with Missing Data* 第三版 Wiley，2020 中进行了综述。

4.4.1 Data Imputation

丢弃不完整实例的另一种方法是填补缺失条目的值。*Data imputation* 是推断缺失条目值的过程。看待填补的一种方式是将其视为对方程（4.37）的近似，其中我们找到

$$\hat{D}_{\text{mis}} = \arg\max_{D_{\text{mis}}} p(D_{\text{mis}} \mid D_{\text{obs}}) \tag{4.38}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

一旦我们得到插补的缺失值，就可以利用这些数据来得出最大后验估计：

$$\hat{\theta} = \arg\max_{\theta} p(\theta \mid D_{\text{obs}}) \approx \arg\max_{\theta} p(\theta \mid D_{\text{obs}}, \hat{D}_{\text{mis}}) \tag{4.39}$$

或者，我们也可以采用最大似然法。

求解方程（4.38）在计算上可能仍具有挑战性。对于离散数据集，一种简单的方法是用最常观察到的值替换缺失项，称为*marginal mode*。例如，在表4.1中，我们可能会将所有缺失的A值替换为其边际众数1。

A	B	C
−6.5	0.9	4.2
?	4.4	9.2
7.8	?	?
?	?	?

表4.2. 连续值数据示例。

连续数据通常没有重复值。然而，我们可以对连续值拟合一个分布，然后使用所得分布的众数。例如，我们可以对表4.2中的数据拟合一个高斯分布，然后用与每个变量相关的观测值的均值来填补缺失项。图4.6中左上角的图展示了这种方法对二维数据的效果。红线显示了缺失第一或第二分量的值如何与它们的插补对应值配对。然后，我们可以使用观测数据和插补数据来得到联合高斯分布参数的最大似然估计。正如我们所看到的，这种插补方法并不总能产生合理的预测，而且学习到的模型相当差。

如果我们考虑观测变量和未观测变量之间的概率关系，往往能做得更好。在图4.6中，两个变量之间明显存在相关性；因此，知道一个变量的值可以帮助预测另一个变量的值。一种常见的插补方法，称为*nearest-neighbor imputation*，是使用与在观测变量上定义的某种距离度量最近的实例相关的值。图4.6右上角的图使用欧几里得距离进行插补。这种方法往往能带来更好的插补效果和学习到的分布。

另一种方法是将分布拟合到完全观测的数据上，然后利用该分布推断缺失值。我们可以使用上一章中的推断算法来执行这一推断。例如，如果我们的数据是离散的，并且我们可以假设一个贝叶斯网络结构，我们可以利用变量消元或吉布斯采样，从观测变量出发，为一个实例生成缺失变量上的分布。基于这个分布，我们可能采用均值或众数来填充缺失值。或者，我们可以

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

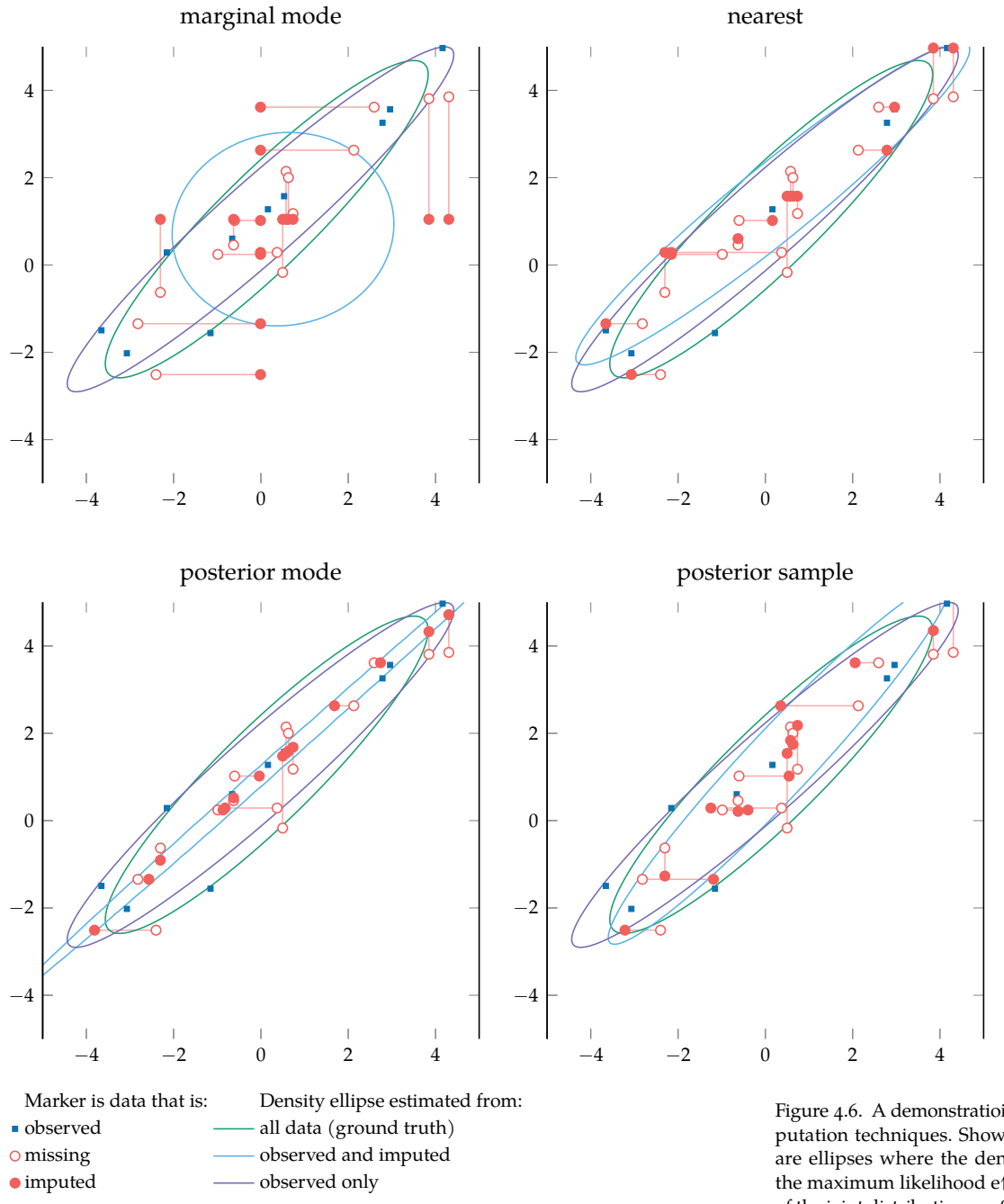


Figure 4.6. A demonstratioim-的数量putation techniques. Shown 此处are ellipses where the den密度为the maximum likelihood e估计of the joint distribution equ0.02。

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

从这个分布中抽取一个样本。如果我们的数据是连续的，并且我们可以假设数据联合高斯分布，我们可以使用算法3.11来推断后验分布。图4.6底部的图展示了使用这些后验众数和后验抽样方法进行的插补。

4.4.2 Expectation-Maximization

expectation-maximization (*EM*)类方法涉及分布参数估计$\hat{\theta}$的迭代改进。[8]我们从一个初始的$\hat{\theta}$开始，

这可能是一个猜测，从分布参数的先验分布中随机抽样，或者使用第4.4.1节中讨论的方法之一进行估计。在每次迭代中，我们执行一个两步过程来更新$\hat{\theta}$。

8 期望最大化算法由 A. P. Dempster、N. M. Laird 和 D. B. Rubin 提出，“Maximum Likelihood from Incomplete Data via the EM Algorithm,” *Journal of the Royal Statistical Society, Series B (Methodological)*, vol. 39, no. 1, pp. 1–38, 1977.

第一步称为*expectation step* (*E-step*)，在此我们使用θ的当前估计来推断数据的补全。例如，如果我们使用离散贝叶斯网络对数据建模，我们可以使用一种推断算法来推断每个实例缺失项上的分布。在提取计数时，我们应用与补全的似然成正比的权重，如例4.3所示。在存在许多缺失变量的情况下，可能的补全可能太多而难以实际枚举，这使得基于采样的方法具有吸引力。当我们的变量是连续的时，我们可能也希望使用采样作为近似方法。

第二步称为 *maximization step* (*M-step*)，在此我们尝试找到一个新$\hat{\theta}$，使完整数据的似然最大化。如果我们有一个离散贝叶斯网络，其加权计数如示例4.3所示，那么我们可以执行本章前面讨论的相同最大似然估计。或者，如果我们想纳入先验，也可以使用最大后验估计。

这种方法不能保证收敛到使观测数据似然最大化的模型参数，但在实践中可以很好地工作。为了降低算法仅收敛到局部最优的风险，我们可以从参数空间中的许多不同初始点运行算法至收敛。我们只需在最后选择使似然最大化的参数估计结果。

期望最大化甚至可以用于为数据中完全未观察到的变量插补值。这样的变量称为*latent variables*。为了说明，假设我们有一个贝叶斯网络$Z \rightarrow X$，其中X是连续的

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

假设我们有一个带有 $A \rightarrow B$ 的二元贝叶斯网络。我们首先假设 $\hat{\theta}$ 意味着

$$P(a^1) = 0.5 \qquad P(b^1 \mid a^0) = 0.2 \qquad P(b^1 \mid a^1) = 0.6$$

使用这些参数，我们可以将含有缺失值的数据集（左）扩展为一个包含所有可能个体补全的加权数据集（右）：

A	B
1	1
0	1
0	?
?	0

A	B	weight
1	1	1
0	1	1
0	0	$1 - P(b^1 \mid a^0) = 0.8$
0	1	$P(b^1 \mid a^0) = 0.2$
0	0	$\alpha P(a^0)P(b^0 \mid a^0) = \alpha 0.4 = 2/3$
1	0	$\alpha P(a^1)P(b^0 \mid a^1) = \alpha 0.2 = 1/3$

这里的α是计算中的一个归一化常数，它确保每个实例都被扩展为权重之和为1的实例。计数矩阵则是

$$\begin{bmatrix} (2 + 2/3) & (1 + 1/3) \end{bmatrix} \qquad \begin{bmatrix} (0.8 + 2/3) & 1.2 \\ 1/3 & 1 \end{bmatrix}$$

示例4.3. 使用假设的模型参数扩展不完整数据集。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Z是离散的，可以取三个值之一。我们的模型假设$p(x \mid z)$是条件高斯分布。我们的数据集仅包含X的值，但不包含Z的值。我们从初始的$\hat{\theta}$开始，并利用它来推断一个关于Z值上的概率分布，给定每个实例的X值。然后，条目完成的分布被用来更新我们对$P(Z)$和$P(X \mid Z)$参数估计的值，如例4.4所示。我们迭代直至收敛，这通常很快发生。在这个例子中我们获得的参数定义了一个高斯混合模型，该模型在第2.2.2节中介绍。

4.5 Summary

- 参数学习涉及从数据中推断概率模型的参数。
- 一种参数学习的最大似然方法涉及最大化似然函数，对于某些模型，这可以通过解析方法完成。
- 一种贝叶斯方法用于参数学习，涉及利用贝叶斯规则推断底层参数的概率分布。
- Beta 分布和 Dirichlet 分布是贝叶斯先验的示例，它们很容易根据证据进行更新。
- 与假设概率模型具有固定参数化的参数学习不同，非参数学习使用随数据量增长而扩展的表示。
- 我们可以使用数据插补或期望最大化等方法来处理从缺失数据中学习参数的问题，其中我们基于观测值进行推断。

4.6 Exercises

练习 4.1。假设安娜正在投篮球罚球。在我们看她比赛之前，我们先对她每次投篮成功命中篮筐的概率设定一个独立的均匀先验。我们观察她投了三次篮，其中两次成功命中。我们赋予她下一次投篮命中的概率是多少?

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

示例4.4。期望最大化应用于学习高斯混合模型的参数。

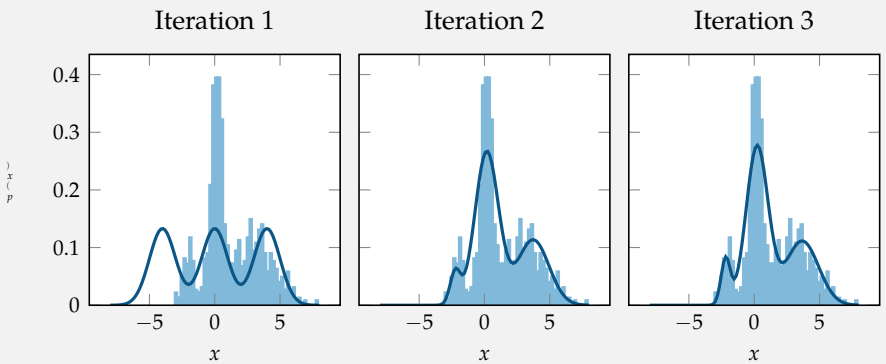
我们有一个贝叶斯网络 $Z \rightarrow X$，其中 Z 是一个具有三个取值的离散潜变量，X 是连续的，$p(x \mid z)$ 被建模为条件高斯分布。因此，我们有定义 $P(z^1)$、$P(z^2)$ 和 $P(z^3)$ 的参数，以及针对与 Z 的不同取值相关联的三个高斯分布各自的 μ_i 和 σ_i。在这个例子中，我们使用一个初始参数向量 $\hat{\theta}$，它指定 $P(z^i) =$ 为 1/3，并且对所有 i 指定 $\sigma_i =$ 为 1。我们将均值分别设为 $\mu_1 = -4$、$\mu_2 = 0$ 和 $\mu_3 = 4$。

假设我们的数据中第一个实例的 $X =$ 为 4.2。我们希望推断该实例上 Z 的分布：

$$P(z^i \mid X = 4.2) = \frac{P(z^i)\,\mathcal{N}(4.2 \mid \mu_i, \sigma_i^2)}{\sum_j P(z^j)\,\mathcal{N}(4.2 \mid \mu_j, \sigma_j^2)}$$

我们为数据集中的所有实例计算该分布。对于加权完成，我们可以获得$\hat{\theta}$的新估计。我们通过取数据集中各实例的均值来估计$P(z^i)$。为了估计μ_i和σ_i，我们使用数据集中各实例上X值的均值和标准差，并以与各实例相关的z^i的概率进行加权。

我们重复该过程直至收敛。此处图表展示了三次迭代。直方图基于X的值生成。深蓝色函数表示推断出的密度。到第三次迭代时，我们的高斯混合模型参数已紧密贴合数据分布。



© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: 我们将投篮命中的概率记为 θ。由于我们从均匀先验 Beta(1, 1) 开始，并观察到两次命中和一次未命中，那么我们的后验为 $\text{Beta}(1+2, 1+1) = \text{Beta}(3, 2)$。我们想按如下方式计算投篮命中的概率：

$$P(\text{basket}) = \int P(\text{basket} \mid \theta)\, \text{Beta}(\theta \mid 3, 2)\, \mathrm{d}\theta = \int \theta\, \text{Beta}(\theta \mid 3, 2)\, \mathrm{d}\theta$$

这个表达式正是beta分布的期望（或均值），它给我们$P(\text{basket}) = 3/5$。

练习4.2。考虑一个连续随机变量X，它遵循由μ和b参数化的*Laplace distribution*，其密度为

$$p(x \mid \mu, b) = \frac{1}{2b} \exp\left(-\frac{|x-\mu|}{b}\right)$$

计算拉普拉斯分布参数的最大似然估计，给定数据集 D，包含 m 个独立观测值 $x_{1:m}$。注意 $\partial |u| / \partial x = \text{sign}(u) \partial u / \partial x$，其中 sign 函数返回其参数的正负号。

Solution: 由于观测是独立的，我们可以将对数似然函数表示为求和形式：

$$\begin{aligned}
\ell(\mu, b) &= \sum_{i=1}^{m} \log\left[\frac{1}{2b} \exp\left(-\frac{|x_i - \mu|}{b}\right)\right] \\
&= -\sum_{i=1}^{m} \log 2b - \sum_{i=1}^{m} \frac{|x_i - \mu|}{b} \\
&= -m \log 2b - \frac{1}{b} \sum_{i=1}^{m} |x_i - \mu|
\end{aligned}$$

为了获得真实参数 μ 和 b 的最大似然估计，我们对对数似然函数针对每个参数求偏导数，令其为零，并解出每个参数。首先，我们解$\hat{\mu}$：

$$\begin{aligned}
\tfrac{\partial}{\partial \mu} \ell(\mu, b) &= \frac{1}{\hat{b}} \sum_{i=1}^{m} \text{sign}(x_i - \mu) \\
0 &= \frac{1}{\hat{b}} \sum_{i=1}^{m} \text{sign}(x_i - \hat{\mu}) \\
0 &= \sum_{i=1}^{m} \text{sign}(x_i - \hat{\mu}) \\
\hat{\mu} &= \text{median}(x_{1:m})
\end{aligned}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

现在，解出$\hat{b}$:

$$\begin{aligned}\frac{\partial}{\partial b}\ell(\mu, b) &= -\frac{m}{b} + \frac{1}{b^2}\sum_{i=1}^{m}|x_i - \hat{\mu}| \\ 0 &= -\frac{m}{\hat{b}} + \frac{1}{\hat{b}^2}\sum_{i=1}^{m}|x_i - \hat{\mu}| \\ \frac{m}{\hat{b}} &= \frac{1}{\hat{b}^2}\sum_{i=1}^{m}|x_i - \hat{\mu}| \\ \hat{b} &= \frac{1}{m}\sum_{i=1}^{m}|x_i - \hat{\mu}|\end{aligned}$$

因此，拉普拉斯分布参数的最大似然估计为$\hat{\mu}$，即观测值的中位数，以及 $\hat{b}$，即中位数绝对偏差的平均值。

练习 4.3。本问题探讨最大似然估计在 *censored data* 中的应用，其中某些测量值仅部分已知。假设我们正在为四轴飞行器无人机制造电动机，并希望建立一个关于它们持续多久才失效的模型。尽管可能有更适合用于对组件可靠性建模的分布，[9] 我们将使用一个由以下参数参数化的 *exponential distribution*:

λ with probability density function $\lambda \exp(-\lambda x)$ and cumulative distribution function $1 - \exp(-\lambda x)$。我们飞行了五架无人机。其中三架分别在132小时、42小时和89小时后发生电机故障。另外两架在200小时后没有发生故障，我们停止了测试；我们不知道它们的故障时间；我们只知道它们大于200小时。根据这些数据，λ的最大似然估计是多少?

[9] K. S. Trivedi and A. Bobbio, *Reliability and Availability Engineering*. Cambridge University Press, 2017.

Solution: 这个问题有 $n =$ 个完全观测的测量值和 $m =$ 个删失测量值。我们用 t_i 表示第 i 个完全观测的测量值，用 $\underline{t}_j$ 表示第 j 个删失测量值。在 $\underline{t}_j$ 以上的单个测量值的似然是累积分布函数的补集，即 $\exp(-\lambda \underline{t}_j)$。因此，数据的似然是

$$\left(\prod_{i=1}^{n}\lambda e^{-\lambda t_i}\right)\left(\prod_{j=1}^{m}e^{-\lambda \underline{t}_j}\right)$$

我们使用我们的最大化对数似然的标准方法，即 g 给定为

$$\ell(\lambda) = \sum_{i=1}^{n}(\log\lambda - \lambda t_i) + \sum_{j=1}^{m} -\lambda \underline{t}_j$$

关于 λ 的导数是

$$\frac{\partial \ell}{\partial \lambda} = \frac{n}{\lambda} - \sum_{i=1}^{n} t_i - \sum_{j=1}^{m} \underline{t}_j$$

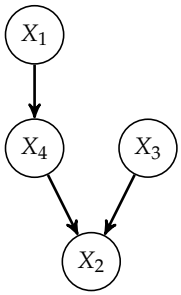
© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

将这个导数设为0，我们可以解出λ，从而得到最大似然估计：

$$\hat{\lambda} = \frac{n}{\sum_{i=1}^{n} t_i + \sum_{j=1}^{m} \underline{t}_j} = \frac{3}{132 + 42 + 89 + 200 + 200} \approx 0.00452$$

指数分布的均值为$1/\lambda$，因此我们问题中的均值为221小时。

练习4.4。我们有一个贝叶斯网络，其中变量$X_{1:3}$可以取$\{1, 2\}$中的值，而X_4可以取$\{1, 2, 3\}$中的值。给定观察值$o_{1:m}$的数据集D，如图所示，生成相关条件分布参数θ的最大似然估计。



$$\mathbf{D} = \begin{bmatrix} 1 & 2 & 1 & 1 & 1 & 2 & 1 & 2 & 1 & 1 \\ 2 & 2 & 2 & 1 & 2 & 1 & 1 & 1 & 2 & 1 \\ 2 & 2 & 2 & 1 & 1 & 1 & 1 & 1 & 2 & 1 \\ 3 & 2 & 1 & 1 & 1 & 3 & 3 & 1 & 1 & 1 \end{bmatrix}$$

Solution: 我们可以通过遍历数据集并存储计数，为每个节点生成大小为$q_i \times r_i$的计数矩阵$\mathbf{M}_i$。然后，我们对计数矩阵中的每一行进行归一化处理，得到包含参数最大似然估计的矩阵：

$$\mathbf{M}_1 = \begin{bmatrix} 7 & 3 \end{bmatrix} \qquad \mathbf{M}_2 = \begin{bmatrix} 3 & 1 \\ 0 & 0 \\ 2 & 0 \\ 0 & 2 \\ 0 & 1 \\ 0 & 1 \end{bmatrix} \qquad \mathbf{M}_3 = \begin{bmatrix} 6 & 4 \end{bmatrix} \qquad \mathbf{M}_4 = \begin{bmatrix} 5 & 0 & 2 \\ 1 & 1 & 1 \end{bmatrix}$$

$$\hat{\theta}_1 = \begin{bmatrix} 0.7 & 0.3 \end{bmatrix} \quad \hat{\theta}_2 = \begin{bmatrix} 0.75 & 0.25 \\ \text{NAN} & \text{NAN} \\ 1.0 & 0.0 \\ 0.0 & 1.0 \\ 0.0 & 1.0 \\ 0.0 & 1.0 \end{bmatrix} \quad \hat{\theta}_3 = \begin{bmatrix} 0.6 & 0.4 \end{bmatrix} \quad \hat{\theta}_4 \approx \begin{bmatrix} 0.71 & 0.0 & 0.29 \\ 0.33 & 0.33 & 0.34 \end{bmatrix}$$

练习 4.5. 我们有一枚偏斜的硬币，我们想要估计伯努利参数 ϕ，该参数指定了硬币正面朝上的概率。如果第一次投掷结果为正面（$o_1 = 1$），请回答以下问题：

- ϕ 的最大似然估计是什么?

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

- 使用均匀先验，ϕ的最大后验估计是什么?
- 使用均匀先验，我们对 ϕ 的后验分布的期望是什么?

Solution: 由于我们的第一次抛掷落在正面，我们有$n = 1$次成功和$m = 1$次试验。

- ϕ 的最大似然估计是 $n/m = 1$。
- 使用均匀的Beta(1, 1)先验，后验分布为Beta$(1 + n, 1 + m - n)$ = Beta(2, 1)。ϕ的最大后验估计或后验分布的众数是

$$\frac{\alpha - 1}{\alpha + \beta - 2} = \frac{2 - 1}{2 + 1 - 2} = 1$$

- 后验分布的均值是

$$\frac{\alpha}{\alpha + \beta} = \frac{2}{2 + 1} = \frac{2}{3}$$

练习4.6。假设我们得到了以下数据集中有一个缺失值。如果假设边缘分布为高斯分布，使用边缘众数插补将会填充什么值？使用最近邻插补将会填充什么值?

X_1	X_2
0.5	1.0
?	0.3
−0.6	−0.3
0.1	0.2

Solution: 假设{v20}的边缘分布是高斯分布，我们可以计算边缘众数，即高斯分布的均值参数:

$$\mu = \frac{1}{m}\sum_{i=1}^{m} x_i = \frac{0.5 - 0.6 + 0.1}{3} = 0$$

因此，对于边际众数插补，缺失值将被设为0。对于最近邻插补，距离$X_2 = 0.3$最近的样本是第四个样本，因此缺失值将被设为0.1。

练习4.7. 假设我们有一个包含两个变量$X_{1:2}$的数据集，其中存在若干缺失值。我们假定$X_{1:2}$联合高斯分布，并利用完全观测的样本拟合以下分布:

$$\begin{bmatrix} X_1 \\ X_2 \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} 5 \\ 2 \end{bmatrix}, \begin{bmatrix} 4 & 1 \\ 1 & 2 \end{bmatrix} \right)$$

对于样本 $X_2 = 1.5$，使用后验众数插补时，X_1 的插补值是什么？对于后验样本插补，我们需要从哪个分布中抽样?

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: 由于我们假设 $X_{1:2}$ 联合高斯分布，给定 X_2 下 X_1 的后验分布也是高斯分布，其众数即为后验分布的均值参数。我们可以按照以下方式计算后验分布的均值：

$$p(x_1 \mid x_2) = \mathcal{N}\left(x_1 \mid \mu_{x_1|x_2}, \sigma^2_{x_1|x_2}\right)$$
$$\mu_{x_1|x_2=1.5} = 5 + (1)(2)^{-1}(1.5 - 2) = 4.75$$

因此，对于后验众数插补，缺失值将被设为4.75。对于后验样本插补，我们将从$X_1 \sim \mathcal{N}($4.75, 3.5)中抽取一个值。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

5 Structure Learning

本书前几章假设概率模型的结构是已知的。本章讨论从数据中学习模型结构的方法。[1] 我们首先解释如何计算概率

给定数据的情况下，关于图结构的概率。通常，我们希望最大化这个概率。由于可能的图结构空间通常太大而无法枚举，我们还将讨论高效搜索这一空间的方法。

1 贝叶斯网络结构学习的概述可在以下教科书中找到：D. Koller 和 N. Friedman，*Probabilis-*。MIT Press，2009年；R. E. Neapolitan，*Learning Bayesian Net-*。Prentice Hall，2003年。

tic Graphical Models: Principles and Techniques

works

5.1 Bayesian Network Scoring

我们希望能够基于网络结构G对数据的建模效果来对其进行评分。结构学习的最大后验方法涉及找到一个G，使得$P(G \mid D)$最大化。我们首先解释如何基于$P(G \mid D)$计算贝叶斯分数，以衡量G对数据的建模效果。然后，我们解释如何在网络空间中搜索得分最高的网络。与贝叶斯网络中的推断类似，可以证明对于一般图和输入数据，学习贝叶斯网络的结构是NP难的。[2]

2 参见 D. M. Chickering，“学习贝叶斯网络是NP完全的”，载于*Learning from Data:*，D. Fisher 和 H.-J. Lenz 编，Springer，1996年，第121–130页。D. M. Chickering、D. Heckerman 和 C. Meek，“贝叶斯网络的大样本学习是NP困难的”，*Journal of Machine Learning Research*，第5卷，第1287–1330页，2004年。

Artificial Intelligence and Statis-

V

我们使用贝叶斯法则和全概率公式计算 $P(G \mid D)$：

$$P(G \mid D) \propto P(G)P(D \mid G) \tag{5.1}$$

$$= P(G) \int P(D \mid \theta, G) p(\theta \mid G) \, \mathrm{d}\theta \tag{5.2}$$

其中 θ 包含上一章介绍的网络参数。对 θ 进行积分得到 [3]

$$P(G \mid D) = P(G) \prod_{i=1}^{n} \prod_{j=1}^{q_i} \frac{\Gamma(\alpha_{ij0})}{\Gamma(\alpha_{ij0} + m_{ij0})} \prod_{k=1}^{r_i} \frac{\Gamma(\alpha_{ijk} + m_{ijk})}{\Gamma(\alpha_{ijk})} \tag{5.3}$$

3 推导过程参见 G. F. Cooper 和 E. Herskovits 的附录，“一种从数据归纳概率网络的贝叶斯方法”，*Machine Learn-*，第4卷第9期，第309–347页，1992年。

ing

其中 α_{ijk} 的值是伪计数，而 m_{ijk} 是计数，如前一章所介绍。我们还定义

$$\alpha_{ij0} = \sum_{k=1}^{r_i} \alpha_{ijk} \qquad m_{ij0} = \sum_{k=1}^{r_i} m_{ijk} \tag{5.4}$$

找到使方程（5.2）最大化的G，等同于找到使所谓的*Bayesian score*最大化的G：

$$\log P(G \mid D) = \log P(G) + \sum_{i=1}^{n} \sum_{j=1}^{q_i} \left(\log \left(\frac{\Gamma(\alpha_{ij0})}{\Gamma(\alpha_{ij0} + m_{ij0})} \right) + \sum_{k=1}^{r_i} \log \left(\frac{\Gamma(\alpha_{ijk} + m_{ijk})}{\Gamma(\alpha_{ijk})} \right) \right) \tag{5.5}$$

贝叶斯分数在数值计算上更为便捷，因为将小数的对数相加比将小数相乘更容易。许多软件库能够直接计算伽马函数的对数。

文献中已经探索了多种图先验，尽管在实践中通常使用均匀先验，此时可以从方程（5.5）中贝叶斯分数的计算中删除log {v*}。算法5.1提供了实现。

```
function bayesian_score_component(M, α)
    p =  sum(loggamma.(α + M))
    p -= sum(loggamma.(α))
    p += sum(loggamma.(sum(α,dims=2)))
    p -= sum(loggamma.(sum(α,dims=2) + sum(M,dims=2)))
    return p
end

function bayesian_score(vars, G, D)
    n = length(vars)
    M = statistics(vars, G, D)
    α = prior(vars, G)
    return sum(bayesian_score_component(M[i], α[i]) for i in 1:n)
end
```

算法5.1。一种用于计算给定数据 `D` 下变量列表 `vars` 和图 `G` 的贝叶斯评分的算法。该方法对所有由算法4.2生成的 i、j 和 k 使用均匀先验 $\alpha_{ijk} = 1$。`loggamma` 函数由 `SpecialFunctions.jl` 提供。第4章介绍了 `statistics` 和 `prior` 函数。注意 $\log(\Gamma(\alpha)/\Gamma(\alpha + m)) = \log \Gamma(\alpha) - \log \Gamma(\alpha + m)$，以及 $\log \Gamma(1) = 0$。

相对于贝叶斯分数优化结构的一个副产品是，我们能够在给定可用数据的情况下，找到模型复杂度的正确平衡。我们不希望模型错过捕捉变量间重要关系的机会，但也不希望模型因参数过多而无法从有限数据中得到充分学习。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

为了说明贝叶斯得分如何帮助我们平衡模型复杂度，考虑图5.1中的网络。A的值微弱地影响B的值，而C独立于其他变量。我们从这个“真实”模型中抽样以生成数据D，然后尝试学习模型结构。涉及三个变量的可能网络结构有25种，但我们将重点关注图5.2中模型的得分。

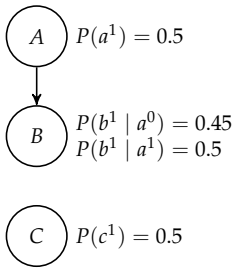


图5.1. 一个简单的贝叶斯网络，用于说明贝叶斯得分如何帮助我们平衡模型的复杂度。

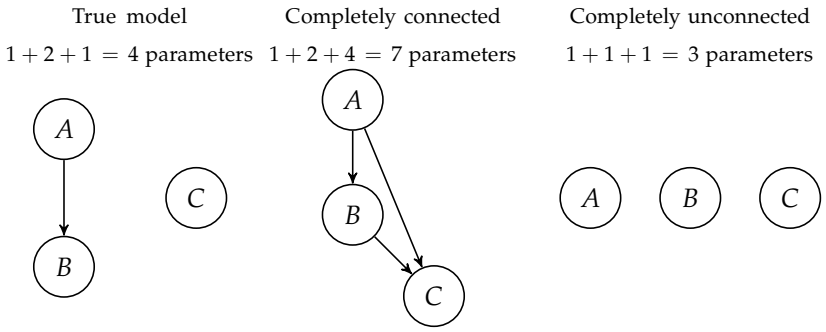


图5.2. 三种复杂度不同的贝叶斯网络结构。

图5.3展示了随着数据量的增加，完全连接模型与未连接模型的贝叶斯评分与真实模型的比较情况。在图中，我们减去了真实模型的评分，因此高于0的值表明，在可用数据下，该模型提供了比真实模型更好的表示。该图显示，当样本数少于约5×10^3时，未连接模型的表现优于真实模型。完全连接模型从未优于真实模型，但在约10^4个样本时开始优于未连接模型，因为此时有足够的数据来充分估计其七个独立参数。

5.2 Directed Graph Search

在*directed graph search*中，我们在有向无环图的空间中搜索能够最大化贝叶斯得分的图。可能的贝叶斯网络结构的空间以超指数级增长。[4] 当有10个节点时，存在4.2×10^{18} 种可能的有向无环图。当有20个节点时，存在2.4×10^{72}种可能。除了节点数量较少的贝叶斯网络外，我们无法通过枚举所有可能的结构来寻找评分最高的网络。因此，我们必须依赖搜索策略。

4 R. W. Robinson, “Counting Labeled Acyclic Digraphs,” in *New Directions in the Theory of Graphs*, 1973年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

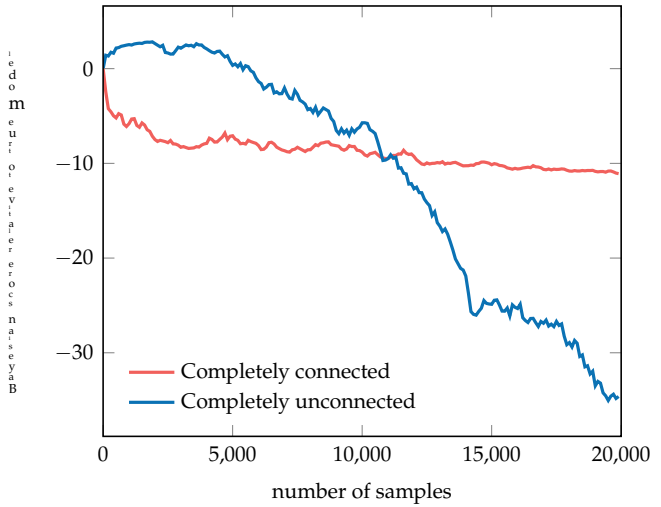


图5.3. 贝叶斯网络结构学习在模型复杂性与可用数据之间进行权衡。完全连接的模型从未超越真实模型，而完全未连接的模型在抽取了大约超过5×10^3个样本后，最终表现不佳。这一结果表明，在数据稀缺时，较简单的模型可能胜过复杂的模型——即使样本是由更复杂的模型生成的。

幸运的是，搜索是一个普遍问题，多年来已经研究了各种各样的通用搜索算法。

最常见的搜索策略之一称为$K2$。[5] 搜索（算法
算法5.2）在多项式时间内运行，但不保证能找到全局最优的网络结构。它可以使用任何评分函数，但由于其能够平衡模型复杂性和可用数据量，常与贝叶斯评分一起使用。K2从一个没有有向边的图开始，然后根据提供的顺序遍历变量，以最大化提高评分的方式贪婪地向节点添加父节点。K2通常会对任一节点的父节点数量设定上限，以减少所需的计算量。原始的K2算法假设对所有i、j和k的$\alpha_{ijk} = 1$采用单位均匀Dirichlet先验，但原则上可以使用任何先验。

5 该名称源于它是一个名为Kutató的系统的演进。该算法由G. F. Cooper和E. Herskovits在“A Bayesian Method for the Induction of Probabilistic Networks from Data,” *Machine Learning*, vol. 4, no. 9, pp. 309–347, 1992中提出。

一种通用的搜索策略是*local search*，有时被称为*hill climbing*。算法5.3提供了这一概念的实现。我们从初始图开始，然后移动到得分最高的邻居。图的邻域包括仅通过一个基本图操作即可到达的图，这些基本图操作包括引入一条边、删除一条边和反转一条边。当然，并非所有操作都能从特定的

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct K2Search
    ordering::Vector{Int} # variable ordering
end

function fit(method::K2Search, vars, D)
    G = SimpleDiGraph(length(vars))
    for (k,i) in enumerate(method.ordering[2:end])
        y = bayesian_score(vars, G, D)
        while true
            y_best, j_best = -Inf, 0
            for j in method.ordering[1:k]
                if !has_edge(G, j, i)
                    add_edge!(G, j, i)
                    y′ = bayesian_score(vars, G, D)
                    if y′ > y_best
                        y_best, j_best = y′, j
                    end
                    rem_edge!(G, j, i)
                end
            end
            if y_best > y
                y = y_best
                add_edge!(G, j_best, i)
            else
                break
            end
        end
    end
    return G
end
```

算法5.2. 使用指定变量顺序对 directed acyclic graphs 空间进行K2搜索。该变量顺序在结果图中强制了一个拓扑排序。`fit` 函数接受一个有序变量列表 `vars` 和一个数据集合 `D`。该方法从空图开始，迭代地添加能够最大程度改进贝叶斯评分的下一个父节点。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

图，并且引入环的操作是无效的。搜索持续进行，直到当前图的得分不低于其任何相邻图。

局部搜索的*opportunistic*版本在算法5.3中实现。这种方法不是每次迭代都生成所有图邻居，而是生成一个随机邻居，如果其贝叶斯分数高于当前图，则接受它。

```
struct LocalDirectedGraphSearch
    G     # initial graph
    k_max # number of iterations
end

function rand_graph_neighbor(G)
    n = nv(G)
    i = rand(1:n)
    j = mod1(i + rand(2:n)-1, n)
    G′ = copy(G)
    has_edge(G, i, j) ? rem_edge!(G′, i, j) : add_edge!(G′, i, j)
    return G′
end

function fit(method::LocalDirectedGraphSearch, vars, D)
    G = method.G
    y = bayesian_score(vars, G, D)
    for k in 1:method.k_max
        G′ = rand_graph_neighbor(G)
        y′ = is_cyclic(G′) ? -Inf : bayesian_score(vars, G′, D)
        if y′ > y
            y, G = y′, G′
        end
    end
    return G
end
```

算法5.3. 局部有向图搜索，该算法从初始有向图`G`开始，并在贝叶斯得分更高时机会性地移动到随机图邻居。它重复此过程进行`k_max`次迭代。随机图邻居通过添加或删除单条边来生成。此算法可以扩展以包括反转边的方向。添加边可能导致图含有循环，在这种情况下，我们分配一个分数$-\infty$。

局部搜索可能会陷入 *local optima*，从而无法找到全局最优的网络结构。针对局部最优问题，人们提出了多种策略，包括以下几种：[6]

- *Randomized restart*一旦找到局部最优解，只需从搜索空间中的随机点重新开始搜索。
- *Simulated annealing*相反，不是总是移动到适应度最高的邻居，搜索可以根据某种随机探索策略访问适应度较低的邻居。随着搜索的进行，随机性在

6 优化领域相当广阔，已发展出许多方法来解决局部最优问题。本教科书提供了一个概览：M. J. Kochenderfer 和 T. A. Wheeler, *Algorithms for Opti-*。MIT Press, 2019。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

探索根据特定计划减少。这种方法被称为模拟退火，因为它受到冶金退火的启发。

- *Genetic algorithms*该过程从搜索空间中一组初始的随机点开始，这些点表示为二进制字符串。字符串中的每一位表示两个节点之间是否存在箭头。因此，字符串操作允许在 有向图空间中进行搜索。种群中的个体按其得分成比例地繁殖。被选中繁殖的个体会通过遗传交叉随机重组其字符串，这包括在两个随机选中的个体上选择一个交叉点，然后交换该点之后的字符串。突变也通过随机翻转字符串中的比特被随机引入种群。进化过程持续进行，直到在搜索空间中找到一个令人满意的点。

- *Memetic algorithms*. 这种方法有时称为*genetic local search*，它只是遗传算法和局部搜索的结合。在遗传重组之后，对个体进行局部搜索。

- *Tabu search*先前的方法可以进行增强，以维护一个包含搜索空间中最近访问点的*tabu list*。搜索算法避免选择禁忌列表中的邻居。

某些搜索策略在特定数据集上可能比其他策略更有效，但总体而言，寻找全局最优解仍然是NP难问题。然而，许多应用并不需要全局最优的网络结构，局部最优结构通常是可以接受的。

5.3 Markov Equivalence Classes

如前所述，贝叶斯网络的结构编码了一组条件独立性假设。在尝试学习贝叶斯网络结构时，一个重要的观察是，两个不同的图可以编码相同的独立性假设。作为一个简单的例子，两变量网络 $A \rightarrow B$ 与 $A \leftarrow B$ 具有相同的独立性假设。仅基于数据，我们无法证明 A 和 B 之间边的方向。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

如果两个网络编码相同的条件独立假设，我们称它们是 *Markov equivalent*。可以证明，两个图是马尔可夫等价的当且仅当它们具有（1）相同的边，不考虑方向；（2）相同的非道德v结构集合。一个 *immoral v-structure* 是一个v结构 $X \rightarrow Y \leftarrow Z$，其中 X 和 Z 不直接相连，如图5.4所示。一个 *Markov equivalence class* 是一个包含所有彼此马尔可夫等价的有向无环图的集合。检查马尔可夫等价的方法见算法5.4。

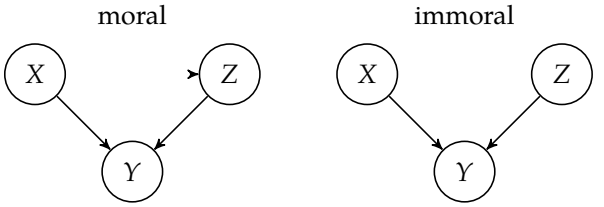


图5.4. 道德与不道德的{v*}结构。

通常，属于同一马尔可夫等价类的两个结构可能会得到不同的分数。然而，如果使用带有狄利克雷先验的贝叶斯分数，使得 $\kappa = \sum_j \sum_k \alpha_{ijk}$ 对于所有 i 都是常数，那么两个马尔可夫等价结构会被赋予相同的分数。[7] 这样的先验称为 *BDe*，并且一个特殊的情况是先验 *BDeu*，[8] 分配为 $\alpha_{ijk} = \kappa/(q_i r_i)$。尽管通常使用均匀先验$\alpha_{ijk} = 1$并不总是导致同一等价类中的结构被赋予相同的分数，它们通常相当接近。将所有同一类中的结构赋予相同分数的评分函数称为 *score equivalent*。

7 这由D. Heckerman、D. Geiger和D. M. Chickering在“学习贝叶斯网络：知识与统计数据的结合”中展示，*Machine*，第20卷，第3期，第197-243页，1995年。

8 W. L. Buntine，《贝叶斯网络上的理论精化》，载于 *Conference on Uncertainty in Artificial Intelligence (UAI)*，1991年。

5.4 Partially Directed Graph Search

一个马尔可夫等价类可以表示为一个 *partially directed graph*，有时称为 *essential graph* 或 *directed acyclic graph pattern*。部分有向图可以同时包含有向边和无向边。图5.5展示了一个编码马尔可夫等价类的部分有向图的示例。一个有向无环图 G 是部分有向图 G' 所编码的马尔可夫等价类的成员，当且仅当 G 与 G' 在不考虑方向的情况下具有相同的边，并且与 G' 具有相同的不道德v结构。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
function are_markov_equivalent(G, H)
    if nv(G) != nv(H) || ne(G) != ne(H) ||
        !all(has_edge(H, e) || has_edge(H, reverse(e))
                                        for e in edges(G))
        return false
    end
    for (I, J) in [(G,H), (H,G)]
        for c in 1:nv(I)
            parents = inneighbors(I, c)
             for (a, b) in subsets(parents, 2)
                 if !has_edge(I, a, b) && !has_edge(I, b, a) &&
                    !(has_edge(J, a, c) && has_edge(J, b, c))
                     return false
                 end
             end
        end
    end

    return true
end
```

算法5.4。一种用于判定有向无环图G和H是否马尔可夫等价的方法。来自IterTools.jl的subsets函数返回给定集合及指定大小的所有子集。

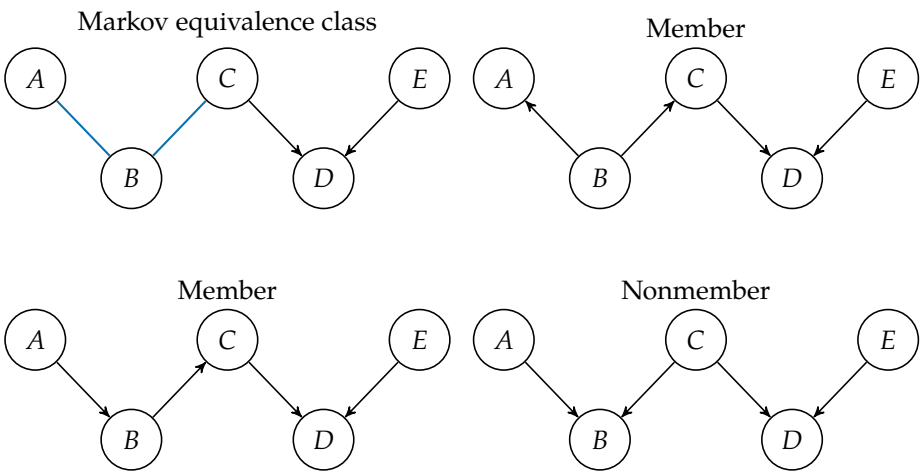


图5.5. 一个马尔可夫等价类及其成员示例和一个非成员示例。该非成员不属于马尔可夫等价类，因为它引入了一个不道德v-结构，$A \rightarrow B \leftarrow C$，这在部分有向图中未标明。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们不必搜索有向无环图的空间，而是可以搜索由部分有向图表示的马尔可夫等价类的空间。[9]
尽管马尔可夫等价类的空间当然小于有向无环图的空间，但并没有显著减小；有向无环图与等价类的比例很快渐近于大约3.7。[10]

关于如何搜索这个空间的具体细节由 D. M. Chickering 在“Learning Equivalence Classes of Bayesian-Network Structures,” *Journal of Machine Learning Research*, vol. 2, pp. 445–498, 2002 中提供。

10 S. B. Gillispie 和 M. D. Perlman，“无环有向图模型的马尔可夫等价类的大小分布”，*Artificial Intelligence*，第141卷，第1–2期，第137–155页，2002年。

在有向无环图空间中进行爬山法的一个问题是，邻域可能包含其他处于同一等价类且得分相同的图，这可能导致搜索陷入局部最优。搜索等价类空间允许我们跳到当前等价类之外的不同有向无环图。

可以使用第5.2节中介绍的任何通用搜索策略。如果采用某种局部搜索形式，则需要定义定义图的邻域的局部图操作。局部图操作的示例包括：

- 如果X和Y之间不存在边，则添加$X - Y$或$X \rightarrow Y$。
- 如果 $X - Y$ 或 $X \rightarrow Y$，则移除 X 和 Y 之间的边。
- 如果 $X \rightarrow Y$，则反转边的方向以得到 $X \leftarrow Y$。
- 如果 $X - Y - Z$，则添加 $X \rightarrow Y \leftarrow Z$。

为了给部分有向图评分，我们生成其马尔可夫等价类的一个成员，并计算其得分。

5.5 Summary

- 将贝叶斯网络拟合到数据需要选择决定变量之间条件依赖关系的贝叶斯网络结构。
- 结构学习的贝叶斯方法最大化贝叶斯得分，该得分与给定数据集的图结构概率相关。
- 贝叶斯评分对于较小的数据集倾向于更简单的结构，而对于较大的数据集则支持更复杂的结构。
- 可能结构的数量随变量数量呈超指数增长，并且寻找使贝叶斯分数最大化的结构是NP难问题。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

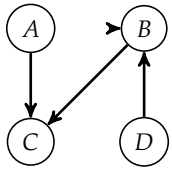
- 像K2和局部搜索这样的有向图搜索算法虽然可能高效，但并不能保证最优性。
- 像部分有向图搜索这样的方法遍历马尔可夫等价类的空间，这可能比搜索有向无环图的更大空间更有效率。

5.6 Exercises

练习5.1. 一个有m个节点的无边的有向无环图有多少个邻居?

Solution: 在三个基本图操作中，我们只能添加边。我们可以向无边有向无环图中添加任意边，而它仍将保持无环。有$m(m-1) = m^2 - m$个节点对，因此有那么多邻居。

练习5.2。以下贝叶斯网络的邻域中有多少个网络?

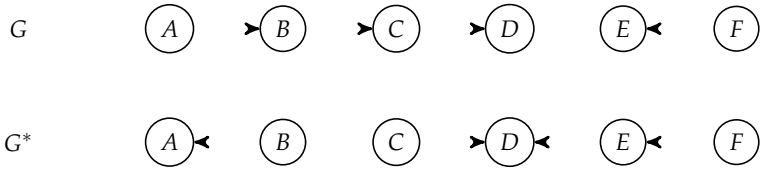


Solution: 我们可以执行以下图操作：

- 添加 $A \to D$、$D \to A$、$D \to C$
- 删除 $A \to B$、$A \to C$、$B \to C$、$D \to B$
- 翻转 $A \to B$、$B \to C$、$D \to B$

因此，邻域中有10个贝叶斯网络。

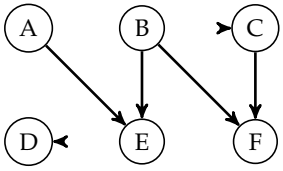
练习5.3。假设我们从一个贝叶斯网络G开始进行局部搜索。为了收敛到最优贝叶斯网络G^*，最少需要执行多少次局部搜索迭代?



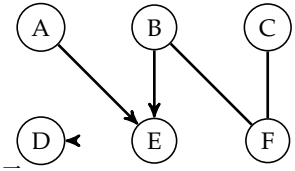
© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: 在每次迭代中，局部搜索可以从原始网络移动到其邻域内的一个网络，该邻域至多与原始网络相差一个边操作。由于G和G^*的边之间存在三处差异，从G进行局部搜索至少需要三次迭代才能到达G^*。一个潜在的最小局部搜索迭代序列可能包括翻转$A \rightarrow B$、移除$B \rightarrow C$和添加$E \rightarrow D$。我们假设，使用这些边操作形成的图在所考虑的邻域中获得了所有图中最高的贝叶斯分数。

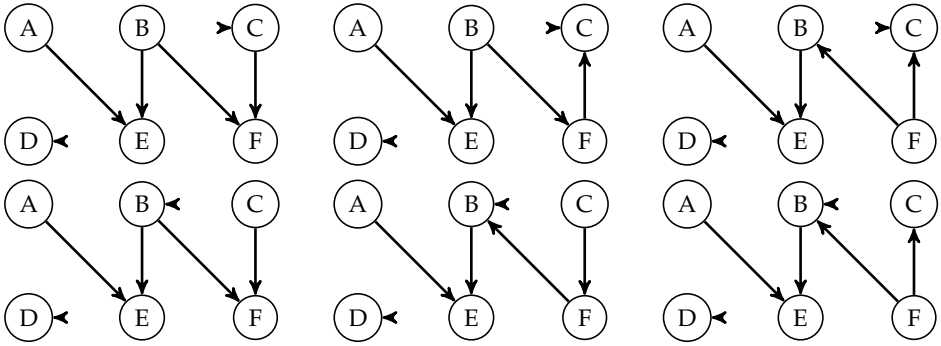
练习5.4。绘制表示以下贝叶斯网络马尔可夫等价类的部分有向无环图。这个马尔可夫等价类中有多少个图?



Solution: 马尔可夫等价类可以用以下部分有向无环图表示：



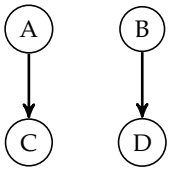
这个马尔可夫等价类中有六个网络，如下所示：



© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

练习5.5。给出一个具有四个节点的部分有向无环图的例子，该图未定义非空马尔可夫等价类。

Solution: 考虑以下部分有向无环图：



我们不能用有向边替换无向边，因为这样做会引入一个新的v结构。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

6 Simple Decisions

本章介绍 *simple decisions* 的概念，其中我们在不确定性下做出单个决策。[1] 我们将从以下方面研究决策问题：

从*utility theory*的角度来看，这涉及将代理的偏好建模为不确定结果上的实值函数。[2] 本章开始

通过讨论关于理性偏好的一小组约束条件如何导致效用函数的存在。这个效用函数可以从一系列偏好查询中推断出来。然后我们引入最大期望效用原则作为理性的定义，这是*decision theory*中的一个核心概念，将在本书中作为决策的驱动原则使用。[3] 我们展示

决策问题如何表示为决策网络，并展示求解最优决策的算法。引入了信息价值的概念，它衡量通过观察额外变量所获得的效用。本章最后简要讨论了人类决策并不总是与最大期望效用原则一致。

1 简单的决策与序列问题相比是简单的，而序列问题是本书其余部分的重点。不过，简单的决策在求解上未必简单。

2 Schoemaker 对效用理论的发展进行了概述。参见 P. J. H. Schoemaker，“The Expected Utility Model: Its Variants, Purposes, Evidence and Limitations”，*Journal of Economic Literature*，第 20 卷，第 2 期，第 529–563 页，1982 年。Fishburn 综述了该领域。参见 P. C. Fishburn，“Utility Theory”，*Management Science*，第 14 卷，第 5 期，第 335–378 页，1968 年。

3 决策理论领域的综述由M. Peterson提供，*An Introduction to Decision Theory*。剑桥大学出版社，2009年。

6.1 Constraints on Rational Preferences

我们在第2章开始讨论不确定性时，确定了比较不同陈述中我们信念程度的必要性。本章要求具备比较两种不同结果期望程度的能力。我们使用以下运算符来表达我们的偏好：

- $A \succ B$ 如果我们更偏好A而非B。
- $A \sim B$ 如果我们在A和B之间无差异。
- $A \succeq B$ 如果我们更偏好A胜过B或无差异。

正如信念可以是主观的，偏好也可以是主观的。

除了比较事件外，我们的偏好操作符还可用于比较不确定结果下的偏好。一个*lottery*是与一组结果相关的一组概率。例如，如果$S_{1:n}$是一组结果，而$p_{1:n}$是它们相应的概率，则涉及这些结果和概率的彩票表示为

$$[S_1 : p_1;\ \ldots;\ S_n : p_n] \tag{6.1}$$

实值效用度量的存在性源于对偏好的一组假设。[4] 从这个效用函数出发，可以定义它

意味着在不确定性下做出理性决策。正如我们对信念施加了一组约束，我们也将对偏好施加一些约束：[5]

- *Completeness.* 以下恰好一项成立：$A \succ B$、$B \succ A$ 或 $A \sim B$。
- *Transitivity.* 如果 $A \succeq B$ 和 $B \succeq C$，则 $A \succeq C$。
- *Continuity.* 如果 $A \succeq C \succeq B$，则存在一个概率 p，使得 $[A : p; B : 1 - p] \sim C$。
- *Independence.* 如果 $A \succ B$，则对于任意 C 和概率 p，$[A : p; C : 1 - p] \succeq [B : p; C : 1 - p]$。

这些是对*rational preferences*的约束。它们并未涉及实际人类的偏好；事实上，有强有力的证据表明人类并不总是理性的（这一点在第6.7节将进一步讨论）。本书的目标是从计算的角度理解理性决策，以便我们能够构建有用的系统。将该理论扩展至理解人类决策的可能性，仅属于次要兴趣。

4 期望效用理论由瑞士数学家和物理学家丹尼尔·伯努利（1700–1782）于1738年提出。参见 D. Bernoulli，“关于风险度量新理论的阐述”，*Econometrica*，第22卷，第1期，第23–36页，1954年。

5 这些约束有时被称为 *von Neumann–Morgenstern axioms*，以匈牙利裔美国数学家和物理学家约翰·冯·诺依曼（1903–1957）和奥地利裔美国经济学家奥斯卡·摩根斯坦（1902–1977）命名。他们提出了这些公理的一个变体。参见 J. von Neumann和O. Morgenstern, *Theory of Games and Economic Behavior*。普林斯顿大学出版社，1944年。P. Anand讨论了这些公理的批评，见“偏好公理真的理性吗？”*Theory and Decision*，第23卷，第2期，第189–214页，1987年。

6.2 Utility Functions

正如对不同陈述合理性比较的约束导致实值概率测度的存在，对理性偏好的约束也导致实值 *utility* 测度的存在。从我们对理性偏好的约束可以得出，存在一个实值效用函数 U，使得

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

- $U(A) > U(B)$ 当且仅当$A \succ B$，且
- $U(A) = U(B)$ 当且仅当 $A \sim B$。

效用函数在 *positive affine transformation* 上是唯一的。换句话说，对于任意常数 $m > 0$ 和 b，$U'(S) = mU(S) + b$ 当且仅当由 U' 诱导的偏好与 U 相同。效用就像温度：你可以用开尔文、摄氏度或华氏度来比较温度，它们彼此之间都是仿射变换。

根据理性偏好的约束条件，彩票的效用由下式给出：

$$U([S_1 : p_1;\ \ldots;\ S_n : p_n]) = \sum_{i=1}^{n} p_i U(S_i) \tag{6.2}$$

例6.1应用此方程来计算涉及碰撞避免系统的结果的效用。

示例 6.1. 一个涉及碰撞避免系统结果的彩票。

假设我们正在构建一个防撞系统。飞机相遇的结果由系统是否发出警报（A）以及是否发生碰撞（C）决定。由于A和C是二元的，因此有四种可能的结果。只要我们的偏好是理性的，我们就可以用四个参数来表示可能彩票空间上的效用函数：$U(a^0, c^0)$、$U(a^1, c^0)$、$U(a^0, c^1)$，以及$U(a^1, c^1)$。例如，

$$U([a^0, c^0 : 0.5;\quad a^1, c^0 : 0.3;\quad a^0, c^1 : 0.1;\quad a^1, c^1 : 0.1])$$

等于

$$0.5U(a^0, c^0) + 0.3U(a^1, c^0) + 0.1U(a^0, c^1) + 0.1U(a^1, c^1)$$

如果效用函数是有界的，那么我们可以定义一个*normalized utility function*，其中最佳可能结果的效用被赋值为1，而最差可能结果的效用被赋值为0。其他每一个结果的效用则根据需要进行缩放和平移。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

6.3 Utility Elicitation

在构建决策或决策支持系统时，从人类或一群人类中推断效用函数通常是有帮助的。这种方法被称为*utility elicitation*或*preference elicitation*。[6]一种实现这一目标的方式是将最差结果 $\underline{S}$ 的效用固定为0，最佳结果 S 的效用固定为1。只要结果的效用是有界的，我们就可以在不改变偏好的情况下平移和缩放效用。如果我们想要确定结果 S 的效用，那么我们确定概率 p，使得 $S \sim [S : p; \underline{S} : 1 - p]$。由此可得 $U(S) = p$。例6.2应用此过程来确定与避碰问题相关的效用函数。

6 P. H. Farquhar 在“效用评估方法”中综述了多种效用 elicitation 方法，*Management Science*，第 30 卷，第 11 期，第 1283–1300 页，1984 年。

在我们的避碰示例中，最理想的情况是不发出警报且不发生碰撞，因此我们设定 $U(a^0、c^0) =$ 为1。最糟糕的情况是发出警报并发生碰撞，所以我们设定 $U(a^1、c^1) =$ 为0。我们定义彩票 $L(p)$ 为 $[a^0, c^0 : p; a^1, c^1 : 1 - p]$。为了确定 $U(a^1、c^0)$，我们必须找到满足 $(a^1, c^0) \sim L(p)$ 的 p。同样地，为了确定 $U(a^0、c^1)$，我们要找到满足 $(a^0, c^1) \sim L(p)$ 的 p。

示例 6.2. 应用于避免碰撞的效用引出。

人们可能会倾向于使用货币价值来推断效用函数。例如，如果我们正在构建一个用于管理野火的决策支持系统，可能会倾向于根据财产损失所导致的货币成本和部署灭火资源的货币成本来定义效用函数。然而，经济学中众所周知，财富的效用通常并不是线性的。[7] 如果效用与财富之间存在线性关系，那么决策应当以最大化期望货币价值为准则。试图最大化期望货币价值的人不会购买保险，因为保险单的期望货币价值通常为负值。

7 H. Markowitz，“财富的效用”，*Journal of Political Econ-omy*，第60卷，第2期，第151-158页，1952年。

与其试图最大化预期财富，我们通常希望最大化财富的预期效用。当然，不同的人有不同的效用函数。图6.1展示了一个效用函数的例子。对于少量财富，曲线大致呈线性，其中100美元大约是50美元的两倍好。然而，对于较大量的财富，曲线趋于平缓；毕竟，

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

1000美元对亿万富翁的价值低于对普通人的价值。曲线的这种平缓化有时被称为*diminishing marginal utility*。

在讨论货币效用函数时，这里列出的三个术语经常被使用。为了说明这一点，假设 A 表示获得50美元，B 表示有50%的机会赢得100美元。

- *Risk neutral*. 效用函数是线性的。在50美元和50%概率赢得100美元（$A \sim B$）之间没有偏好。
- *Risk seeking*. 效用函数是凸的。偏好于有50%的机会赢得100美元（$A \prec B$）。
- *Risk averse*. 效用函数是凹的。对于$50 ($A \succ B$)存在偏好。

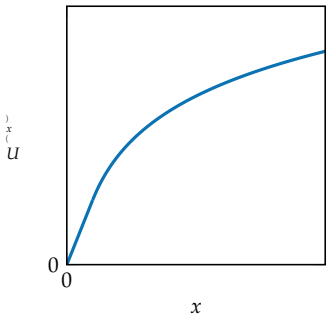


图6.1。财富的效用x通常被建模为在小值时呈线性，而在较大值时呈凹性，表现出风险规避。

有几种常见的函数形式可用于建模标量数量的风险厌恶，[8]如财富或医院床位的可用性。一种是*quadratic utility*：

$$U(x) = \lambda x - x^2 \tag{6.3}$$

其中参数 $\lambda > 0$ 控制风险厌恶程度。由于在建模诸如财富等数量的效用时，我们通常希望该效用函数单调递增，因此我们会将此函数上限设为 $x = \lambda/2$。过了这个点之后，效用开始下降。另一个简单的形式是 *exponential utility*：

$$U(x) = 1 - e^{-\lambda x} \tag{6.4}$$

其中 $\lambda > 0$。尽管它具有方便的数学形式，但通常不被视为财富效用的合理模型。另一种选择是 *power utility*：

$$U(x) = \frac{x^{1-\lambda} - 1}{1 - \lambda} \tag{6.5}$$

其中 $\lambda \geq$ 为 0，$\lambda \neq$ 为 1。*logarithmic utility*

$$U(x) = \log x \tag{6.6}$$

当 $x >$ 为 0 时，可以看作是幂效用函数中 $\lambda \to$ 为 1 的特殊情况。图 6.2 展示了幂效用函数的图像，并将对数效用函数作为其特殊情况之一。

8 这些函数形式在经济学和金融学中已得到充分研究。J. E. Ingersoll, *Theory of Fi-nancial Decision Making*. Rowman and Littlefield Publishers, 1987.

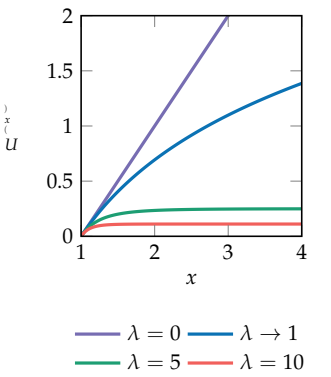


图6.2. 幂效用函数。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

6.4 Maximum Expected Utility Principle

我们关注的是在世界状态知识不完美的情况下做出理性决策的问题。假设我们有一个概率模型 $P(s' \mid o、a)$，它表示在观察到 o 并采取行动 a 的情况下，世界状态变为 s' 的概率。我们有一个效用函数 $U(s')$，它编码了我们对结果空间的偏好。在给定观察 o 的情况下，采取行动 a 的 *expected utility* 由下式给出

$$EU(a \mid o) = \sum_{s'} P(s' \mid a, o) U(s') \tag{6.7}$$

*principle of maximum expected utility*说，理性主体应选择最大化期望效用的行动：

$$a^* = \arg\max_a EU(a \mid o) \tag{6.8}$$

因为我们感兴趣的是构建理性智能体，方程（6.8）在本书中起着核心作用。[9] 例6.3将此原则应用于一个简单的决策问题。

9 最大期望效用原理对人工智能领域的重要性由S. Russell和P. Norvig讨论，*Artificial Intelligence: A Modern Approach*，第4版。Pearson，2021年。

6.5 Decision Networks

一个 *decision network*，有时被称为 *influence diagram*，是对贝叶斯网络的泛化，加入了动作节点和效用节点，以便我们可以紧凑地表示定义决策问题的概率和效用模型。[10] 该前一部分中的状态、动作和观测空间可以被分解，决策网络的结构捕捉了各个组件之间的关系。

10 关于决策网络的广泛讨论可在 F. V. Jensen 和 T. D. Nielsen 的 *Bayesian Networks and Decision Graphs*，第2版，Springer，2007 中找到。

决策网络由三种类型的节点组成：

- 一个 *chance node* 对应于一个随机变量（由圆圈表示）。
- 一个 *action node* 对应于一个决策变量（由正方形表示）。
- 一个 *utility node* 对应于一个效用变量（由菱形表示），且不能有子节点。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

例6.3。将最大期望效用原则应用于是否带伞的简单决策。

假设我们正根据目的地的天气预报来决定度假时是否带伞。我们观察到预报 o，它可能是雨或晴。我们的行动 a 要么是带伞，要么是不带伞。由此产生的状态 s' 是是否带伞与目的地是晴天还是雨天的组合。我们的概率模型如下：

o	a	s'	$P(s' \mid a, o)$
forecast rain	bring umbrella	rain with umbrella	0.9
forecast rain	leave umbrella	rain without umbrella	0.9
forecast rain	bring umbrella	sun with umbrella	0.1
forecast rain	leave umbrella	sun without umbrella	0.1
forecast sun	bring umbrella	rain with umbrella	0.2
forecast sun	leave umbrella	rain without umbrella	0.2
forecast sun	bring umbrella	sun with umbrella	0.8
forecast sun	leave umbrella	sun without umbrella	0.8

如表所示，我们假设预测并不完美；降雨预测准确率为90%，晴天预测准确率为80%。此外，我们假设带伞不会影响天气，尽管有人可能对此假设提出质疑。效用函数如下：

s'	$U(s')$
rain with umbrella	-0.1
rain without umbrella	-1
sun with umbrella	0.9
sun without umbrella	1

我们可以使用公式(6.7)计算如果我们预报下雨时带伞的预期效用：

$$EU(\text{bring umbrella} \mid \text{forecast rain}) = 0.9 \times -0.1 + 0.1 \times 0.9 = 0$$

同样地，如果我们利用方程(6.7)预测到下雨，我们可以计算不带伞的预期效用：

$$EU(\text{leave umbrella} \mid \text{forecast rain}) = 0.9 \times -1 + 0.1 \times 1 = -0.8$$

因此，我们将需要带上雨伞。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

有三种有向边：

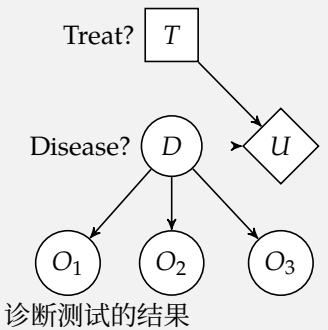
- 一个*conditional edge*以机会节点结束，表示该机会节点的不确定性取决于其所有父节点的值。
- 一个 *informational edge* 以动作节点结束，表明与该节点相关的决策是在了解其父节点值的情况下做出的。（这些边通常用虚线绘制，有时为了简化会从图中省略。）
- 一个 *functional edge* 终止于效用节点，并表明该效用节点由其父节点的结果决定。

与贝叶斯网络一样，决策网络也不能有环。与一个动作相关联的效用等于所有效用节点上的值之和。例6.4说明了决策网络如何在给定诊断测试结果的情况下，对是否治疗某种疾病的问题进行建模。

例6.4. 一个决策网络示例，用于根据诊断测试的信息模拟是否治疗某种疾病。

我们有一组来自诊断测试的结果，这些结果可能表明某种特定疾病的存在。鉴于对测试的了解，我们需要决定是否采取治疗措施。效用是是否采取治疗以及疾病是否实际存在的函数。条件边将D连接到O_1、O_2和O_3。信息边在图中未明确显示，但它们会将观测结果连接到T。功能边将T和D连接到U。

T	D	$U(T,D)$
0	0	0
0	1	−10
1	0	−1
1	1	−1



求解一个简单问题（算法6.1）需要遍历所有可能的决策实例，以找到使期望效用最大化的决策。对于每一个

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

在实例化时，我们计算相关的期望效用。我们首先实例化动作节点和观察到的机会节点。然后，我们可以应用任何推理算法来计算效用函数输入的后验分布。期望效用是效用节点上值的总和。例6.5展示了如何将这一过程应用于我们的持续示例。

```
struct SimpleProblem
    bn::BayesianNetwork
    chance_vars::Vector{Variable}
    decision_vars::Vector{Variable}
    utility_vars::Vector{Variable}
    utilities::Dict{Symbol, Vector{Float64}}
end

function solve(𝒫::SimpleProblem, evidence, M)
    query = [var.name for var in 𝒫.utility_vars]
    U(a) = sum(𝒫.utilities[uname][a[uname]] for uname in query)
    best = (a=nothing, u=-Inf)
    for assignment in assignments(𝒫.decision_vars)
        evidence = merge(evidence, assignment)
        ϕ = infer(M, 𝒫.bn, query, evidence)
        u = sum(p*U(a) for (a, p) in ϕ.table)
        if u > best.u
            best = (a=assignment, u=u)
        end
    end
    return best
end
```

算法6.1. 决策网络中的简单问题。决策网络是一种贝叶斯网络，包含机会变量、决策变量和效用变量。效用变量被视为确定性变量。由于我们的贝叶斯网络中的变量取值来自1：r_i，效用变量通过`utilities`字段映射到实数值。例如，如果我们有一个效用变量`:u1`，与该变量关联的第`i`个效用是`utilities[:u1][i]`。`solve`函数以问题、证据和推理方法作为输入，返回对决策变量的最佳赋值及其相关的期望效用。

已经开发了多种方法来提高评估决策网络的效率。[11] 其中一种方法涉及移除动作节点和机会节点

search

如果决策网络中的节点没有子节点，根据条件边、信息边或功能边的定义，我们可以将其移除。在例6.5中，我们可以移除O_2和O_3，因为它们没有子节点。但是，我们不能移除O_1，因为我们将其视为已观察到的，这表明存在一条从O_1到T（的信息边，尽管这条边没有明确画出）。

6.6 Value of Information

我们基于所观察到的情况做出决策。在许多应用中，自然而然地想要量化*value of information*，即观察额外变量预期能增加我们效用的程度。[12] 例如，在疾病治疗中

11 R. D. Shachter，“评估影响图”，*Operations Re-*，第34卷，第6期，第871-882页，1986年。R. D. Shachter，“概率推断与影响图”，*Operations Research*，第36卷，第4期，第589-604页，1988年。

12 R. A. Howard，“信息价值理论”，*IEEE Transactions on Systems Science and Cybernetics*，第2卷，第1期，第22-26页，1966年。关于决策网络的应用可见：S. L. Dittmer和F. V. Jensen，“影响图中的短视信息价值”，载于*Conference on Uncertainty in Artificial Intelligence (UAI)*，1997年。R. D. Shachter，“信息价值的高效计算”，载于*Conference on Uncertainty in Artificial Intelligence (UAI)*，1999年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们可以使用公式 (6.7) 来计算在示例 6.4 的决策网络中治疗某种疾病的期望效用。目前，我们假设仅获得第一次诊断测试的结果，并且结果为阳性。如果我们想在图中明确表示第一次诊断测试的信息，那么我们会从 O_1 到 T 画一条信息边，并且我们会有

$$EU(t^1 \mid o_1^1) = \sum_{o_3} \sum_{o_2} \sum_{d} P(d, o_2, o_3 \mid t^1, o_1^1) U(t^1, d, o_1^1, o_2, o_3)$$

我们可以使用贝叶斯网络的链式法则和条件概率的定义来计算 $P(d、o_2、o_3 \mid t^1、o_1^1)$。由于效用节点仅取决于疾病是否存在以及我们是否对其进行治疗，我们可以将 $U(t^1、d、o_1^1、o_2、o_3)$ 简化为 $U(t^1、d)$。因此，

$$EU(t^1 \mid o_1^1) = \sum_{d} P(d \mid t^1, o_1^1) U(t^1, d)$$

前一章介绍的任何精确或近似推断方法均可用于评估 $P(d \mid t^1、o_1^1)$。为了决定是否应用某种处理，我们计算 $EU(t^1 \mid o_1^1)$ 和 $EU(t^0 \mid o_1^1)$，并做出能够提供最高期望效用的决策。

示例 6.5. 诊断测试问题的决策网络评估。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

在示例6.5的应用中，我们假设仅观测到了o_1^1。仅凭这一项诊断测试的阳性结果，我们或许会决定不进行治疗。然而，进行额外的诊断测试可能有益，以减少未治疗实际存在的疾病的风险。

在计算信息的价值时，我们将使用$EU^*(o)$来表示在给定观察o的情况下，最优行动的期望效用。关于变量O'的信息价值，在给定o的情况下，是

$$VOI(O' \mid o) = \left(\sum_{o'} P(o' \mid o) EU^*(o, o') \right) - EU^*(o) \tag{6.9}$$

换句话说，关于某个变量的信息价值在于当该变量被观测到时，期望效用的增加量。算法6.2提供了这一方法的具体实现。

```
function value_of_information(𝒫, query, evidence, M)
    ϕ = infer(M, 𝒫.bn, query, evidence)
    voi = -solve(𝒫, evidence, M).u
    query_vars = filter(v->v.name ∈ query, 𝒫.chance_vars)
    for o′ in assignments(query_vars)
        oo′ = merge(evidence, o′)
        p = ϕ.table[o′]
        voi += p*solve(𝒫, oo′, M).u
    end
    return voi
end
```

算法6.2。一种计算查询`query`的信息价值的方法，给定观察到的机会变量及其值`evidence`。该方法还接受一个简单问题$\mathcal{P}$和一个推理策略`M`。

信息的价值永远不会为负。只有当额外的观察可能导致不同的最优决策时，期望效用才可能增加。如果观察一个新变量 O' 对行动的选择没有影响，那么 $EU^*(o、o') = EU^*(o)$ 对于所有 o'，在这种情况下，方程 (6.9) 的值为0。例如，如果最优决策是不论 *diagnostic test* 的结果如何都治疗该疾病，那么观察测试结果的价值为0。

信息的价值仅体现在通过观察所增加的期望效用。进行特定观察可能伴随成本。某些诊断测试可能成本低廉，如测量体温；而其他诊断测试则更为昂贵且侵入性更强，比如腰椎穿刺。腰椎穿刺所获得的信息价值可能远高于体温测量，但测试的成本也应纳入考虑。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

信息价值是选择观察对象时一个重要且常用的指标。有时，信息价值指标被用来确定合适的观察序列。每次观察之后，都会为剩余的未观察变量确定信息价值。然后，选择信息价值最大的未观察变量进行观察。如果进行不同观察会产生成本，那么在确定观察哪个变量时，会从信息价值中减去这些成本。这个过程会持续进行，直到观察更多变量不再有益为止。随后选择最优行动。这种贪心式的观察选择只是一种启发式方法；它可能并不代表真正最优的观察序列。观察的最优选择可以通过使用后续章节中介绍的序贯决策技术来确定。

6.7 Irrationality

决策理论是一个*normative theory*，它是规定性的，而不是*descriptive theory*，后者是对人类行为的预测。人类判断和偏好往往不遵循第6.1节所概述的理性规则。[13] 即使是人类专家

可能具有不一致的偏好集，这在设计试图最大化期望效用的决策支持系统时可能会成为问题。

例6.6表明，确定性往往夸大了相对于仅是可能的损失而言确定的损失。这一*certainty effect*同样适用于收益情况。较小的确定收益往往优于大得多的但仅有可能的收益，这种情况下，理性公理必然被违背。

例 6.7 展示了 *framing effect*，即人们根据选项是被呈现为损失还是收益来做出决策。许多其他认知偏差也可能导致偏离效用理论所规定的行为。[14] 特别

在试图从人类专家那里获取效用函数以构建决策支持系统时，必须谨慎行事。尽管决策支持系统的建议可能是理性的，但在某些情况下，它们可能不完全反映人类的偏好。

13 Kahneman和Tversky对期望效用理论进行了批判，并引入了一种称为*prospect theory*的替代模型，该模型似乎更符合人类行为。D. Kahneman和A. Tversky，“前景理论：风险决策分析”，*Econometrica*，第47卷，第2期，第263-292页，1979年。

14 最近有几本书讨论了明显的人类非理性行为。D. Ariely, *Predictably Irrational: The Hidden Forces That Shape Our Decisions*。Harper，2008年。J. Lehrer, *How We Decide*。Houghton Mifflin，2009年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

例6.6。一项实验表明，确定性往往会夸大相对于仅仅是可能的损失而言的确定性损失。A. Tversky和D. Kahneman，“决策框架与选择心理学”，*Sci-*，第211卷，第4481期，第453-458页，1981年。

Tversky和Kahneman研究了在大学课堂环境中回答问卷的大学生的偏好。他们向学生提出了应对流行病的问题。学生需要在以下两种结果之间透露他们的偏好：

ence

- A: 100% 损失 75 条生命的几率
- B：80 % 的概率失去100条生命

多数情况下优先选择B而非A。由公式(6.2)可知

$$U(\text{lose } 75) < 0.8U(\text{lose } 100) \tag{6.10}$$

随后，他们被要求在以下两种结果中做出选择：

- C: 10% 的几率损失 75 条生命
- D：8% 的几率失去 100 条生命

大多数人更偏好 C 而非 D。因此，$0.1U(\text{失去}75) > 0.08U(\text{失去}100)$。我们将两边乘以10，得到

$$U(\text{lose } 75) > 0.8U(\text{lose } 100) \tag{6.11}$$

当然，方程(6.10)和(6.11)导致了矛盾。我们并未对$U(\text{失去}75)$和$U(\text{失去}100)$的实际价值做出任何假设——我们甚至没有假定失去100条生命比失去75条生命更糟糕。由于方程(6.2)直接源自6.1节中给出的冯·诺依曼-摩根斯坦公理，因此必然至少违反了其中一条公理，尽管许多选择B和C的人似乎认为这些公理是可接受的。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Tversky和Kahneman通过一个假设情景展示了*framing effect*，其中预期一种流行病会导致600人死亡。他们向学生展示了以下两种结果：

- E：200人将获救。
- F：1/3 的概率将拯救 600 人，2/3 的概率无人得救。

大多数学生在E和F之间选择了E。然后，他们要求他们在以下选项之间做出选择：

- G：400人会死亡。
- H：1/3 的概率没有人会死，2/3 的概率600人会死。

大多数学生选择了H而不是G，尽管E等同于G，且F等同于H。这种不一致性是由于问题的表述方式造成的。

例6.7. 一项展示框架效应的实验。A. Tversky和D. Kahneman，“决策的框架与选择的心理学”，*Science*，第211卷，第4481期，第453-458页，1981年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

6.8 Summary

- 理性决策结合了概率论和效用理论。
- 效用函数的存在性源自对理性偏好的约束。

- 理性的决策是最大化期望效用的决策。
- 决策问题可以使用决策网络来建模，决策网络是贝叶斯网络的扩展，包含了动作和效用。
- 求解一个简单的决策涉及贝叶斯网络中的推断，因此是NP难的。

- 信息价值衡量的是当观测到一个新变量时，期望效用所获得的增益。

- 人类并不总是理性的。

6.9 Exercises

练习6.1。假设我们有一个效用函数$U(s)$，具有有限的最大值U和有限的最小值$\underline{U}$。保持相同偏好的相应规范化效用函数$\hat{U}(s)$是什么?

Solution: 归一化效用函数的最大值为1，最小值为0。偏好关系在仿射变换下保持不变，因此我们确定$U(s)$的仿射变换，使其匹配单位边界。此变换为

$$\hat{U}(s) = \frac{U(s) - \underline{U}}{\overline{U} - \underline{U}} = \frac{1}{\overline{U} - \underline{U}} U(s) - \frac{\underline{U}}{\overline{U} - \underline{U}}$$

练习6.2。如果$A \succeq C \succeq B$且每个结果的效用分别是$U(A) = 450$，$U(B) = -150$，和$U(C) = 60$，那么对于A和B的彩票，什么情况下会使我们在该彩票与C之间无差异?

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: 定义在 A 和 B 上的彩票为 $[A : p; B : 1 - p]$。要使该彩票与 C ($[A : p; B : 1 - p] \sim C$) 之间满足无差异，我们必须有 $U([A : p; B : 1 - p]) = U(C)$。因此，我们必须计算满足等式的 p

$$\begin{aligned} U([A:p;B:1-p]) &= U(C) \\ pU(A) + (1-p)U(B) &= U(C) \\ p &= \frac{U(C) - U(B)}{U(A) - U(B)} \\ p &= \frac{60 - (-150)}{450 - (-150)} = 0.35 \end{aligned}$$

这意味着彩票 $[A : 0.35; B : 0.65]$ 与 C 一样令人向往。

练习 6.3。假设对于三个结果 A、B 和 C 上的效用函数 U，有 $U(A) = 5$、$U(B) = 20$ 和 $U(C) = 0$。我们面临一个选择：一个是以 50% 的概率得到 B、以 50% 的概率得到 C 的彩票，另一个是保证得到 A 的彩票。计算偏好的彩票，并证明在 $m = 2$ 和 $b = 30$ 的正仿射变换下，我们仍然偏好同一彩票。

Solution: 第一个彩票由 $[A : 0.0; B : 0.5; C : 0.5]$ 给出，第二个彩票由 $[A : 1.0; B : 0.0; C : 0.0]$ 给出。每个彩票的原始效用由下式给出：

$$\begin{aligned} U([A:0.0;B:0.5;C:0.5]) &= 0.0U(A) + 0.5U(B) + 0.5U(C) = 10 \\ U([A:1.0;B:0.0;C:0.0]) &= 1.0U(A) + 0.0U(B) + 0.0U(C) = 5 \end{aligned}$$

因此，由于 $U([A : 0.0; B : 0.5; C : 0.5]) > U([A : 1.0; B : 0.0; C : 0.0])$，我们更偏好第一个彩票。在正仿射变换 $m = 2$ 和 $b = 30$ 下，我们的新效用可以计算为 $U' = 2U + 30$。那么新的效用分别是 $U'(A) = 40$、$U'(B) = 70$ 和 $U'(C) = 30$。每个彩票的新效用是

$$\begin{aligned} U'([A:0.0;B:0.5;C:0.5]) &= 0.0U'(A) + 0.5U'(B) + 0.5U'(C) = 50 \\ U'([A:1.0;B:0.0;C:0.0]) &= 1.0U'(A) + 0.0U'(B) + 0.0U'(C) = 40 \end{aligned}$$

由于 $U'([A : 0.0; B : 0.5; C : 0.5]) > U'([A : 1.0; B : 0.0; C : 0.0])$，我们保持对第一个彩票的偏好。

练习 6.4. 证明等式 (6.5) 中的幂效用函数对于所有 $x > 0$ 和 $\lambda > 0$ 且 $\lambda \neq 1$ 的情况都是风险规避的。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: 风险厌恶意味着效用函数是凹的，这要求效用函数的二阶导数为负。效用函数及其导数计算如下：

$$U(x) = \frac{x^{1-\lambda} - 1}{1-\lambda}$$
$$\frac{dU}{dx} = \frac{1}{x^{\lambda}}$$
$$\frac{d^2U}{dx^2} = \frac{-\lambda}{x^{\lambda+1}}$$

对于 $x > 0$ 和 $\lambda > 0$，$\lambda \neq 1$，$x^{\lambda+1}$ 是一个正数提升到正指数，保证为正。将其乘以 $-\lambda$ 保证二阶导数为负。因此，对于所有 $x > 0$ 和 $\lambda > 0$，$\lambda \neq 1$，幂效用函数是风险厌恶的。

练习 6.5。使用例 6.3 中给出的参数，计算如果我们预报晴天，带伞的期望效用，以及如果我们预报晴天，不带伞的期望效用。给定我们预报晴天，最大化我们期望效用的行动是什么？

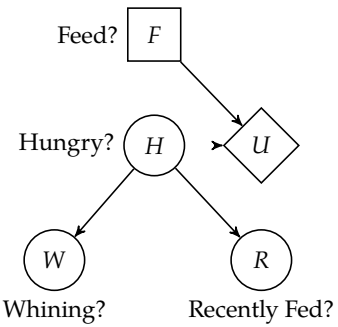
Solution:

$$EU(\text{bring umbrella} \mid \text{forecast sun}) = 0.2 \times -0.1 + 0.8 \times 0.9 = 0.7$$
$$EU(\text{leave umbrella} \mid \text{forecast sun}) = 0.2 \times -1.0 + 0.8 \times 1.0 = 0.6$$

如果预报有雨，最大化我们期望效用的行动是带伞！

练习 6.6。假设我们试图根据小狗饥饿的可能性（H）来最优地决定是否喂食（F）我们的新小狗。我们可以观察小狗是否在哀鸣（W）以及是否最近有其他人喂过小狗（R）。喂食与饥饿的每种组合的效用以及决策网络表示如下：

F	H	$U(F,H)$
0	0	0.0
0	1	−1.0
1	0	−0.5
1	1	−0.1



© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

给定 $P(h^1 \mid w^1) = 0.78$，如果我们观察到小狗呜咽（w^1），那么不喂小狗（f^0）和喂小狗（f^1）的期望效用分别是多少？最优行动是什么？

Solution: 我们首先定义期望效用，并认识到效用仅取决于H和F：

$$EU(f^0 \mid w^1) = \sum_h P(h \mid w^1) U(f^0, h)$$

现在，我们可以计算在幼犬呜咽的情况下喂食幼犬的预期效用，并采用与之前类似的方法，计算在幼犬呜咽的情况下不喂食幼犬的预期效用：

$$EU(f^0 \mid w^1) = 0.22 \times 0.0 + 0.78 \times -1.0 = -0.78$$
$$EU(f^1 \mid w^1) = 0.22 \times -0.5 + 0.78 \times -0.1 = -0.188$$

因此，最佳行动是喂小狗（f^1），因为这最大化我们的预期效用$EU^*(w^1) = -0.188$。

练习 6.7。利用练习 6.6 的结果，如果 $P(r^1 \mid w^1) = 0.2$，$P(h^1 \mid w^1、r^0) = 0.9$，且 $P(h^1 \mid w^1、r^1) = 0.3$，那么在观察到小狗在呜咽（$w^1$）的情况下，询问别人小狗最近是否被喂过的信息价值是多少？

Solution: 我们感兴趣的是计算

$$\text{VOI}(R \mid w^1) = \left(\sum_r P(r \mid w^1) EU^*(w^1, r) \right) - EU^*(w^1)$$

我们首先计算所有 f 和 r 对应的 $EU(f \mid w^1、r)$。按照与练习6.6类似的推导，我们有

$$EU(f^0 \mid w^1, r^0) = \sum_h P(h \mid w^1, r^0) U(f^0, h)$$

因此，对于每个 F 和 R 的组合，我们有以下期望效用：

$$EU(f^0 \mid w^1, r^0) = \sum_h P(h \mid w^1, r^0) U(f^0, h) = 0.1 \times 0.0 + 0.9 \times -1.0 = -0.9$$
$$EU(f^1 \mid w^1, r^0) = \sum_h P(h \mid w^1, r^0) U(f^1, h) = 0.1 \times -0.5 + 0.9 \times -0.1 = -0.14$$
$$EU(f^0 \mid w^1, r^1) = \sum_h P(h \mid w^1, r^1) U(f^0, h) = 0.7 \times 0.0 + 0.3 \times -1.0 = -0.3$$
$$EU(f^1 \mid w^1, r^1) = \sum_h P(h \mid w^1, r^1) U(f^1, h) = 0.7 \times -0.5 + 0.3 \times -0.1 = -0.38$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

最优期望效用为

$$EU^*(w^1, r^0) = -0.14$$
$$EU^*(w^1, r^1) = -0.3$$

现在，我们可以计算信息的价值：

$$\text{VOI}(R \mid w^1) = 0.8(-0.14) + 0.2(-0.3) - (-0.188) = 0.016$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

第二部分

序列问题

到目前为止，我们假设是在某一时刻做出单一决策，但许多重要问题要求我们做出一系列决策。最大期望效用的相同原则仍然适用，但在序列情境下进行最优决策需要对未来动作和观察序列进行推理。本书的这一部分将讨论随机环境中的序列决策问题。我们将重点关注在假定模型已知且环境完全可观测的情况下，序列决策问题的一般表述。稍后我们将放宽这两个假设。我们的讨论将从引入 *Markov decision process* (*MDP*) 开始，这是序列决策问题的标准数学模型。我们将讨论若干寻求精确解的方法。由于大型问题有时无法高效地求得精确解，我们将讨论一组离线和在线近似求解方法，以及一类涉及直接搜索参数化决策策略空间的方法。最后，我们将讨论用于验证我们的决策策略在部署到现实世界时能否按预期表现的方法。

7 Exact Solution Methods

本章介绍一个称为 *Markov decision process* (*MDP*) 的模型，用于表示我们行动的效果不确定的序贯决策问题。[1] 我们首先描述该模型，它同时指定了随机系统的动力学以及与其演化相关的效用。可以使用不同的算法来计算与决策策略相关的效用，并搜索最优策略。在某些假设下，我们可以找到MDP的精确解。后续章节将讨论往往能更好地扩展到更大问题的近似方法。

1 这类模型最初是在20世纪50年代研究的。R. E. Bellman, *Dynamic Programming*。普林斯顿大学出版社，1957年。现代处理方法可见于M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*。Wiley，2005年。

7.1 Markov Decision Processes

在MDP（算法7.1）中，我们根据观察到的状态s_t，在时间t选择动作a_t。随后，我们获得奖励r_t。其中，*action space* $\mathcal{A}$是可执行动作的集合，而*state space* $\mathcal{S}$是可能状态的集合。部分算法假定这些集合是有限的，但一般情况下并非必需。状态依据当前状态及我们所采取的动作以概率方式演化。假设下一状态仅依赖于当前状态和动作，而不依赖于任何先前状态或动作，这一假设被称为*Markov assumption*。

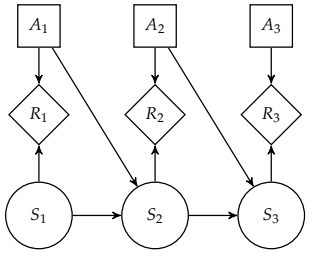


图7.1. MDP决策网络图。

MDP 可以使用如图 7.1 所示的决策网络来表示。从 $A_{1:t-1}$ 和 $S_{1:t}$ 到 A_t 存在信息边（此处未显示）。效用函数分解为奖励 $R_{1:t}$。我们重点关注 *stationary* MDP，其中 $P(S_{t+1} \mid S_t, A_t)$ 和 $P(R_t \mid S_t, A_t)$ 不随时间变化。平稳 MDP 可以通过如图 7.2 所示的动态决策图紧凑地表示。*state transition model* $T(s' \mid s, a)$ 表示在执行动作 a 后从状态 s 转移到 s' 的概率。*reward function* $R(s, a)$ 表示从状态 s 执行动作 a 时获得的期望奖励。

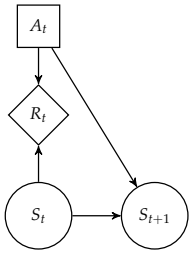


图7.2. 固定MDP决策网络图。所有MDP都具有这一通用结构。

奖励函数是s和a的确定性函数，因为它表示一个期望值，但奖励在环境中可能是随机生成的，甚至可能依赖于产生的下一状态。[2] 例7.1展示了如何构建一个将碰撞避免问题视为MDP。

2 例如，如果奖励根据 $R(s、a、s')$ 给出的下一状态来决定，那么预期奖励函数将是

$$R(s,a) = \sum_{s'} T(s' \mid s,a) R(s,a,s')$$

飞机碰撞避免问题可以形式化为一个MDP。状态表示我方飞机和入侵飞机的位置与速度，动作表示我们是爬升、下降还是保持水平。与其他飞机相撞会获得一个大的负奖励，而爬升或下降则会获得一个小的负奖励。

nal of the Operational Research Society

给定当前状态的知识，我们必须决定是否需要进行避让机动。这个问题的挑战在于飞机的位置以概率方式演变，我们既要确保尽早开始机动以避免碰撞，又要足够晚以避免不必要的机动。

例7.1. 将飞机防撞问题构建为MDP。D. J. White在“马尔可夫决策过程应用综述”中讨论了众多其他现实世界的应用，*Jour-*，第44卷，第11期，第1073–1096页，1993年。

```
struct MDP
    γ  # discount factor
    𝒮  # state space
    𝒜  # action space
    T  # transition function
    R  # reward function
    TR # sample transition and reward
end
```

算法 7.1. MDP 的数据结构。我们稍后将使用 `TR` 字段，根据当前状态和动作采样下一个状态和奖励：`s′, r = TR(s, a)`。在数学写作中，MDP 有时以元组的形式定义，该元组由 MDP 的各个组成部分组成，写作 $(\mathcal{S}, \mathcal{A}, T, R, \gamma)$。

MDP中的奖励被视为可加分解效用函数的组成部分。在一个具有n个决策的*finite horizon*问题中，与一系列奖励$r_{1:n}$相关的效用简单地是

$$\sum_{t=1}^{n} r_t \tag{7.1}$$

奖励之和有时被称为*return*。

在一个 *infinite horizon* 问题中，若决策数量无界，奖励之和可能变为无穷大。[3] 有几种定义效用的方法

在无限 horizon 问题中，就个体奖励而言。一种方法是施加

3 假设策略 A 每时间步获得1的奖励，而策略 B 每时间步获得100的奖励。直观上，理性的代理人应该更偏好策略 B 而非策略 A，但两者提供的无限预期效用是相同的。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

一个介于0和1之间的*discount factor* γ。效用由下式给出

$$\sum_{t=1}^{\infty} \gamma^{t-1} r_t \tag{7.2}$$

这个值有时被称为 *discounted return*。只要 $0 \leq \gamma < 1$ 且奖励是有限的，效用将是有限的。折扣因子使得现在的奖励比未来的奖励更有价值，这一概念也出现在经济学中。

在无限期问题中定义效用的另一种方法是使用*averagereward*，也称为 *average return*，由下式给出

$$\lim_{n \to \infty} \frac{1}{n} \sum_{t=1}^{n} r_t \tag{7.3}$$

这种表述方式可能具有吸引力，因为我们无需选择折扣因子，但通常这种表述与折扣因子接近1的折现回报之间并无实际差异。由于折现回报在计算上往往更为简便，我们将重点放在折现公式上。

一个 *policy* 告诉我们在给定的过去状态和动作历史下应选择什么动作。在时间 t 给定 *history* $h_t = (s_{1:t}, a_{1:t-1})$ 时，要选择的动作写作 $\pi_t(h_t)$。由于未来状态和奖励仅取决于当前状态和动作（如图 7.1 中的条件独立假设所示），我们可以将注意力限制在仅依赖于当前状态的策略上。此外，我们将主要关注 *deterministic policies*，因为在 MDP 中保证存在一个确定性最优策略。后面的章节将讨论 *stochastic policies*，其中 $\pi_t(a_t \mid s_t)$ 表示策略在时间 t 于状态 s_t 中选择动作 a_t 所赋予的概率。

在具有平稳转移和奖励的无限时域问题中，我们可以进一步将注意力限制在不依赖于时间的 *stationary policies* 上。我们将平稳策略 π 在状态 s 下所关联的动作写作 $\pi(s)$，省略时间下标。然而在有限时域问题中，根据剩余时间步数选择不同的动作可能是有益的。例如，打篮球时，除非比赛只剩下几秒钟，否则尝试半场投篮通常不是好的策略。我们可以通过将时间作为状态变量纳入，使平稳策略考虑时间因素。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

从状态 s 执行 π 的期望效用记为 $U^\pi(s)$。在 MDP 的语境中，U^π 通常被称为 *value function*。一个 *optimal policy* π^* 是一个最大化期望效用的策略：[4]

$$\pi^*(s) = \arg\max_\pi U^\pi(s) \tag{7.4}$$

对于所有状态 s。根据模型的不同，可能存在多个最优策略。与最优策略 π^* 相关的值函数被称为 *optimal value function*，并表示为 U^*。

4 这样做与第6.4节中引入的最大期望效用原则是一致的。

最优策略可以通过使用一种称为*dynamic programming*,[5]的计算技术来找到，该技术涉及通过简化一个复杂问题来
将其以递归方式分解为更简单的子问题。尽管我们将重点关注用于MDP的动态规划算法，但动态规划是一种通用技术，可应用于各种其他问题。例如，动态规划可用于计算斐波那契数列以及寻找两个字符串之间的最长公共子序列。[6]一般来说，使用动态规划求解MDP的算法

5 “动态规划”一词由美国数学家理查德·欧内斯特·贝尔曼（1920–1984）创造。动态指的是问题是时变的，而规划指的是找到最优方案或决策策略的方法论。R. Bellman, *Eye of the Hurricane: An Autobiography*。World Scientific，1984。

比暴力方法高效得多。

6 T. H. Cormen, C. E. Leiserson, R. L. Rivest, 和 C. Stein, *Introduction to Algorithms*, 第3版. MIT出版社, 2009年.

7.2 Policy Evaluation

在我们讨论如何计算最优策略之前，我们将讨论 *policy evaluation*，其中我们计算值函数 U^π。策略评估可以迭代进行。如果策略执行一步，效用为 $U_1^\pi(s) = R(s, \pi(s))$。进一步的步骤可以从 *lookahead* 方程获得：

$$U_{k+1}^\pi(s) = R(s, \pi(s)) + \gamma \sum_{s'} T(s' \mid s, \pi(s)) U_k^\pi(s') \tag{7.5}$$

该方程在算法7.2中实现。迭代策略评估在算法7.3中实现。图7.3展示了若干次迭代。

在给定足够的前瞻方程迭代次数的情况下，价值函数U^π可以被计算到任意精度。收敛性得到保证，因为方程（7.5）中的更新是一个*contraction mapping* (，在附录A.15中进行了回顾)。[7]
收敛时，以下等式成立：

$$U^\pi(s) = R(s, \pi(s)) + \gamma \sum_{s'} T(s' \mid s, \pi(s)) U^\pi(s') \tag{7.6}$$

7 见练习7.12。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
function lookahead(𝒫::MDP, U, s, a)
    𝒮, T, R, γ = 𝒫.𝒮, 𝒫.T, 𝒫.R, 𝒫.γ
    return R(s,a) + γ*sum(T(s,a,s′)*U(s′) for s′ in 𝒮)
end
function lookahead(𝒫::MDP, U::Vector, s, a)
    𝒮, T, R, γ = 𝒫.𝒮, 𝒫.T, 𝒫.R, 𝒫.γ
    return R(s,a) + γ*sum(T(s,a,s′)*U[i] for (i,s′) in enumerate(𝒮))
end
```

算法7.2. 用于根据状态 `s` 和动作 `a`，并利用对MDP $\mathcal{P}$ 的价值函数 `U` 的估计，来计算前瞻状态-动作值的函数。第二个版本处理当 `U` 是向量的情况。

```
function iterative_policy_evaluation(𝒫::MDP, π, k_max)
    𝒮, T, R, γ = 𝒫.𝒮, 𝒫.T, 𝒫.R, 𝒫.γ
    U = [0.0 for s in 𝒮]
    for k in 1:k_max
        U = [lookahead(𝒫, U, s, π(s)) for s in 𝒮]
    end
    return U
end
```

算法7.3. 迭代策略评估，该算法使用`k_max`次迭代，针对具有离散状态和动作空间的MDP $\mathcal{P}$，迭代计算策略π的价值函数。

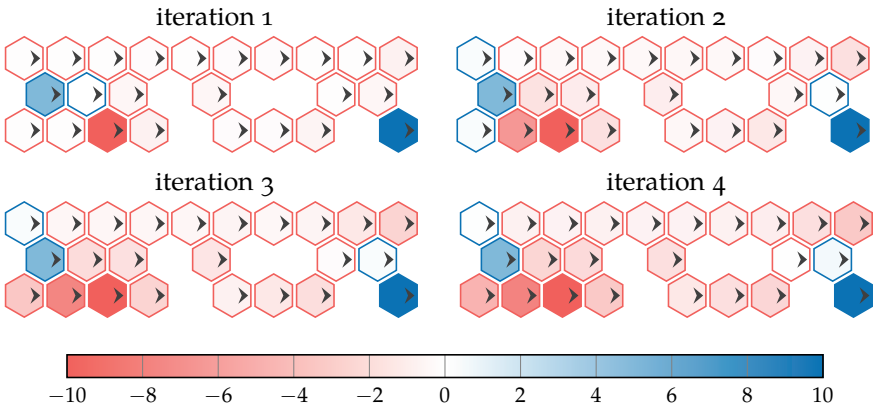


图7.3. 用于评估六边形世界问题（见附录F.1）中向东移动策略的迭代策略评估。箭头表示策略推荐的方向（即始终向东移动），颜色表示与状态相关的价值。价值随每次迭代而变化。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

该等式称为 *Bellman expectation equation*.[8]

策略评估可以通过直接求解贝尔曼期望方程中的方程组来完成，而无需迭代。方程（7.6）定义了一组包含 $|S|$ 个线性方程和 $|S|$ 个未知数的系统，这些未知数对应于每个状态的价值。求解该方程组的一种方法是首先将其转换为矩阵形式：

Programming

$$\mathbf{U}^\pi = \mathbf{R}^\pi + \gamma \mathbf{T}^\pi \mathbf{U}^\pi \tag{7.7}$$

其中 $\mathbf{U}^\pi$ 和 $\mathbf{R}^\pi$ 是以向量形式表示的效用函数和奖励函数，包含 $|\mathcal{S}|$ 个分量。$|\mathcal{S}| \times |\mathcal{S}|$ 矩阵 $\mathbf{T}^\pi$ 包含状态转移概率，其中 T_{ij}^π 是从第 i 个状态转移到第 j 个状态的概率。

价值函数如下所示：

$$\mathbf{U}^\pi - \gamma \mathbf{T}^\pi \mathbf{U}^\pi = \mathbf{R}^\pi \tag{7.8}$$

$$(\mathbf{I} - \gamma \mathbf{T}^\pi)\mathbf{U}^\pi = \mathbf{R}^\pi \tag{7.9}$$

$$\mathbf{U}^\pi = (\mathbf{I} - \gamma \mathbf{T}^\pi)^{-1}\mathbf{R}^\pi \tag{7.10}$$

该方法在算法7.4中实现。以这种方式求解$\mathbf{U}^\pi$需要$O(|\mathcal{S}|^3)$时间。该方法用于评估图7.4中的策略。

8 这个方程以理查德·E·贝尔曼命名，他是动态规划的先驱之一。R. E. Bellman, *Dynamic*。普林斯顿大学出版社，1957年。

```
function policy_evaluation(𝒫::MDP, π)
    𝒮, R, T, γ = 𝒫.𝒮, 𝒫.R, 𝒫.T, 𝒫.γ
    R′ = [R(s, π(s)) for s in 𝒮]
    T′ = [T(s, π(s), s′) for s in 𝒮, s′ in 𝒮]
    return (I - γ*T′)\R′
end
```

算法7.4。精确策略评估，针对具有离散状态和动作空间的MDP $\mathcal{P}$，计算策略π的价值函数。

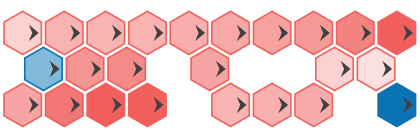


图7.4。用于评估六边形世界问题中向东移动策略的精确策略评估。精确解包含的值低于图7.3中迭代策略评估前几步所包含的值。如果我们对迭代策略评估运行更多迭代，它将收敛到相同的值函数。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

7.3 Value Function Policies

上一节展示了如何计算与策略关联的价值函数。本节展示如何从价值函数中提取策略，我们稍后在生成最优策略时会用到这一点。给定一个价值函数 U，它可能对应于也可能不对应于最优价值函数，我们可以构造一个策略 π，使公式（7.5）中引入的前瞻方程最大化：

$$\pi(s) = \arg\max_a \left(R(s,a) + \gamma \sum_{s'} T(s' \mid s,a) U(s') \right) \tag{7.11}$$

我们将此策略称为关于U的*greedy policy*。如果U是最优值函数，那么提取的策略就是最优的。算法7.5实现了这一思想。

表示策略的另一种方法是使用*action value function*，有时称为*Q-function*。动作价值函数表示从状态s开始，采取行动a，然后继续遵循关于Q的贪婪策略时的预期回报：

$$Q(s,a) = R(s,a) + \gamma \sum_{s'} T(s' \mid s,a) U(s') \tag{7.12}$$

从该动作价值函数中，我们可以得到价值函数，

$$U(s) = \max_a Q(s,a) \tag{7.13}$$

以及政策 {v*}

$$\pi(s) = \arg\max_a Q(s,a) \tag{7.14}$$

对于离散问题，显式存储 Q 需要 $O(|\mathcal{S}| \times |\mathcal{A}|)$ 存储空间，而非 U 的 $O(|\mathcal{S}|)$ 存储空间，但我们不必使用 R 和 T 来提取策略。

策略也可以使用 *advantage function* 来表示，它衡量了采取某项行动相较于贪婪行动的优势。它是根据 Q 和 U 之间的差异来定义的：

$$A(s,a) = Q(s,a) - U(s) \tag{7.15}$$

贪婪动作具有零优势，而非贪婪动作则具有负优势。在本书后续将要讨论的一些算法中，会使用U表示法，而其他算法则会采用Q或A。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct ValueFunctionPolicy
    𝒫 # problem
    U # utility function
end

function greedy(𝒫::MDP, U, s)
    u, a = findmax(a→lookahead(𝒫, U, s, a), 𝒫.𝒜)
    return (a=a, u=u)
end

(π::ValueFunctionPolicy)(s) = greedy(π.𝒫, π.U, s).a
```

算法7.5。从MDP $\mathcal{P}$的价值函数U中提取的价值函数策略。greedy函数将在其他算法中使用。

7.4 Policy Iteration

Policy iteration (算法 7.6) 是计算最优策略的一种方法。它涉及在策略评估（第7.2节）和通过贪婪策略（算法7.5）进行策略改进之间迭代。策略迭代保证从任何初始策略收敛。它在有限次迭代中收敛，因为策略的数量是有限的，并且如果有改进的空间，每次迭代都会改进策略。尽管可能策略的数量随状态数呈指数增长，策略迭代通常收敛得很快。图7.5展示了在六边形世界问题上的策略迭代。

```
struct PolicyIteration
    π # initial policy
    k_max # maximum number of iterations
end

function solve(M::PolicyIteration, 𝒫::MDP)
    π, 𝒮 = M.π, 𝒫.𝒮
    for k = 1:M.k_max
        U = policy_evaluation(𝒫, π)
        π′ = ValueFunctionPolicy(𝒫, U)
        if all(π(s) == π′(s) for s in 𝒮)
            break
        end
        π = π′
    end
    return π
end
```

算法7.6. 策略迭代，它迭代地改进初始策略π，以获取具有离散状态和动作空间的MDP $\mathcal{P}$ 的最优策略。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

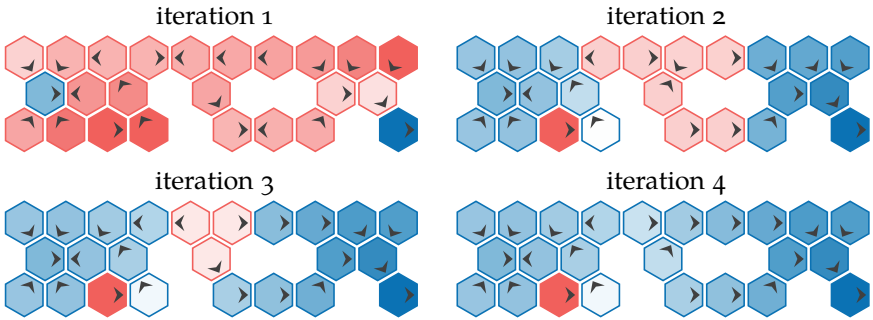


图7.5。策略迭代用于在六边形世界问题中迭代改进初始的向东移动策略，以获得最优策略。在第一次迭代中，我们看到与向东移动策略关联的值函数，以及指示相对于该值函数贪婪的策略的箭头。策略迭代在四次迭代后收敛；如果我们运行第五次或更多次迭代，我们将得到相同的策略。

策略迭代往往成本高昂，因为我们必须对每次迭代中的策略进行评估。策略迭代的一个变体称为*modified policy iteration*[9]使用迭代策略评估而不是精确策略评估来近似值函数。我们可以在策略改进的步骤之间选择策略评估的迭代次数。如果我们在步骤之间只使用一次迭代，那么这种方法就与值迭代相同。

9 M. L. Puterman 和 M. C. Shin，“折扣马尔可夫决策问题的修改策略迭代算法”，*Management Sci.*，第24卷，第11期，第1127-1137页，1978年。

7.5 Value Iteration

Value iteration 是策略迭代的一种替代方法，因其简便性而常被采用。与策略改进不同，值迭代直接更新价值函数。它从任意有界的价值函数U开始，这意味着对于所有s，有$|U(s)| < \infty$。一种常见的初始化方式是对所有s，$U(s) = 0$。

值函数可以通过应用*Bellman backup*来改进，也称为*Bellman update*：[10]

$$U_{k+1}(s) = \max_a \left(R(s,a) + \gamma \sum_{s'} T(s' \mid s,a) U_k(s') \right) \tag{7.16}$$

10 它被称为备份操作，因为它将信息从未来状态传回某一状态。

该备份过程在算法7.7中实现。

```
function backup(𝒫::MDP, U, s)
    return maximum(lookahead(𝒫, U, s, a) for a in 𝒫.𝒜)
end
```

算法7.7。应用于MDP $\mathcal{P}$的备份过程，它在状态s处改进价值函数U。

反复应用此更新保证会收敛到最优价值函数。与迭代策略评估类似，我们可以利用这一更新的事实：

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

是一个收缩映射以证明收敛性。[11] 这个最优策略是保证的 [11] 见练习7.1 3.
以满足 *Bellman optimality equation*：

$$U^*(s) = \max_a \left(R(s,a) + \gamma \sum_{s'} T(s' \mid s,a) U^*(s') \right) \tag{7.17}$$

一旦该等式成立，进一步应用贝尔曼备份不会改变价值函数。最优策略可以从 U^* 中使用方程 (7.11) 提取。价值迭代在算法 7.8 中实现，并应用于图 7.6 中的六边形世界问题。

算法7.8中的实现在固定次数的迭代后停止，但通常也会基于值 $\|U_{k+1} - U_k\|_\infty$的最大变化提前终止迭代，这被称为*Bellman residual*。如果贝尔曼残差降至阈值δ以下，则迭代终止。贝尔曼残差为δ保证了通过值迭代获得的最优值函数在U^*的$\epsilon = \delta\gamma/(1-\gamma)$范围内。[12]折扣因子越接近1，显著膨胀

这个误差会导致收敛速度变慢。如果我们大幅折扣未来的奖励（γ 接近 0），那么我们就不需要向未来迭代那么多。这一效应在示例7.2中得到了展示。

12 参见练习7.8。

知道估计值函数与最优值函数的最大偏差$\|U_k - U^*\|_\infty < \epsilon$，使我们能够限制在提取策略$\pi$下获得的奖励与最优策略$\pi^*$下的最大偏差。这个 *policy loss* $\|U^\pi - U^*\|_\infty$由$2\epsilon\gamma/(1-\gamma)$所界定。[13]

13 S. P. Singh 和 R. C. Yee，“近似最优值函数损失的上界”，*Machine Learning*，第16卷，第3期，第227-233页，1994年。

```
struct ValueIteration
    k_max # maximum number of iterations
end

function solve(M::ValueIteration, 𝒫::MDP)
    U = [0.0 for s in 𝒫.𝒮]
    for k = 1:M.k_max
        U = [backup(𝒫, U, s) for s in 𝒫.𝒮]
    end
    return ValueFunctionPolicy(𝒫, U)
end
```

算法7.8。值迭代，通过迭代改进值函数`U`，以获取具有离散状态和动作空间的MDP $\mathcal{P}$的最优策略。该方法在`k_max`次迭代后终止。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

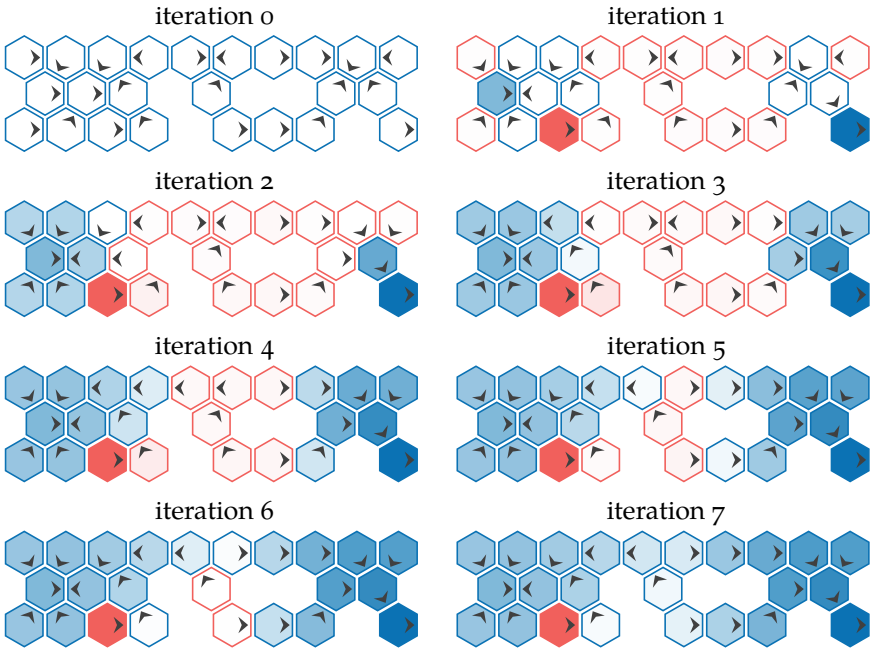
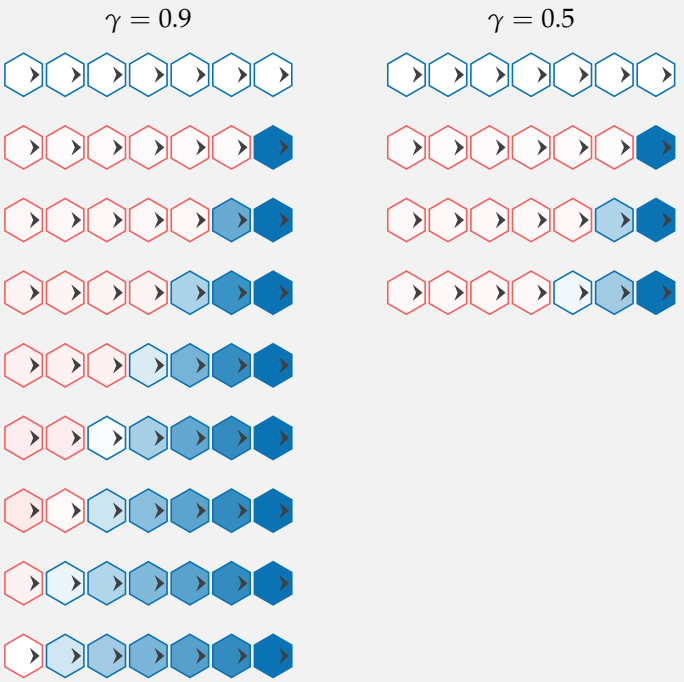


图7.6. 六边形世界问题中通过值迭代获得最优策略。每个六边形根据价值函数着色，箭头指示了相对于该价值函数的贪婪策略。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

考虑六边形世界问题的一个简单变体，由一排瓷砖组成，末端有一个单一的消耗瓷砖产生10的奖励。折扣因子直接影响消耗瓷砖的奖励沿直线传播到其他瓷砖的速率，从而影响价值迭代收敛的速度。



例7.2. 折现因子对值迭代收敛性的影响。在每种情况下，值迭代运行至贝尔曼残差小于1为止。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

7.6 Asynchronous Value Iteration

值迭代往往计算量大，因为每次迭代都会更新价值函数U_k中的每个条目以获得U_{k+1}。在*asynchronous valueiteration*中，每次迭代仅更新状态的一个子集。只要每个状态被更新无限次，异步值迭代仍保证收敛到最优价值函数。

一种常见的异步值迭代方法，*Gauss-Seidel value iteration* (算法7.9)，按状态顺序进行扫描，并就地应用贝尔曼更新：

$$U(s) \leftarrow \max_a \left(R(s,a) + \gamma \sum_{s'} T(s' \mid s,a) U(s') \right) \tag{7.18}$$

计算上的节省在于不必在每次迭代时在内存中构造第二个值函数。根据所选的排序方式，Gauss-Seidel 值迭代可以比标准值迭代更快地收敛。[14]在

一些问题中，状态包含一个随时间确定性地向前递增的时间索引。如果我们从最后一个时间索引开始应用高斯-赛德尔值迭代并反向进行，这一过程有时被称为*backward induction value iteration*。状态排序影响的示例见例7.3。

14 在Gauss-Seidel价值迭代中，较差的排序不会导致算法比标准价值迭代更慢。

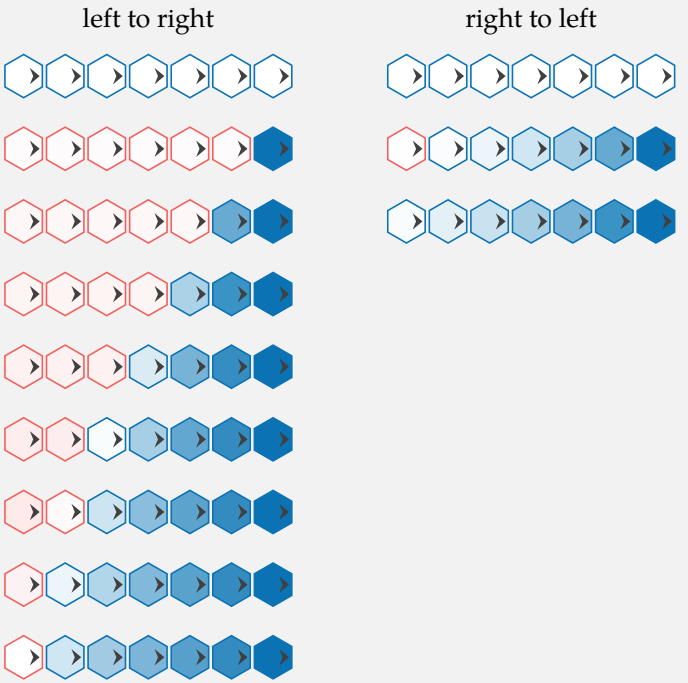
```
struct GaussSeidelValueIteration
    k_max # maximum number of iterations
end

function solve(M::GaussSeidelValueIteration, 𝒫::MDP)
    U = [0.0 for s in 𝒫.𝒮]
    for k = 1:M.k_max
        for (i, s) in enumerate(𝒫.𝒮)
            U[i] = backup(𝒫, U, s)
        end
    end
    return ValueFunctionPolicy(𝒫, U)
end
```

算法7.9. 异步值迭代，其更新状态的方式不同于值迭代，通常能节省计算时间。该方法在`k_max`次迭代后终止。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

考虑例7.2中六边形世界问题的线性变体。我们可以使用异步价值迭代来解决同样的问题。状态顺序直接影响到从消耗瓦片到其他瓦片的奖励传播速度，从而影响方法的收敛速度。



例 7.3. 状态排序对异步值迭代收敛性的影响。在这种情况下，从右到左评估允许在远少的迭代次数内实现收敛。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

7.7 Linear Program Formulation

将寻找最优策略的问题形式化为一个*linear program*，这是一个具有线性目标函数和一组线性等式或不等式约束的优化问题。一旦问题被表示为线性规划，我们就可以使用众多线性规划求解器之一。[15]

15 关于线性规划概述，见 R. Vanderbei, *Linear Programming, Foundations and Extensions*，第4版，Springer，2014年。

为了展示如何将贝尔曼最优方程转换为线性规划，我们首先将贝尔曼最优方程中的等式替换为一组不等式约束，同时在每个状态s下最小化$U(s)$：[16]

直观地说，我们希望降低所有状态s下的值$U(s)$，以便将不等式约束转换为等式约束。因此，我们最小化所有效用之和。

$$\begin{aligned} &\text{minimize} \sum_s U(s) \\ &\text{subject to } U(s) \geq \max_a \left(R(s,a) + \gamma \sum_{s'} T(s' \mid s,a) U(s') \right) \text{ for all } s \end{aligned} \tag{7.19}$$

优化中的变量是每个状态的效用值。一旦我们知道了这些效用值，就可以利用方程（7.11）提取出最优策略。

不等式约束中的最大化可以用一组线性约束代替，从而构成一个线性规划：

$$\begin{aligned} &\text{minimize} \sum_s U(s) \\ &\text{subject to } U(s) \geq R(s,a) + \gamma \sum_{s'} T(s' \mid s,a) U(s') \text{ for all } s \text{ and } a \end{aligned} \tag{7.20}$$

在方程（7.20）所示的线性规划中，变量数等于状态数，约束数等于状态数乘以动作数。因为线性规划可以在多项式时间内求解，[17] MDP 也可以在多项式时间内求解。尽管一个

17 这是由 L. G. Khachiyan 在“线性规划中的多项式算法”中证明的，*USSR Computational Mathematics and Mathematical Physics*，第20卷，第1期，第53–72页，1980年。现代算法在实践中往往更为高效。

线性规划方法提供了这种渐近复杂度保证，但在实践中，简单地使用价值迭代往往更为高效。算法7.10提供了这一实现。

7.8 Linear Systems with Quadratic Reward

到目前为止，我们假设了离散的状态和动作空间。本节放宽这一假设，允许连续、向量值状态和动作。离散问题的贝尔曼最优方程可以修改如下：[18]

18 本节假设问题是无折扣且有限时域的，但这些方程可以很容易地推广。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct LinearProgramFormulation end

function tensorform(𝒫::MDP)
    𝒮, 𝒜, R, T = 𝒫.𝒮, 𝒫.𝒜, 𝒫.R, 𝒫.T
    𝒮′ = eachindex(𝒮)
    𝒜′ = eachindex(𝒜)
    R′ = [R(s,a) for s in 𝒮, a in 𝒜]
    T′ = [T(s,a,s′) for s in 𝒮, a in 𝒜, s′ in 𝒮]
    return 𝒮′, 𝒜′, R′, T′
end

solve(𝒫::MDP) = solve(LinearProgramFormulation(), 𝒫)

function solve(M::LinearProgramFormulation, 𝒫::MDP)
    𝒮, 𝒜, R, T = tensorform(𝒫)
    model = Model(GLPK.Optimizer)
    @variable(model, U[𝒮])
    @objective(model, Min, sum(U))
    @constraint(model, [s=𝒮,a=𝒜], U[s] ≥ R[s,a] + 𝒫.γ*T[s,a,:]⋅U)
    optimize!(model)
    return ValueFunctionPolicy(𝒫, value.(U))
end
```

算法 7.10. 使用线性规划形式求解离散MDP的方法。为便于指定线性规划，我们定义了一个函数，用于将MDP转换为其张量形式，其中状态和动作由整数索引组成，奖励函数是一个矩阵，转移函数是一个三维张量。它使用`JuMP.jl`包进行数学规划。优化器被设置为使用`GLPK.jl`，但也可以使用其他优化器。我们还定义了MDP的默认求解行为，以使用这种形式。

$$U_{h+1}(\mathbf{s}) = \max_{\mathbf{a}}\left(R(\mathbf{s},\mathbf{a}) + \int T(\mathbf{s}' \mid \mathbf{s},\mathbf{a})U_h(\mathbf{s}')\,\mathrm{d}\mathbf{s}'\right) \tag{7.21}$$

在方程（7.16）中，s 和 a 被替换为它们的向量等价形式，求和被替换为积分，而 T 提供的是概率密度而非概率质量。对于任意连续转移分布和奖励函数，计算方程（7.21）并不直接。

在某些情况下，对于具有连续状态和动作空间的MDPs，确实存在精确解法。[19] 特别地，如果一个问题具有*linear dynamics*并且拥有*quadratic reward*，那么最优策略可以以闭式形式高效地找到。这样的系统在控制理论中被称为*linear quadratic regulator* (*LQR*)，并且已被充分研究。[20]

如果从状态s执行动作a后的下一个状态s′由以下形式的方程确定，则该问题具有线性动态：

$$\mathbf{s}' = \mathbf{T}_s\mathbf{s} + \mathbf{T}_a\mathbf{a} + \mathbf{w} \tag{7.22}$$

其中 T_s 和 T_a 是矩阵，而 w 是从一个均值、有限方差分布中抽取的随机扰动，该分布不依赖于 s 和 a。一种常见的选择是多元高斯分布。

19 有关详细概述，请参见 D. P. Bertsekas 的《*Dynamic Programming and Optimal Control*》第一卷第四章。Athena Scientific，2007年。

20 关于LQR及其他相关控制问题的简明总结，参见A. Shaiju和I. R. Petersen，“离散时间LQR、LQG、LEQG及极小极大LQG最优控制问题的公式，”*IFAC Proceedings Volumes*，第41卷，第2期，第8773–8778页，2008年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

如果一个奖励函数可以写成以下形式，则它是二次的：[21]

$$R(\mathbf{s}, \mathbf{a}) = \mathbf{s}^\top \mathbf{R}_s \mathbf{s} + \mathbf{a}^\top \mathbf{R}_a \mathbf{a} \tag{7.23}$$

其中 R_s 和 R_a 是决定状态和动作分量组合如何贡献奖励的矩阵。我们额外要求 R_s 为半负定，且 R_a 为负定。这样的奖励函数会惩罚偏离 0 的状态和动作。

21 第三项，$2\mathbf{s}^\top \mathrm{R}_{sa}\mathbf{a}$，也可以包括在内。例如，参见Shaiju和Petersen（2008）。

线性动力学与二次奖励的问题在控制理论中很常见，人们常寻求调控一个过程，使其不偏离期望值太远。二次成本对远离原点的状态赋予的代价远高于靠近原点的状态。具有线性动力学和二次奖励的问题的最优策略有一个解析的闭式解。许多MDP可以用线性二次MDP近似并求解，通常能为原问题生成合理的策略。

将转移函数和奖励函数代入公式（7.21）得出

$$U_{h+1}(\mathbf{s}) = \max_{\mathbf{a}} \left(\mathbf{s}^\top \mathbf{R}_s \mathbf{s} + \mathbf{a}^\top \mathbf{R}_a \mathbf{a} + \int p(\mathbf{w}) U_h(\mathbf{T}_s \mathbf{s} + \mathbf{T}_a \mathbf{a} + \mathbf{w}) \, \mathrm{d}\mathbf{w} \right) \tag{7.24}$$

其中 $p(\mathrm{w})$ 是随机零均值扰动 w 的概率密度。最优一步价值函数为

$$U_1(\mathbf{s}) = \max_{\mathbf{a}} \left(\mathbf{s}^\top \mathbf{R}_s \mathbf{s} + \mathbf{a}^\top \mathbf{R}_a \mathbf{a} \right) = \mathbf{s}^\top \mathbf{R}_s \mathbf{s} \tag{7.25}$$

其最优动作为 $= 0$。

我们将通过归纳法证明 $U_h(\mathrm{s})$ 具有二次型 $\mathrm{s}^\top \mathrm{V}_h \mathrm{s} + q_h$，其中矩阵 V_h 是对称的。对于一步值函数，$\mathrm{V}_1 = \mathrm{R}_s$ 且 $q_1 = 0$。将此二次型代入方程（7.24）可得

$$U_{h+1}(\mathbf{s}) = \mathbf{s}^\top \mathbf{R}_s \mathbf{s} + \max_{\mathbf{a}} \left(\mathbf{a}^\top \mathbf{R}_a \mathbf{a} + \int p(\mathbf{w}) \left((\mathbf{T}_s \mathbf{s} + \mathbf{T}_a \mathbf{a} + \mathbf{w})^\top \mathbf{V}_h (\mathbf{T}_s \mathbf{s} + \mathbf{T}_a \mathbf{a} + \mathbf{w}) + q_h \right) \mathrm{d}\mathbf{w} \right) \tag{7.26}$$

通过展开并利用$\int p(\mathrm{w}) \, \mathrm{dw} = 1$以及$\int \mathrm{w} p(\mathrm{w}) \, \mathrm{dw} = 0$这一事实，可以简化此式：

$$\begin{aligned} U_{h+1}(\mathbf{s}) = {} & \mathbf{s}^\top \mathbf{R}_s \mathbf{s} + \mathbf{s}^\top \mathbf{T}_s^\top \mathbf{V}_h \mathbf{T}_s \mathbf{s} \\ & + \max_{\mathbf{a}} \left(\mathbf{a}^\top \mathbf{R}_a \mathbf{a} + 2\mathbf{s}^\top \mathbf{T}_s^\top \mathbf{V}_h \mathbf{T}_a \mathbf{a} + \mathbf{a}^\top \mathbf{T}_a^\top \mathbf{V}_h \mathbf{T}_a \mathbf{a} \right) \\ & + \int p(\mathbf{w}) \left(\mathbf{w}^\top \mathbf{V}_h \mathbf{w} \right) \mathrm{d}\mathbf{w} + q_h \end{aligned} \tag{7.27}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们可以通过对a求导并将其设为0来获得最优动作：[22]

$$\begin{aligned}\mathbf{0} &= \left(\mathbf{R}_a + \mathbf{R}_a^\top\right)\mathbf{a} + 2\mathbf{T}_a^\top \mathbf{V}_h \mathbf{T}_s \mathbf{s} + \left(\mathbf{T}_a^\top \mathbf{V}_h \mathbf{T}_a + \left(\mathbf{T}_a^\top \mathbf{V}_h \mathbf{T}_a\right)^\top\right)\mathbf{a} \\ &= 2\mathbf{R}_a \mathbf{a} + 2\mathbf{T}_a^\top \mathbf{V}_h \mathbf{T}_s \mathbf{s} + 2\mathbf{T}_a^\top \mathbf{V}_h \mathbf{T}_a \mathbf{a}\end{aligned} \tag{7.28}$$

22 回忆一下

$$\nabla_{\mathbf{x}} \mathbf{A}\mathbf{x} = \mathbf{A}^\top$$
$$\nabla_{\mathbf{x}} \mathbf{x}^\top \mathbf{A}\mathbf{x} = (\mathbf{A} + \mathbf{A}^\top)\mathbf{x}$$

求解最优动作得到[23]

$$\mathbf{a} = -\left(\mathbf{R}_a + \mathbf{T}_a^\top \mathbf{V}_h \mathbf{T}_a\right)^{-1} \mathbf{T}_a^\top \mathbf{V}_h \mathbf{T}_s \mathbf{s} \tag{7.29}$$

23 矩阵 $\mathrm{R}_a + \mathrm{T}_a^\top \mathrm{V}_h \mathrm{T}_a$ 是负定的，因此是可逆的。

将最优动作代入 $U_{h+1}(\mathrm{s})$ 得到我们寻求的二次型 $U_{h+1}(\mathrm{s}) = \mathrm{s}^\top \mathrm{V}_{h+1} \mathrm{s} + q_{h+1}$，其中[24]

$$\mathbf{V}_{h+1} = \mathbf{R}_s + \mathbf{T}_s^\top \mathbf{V}_h^\top \mathbf{T}_s - \left(\mathbf{T}_a^\top \mathbf{V}_h \mathbf{T}_s\right)^\top \left(\mathbf{R}_a + \mathbf{T}_a^\top \mathbf{V}_h \mathbf{T}_a\right)^{-1} \left(\mathbf{T}_a^\top \mathbf{V}_h \mathbf{T}_s\right) \tag{7.30}$$

cati equation

24 这个方程有时被称为 *discrete-time Ric-*，以威尼斯数学家雅科波·里卡蒂（1676–1754）命名。

和

$$q_{h+1} = \sum_{i=1}^{h} \mathbb{E}_{\mathbf{w}}\left[\mathbf{w}^\top \mathbf{V}_i \mathbf{w}\right] \tag{7.31}$$

如果 $\mathrm{w} \sim \mathcal{N}(0, \boldsymbol{\Sigma})$，那么

$$q_{h+1} = \sum_{i=1}^{h} \mathrm{Tr}(\boldsymbol{\Sigma} \mathbf{V}_i) \tag{7.32}$$

我们可以从$\mathrm{V}_1 =$、R_s和$q_1 = 0$开始，通过迭代使用方程(7.30)和(7.31)，计算出任意视界h内的V_h和q_h。h步策略的最优行动直接来自方程(7.29)：

$$\pi_h(\mathbf{s}) = -\left(\mathbf{T}_a^\top \mathbf{V}_{h-1} \mathbf{T}_a + \mathbf{R}_a\right)^{-1} \mathbf{T}_a^\top \mathbf{V}_{h-1} \mathbf{T}_s \mathbf{s} \tag{7.33}$$

注意，最优动作与零均值扰动分布[25]无关。然而，扰动的方差确实会影响预期效用。算法7.11提供了实现。例7.4在一个具有线性高斯动力学的简单问题上演示了这一过程。

25 在这种情况下，我们可以在不改变最优策略的前提下，用其期望值替换随机扰动。这一性质被称为 *certainty equivalence*。

7.9 Summary

- 具有有界奖励的离散MDP可以通过动态规划精确求解。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct LinearQuadraticProblem
    Ts # transition matrix with respect to state
    Ta # transition matrix with respect to action
    Rs # reward matrix with respect to state (negative semidefinite)
    Ra # reward matrix with respect to action (negative definite)
    h_max # horizon
end

function solve(𝒫::LinearQuadraticProblem)
    Ts, Ta, Rs, Ra, h_max = 𝒫.Ts, 𝒫.Ta, 𝒫.Rs, 𝒫.Ra, 𝒫.h_max
    V = zeros(size(Rs))
    πs = Any[s -> zeros(size(Ta, 2))]
    for h in 2:h_max
        V = Ts'*(V - V*Ta*((Ta'*V*Ta + Ra) \ Ta'*V))*Ts + Rs
        L = -(Ta'*V*Ta + Ra) \ Ta' * V * Ts
        push!(πs, s -> L*s)
    end
    return πs
end
```

算法7.11。一种计算最优策略的方法，适用于具有由矩阵`Ts`和`Ta`参数化的随机线性动态以及由矩阵`Rs`和`Ra`参数化的二次奖励的`h_max`步视界MDP。该方法返回一个策略向量，其中条目h产生h步策略中的最优首个动作。

- 对于此类问题，策略评估可以通过矩阵求逆精确完成，也可以通过迭代算法近似。
- 策略迭代可以通过在策略评估和策略改进之间迭代来求解最优策略。
- 值迭代和异步值迭代通过直接迭代价值函数来节省计算。
- 寻找最优策略的问题可以框架为一个线性规划，并在多项式时间内求解。
- 具有线性转移函数和二次奖励的连续问题可以被精确求解。

7.10 Exercises

练习7.1. 证明对于无限序列的恒定奖励（所有t时$r_t = r$），无限视界折扣回报收敛于$r/(1-\gamma)$。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

考虑一个连续MDP，其中状态由标量位置和速度$s = [x, v]$组成。动作是标量加速度a，每个动作在一个时间步长$\Delta t = 1$内执行。从$s_0 = [-10, 0]$出发，给定二次奖励，找到最优的五步策略：

$$R(\mathbf{s}, a) = -x^2 - v^2 - 0.5a^2$$

使得系统趋向于在 $s = 0$ 处静止。

转移动态为

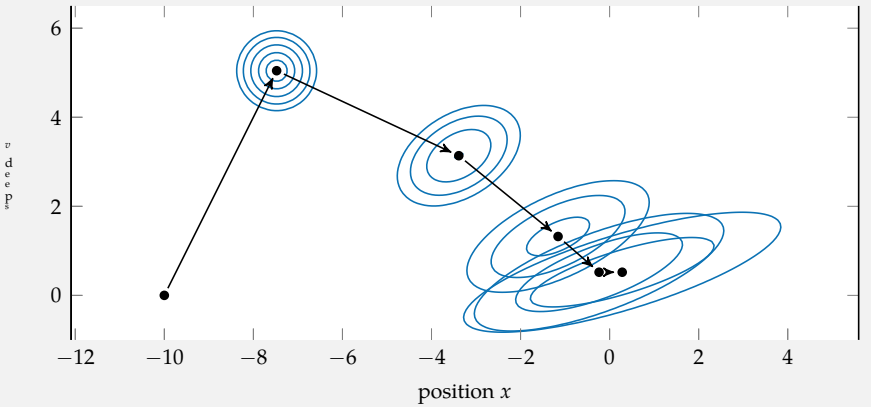
$$\begin{bmatrix} x' \\ v' \end{bmatrix} = \begin{bmatrix} x + v\Delta t + \frac{1}{2}a\Delta t^2 + w_1 \\ v + a\Delta t + w_2 \end{bmatrix} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ v \end{bmatrix} + \begin{bmatrix} 0.5\Delta t^2 \\ \Delta t \end{bmatrix} [a] + \mathbf{w}$$

其中w是从均值为零、协方差为0.1I的多元高斯分布中抽取的。

The reward matrices are $\mathbf{R}_s = -\mathbf{I}$ and $\mathbf{R}_a = -[0.5]$.

得到的最优策略是：

$$\begin{aligned}
\pi_1(\mathbf{s}) &= \begin{bmatrix} 0 & 0 \end{bmatrix} \mathbf{s} \\
\pi_2(\mathbf{s}) &= \begin{bmatrix} -0.286 & -0.857 \end{bmatrix} \mathbf{s} \\
\pi_3(\mathbf{s}) &= \begin{bmatrix} -0.462 & -1.077 \end{bmatrix} \mathbf{s} \\
\pi_4(\mathbf{s}) &= \begin{bmatrix} -0.499 & -1.118 \end{bmatrix} \mathbf{s} \\
\pi_5(\mathbf{s}) &= \begin{bmatrix} -0.504 & -1.124 \end{bmatrix} \mathbf{s}
\end{aligned}$$



示例 7.4. 使用线性转移函数和二次奖励求解有限时域 MDP。该图展示了系统从 $[-10, 0]$ 开始的进展。蓝色等高线表示每次迭代中状态上的高斯分布。初始信念是圆形的，但随着我们使用卡尔曼滤波器向前传播信念，它会变形为非圆形。

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

Solution: 我们可以通过以下步骤证明折现常数奖励的无限序列收敛到 $r/(1-\gamma)$:

$$\begin{aligned}\sum_{t=1}^{\infty} \gamma^{t-1} r_t &= r + \gamma^1 r + \gamma^2 r + \cdots \\ &= r + \gamma \sum_{t=1}^{\infty} \gamma^{t-1} r_t\end{aligned}$$

我们可以将求和移到左边并提取出$(1-\gamma)$:

$$(1-\gamma) \sum_{t=1}^{\infty} \gamma^{t-1} r = r$$

$$\sum_{t=1}^{\infty} \gamma^{t-1} r = \frac{r}{1-\gamma}$$

练习7.2。假设我们有一个由五个状态组成的MDP，$s_{1:5}$，以及两个动作：停留（a_S）和继续（a_C）。我们有以下内容:

$$\begin{aligned}T(s_i \mid s_i, a_S) &= 1 \text{ for } i \in \{1,2,3,4\} \\ T(s_{i+1} \mid s_i, a_C) &= 1 \text{ for } i \in \{1,2,3,4\} \\ T(s_5 \mid s_5, a) &= 1 \text{ for all actions } a \\ R(s_i, a) &= 0 \text{ for } i \in \{1,2,3,5\} \text{ and for all actions } a \\ R(s_4, a_S) &= 0 \\ R(s_4, a_C) &= 10\end{aligned}$$

如果最优值$U^*(s_1)$ =为1，则折扣因子γ是多少?

Solution: $U^*(s_1)$ 的最优值与从 s_1 开始遵循最优策略 π^* 相关。给定转移模型，从 s_1 出发的最优策略是一直继续，直到到达 s_5，这是一个终止状态，在此状态下我们无法再转移到其他状态或累积额外奖励。因此，s_1 的最优值可以计算为

$$\begin{aligned}U^*(s_1) &= \sum_{t=1}^{\infty} \gamma^{t-1} r_t \\ U^*(s_1) &= R(s_1, a_C) + \gamma^1 R(s_2, a_C) + \gamma^2 R(s_3, a_C) + \gamma^3 R(s_4, a_C) + \gamma^4 R(s_5, a_C) + \cdots \\ U^*(s_1) &= 0 + \gamma^1 \times 0 + \gamma^2 \times 0 + \gamma^3 \times 10 + \gamma^4 \times 0 + 0 \\ 1 &= 10\gamma^3\end{aligned}$$

因此，折扣因子为$\gamma = 0.1^{1/3} \approx 0.464$。

练习 7.3. 执行 k 步迭代策略评估的时间复杂度是多少?

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: 迭代策略评估需要计算前瞻方程：

$$U_{k+1}^{\pi}(s) = R(s, \pi(s)) + \gamma \sum_{s'} T(s' \mid s, \pi(s)) U_k^{\pi}(s')$$

在单个状态下更新值需要对所有$|\mathcal{S}|$状态求和。对所有状态进行单次迭代时，我们必须执行此操作$|\mathcal{S}|$次。因此，迭代策略评估的k步的时间复杂度为$O(k|\mathcal{S}|^2)$。

练习7.4。假设我们有一个具有六个状态 $s_{1:6}$ 和四个动作 $a_{1:4}$ 的MDP。利用以下动作价值函数 $Q(s, a)$ 的表格形式，计算 $U(s)$，$\pi(s)$ 和 $A(s, a)$。

$Q(s,a)$	a_1	a_2	a_3	a_4
s_1	0.41	0.46	0.37	0.37
s_2	0.50	0.55	0.46	0.37
s_3	0.60	0.50	0.38	0.44
s_4	0.41	0.50	0.33	0.41
s_5	0.50	0.60	0.41	0.39
s_6	0.71	0.70	0.61	0.59

Solution: 我们可以使用以下等式计算 $U(s)$、$\pi(s)$ 和 $A(s、a)$：

$$U(s) = \max_a Q(s, a) \qquad \pi(s) = \arg\max_a Q(s, a) \qquad A(s, a) = Q(s, a) - U(s)$$

s	$U(s)$	$\pi(s)$	$A(s,a_1)$	$A(s,a_2)$	$A(s,a_3)$	$A(s,a_4)$
s_1	0.46	a_2	−0.05	0.00	−0.09	−0.09
s_2	0.55	a_2	−0.05	0.00	−0.09	−0.18
s_3	0.60	a_1	0.00	−0.10	−0.22	−0.16
s_4	0.50	a_2	−0.09	0.00	−0.17	−0.09
s_5	0.60	a_2	−0.10	0.00	−0.19	−0.21
s_6	0.71	a_1	0.00	−0.01	−0.10	−0.12

练习 7.5。假设我们有一个三格、直线型的六边形世界（附录 F.1），其中最右侧的格子是吸收状态。当我们在最右侧状态采取任何行动时，我们会获得 10 的奖励，并被传送到第四个终止状态，在那里我们不再获得任何奖励。使用折扣因子 $\gamma = 0.9$，并执行一步策略迭代，其中初始策略 π 让我们在第一个格子向东移动，在第二个格子向东北移动，在第三个格子向西南移动。对于策略评估步骤，写出转移矩阵 $\mathbf{T}^{\pi}$ 和奖励向量 $\mathbf{R}^{\pi}$，然后直接使用矩阵求逆求解无限时域价值函数 $\mathbf{U}^{\pi}$。对于策略改进步骤，通过最大化前瞻方程来计算更新后的策略 π'。

Solution: 对于策略评估步骤，我们使用公式(7.10)，此处重述：

$$\mathbf{U}^{\pi} = (\mathbf{I} - \gamma \mathbf{T}^{\pi})^{-1} \mathbf{R}^{\pi}$$

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

使用为终止状态额外增加一个状态的方式构造转移矩阵$\mathbf{T}^{\pi}$和奖励向量$\mathbf{R}^{\pi}$，我们可以求解无限时域价值函数$\mathbf{U}^{\pi}$：[26]

$$\mathbf{U}^{\pi} = \left(\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} - (0.9) \begin{bmatrix} 0.3 & 0.7 & 0 & 0 \\ 0 & 0.85 & 0.15 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \right)^{-1} \begin{bmatrix} -0.3 \\ -0.85 \\ 10 \\ 0 \end{bmatrix} \approx \begin{bmatrix} 1.425 \\ 2.128 \\ 10 \\ 0 \end{bmatrix}$$

对于策略改进步骤，我们应用公式(7.11)，使用更新后的价值函数。arg max项中的动作对应于 a_E、a_{NE}、a_{NW}、a_W、a_{SW} 和 a_{SE}：

$$\begin{aligned} \pi(s_1) &= \arg\max(1.425, 0.527, 0.283, 0.283, 0.283, 0.527) = a_E \\ \pi(s_2) &= \arg\max(6.575, 2.128, 0.970, 1.172, 0.970, 2.128) = a_E \\ \pi(s_3) &= \arg\max(10, 10, 10, 10, 10, 10) \text{ (all actions are equally desirable)} \end{aligned}$$

练习7.6。对练习7.5中的问题执行两步值迭代，初始值函数为所有s的$U_0(s) = 0$。

Solution: 我们需要使用贝尔曼备份（方程（7.16））来迭代更新价值函数。最大项中的动作对应于a_E、a_{NE}、a_{NW}、a_W、a_{SW}和a_{SE}。对于我们的第一次迭代，所有状态的价值函数均为零，因此我们只需考虑奖励部分：

$$\begin{aligned} U_1(s_1) &= \max(-0.3, -0.85, -1, -1, -1, -0.85) = -0.3 \\ U_1(s_2) &= \max(-0.3, -0.85, -0.85, -0.3, -0.85, -0.85) = -0.3 \\ U_1(s_3) &= \max(10, 10, 10, 10, 10, 10) = 10 \end{aligned}$$

对于第二次迭代，

$$\begin{aligned} U_2(s_1) &= \max(-0.57, -1.12, -1.27, -1.27, -1.27, -1.12) = -0.57 \\ U_2(s_2) &= \max(5.919, 0.271, -1.12, -0.57, -1.12, 0.271) = 5.919 \\ U_2(s_3) &= \max(10, 10, 10, 10, 10, 10) = 10 \end{aligned}$$

练习 7.7。对练习 7.5 中的问题应用一次异步价值迭代扫描，初始价值函数 $U_0(s) =$ 对所有 s 均为 0。从右到左更新状态。

26 六边形世界问题定义了 $R(s、a、s')$，因此为了生成 $\mathbf{R}^{\pi}$ 的条目，我们必须计算

$$R(s,a) = \sum_{s'} T(s' \mid s,a) R(s,a,s')$$

例如，-0.3 来自向东移动导致与边界碰撞的 30% 概率，其代价为 -1。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: 我们使用贝尔曼备份（方程（7.16））按照我们的顺序迭代更新每个状态上的值函数。最大项中的动作对应于a_E、a_{NE}、a_{NW}、a_W、a_{SW}和a_{SE}：

$$\begin{aligned}
U(s_3) &= \max(10, 10, 10, 10, 10, 10) = 10 \\
U(s_2) &= \max(6, 0.5, -0.85, -0.3, -0.85, 0.5) = 6 \\
U(s_1) &= \max(3.48, -0.04, -1, -1, -1, -0.04) = 3.48
\end{aligned}$$

练习 7.8。证明如果 δ 的贝尔曼残差，那么通过值迭代获得的价值函数在每个状态 s 上都处于 $U^*(s)$ 的 $\delta\gamma/(1-\gamma)$ 范围内。

Solution: 对于给定的 U_k，假设我们知道 $\|U_k - U_{k-1}\|_\infty < \delta$。那么我们在下一次迭代中界定改进：

$$\begin{aligned}
U_{k+1}(s) - U_k(s) &= \max_a \left(R(s,a) + \gamma \sum_{s'} T(s' \mid s,a) U_k(s') \right) \\
&\quad - \max_a \left(R(s,a) + \gamma \sum_{s'} T(s' \mid s,a) U_{k-1}(s') \right) \\
&< \max_a \left(R(s,a) + \gamma \sum_{s'} T(s' \mid s,a) U_k(s') \right) \\
&\quad - \max_a \left(R(s,a) + \gamma \sum_{s'} T(s' \mid s,a) \left(U_k(s') - \delta \right) \right) \\
&= \delta\gamma
\end{aligned}$$

同样地，

$$\begin{aligned}
U_{k+1}(s) - U_k(s) &> \max_a \left(R(s,a) + \gamma \sum_{s'} T(s' \mid s,a) U_k(s') \right) \\
&\quad - \max_a \left(R(s,a) + \gamma \sum_{s'} T(s' \mid s,a) \left(U_k(s') + \delta \right) \right) \\
&= -\delta\gamma
\end{aligned}$$

经过无限次迭代后的累积改进因此受限于

$$\|U^*(s) - U_k(s)\|_\infty < \sum_{i=1}^{\infty} \delta\gamma^i = \frac{\delta\gamma}{1-\gamma}$$

一个δ的贝尔曼残差因此保证了通过值迭代得到的最优值函数在U^*的$\delta\gamma/(1-\gamma)$范围内。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

练习7.9。假设我们在专家策略上运行策略评估以获得价值函数。如果相对于该价值函数贪婪行动等价于专家策略，我们可以推断出关于专家策略的什么结论?

Solution: 我们从贝尔曼最优方程得知，基于最优价值函数的贪心前瞻是平稳的。如果贪心策略与专家策略一致，那么这两个策略都是最优的。

练习 7.10. 展示如何重新表述一个具有二次奖励函数 $R(\mathrm{s}、\mathrm{a}) = \mathrm{s}^\top \mathrm{R}_s \mathrm{s} + \mathrm{a}^\top \mathrm{R}_a \mathrm{a}$ 的 LQR 问题，使得奖励函数中包含 s 和 a 的线性项。

Solution: 我们可以引入一个始终等于1的额外状态维度，从而得到一个具有线性动力学的新系统：

$$\begin{bmatrix} \mathbf{s}' \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{T}_s & \mathbf{0} \\ \mathbf{0}^\top & 1 \end{bmatrix} \begin{bmatrix} \mathbf{s} \\ 1 \end{bmatrix} + \mathbf{T}_a \mathbf{a}$$

增强系统的奖励函数现在可以包含线性状态奖励项：

$$\begin{bmatrix} \mathbf{s} \\ 1 \end{bmatrix}^\top \mathbf{R}_{\text{augmented}} \begin{bmatrix} \mathbf{s} \\ 1 \end{bmatrix} = \mathbf{s}^\top \mathbf{R}_s \mathbf{s} + 2\mathbf{r}_{s,\text{linear}}^\top \mathbf{s} + r_{s,\text{scalar}}$$

类似地，我们可以加入一个始终为1的额外动作维度，以获得线性动作奖励项。

练习 7.11. 为什么在例7.4中获得的最优策略，当视野更广时，会产生幅度更大的动作?

Solution: 示例7.4中的问题具有二次奖励，惩罚偏离原点的行为。视界越长，可累积的负奖励越大，使得尽早达到原点更为值得。

练习7.12。证明迭代策略评估收敛于方程(7.6)的解。

Solution: 考虑应用于策略 π 的迭代策略评估，如公式 (7.5) 所示：

$$U_{k+1}^\pi(s) = R(s, \pi(s)) + \gamma \sum_{s'} T(s' \mid s, \pi(s)) U_k^\pi(s')$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

让我们定义一个算子 B_π，并将其重写为 $U_{k+1}^\pi = B_\pi U_k^\pi$。我们可以证明 B_π 是一个压缩映射：

$$\begin{aligned}
B_\pi U^\pi(s) &= R(s,\pi(s)) + \gamma \sum_{s'} T(s' \mid s, \pi(s)) U^\pi(s') \\
&= R(s,\pi(s)) + \gamma \sum_{s'} T(s' \mid s, \pi(s)) \big(U^\pi(s') - \hat{U}^\pi(s') + \hat{U}^\pi(s')\big) \\
&= B_\pi \hat{U}^\pi(s) + \gamma \sum_{s'} T(s' \mid s, \pi(s)) \big(U^\pi(s') - \hat{U}^\pi(s')\big) \\
&\leq B_\pi \hat{U}^\pi(s) + \gamma \|U^\pi - \hat{U}^\pi\|_\infty
\end{aligned}$$

因此，$\|B_\pi U^\pi - B_\pi \hat{U}^\pi\|_\infty \leq \alpha \|U^\pi - \hat{U}^\pi\|_\infty$对于$\alpha = \gamma$，意味着$B_\pi$是一个收缩映射。正如附录A.15所讨论的，$\lim_{t\to\infty} B_\pi^t U_1^\pi$收敛于一个唯一的不动点$U^\pi$，对此有$U^\pi = B_\pi U^\pi$。

练习 7.13。证明值迭代收敛到唯一解。

Solution: 值迭代更新（公式(7.16)）是

$$U^{k+1}(s) = \max_a \left(R(s,a) + \gamma \sum_{s'} T(s' \mid s, a) U_k(s') \right)$$

我们将贝尔曼算子表示为 B，并将贝尔曼备份的应用重写为 $U_{k+1} = BU_k$。与之前的问题一样，如果 B 是一个压缩映射，那么重复将 B 应用于 U 将收敛到一个唯一的不动点。

我们可以证明 B 是一个压缩映射：

$$\begin{aligned}
BU(s) &= \max_a \left(R(s,a) + \gamma \sum_{s'} T(s' \mid s, a) U(s') \right) \\
&= \max_a \left(R(s,a) + \gamma \sum_{s'} T(s' \mid s, a) \big(U(s') - \hat{U}(s') + \hat{U}(s')\big) \right) \\
&\leq B\hat{U}(s) + \gamma \max_a \sum_{s'} T(s' \mid s, a) \big(U(s') - \hat{U}(s')\big) \\
&\leq B\hat{U}(s) + \alpha \|U - \hat{U}\|_\infty
\end{aligned}$$

对于 $\alpha = \gamma \max_s \max_a \sum_{s'} T(s' \mid s、 a)$，其中 $0 \leq \alpha < 1$。因此，$\|BU - B\hat{U}\|_\infty \leq \alpha \|U - \hat{U}\|_\infty$，这意味着 B 是一个压缩映射。

练习7.14。证明价值迭代收敛的点对应于最优价值函数。

Solution: 令 U 为值迭代产生的值函数。我们希望证明 $U = U^*$。在收敛时，我们有 $BU = U$。令 U_0 为将所有状态映射到 0 的值函数。对于任意策略 π，由 B_π 的定义可得 $B_\pi U_0 \leq BU_0$。类似地，$B_\pi^t U_0 \leq B^t U_0$。因为当 $t \to \infty$ 时，$B_{\pi^*}^t U_0 \to U^*$ 和 $B^t U_0 \to U$，所以有 $U^* \leq U$，这只有在 $U = U^*$ 的情况下才成立。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

练习 7.15. 假设我们有一个带有干扰 $w \sim \mathcal{N}(0, \Sigma)$ 和二次奖励的线性高斯问题。证明效用函数中标量项的形式为：

$$q_{h+1} = \sum_{i=1}^{h} \mathbb{E}_{\mathbf{w}}\left[\mathbf{w}^\top \mathbf{V}_i \mathbf{w}\right] = \sum_{i=1}^{h} \text{Tr}(\Sigma \mathbf{V}_i)$$

您可能想使用*trace trick*：

$$\mathbf{x}^\top \mathbf{A}\mathbf{x} = \text{Tr}\left(\mathbf{x}^\top \mathbf{A}\mathbf{x}\right) = \text{Tr}\left(\mathbf{A}\mathbf{x}\mathbf{x}^\top\right)$$

Solution: 如果 $\mathbb{E}_{\mathbf{w}}\left[w^\top V_i w\right] = \text{Tr}(\Sigma V_i)$，该方程成立。我们的推导是

$$\begin{aligned}
\mathop{\mathbb{E}}_{\mathbf{w}\sim\mathcal{N}(\mathbf{0},\Sigma)}\left[\mathbf{w}^\top \mathbf{V}_i \mathbf{w}\right] &= \mathop{\mathbb{E}}_{\mathbf{w}\sim\mathcal{N}(\mathbf{0},\Sigma)}\left[\text{Tr}\left(\mathbf{w}^\top \mathbf{V}_i \mathbf{w}\right)\right] \\
&= \mathop{\mathbb{E}}_{\mathbf{w}\sim\mathcal{N}(\mathbf{0},\Sigma)}\left[\text{Tr}\left(\mathbf{V}_i \mathbf{w}\mathbf{w}^\top\right)\right] \\
&= \text{Tr}\left(\mathop{\mathbb{E}}_{\mathbf{w}\sim\mathcal{N}(\mathbf{0},\Sigma)}\left[\mathbf{V}_i \mathbf{w}\mathbf{w}^\top\right]\right) \\
&= \text{Tr}\left(\mathbf{V}_i \mathop{\mathbb{E}}_{\mathbf{w}\sim\mathcal{N}(\mathbf{0},\Sigma)}\left[\mathbf{w}\mathbf{w}^\top\right]\right) \\
&= \text{Tr}(\mathbf{V}_i \Sigma) \\
&= \text{Tr}(\Sigma \mathbf{V}_i)
\end{aligned}$$

练习7.16。在方程(7.31)给出的LQR最优值函数中，标量项q的作用是什么?

$$q_{h+1} = \sum_{i=1}^{h} \mathbb{E}_{\mathbf{w}}\left[\mathbf{w}^\top \mathbf{V}_i \mathbf{w}\right]$$

Solution: 如果对于所有非零的x，有$x^\top Mx > 0$，则矩阵M是正定的。在方程(7.31)中，每个V_i都是半负定的，因此对于所有的w，都有$w^\top Vw \leq 0$。因此，这些q项保证为非正。这是可以预见的，因为在LQR问题中无法获得正回报，我们转而寻求最小化成本。

q 标量是二次最优值函数中的偏移量：

$$U(\mathbf{s}) = \mathbf{s}^\top \mathbf{V}\mathbf{s} + q$$

每个 q 代表基线奖励，$s^\top Vs$ 项围绕其波动。我们知道 V 是负定的，所以 $s^\top Vs \leq 0$，因此 q 表示如果位于原点 $s = 0$ 时可能获得的期望奖励。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

8 Approximate Value Functions

到目前为止，我们假设价值函数可以用表格表示。表格仅对小规模、离散的问题是有用的表示。状态空间较大的问题可能需要不可行的内存量，而上一章讨论的精确方法可能需要不可行的计算量。对于此类问题，我们通常不得不求助于*approximate dynamicprogramming*，其中解可能不是精确的。[1]一种近似的方法

1 关于这一主题更深入的论述由 W. B. Powell 提供，*Approxi-mate Dynamic Programming: Solving the Curses of Dimensionality*，第2版。Wiley，2011年。相关见解可从多个领域中得出，如 W. B. Powell 所讨论，*Re-inforcement Learning and Stochastic Optimization*。Wiley，2022年。

解决方案是使用*value function approximation*，这是本章的主题。我们将讨论逼近价值函数的不同方法，以及如何结合动态规划来推导近似最优策略。

8.1 Parametric Representations

我们将使用$U_\theta(s)$来表示值函数的*parametric representation*，其中θ是*parameters*的向量。有多种方式来表示$U_\theta(s)$，其中一些将在本章后面提到。假设我们有这样的近似，我们可以根据以下方式提取动作

$$\pi(s) = \arg\max_a \left(R(s,a) + \gamma \sum_{s'} T(s' \mid s,a) U_\theta(s') \right) \tag{8.1}$$

值函数近似常用于连续状态空间的问题中，此时上述求和可以被积分替代。该积分可以使用转移模型样本进行近似。

公式 (8.1) 中计算的另一种替代方案是近似动作价值函数 $Q(s、a)$。如果我们使用 $Q_\theta(s、a)$ 来表示我们的参数化

近似地，我们可以根据下式获得一个动作

$$\pi(s) = \arg\max_a Q_\theta(s, a) \tag{8.2}$$

本章讨论如何在有限状态集 $S = s_{1:m}$ 上应用动态规划，以得到值函数在整个状态空间上的参数化近似。可以使用不同的方案来生成这个集合。如果状态空间维度相对较低，我们可以定义一个网格。另一种方法是从状态空间中进行随机采样。然而，有些状态比其他状态更可能被遇到，因此在构建值函数时更为重要。我们可以通过使用某种策略（也许最初是随机的）从一组合理的初始状态运行模拟，使采样偏向更重要的状态。

可以使用迭代方法来增强我们在S中状态处的价值函数近似。我们交替进行通过动态规划改进我们在S处的价值估计，并在这些状态处重新拟合我们的近似。算法8.1提供了一种实现，其中动态规划步骤由价值迭代中所做的贝尔曼备份组成（见7.5节）。可以为动作价值近似Q_θ创建类似的算法。[2]

2 其他几类优化值函数近似的方法由A. Geramifard、T. J. Walsh、S. Tellex、G. Chowdhary、N. Roy和J. P. How在“动态规划与强化学习的线性函数近似教程”中综述，*Foundations and Trends in Machine Learning*，第6卷，第4期，第375-451页，2013年。

```
struct ApproximateValueIteration
    U0    # initial parameterized value function that supports fit!
    S     # set of discrete states for performing backups
    k_max # maximum number of iterations
end

function solve(M::ApproximateValueIteration, 𝒫::MDP)
    U0, S, k_max = M.U0, M.S, M.k_max
    for k in 1:k_max
        U = [backup(𝒫, U0, s) for s in S]
        fit!(U0, S, U)
    end
    return ValueFunctionPolicy(𝒫, U0)
end
```

算法8.1. 参数化值函数近似为`U0`的MDP的近似值迭代。我们在`S`中的状态执行备份（定义在算法7.7中），得到一个效用向量`U`。然后我们调用`fit!(U0, S, U)`，它修改参数表示`U0`，以更好地匹配`S`中的状态值与`U`中的效用。不同的参数化近似对`fit!`有不同的实现。

本章讨论的所有参数化表示都可以与算法 8.1 一起使用。要与该算法一起使用，一种表示需要支持对 U_θ 的评估，以及对 S 中各个点处的效用估计值拟合 U_θ。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们可以将参数表示分为两类。第一类包括*local approximation*方法，其中θ对应于S中各状态下的值。为了在任意状态s下评估$U_\theta(s)$，我们对存储在S中的值进行加权求和。第二类包括*global approximation*方法，其中θ与S中各状态下的值没有直接关系。事实上，θ的分量可能远少于甚至远多于S中的状态数。

局部近似和许多全局近似都可以视为一个*linear function approximation* $U_\theta(s) = \theta^\top \beta(s)$，其中方法的不同之处在于它们如何定义向量函数$\beta$。在局部近似方法中，$\beta(s)$决定了如何对$S$中状态的效用进行加权，以近似状态$s$处的效用。权重通常为非负且总和为1。在许多全局近似方法中，$\beta(s)$被视为一组基函数，这些基函数以线性方式组合，以获得任意s的近似。

我们也可以使用线性函数来近似动作价值函数，$Q_\theta(s, a) = \theta^\top \beta(s, a)$。在局部近似的背景下，我们可以通过选择一组有限的动作 $A \subset \mathcal{A}$ 来在连续动作空间上提供近似。我们的参数向量 θ 将由 $|S| \times |A|$ 个分量组成，每个分量对应一个状态-动作值。我们的函数 $\beta(s, a)$ 将返回一个具有相同数量分量的向量，该向量指定如何将我们的有限状态-动作值集合加权组合，以获得与状态 s 和动作 a 相关的效用估计。

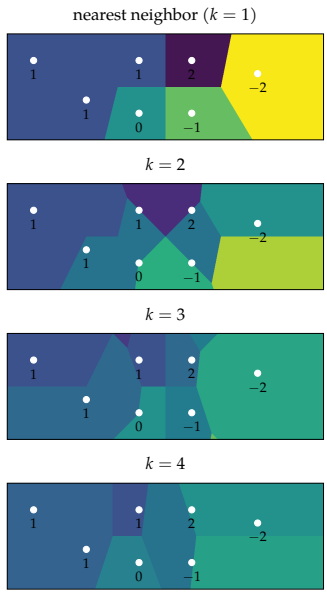


图8.1。在二维连续状态空间中，通过计算每个状态点根据欧几里得距离最近的k个邻居的效用值的平均值来近似状态值。由此得到的值函数是分段常数。

8.2 Nearest Neighbor

一种简单的局部近似方法是使用 S 中状态的值，即 s 的 *nearest neighbor*。为了使用这种方法，我们需要一个 *distance metric*（见附录A.3）。我们用 $d(s, s')$ 来表示两个状态 s 和 s' 之间的距离。近似价值函数则是 $U_\theta(s) = \theta_i$，其中 $i = \arg\min_{j \in 1:m} d(s_j, s)$。图8.1展示了使用最近邻方案表示的价值函数的一个例子。

我们可以将这种方法推广，以平均k-*nearestneighbors*的值。这种方法仍然会产生分段常值函数，但为k取不同的值可以得到更好的近似。图8.1展示了使用不同k值近似得到的价值函数示例。算法8.2给出了这一方法的实现。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
mutable struct NearestNeighborValueFunction
    k # number of neighbors
    d # distance function d(s, s′)
    S # set of discrete states
    θ # vector of values at states in S
end

function (U0::NearestNeighborValueFunction)(s)
    dists = [U0.d(s,s′) for s′ in U0.S]
    ind = sortperm(dists)[1:U0.k]
    return mean(U0.θ[i] for i in ind)
end

function fit!(U0::NearestNeighborValueFunction, S, U)
    U0.θ = U
    return U0
end
```

算法8.2。k最近邻方法，该方法基于`S`中由距离函数`d`确定的`k`个最近状态来近似状态`s`的值。向量`θ`包含`S`中状态的值。通过使用专用数据结构，例如在`NearestNeighbors.jl`中实现的kd树，可以实现更高的效率。

8.3 Kernel Smoothing

另一种局部近似方法是*kernel smoothing*，其中S中的状态的效用在整个状态空间上被平滑。这种方法需要定义一个*kernel function* $k(s, s')$，它将状态对s和s'关联起来。我们通常希望对于更接近的状态，$k(s, s')$更高，因为这些值告诉我们在S中与状态相关联的效用如何加权在一起。这种方法产生以下线性近似：

$$U_\theta(s) = \sum_{i=1}^{m} \theta_i \beta_i(s) = \theta^\top \beta(s) \tag{8.3}$$

其中

$$\beta_i(s) = \frac{k(s, s_i)}{\sum_{j=1}^{m} k(s, s_j)} \tag{8.4}$$

算法8.3提供了这一点的实现。

定义核函数的方法有很多种。我们可以将核函数简单地定义为状态之间距离的倒数：

$$k(s, s') = \max(d(s, s'), \epsilon)^{-1} \tag{8.5}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

其中 ϵ 是一个小的正常数，以避免在 $s = s'$ 时除零。图8.2展示了使用几种距离函数的值近似。正如我们所见，核平滑可以产生平滑的值函数近似，与 k-最近邻相比。图8.3将这一核应用于离散六边形世界问题，并展示了近似值迭代（算法8.1）的几次迭代结果。图8.4显示了针对连续状态空间的山地车问题（附录F.4）学习到的值函数和策略。

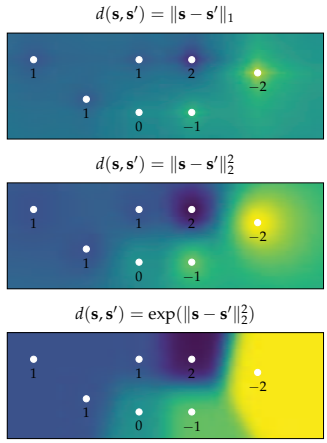


图8.2. 通过基于与几个已知值的状态的接近程度分配值，来近似二维连续状态空间中状态的值。近似是使用几种距离函数构建的。

另一个常见的核是 *Gaussian kernel*：

$$k(s, s') = \exp\left(-\frac{d(s, s')^2}{2\sigma^2}\right) \tag{8.6}$$

其中 σ 控制平滑程度。

```
mutable struct LocallyWeightedValueFunction
    k # kernel function k(s, s′)
    S # set of discrete states
    θ # vector of values at states in S
end

function (U0::LocallyWeightedValueFunction)(s)
    w = normalize([U0.k(s,s′) for s′ in U0.S], 1)
    return U0.θ ⋅ w
end

function fit!(U0::LocallyWeightedValueFunction, S, U)
    U0.θ = U
    return U0
end
```

算法8.3. 局部加权值函数近似，由核函数`k`和状态`S`中的效用向量`θ`定义。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

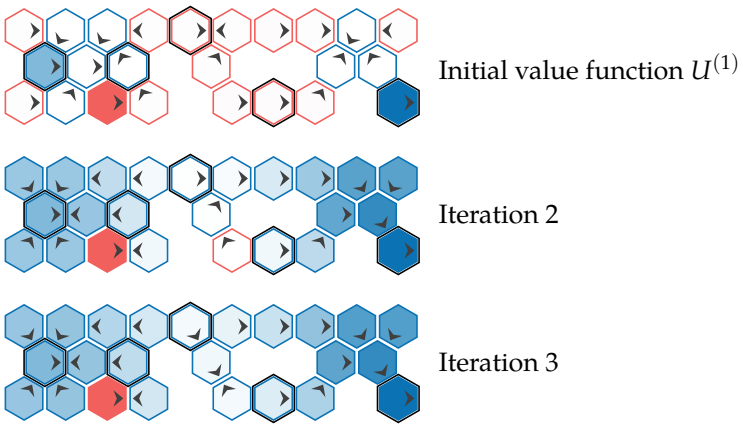


图8.3. 局部近似值迭代用于在六边形世界问题上迭代改进近似价值函数。五个勾勒出的状态用于近似价值函数。其余状态的价值使用距离函数$\|s - s'\|_2^2$进行近似。由此产生的策略合理但并非最优。正奖励以蓝色显示，负奖励以红色显示。

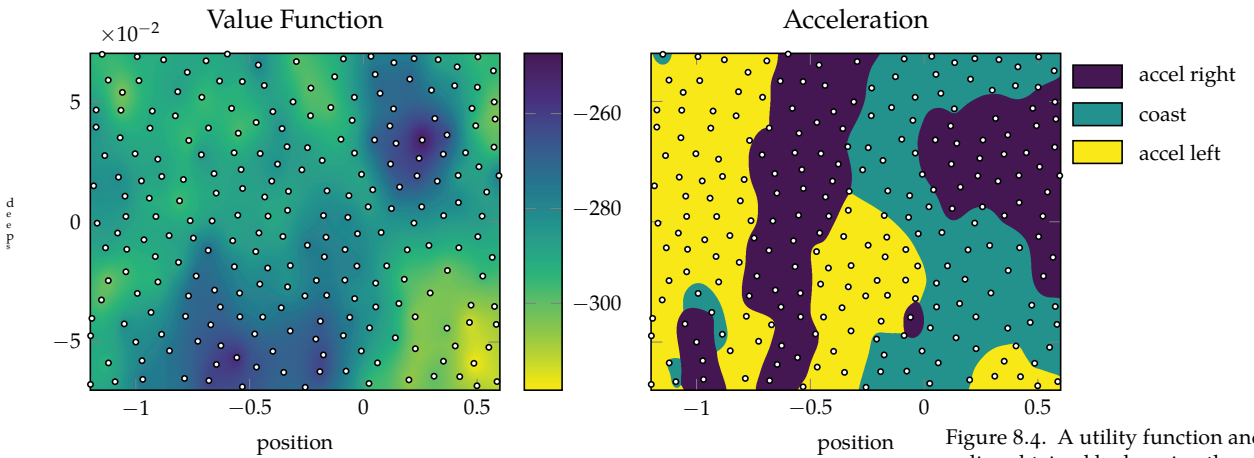


Figure 8.4. A utility function and policy obtained by learning the action values for a finite set of states (white) in the mountain car problem using the distance function $\|\mathbf{s} - \mathbf{s}'\|_2 + 0.1$.

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

8.4 Linear Interpolation

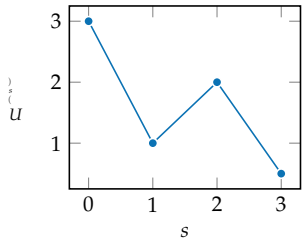


图8.5。一维线性插值沿连接两点的线段产生插值结果。

Linear interpolation 是另一种常见的局部近似方法。一维情况很直接，其中状态s在状态s_1和s_2之间的近似值为

$$U_\theta(s) = \alpha\theta_1 + (1-\alpha)\theta_2 \tag{8.7}$$

伴随 $\alpha = (s_2 - s)/(s_2 - s_1)$。此情况如图8.5和图8.6所示。

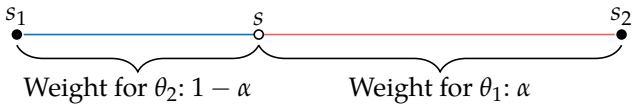
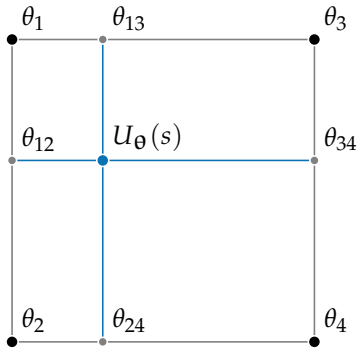


图8.6. 在一维空间中，分配给每个点的权重与插值状态相对侧线段的长度成正比。

线性插值可以扩展到多维网格。在二维情况下，称为*bilinear interpolation*，我们在四个顶点之间进行插值。双线性插值通过单维线性插值完成，每个轴上进行一次，需要利用网格顶点处的四个状态。这种插值方法如图8.7所示。



θ_{12} = 1D interpolation between θ_1 and θ_2 along the vertical axis
θ_{24} = 1D interpolation between θ_2 and θ_4 along the horizontal axis
θ_{13} = 1D interpolation between θ_1 and θ_3 along the horizontal axis
θ_{34} = 1D interpolation between θ_3 and θ_4 along the vertical axis

$$U_\theta(s) = \begin{cases} \text{1D interpolation between } \theta_{12} \text{ and } \theta_{34} \text{ along the horizontal axis} \\ \text{or} \\ \text{1D interpolation between } \theta_{13} \text{ and } \theta_{24} \text{ along the vertical axis} \end{cases}$$

图8.7. 二维网格上的线性插值是通过依次在每个轴上进行线性插值来实现的，顺序不限。

给定四个顶点，坐标分别为 $s_1 = [x_1, y_1]$、$s_2 = [x_1, y_2]$、$s_3 = [x_2, y_1]$ 和 $s_4 = [x_2, y_2]$，以及一个样本状态 $s = [x, y]$，插值结果为

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

$$
\begin{aligned}
U_\theta(s) &= \alpha\theta_{12} + (1-\alpha)\theta_{34} && (8.8)\\
&= \frac{x_2 - x}{x_2 - x_1}\theta_{12} + \frac{x - x_1}{x_2 - x_1}\theta_{34} && (8.9)\\
&= \frac{x_2 - x}{x_2 - x_1}(\alpha\theta_1 + (1-\alpha)\theta_2) + \frac{x - x_1}{x_2 - x_1}(\alpha\theta_3 + (1-\alpha)\theta_4) && (8.10)\\
&= \frac{x_2 - x}{x_2 - x_1}\left(\frac{y_2 - y}{y_2 - y_1}\theta_1 + \frac{y - y_1}{y_2 - y_1}\theta_2\right) + \frac{x - x_1}{x_2 - x_1}\left(\frac{y_2 - y}{y_2 - y_1}\theta_3 + \frac{y - y_1}{y_2 - y_1}\theta_4\right) && (8.11)\\
&= \frac{(x_2 - x)(y_2 - y)}{(x_2 - x_1)(y_2 - y_1)}\theta_1 + \frac{(x_2 - x)(y - y_1)}{(x_2 - x_1)(y_2 - y_1)}\theta_2 + \frac{(x - x_1)(y_2 - y)}{(x_2 - x_1)(y_2 - y_1)}\theta_3 + \frac{(x - x_1)(y - y_1)}{(x_2 - x_1)(y_2 - y_1)}\theta_4 && (8.12)
\end{aligned}
$$

所得插值根据每个顶点对向象限的面积对其加权，如图8.8所示。

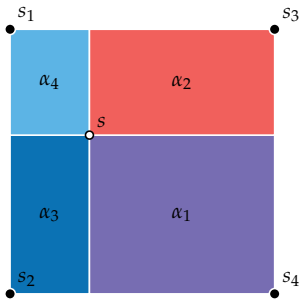


图 8.8. 在二维网格上进行线性插值时，每个顶点的贡献等于其相对象限的相对面积：$U_\theta(s) = \alpha_1\theta_1 + \alpha_2\theta_2 + \alpha_3\theta_3 + \alpha_4\theta_4$。

Multilinear interpolation 在d维中，类似地通过沿每个轴线性插值实现，需要2^d个顶点。此处，每个顶点的效用根据相对超矩形的体积加权。多线性插值在算法8.4中实现。图8.9展示了这种方法在二维状态空间上的应用。

8.5 Simplex Interpolation

多线性插值在高维情况下可能效率低下。与对2^d个点进行加权贡献不同，*simplex interpolation*只考虑给定状态邻域内的$d+1$个点，以生成与已知样本点匹配的连续曲面。

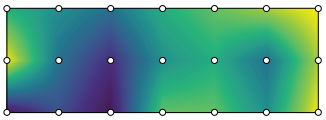


图8.9. 在3 × 7网格上的二维线性插值。

我们从一个多维网格开始，将每个单元格划分为$d!$ *simplexes*，这些是由$d+1$个顶点的*convex hull*定义的多维三角形推广。这个过程被称为*Coxeter-Freudenthal-Kuhn triangulation*，[3]并且它确保任何两个共享一个面的单纯形在该面上产生等值，从而在插值时产生连续性，如图8.10所示。

3 A. W. Moore, "Simplicial Mesh Generation with Applications"，博士学位论文，康奈尔大学，1992年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
mutable struct MultilinearValueFunction
    o # position of lower-left corner
    δ # vector of widths
    θ # vector of values at states in S
end

function (U0::MultilinearValueFunction)(s)
    o, δ, θ = U0.o, U0.δ, U0.θ
    Δ = (s - o)./δ
    # Multidimensional index of lower-left cell
    i = min.(floor.(Int, Δ) .+ 1, size(θ) .- 1)
    vertex_index = similar(i)
    d = length(s)
    u = 0.0
    for vertex in 0:2^d-1
        weight = 1.0
        for j in 1:d
            # Check whether jth bit is set
            if vertex & (1 << (j-1)) > 0
                vertex_index[j] = i[j] + 1
                weight *= Δ[j] - i[j] + 1
            else
                vertex_index[j] = i[j]
                weight *= i[j] - Δ[j]
            end
        end
        u += θ[vertex_index...]*weight
    end
    return u
end

function fit!(U0::MultilinearValueFunction, S, U)
    U0.θ = U
    return U0
end
```

算法8.4。一种进行多线性插值以估计状态向量 `s` 的值的方法，对于已知状态值 `θ` 在由左下顶点 `o` 和宽度向量 `δ` 定义的网格上。网格的顶点均可写为 `o + δ.*i`，对于某个非负整数向量 `i`。该软件包 `Interpolations.jl` 还提供了多线性和其他插值方法。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

为了说明这一点，假设我们已经对包含状态的单元进行了平移和缩放，使得最低顶点为0，对角相对的顶点为1。对于1的每个排列：d，都有一个单纯形。由排列p给出的单纯形是满足以下条件的点x的集合：

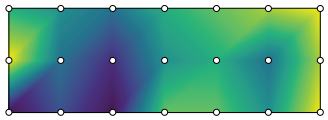


图8.10. 在3 × 7网格上的二维单纯形插值。

$$0 \leq x_{p_1} \leq x_{p_2} \leq \cdots \leq x_{p_d} \leq 1 \tag{8.13}$$

图8.11显示了针对单位立方体所获得的单纯形。

单纯形插值首先将状态向量 s 平移和缩放至其对应单元格的单位超立方体，以获得 s{v*}。然后，对 s{v*} 中的元素进行排序，以确定包含 s{v*} 的单纯形。s{v*} 处的效用随后可以通过该单纯形顶点的唯一线性组合来表示。

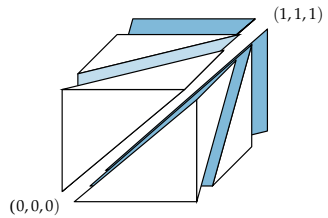


图8.11. 单位立方体的三角剖分。依据A. W. Moore的博士论文《具有应用的单纯形网格生成》，康奈尔大学，1992年，图2.1。

示例8.1给出了单纯形插值的一个例子。该过程在算法8.5中实现。

考虑由排列 p = [3, 1, 2] 给出的三维单纯形，使得单纯形内的点满足 0 ≤ x_3 ≤ x_1 ≤ x_2 ≤ 1。该单纯形的顶点为 (0, 0, 0)、(0, 1, 0)、(1, 1, 0) 和 (1, 1, 1)。

任何属于单纯形的点s因此可以通过顶点的加权来表示：

$$\begin{bmatrix} s_1 \\ s_2 \\ s_3 \end{bmatrix} = w_1 \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + w_2 \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} + w_3 \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} + w_4 \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

我们可以依次确定最后三个权重的值：

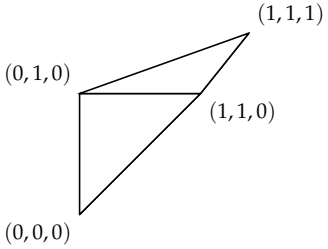
$$w_4 = s_3 \qquad w_3 = s_1 - w_4 \qquad w_2 = s_2 - w_3 - w_4$$

我们通过强制权重之和为1来得到w_1。

如果 s = [0.3, 0.7, 0.2]，则权重为

$$w_4 = 0.2 \qquad w_3 = 0.1 \qquad w_2 = 0.4 \qquad w_1 = 0.3$$

例8.1. 三维空间中的单纯形插值。



© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
mutable struct SimplexValueFunction
    o # position of lower-left corner
    δ # vector of widths
    θ # vector of values at states in S
end

function (U0::SimplexValueFunction)(s)
    Δ = (s - U0.o)./U0.δ
    # Multidimensional index of upper-right cell
    i = min.(floor.(Int, Δ) .+ 1, size(U0.θ) .- 1) .+ 1
    u = 0.0
    s′ = (s - (U0.o + U0.δ.*(i.-2))) ./ U0.δ
    p = sortperm(s′) # increasing order
    w_tot = 0.0
    for j in p
        w = s′[j] - w_tot
        u += w*U0.θ[i...]
        i[j] -= 1
        w_tot += w
    end
    u += (1 - w_tot)*U0.θ[i...]
    return u
end

function fit!(U0::SimplexValueFunction, S, U)
    U0.θ = U
    return U0
end
```

算法8.5。一种用于进行单纯形插值的方法，以估计状态向量`s`的值，该值基于已知的状态值`θ`，在一个由左下顶点`o`和宽度向量`δ`定义的网格上。网格的所有顶点可以表示为`o + δ.*i`，其中`i`是非负整数向量。单纯形插值也在通用`GridInterpolations.jl`包中实现。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

8.6 Linear Regression

一种简单的全局近似方法是*linear regression*，其中$U_\theta(s)$是*basis functions*的线性组合，通常也称为*features*。这些基函数通常是状态s的非线性函数，并被组合成一个向量函数$\beta(s)$或$\beta(s、a)$，从而得到近似

$$U_\theta(s) = \theta^\top \beta(s) \qquad Q_\theta(s,a) = \theta^\top \beta(s,a) \tag{8.14}$$

尽管我们的近似对于基函数是线性的，但对于潜在的状态变量来说，所得到的近似可能是非线性的。图8.12展示了这一概念。例8.2提供了一个使用多项式基函数对连续山地车问题进行全局线性价值近似的例子，其结果是对状态变量的非线性价值函数近似。

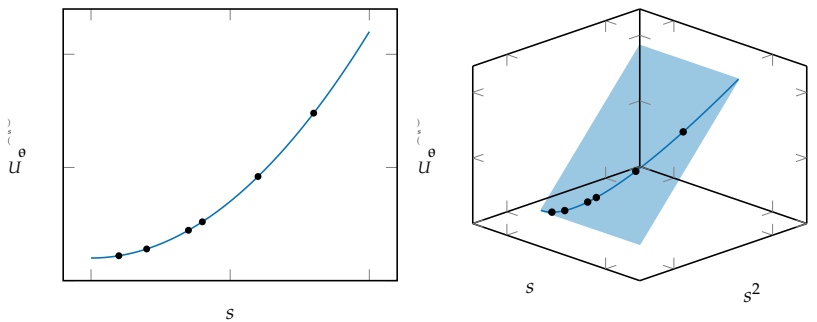


图8.12. 非线性基函数的线性回归在高维空间中是线性的。这里，多项式回归可以看作是在三维空间中的线性。该函数存在于由其基构成的平面上，但由于各项并非独立，它并未占据整个平面。

增加更多的基函数通常能提高在S中状态匹配目标效能的能力，但过多的基函数可能导致在其他状态下的近似效果不佳。存在一些原则性的方法为我们的回归模型选择合适的基函数集。[4]

拟合线性模型涉及确定向量 θ，使其在状态 $S = s_{1:m}$ 下的预测平方误差最小。如果与这些状态相关的效用记为 $u_{1:m}$，那么我们要找到使下式最小化的 θ。

$$\sum_{i=1}^{m} (\hat{U}_\theta(s_i) - u_i)^2 = \sum_{i=1}^{m} (\theta^\top \beta(s_i) - u_i)^2 \tag{8.15}$$

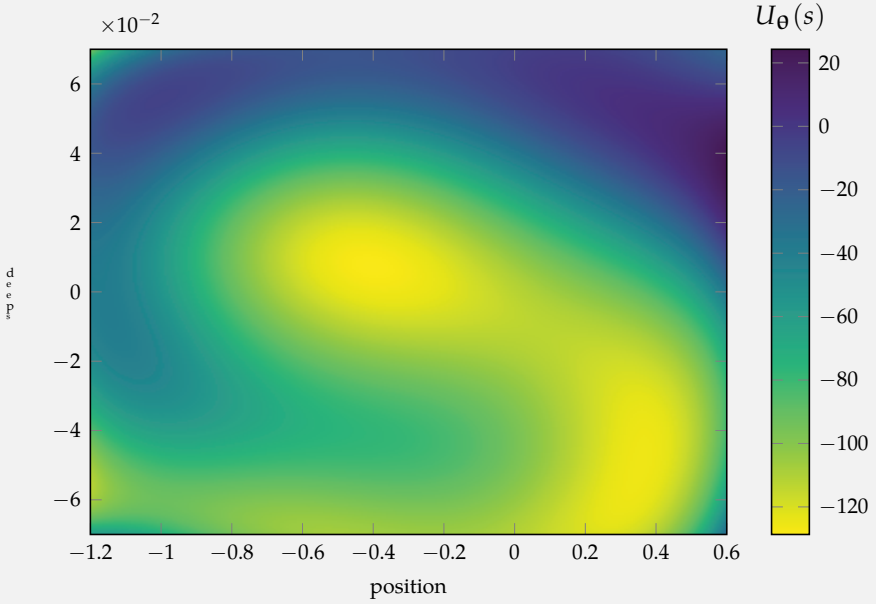
4 参见 M. J. Kochenderfer 和 T. A. Wheeler 的 *Algorithms for Optimization* 第14章。MIT Press，2019年。或参见 T. Hastie、R. Tibshirani 和 J. Friedman 的 *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* 第7章，第2版，Springer Series in Statistics，2001年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们可以使用线性近似来近似山地车问题的价值函数。该问题具有一个连续的状态空间，由位置 x 和速度 v 两个维度组成。以下是高达六次的基函数：

$$\boldsymbol{\beta}(s) = \begin{bmatrix} 1, & & & & & & \\ x, & v, & & & & & \\ x^2, & xv, & v^2, & & & & \\ x^3, & x^2v, & xv^2, & v^3, & & & \\ x^4, & x^3v, & x^2v^2, & xv^3, & v^4, & & \\ x^5, & x^4v, & x^3v^2, & x^2v^3, & xv^4, & v^5, & \\ x^6, & x^5v, & x^4v^2, & x^3v^3, & x^2v^4, & xv^5, & v^6 \end{bmatrix}$$

这是一个近似值函数拟合专家策略状态-价值对的图表：



示例 8.2. 使用线性近似来估计山地车的价值函数。基函数的选择会产生很大的差异。山地车的最优价值函数是非线性的，具有螺旋形状和不连续性。即使是六次多项式也无法产生完美的拟合。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

最优的 {v*} 可以通过一些简单的矩阵运算来计算。我们首先构造一个矩阵 X，其中 m 行 $X_{i,:}$ 中的每一行都包含 $\beta(s_i)^\top$。[5] 它可以可以证明，使平方误差最小化的 θ 的值为

$$\theta = \left(X^\top X\right)^{-1} X^\top u_{1:m} = X^+ u_{1:m} \tag{8.16}$$

其中 X^+ 是矩阵 X 的 *Moore-Penrose pseudoinverse*。伪逆通常通过首先计算 *singular value decomposition*，$X = U\Sigma U^*$ 来实现。然后我们有

$$X^+ = U\Sigma^+ U^* \tag{8.17}$$

对角矩阵Σ的伪逆是通过取对角线上每个非零元素的倒数，然后转置结果得到的。

图8.13展示了S中状态的效用如何用几个基函数族进行拟合。基函数的不同选择导致不同的误差。

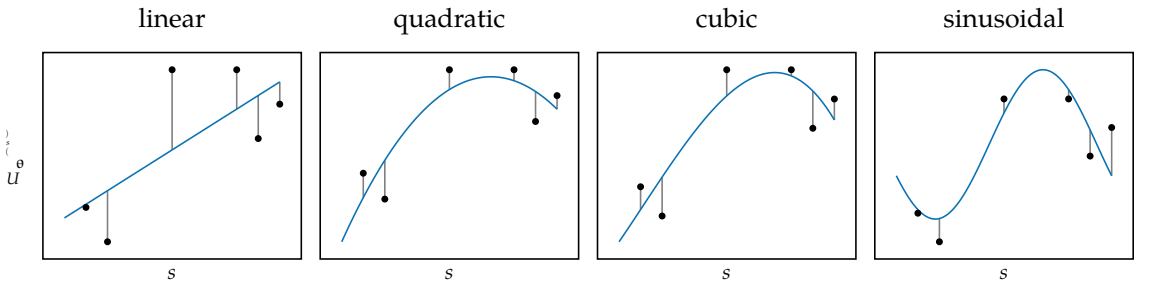


图 8.13. 不同基函数族的线性回归。

算法8.6提供了评估和拟合价值函数线性回归模型的一种实现方法。示例8.3通过山地车问题展示了这一方法。

8.7 Neural Network Regression

Neural network regression 使我们无需像线性回归那样构建一组适当的基函数。相反，使用*neural network*来表示我们的值函数。关于神经网络的综述，请参见附录D。神经网络的输入将是状态变量，输出将是效用估计值。参数θ将对应于神经网络中的*weights*。

5 关于线性回归所涉及的数学以及更高级技术的概述，参见 T. Hastie、R. Tibshirani 和 J. Friedman，*The Elements of Statistical Learning, Data Mining, Inference, and Prediction*，第2版。Springer-Verlag，2001年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
mutable struct LinearRegressionValueFunction
    β # basis vector function
    θ # vector of parameters
end

function (U0::LinearRegressionValueFunction)(s)
    return U0.β(s) ⋅ U0.θ
end

function fit!(U0::LinearRegressionValueFunction, S, U)
    X = hcat([U0.β(s) for s in S]...)'
    U0.θ = pinv(X)*U
    return U0
end
```

算法8.6. 由基向量函数β和参数向量θ定义的线性回归价值函数近似。函数pinv实现了伪逆。Julia和其他语言支持*backslash*运算符，这允许我们在fit!函数中用X \ U代替pinv(X)*U。

如附录D所述，我们可以优化网络权重以实现特定目标。在近似动态规划的背景下，我们希望最小化预测误差，正如我们在上一节所做的那样。然而，最小化平方误差不能通过简单的矩阵运算完成。相反，我们通常必须依赖诸如梯度下降等优化技术。幸运的是，计算神经网络的梯度可以通过直接应用导数链式法则来精确完成。

8.8 Summary

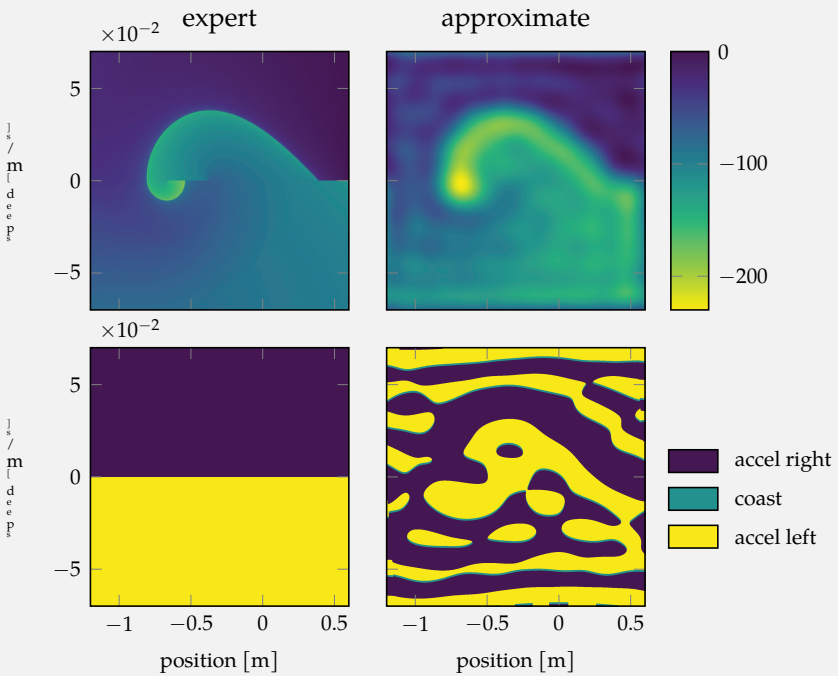
- 对于大型或连续问题，我们可以尝试寻找由价值函数的参数化模型表示的近似策略。
- 本章所采取的方法涉及在有限状态集上迭代应用动态规划的步骤，并改进我们的参数近似。
- 局部近似技术根据已知值的邻近状态的值来近似值函数。
- 多种局部近似技术包括最近邻、核平滑、线性插值和单纯形插值。
- 全局近似技术包括线性回归和神经网络回归。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们可以应用线性回归来为山地车问题学习一个价值函数。最优价值函数呈螺旋形式，这很难用多项式基函数近似（见例8.2）。我们使用傅里叶基函数，其组成部分采用以下形式：

$$
\begin{aligned}
b_0(x) &= 1/2 \\
b_{s,i}(x) &= \sin(2\pi i x/T) \text{ for } i = 1, 2, \ldots \\
b_{c,i}(x) &= \cos(2\pi i x/T) \text{ for } i = 1, 2, \ldots
\end{aligned}
$$

其中 T 是组件域的宽度。多维傅里叶基函数是状态空间轴上所有一维分量的组合。这里我们采用八阶近似，因此 i 的范围可达 8。专家策略是沿运动方向加速。



示例 8.3. 使用傅里叶基进行线性回归来近似山地车问题（附录 F.4）的价值函数。展示了价值函数（顶行）及由此产生的策略（底行）。尽管使用了八阶傅里叶基函数，全局近似的价值函数拟合效果仍不佳。由此产生的近似策略与专家策略相差甚远。山地车问题中的小时间步长导致价值函数景观的微小变化都会影响策略。最优效用函数往往具有复杂的几何形态，这使得全局基函数难以捕捉。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

- 当线性回归与适当的非线性基函数选择相结合时，可以获得非线性效用函数。
- 神经网络回归使我们无需指定基函数，但拟合它们更为复杂，通常需要我们使用梯度下降来调整对值函数的参数化近似。

8.9 Exercises

练习 8.1。本章介绍的价值函数近似方法大多假设状态空间是连续的。六边形世界问题，附录 F.1，是离散的，但其大多数状态可以映射到二维位置。然而，它确实有一个额外的终止状态，产生零奖励，该状态没有二维位置。如何修改本章中的连续价值函数近似方法以处理这样的状态?

Solution: 六边形世界问题要求智能体在二维六边形网格中导航。然而，智能体可以从几个网格六边形之一进入一个单一终止状态。这个单一终止状态对价值函数近似方法提出了挑战，因为这些方法通常依赖邻近性来推断状态的价值。

虽然终端状态可以投影到与其他状态相同的状态空间，可能相距甚远，但这种做法仍会在终端状态的价值计算中强行引入一种邻近性。为一个本应与多个前驱状态等距的状态选择单一位置，会引入偏差。

一种替代方案是将终止状态视为特殊情况。可以修改核函数，使其产生终止状态与任何其他状态之间的无限距离。

另一个选项是调整问题，使得每个具有终止奖励的六边形都有一个终止状态。每个终止状态可以与其前驱状态重合，但需在额外维度上进行偏移。这种变换保持了邻近性，但代价是增加了状态数量。

练习 8.2。表格表示是线性近似值函数的一个特例。证明对于任何离散问题，表格表示都可以构造成线性近似值函数。

Solution: 考虑一个具有 m 个状态 $s_{1:m}$ 和 n 个动作 $a_{1:n}$ 的离散 MDP。表格表示将值与每个状态或状态-动作对相关联。我们可以使用线性近似值函数恢复相同的行为。我们将指示函数与每个状态或状态-动作对相关联，当输入为给定状态或状态-动作对时其值为 1，否则为 0：

$$\beta_i(s) = (s = s_i) = \begin{cases} 1 & \text{if } s = s_i \\ 0 & \text{otherwise} \end{cases}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

或

$$\beta_{ij}(s,a) = ((s,a) = (s_i,a_j)) = \begin{cases} 1 & \text{if } (s,a) = (s_i,a_j) \\ 0 & \text{otherwise} \end{cases}$$

练习 8.3. 假设我们面临一个具有连续状态和动作空间的问题，并且我们希望构建动作价值函数 $Q(s、a) = \theta^\top \beta(s、a)$ 的局部近似和全局近似。对于全局近似，我们选择基函数

$$\beta(s,a) = \left[1, s, a, s^2, sa, a^2\right]$$

给定一组100个状态$S = s_{1:100}$和一组五个动作$A = a_{1:5}$，对于局部近似方法，θ中有多少个参数？对于指定的全局近似方法，θ中有多少个参数？

Solution: 在局部近似方法中，状态-动作值是参数。在θ中，我们将有$|S| \times |A|$ =100×5=500个参数。在全局近似方法中，基函数的系数是参数。由于β(s中有六个分量，a)，我们将在θ中有六个参数。

练习 8.4。给定状态 $s_1 = (4, 5)$、$s_2 = (2, 6)$ 和 $s_3 = (-1、-1)$，以及它们对应的值 $U(s_1) = 2$、$U(s_2) = 10$ 和 $U(s_3) = 30$。使用 L_1 距离度量、L_2 距离度量和 L_∞ 距离度量，计算状态 $s = (1, 2)$ 处的值，采用 2-最近邻局部近似。

Solution: 我们将从s到点$s' \in S$的距离列在此处：

$s' \in S$	L_1	L_2	L_∞
$s_1 = (4,5)$	6	$\sqrt{18}$	3
$s_2 = (2,6)$	5	$\sqrt{17}$	4
$s_3 = (-1,-1)$	5	$\sqrt{13}$	3

使用 L_1 范数，我们估计 $U(s) = (10 + 30)/2 = 20$。使用 L_2 范数，我们估计 $U(s) = (10 + 30)/2 = 20$。使用 L_∞ 范数，我们估计 $U(s) = (2 + 30)/2 = 16$。

练习 8.5。我们希望根据两个状态 $S = \{s_1、s_2\}$ 的值来估计状态 s 的值。如果我们想使用局部近似值迭代，以下哪些加权函数是有效的？如果它们无效，如何修改这些加权函数使其有效？

- $\beta(s) = [1, 1]$
- $\beta(s) = [1 - \lambda, \lambda]$ 其中 $\lambda \in [0, 1]$
- $\beta(s) = \left[e^{(s-s}1_)2, e^{(s-s_2)^2}\right]$

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

Solution: 第一组加权函数无效，因为它违反了约束条件 $\sum_i \beta_i(s) = 1$。我们可以通过按其和进行归一化来修正加权函数：

$$\boldsymbol{\beta}(s) = \left[\frac{1}{1+1}, \frac{1}{1+1}\right] = \left[\frac{1}{2}, \frac{1}{2}\right]$$

第二组权重函数是有效的。第三组权重函数无效，因为它违反了约束条件 $\sum_i \beta_i(s) = 1$。我们可以通过按其总和归一化来修改权重函数：

$$\boldsymbol{\beta}(s) = \left[\frac{e^{(s-s_1)^2}}{e^{(s-s_1)^2}+e^{(s-s_2)^2}}, \frac{e^{(s-s_2)^2}}{e^{(s-s_1)^2}+e^{(s-s_2)^2}}\right]$$

练习8.6。证明双线性插值在（非零）线性网格缩放变换下保持不变。

Solution: 很容易证明，插值值对一轴或两轴上的线性缩放是不变的，例如，$\tilde{U}_{\boldsymbol{\theta}}(\tilde{s}) = U_{\boldsymbol{\theta}}(s)$。我们通过将所有 x- 和 y-值替换为其缩放版本 $\tilde{x} = \beta x$ 和 $\tilde{y} = \gamma y$，并证明网格缩放相互抵消来展示这一点：

$$\begin{aligned}
\tilde{U}_{\boldsymbol{\theta}}(\tilde{s}) &= \frac{(\tilde{x}_2 - \tilde{x})(\tilde{y}_2 - \tilde{y})}{(\tilde{x}_2 - \tilde{x}_1)(\tilde{y}_2 - \tilde{y}_1)}\theta_1 + \frac{(\tilde{x}_2 - \tilde{x})(\tilde{y} - \tilde{y}_1)}{(\tilde{x}_2 - \tilde{x}_1)(\tilde{y}_2 - \tilde{y}_1)}\theta_2 + \frac{(\tilde{x} - \tilde{x}_1)(\tilde{y}_2 - \tilde{y})}{(\tilde{x}_2 - \tilde{x}_1)(\tilde{y}_2 - \tilde{y}_1)}\theta_3 + \frac{(\tilde{x} - \tilde{x}_1)(\tilde{y} - \tilde{y}_1)}{(\tilde{x}_2 - \tilde{x}_1)(\tilde{y}_2 - \tilde{y}_1)}\theta_4 \\
\tilde{U}_{\boldsymbol{\theta}}(\tilde{s}) &= \frac{\beta(x_2 - x)\gamma(y_2 - y)}{\beta(x_2 - x_1)\gamma(y_2 - y_1)}\theta_1 + \frac{\beta(x_2 - x)\gamma(y - y_1)}{\beta(x_2 - x_1)\gamma(y_2 - y_1)}\theta_2 + \frac{\beta(x - x_1)\gamma(y_2 - y)}{\beta(x_2 - x_1)\gamma(y_2 - y_1)}\theta_3 + \frac{\beta(x - x_1)\gamma(y - y_1)}{\beta(x_2 - x_1)\gamma(y_2 - y_1)}\theta_4 \\
\tilde{U}_{\boldsymbol{\theta}}(\tilde{s}) &= \frac{(x_2 - x)(y_2 - y)}{(x_2 - x_1)(y_2 - y_1)}\theta_1 + \frac{(x_2 - x)(y - y_1)}{(x_2 - x_1)(y_2 - y_1)}\theta_2 + \frac{(x - x_1)(y_2 - y)}{(x_2 - x_1)(y_2 - y_1)}\theta_3 + \frac{(x - x_1)(y - y_1)}{(x_2 - x_1)(y_2 - y_1)}\theta_4 \\
\tilde{U}_{\boldsymbol{\theta}}(\tilde{s}) &= U_{\boldsymbol{\theta}}(s)
\end{aligned}$$

练习8.7. 给定四个状态 $s_1 = [0, 5]$、$s_2 = [0, 25]$、$s_3 = [1, 5]$ 和 $s_4 = [1, 25]$，以及一个样本状态 $s = [0.7, 10]$，为任意 $\boldsymbol{\theta}$ 生成插值方程 $U_{\boldsymbol{\theta}}(s)$。

Solution: 双线性插值的一般形式在公式（8.12）中给出，并在此重述。为了生成插值函数，我们将数值代入方程并进行简化：

$$\begin{aligned}
U_{\boldsymbol{\theta}}(s) &= \frac{(x_2 - x)(y_2 - y)}{(x_2 - x_1)(y_2 - y_1)}\theta_1 + \frac{(x_2 - x)(y - y_1)}{(x_2 - x_1)(y_2 - y_1)}\theta_2 + \frac{(x - x_1)(y_2 - y)}{(x_2 - x_1)(y_2 - y_1)}\theta_3 + \frac{(x - x_1)(y - y_1)}{(x_2 - x_1)(y_2 - y_1)}\theta_4 \\
U_{\boldsymbol{\theta}}(s) &= \frac{(1 - 0.7)(25 - 10)}{(1 - 0)(25 - 5)}\theta_1 + \frac{(1 - 0.7)(10 - 5)}{(1 - 0)(25 - 5)}\theta_2 + \frac{(0.7 - 0)(25 - 10)}{(1 - 0)(25 - 5)}\theta_3 + \frac{(0.7 - 0)(10 - 5)}{(1 - 0)(25 - 5)}\theta_4 \\
U_{\boldsymbol{\theta}}(s) &= \frac{9}{40}\theta_1 + \frac{3}{40}\theta_2 + \frac{21}{40}\theta_3 + \frac{7}{40}\theta_4
\end{aligned}$$

练习 8.8。按照例 8.1，对于状态的单纯形插值权重是什么？
$\mathrm{s} = [0.4, 0.95, 0.6]$?

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: 对于给定的状态 s，我们有 $0 \leq x_1 \leq x_3 \leq x_2 \leq 1$，因此我们的排列向量是 p = [1, 3, 2]。单纯形的顶点可以通过从 (0, 0, 0) 开始，并按照排列向量的逆序将每个 0 改为 1 来生成。因此，单纯形的顶点是 (0, 0, 0)、(0, 1, 0)、(0, 1, 1) 和 (1, 1, 1)。

任何属于单纯形的点s因此可以通过顶点的加权来表示：

$$\begin{bmatrix} s_1 \\ s_2 \\ s_3 \end{bmatrix} = w_1 \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + w_2 \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} + w_3 \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} + w_4 \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

我们可以按相反顺序确定权重的值，最后通过应用权重之和必须为1的约束来求解 w_1。然后，我们可以计算s = [0.4, 0.95, 0.6]的权重：

$$\begin{array}{llll} w_4 = s_1 & w_3 = s_3 - w_4 & w_2 = s_2 - w_3 - w_4 & w_1 = 1 - w_2 - w_3 - w_4 \\ w_4 = 0.4 & w_3 = 0.2 & w_2 = 0.35 & w_1 = 0.05 \end{array}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

9 Online Planning

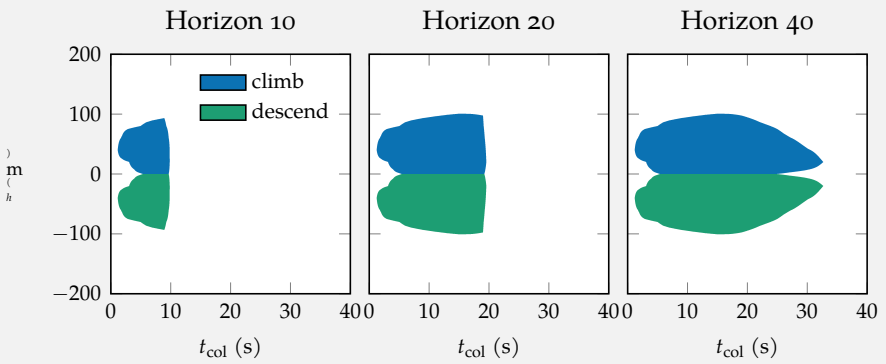
到目前为止，我们讨论的解决方法都是在实际问题执行任何动作之前，离线计算策略。即使是离线近似方法，在许多高维问题中也可能难以处理。本章讨论*online planning*方法，这些方法基于对从当前状态可达状态的推理来寻找动作。*reachable state space*通常比完整状态空间小几个数量级，与离线方法相比，这可以显著减少存储和计算需求。我们将讨论各种旨在使在线规划高效的算法，包括修剪状态空间、采样以及沿着看起来更有希望的轨迹进行更深入的规划。

9.1 Receding Horizon Planning

在*receding horizon planning*中，我们从当前状态规划到最大固定视界或深度d。然后，我们从当前状态执行动作，过渡到下一个状态，并重新规划。本章讨论的在线规划方法遵循这种滚动视界规划方案。它们的区别在于如何探索不同的行动方案。

应用滚动时域规划的一个挑战是确定合适的深度。更深的规划通常需要更多计算。对于某些问题，较浅的深度可能相当有效；我们在每一步重新规划这一事实可以弥补我们缺乏长期建模的不足。在其他问题中，可能需要更大的规划深度，以便我们的规划器能够被驱动向目标或远离不安全状态，如例9.1所示。

假设我们要将滚动时域规划应用于飞机防撞。目标是在必要时提供下降或爬升建议以避免碰撞。当相对于入侵者h的高度在± 50米以内且潜在碰撞时间t_{col}为零时，就会发生碰撞。我们需要进行足够深入的规划，以便能够足够早地提供建议，从而以高置信度避免碰撞。这里的图表展示了具有不同深度的滚动时域规划器将采取的行动。



如果深度为 $d = 10$，我们仅在碰撞前10秒内提供建议。由于车辆动力学的限制和其他飞机行为的不确定性，这么晚提供建议会危及安全。使用 $d = 20$时，我们可以做得更好，但在某些情况下，我们希望能稍早一些发出警报，以进一步降低碰撞风险。没有动机规划超过 $d = 40$的深度，因为我们不需要在潜在碰撞发生前那么早就建议任何机动。

例9.1 不同规划深度下用于避免碰撞的滚动时域规划。该问题涉及四个状态变量。这些图展示了在假设飞行器当前处于水平状态且尚未收到咨询的情况下状态空间的切片。横轴是碰撞时间 t_{col}，纵轴是我们相对于入侵者的高度 h。附录F.6提供了关于该问题的更多细节。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

9.2 Lookahead with Rollouts

第8章涉及通过使用一步前瞻来提取相对于近似值函数U的贪婪策略。[1]一个简单的在线
策略涉及根据通过模拟估计到深度d的价值贪婪地行动。为了运行模拟，我们需要一个用于模拟的策略。当然，我们不知道最优策略，但我们可以使用所谓的*rollout policy*代替。展开策略通常是随机的，动作从分布$a \sim \pi(s)$中抽取。为了生成这些展开模拟，我们使用*generative models* $s' \sim T(s, a)$来从分布$T(s' \mid s、a)$中生成后继状态s'。这种生成模型可以通过从随机数生成器中抽取来实现，与显式表示分布$T(s' \mid s、a)$相比，这在实践中可能更容易实现。

1 前瞻策略最初是在算法7.2中作为我们讨论精确解法方法的一部分引入的。

算法9.1将一步前瞻与通过rollout估计的值相结合。这种方法通常比原始rollout策略的行为更好，但并不能保证最优性。它可以被视为策略迭代算法（第7.4节）中使用的策略改进的一种近似形式。该算法的一个简单变体是使用多次rollout来获得对期望折扣回报的更好估计。如果我们对每个动作及结果状态运行m次模拟，时间复杂度为$O(m \times |\mathcal{A}| \times |\mathcal{S}| \times d)$。

9.3 Forward Search

Forward search 从初始状态 s 开始，通过扩展所有可能的转移直到深度 d，确定最佳行动。这些扩展形成一棵 *search tree*。[2] 此类搜索树的最坏情况分支因子为$|\mathcal{S}| \times |\mathcal{A}|$，得出
$O((|\mathcal{S}| \times |\mathcal{A}|)^d)$的计算复杂度。图9.1展示了应用于一个包含三个状态和两个动作的问题的搜索树。图9.2直观展示了在六边形世界问题进行前向搜索期间访问的状态。

2 树的探索以 *depth-first search* 的形式进行。附录E在确定性背景下回顾了深度优先搜索和其他标准搜索算法。

算法9.2递归调用自身至指定深度。一旦达到指定深度，它就使用由函数U提供的效用估计。如果我们仅仅想规划到指定的时间范围，我们将$U(s) =$设为0。如果我们的问题需要规划超出我们能够在线计算承担的深度，我们可以使用离线获得的价值函数估计，例如，使用上一章描述的价值函数近似之一。结合

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct RolloutLookahead
    𝒫 # problem
    π # rollout policy
    d # depth
end

randstep(𝒫::MDP, s, a) = 𝒫.TR(s, a)

function rollout(𝒫, s, π, d)
    ret = 0.0
    for t in 1:d
        a = π(s)
        s, r = randstep(𝒫, s, a)
        ret += 𝒫.γ^(t-1) * r
    end
    return ret
end

function (π::RolloutLookahead)(s)
    U(s) = rollout(π.𝒫, s, π.π, π.d)
    return greedy(π.𝒫, U, s).a
end
```

算法9.1。一个在问题$\mathcal{P}$中从状态s执行策略π的rollout到深度d的函数。它返回总折扣奖励。该函数可与算法7.5中引入的`greedy`函数一起使用，以生成一个可能优于原始rollout策略的动作。我们稍后会将此算法用于MDP以外的问题，这要求我们只需适当地修改`randstep`。

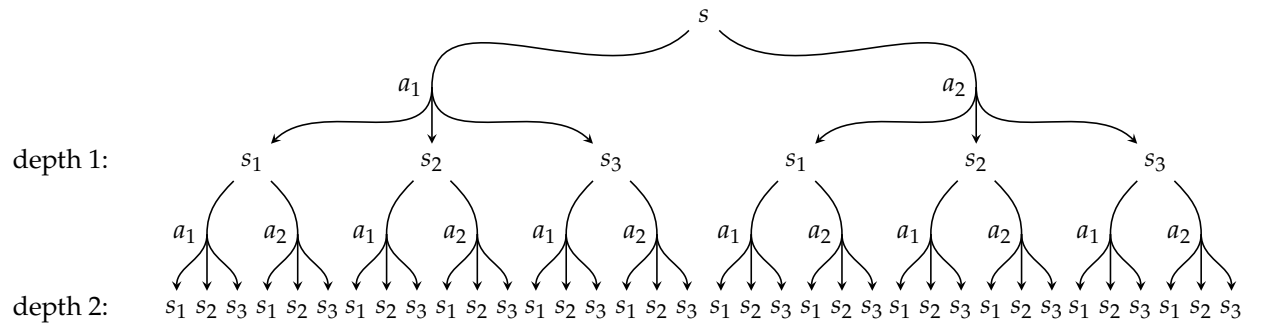


图9.1. 一个具有三种状态和两种动作问题的前向搜索树。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

以这种方式结合在线和离线方法有时被称为*hybrid planning*。

```
struct ForwardSearch
    𝒫 # problem
    d # depth
    U # value function at depth d
end

function forward_search(𝒫, s, d, U)
    if d ≤ 0
        return (a=nothing, u=U(s))
    end
    best = (a=nothing, u=-Inf)
    U′(s) = forward_search(𝒫, s, d-1, U).u
    for a in 𝒫.𝒜
        u = lookahead(𝒫, U′, s, a)
        if u > best.u
            best = (a=a, u=u)
        end
    end
    return best
end

(π::ForwardSearch)(s) = forward_search(π.𝒫, s, π.d, π.U).a
```

算法9.2. 针对当前状态 `s` 中的问题 $\mathcal{P}$，在线寻找近似最优动作的前向搜索算法。搜索执行至深度 `d`，此时通过近似价值函数 `U` 估计终端值。返回的命名元组由最佳动作 `a` 及其有限视界期望值 `u` 组成。问题类型不限于MDP；第22.2节在部分可观测问题的背景下使用同一算法，但对 `lookahead` 采用了不同的实现。

9.4 Branch and Bound

Branch and bound (算法9.3)试图避免前向搜索的指数级计算复杂度。它通过推理值函数的边界来修剪分支。该算法需要知道值函数的下界$\underline{U}(s)$以及动作值函数的上界$Q(s、a)$。下界用于评估最大深度处的状态。这个下界通过贝尔曼更新向上传播到树中。如果我们发现某个状态下某个动作的上界低于该状态下先前已探索动作的下界，那么我们无需探索该动作，从而可以将相关的子树*prune*从考虑中排除。

分支定界将给出与正向搜索相同的结果，但其效率可能会更高，具体取决于被剪枝的分支数量。分支定界的最坏情况复杂度仍与正向搜索相同。为了便于

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

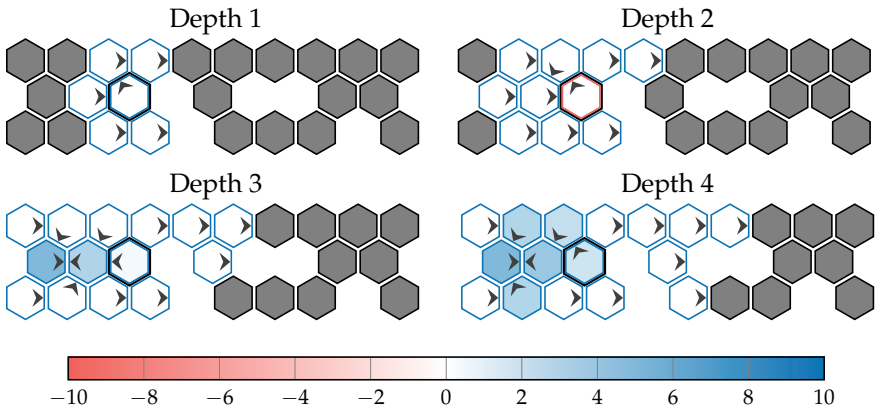


图9.2. 应用于六边形世界问题的前向搜索，最大深度为四。搜索可以多次访问一个节点。对于已访问状态的动作和颜色，是根据搜索树中该状态最浅且价值最高的节点来选择的。初始状态有一个额外的黑色边框。

```
struct BranchAndBound
    𝒫   # problem
    d   # depth
    Ulo # lower bound on value function at depth d
    Qhi # upper bound on action value function
end

function branch_and_bound(𝒫, s, d, Ulo, Qhi)
    if d ≤ 0
        return (a=nothing, u=Ulo(s))
    end
    U′(s) = branch_and_bound(𝒫, s, d-1, Ulo, Qhi).u
    best = (a=nothing, u=-Inf)
    for a in sort(𝒫.𝒜, by=a->Qhi(s,a), rev=true)
        if Qhi(s, a) < best.u
            return best # safe to prune
        end
        u = lookahead(𝒫, U′, s, a)
        if u > best.u
            best = (a=a, u=u)
        end
    end
    return best
end

(π::BranchAndBound)(s) = branch_and_bound(π.𝒫, s, π.d, π.Ulo, π.Qhi).a
```

算法 9.3. 用于在线寻找离散 MDP $\mathcal{P}$ 中从当前状态 `s` 出发的近似最优动作的分支定界算法。搜索深度为 `d`，值函数下界为 `Ulo`，动作价值函数上界为 `Qhi`。返回的命名元组由最优动作 `a` 及其有限时域期望值 `u` 组成。该算法同样用于 POMDP。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

剪枝时，操作按照上界的降序遍历。如例9.2所示，更紧的边界通常会导致更多的剪枝。

考虑将分支定界应用于山地车问题。我们可以使用启发式策略的值函数作为下界 $\underline{U}(s)$，例如始终沿运动方向加速的启发式策略。对于上界 $Q([x, v], a)$，我们可以使用在没有坡的情况下朝目标加速时的期望回报。分支定界访问的状态数约为前向搜索的三分之一。

例9.2. 分支定界法应用于山地车问题（附录F.4）。分支定界法相比前向搜索能实现显著的加速。

9.5 Sparse Sampling

一种被称为*sparse sampling*[3] (算法9.4)的方法试图减少前向搜索和分支定界的分支因子。我们不是对所有可能的下一状态进行分支，而是只考虑下一状态的有限数量的样本。尽管对下一状态进行采样会导致近似，但这种方法在实践中可以很好地工作，并且可以显著减少计算量。如果我们为搜索树中的每个动作节点抽取 m 个下一状态样本，则计算复杂度为 $O\left((m \times |\mathcal{A}|)^d\right)$，它仍然随深度呈指数增长，但不再依赖于状态空间的大小。图 9.3 展示了一个示例。

3 M. J. Kearns、Y. Mansour 和 A. Y. Ng，“一种用于大型马尔可夫决策过程中近似最优规划的稀疏采样算法”，*Machine Learning*，第 49 卷，第 2–3 期，第 193–208 页，2002 年。

9.6 Monte Carlo Tree Search

Monte Carlo tree search (算法9.5)通过从当前状态运行m次模拟，避免了horizon中的指数复杂度。[4]在这些模拟过程中，算法更新动作值函数 $Q(s, a)$ 的估计值，以及特定状态-动作对被选择次数的记录 $N(s, a)$。在从当前状态 s 运行这些 m 次模拟后，我们只需选择使 $Q(s, a)$ 的估计值最大化的动作。

4 关于一项综述，见 C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton，“A Survey of Monte Carlo Tree Search Methods，” *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 1–43, 2012.

模拟（算法9.6）首先遍历已探索的状态空间，该空间由我们已有Q和N估计的状态组成。我们遵循探索策略，从各个状态中选择动作。一种常见的方法是选择使*UCB1 exploration heuristic*最大化的动作：[5]

5 UCB 代表上置信界。这是 P. Auer、N. Cesa-Bianchi 和 P. Fischer 在“多臂老虎机问题的有限时间分析”，*Machine Learning*，第 47 卷，第 2-3 期，第 235-256 页，2002 年中讨论的众多策略之一。该方程源自切尔诺夫-霍夫丁界。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct SparseSampling
    𝒫 # problem
    d # depth
    m # number of samples
    U # value function at depth d
end

function sparse_sampling(𝒫, s, d, m, U)
    if d ≤ 0
        return (a=nothing, u=U(s))
    end
    best = (a=nothing, u=-Inf)
    for a in 𝒫.𝒜
        u = 0.0
        for i in 1:m
            s′, r = randstep(𝒫, s, a)
            a′, u′ = sparse_sampling(𝒫, s′, d-1, m, U)
            u += (r + 𝒫.γ*u′) / m
        end
        if u > best.u
            best = (a=a, u=u)
        end
    end
    return best
end

(π::SparseSampling)(s) = sparse_sampling(π.𝒫, s, π.d, π.m, π.U).a
```

算法9.4。稀疏采样算法，用于在线寻找离散问题$\mathcal{P}$从当前状态s到深度d的近似最优动作，每个动作有m个样本。返回的命名元组包含最佳动作a及其有限时域期望值u。

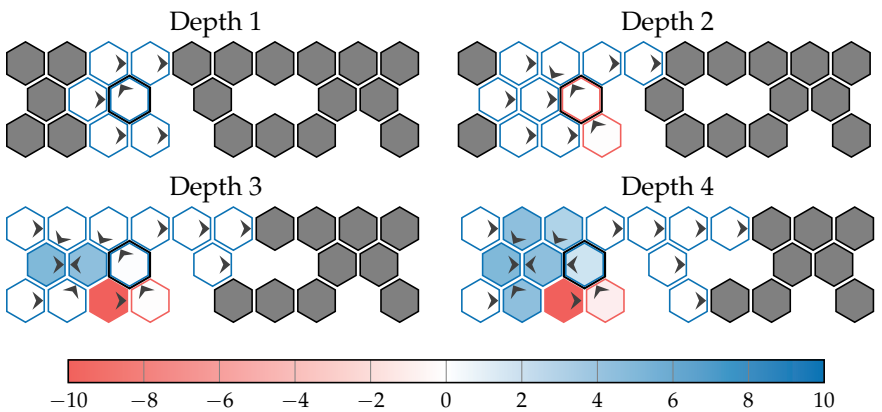


图9.3. 在六边形世界问题中应用了带有 $m = 10$ 的稀疏采样。访问过的图块根据其估计值着色。带边框的图块是初始状态。与图9.2中的前向搜索进行比较。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct MonteCarloTreeSearch
    𝒫 # problem
    N # visit counts
    Q # action value estimates
    d # depth
    m # number of simulations
    c # exploration constant
    U # value function estimate
end

function (π::MonteCarloTreeSearch)(s)
    for k in 1:π.m
        simulate!(π, s)
    end
    return argmax(a→π.Q[(s,a)], π.𝒫.𝒜)
end
```

算法9.5. 蒙特卡洛树搜索策略，用于从当前状态s中找到近似最优动作。

$$Q(s,a) + c\sqrt{\frac{\log N(s)}{N(s,a)}} \tag{9.1}$$

其中$N(s) = \sum_a N(s, a)$是s的总访问次数，而c是一个探索参数，用于缩放未探索动作的价值。第二项对应于一个*exploration bonus*。如果$N(s, a) = 0$，则奖励定义为无穷大。在分母中有$N(s, a)$的情况下，对于尝试频率较低的动作，探索奖励更高。算法9.7实现了这一探索策略。我们将在第15章后面讨论许多其他探索策略。

当我们执行算法 9.7 指定的动作时，我们会进入从生成模型 $T(s, a)$ 中采样的新状态，类似于稀疏采样方法。我们增加访问计数 $N(s, a)$ 并更新 $Q(s, a)$ 以维持均值。

在某个时刻，我们或达到最大深度，或到达一个尚未探索的状态。若到达未探索状态s，我们为每个动作a初始化$N(s, a)$及$Q(s, a)$为零。根据问题的先验专家知识，我们可修改算法9.6，将这些计数和价值估计初始化为其他值。初始化N和Q后，我们返回状态s的价值估计。通常，我们通过使用第9.2节所述过程的某个策略的rollout来估计此价值。

示例9.3至9.7通过蒙特卡洛树搜索应用于2048问题的实例进行说明。图9.4展示了在2048上运行蒙特卡洛树搜索生成的搜索树。示例9.8讨论了采用不同策略估计值的影响。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
function simulate!(π::MonteCarloTreeSearch, s, d=π.d)
    if d ≤ 0
        return π.U(s)
    end
    𝒫, N, Q, c = π.𝒫, π.N, π.Q, π.c
    𝒜, TR, γ = 𝒫.𝒜, 𝒫.TR, 𝒫.γ
    if !haskey(N, (s, first(𝒜)))
        for a in 𝒜
            N[(s,a)] = 0
            Q[(s,a)] = 0.0
        end
        return π.U(s)
    end
    a = explore(π, s)
    s′, r = TR(s,a)
    q = r + γ*simulate!(π, s′, d-1)
    N[(s,a)] += 1
    Q[(s,a)] += (q-Q[(s,a)])/N[(s,a)]
    return q
end
```

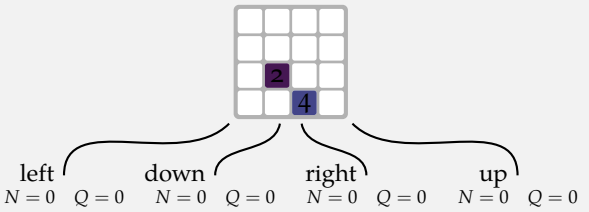
算法9.6. 从状态s开始到深度d运行蒙特卡洛树搜索模拟的方法。

```
bonus(Nsa, Ns) = Nsa == 0 ? Inf : sqrt(log(Ns)/Nsa)

function explore(π::MonteCarloTreeSearch, s)
    𝒜, N, Q, c = π.𝒫.𝒜, π.N, π.Q, π.c
    Ns = sum(N[(s,a)] for a in 𝒜)
    return argmax(a->Q[(s,a)] + c*bonus(N[(s,a)], Ns), 𝒜)
end
```

算法9.7。在蒙特卡洛树搜索中，用于确定搜索树中遍历哪些节点的一种探索策略。该策略由状态-动作访问计数字典`N`和值`Q`，以及一个探索参数`c`决定。当`N[(s,a)] = 0`时，策略返回无穷大。

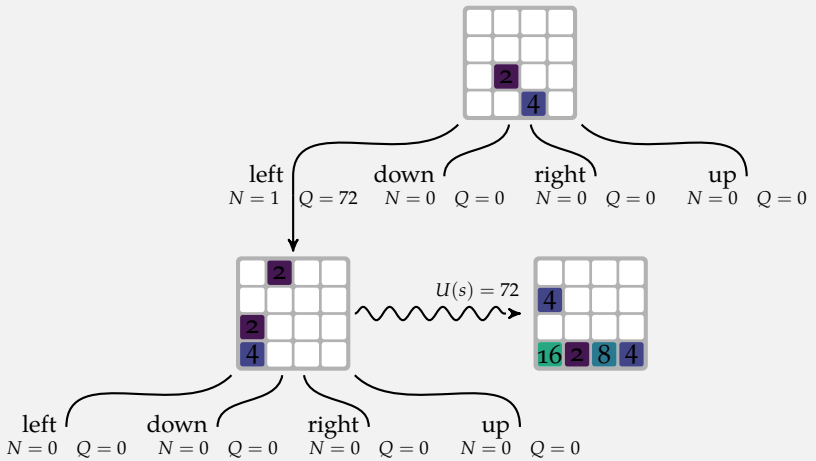
考虑使用蒙特卡洛树搜索来玩2048（附录F.2），最大深度为$d = 10$，探索参数为$c = 100$，并通过10步随机模拟来估计$U(s)$。我们的首次模拟会扩展起始状态。对于初始状态下的每个动作，计数和价值都会被初始化：



例9.3. 使用蒙特卡洛树搜索求解2048的一个示例。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

第二次模拟首先根据方程（9.1）中的探索策略，从初始状态中选择最佳动作。由于所有状态具有相同的值，我们任意选择第一个动作 `left`。接着，我们采样一个新的后继状态并将其扩展，初始化相关的计数和价值估计。从该后继状态进行模拟，并利用其值来更新 `left` 的值：



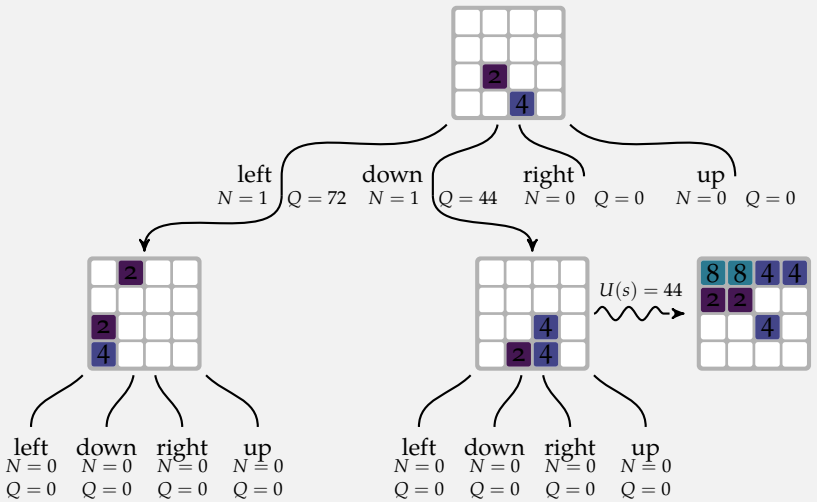
例9.4. 使用蒙特卡洛树搜索解决2048游戏的（续）示例。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

第三次模拟首先选择第二个动作 `down`，因为它由于对未探索动作的探索奖励而具有无限价值。第一个动作具有有限价值：

$$Q(s_0, \texttt{left}) + c\sqrt{\frac{\log N(s_0)}{N(s_0, \texttt{left})}} = 72 + 100\sqrt{\frac{\log 1}{1}} = 72$$

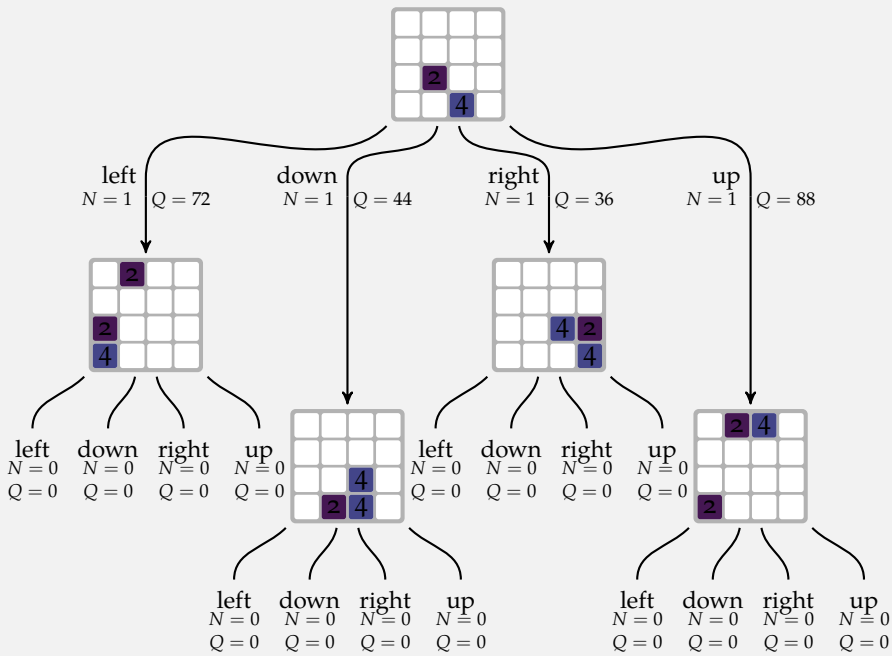
我们采取 `down` 动作并采样一个新的后继状态，该状态被扩展。从该后继状态运行一次 rollout，并使用其值来更新 `down` 的值：



示例 9.5. 使用蒙特卡洛树搜索解决 2048 的一个（续）例子。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

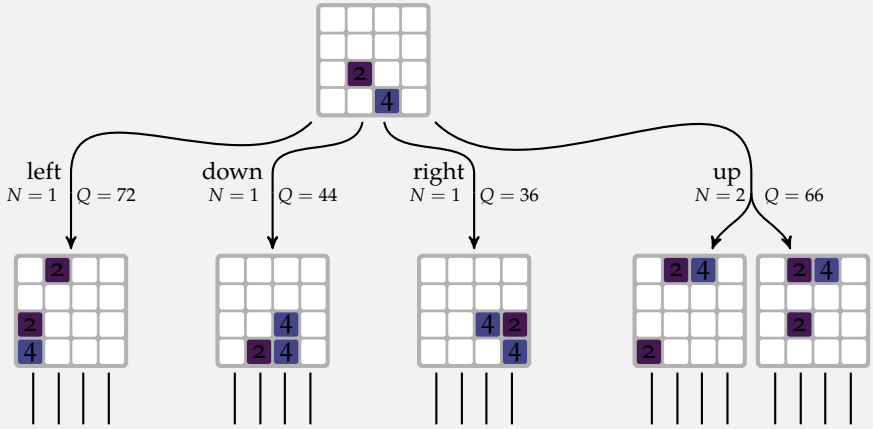
接下来的两个模拟分别选择`right`和`up`。结果如下：



例9.6。一个（续）使用蒙特卡洛树搜索解决2048的示例。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

在第五次模拟中，up 的值最高。在源状态下采取 up 后的后继状态不一定与第一次选择 up 时相同。我们评估 $U(s) = 44$ 并将访问计数更新为 2，估计值更新为 $Q \leftarrow 88 + (44 - 88)/2 = 66$。创建了一个新的后继节点：



例9.7。一个使用蒙特卡洛树搜索解决2048游戏的（续）示例。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

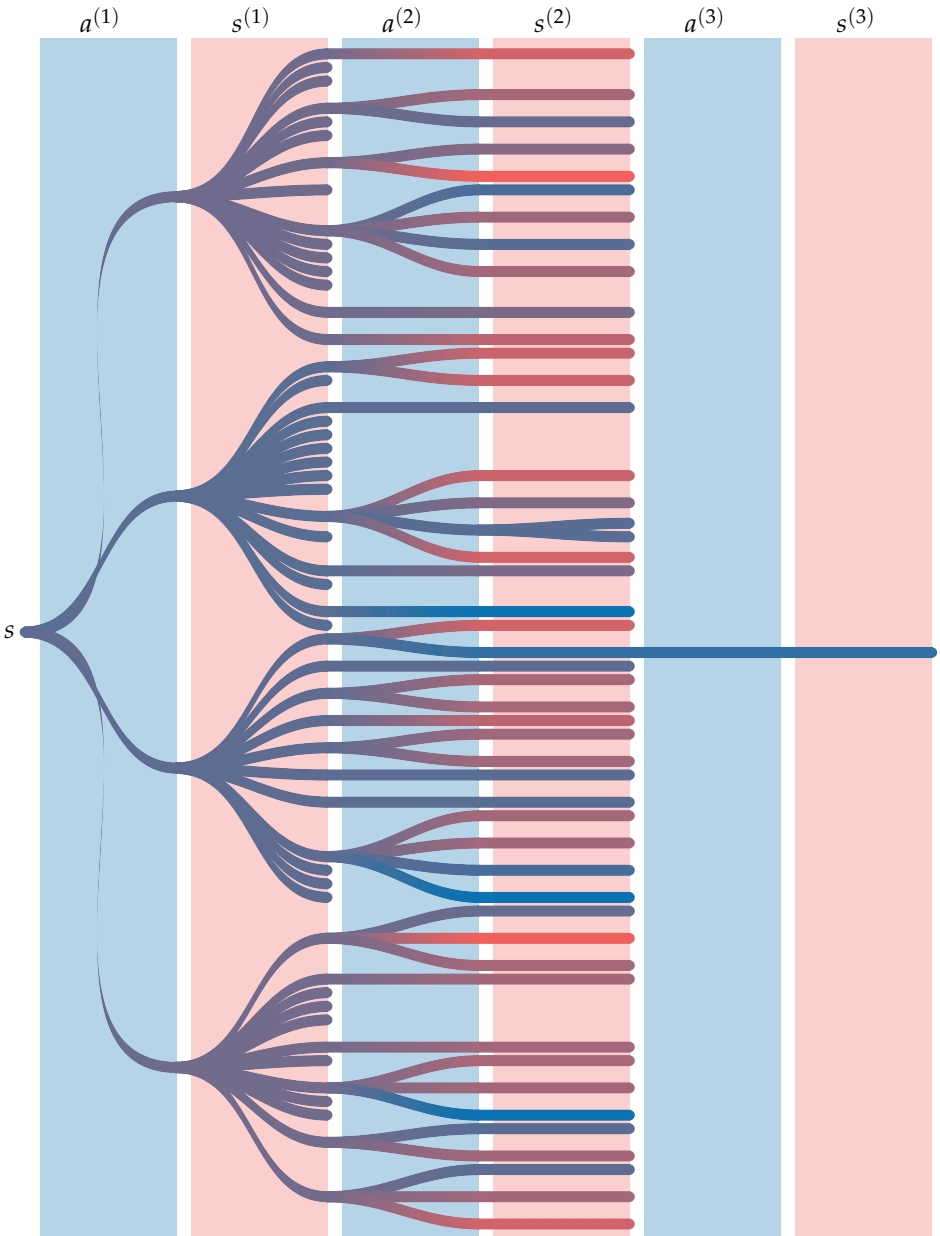


图9.4. 经过100次模拟后2048游戏上的蒙特卡洛树搜索树。通常，针对MDPs的蒙特卡洛树搜索会生成一个搜索图，因为到达同一状态可能存在多种路径。树中的颜色表示节点上的估计值，高值以蓝色显示，低值以红色显示。由于2048游戏中每个动作都有许多可达状态，这棵树较浅，但分支因子相当高。

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

在蒙特卡洛树搜索中，展开并不是估计效用值的唯一手段。针对特定问题，我们往往可以定制评估函数来辅助引导算法。例如，我们可以通过使用返回各瓦片值加权和的评估函数，促使蒙特卡洛树搜索在2048中对瓦片进行排序：

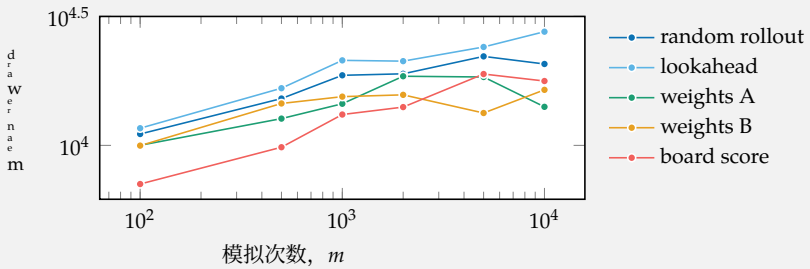
1	2	3	4
2	3	4	5
3	4	5	6
4	5	6	7

启发式A权重

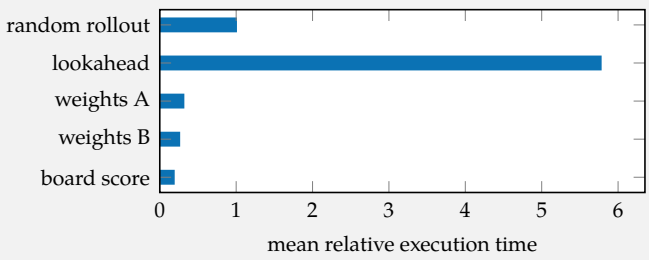
0	1	2	3
7	6	5	4
8	9	10	11
15	14	13	12

启发式B权重

这里的图表比较了在2048游戏中使用蒙特卡洛树搜索的几种策略：采用均匀随机策略的模拟、采用一步前瞻策略的模拟、两种评估函数，以及使用当前棋盘得分的情况：



Rollouts 表现良好，但需要更多的执行时间。这里我们绘制了从起始状态开始的 $m = 100$ 相对于随机 rollout 的平均执行时间：



例9.8。蒙特卡洛树搜索的性能随模拟次数的变化而变化，并且随着棋盘评估方法的改变而变化。启发式棋盘评估往往效率较高，并且在运行次数较少时能更有效地引导搜索。前瞻性展开评估所需时间大约是启发式评估的18倍。

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

这种基本的蒙特卡洛树搜索算法有多种变体，能够更好地处理大型动作和状态空间。我们可以使用 *progressive widening*，而不是扩展所有动作。从状态 s 考虑的动作数量被限制为 $\theta_1 N(s)^{\theta_2}$，其中 θ_1 和 θ_2 是超参数。类似地，我们可以用同样的方式限制从状态 s 采取动作 a 后产生的状态数量，这被称为 *double progressive widening*。如果在状态 s 采取动作 a 后已经模拟的状态数量低于 $\theta_3 N(s、a)^{\theta_4}$，那么我们采样一个新状态；否则，我们按与其被访问次数成比例的概率从之前采样的状态中采样一个。这种策略可用于处理大型以及连续的动作和状态空间。[6]

6 A. Couëtoux, J.-B. Hoock, N. Sokolovska, O. Teytaud, and N. Bonnard, “连续置信上界树”，收录于 *Learning and In-*，2011年。 *telligent Optimization* (*LION*)

9.7 Heuristic Search

Heuristic search (算法9.8)使用了从当前状态s出发，针对价值函数U的贪婪策略的m次模拟。[7] 价值函数U初始化为 *tificial Intelligence*

到一个值函数U的上界，这被称为*heuristic*。当我们运行这些模拟时，我们通过 *with Markov Decision Processes: An AI Perspective*
前瞻来更新我们对U的估计。运行这些模拟后，我们只需从s相对于U选择贪婪动作。图9.5显示了U和贪婪策略随模拟次数的变化。

7 A. G. Barto、S. J. Bradtke和S. P. Singh, “利用实时动态规划学习行动”，*Ar-*，第72卷，第1-2期，第81-138页，1995年。其他形式的启发式搜索由Mausam和A. Kolobov讨论，*Planning*。Morgan & Claypool Publishers，2012年。

启发式搜索保证会收敛到最优效用函数，只要启发式U确实是价值函数的上界。[8] 效率

搜索的效率取决于上界的紧度。遗憾的是，在实际操作中获取紧上界往往颇为困难。尽管一个并非真正上界的启发式方法可能无法收敛至最优策略，但它仍可能收敛到一个表现良好的策略。其时间复杂度为$O(m \times d \times |S| \times |A|)$。

8 这样的启发式方法被称为 *admissible heuristic*。

9.8 Labeled Heuristic Search

Labeled heuristic search (算法9.9)是启发式搜索的一种变体，它通过价值更新进行模拟，同时根据状态的值是否已解决来进行标记。[9]我们称一个状态s为已解决，如果其效用残差低于一个

阈值 $\delta > 0$:

$$|U_{k+1}(s) - U_k(s)| < \delta \tag{9.2}$$

9 B. Bonet 和 H. Geffner, “标记RTDP：改进实时动态规划的收敛性”，载于 *International Conference on Automated Planning and Scheduling* (*ICAPS*)，2003年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct HeuristicSearch
    𝒫   # problem
    Uhi # upper bound on value function
    d   # depth
    m   # number of simulations
end

function simulate!(π::HeuristicSearch, U, s)
    𝒫 = π.𝒫
    for d in 1:π.d
        a, u = greedy(𝒫, U, s)
        U[s] = u
        s = rand(𝒫.T(s, a))
    end
end

function (π::HeuristicSearch)(s)
    U = [π.Uhi(s) for s in π.𝒫.𝒮]
    for i in 1:π.m
        simulate!(π, U, s)
    end
    return greedy(π.𝒫, U, s).a
end
```

算法9.8。启发式搜索从初始状态`s`开始，运行`m`次模拟，达到深度`d`。搜索由启发式初始值函数`Uhi`引导，如果该函数是最优值函数的上界，则在模拟的极限下可达到最优性。

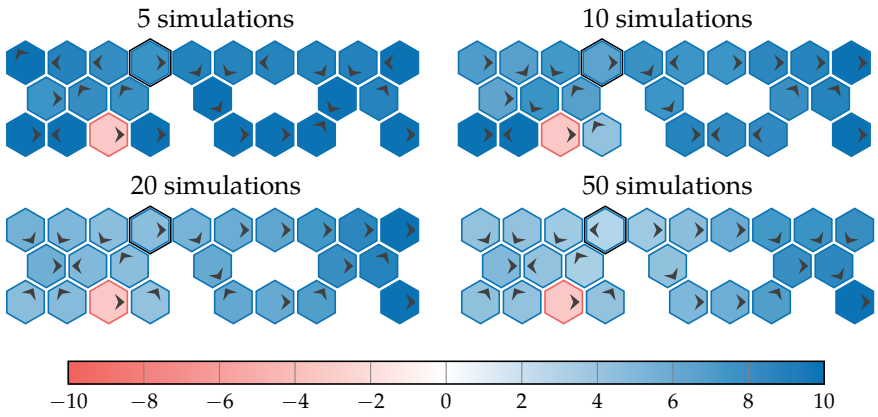


图9.5. 启发式搜索通过带有贝尔曼更新的模拟来改进六边形世界问题上的价值函数，从而从初始状态获得策略，此处用额外的黑色边框表示。这些模拟在启发式{v*}为10的情况下运行至深度8。每个六边形根据该迭代中的效用函数值着色。我们看到算法最终找到了最优策略。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们进行带有值更新的模拟，直到当前状态被解决。与上一节中固定运行迭代次数的启发式搜索不同，这种标记过程将计算努力集中在状态空间中最重要的区域。

```
struct LabeledHeuristicSearch
    𝒫     # problem
    Uhi   # upper bound on value function
    d     # depth
    δ     # gap threshold
end

function (π::LabeledHeuristicSearch)(s)
    U, solved = [π.Uhi(s) for s in 𝒫.𝒮], Set()
    while s ∉ solved
        simulate!(π, U, solved, s)
    end
    return greedy(π.𝒫, U, s).a
end
```

算法9.9。标记启发式搜索，从当前状态开始模拟至深度`d`，直到当前状态被解决。搜索由值函数`Uhi`的启发式上界引导，并维护一个不断增长的已解决状态集合。当状态的效用残差低于`δ`时，认为该状态已解决。返回一个值函数策略。

在有标签的启发式搜索（算法9.10）中，模拟开始时先遵循一种相对于我们估计的价值函数U贪婪的策略，运行到最大深度d，类似于前一节中的启发式搜索。如果在模拟到达深度d之前，我们遇到了在先前模拟中已被标记为已解决的状态，则可能提前终止模拟。

```
function simulate!(π::LabeledHeuristicSearch, U, solved, s)
    visited = []
    for d in 1:π.d
        if s ∈ solved
            break
        end
        push!(visited, s)
        a, u = greedy(π.𝒫, U, s)
        U[s] = u
        s = rand(π.𝒫.T(s, a))
    end
    while !isempty(visited)
        if label!(π, U, solved, pop!(visited))
            break
        end
    end
end
```

算法9.10。模拟从当前状态运行到最大深度`d`。我们在深度`d`处停止模拟，或者如果遇到属于集合`solved`中的状态也停止。模拟结束后，我们对访问过的状态按相反顺序调用`label!`。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

每次模拟后，我们逆序迭代该模拟过程中访问过的状态，对每个状态执行标记程序，并在发现未解决的状态时停止。标记程序（算法9.11）在*greedy envelope*的s中搜索状态，这些状态定义为在基于U的贪婪策略下从s可达的状态。如果s的贪婪包络中存在效用残差大于阈值δ的状态，则认为状态s未解决。如果没有找到这样的状态，那么s被标记为已解决——以及s的贪婪包络中的所有状态，因为它们也必定已经收敛。如果找到了效用残差足够大的状态，则在搜索贪婪包络期间遍历的所有状态的效用都会被更新。

图9.6显示了几个不同的贪婪包络。图9.7展示了标记启发式搜索单次迭代中所经过的状态。图9.8展示了在六边形世界问题上的启发式搜索进程。

9.9 Open-Loop Planning

本章讨论的在线方法，以及前几章讨论的离线方法，都是*closed-loop planning*的例子，即在规划过程中考虑未来状态信息。[10]通常，

10 此上下文中的循环指的是第1.1节中介绍的观察-行动循环。

open-loop planning 可以提供令人满意的最优闭环计划近似，同时通过避免对未来信息获取的推理而极大地提高计算效率。有时这种开环规划方法被称为*model predictive control*。[11]如同滚动

11 F. Borrelli, A. Bemporad, 和 M. Morari, *Predictive Control for Linear and Hybrid Systems*. 剑桥大学出版社，2019。

滚动时域控制，模型预测控制求解开环问题，从当前状态执行动作，转移到下一状态，然后重新规划。

开环计划可以表示为一系列动作，直到深度 d。规划过程简化为一个优化问题：

$$\underset{a_{1:d}}{\text{maximize}} \quad U(a_{1:d}) \tag{9.3}$$

其中 $U(a_{1:d})$ 是在执行动作序列 $a_{1:d}$ 时的期望回报。根据应用的不同，这个优化问题可能是凸的，或者可以转化为凸近似，这意味着它可以使用多种算法快速求解。[12] 本节稍后，我们将讨论几种不同的

12 附录A.6回顾凸性。凸优化的入门介绍由S. Boyd和L. Vandenberghe提供，*Convex Optimization*。剑桥大学出版社，2004年。

timization

可以将方程（9.3）转化为凸问题的公式。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
function expand(π::LabeledHeuristicSearch, U, solved, s)
    𝒫, δ = π.𝒫, π.δ
    𝒮, 𝒜, T = 𝒫.𝒮, 𝒫.𝒜, 𝒫.T
    found, toexpand, envelope = false, Set(s), []
    while !isempty(toexpand)
        s = pop!(toexpand)
        push!(envelope, s)
        a, u = greedy(𝒫, U, s)
        if abs(U[s] - u) > δ
            found = true
        else
            for s′ in 𝒮
                if T(s,a,s′) > 0 && s′ ∉ (solved ∪ envelope)
                    push!(toexpand, s′)
                end
            end
        end
    end
    return (found, envelope)
end

function label!(π::LabeledHeuristicSearch, U, solved, s)
    if s ∈ solved
        return false
    end
    found, envelope = expand(π, U, solved, s)
    if found
        for s ∈ reverse(envelope)
            U[s] = greedy(π.𝒫, U, s).u
        end
    else
        union!(solved, envelope)
    end
    return found
end
```

算法9.11。`label!`函数将尝试在`s`的贪心包络中找到一个效用残差超过阈值`δ`的状态。函数`expand`计算`s`的贪心包络，并确定这些状态中是否有任何状态的效用残差超过阈值。如果某个状态的残差超过阈值，则我们更新包络中状态的效用。否则，我们将该包络添加到已解决状态的集合中。

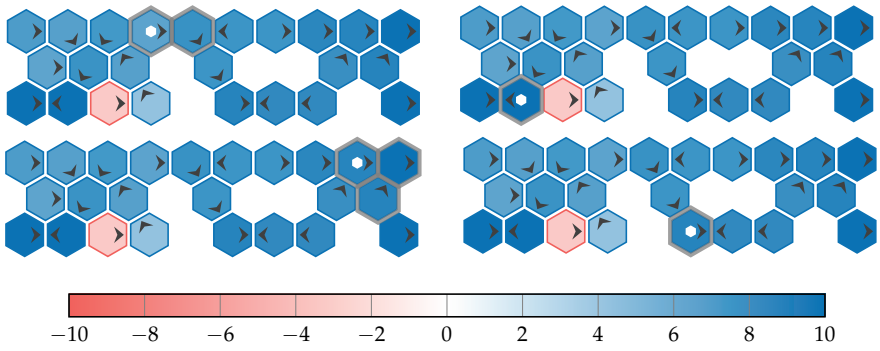


图9.6. 在六边形世界问题中，针对价值函数可视化了几种状态的$\delta = 1$的贪婪包络。该价值函数是通过从初始状态（以白色六边形中心表示）出发，运行基本启发式搜索10次迭代，达到最大深度8获得的。我们发现，以灰色轮廓标示的贪婪包络的大小可根据状态的不同而有很大差异。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com



图9.7. 标记启发式搜索的单次迭代进行探索性运行（箭头所示），随后进行标记（六边形边界）。在此迭代中，仅两个状态被标记：隐藏的终端状态和带有六边形边界的状态。探索性运行和标记步骤均更新价值函数。

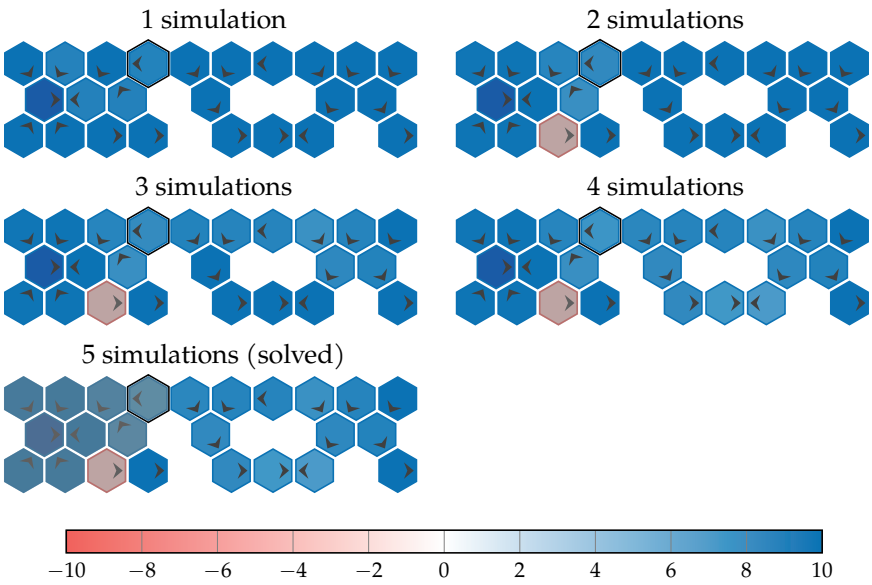


图 9.8. 在六边形世界问题中使用 $\delta = 1$ 和启发式 $U(s) = 10$ 的启发式搜索进展。每次迭代中已解决的状态用灰色涂层覆盖。已解决状态的集合从终止奖励状态向具有深色边框的初始状态扩展。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

开环规划通常使我们能够在高维空间中设计有效的决策策略，而在这些空间中，闭环规划在计算上是不可行的。这类规划通过不考虑未来信息来获得这种效率。例9.9提供了一个简单的实例，说明即使在考虑到随机性的情况下，开环规划也可能导致糟糕的决策。

考虑一个包含九个状态的问题，如边栏所示，从初始状态 s_1 开始有两个决策步骤。在我们的决策中，我们必须在向上走（蓝色箭头）和向下走（绿色箭头）之间做出选择。这些行动的效果是确定性的，唯一的例外是，如果我们从 s_1 向上走，那么我们有一半的时间最终到达状态 s_2，另一半时间到达状态 s_3。我们在状态 s_5 和 s_7 中获得 30 的奖励，在状态 s_8 和 s_9 中获得 20 的奖励，如图所示。

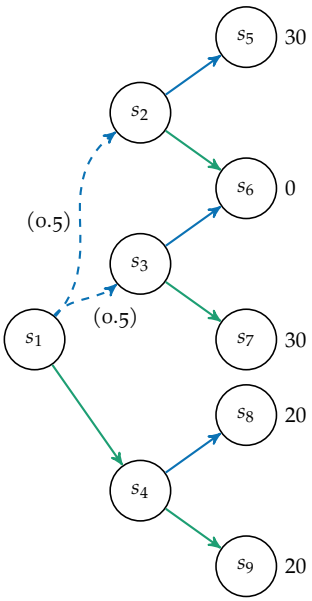
恰好有四个开环计划：(上，上)、(上，下)、(下，上)和(下，下)。在这个简单的例子中，很容易计算它们的预期效用：

- $U(\text{上，上}) = 0.5 \times 30 + 0.5 \times 0 = 15$
- $U(\text{上、下}) = 0.5 \times 0 + 0.5 \times 30 = 15$
- $U(\text{下、上}) = 20$
- $U(\text{下，下}) = 20$

根据开环计划集，最好从 s_1 中选择向下，因为我们的预期奖励是20而不是15。

相比之下，闭环规划考虑到了我们可以根据第一个动作的观察结果来做出下一个决策这一事实。如果我们选择从s_1向上走，那么我们可以根据最终到达的是s_2还是s_3来决定向下还是向上，从而保证获得30的奖励。

例9.9. 开环规划的次优性。



© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

9.9.1 Deterministic Model Predictive Control

使$U(a_{1:d})$易于优化的常见近似方法是假设确定性动力学：

$$\begin{aligned} \underset{a_{1:d}, s_{2:d}}{\text{maximize}} \quad & \sum_{t=1}^{d} \gamma^t R(s_t, a_t) \\ \text{subject to} \quad & s_{t+1} = T(s_t, a_t), \quad t \in 1:d-1 \end{aligned} \tag{9.4}$$

其中 s_1 是当前状态，而 $T(s、a)$ 是一个确定性转移函数，它返回从状态 s 采取动作 a 所得到的状态。从随机转移函数生成合适的确定性转移函数的一种常见策略是使用最可能的转移。如果方程 (9.4) 中的动态是线性的，并且奖励函数是凸的，那么该问题是凸的。

示例 9.10 提供了一个涉及在避开障碍物并最小化加速度努力的同时导航至目标状态的实例。状态空间和动作空间都是连续的，我们可以在不到一秒的时间内找到解决方案。每一步之后重新规划可以帮助补偿随机性或意外事件。例如，如果障碍物移动，我们可以重新调整我们的计划，如图 9.9 所示。

9.9.2 Robust Model Predictive Control

我们可以改变问题的表述方式，以提供对结果不确定性的鲁棒性。有许多 *robust model predictive control* 公式，[13] 但其中一种涉及在给定最坏情况状态转移的情况下选择最佳开环计划。该公式将 $T(s、a)$ 定义为 *uncertainty set*，由在状态 s 中采取动作 a 可能产生的所有状态组成。换句话说，不确定性集是分布 $T(\cdot \mid s、a)$ 的支撑集。针对最坏情况状态转移进行优化，需要将方程 (9.4) 中的优化问题转化为一个 *minimax* 问题：

13 A. Bemporad 和 M. Morari，《鲁棒模型预测控制：综述》，收录于 *Robustness in Identification and Control*，A. Garulli, A. Tesi 和 A. Vicino 编，Springer，1999年，第207–226页。

$$\begin{aligned} \underset{a_{1:d}}{\text{maximize}} \quad & \underset{s_{2:d}}{\text{minimize}} \sum_{t=1}^{d} \gamma^t R(s_t, a_t) \\ \text{subject to} \quad & s_{t+1} \in T(s_t, a_t), \quad t \in 1:d-1 \end{aligned} \tag{9.5}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

在这个问题中，我们的状态 s 表示代理的二维位置与其二维速度向量拼接，初始时 s 设为 [0, 0, 0, 0]。我们的动作 a 是一个加速度向量，其中每个分量必须在 ± 1 之间。在每一步，我们使用动作更新速度，并使用速度更新位置。我们的目标是达到目标状态 $s_{goal} = [10, 10, 0, 0]$。我们计划最多 $d = 10$ 步，且无折扣。每一步，我们累积成本 $\|a_t\|_2^2$ 以最小化加速度努力。在最后一步，我们希望尽可能接近目标状态，惩罚为 $100\|s_d - s_{goal}\|_2^2$。我们还必须确保避开一个以 [3, 4] 为中心、半径为 2 的圆形障碍物。我们可以将这个问题表述如下，并从计划中提取第一个动作：

```
model = Model(Ipopt.Optimizer)
d = 10
current_state = zeros(4)
goal = [10,10,0,0]
obstacle = [3,4]
@variables model begin
    s[1:4, 1:d]
    -1 ≤ a[1:2,1:d] ≤ 1
end
# velocity update
@constraint(model, [i=2:d,j=1:2], s[2+j,i] == s[2+j,i-1] + a[j,i-1])
# position update
@constraint(model, [i=2:d,j=1:2], s[j,i] == s[j,i-1] + s[2+j,i-1])
# initial condition
@constraint(model, s[:,1] .== current_state)
# obstacle
@constraint(model, [i=1:d], sum((s[1:2,i] - obstacle).^2) ≥ 4)
@objective(model, Min, 100*sum((s[:,d] - goal).^2) + sum(a.^2))
optimize!(model)
action = value.(a[:,1])
```

例 9.10。确定性环境中的开环规划。我们尝试找到一条绕过圆形障碍物的路径。此实现使用 `JuMP.jl` 接口调用 Ipopt 求解器。A. Wächter 和 L. T. Biegler，“关于大规模非线性规划的内点滤波线搜索算法的实现”，*Mathematical Programming*，第 106 卷，第 1 期，第 25–57 页，2005 年。doi:10.1007/s10107-004-0559-y。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

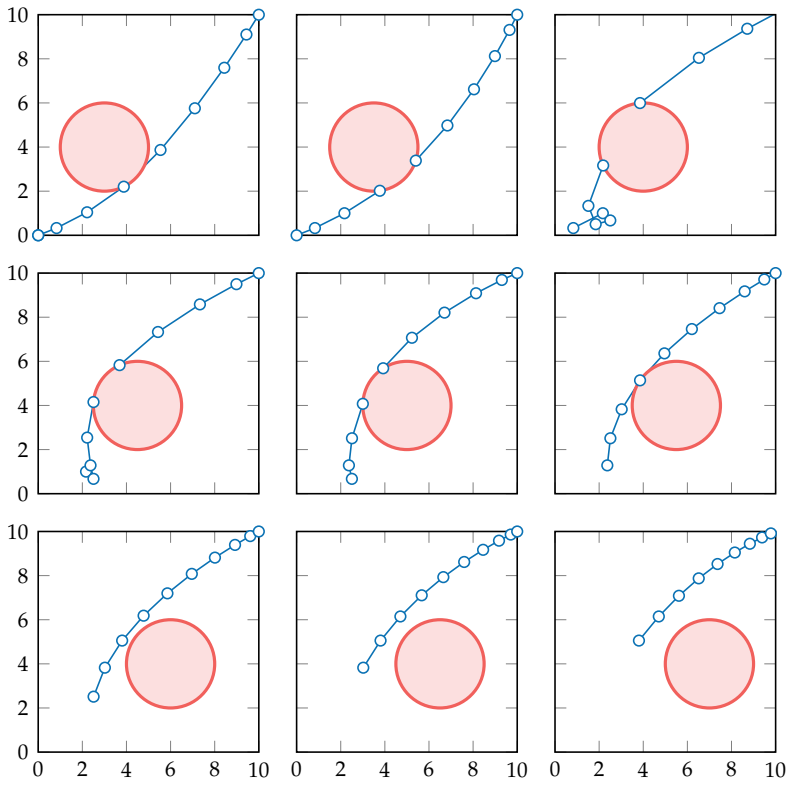


图9.9. 应用于例9.10问题的模型预测控制，增加了一个移动障碍物。序列从左到右、从上到下进行。最初，我们有一个从障碍物右侧通过的方案，但在第三格中，我们不得不改变主意，从左侧通过。我们得稍微绕一下，以最小的努力调整我们的速度向量。当然，如果我们的规划过程知道障碍物正朝特定方向移动，我们本可以制定出更好的路径（根据我们的效用函数）。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

遗憾的是，这种表述可能导致极为保守的行为。若我们将例9.10调整以模拟障碍物运动中的不确定性，即便在规划相对短视界时，不确定性的累积也可能变得相当大。减少不确定性累积的一种方法是限制由$T(s、a)$输出的不确定性集合仅包含比如95%的概率质量。此方法的另一个问题是，极小极大优化问题往往非凸且难以求解。

9.9.3 *Multiforecast Model Predictive Control*

解决方程（9.5）中极小极大问题所面临的计算挑战的一种方法是使用m预测情景，每个情景遵循其自身的确定性转移函数。[14]这类问题有多种表述方式，

multiforecast model predictive control，这是一种*hindsight optimization*类型。[15] 一常见方法是将确定性转移函数依赖于步骤 k、$T_i(s、a、k)$，这等同于扩展状态空间以包含深度。示例 9.11 演示了如何对线性高斯模型进行这样的处理。

14 S. Garatti 和 M. C. Campi，“控制设计中的鲁棒性调节：原理与算法”，*IEEE Control Systems Mag-azine*，第33卷，第2期，第36-51页，2013年。

15 这被称为后见之明优化，因为它代表了一种利用只有在事后才能获知的行动结果知识来进行优化的解决方案。E. K. P. Chong, R. L. Givan, 和 H. S. Chang, “A Framework for Simulation-Based Network Control via Hindsight Optimization,” in *IEEE Conference on Decision and Control (CDC)*, 2000.

假设我们有一个线性高斯动力学问题：

$$T(\mathbf{s}' \mid \mathbf{s}, \mathbf{a}) = \mathcal{N}(\mathbf{T}_s\mathbf{s} + \mathbf{T}_a\mathbf{a}, \mathbf{\Sigma})$$

图9.9中的问题是线性的，没有不确定性，但如果我们允许障碍物在每一步都按照高斯分布移动，那么动力学就变成了线性高斯。我们可以使用一组m个预测情景来近似动力学，每个情景包含d步。我们可以抽取$m \times d$个样本$\epsilon_{ik} \sim \mathcal{N}(0、\mathbf{\Sigma})$，并定义确定性转移函数：

$$T_i(\mathbf{s}, \mathbf{a}, k) = \mathbf{T}_s\mathbf{s} + \mathbf{T}_a\mathbf{a} + \epsilon_{ik}$$

示例 9.11. 多预测模型预测控制中的线性高斯转移动力学建模。

我们尝试为最差采样场景找到最佳动作序列：

$$\begin{aligned} &\underset{a_{1:d}}{\text{maximize}} \quad \underset{i,\, s_{2:d}}{\text{minimize}} \sum_{k=1}^{d} \gamma^k R(s_k, a_k) \\ &\text{subject to} \quad s_{k+1} = T_i(s_k, a_k, k), \quad k \in 1:d-1 \end{aligned} \tag{9.6}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

这个问题比原始的鲁棒问题更容易解决。

我们也可以使用多预测方法来优化平均情况。[16] 该公式化类似于方程（9.6），只是我们将最小化替换为期望，并允许针对不同场景采取不同的行动序列，但要求第一个行动必须一致：

16 该方法被N. Moehle、E. Busseti、S. Boyd和M. Wytock应用于优化潮流策略，见“动态能源管理”，收录于 *Large Scale Optimization in Supply Chains and Smart Manufacturing*，Springer，2019年，第89–126页。

$$\begin{aligned} \underset{a_{1:d}^{(1:m)}, s_{2:d}^{(i)}}{\text{maximize}} \quad & \frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{d}\gamma^k R(s_k^{(i)}, a_k^{(i)}) \\ \text{subject to} \quad & s_{k+1}^{(i)} = T_i(s_k^{(i)}, a_k^{(i)}, k), \quad k \in 1:d-1, i \in 1:m \\ & a_1^{(i)} = a_1^{(j)}, \qquad\qquad i \in 1:m, j \in 1:m \end{aligned} \tag{9.7}$$

这种表述可以实现稳健的行为而不过于保守，同时仍保持计算上的可行性。通过增加预测情景{v*}的数量，方程(9.6)和(9.7)中的两种表述都可以在牺牲额外计算量的情况下变得更加稳健。

9.10 Summary

- 在线方法从当前状态进行规划，将计算集中在可达的状态上。
- 滚动时域规划涉及规划到某个时域，然后在每一步重新规划。
- 使用前瞻与展开（rollouts）涉及根据通过模拟展开策略估算的值贪婪地行动；与其他算法相比，它在计算上更高效，但对性能没有保证。
- 前向搜索考虑达到一定深度的所有状态-动作转移，导致计算复杂度在状态数量和动作数量上均呈指数增长。
- 分支定界使用上界和下界函数来剪枝搜索树中那些在期望上不会带来更好结果的部分。
- 稀疏采样通过限制每个搜索节点的采样转移数量，避免了状态数量上的指数复杂度。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

- 蒙特卡洛树搜索通过采取平衡探索与利用的行动，引导搜索走向搜索空间中有前景的区域。
- 启发式搜索运行的是这样一种策略模拟：该策略相对于一个沿途使用前瞻进行更新的价值函数是贪婪的。
- 带标签的启发式搜索通过不重新评估值已收敛的状态来减少计算量。
- 开环规划旨在找到最佳可能的动作序列，并且如果优化问题是凸的，它可以计算高效。

9.11 Exercises

练习 9.1. 为什么分支定界法的最坏情况计算复杂度与前向搜索相同?

Solution: 在最坏的情况下，分支定界法永远不会剪枝，导致遍历与正向搜索相同的搜索树，具有相同的复杂度。

练习 9.2. 给定两个可接受的启发式函数 h_1 和 h_2，我们如何在启发式搜索中同时使用它们?

Solution: 创建一个新的启发式 $h(s) = \min\{h_1(s)、h_2(s)\}$ 并使用它来替代。这个新的启发式保证是可接受的，并且不会比 h_1 或 h_2 中的任何一个给出更差的边界。$h_1(s) \geq U^*(s)$ 和 $h_2(s) \geq U^*(s)$ 都意味着 $h(s) \geq U^*(s)$。

练习 9.3. 给定两个不可接受启发式函数h_1和h_2，描述一种在启发式搜索中同时使用它们的方法。

Solution: 我们可以定义一个新的启发式函数 $h_3(s) = \max(h_1(s), h_2(s))$，以得到一个可能可采纳的，或者“较少不可采纳”的启发式函数。它可能收敛得更慢，但它可能更有可能不会错过更好的解决方案。

练习 9.4。假设我们有一个状态空间为 $\mathcal{S}$、动作空间为 $\mathcal{A}$ 的离散 MDP，并且我们希望执行深度为 d 的前向搜索。由于计算限制以及我们必须模拟到深度 d 的要求，我们决定通过以更粗的尺度 $|\mathcal{S}'| < |\mathcal{S}|$ 和 $|\mathcal{A}'| < |\mathcal{A}|$ 对原始状态和动作空间重新离散化，来生成新的、更小的状态和动作空间。就原始状态和动作空间而言，为了使前向搜索的计算复杂度相对于我们原始状态和动作空间的大小近似与深度无关，即 $O(|\mathcal{S}||\mathcal{A}|)$，新的状态和动作空间的大小需要是多少?

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: 我们需要

$$|\mathcal{S}'| = |\mathcal{S}|^{\frac{1}{d}} \qquad \text{and} \qquad |\mathcal{A}'| = |\mathcal{A}|^{\frac{1}{d}}$$

这导致以下复杂度：

$$O\left(|\mathcal{S}'|^d |\mathcal{A}'|^d\right) = O\left(\left(|\mathcal{S}|^{\frac{1}{d}}\right)^d \left(|\mathcal{A}|^{\frac{1}{d}}\right)^d\right) = O(|\mathcal{S}||\mathcal{A}|)$$

练习 9.5. 在前一个练习的基础上，假设我们现在想保留动作空间中的所有原始动作，仅重新离散化状态空间。为了使前向搜索的计算复杂度相对于我们原始状态和动作空间的大小大致深度不变，新的状态空间的大小需要是多少？

Solution: 前向搜索的计算复杂度由 $O\left((|\mathcal{S}||\mathcal{A}|)^d\right)$ 给出，也可以写作 $O\left(|\mathcal{S}|^d |\mathcal{A}|^d\right)$。因此，为了使我们的粗粒度状态空间导致的前向搜索相对于原始状态和动作空间的大小近似深度不变，我们需要

$$|\mathcal{S}'| = \left(\frac{|\mathcal{S}|}{|\mathcal{A}|^{d-1}}\right)^{\frac{1}{d}}$$

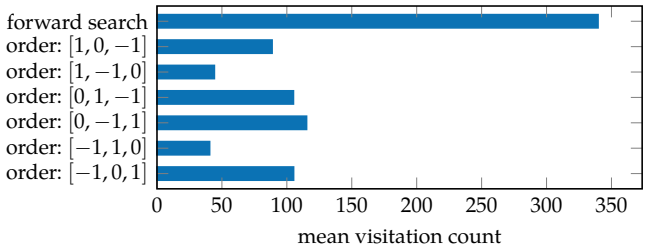
这为我们提供了：

$$O\left(|\mathcal{S}'|^d |\mathcal{A}'|^d\right) = O\left(\left[\left(\frac{|\mathcal{S}|}{|\mathcal{A}|^{d-1}}\right)^{\frac{1}{d}}\right]^d |\mathcal{A}|^d\right) = O\left(|\mathcal{S}| \frac{|\mathcal{A}|^d}{|\mathcal{A}|^{d-1}}\right) = O(|\mathcal{S}||\mathcal{A}|)$$

练习 9.6。改变动作空间的顺序是否会导致前向搜索采取不同的动作？改变动作空间的顺序是否会导致分支定界采取不同的动作？动作空间的顺序是否会影响分支定界访问的状态数量？

Solution: 前向搜索枚举所有可能的未来动作。如果预期效用存在并列情况，它可能会返回不同的动作。分支定界通过按上界排序，在规划范围内保持了与前向搜索相同的最优性保证。当上界对两个或更多动作产生相同的期望值时，动作空间的排序可能会影响分支定界的访问率。下面我们在例9.2中的修改山地车问题上展示这种效应。该图比较了不同动作排序下，前向搜索与分支定界到深度6所访问的状态数量。分支定界访问的状态数始终远少于前向搜索，但动作排序仍可能影响状态访问。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com



练习 9.7. 使用 $m = |\mathcal{S}|$ 的稀疏采样是否等同于前向搜索?

Solution: 不。虽然在$O\left(|\mathcal{S}|^d|\mathcal{A}|^d\right)$处计算复杂度相同，但前向搜索会在状态空间中的所有状态上进行分支，而稀疏采样则会在随机抽样的$|\mathcal{S}|$个状态上进行分支。

练习 9.8。给定一个 MDP，$|\mathcal{S}| = 10$，$|\mathcal{A}| = 3$，且对所有 s 和 a 有均匀转移分布 $T(s' \mid s, a) = 1/|\mathcal{S}|$，那么使用 $m = |\mathcal{S}|$ 个样本和深度 $d = 1$ 的稀疏采样产生与深度 $d = 1$ 的前向搜索完全相同的搜索树的概率是多少?

Solution: 对于前向搜索和稀疏采样，我们都从当前状态节点对所有动作进行分支。对于前向搜索，在每个动作节点上，我们对所有状态进行分支；而对于稀疏采样，我们将对$m = |\mathcal{S}|$个采样状态进行分支。如果这些采样状态恰好等于状态空间，则该动作分支等同于前向搜索中产生的分支。因此，对于单个动作分支，我们有：

第一个状态唯一的概率 10 10 第二个状态唯一的概率（不等于第一个状态） 910 第—三个状态唯一的概率（不等于第一个或第二个状态） 810 ... ...

—

—

由于这些采样状态中的每一个都是独立的，这导致状态空间中所有独特状态被选中的概率为

$$\frac{10 \times 9 \times 8 \times \cdots}{10 \times 10 \times 10 \times \cdots} = \frac{10!}{10^{10}} \approx 0.000363$$

由于不同动作分支中采样的各个状态是独立的，因此三个动作分支均采样到状态空间中唯一状态的概率为

$$\left(\frac{10!}{10^{10}}\right)^3 \approx (0.000363)^3 \approx 4.78 \times 10^{-11}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

练习9.9。给定以下$Q(s, a)$和$N(s, a)$的表格，使用方程(9.1)中的上置信界，计算每个状态下MCTS遍历动作，探索参数$c_1 =$为10，再次为$c_2 =$为20。

	$Q(s,a_1)$	$Q(s,a_2)$
s_1	10	-5
s_2	12	10

	$N(s,a_1)$	$N(s,a_2)$
s_1	27	4
s_2	32	18

Solution: 对于第一个探索参数$c_1 = 10$，我们将每个状态-动作对的上置信界制成表格，并为每个状态选择使界最大化的动作：

	$UCB(s,a_1)$	$UCB(s,a_2)$	$\arg\max_a UCB(s,a)$
s_1	$10 + 10\sqrt{\frac{\log 31}{27}} \approx 13.566$	$-5 + 10\sqrt{\frac{\log 31}{4}} \approx 4.266$	a_1
s_2	$12 + 10\sqrt{\frac{\log 50}{32}} \approx 15.496$	$10 + 10\sqrt{\frac{\log 50}{18}} \approx 14.662$	a_1

而对于 $c_2 = 20$，我们有：

	$UCB(s,a_1)$	$UCB(s,a_2)$	$\arg\max_a UCB(s,a)$
s_1	$10 + 20\sqrt{\frac{\log 31}{27}} \approx 17.133$	$-5 + 20\sqrt{\frac{\log 31}{4}} \approx 13.531$	a_1
s_2	$12 + 20\sqrt{\frac{\log 50}{32}} \approx 18.993$	$10 + 20\sqrt{\frac{\log 50}{18}} \approx 19.324$	a_2

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

10 Policy Search

Policy search 涉及在策略空间中搜索，而不直接计算价值函数。策略空间通常比状态空间维度更低，并且通常可以更高效地进行搜索。策略优化通过优化*parameterized policy*中的参数来最大化效用。这种参数化策略可以采取多种形式，例如神经网络、决策树和计算机程序。本章首先讨论一种在给定初始状态分布的情况下估计策略价值的方法。然后，我们将讨论不使用策略梯度估计的搜索方法，将梯度方法留到下一章。尽管局部搜索在实践中可能非常有效，但我们还将讨论一些可以避免局部最优的替代优化方法。[1]

1 还有许多其他优化方法，如 M. J. Kochenderfer 和 T. A. Wheeler 所讨论的，*Algorithms for Optimization*，MIT Press，2019年。

10.1 Approximate Policy Evaluation

如第7.2节所介绍，当从状态s遵循策略π时，我们可以计算预期折扣回报。这个预期折扣回报$U^\pi(s)$可以通过迭代（算法7.3）或在状态空间离散且相对较小时通过矩阵运算（算法7.4）来计算。我们可以利用这些结果来计算π的预期折扣回报：

$$U(\pi) = \sum_s b(s) U^\pi(s) \tag{10.1}$$

假设一个 *initial state distribution* $b(s)$。

在本章中，我们将一直使用$U(\pi)$的定义。然而，当状态空间庞大或连续时，我们往往无法精确计算$U(\pi)$。相反，我们可以通过采样*trajectories*来近似$U(\pi)$，*trajectories*包含状态、动作、

并且遵循π时获得奖励。$U(\pi)$的定义可以重写为

$$U(\pi) = \mathbb{E}_\tau[R(\tau)] = \int p_\pi(\tau) R(\tau)\, \mathrm{d}\tau \tag{10.2}$$

其中$p_\pi(\tau)$是与轨迹τ相关联的概率密度，当遵循策略π时，从初始状态分布b开始。*trajectory reward* $R(\tau)$是与τ相关的折扣回报。图10.1展示了根据从初始状态分布中采样的轨迹计算$U(\pi)$的过程。

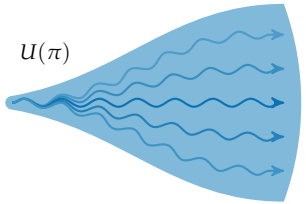


图10.1. 与初始状态分布下的策略相关的效用，是根据给定策略下所有可能轨迹的回报计算得出的，这些回报根据其可能性进行加权。

Monte Carlo policy evaluation（算法10.1）涉及用π的m轨迹展开来近似方程（10.2）：

$$U(\pi) \approx \frac{1}{m} \sum_{i=1}^{m} R(\tau^{(i)}) \tag{10.3}$$

其中 $\tau^{(i)}$ 是第 i 个轨迹样本。

```
struct MonteCarloPolicyEvaluation
    𝒫 # problem
    b # initial state distribution
    d # depth
    m # number of samples
end

function (U::MonteCarloPolicyEvaluation)(π)
    R(π) = rollout(U.𝒫, rand(U.b), π, U.d)
    return mean(R(π) for i = 1:U.m)
end

(U::MonteCarloPolicyEvaluation)(π, θ) = U(s->π(θ, s))
```

算法 10.1. 策略 `π` 的蒙特卡洛策略评估。该方法根据问题 $\mathcal{P}$ 指定的动态特性，运行 `m` 次推演至深度 `d`。每次推演均从状态分布 `b` 中采样的初始状态开始。此算法块的最后一行评估了一个由 `θ` 参数化的策略 `π`，这对于本章中尝试寻找能使 `U` 最大化的 `θ` 值的算法而言将十分有用。

蒙特卡洛策略评估是随机的。对同一策略多次评估式（10.1）可能会得到不同的估计值。增加 rollout 次数会降低评估的方差，如图 10.2 所示。

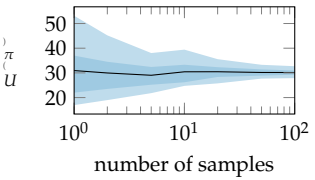


图10.2. 深度和样本数量对cart-pole问题（附录F.3）上均匀随机策略的蒙特卡洛策略评估的影响。随着样本数量的增加，方差减小。蓝色区域表示$U(\pi)$的5%至95%和25%至75%的经验分位数。

我们将使用 π_θ 来表示由 θ 参数化的策略。为方便起见，在没有歧义的情况下，我们将使用 $U(\theta)$ 作为 $U(\pi_\theta)$ 的简写。参数 θ 可以是一个向量或其他更复杂的表示。例如，我们可能希望使用具有特定结构的神经网络来表示我们的策略。我们将使用 θ 来表示网络中的权重。许多优化算法假设 θ 是一个具有固定数量

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

组件。其他优化算法允许更灵活的表示，包括决策树或计算表达式之类的表示。[2]

2 我们在这里不会讨论这些表示，但其中一些已在`ExprOptimization.jl`中实现。

10.2 Local Search

一种常见的优化方法是*local search*，我们从初始参数化开始，在搜索空间中逐步从邻居移动到邻居，直到收敛。我们在第5章中讨论了这种方法，在针对贝叶斯评分优化贝叶斯网络结构的背景下。在这里，我们正在优化由θ参数化的策略。我们试图找到一个使$U(\theta)$最大化的θ值。

有许多局部搜索算法，但本节将重点介绍 *Hooke-Jeeves method* (算法 10.2)。[3] 该算法假设我们的策略

由 n 维向量 θ 参数化。该算法从当前 θ 出发，沿每个坐标方向迈出大小为 $\pm\alpha$ 的一步。这 $2n$ 个点对应于 θ 的邻域。如果没有找到策略的改进，则将步长 α 按某个因子减小。如果找到改进，则移动到最佳点。该过程持续进行，直到 α 降至某个阈值 $\epsilon > 0$ 以下。例 10.1 提供了一个涉及策略优化的示例，图 10.3 展示了这一过程。

3 R. Hooke 和 T. A. Jeeves，“数值与统计问题的直接搜索解法，” *Journal of the*，第8卷，第2期，第212-229页，1961年。*ACM (JACM)*

10.3 Genetic Algorithms

像Hooke-Jeeves方法这样的局部搜索算法的一个潜在问题是，优化过程可能会陷入局部最优。为了应对这一挑战，存在多种方法，它们涉及维护一个由参数空间中样本点组成的*population*，并行地根据我们的目标评估这些点，然后以某种方式重新组合它们，驱动群体向全局最优解进化。*genetic algorithm*[4]便是这样一种方法，它灵感来源于

从生物进化中汲取灵感。这是一种通用的优化方法，但在优化策略方面已取得了成功。例如，这种方法已被用于优化Atari视频游戏的策略，其中策略参数对应于神经网络中的权重。[5]

这种方法的一个简单版本（算法 10.3）从一个包含 m 个随机参数化的种群开始，即 $\theta^{(1)}, \ldots, \theta^{(m)}$。我们为每个样本计算 $U(\theta^{(i)})$

4 D. E. Goldberg, *Genetic Algorithms*. Addison-W *in Search, Optimization, and Machine Learning*

5 F. P. Such, V. Madhavan, E. Conti, J. Lehman, K. O. Stanley, and J. Clune, “Deep Neuroevolution: Genetic Algorithms Are a Competitive Alternative for Training Deep Neural Networks for Reinforcement Learning,” 2017. arXiv: 1712.06567v3. 本节中的实现遵循了他们相对简单的公式化表述。他们的表述不包含交叉，而交叉通常用于在整个种群中混合参数化。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct HookeJeevesPolicySearch
    θ # initial parameterization
    α # step size
    c # step size reduction factor
    ϵ # termination step size
end

function optimize(M::HookeJeevesPolicySearch, π, U)
    θ, θ′, α, c, ϵ = copy(M.θ), similar(M.θ), M.α, M.c, M.ϵ
    u, n = U(π, θ), length(θ)
    while α > ϵ
        copyto!(θ′, θ)
        best = (i=0, sgn=0, u=u)
        for i in 1:n
            for sgn in (-1,1)
                θ′[i] = θ[i] + sgn*α
                u′ = U(π, θ′)
                if u′ > best.u
                    best = (i=i, sgn=sgn, u=u′)
                end
            end
            θ′[i] = θ[i]
        end
        if best.i != 0
            θ[best.i] += best.sgn*α
            u = best.u
        else
            α *= c
        end
    end
    return θ
end
```

算法10.2. 使用Hooke-Jeeves方法进行策略搜索，该方法返回一个相对于U优化过的θ。策略π以参数θ和状态s作为输入。此实现从θ的初始值开始。如果没有任何邻居改进目标，则步长α会按因子c减小。迭代会一直运行，直到步长小于ϵ。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

示例 10.1. 使用策略优化算法来优化随机策略的参数。

假设我们要针对附录 F.5 中描述的简单调节器问题优化策略。我们定义了一个由 θ 参数化的随机策略 π，使得动作根据

$$a \sim \mathcal{N}(\theta_1 s, (|\theta_2| + 10^{-5})^2) \tag{10.4}$$

以下代码定义了参数化随机策略 π、评估函数 U 和方法 M。然后调用 optimize(M, π, U)，它为 θ 返回一个优化值。在这种情况下，我们使用 Hooke-Jeeves 方法，但本章讨论的其他方法也可以作为 M 传入：

```
function π(θ, s)
    return rand(Normal(θ[1]*s, abs(θ[2]) + 0.00001))
end
b, d, n_rollouts = Normal(0.3,0.1), 10, 3
U = MonteCarloPolicyEvaluation(𝒫, b, d, n_rollouts)
θ, α, c, ϵ = [0.0,1.0], 0.75, 0.75, 0.01
M = HookeJeevesPolicySearch(θ, α, c, ϵ)
θ = optimize(M, π, U)
```

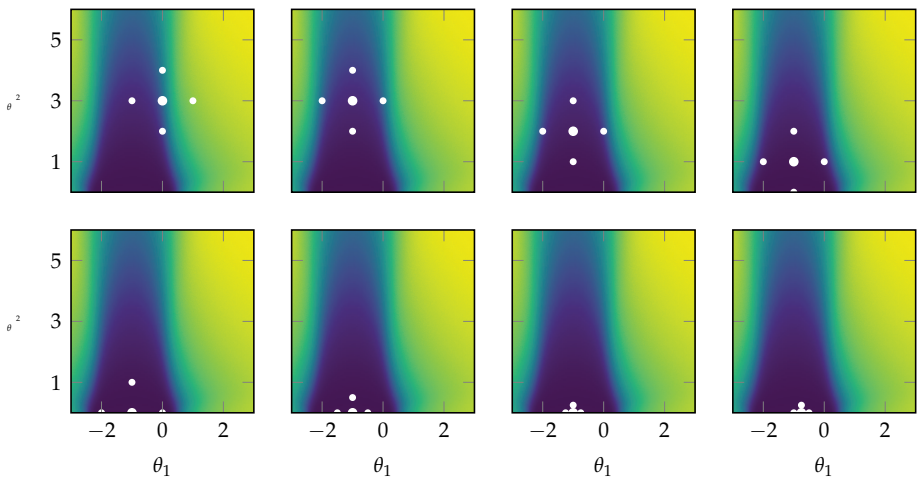


图10.3。Hooke-Jeeves方法应用于优化例10.1中讨论的简单调节器问题中的策略。每次迭代的评估结果以白点表示。迭代从左到右、从上到下进行，背景颜色根据期望效用着色，黄色表示较低的效用，深蓝色表示较高的效用。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

i 在种群中。由于这些评估可能涉及许多展开模拟，因此计算成本很高，它们通常并行运行。这些评估帮助我们识别出*elite samples*，即根据U排名前m_{elite}的样本。

下一次迭代的种群生成方式是通过反复选择一个随机精英样本 θ 并对其施加各向同性高斯噪声 $\theta + \sigma\epsilon$ 进行扰动，产生 $m -$ 个新的参数化，其中 $\epsilon \sim \mathcal{N}(0, \mathrm{I})$。最佳参数化（未扰动）被纳入作为第 m 个样本。由于评估涉及随机仿真，该算法的一种变体可能涉及运行额外仿真以帮助确定哪个精英样本真正最优。图 10.4 展示了该方法在一个样本问题中的若干次迭代，即 *generations*。

```
struct GeneticPolicySearch
    θs      # initial population
    σ       # initial standard deviation
    m_elite # number of elite samples
    k_max   # number of iterations
end

function optimize(M::GeneticPolicySearch, π, U)
    θs, σ = M.θs, M.σ
    n, m = length(first(θs)), length(θs)
    for k in 1:M.k_max
        us = [U(π, θ) for θ in θs]
        sp = sortperm(us, rev=true)
        θ_best = θs[sp[1]]
        rand_elite() = θs[sp[rand(1:M.m_elite)]]
        θs = [rand_elite() + σ.*randn(n) for i in 1:(m-1)]
        push!(θs, θ_best)
    end
    return last(θs)
end
```

算法10.3. 一种遗传策略搜索方法，用于迭代更新一组策略参数化 `θs`，它接受一个策略评估函数 `U`、一个策略 `π(θ, s)`、一个扰动标准差 `σ`、一个精英样本数 `m_elite` 和一个迭代次数 `k_max`。每次迭代中最好的 `m_elite` 个样本用于生成后续迭代的样本。

10.4 Cross Entropy Method

cross entropy method (算法10.4)涉及在每次迭代时更新策略参数空间上的 *search distribution*。[6]我们对此进行参数化

搜索分布 $p(\theta \mid \psi)$ 与 ψ。[7] 该分布可以属于任何分布族，但高斯分布是一个常见的选择，其中 ψ 表示均值，并且

6 S. Mannor、R. Y. Rubinstein 和 Y. Gat，“用于快速策略搜索的交叉熵方法”，载于 *International Conference on Machine Learning (ICML)*，2003年

7 通常，θ 和 ψ 是向量，但由于此方法不要求这一假设，我们在本节中不会将它们加粗。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

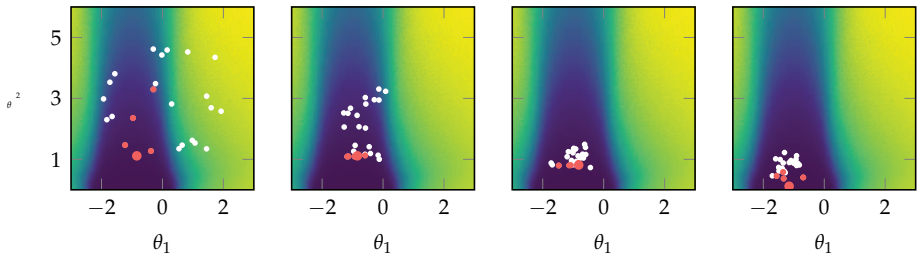


图10.4. 使用每次迭代25个样本，并应用σ =为0.25的遗传策略搜索解决简单调节器问题。每一代中的五个精英样本以红色显示，最佳样本由一个更大的点表示。

分布的协方差。目标是找到一个ψ^*的值，使得当θ从搜索分布中抽取时，最大化$U(\theta)$的期望：

$$\psi^* = \arg\max_{\psi} \mathop{\mathbb{E}}_{\theta \sim p(\cdot \mid \psi)}[U(\theta)] = \arg\max_{\psi} \int U(\theta) p(\theta \mid \psi) \, \mathrm{d}\theta \tag{10.5}$$

直接最大化方程(10.5)通常是计算上不可行的。交叉熵方法所采取的策略是从ψ的初始值开始，通常选择该值以使分布覆盖相关参数空间。在每次迭代中，我们从相关联的分布中抽取m个样本，然后更新ψ以拟合精英样本。对于拟合，我们通常使用最大似然估计（第4.1节）。[8] 我们在固定次数的迭代后停止，或者直到搜索分布高度集中于一个最优解。图10.5展示了该算法在一个简单问题上的应用。

8 最大似然估计对应于选择ψ，使得搜索分布与精英样本之间的*cross entropy*见附录A.9)最小化。

10.5 Evolution Strategies

Evolution strategies[9] 更新由向量 $\boldsymbol{\psi}$ 参数化的搜索分布，在在每次迭代中，他们不是将分布拟合到一组精英样本上，而是通过沿着梯度方向迈出一步来更新分布。[10]

目标函数在方程（10.5）中的梯度可以按以下方式计算：[11]

9 D. Wierstra, T. Schaul, T. Glasmachers, Y. Sun, J. Peters 和 J. Schmidhuber, “自然进化策略”，*Journal of Machine Learning Research*，第15卷，第949–980页，2014年。

10 我们实际上是在进行梯度上升，这在附录A.11中有回顾。

11 策略参数θ在此处未加粗，因为它不需要是向量。然而，$\boldsymbol{\psi}$以粗体表示，因为当处理目标梯度时，我们要求它是一个向量。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct CrossEntropyPolicySearch
    p       # initial distribution
    m       # number of samples
    m_elite # number of elite samples
    k_max   # number of iterations
end

function optimize_dist(M::CrossEntropyPolicySearch, π, U)
    p, m, m_elite, k_max = M.p, M.m, M.m_elite, M.k_max
    for k in 1:k_max
        θs = rand(p, m)
        us = [U(π, θs[:,i]) for i in 1:m]
        θ_elite = θs[:,sortperm(us)[(m-m_elite+1):m]]
        p = Distributions.fit(typeof(p), θ_elite)
    end
    return p
end

function optimize(M, π, U)
    return Distributions.mode(optimize_dist(M, π, U))
end
```

算法10.4. 交叉熵策略搜索，该算法迭代地改进初始设定为`p`的搜索分布。此算法以参数化策略`π(θ, s)`和策略评估函数`U`作为输入。在每次迭代中，抽取`m`个样本，并使用前`m_elite`个样本来重新拟合分布。算法在`k_max`次迭代后终止。分布`p`可以使用`Distributions.jl`包来定义。例如，我们可以定义

```
μ = [0.0,1.0]
Σ = [1.0 0.0; 0.0 1.0]
p = MvNormal(μ,Σ)
```

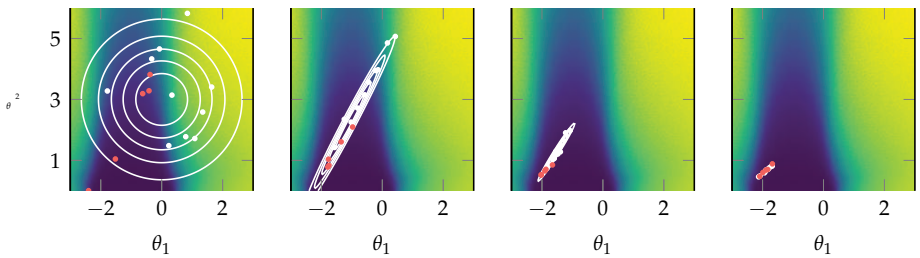


图10.5. 应用于简单调节器问题的交叉熵方法，使用多变量高斯搜索分布。每次迭代中的五个精英样本以红色显示。初始分布设置为 $\mathcal{N}([0, 3], 2I)$。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

$$\nabla_{\psi} \mathop{\mathbb{E}}_{\theta \sim p(\cdot \mid \psi)}[U(\theta)] = \nabla_{\psi} \int U(\theta) p(\theta \mid \psi) \, \mathrm{d}\theta \tag{10.6}$$

$$= \int U(\theta) \nabla_{\psi} p(\theta \mid \psi) \, \mathrm{d}\theta \tag{10.7}$$

$$= \int U(\theta) \nabla_{\psi} p(\theta \mid \psi) \frac{p(\theta \mid \psi)}{p(\theta \mid \psi)} \, \mathrm{d}\theta \tag{10.8}$$

$$= \int \left(U(\theta) \nabla_{\psi} \log p(\theta \mid \psi)\right) p(\theta \mid \psi) \, \mathrm{d}\theta \tag{10.9}$$

$$= \mathop{\mathbb{E}}_{\theta \sim p(\cdot \mid \psi)}\left[U(\theta) \nabla_{\psi} \log p(\theta \mid \psi)\right] \tag{10.10}$$

上面引入对数的做法来自所谓的 *log deriva- tive trick*，它观察到 $\nabla_{\psi} \log p(\theta \mid \psi) = \nabla_{\psi} p(\theta \mid \psi) / p(\theta \mid \psi)$。这一计算需要知道 $\nabla_{\psi} \log p(\theta \mid \psi)$，但我们通常可以解析地计算它，如例 10.2 所讨论的那样。

搜索梯度可以从 m 个样本中估计：$\theta^{(1)}, \ldots, \theta^{(m)} \sim p(\cdot \mid \psi)$:

$$\nabla_{\psi} \mathop{\mathbb{E}}_{\theta \sim p(\cdot \mid \psi)}[U(\theta)] \approx \frac{1}{m} \sum_{i=1}^{m} U(\theta^{(i)}) \nabla_{\psi} \log p(\theta^{(i)} \mid \psi) \tag{10.11}$$

该估计取决于被评估的期望效用，而期望效用本身可能变化很大。我们可以通过 *rank shaping* 使梯度估计更具鲁棒性，它用基于每个样本相对于同一迭代中其他样本的相对性能的权重来替换效用值。m 个样本按期望效用的降序排序。根据使用 $w^{(1)} \geq \cdots \geq w^{(m)}$ 的加权方案，为样本 i 分配权重 $w^{(i)}$。搜索梯度变为

$$\nabla_{\psi} \mathop{\mathbb{E}}_{\theta \sim p(\cdot \mid \psi)}[U(\theta)] \approx \sum_{i=1}^{m} w^{(i)} \nabla_{\psi} \log p(\theta^{(i)} \mid \psi) \tag{10.12}$$

一种常见的加权方案是[12]

$$w^{(i)} = \frac{\max\left(0, \log\left(\frac{m}{2} + 1\right) - \log(i)\right)}{\sum_{j=1}^{m} \max\left(0, \log\left(\frac{m}{2} + 1\right) - \log(j)\right)} - \frac{1}{m} \tag{10.13}$$

这些权重如图10.6所示，偏向更好的样本，并给予大多数样本一个小的负权重。秩次塑形减少了异常值的影响。

算法10.5提供了进化策略方法的一个实现。
图10.7展示了一个搜索进程的示例。

12 N. Hansen 和 A. Ostermeier，“在进化策略中适应任意正态变异分布：协方差矩阵适应”，载于 *IEEE International Conference on Evolutionary Computation*，1996年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

多元正态分布 $\mathcal{N}(\boldsymbol{\mu}、\ \boldsymbol{\Sigma})$，其均值为 $\boldsymbol{\mu}$，协方差为 $\boldsymbol{\Sigma}$，是一个常见的分布族。在 d 维度中的似然函数形式为

$$p(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = (2\pi)^{-\frac{d}{2}} |\boldsymbol{\Sigma}|^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$$

其中 $|\boldsymbol{\Sigma}|$ 是 $\boldsymbol{\Sigma}$ 的行列式。对数似然为

$$\log p(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = -\frac{d}{2}\log(2\pi) - \frac{1}{2}\log|\boldsymbol{\Sigma}| - \frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})$$

可以使用对数似然梯度来更新参数：

$$\nabla_{\boldsymbol{\mu}} \log p(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})$$
$$\nabla_{\boldsymbol{\Sigma}} \log p(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{2}\boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} - \frac{1}{2}\boldsymbol{\Sigma}^{-1}$$

术语$\nabla_{\boldsymbol{\Sigma}}$包含$\boldsymbol{\Sigma}$的每个条目关于对数似然的偏导数。

直接更新 $\boldsymbol{\Sigma}$ 可能无法得到正定矩阵，而这是协方差矩阵所要求的。一种解决方案是将 $\boldsymbol{\Sigma}$ 表示为乘积 $\mathrm{A}^\top\mathrm{A}$，这保证了 $\boldsymbol{\Sigma}$ 保持半正定，然后改为更新 A。将 $\boldsymbol{\Sigma}$ 替换为 $\mathrm{A}^\top\mathrm{A}$ 并对 A 求梯度得到

$$\nabla_{(\mathbf{A})} \log p(\mathbf{x} \mid \boldsymbol{\mu}, \mathbf{A}) = \mathbf{A}\left[\nabla_{\boldsymbol{\Sigma}} \log p(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) + \nabla_{\boldsymbol{\Sigma}} \log p(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma})^\top\right]$$

示例 10.2. 多元高斯分布对数似然梯度方程的推导。关于原始推导以及处理正定协方差矩阵的几种更复杂的解决方案，请参见 D. Wierstra、T. Schaul、T. Glasmachers、Y. Sun、J. Peters 和 J. Schmidhuber，“Natural Evolution Strategies,” *Journal of Machine Learning Research*, vol. 15, pp. 949–980, 2014。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

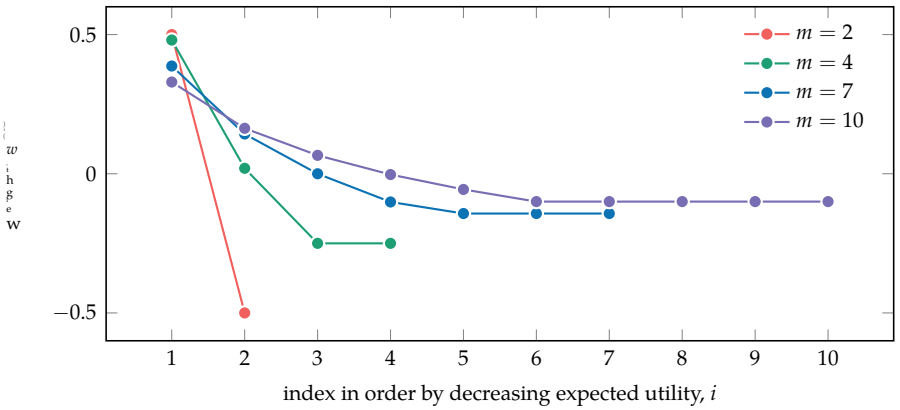


图10.6。使用等式（10.13）构造的几种加权。

```
struct EvolutionStrategies
    D       # distribution constructor
    ψ       # initial distribution parameterization
    ∇logp   # log search likelihood gradient
    m       # number of samples
    α       # step factor
    k_max   # number of iterations
end

function evolution_strategy_weights(m)
    ws = [max(0, log(m/2+1) - log(i)) for i in 1:m]
    ws ./= sum(ws)
    ws .-= 1/m
    return ws
end

function optimize_dist(M::EvolutionStrategies, π, U)
    D, ψ, m, ∇logp, α = M.D, M.ψ, M.m, M.∇logp, M.α
    ws = evolution_strategy_weights(m)
    for k in 1:M.k_max
        θs = rand(D(ψ), m)
        us = [U(π, θs[:,i]) for i in 1:m]
        sp = sortperm(us, rev=true)
        ∇ = sum(w.*∇logp(ψ, θs[:,i]) for (w,i) in zip(ws,sp))
        ψ += α.*∇
    end
    return D(ψ)
end
```

算法10.5. 一种用于更新策略参数化搜索分布 `D(ψ)` 的进化策略方法，适用于策略 `π(θ, s)`。此实现还采用初始搜索分布参数化 `ψ`、对数搜索似然梯度 `∇logp(ψ, θ)`、策略评估函数 `U` 以及迭代次数 `k_max`。在每次迭代中，抽取 `m` 个参数化样本，并用于估计搜索梯度。随后，以步长因子 `α` 应用该梯度。我们可以使用 `Distributions.jl` 来定义 `D(ψ)`。例如，如果我们想定义 `D` 来构建具有给定均值 `ψ` 和固定协方差 `Σ` 的高斯分布，我们可以使用

```
D(ψ) = MvNormal(ψ, Σ)。
```

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

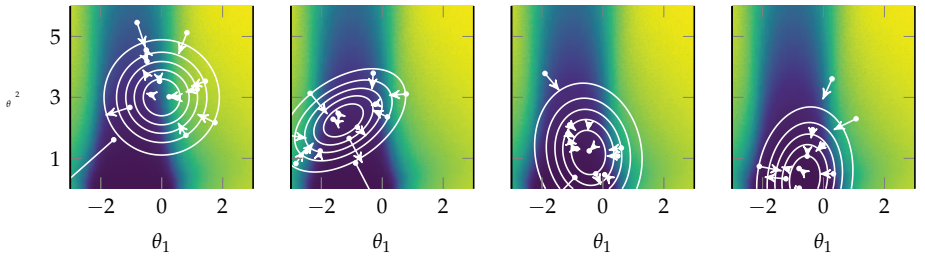


图10.7. 应用多元高斯搜索分布的进化策略（算法10.5）解决简单调节器问题。样本以白色显示，及其搜索梯度贡献，$w\nabla \log p$。

10.6 Isotropic Evolutionary Strategies

上一节介绍了能够处理一般搜索分布的进化策略。本节将假设搜索分布为*spherical*或*isotropic*高斯分布，其中协方差矩阵形式为$\sigma^2\mathbf{I}$.[13]在此假设下，该分布的期望效用

在方程 (10.5) 中引入的 [14] 简化为

$$\mathop{\mathbb{E}}_{\boldsymbol{\theta}\sim\mathcal{N}(\boldsymbol{\psi},\sigma^2\mathbf{I})}[U(\boldsymbol{\theta})] = \mathop{\mathbb{E}}_{\boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\mathbf{I})}[U(\boldsymbol{\psi}+\sigma\boldsymbol{\epsilon})] \tag{10.14}$$

$\boldsymbol{\mu}+$

搜索梯度简化为

$$\begin{aligned}
\nabla_{\boldsymbol{\psi}} \mathop{\mathbb{E}}_{\boldsymbol{\theta}\sim\mathcal{N}(\boldsymbol{\psi},\sigma^2\mathbf{I})}[U(\boldsymbol{\theta})] &= \mathop{\mathbb{E}}_{\boldsymbol{\theta}\sim\mathcal{N}(\boldsymbol{\psi},\sigma^2\mathbf{I})}\left[U(\boldsymbol{\theta})\nabla_{\boldsymbol{\psi}}\log p(\boldsymbol{\theta}\mid\boldsymbol{\psi},\sigma^2\mathbf{I})\right] && (10.15)\\
&= \mathop{\mathbb{E}}_{\boldsymbol{\theta}\sim\mathcal{N}(\boldsymbol{\psi},\sigma^2\mathbf{I})}\left[U(\boldsymbol{\theta})\frac{1}{\sigma^2}(\boldsymbol{\theta}-\boldsymbol{\psi})\right] && (10.16)\\
&= \mathop{\mathbb{E}}_{\boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\mathbf{I})}\left[U(\boldsymbol{\psi}+\sigma\boldsymbol{\epsilon})\frac{1}{\sigma^2}(\sigma\boldsymbol{\epsilon})\right] && (10.17)\\
&= \frac{1}{\sigma}\mathop{\mathbb{E}}_{\boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\mathbf{I})}[U(\boldsymbol{\psi}+\sigma\boldsymbol{\epsilon})\boldsymbol{\epsilon}] && (10.18)
\end{aligned}$$

算法10.6提供了该策略的实现。此实现包含了*mirrored sampling*.[15]我们从搜索中采样$m/2$个值

分布，然后通过围绕均值镜像生成另外的$m/2$个样本。镜像样本减少了梯度估计的方差。[16]

使用这种技术的好处如图10.8所示。

13 将这种方法应用于策略搜索的一个例子由 T. Salimans、J. Ho、X. Chen、S. Sidor 和 I. Sutskever 在 2017 年的 “Evolution Strategies as a Scalable Alternative to Reinforcement Learning” 中进行了探讨。arXiv: 1703.03864v2。

14 通常，如果 $\mathbf{A}^\top\mathbf{A} = \boldsymbol{\Sigma}$，那么 $\boldsymbol{\theta} = \mathbf{A}^\top\boldsymbol{\epsilon}$ 将 $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ 变换为一个样本 $\boldsymbol{\theta} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$。

15 D. Brockhoff, A. Auger, N. Hansen, D. Arnold, 和 T. Hohm, “进化策略的镜像采样与序列选择”，收录于 *International Confer-*, 2010. *ence on Parallel Problem Solving from Nature*

16 该技术由 T. Salimans、J. Ho、X. Chen、S. Sidor 和 I. Sutskever 在 2017 年实现，论文为 “Evolution Strategies as a Scalable Alternative to Reinforcement Learning”，arXiv: 1703.03864v2。他们还包含了其他技术，包括权重衰减。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct IsotropicEvolutionStrategies
    ψ       # initial mean
    σ       # initial standard deviation
    m       # number of samples
    α       # step factor
    k_max   # number of iterations
end

function optimize_dist(M::IsotropicEvolutionStrategies, π, U)
    ψ, σ, m, α, k_max = M.ψ, M.σ, M.m, M.α, M.k_max
    n = length(ψ)
    ws = evolution_strategy_weights(2*div(m,2))
    for k in 1:k_max
        ϵs = [randn(n) for i in 1:div(m,2)]
        append!(ϵs, -ϵs) # weight mirroring
        us = [U(π, ψ + σ.*ϵ) for ϵ in ϵs]
        sp = sortperm(us, rev=true)
        ∇ = sum(w.*ϵs[i] for (w,i) in zip(ws,sp)) / σ
        ψ += α.*∇
    end
    return MvNormal(ψ, σ)
end
```

算法10.6。一种演化策略方法，用于更新均值ψ和协方差$\sigma^2 I$下的各向同性多元高斯搜索分布，该分布覆盖策略参数化，针对策略`π(θ, s)`。此实现还接受策略评估函数`U`、步长因子`α`和迭代次数`k_max`。在每次迭代中，抽取`m/2`个参数化样本并进行镜像，然后用于估计搜索梯度。

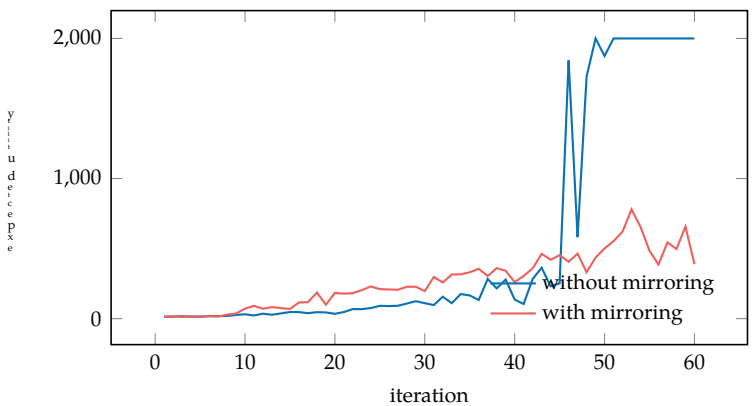


图10.8。镜像采样对各向同性进化策略影响的演示。使用`m = 10`和`σ = 0.25`，在车杆问题（附录F.3）上训练了两层神经网络策略，每次评估有六次 rollout。镜像采样显著加快并稳定了学习过程。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

10.7 Summary

- 蒙特卡洛策略评估涉及使用从初始状态分布中采样的状态进行大量模拟，计算与策略相关的期望效用。
- 局部搜索方法，如Hooke-Jeeves方法，基于微小的局部变化来改进策略。
- 遗传算法在参数空间中维护一个点群，以不同的方式重新组合它们，试图将种群推向全局最优。
- 交叉熵方法通过在每次迭代中将分布重新拟合到精英样本，迭代地改进策略参数上的搜索分布。
- 进化策略试图利用来自该分布样本的梯度信息来改进搜索分布。
- 各向同性进化策略假设搜索分布是各向同性的高斯分布。

10.8 Exercises

练习 10.1. 在蒙特卡洛策略评估中，效用估计的方差如何受到样本数量的影响?

Solution: 蒙特卡洛策略评估的方差是m个样本均值的方差。假设这些样本是独立的，因此均值的方差是单次回放评估的方差除以样本量：

$$\text{Var}[\hat{U}(\pi)] = \text{Var}\left[\frac{1}{m}\sum_{i=1}^{m} R(\tau^{(i)})\right] = \frac{1}{m^2}\text{Var}\left[\sum_{i=1}^{m} R(\tau^{(i)})\right] = \frac{1}{m^2}\left(\sum_{i=1}^{m}\text{Var}\left[R(\tau^{(i)})\right]\right) = \frac{1}{m}\text{Var}_\tau[R(\tau)]$$

其中$\hat{U}(\pi)$ 是来自蒙特卡洛策略评估的效用，而 $R(\tau)$ 是采样轨迹 τ 的轨迹奖励。因此，样本方差随着$1/m$减小。

练习10.2. 改变样本数量m和精英样本数量m_{elite}对交叉熵策略搜索有何影响?

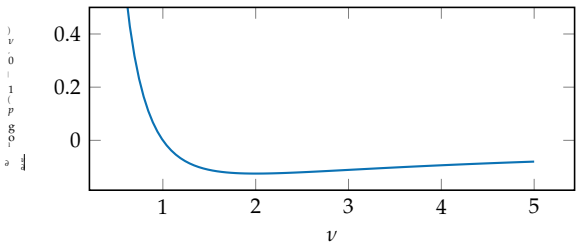
© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: 每次迭代的计算成本与样本数量呈线性关系。更多样本将更好地覆盖搜索空间，从而有更大机会识别出更优的精英样本以改进策略。精英样本的数量也有影响。将所有样本都设为精英样本不会为改进过程提供反馈。精英样本太少可能导致过早收敛到次优解。

练习 10.3. 考虑使用具有单变量高斯分布的进化策略，$\theta \sim \mathcal{N}(\mu, \nu)$。关于方差 ν 的搜索梯度是什么？当方差变得很小时会出现什么问题？

Solution: 搜索梯度是对数似然的梯度 od:

$$\begin{aligned}\frac{\partial}{\partial \nu} \log p(x \mid \mu, \nu) &= \frac{\partial}{\partial \nu} \log \frac{1}{\sqrt{2\pi\nu}} \exp\left(-\frac{(x-\mu)^2}{2\nu}\right) \\ &= \frac{\partial}{\partial \nu}\left(-\frac{1}{2}\log(2\pi) - \frac{1}{2}\log(\nu) - \frac{(x-\mu)^2}{2\nu}\right) \\ &= -\frac{1}{2\nu} + \frac{(x-\mu)^2}{2\nu^2}\end{aligned}$$



我们发现，当方差趋近于零时，梯度趋于无穷大。这是一个问题，因为当搜索分布收敛时，方差应当很小。非常大的梯度可能导致简单的上升方法过冲最优解。

练习10.4。公式(10.14)以搜索分布$\boldsymbol{\theta} \sim \mathcal{N}(\boldsymbol{\psi}、\boldsymbol{\Sigma})$的形式定义了目标。与直接使用公式(10.1)中的期望效用目标优化$\boldsymbol{\theta}$相比，这个目标有何优势?

Solution: 在策略参数周围加入高斯噪声可以平滑原始目标中的不连续性，从而使优化更加可靠。

练习10.5。在本章中，哪些方法最适合于这样一个事实：在给定问题中，多种策略类型可能表现良好?

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: Hooke-Jeeves方法改进单一策略参数化，因此无法保留多个策略。交叉熵方法和进化策略都使用搜索分布。为了成功表示多种类型的策略，必须使用多模态分布。一种常见的多模态分布是高斯混合。高斯混合无法通过解析方式拟合，但可以通过期望最大化（EM）可靠地拟合，如例4.4所示。如果种群规模足够大，遗传算法可以保留多个策略。

练习 10.6。假设我们有一个参数化策略 π_θ，我们希望使用 Hooke-Jeeves 方法对其进行优化。如果我们将参数 $\theta =$ 初始化为 0，且效用函数为 $U(\theta) = -3\theta^2 + 4\theta + 1$，那么在 Hooke-Jeeves 方法的第一轮迭代中，仍能保证策略改进的最大步长 α 是多少?

Solution: Hooke-Jeeves方法沿每个坐标方向在中心点$\pm\alpha$处评估目标函数。为了确保Hooke-Jeeves搜索的第一次迭代能有所改进，新点处至少有一个目标函数值必须改善目标函数值。对于我们的策略优化问题，这意味着我们正在寻找最大的步长α，使得$U(\theta+\alpha)$或$U(\theta-\alpha)$大于$U(\theta)$。

由于底层效用函数是抛物线且凹的，仍然能够带来改进的最大步长略小于当前点处抛物线的宽度。因此，我们计算抛物线上与当前点相对的点，θ'，在该点处有 $U(\theta') = U(\theta)$:

$$
\begin{aligned}
U(\theta) &= -3\theta^2 + 4\theta + 1 = -3(0)^2 + 4(0) + 1 = 1 \\
U(\theta) &= U(\theta') \\
1 &= -3\theta'^2 + 4\theta' + 1 \\
0 &= -3\theta'^2 + 4\theta' + 0 \\
\theta' &= \frac{-4 \pm \sqrt{4^2 - 4(-3)(0)}}{2(-3)} = \frac{-4 \pm 4}{-6} = \frac{2 \pm 2}{3} = \left\{0, \tfrac{4}{3}\right\}
\end{aligned}
$$

抛物线上与当前点相对的点因此是 $\theta' = \frac{4}{3}$。θ 与 θ' 之间的距离是 $\frac{4}{3} - 0 = \frac{4}{3}$。因此，我们在第一次迭代中能采取且仍能保证改进的最大步长略小于 $\frac{4}{3}$。

练习 10.7. 假设我们有一个由单个参数 θ 参数化的策略。我们采用进化策略方法，其搜索分布遵循伯努利分布 $p(\theta \mid \psi) = \psi^\theta(1-\psi)^{1-\theta}$。计算对数似然梯度 ∇_ψ 对数 $p(\theta \mid \psi)$。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: 对数似然梯度可以计算如下：

$$\begin{aligned}
p(\theta \mid \psi) &= \psi^{\theta}(1-\psi)^{1-\theta} \\
\log p(\theta \mid \psi) &= \log\left(\psi^{\theta}(1-\psi)^{1-\theta}\right) \\
\log p(\theta \mid \psi) &= \theta \log \psi + (1-\theta)\log(1-\psi) \\
\nabla_{\psi} \log p(\theta \mid \psi) &= \frac{d}{d\psi}\left[\theta \log \psi + (1-\theta)\log(1-\psi)\right] \\
\nabla_{\psi} \log p(\theta \mid \psi) &= \frac{\theta}{\psi} - \frac{1-\theta}{1-\psi}
\end{aligned}$$

练习10.8。给定m =个3样本，计算用于排名塑形的搜索梯度估计的样本权重。

Solution: 我们首先计算方程（10.13）中第一项的分子，对于所有 i：

$$\begin{aligned}
i = 1 \qquad & \max\left(0, \log\left(\tfrac{3}{2}+1\right) - \log 1\right) = \log \tfrac{5}{2} \\
i = 2 \qquad & \max\left(0, \log\left(\tfrac{3}{2}+1\right) - \log 2\right) = \log \tfrac{5}{4} \\
i = 3 \qquad & \max\left(0, \log\left(\tfrac{3}{2}+1\right) - \log 3\right) = 0
\end{aligned}$$

现在，我们计算权重：

$$\begin{aligned}
w^{(1)} &= \frac{\log \frac{5}{2}}{\log \frac{5}{2} + \log \frac{5}{4} + 0} - \frac{1}{3} = 0.47 \\
w^{(2)} &= \frac{\log \frac{5}{4}}{\log \frac{5}{2} + \log \frac{5}{4} + 0} - \frac{1}{3} = -0.14 \\
w^{(3)} &= \frac{0}{\log \frac{5}{2} + \log \frac{5}{4} + 0} - \frac{1}{3} = -0.33
\end{aligned}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

11 Policy Gradient Estimation

前一章讨论了直接优化策略参数以最大化期望效用的几种方法。在许多应用中，使用效用相对于策略参数的梯度来指导优化过程通常很有用。本章讨论了几种从轨迹 rollout 中估计该梯度的方法。[1] 一个主要挑战是

1关于该主题的额外资源是M. C. Fu的“梯度估计”，收录于*Simulation*，S. G. Henderson和B. L. Nelson编辑，爱思唯尔，2006年，第575-616页。

这种方法的方差是由于轨迹的随机性质导致的估计方差，这种随机性源自环境以及我们对环境的探索。下一章将讨论如何使用这些算法来估计梯度，以实现策略优化。

11.1 Finite Difference

Finite difference 方法通过函数评估中的微小变化来估计函数的梯度。回顾单变量函数 f 的导数是

$$\frac{\mathrm{d}f}{\mathrm{d}x}(x) = \lim_{\delta \to 0} \frac{f(x+\delta) - f(x)}{\delta} \tag{11.1}$$

x 处的导数可以通过一个足够小的步长 $\delta > 0$ 来近似：

$$\frac{\mathrm{d}f}{\mathrm{d}x}(x) \approx \frac{f(x+\delta) - f(x)}{\delta} \tag{11.2}$$

此近似在图11.1中说明。

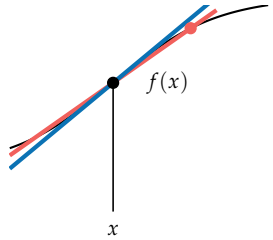


图11.1. 有限差分法通过评估x附近的一点来近似$f(x)$的导数。有限差分近似，以红色表示，与以蓝色表示的真正导数并不完全匹配。

多元函数 f 在输入长度为 n 时的梯度是

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}(\mathbf{x}), \ldots, \frac{\partial f}{\partial x_n}(\mathbf{x})\right] \tag{11.3}$$

有限差分可以应用于每个维度来估计梯度。

在策略优化的背景下，我们希望估计遵循参数为θ的策略所带来的期望效用的梯度：

$$\nabla U(\theta) = \left[\frac{\partial U}{\partial \theta_1}(\theta), \ldots, \frac{\partial U}{\partial \theta_n}(\theta)\right] \tag{11.4}$$

$$\approx \left[\frac{U(\theta + \delta \mathbf{e}^{(1)}) - U(\theta)}{\delta}, \ldots, \frac{U(\theta + \delta \mathbf{e}^{(n)}) - U(\theta)}{\delta}\right] \tag{11.5}$$

其中 e(i) 是第 i 个 *standard basis* 向量，除第 i 个分量设为 1 外，其余均为零。

如第10.1节所述，我们需要模拟策略滚动以估计{v*}。我们可以使用算法11.1生成轨迹。根据这些轨迹，我们可以计算其回报并估计与策略相关的效用。算法11.2通过为每个组件模拟m次滚动并对回报取平均，实现了方程(11.5)中的梯度估计。

```
function simulate(𝒫::MDP, s, π, d)
    τ = []
    for i = 1:d
        a = π(s)
        s′, r = 𝒫.TR(s,a)
        push!(τ, (s,a,r))
        s = s′
    end
    return τ
end
```

算法11.1. 一种生成与问题$\mathcal{P}$相关的轨迹的方法，该轨迹从状态s开始，执行策略π至深度d。它创建一个包含状态-动作-奖励元组的向量τ。

得出策略梯度准确估计的一个主要挑战是轨迹奖励的方差可能相当高。减少梯度估计中由此产生的方差的一种方法是让每个rollout共享相同的随机生成器种子。[2]这种方法可能会有帮助，例如，在某些情况下，某次回放（rollout）恰好在早期就遇到了低概率的过渡。由于随机生成器是共享的，其他回放也会有同样的倾向，它们的奖励也会以同样的方式偏向。

2 这种随机种子共享用于PEGASUS算法。A. Y. Ng和M. Jordan，“大规模MDP和POMDP的策略搜索方法”，载于*Conference on Uncertainty in Artificial Intelligence*，2000年

策略表示对策略梯度有显著影响。例11.1展示了策略梯度对策略参数化的敏感性。当参数尺度不同的时候，策略优化的有限差分方法可能表现不佳。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct FiniteDifferenceGradient
    𝒫 # problem
    b # initial state distribution
    d # depth
    m # number of samples
    δ # step size
end

function gradient(M::FiniteDifferenceGradient, π, θ)
    𝒫, b, d, m, δ, γ, n = M.𝒫, M.b, M.d, M.m, M.δ, M.𝒫.γ, length(θ)
    Δθ(i) = [i == k ? δ : 0.0 for k in 1:n]
    R(τ) = sum(r*γ^(k-1) for (k, (s,a,r)) in enumerate(τ))
    U(θ) = mean(R(simulate(𝒫, rand(b), s->π(θ, s), d)) for i in 1:m)
    ΔU = [U(θ + Δθ(i)) - U(θ) for i in 1:n]
    return ΔU ./ δ
end
```

算法11.2. 一种使用有限差分估计策略梯度的方法，适用于问题$\mathcal{P}$、参数化策略`π(θ, s)`以及策略参数化向量`θ`。效用估计基于`m`次展开至深度`d`。步长由`δ`给出。

考虑一个单状态、单步的MDP，具有一维连续动作空间和奖励函数$R(s, a) = a$。在这种情况下，较大的动作会产生较高的奖励。

假设我们有一个随机策略 v_{14}，该策略根据在 v_{15} 和 v_{16} 之间的均匀分布对 v_{17} 采取行动。期望值为

$$U(\theta) = \mathbb{E}[a] = \int_{\theta_1}^{\theta_2} a \frac{1}{\theta_2 - \theta_1} \, \mathrm{d}a = \frac{\theta_1 + \theta_2}{2}$$

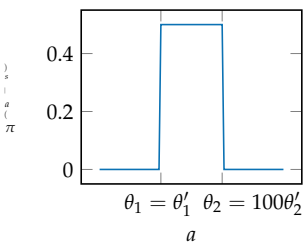
策略梯度是

$$\nabla U(\theta) = [1/2, 1/2]$$

该策略可以重新参数化，从θ_1'到$100\theta_2'$之间的均匀分布中抽取动作，对于$100\theta_2' > \theta_1'$。现在期望奖励为$(\theta_1' + 100\theta_2')/2$，而策略梯度是$[1/2, 50]$。

这两种参数化可以表示相同的策略，但它们的梯度差异很大。为第二种策略找到合适的扰动标量要困难得多，因为参数的尺度变化很大。

例11.1。一个说明策略参数化如何对策略梯度产生显著影响的例子。



© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

11.2 Regression Gradient

与其像上一节那样沿着每个坐标轴取固定步长来估计 θ 处的梯度，我们可以使用 *linear regression*[3] 来估计
来自θ的随机扰动结果的梯度。这些扰动存储在一个矩阵中，如下所示：[4]

$$\Delta\boldsymbol{\Theta} = \begin{bmatrix} (\Delta\boldsymbol{\theta}^{(1)})^\top \\ \vdots \\ (\Delta\boldsymbol{\theta}^{(m)})^\top \end{bmatrix} \tag{11.6}$$

更多的策略参数扰动往往会产生更好的梯度估计。[5]

对于这些扰动中的每一个，我们执行一次回滚并估计效用的变化：[6]

$$\Delta\mathbf{U} = \left[U(\boldsymbol{\theta} + \Delta\boldsymbol{\theta}^{(1)}) - U(\boldsymbol{\theta}), \ldots, U(\boldsymbol{\theta} + \Delta\boldsymbol{\theta}^{(m)}) - U(\boldsymbol{\theta})\right] \tag{11.7}$$

使用线性回归的策略梯度估计为[7]

$$\nabla U(\boldsymbol{\theta}) \approx \Delta\boldsymbol{\Theta}^{+}\Delta\mathbf{U} \tag{11.8}$$

算法11.3提供了这种方法的实现，其中扰动是从半径为δ的超球面上均匀抽取的。
例11.2通过一个简单函数演示了这种方法。

11.3 Likelihood Ratio

梯度估计的*likelihood ratio*方法[8]使用了以下形式的解析表达式：
$\nabla\pi_\theta$ 为了提高我们对$\nabla U(\theta)$的估计。回顾方程(10.2)可知

$$U(\boldsymbol{\theta}) = \int p_\theta(\tau) R(\tau)\, \mathrm{d}\tau \tag{11.9}$$

3 线性回归在 8.6 节中介绍。

4 这种通用方法有时被J. C. Spall称为*simultaneous*，*perturbation stochastic approximation*。*Introduction to Stochastic Search and Optimization*。Wiley，2003。与线性回归的一般联系由J. Peters和S. Schaal提供，“使用策略梯度的运动技能强化学习”，*Neural Networks*，第21卷，第4期，第682–697页，2008年。

5 一个推荐的经验法则是，扰动数量大约是参数数量的两倍。

6 这个方程显示了 *forward difference*。也可以使用其他有限差分公式，例如中心差分。

7 如第8.6节所述，X^+表示X的伪逆。

8 P. W. Glynn，“随机系统的似然比梯度估计”，*Communications of the ACM*，第33卷，第10期，第75–84页，1990年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct RegressionGradient
    𝒫 # problem
    b # initial state distribution
    d # depth
    m # number of samples
    δ # step size
end

function gradient(M::RegressionGradient, π, θ)
    𝒫, b, d, m, δ, γ = M.𝒫, M.b, M.d, M.m, M.δ, M.𝒫.γ
    Δθ = [δ.*normalize(randn(length(θ)), 2) for i = 1:m]
    R(τ) = sum(r*γ^(k-1) for (k, (s,a,r)) in enumerate(τ))
    U(θ) = R(simulate(𝒫, rand(b), s->π(θ,s), d))
    ΔU = [U(θ + Δθ) - U(θ) for Δθ in Δθ]
    return pinv(reduce(hcat, Δθ)') * ΔU
end
```

算法11.3. 一种使用有限差分估计策略梯度的方法，适用于MDP $\mathcal{P}$、随机参数化策略 `π(θ, s)` 和策略参数化向量 `θ`。策略变异向量通过归一化正态分布样本并按扰动标量 `δ` 缩放生成。共生成 `m` 个参数扰动，每个扰动在从 `b` 抽取的初始状态到深度 `d` 的rollout中进行评估，并与原始策略参数化进行比较。

因此，

$$\nabla U(\theta) = \nabla_\theta \int p_\theta(\tau) R(\tau)\, \mathrm{d}\tau \tag{11.10}$$

$$= \int \nabla_\theta p_\theta(\tau) R(\tau)\, \mathrm{d}\tau \tag{11.11}$$

$$= \int p_\theta(\tau) \frac{\nabla_\theta p_\theta(\tau)}{p_\theta(\tau)} R(\tau)\, \mathrm{d}\tau \tag{11.12}$$

$$= \mathbb{E}_\tau\left[\frac{\nabla_\theta p_\theta(\tau)}{p_\theta(\tau)} R(\tau)\right] \tag{11.13}$$

该方法的名称来源于这种轨迹似然比。这个似然比可以看作是在轨迹奖励上的似然加权采样（第3.7节）中的一个权重。

应用对数导数技巧，[9]我们有

$$\nabla U(\theta) = \mathbb{E}_\tau[\nabla_\theta \log p_\theta(\tau) R(\tau)] \tag{11.14}$$

我们可以使用轨迹展开来估计这个期望值。对于每条轨迹τ，我们需要计算乘积$\nabla_\theta \log p_\theta(\tau) R(\tau)$。回想一下，$R(\tau)$是与轨迹$\tau$相关的回报。如果我们有一个随机策略，[10]梯度

∇_θ 日志 $p_\theta(\tau)$ 是

$$\nabla_\theta \log p_\theta(\tau) = \sum_{k=1}^{d} \nabla_\theta \log \pi_\theta(a^{(k)} \mid s^{(k)}) \tag{11.15}$$

9 对数导数技巧在 10.5 节中引入。它利用了以下等式：

$$\nabla_\theta \log p_\theta(\tau) = \nabla_\theta p_\theta(\tau) / p_\theta(\tau)$$

10 我们使用 $\pi_\theta(a \mid s)$ 来表示策略 π_θ 从状态 s 采取动作 a 的概率（无论是密度还是质量）。

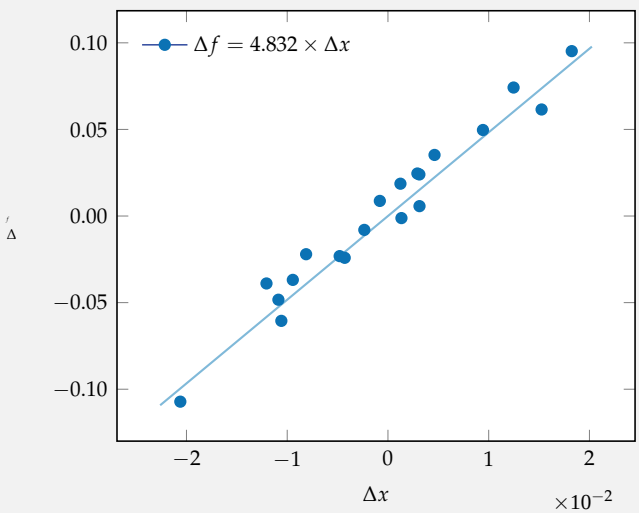
© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

例11.2. 在一维函数上使用回归梯度方法。

我们希望应用回归梯度来估计一个简单的一维函数$f(x) = x^2$在$x_0 =$处的梯度，该梯度基于从$m =$个样本中选取的 20 个样本进行评估。为了模拟策略评估中固有的随机性，我们在函数评估中加入噪声。我们生成一组扰动ΔX，其采样自$\mathcal{N}(0、\delta^2)$，并对每个扰动Δx（属于ΔX）评估$f(x_0 + \Delta x) - f(x_0)$。随后，我们可以用以下代码估计一维梯度（或导数）$\Delta X^+ \Delta F$:

```
f(x) = x^2 + 1e-2*randn()
m = 20
δ = 1e-2
ΔX = [δ.*randn() for i = 1:m]
x0 = 2.0
ΔF = [f(x0 + Δx) - f(x0) for Δx in ΔX]
pinv(ΔX) * ΔF
```

此处展示了样本和线性回归。回归线的斜率接近精确解4:



© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

因为 $p_\theta(\tau)$ 的形式为

$$p_\theta(\tau) = p(s^{(1)}) \prod_{k=1}^{d} T(s^{(k+1)} \mid s^{(k)}, a^{(k)}) \pi_\theta(a^{(k)} \mid s^{(k)}) \tag{11.16}$$

其中$s^{(k)}$和$a^{(k)}$分别是轨迹τ中的第k个状态和动作。算法11.4提供了一个实现，其中采样m条轨迹以得出梯度估计。例11.3说明了该过程。

如果我们有一个确定性策略，那么梯度计算需要：[11]

$$\nabla_\theta \log p_\theta(\tau) = \nabla_\theta \log \left[p(s^{(1)}) \prod_{k=1}^{d} T(s^{(k+1)} \mid s^{(k)}, \pi_\theta(s^{(k)})) \right] \tag{11.17}$$

$$= \sum_{k=1}^{d} \nabla_\theta \pi_\theta(s^{(k)}) \frac{\partial}{\partial a^{(k)}} \log T(s^{(k+1)} \mid s^{(k)}, a^{(k)}) \tag{11.18}$$

方程（11.17）和（11.18）要求知道转移似然，这与用于随机策略的方程（11.15）形成对比。

11 许多问题具有向量值动作 $a \in \mathbb{R}^n$。在这种情况下，$\nabla_\theta \pi_\theta(s^{(k)})$ 被替换为一个雅可比矩阵，其第 j 列为关于第 j 个动作分量的梯度，并且 $\frac{\partial}{\partial a^{(k)}} \log T(s^{(k+1)} \mid s^{(k)}, a^{(k)})$ 被替换为动作梯度。

```
struct LikelihoodRatioGradient
    𝒫 # problem
    b # initial state distribution
    d # depth
    m # number of samples
    ∇logπ # gradient of log likelihood
end

function gradient(M::LikelihoodRatioGradient, π, θ)
    𝒫, b, d, m, ∇logπ, γ = M.𝒫, M.b, M.d, M.m, M.∇logπ, M.𝒫.γ
    πθ(s) = π(θ, s)
    R(τ) = sum(r*γ^(k-1) for (k, (s,a,r)) in enumerate(τ))
    ∇U(τ) = sum(∇logπ(θ, a, s) for (s,a) in τ)*R(τ)
    return mean(∇U(simulate(𝒫, rand(b), πθ, d)) for i in 1:m)
end
```

算法11.4. 一种使用似然比技巧估计MDP $\mathcal{P}$初始状态分布为`b`的策略`π(s)`的策略梯度的方法。相对于参数化向量`θ`的梯度是使用对数策略梯度`∇logπ`从深度为`d`的`m`次展开估计的。

11.4 Reward-to-Go

似然比策略梯度方法是无偏的，但方差较高。例11.4回顾了偏差与方差。随着推演深度的增加，由于动作、状态与

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

示例 11.3. 在简单问题中应用似然比技巧来估计策略梯度。

考虑例11.1中的单步、单状态问题。假设我们有一个随机策略π_θ，它根据高斯分布$\mathcal{N}(\theta_1、\theta_2^2)$采样其动作，其中$\theta_2^2$是方差。

$$\begin{aligned}\log \pi_\theta(a \mid s) &= \log\left(\frac{1}{\sqrt{2\pi\theta_2^2}} \exp\left(-\frac{(a-\theta_1)^2}{2\theta_2^2}\right)\right) \\ &= -\frac{(a-\theta_1)^2}{2\theta_2^2} - \frac{1}{2}\log\left(2\pi\theta_2^2\right)\end{aligned}$$

对数策略似然的梯度是

$$\begin{aligned}\frac{\partial}{\partial\theta_1}\log \pi_\theta(a \mid s) &= \frac{a-\theta_1}{\theta_2^2} \\ \frac{\partial}{\partial\theta_2}\log \pi_\theta(a \mid s) &= \frac{(a-\theta_1)^2-\theta_2^2}{\theta_2^3}\end{aligned}$$

假设我们运行三次带有$\theta = [0, 1]$的滚动，采取行动$\{0.5、-1、0.7\}$，并接收相同的奖励（$R(s、a) = a$）。估计的策略梯度是

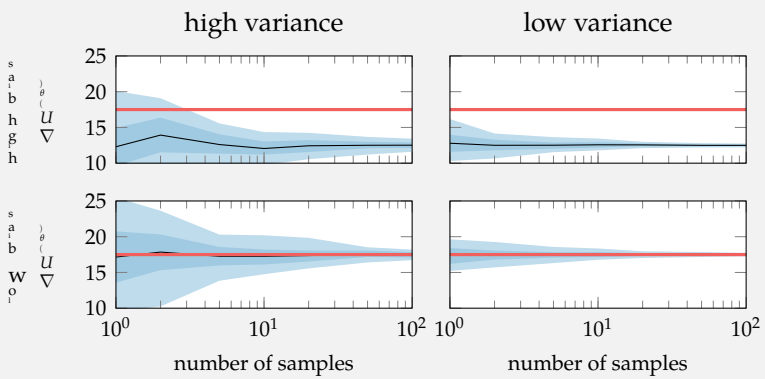
$$\begin{aligned}\nabla U(\theta) &\approx \frac{1}{m}\sum_{i=1}^{m} \nabla_\theta \log p_\theta(\tau^{(i)}) R(\tau^{(i)}) \\ &= \frac{1}{3}\left(\begin{bmatrix}0.5\\-0.75\end{bmatrix}0.5 + \begin{bmatrix}-1.0\\0.0\end{bmatrix}(-1) + \begin{bmatrix}0.7\\-0.51\end{bmatrix}0.7\right) \\ &= [0.58, -0.244]\end{aligned}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

例 11.4. 估计 $\nabla U(\theta)$ 时偏差与方差的经验演示。

当从一组模拟中估计感兴趣的某个量时，我们通常希望使用既具有低*bias*又具有低*variance*的方案。在本章中，我们想要估计$\nabla U(\theta)$。一般来说，模拟样本越多，我们就能得到更好的估计。有些方法可能会导致偏差，即使有无限多的样本，也无法得出准确的估计。有时，具有非零偏差的方法如果同时具有低方差，可能仍然具有吸引力，这意味着它们需要较少的样本就能收敛。

这里是四种名义方法对 $\nabla U(\theta)$ 进行估计的估计值图。真实值为17.5，如红线所示。我们对每种方法进行了100次模拟，每次模拟100次。随着样本数量的增加，方差减小。蓝色区域表示估计值的5%至95%和25%至75%的经验分位数。



© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

跨时间步的奖励。*reward-to-go*方法试图减少估计中的方差。

为了推导这种方法，我们从展开式（11.14）开始：

$$\nabla U(\theta) = \mathbb{E}_\tau\left[\left(\sum_{k=1}^{d} \nabla_\theta \log \pi_\theta(a^{(k)} \mid s^{(k)})\right)\left(\sum_{k=1}^{d} r^{(k)}\gamma^{k-1}\right)\right] \tag{11.19}$$

令 $f^{(k)}$ 替换 $\nabla_\theta \log \pi_\theta(a^{(k)} \mid s^{(k)})$ 以便处理。随后我们展开如下：

$$\nabla U(\theta) = \mathbb{E}_\tau\left[\left(\sum_{k=1}^{d} f^{(k)}\right)\left(\sum_{k=1}^{d} r^{(k)}\gamma^{k-1}\right)\right] \tag{11.20}$$

$$= \mathbb{E}_\tau\left[\left(f^{(1)} + f^{(2)} + f^{(3)} + \cdots + f^{(d)}\right)\left(r^{(1)} + r^{(2)}\gamma + r^{(3)}\gamma^2 + \cdots + r^{(d)}\gamma^{d-1}\right)\right] \tag{11.21}$$

$$= \mathbb{E}_\tau\begin{bmatrix} f^{(1)}r^{(1)} + f^{(1)}r^{(2)}\gamma + f^{(1)}r^{(3)}\gamma^2 + \cdots + f^{(1)}r^{(d)}\gamma^{d-1} \\ +f^{(2)}r^{(1)} + f^{(2)}r^{(2)}\gamma + f^{(2)}r^{(3)}\gamma^2 + \cdots + f^{(2)}r^{(d)}\gamma^{d-1} \\ +f^{(3)}r^{(1)} + f^{(3)}r^{(2)}\gamma + f^{(3)}r^{(3)}\gamma^2 + \cdots + f^{(3)}r^{(d)}\gamma^{d-1} \\ \vdots \\ +f^{(d)}r^{(1)} + f^{(d)}r^{(2)}\gamma + f^{(d)}r^{(3)}\gamma^2 + \cdots + f^{(d)}r^{(d)}\gamma^{d-1} \end{bmatrix} \tag{11.22}$$

第一个奖励 $r^{(1)}$ 仅受第一个动作影响。因此，它对策略梯度的贡献不应依赖于后续的时间步。我们可以如下移除其他此类违反因果关系的项：[12]

12 术语$\sum_{\ell=k}^{d} r^{(\ell)}\gamma^{\ell-k}$通常被称为步骤$k$中的*reward-to-go*。

$$\nabla U(\theta) = \mathbb{E}_\tau\begin{bmatrix} f^{(1)}r^{(1)} + f^{(1)}r^{(2)}\gamma + f^{(1)}r^{(3)}\gamma^2 + \cdots + f^{(1)}r^{(d)}\gamma^{d-1} \\ + f^{(2)}r^{(2)}\gamma + f^{(2)}r^{(3)}\gamma^2 + \cdots + f^{(2)}r^{(d)}\gamma^{d-1} \\ + f^{(3)}r^{(3)}\gamma^2 + \cdots + f^{(3)}r^{(d)}\gamma^{d-1} \\ \vdots \\ + f^{(d)}r^{(d)}\gamma^{d-1} \end{bmatrix} \tag{11.23}$$

$$= \mathbb{E}_\tau\left[\sum_{k=1}^{d} \nabla_\theta \log \pi_\theta(a^{(k)} \mid s^{(k)})\left(\sum_{\ell=k}^{d} r^{(\ell)}\gamma^{\ell-1}\right)\right] \tag{11.24}$$

$$= \mathbb{E}_\tau\left[\sum_{k=1}^{d} \nabla_\theta \log \pi_\theta(a^{(k)} \mid s^{(k)})\left(\gamma^{k-1}\sum_{\ell=k}^{d} r^{(\ell)}\gamma^{\ell-k}\right)\right] \tag{11.25}$$

$$= \mathbb{E}_\tau\left[\sum_{k=1}^{d} \nabla_\theta \log \pi_\theta(a^{(k)} \mid s^{(k)})\gamma^{k-1} r^{(k)}_{\text{to-go}}\right] \tag{11.26}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

算法11.5提供了这一点的实现。

注意，在由θ参数化的策略下，状态-动作对(s, a)的回报实际上是对该状态下状态-动作价值$Q_\theta(s、a)$的近似。动作价值函数如果已知，可以用来获得策略梯度：

$$\nabla U(\theta) = \mathbb{E}_\tau\left[\sum_{k=1}^{d} \nabla_\theta \log \pi_\theta(a^{(k)} \mid s^{(k)}) \gamma^{k-1} Q_\theta\left(s^{(k)}, a^{(k)}\right)\right] \tag{11.27}$$

```
struct RewardToGoGradient
    𝒫 # problem
    b # initial state distribution
    d # depth
    m # number of samples
    ∇logπ # gradient of log likelihood
end

function gradient(M::RewardToGoGradient, π, θ)
    𝒫, b, d, m, ∇logπ, γ = M.𝒫, M.b, M.d, M.m, M.∇logπ, M.𝒫.γ
    πθ(s) = π(θ, s)
    R(τ, j) = sum(r*γ^(k-1) for (k,(s,a,r)) in zip(j:d, τ[j:end]))
    ∇U(τ) = sum(∇logπ(θ, a, s)*R(τ,j) for (j, (s,a,r)) in enumerate(τ))
    return mean(∇U(simulate(𝒫, rand(b), πθ, d)) for i in 1:m)
end
```

算法11.5。一种利用奖励累计值来估计策略`π(s)`的策略梯度的方法，适用于初始状态分布为`b`的MDP $\mathcal{P}$。该方法基于`m`次深度为`d`的模拟轨迹，利用对数策略梯度`∇logπ`来估计关于参数化向量`θ`的梯度。

11.5 Baseline Subtraction

我们可以进一步建立在上一节中提出的方法之上，通过从奖励到目标[13]中减去一个*baseline*值来减少方差的梯度估计。这种减法不会使梯度产生偏差。

13 我们也可以从状态-动作值中减去一个基线。

我们现在减去一个基线 {v*}：

$$\nabla U(\theta) = \mathbb{E}_\tau\left[\sum_{k=1}^{d} \nabla_\theta \log \pi_\theta(a^{(k)} \mid s^{(k)}) \gamma^{k-1}\left(r_{\text{to-go}}^{(k)} - r_{\text{base}}(s^{(k)})\right)\right] \tag{11.28}$$

为了证明基线减法不会使梯度产生偏差，我们首先展开：

$$\nabla U(\theta) = \mathbb{E}_\tau\left[\sum_{k=1}^{d} \nabla_\theta \log \pi_\theta(a^{(k)} \mid s^{(k)}) \gamma^{k-1} r_{\text{to-go}}^{(k)} - \sum_{k=1}^{d} \nabla_\theta \log \pi_\theta(a^{(k)} \mid s^{(k)}) \gamma^{k-1} r_{\text{base}}(s^{(k)})\right] \tag{11.29}$$

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

linearity of expectation 表明 $\mathbb{E}[a+b] = \mathbb{E}[a] + \mathbb{E}[b]$，因此只需证明：若对每个步骤 k，期望相关基线项为 0，则方程 (11.29) 等价于方程 (11.26)：

$$\mathbb{E}_\tau\left[\nabla_\theta \log \pi_\theta(a^{(k)} \mid s^{(k)})\gamma^{k-1} r_{\text{base}}(s^{(k)})\right] = \mathbf{0} \tag{11.30}$$

我们首先将期望转化为嵌套期望，如图11.2所示：

$$\mathbb{E}_\tau\left[\nabla_\theta \log \pi_\theta(a^{(k)} \mid s^{(k)})\gamma^{k-1} r_{\text{base}}(s^{(k)})\right] = \mathbb{E}_{\tau_{1:k}}\left[\mathbb{E}_{\tau_{k+1:d}}\left[\nabla_\theta \log \pi_\theta(a^{(k)} \mid s^{(k)})\gamma^{k-1} r_{\text{base}}(s^{(k)})\right]\right] \tag{11.31}$$

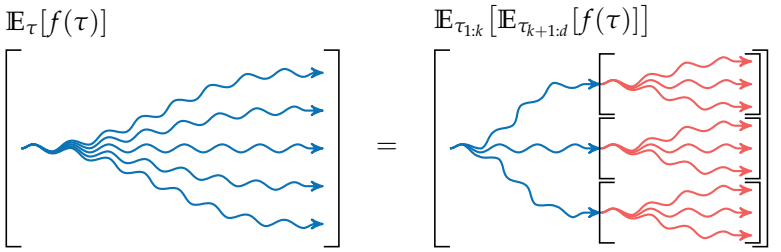


图11.2。从策略中采样得到的轨迹的函数期望可以视为对子轨迹嵌套期望的期望。数学推导见练习11.4。

我们继续推导，使用与第11.3节相同的对数导数技巧：

$$\begin{aligned}
&\mathbb{E}_{\tau_{1:k}}\left[\mathbb{E}_{\tau_{k+1:d}}\left[\nabla_\theta \log \pi_\theta(a^{(k)} \mid s^{(k)})\gamma^{k-1} r_{\text{base}}(s^{(k)})\right]\right] \\
&\quad = \mathbb{E}_{\tau_{1:k}}\left[\gamma^{k-1} r_{\text{base}}(s^{(k)})\ \mathbb{E}_{\tau_{k+1:d}}\left[\nabla_\theta \log \pi_\theta(a^{(k)} \mid s^{(k)})\right]\right] && (11.32)\\
&\quad = \mathbb{E}_{\tau_{1:k}}\left[\gamma^{k-1} r_{\text{base}}(s^{(k)})\ \mathbb{E}_{a^{(k)}}\left[\nabla_\theta \log \pi_\theta(a^{(k)} \mid s^{(k)})\right]\right] && (11.33)\\
&\quad = \mathbb{E}_{\tau_{1:k}}\left[\gamma^{k-1} r_{\text{base}}(s^{(k)}) \int \nabla_\theta \log \pi_\theta(a^{(k)} \mid s^{(k)}) \pi_\theta(a^{(k)} \mid s^{(k)})\, \mathrm{d}a^{(k)}\right] && (11.34)\\
&\quad = \mathbb{E}_{\tau_{1:k}}\left[\gamma^{k-1} r_{\text{base}}(s^{(k)}) \int \frac{\nabla_\theta \pi_\theta(a^{(k)} \mid s^{(k)})}{\pi_\theta(a^{(k)} \mid s^{(k)})} \pi_\theta(a^{(k)} \mid s^{(k)})\, \mathrm{d}a^{(k)}\right] && (11.35)\\
&\quad = \mathbb{E}_{\tau_{1:k}}\left[\gamma^{k-1} r_{\text{base}}(s^{(k)})\, \nabla_\theta \int \pi_\theta(a^{(k)} \mid s^{(k)})\, \mathrm{d}a^{(k)}\right] && (11.36)\\
&\quad = \mathbb{E}_{\tau_{1:k}}\left[\gamma^{k-1} r_{\text{base}}(s^{(k)})\, \nabla_\theta 1\right] && (11.37)\\
&\quad = \mathbb{E}_{\tau_{1:k}}\left[\gamma^{k-1} r_{\text{base}}(s^{(k)})\, \mathbf{0}\right] && (11.38)
\end{aligned}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

因此，减去项 $r_{\text{base}}(s^{(k)})$ 不会使估计产生偏差。这一推导假设了连续的状态和动作空间。同样的结果也适用于离散空间。

我们可以为梯度的每个分量选择不同的 $r_{\text{base}}(s)$，并且我们会选择它们以最小化方差。为简单起见，我们将忽略对 s 的依赖，并将每个基线分量视为常数。[14] 为了紧凑

在推导中书写这些方程时，我们定义

14 一些方法使用{v*}来近似状态依赖的基线。选择合适的基线函数往往很困难。J. Peters 和 S. Schaal，“使用策略梯度的运动技能强化学习”，*Neural Networks*，第21卷，第4期，第682-697页，2008年。

$$\ell_i(a, s, k) = \gamma^{k-1} \frac{\partial}{\partial \theta_i} \log \pi_\theta(a \mid s) \tag{11.39}$$

方程（11.28）中我们的梯度估计的第i个分量的方差是

$$\mathop{\mathbb{E}}_{a,s,r_{\text{to-go}},k}\left[\left(\ell_i(a, s, k)\left(r_{\text{to-go}} - r_{\text{base},i}\right)\right)^2\right] - \mathop{\mathbb{E}}_{a,s,r_{\text{to-go}},k}\left[\ell_i(a, s, k)\left(r_{\text{to-go}} - r_{\text{base},i}\right)\right]^2 \tag{11.40}$$

其中期望是针对轨迹样本中的 $(a, s, r_{\text{to-go}})$ 元组，且 k 是每个元组的深度。

我们刚刚展示了第二项为零。因此，我们可以专注于通过针对基线求导并令其为零，来选择$r_{\text{base},i}$以最小化第一项：

$$\begin{aligned}
&\frac{\partial}{\partial r_{\text{base},i}} \mathop{\mathbb{E}}_{a,s,r_{\text{to-go}},k}\left[\left(\ell_i(a, s, k)\left(r_{\text{to-go}} - r_{\text{base},i}\right)\right)^2\right] \\
&\quad = \frac{\partial}{\partial r_{\text{base},i}}\left(\mathop{\mathbb{E}}_{a,s,r_{\text{to-go}},k}\left[\ell_i(a, s, k)^2 r_{\text{to-go}}^2\right] - 2\mathop{\mathbb{E}}_{a,s,r_{\text{to-go}},k}\left[\ell_i(a, s, k)^2 r_{\text{to-go}} r_{\text{base},i}\right] + r_{\text{base},i}^2 \mathop{\mathbb{E}}_{a,s,k}\left[\ell_i(a, s, k)^2\right]\right) \qquad (11.41) \\
&\quad = -2\mathop{\mathbb{E}}_{a,s,r_{\text{to-go}},k}\left[\ell_i(a, s, k)^2 r_{\text{to-go}}\right] + 2 r_{\text{base},i} \mathop{\mathbb{E}}_{a,s,k}\left[\ell_i(a, s, k)^2\right] = 0 \qquad (11.42)
\end{aligned}$$

求解 $r_{\text{base},i}$ 可得到使方差最小的基线分量：

$$r_{\text{base},i} = \frac{\mathbb{E}_{a,s,r_{\text{to-go}},k}\left[\ell_i(a, s, k)^2 r_{\text{to-go}}\right]}{\mathbb{E}_{a,s,k}\left[\ell_i(a, s, k)^2\right]} \tag{11.43}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct BaselineSubtractionGradient
    𝒫 # problem
    b # initial state distribution
    d # depth
    m # number of samples
    ∇logπ # gradient of log likelihood
end

function gradient(M::BaselineSubtractionGradient, π, θ)
    𝒫, b, d, m, ∇logπ, γ = M.𝒫, M.b, M.d, M.m, M.∇logπ, M.𝒫.γ
    πθ(s) = π(θ, s)
    ℓ(a, s, k) = ∇logπ(θ, a, s)*γ^(k-1)
    R(τ, k) = sum(r*γ^(j-1) for (j,(s,a,r)) in enumerate(τ[k:end]))
    numer(τ) = sum(ℓ(a,s,k).^2*R(τ,k) for (k,(s,a,r)) in enumerate(τ))
    denom(τ) = sum(ℓ(a,s,k).^2 for (k,(s,a)) in enumerate(τ))
    base(τ) = numer(τ) ./ denom(τ)
    trajs = [simulate(𝒫, rand(b), πθ, d) for i in 1:m]
    rbase = mean(base(τ) for τ in trajs)
    ∇U(τ) = sum(ℓ(a,s,k).*(R(τ,k).-rbase) for (k,(s,a,r)) in enumerate(τ))
    return mean(∇U(τ) for τ in trajs)
end
```

算法11.6. 针对MDP $\mathcal{P}$、策略 π 和初始状态分布 b，采用奖励至去及基线减法进行似然比梯度估计。相对于参数向量 θ 的梯度，通过深度为 d 的 m 轮模拟，利用对数策略梯度 ∇logπ 进行估计。

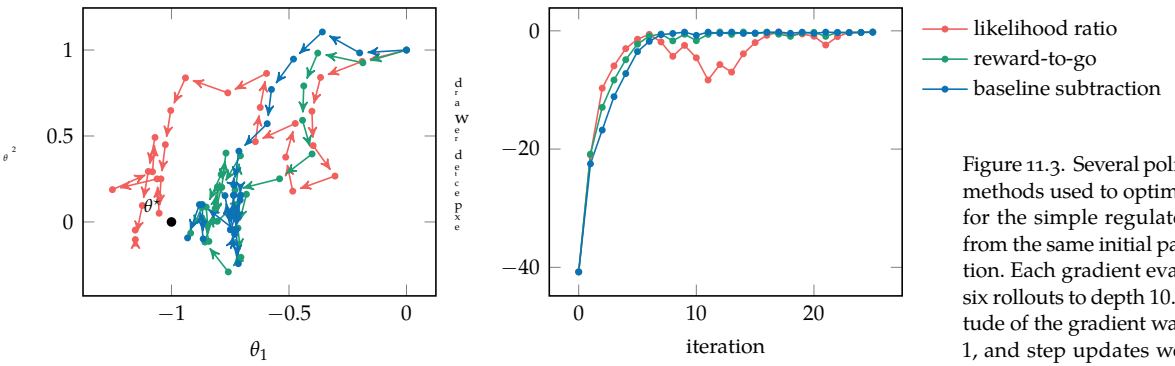


Figure 11.3. Several policy gr梯度 methods used to optimize p政策 for the simple regulator problem from the same initial parame泰里扎- tion. Each gradient evaluation 跑了 six rollouts to depth 10. The m火- tude of the gradient was limited到 1, and step updates were ap应用 with step size 0.2. The optim阿尔波尔- icy parameterization is sho黑色 显示。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

通常使用带有基线减法的似然比策略梯度估计（算法11.6）。[15]。图11.3比较了所讨论的方法
此处。

定性地说，当考虑状态-动作对的梯度贡献时，我们真正关心的是一个动作相对于另一个动作的相对价值。如果某一状态下所有动作都产生相同的高价值，那么梯度中就没有真正的信号，基线减法可以将其归零。我们想要识别出那些产生高于其他动作价值的动作，而不论动作间的平均值如何。

15 这种组合用于称为 *REINFORCE* 的算法类中，由 R. J. Williams 在“用于连接主义强化学习的简单统计梯度跟随算法”，*Machine Learning*，第 8 卷，第 229–256 页，1992 年中引入。

动作值的替代方案是 *advantage*、$A(s、a) = Q(s、a) - U(s)$。在基线减法中使用状态值函数会产生优势。使用优势的策略梯度是无偏的，并且通常具有低得多的方差。梯度计算采用以下形式：

$$\nabla U(\theta) = \mathbb{E}_\tau \left[\sum_{k=1}^{d} \nabla_\theta \log \pi_\theta(a^{(k)} \mid s^{(k)}) \gamma^{k-1} A_\theta\left(s^{(k)}, a^{(k)}\right) \right] \tag{11.44}$$

与状态和动作值函数一样，优势函数通常是未知的。需要第13章中介绍的其他方法来近似它。

11.6 Summary

- 可以使用有限差分来估计梯度。
- 线性回归也可以用来提供更稳健的策略梯度估计。
- 似然比可用于推导出一种不依赖于转移模型的随机策略的策略梯度形式。
- 使用奖励到当前和基线减法可以显著减少策略梯度的方差。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

11.7 Exercises

练习 11.1. 如果我们使用 m 次模拟来估计由 n 维参数向量 θ 定义的给定参数化策略 π_θ 的预期折扣回报，那么使用有限差分方法计算策略梯度总共需要执行多少次模拟?

Solution: 为了使用有限差分方法估计策略梯度，我们需要估计在当前参数向量$U(\theta)$下策略的效用，以及在当前参数向量$U(\theta + \delta e^{(i)})$的所有$n$种变体下的效用，其中$i$ =为1 : n。由于我们使用m次rollouts来估计每一个，我们总共需要进行$m(n+1)$次rollouts。

练习 11.2。假设我们有一个机械臂，能够进行实验操作各种物体。我们希望利用似然比策略梯度或其扩展来训练一个能够高效拾取和移动这些物体的策略。使用确定性策略还是随机策略会更直接，为什么?

Solution: 当与确定性策略一起使用时，似然比策略梯度需要显式表示转移概率。对于现实世界的机器人手臂操作任务，指定准确的显式转移模型将具有挑战性。计算随机策略的策略梯度不需要显式表示转移概率，这使得使用随机策略更为直接。

练习11.3。考虑形式为的策略梯度

$$\nabla_\theta U(\theta) = \mathbb{E}_\tau \left[\sum_{k=1}^{d} \gamma^{k-1} y \, \nabla_\theta \log \pi_\theta(a^{(k)} \mid s^{(k)}) \right]$$

哪个 以下y的哪些值会产生有效的策略梯度? 解释 为什么。

(a) $\gamma^{1-k} \sum_{\ell=1}^{\infty} r^{(\ell)} \gamma^{\ell-1}$

(b) $\sum_{\ell=k}^{\infty} r^{(\ell)} \gamma^{\ell-k}$

(c) $\left(\sum_{\ell=k}^{\infty} r^{(\ell)} \gamma^{\ell-k} \right) - r_{\text{base}}(s^{(k)})$

(d) $U(s^{(k)})$

(e) $Q(s^{(k)}, a^{(k)})$

(f) $A(s^{(k)}, a^{(k)})$

(g) $r^{(k)} + \gamma U(s^{(k+1)}) - U(s^{(k)})$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution:
(a) $\sum_{\ell=1}^{\infty} r^{(\ell)}$ 得到总折扣奖励，即

$$\gamma^{k-1}\gamma^{1-k}\sum_{\ell=1}^{\infty} r^{(\ell)}\gamma^{\ell-1} = \sum_{\ell=1}^{\infty} r^{(\ell)}\gamma^{\ell-1}$$

并产生有效的策略梯度，如公式(11.19)所示。
(b) $\sum_{\ell=k}^{\infty} r^{(\ell)}\gamma^{\ell-k}$ 是回报，产生有效的策略梯度，如公式 (11.26) 所示。

(c) $\left(\sum_{\ell=k}^{\infty} r^{(\ell)}\right) - r_{\text{base}}(s^{(k)})$ 是减去基线的回报，产生有效的策略梯度，如公式 (11.28) 所示。(d) $U(s^{(k)})$ 是状态价值函数，不产生有效的策略梯度。(e) $Q(s^{(k)}, a^{(k)})$ 是状态-动作价值函数，产生有效的策略梯度，如公式 (11.27) 所示。(f) $A(s^{(k)}, a^{(k)})$ 是优势函数，产生有效的策略梯度，如公式 (11.44) 所示。(g) $r^{(k)} + \gamma U(s^{(k+1)}) - U(s^{(k)})$ 是时间差分残差（将在第13章进一步讨论），并且产生有效的策略梯度，因为它是优势函数的无偏近似。

练习11.4。证明对于步骤k，$\mathbb{E}_{\tau\sim\pi}[f(\tau)] = \mathbb{E}_{\tau_{1:k}\sim\pi}[\mathbb{E}_{\tau_{k:d}\sim\pi}[f(\tau)]]$。

Solution: 嵌套期望可以通过将期望写成积分形式，然后再转换回来来证明：

$$\begin{aligned}
&\mathbb{E}_{\tau\sim\pi}[f(\tau)] = \\
&= \int p(\tau) f(\tau)\,\mathrm{d}\tau \\
&= \int \left(p(s^{(1)})\prod_{k=1}^{d} p(s^{(k+1)} \mid s^{(k)}, a^{(k)})\pi(a^{(k)} \mid s^{(k)})\right) f(\tau)\,\mathrm{d}\tau \\
&= \int\int\int\int\cdots\int \left(p(s^{(1)})\prod_{k=1}^{d} p(s^{(k+1)} \mid s^{(k)}, a^{(k)})\pi(a^{(k)} \mid s^{(k)})\right) f(\tau)\,\mathrm{d}s^{(d)}\cdots\mathrm{d}a^{(2)}\,\mathrm{d}s^{(2)}\,\mathrm{d}a^{(1)}\,\mathrm{d}s^{(1)} \\
&= \mathop{\mathbb{E}}_{\tau_{1:k}\sim\pi}\left[\int\int\int\int\cdots\int \left(\prod_{q=k}^{d} p(s^{(q+1)} \mid s^{(q)}, a^{(q)})\pi(a^{(q)} \mid s^{(q)})\right) f(\tau)\,\mathrm{d}s^{(d)}\cdots\mathrm{d}a^{(k+1)}\,\mathrm{d}s^{(k+1)}\,\mathrm{d}a^{(k)}\,\mathrm{d}s^{(k)}\right] \\
&= \mathbb{E}_{\tau_{1:k}\sim\pi}[\mathbb{E}_{\tau_{k:d}\sim\pi}[f(\tau)]]
\end{aligned}$$

练习11.5。我们的回归梯度实现（算法11.3）拟合了一个从扰动到回报差异的线性映射，$U(\theta + \Delta\theta^{(i)}) - U(\theta)$。我们针对每个$m$扰动评估$U(\theta + \Delta\theta^{(i)})$和$U(\theta)$，因此总共重新评估$U(\theta)$达$m$次。我们如何更有效地重新分配样本？

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: 一种方法是评估 $U(\theta)$ 一次，并为每个扰动使用相同的值，从而仅进行 $m + 1$ 次评估。准确估计 $U(\theta)$ 对于准确的回归梯度估计尤为重要。另一种方法是仍然计算 $U(\theta)$ 一次，但使用 m 次展开，从而保持每次迭代的总展开次数不变。这种方法使用的计算量与算法 11.3 相同，但它可以产生更可靠的梯度估计。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

12 Policy Gradient Optimization

我们可以使用策略梯度的估计来驱动参数空间的搜索，以趋向最优策略。上一章概述了估计该梯度的方法。本章解释如何使用这些估计来指导优化。我们从梯度上升开始，它在每次迭代中简单地沿梯度方向迈步。确定步长是一个主要挑战。较大的步长可以更快地接近最优值，但可能会过冲。自然策略梯度修改梯度的方向，以更好地处理参数各分量之间敏感度的变化。我们以信赖域方法结束，该方法以与自然梯度方法完全相同的方式开始，以获得候选策略。然后，它在策略空间中沿着连接原始策略与该候选策略的线段进行搜索，以找到更好的策略。

12.1 Gradient Ascent Update

我们可以使用在附录 A.11 中回顾的 *gradient ascent* () 来找到一个由 θ 参数化的策略，该策略最大化期望效用 $U(\theta)$。梯度上升是一种 *iterated ascent* 方法，它涉及在每次迭代时在参数空间中采取步骤，以试图提高相关策略的质量。本章讨论的所有方法都是迭代上升方法，但它们在如何采取步骤方面有所不同。本节讨论的梯度上升方法沿着 $\nabla U(\theta)$ 的方向采取步骤，这可以使用上一章讨论的方法之一来估计。θ 的更新是

$$\theta \leftarrow \theta + \alpha \nabla U(\theta) \tag{12.1}$$

其中步长等于步长因子$\alpha > 0$乘以梯度的大小。

算法12.1实现了一种采取此类步骤的方法。此方法可调用固定次数的迭代，或直至θ或$U(\theta)$收敛。梯度上升，以及本章讨论的其他算法，并不保证收敛至最优策略。然而，存在一些技巧促使收敛至*locally optimal*策略，即在参数空间中采取无穷小步长也无法得到更优策略。一种方法是每次步骤中衰减步长因子。[1]

1 这种方法以及许多其他方法都由M. J. Kochenderfer 和 T. A. Wheeler在 *Algorithms for Optimization* 中详细涵盖。MIT Press，2019年。

```
struct PolicyGradientUpdate
    ∇U # policy gradient estimate
    α  # step factor
end

function update(M::PolicyGradientUpdate, θ)
    return θ + M.α * M.∇U(θ)
end
```

算法12.1. 策略优化的梯度上升法。它从点θ出发，沿梯度∇U方向迈出一步，步长因子为α。我们可以使用上一章中的任一方法来计算∇U。

非常大的梯度往往会越过最优值，并可能由多种原因引起。某些问题的奖励，例如2048问题（附录F.2），可能会变化几个数量级。使梯度保持可控的一种方法是使用 *gradient scaling*，它在使用梯度估计更新策略参数化之前限制其幅度。梯度通常被限制为具有1的 L_2范数。另一种方法是 *gradient clipping*，它在使用梯度更新策略之前对梯度进行逐元素裁剪。裁剪通常将条目限制在 ± 1之间。这两种技术都在算法12.2中实现。

```
scale_gradient(∇, L2_max) = min(L2_max/norm(∇), 1)*∇
clip_gradient(∇, a, b) = clamp.(∇, a, b)
```

算法12.2. 梯度缩放与裁剪方法。梯度缩放将提供的梯度向量∇的大小限制为L2_max。梯度裁剪将提供的梯度向量∇逐元素限制在a和b之间。

缩放和裁剪在如何影响最终梯度方向方面有所不同，如图12.1所示。缩放不会改变方向，而裁剪则分别影响每个分量。这种差异是否有利取决于具体问题。例如，如果单个分量主导了梯度向量，缩放会将其他分量置零。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

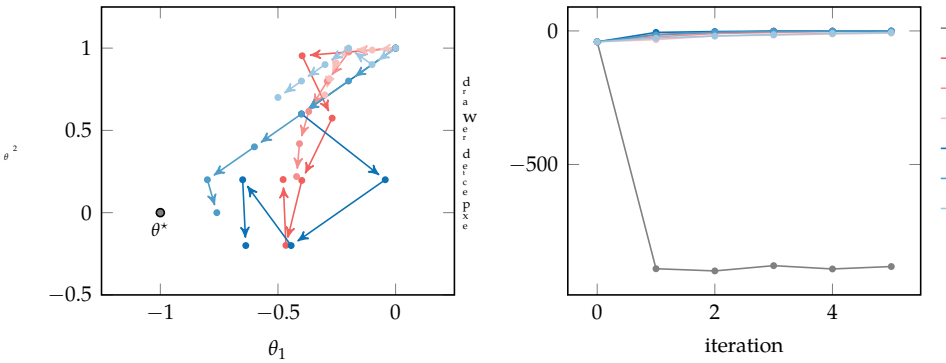


Figure 12.1. 梯度{v*}的影响
ent scaling an应用于
the simple reg模拟器问题。每个
gradient evaluation 运行了 10 次 rollout
to depth 10. Step 更新以 0.2 的步长
应用。最优策略参数化以黑色显示
。

12.2 *Restricted Gradient Update*

本章中其余的算法试图优化目标函数 $U(\theta)$ 的近似，但受到一个约束，即下一步的策略参数 θ' 不能与当前步的 θ 相差太远。该约束的形式为 $g(\theta, \theta') \leq \epsilon$，其中 $\epsilon > 0$ 是算法中的一个自由参数。这些方法在 $U(\theta)$ 的近似和 g 的形式上有所不同。本节介绍一种简单的 *restricted step* 方法。

我们使用从 θ 处的梯度估计得到的一阶泰勒近似（附录 A.12）来近似 U：

$$U(\theta') \approx U(\theta) + \nabla U(\theta)^\top (\theta' - \theta) \tag{12.2}$$

对于约束，我们使用

$$g(\theta, \theta') = \frac{1}{2}(\theta' - \theta)^\top \mathbf{I}(\theta' - \theta) = \frac{1}{2}\|\theta' - \theta\|_2^2 \tag{12.3}$$

我们可以将这个约束视为将步长限制为不超过 $\sqrt{2\epsilon}$。换句话说，我们优化中的可行域是以 θ 为中心、半径为 $\sqrt{2\epsilon}$ 的球。

那么，优化问题是，

$$\begin{aligned} \underset{\theta'}{\text{maximize}} \quad & U(\theta) + \nabla U(\theta)^\top (\theta' - \theta) \\ \text{subject to} \quad & \frac{1}{2}(\theta' - \theta)^\top \mathbf{I}(\theta' - \theta) \leq \epsilon \end{aligned} \tag{12.4}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们可以从目标函数中删除 $U(\theta)$，因为它不依赖于 θ'。此外，在约束条件中，我们可以将不等式改为等式，因为线性目标迫使最优解位于可行域的边界上。这些改变导致了一个等价优化问题：

$$\begin{aligned} \underset{\theta'}{\text{maximize}} \quad & \nabla U(\theta)^\top (\theta' - \theta) \\ \text{subject to} \quad & \frac{1}{2}(\theta' - \theta)^\top \mathbf{I}(\theta' - \theta) = \epsilon \end{aligned} \tag{12.5}$$

这个优化问题可以解析求解：

$$\theta' = \theta + \mathbf{u}\sqrt{\frac{2\epsilon}{\mathbf{u}^\top \mathbf{u}}} = \theta + \sqrt{2\epsilon}\frac{\mathbf{u}}{\|\mathbf{u}\|} \tag{12.6}$$

其中未归一化的搜索方向u就是$\nabla U(\theta)$。当然，我们并不确切知道$\nabla U(\theta)$，但可以使用前一章描述的任何方法来估计它。算法12.3提供了一种实现。

```
struct RestrictedPolicyUpdate
    𝒫     # problem
    b     # initial state distribution
    d     # depth
    m     # number of samples
    ∇logπ # gradient of log likelihood
    π     # policy
    ϵ     # divergence bound
end

function update(M::RestrictedPolicyUpdate, θ)
    𝒫, b, d, m, ∇logπ, π, γ = M.𝒫, M.b, M.d, M.m, M.∇logπ, M.π, M.𝒫.γ
    πθ(s) = π(θ, s)
    R(τ) = sum(r*γ^(k-1) for (k, (s,a,r)) in enumerate(τ))
    τs = [simulate(𝒫, rand(b), πθ, d) for i in 1:m]
    ∇log(τ) = sum(∇logπ(θ, a, s) for (s,a) in τ)
    ∇U(τ) = ∇log(τ)*R(τ)
    u = mean(∇U(τ) for τ in τs)
    return θ + u*sqrt(2*M.ϵ/dot(u,u))
end
```

算法12.3. 受限策略梯度方法在θ处的更新函数，针对初始状态分布为b的问题$\mathcal{P}$。梯度从初始状态分布b估计到深度d，通过m次参数化策略π(θ, s)的模拟，对数策略梯度为∇logπ。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

12.3 Natural Gradient Update

natural gradient 方法[2]是所讨论的限制步长方法的一种变体
在上一节中，为了更好地处理参数空间中某些组件比其他组件更敏感的情况。在此背景下，*Sensitivity*指的是策略的效用相对于参数中某一小变化的变化程度。梯度方法中的敏感度很大程度上取决于策略参数的缩放选择。自然策略梯度方法使搜索方向u对参数缩放不变。图12.2展示了真实梯度与自然梯度之间的差异。

2 S. Amari, “自然梯度在学习中高效工作”，*Neural Computation*，第10卷，第2期，第251–276页，1998年。

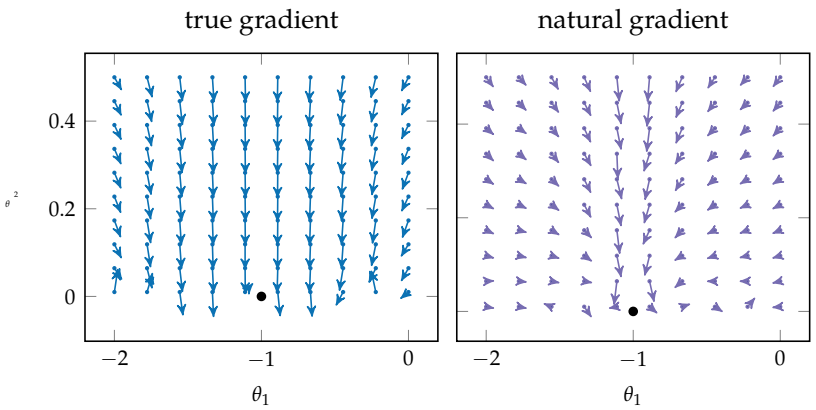


图12.2. 简单调节器问题（见附录F.5）上真实梯度与自然梯度的比较。真实梯度通常强烈指向负θ_2方向，而自然梯度通常指向位于$[-1, 0]$的最优点（黑点）。J. Peters和S. Schaal在“Reinforcement Learning of Motor Skills with Policy Gradients”一文中展示了类似的图，*Neural Networks*，第21卷，第4期，第682–697页，2008年。

自然策略梯度方法使用与上一节相同的一阶目标近似。然而，约束条件有所不同。其直觉是我们希望限制 θ 中的变化，这些变化会导致轨迹分布发生大的变化。衡量分布变化程度的一种方法是使用 *Kullback-Leibler divergence*，或 KL 散度（附录 A.10）。我们可以施加约束

$$g(\theta, \theta') = D_{\text{KL}}\left(p(\cdot \mid \theta) \,||\, p(\cdot \mid \theta')\right) \leq \epsilon \tag{12.7}$$

但相反，我们将采用二阶泰勒近似：

$$g(\theta, \theta') = \frac{1}{2}(\theta' - \theta)^\top \mathbf{F}_\theta (\theta' - \theta) \leq \epsilon \tag{12.8}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

其中 *Fisher information matrix* 具有以下形式：

$$\mathbf{F}_\theta = \int p(\tau \mid \theta) \nabla \log p(\tau \mid \theta) \nabla \log p(\tau \mid \theta)^\top \, \mathrm{d}\tau \tag{12.9}$$

$$= \mathbb{E}_\tau \left[\nabla \log p(\tau \mid \theta) \nabla \log p(\tau \mid \theta)^\top \right] \tag{12.10}$$

由此产生的优化问题是

$$\begin{aligned} \underset{\theta'}{\text{maximize}} \quad & \nabla U(\theta)^\top (\theta' - \theta) \\ \text{subject to} \quad & \frac{1}{2} (\theta' - \theta)^\top \mathbf{F}_\theta (\theta' - \theta) = \epsilon \end{aligned} \tag{12.11}$$

它看起来与方程(12.5)相同，只是这里的单位矩阵I被Fisher矩阵$\mathbf{F}_\theta$所替代。这一差异导致可行域呈现为椭球形状。图12.3展示了二维空间中的一个例子。

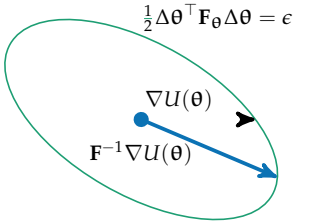


图12.3。自然策略梯度对近似的Kullback-Leibler散度施加了约束。这种约束的形式为椭圆。该椭圆在某些方向上可能被拉长，从而在梯度旋转时允许更大的步长。

这个优化问题可以解析求解，并且其形式与上一节中的更新相同：

$$\theta' = \theta + \mathbf{u} \sqrt{\frac{2\epsilon}{\nabla U(\theta)^\top \mathbf{u}}} \tag{12.12}$$

except that we now have[3]

$$\mathbf{u} = \mathbf{F}_\theta^{-1} \nabla U(\theta) \tag{12.13}$$

我们可以使用采样的轨迹来估计$\mathbf{F}_\theta$和$\nabla U(\theta)$。算法12.4提供了一个实现。

3 该计算可以采用共轭梯度下降法进行，当θ的维度较大时可减少计算量。S. M. Kakade, "A Natural Policy Gradient," 载于 *Advances in Neural Information Processing Systems (NIPS)*, 2001年。

12.4 Trust Region Update

本节讨论了一种在*trust region*内进行搜索的方法，该方法由前一节的椭圆可行域定义。这类方法被称为*trust region policy optimization* (*TRPO*)。[4]其工作原理是通过计算

下一个评估点θ'，该点将由自然策略梯度采取，然后沿着连接θ到θ'的线段进行*line search*。此线搜索阶段的一个关键特性是，对近似目标和约束的评估不需要任何额外的滚动模拟。

4 J. Schulman, S. Levine, P. Moritz, M. Jordan, and P. Abbeel, "信赖域策略优化"，载于 *International Conference on Machine Learning* (*ICML*)，2015年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct NaturalPolicyUpdate
    𝒫     # problem
    b     # initial state distribution
    d     # depth
    m     # number of samples
    ∇logπ # gradient of log likelihood
    π     # policy
    ϵ     # divergence bound
end

function natural_update(θ, ∇f, F, ϵ, τs)
    ∇fθ = mean(∇f(τ) for τ in τs)
    u = mean(F(τ) for τ in τs) \ ∇fθ
    return θ + u*sqrt(2ϵ/dot(∇fθ,u))
end

function update(M::NaturalPolicyUpdate, θ)
    𝒫, b, d, m, ∇logπ, π, γ = M.𝒫, M.b, M.d, M.m, M.∇logπ, M.π, M.𝒫.γ
    πθ(s) = π(θ, s)
    R(τ) = sum(r*γ^(k-1) for (k, (s,a,r)) in enumerate(τ))
    ∇log(τ) = sum(∇logπ(θ, a, s) for (s,a) in τ)
    ∇U(τ) = ∇log(τ)*R(τ)
    F(τ) = ∇log(τ)*∇log(τ)'
    τs = [simulate(𝒫, rand(b), πθ, d) for i in 1:m]
    return natural_update(θ, ∇U, F, M.ϵ, τs)
end
```

算法12.4。给定策略`π(θ, s)`，对于具有初始状态分布`b`的MDP $\mathcal{P}$，自然策略梯度的更新函数。关于参数向量`θ`的自然梯度是根据`m`次深度为`d`的rollout，使用对数策略梯度`∇logπ`估计的。`natural_update`辅助方法根据方程(12.12)，在给定目标梯度`∇f(τ)`和一组轨迹的Fisher矩阵`F(τ)`的情况下进行更新。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

在线搜索阶段，我们不再采用一阶近似，而是使用由优势函数[5]的等式推导出的近似。

$$U(\theta') = U(\theta) + \mathop{\mathbb{E}}_{\tau\sim\pi_{\theta'}}\left[\sum_{k=1}^{d} A_\theta(s^{(k)}, a^{(k)})\right] \tag{12.14}$$

另一种写法是使用 $b_{\gamma,\theta}$，它是策略 π_θ 下状态 s 的 *discounted visitation distribution*，其中

$$b_{\gamma,\theta}(s) \propto P(s^{(1)} = s) + \gamma P(s^{(2)} = s) + \gamma^2 P(s^{(3)} = s) + \cdots \tag{12.15}$$

使用折扣访问分布，目标变为

$$U(\theta') = U(\theta) + \mathop{\mathbb{E}}_{s\sim b_{\gamma,\theta'}}\left[\mathop{\mathbb{E}}_{a\sim\pi_{\theta'}(\cdot|s)}[A_\theta(s,a)]\right] \tag{12.16}$$

我们希望从由 θ 参数化的策略中抽取样本，而不是从 θ' 中抽取，这样我们就不必在线搜索期间运行更多的模拟。只要适当地加权优势，内层期望相关的样本就可以用我们原始策略的样本替换：[6]

$$U(\theta') = U(\theta) + \mathop{\mathbb{E}}_{s\sim b_{\gamma,\theta'}}\left[\mathop{\mathbb{E}}_{a\sim\pi_\theta(\cdot|s)}\left[\frac{\pi_{\theta'}(a \mid s)}{\pi_\theta(a \mid s)} A_\theta(s,a)\right]\right] \tag{12.17}$$

下一步涉及将状态分布替换为 $b_{\gamma,\theta}$。随着 θ' 离 θ 越来越远，近似的质量会下降，但假设在信赖域内是可以接受的。由于 $U(\theta)$ 不依赖于 θ'，我们可以将其从目标函数中剔除。我们还可以从优势函数中剔除状态价值函数，只留下动作价值函数。剩下的部分被称为 *surrogate objective*：

$$f(\theta, \theta') = \mathop{\mathbb{E}}_{s\sim b_{\gamma,\theta}}\left[\mathop{\mathbb{E}}_{a\sim\pi_\theta(\cdot|s)}\left[\frac{\pi_{\theta'}(a \mid s)}{\pi_\theta(a \mid s)} Q_\theta(s,a)\right]\right] \tag{12.18}$$

这个方程可以从用于估计自然梯度更新的同一组轨迹中进行估计。我们可以利用采样轨迹中的剩余回报来估计$Q_\theta(s、a)$。[7]

线搜索中的 *surrogate constraint* 由下式给出

$$g(\theta, \theta') = \mathop{\mathbb{E}}_{s\sim b_{\gamma,\theta}}[D_{\text{KL}}(\pi_\theta(\cdot \mid s) \,||\, \pi_{\theta'}(\cdot \mid s))] \leq \epsilon \tag{12.19}$$

5 这种等式的一个变体在S. M. Kakade和J. Langford的“近似最优近似强化学习”一文的引理6.1中得到了证明，该文发表于*International Conference on Machine Learning* (*ICML*)，2002

6 这一权重来源于*importance sampling*，该内容在附录 A.14 中进行了审查。

7 算法12.5改为使用$\sum_{\ell=k} r^{(\ell)}\gamma^{\ell-1}$，它有效地将剩余奖励折现为$\gamma^{k-1}$。这个折扣是为了使每个样本的贡献与折现后的访问分布相匹配。代理约束也同样被折现。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

线搜索涉及迭代地评估我们的代理目标 f 和代理约束 g，针对策略空间中的不同点。我们从与自然梯度更新相同过程获得的 θ' 开始。然后我们迭代应用

$$\theta' \leftarrow \theta + \alpha(\theta' - \theta) \tag{12.20}$$

直到我们的目标函数在$f(\theta、\theta') > f(\theta、\theta)$上有所改善，并且我们的约束在$g(\theta、\theta') \leq \epsilon$上得到满足。步长因子$0 < \alpha < 1$ 在每次迭代中缩小θ与θ'之间的距离，其中α通常设置为0.5。

算法12.5提供了这种方法的实现。图12.4展示了与自然梯度和线搜索相关的可行区域之间的关系。图12.5在调节器问题上演示了该方法，而例12.1展示了一个简单问题的更新过程。

12.5 Clamped Surrogate Objective

我们可以通过 *clamping*.[8] 避免由于对信任域替代目标过于乐观的估计而导致的有害策略更新。替代目标来自
方程（12.18），在交换动作价值优势后，为

$$\mathop{\mathbb{E}}_{s\sim b_{\gamma,\theta}}\left[\mathop{\mathbb{E}}_{a\sim\pi_\theta(\cdot|s)}\left[\frac{\pi_{\theta'}(a \mid s)}{\pi_\theta(a \mid s)} A_\theta(s,a)\right]\right] \tag{12.21}$$

概率比 {v*} 可能过于乐观。对目标函数采用悲观下界可以显著提高性能：

$$\mathop{\mathbb{E}}_{s\sim b_{\gamma,\theta}}\left[\mathop{\mathbb{E}}_{a\sim\pi_\theta(\cdot|s)}\left[\min\left(\frac{\pi_{\theta'}(a \mid s)}{\pi_\theta(a \mid s)} A_\theta(s,a), \text{clamp}\left(\frac{\pi_{\theta'}(a \mid s)}{\pi_\theta(a \mid s)}, 1-\epsilon, 1+\epsilon\right) A_\theta(s,a)\right)\right]\right] \tag{12.22}$$

其中 ϵ 是一个小的正值[9]，而 clamp(x, a, b) 强制 x 位于 a 和 b. 根据定义，clamp(x, a, b) = $\min\{\max\{x, a\}, b\}$。

仅对概率比进行截断并不能产生下界；我们还必须取截断目标与原始目标的最小值。下界如图12.6所示，连同原始目标和截断目标一起展示。下界的最终结果是，当概率比的变化会导致目标显著改善时，该变化被忽略。因此，使用下界可以防止在这些情况下出现大幅的、往往有害的更新，并消除了对信赖域替代约束方程（12.19）的需要。没有

8 裁剪是所谓 *proximal policy optimization (PPO)* 中的一个关键思想，正如 J. Schulman、F. Wolski、P. Dhariwal、A. Radford 和 O. Klimov 在“Proximal Policy Optimization Algorithms”（2017）中所讨论的。arXiv: 1707.06347v2。

9 虽然这个ϵ并不像之前的算法那样直接充当散度的阈值，但其作用类似。典型值为0.2。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct TrustRegionUpdate
    𝒫     # problem
    b     # initial state distribution
    d     # depth
    m     # number of samples
    π     # policy π(s)
    p     # policy likelihood p(θ, a, s)
    ∇logπ # log likelihood gradient
    KL    # KL divergence KL(θ, θ′, s)
    ϵ     # divergence bound
    α     # line search reduction factor (e.g., 0.5)
end

function surrogate_objective(M::TrustRegionUpdate, θ, θ′, τs)
    d, p, γ = M.d, M.p, M.𝒫.γ
    R(τ, j) = sum(r*γ^(k-1) for (k,(s,a,r)) in zip(j:d, τ[j:end]))
    w(a,s) = p(θ′,a,s) / p(θ,a,s)
    f(τ) = mean(w(a,s)*R(τ,k) for (k,(s,a,r)) in enumerate(τ))
    return mean(f(τ) for τ in τs)
end

function surrogate_constraint(M::TrustRegionUpdate, θ, θ′, τs)
    γ = M.𝒫.γ
    KL(τ) = mean(M.KL(θ, θ′, s)*γ^(k-1) for (k,(s,a,r)) in enumerate(τ))
    return mean(KL(τ) for τ in τs)
end

function linesearch(M::TrustRegionUpdate, f, g, θ, θ′)
    fθ = f(θ)
    while g(θ′) > M.ϵ || f(θ′) ≤ fθ
        θ′ = θ + M.α*(θ′ - θ)
    end
    return θ′
end

function update(M::TrustRegionUpdate, θ)
    𝒫, b, d, m, ∇logπ, π, γ = M.𝒫, M.b, M.d, M.m, M.∇logπ, M.π, M.𝒫.γ
    πθ(s) = π(θ, s)
    R(τ) = sum(r*γ^(k-1) for (k, (s,a,r)) in enumerate(τ))
    ∇log(τ) = sum(∇logπ(θ, a, s) for (s,a) in τ)
    ∇U(τ) = ∇log(τ)*R(τ)
    F(τ) = ∇log(τ)*∇log(τ)'
    τs = [simulate(𝒫, rand(b), πθ, d) for i in 1:m]
    θ′ = natural_update(θ, ∇U, F, M.ϵ, τs)
    f(θ′) = surrogate_objective(M, θ, θ′, τs)
    g(θ′) = surrogate_constraint(M, θ, θ′, τs)
    return linesearch(M, f, g, θ, θ′)
end
```

算法12.5。信赖域策略优化的更新过程，该过程通过线搜索增强自然梯度。它在问题 `𝒫` 中使用策略 `π` 生成 `m` 条轨迹，其初始状态分布为 `b`，深度为 `d`。为了获得线搜索的起点，我们需要策略从当前状态生成特定动作的对数概率梯度，我们将其记为 `∇logπ`。对于替代目标，我们需要概率函数 `p`，它给出我们的策略从当前状态生成特定动作的概率。对于替代约束，我们需要由 π_θ 和 $\pi_{\theta'}$ 生成的动作分布之间的散度。在线搜索的每一步，我们在保持搜索方向的同时缩小所考虑点 `θ′` 和 `θ` 之间的距离。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

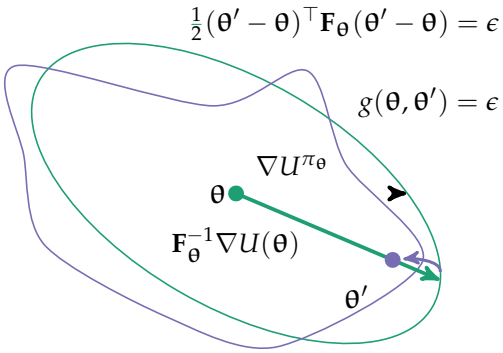


图12.4。信任区域策略优化在由Kullback-Leibler散度的二阶近似生成的椭圆形约束内进行搜索。在计算出自然策略梯度上升方向后，进行线搜索以确保更新后的策略提高策略奖励并遵守散度约束。线搜索从估计的最大步长开始，并沿上升方向减小步长，直到找到满意的点。

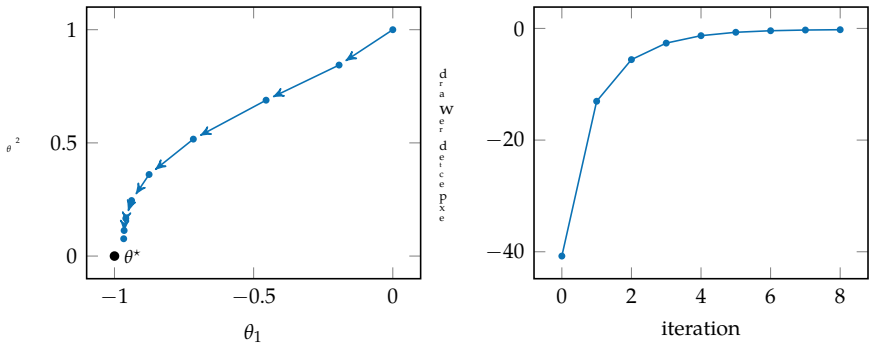


图12.5. 应用于简单调节器问题的信任域策略优化，采用深度为10的轨迹展开，其中ϵ =为1，c =为2。最优策略参数化以黑色显示。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

示例12.1。信赖域策略优化的一次迭代示例。

考虑将TRPO应用于示例11.3中的高斯策略 $\mathcal{N}(\theta_1、\theta_2^2)$，以解决示例11.1中的单状态MDP，其中 $\gamma =$ 为1。回想一下，对数策略似然的梯度是

$$\frac{\partial}{\partial\theta_1}\log\pi_\theta(a \mid s) = \frac{a-\theta_1}{\theta_2^2}$$
$$\frac{\partial}{\partial\theta_2}\log\pi_\theta(a \mid s) = \frac{(a-\theta_1)^2-\theta_2^2}{\theta_2^3}$$

假设我们用 $\theta = [0, 1]$ 运行两次 rollout（这个问题只有一个状态）：

$$\tau_1 = \{(a = r = -0.532), (a = r = \quad 0.597), (a = r = 1.947)\}$$
$$\tau_2 = \{(a = r = -0.263), (a = r = -2.212), (a = r = 2.364)\}$$

估计的Fisher信息矩阵是

$$\begin{aligned}\mathbf{F}_\theta &= \frac{1}{2}\left(\nabla\log p(\tau^{(1)})\nabla\log p(\tau^{(1)})^\top + \nabla\log p(\tau^{(2)})\nabla\log p(\tau^{(2)})^\top\right)\\ &= \frac{1}{2}\left(\begin{bmatrix}4.048 & 2.878\\ 2.878 & 2.046\end{bmatrix} + \begin{bmatrix}0.012 & -0.838\\ -0.838 & 57.012\end{bmatrix}\right) = \begin{bmatrix}2.030 & 1.020\\ 1.019 & 29.529\end{bmatrix}\end{aligned}$$

目标函数梯度为 [2.030, 1.020]。得到的下降方向 u 是 [1, 0]。设定 $\epsilon =$ 0.1，我们计算更新后的参数向量，并得到 $\theta' = [0.314, 1]$。

替代目标函数在θ处的值为1.485。线搜索从θ′开始，此时替代目标函数值为2.110，约束条件产生的结果为0.049。这满足我们的约束条件（因为0.049 $< \epsilon$），因此我们返回新的参数化。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

在约束条件下，我们还可以省去线搜索，直接使用标准的梯度上升方法。

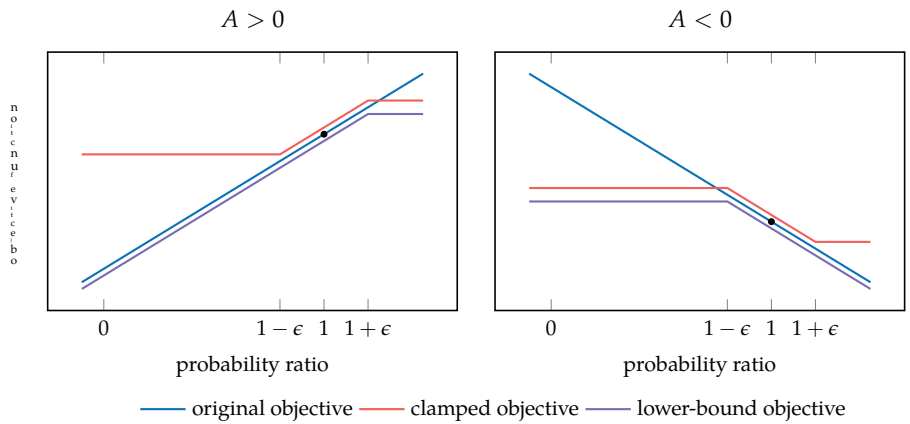


图12.6. 正值和负值优势下下界目标与原始目标及截断目标的对比可视化。黑点表示执行优化的基准点，$\pi_{\theta'}(a \mid s)/\pi_\theta(a \mid s) = 1$。为清晰起见，每个轴上的三条线图垂直分隔。

未截断目标方程（12.21）关于动作值的梯度为

$$\nabla_{\theta'} f(\theta, \theta') = \mathop{\mathbb{E}}_{s\sim b_{\gamma,\theta}}\left[\mathop{\mathbb{E}}_{a\sim\pi_\theta(\cdot|s)}\left[\frac{\nabla_{\theta'}\pi_{\theta'}(a \mid s)}{\pi_\theta(a \mid s)} Q_\theta(s,a)\right]\right] \tag{12.23}$$

其中$Q_\theta(s、a)$可以从reward-to-go估计得到。下界目标方程(12.22)的梯度（带截断）是相同的，只是对于目标被主动截断的经验元组没有贡献。也就是说，如果reward-to-go为正且概率比大于$1+\epsilon$，或者reward-to-go为负且概率比小于$1-\epsilon$，则梯度贡献为零。

与TRPO类似，梯度可以针对由 θ 生成的经验中的参数化 θ' 进行计算。因此，可以使用同一组采样轨迹连续运行多次梯度更新。算法12.6提供了这一实现。

将截断代理目标与图12.7中的其他几个代理目标进行了比较，其中包括TRPO的有效目标折线图：

$$\mathop{\mathbb{E}}_{\substack{s\sim b_{\gamma,\theta}\\ a\sim\pi_\theta(\cdot|s)}}\left[\frac{\pi_{\theta'}(a \mid s)}{\pi_\theta(a \mid s)} A_\theta(s,a) - \beta D_{\text{KL}}(\pi_\theta(\cdot \mid s) \,||\, \pi_{\theta'}(\cdot \mid s))\right] \tag{12.24}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct ClampedSurrogateUpdate
    𝒫     # problem
    b     # initial state distribution
    d     # depth
    m     # number of trajectories
    π     # policy
    p     # policy likelihood
    ∇π    # policy likelihood gradient
    ϵ     # divergence bound
    α     # step size
    k_max # number of iterations per update
end

function clamped_gradient(M::ClampedSurrogateUpdate, θ, θ′, τs)
    d, p, ∇π, ϵ, γ = M.d, M.p, M.∇π, M.ϵ, M.𝒫.γ
    R(τ, j) = sum(r*γ^(k-1) for (k,(s,a,r)) in zip(j:d, τ[j:end]))
    ∇f(a,s,r_togo) = begin
        P = p(θ, a,s)
        w = p(θ′,a,s) / P
        if (r_togo > 0 && w > 1+ϵ) || (r_togo < 0 && w < 1-ϵ)
            return zeros(length(θ))
        end
        return ∇π(θ′, a, s) * r_togo / P
    end
    ∇f(τ) = mean(∇f(a,s,R(τ,k)) for (k,(s,a,r)) in enumerate(τ))
    return mean(∇f(τ) for τ in τs)
end

function update(M::ClampedSurrogateUpdate, θ)
    𝒫, b, d, m, π, α, k_max= M.𝒫, M.b, M.d, M.m, M.π, M.α, M.k_max
    πθ(s) = π(θ, s)
    τs = [simulate(𝒫, rand(b), πθ, d) for i in 1:m]
    θ′ = copy(θ)
    for k in 1:k_max
        θ′ += α*clamped_gradient(M, θ, θ′, τs)
    end
    return θ′
end
```

算法12.6。一种受限替代策略优化的实现，它针对初始状态分布为`b`的MDP $\mathcal{P}$的策略`π(s)`，返回一个新的策略参数化。该实现采样`m`条轨迹至深度`d`，然后使用它们估计后续`k_max`次更新中的策略梯度。使用受限目标的策略梯度是使用策略梯度`∇p`和限幅参数`ϵ`构建的。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

这是信任区域策略目标，其中约束作为某种系数 β 的惩罚来实现。TRPO 通常使用硬约束而不是惩罚，因为在单个问题内选择一个表现良好的 β 值已经很困难，更不用说在多个问题之间了。

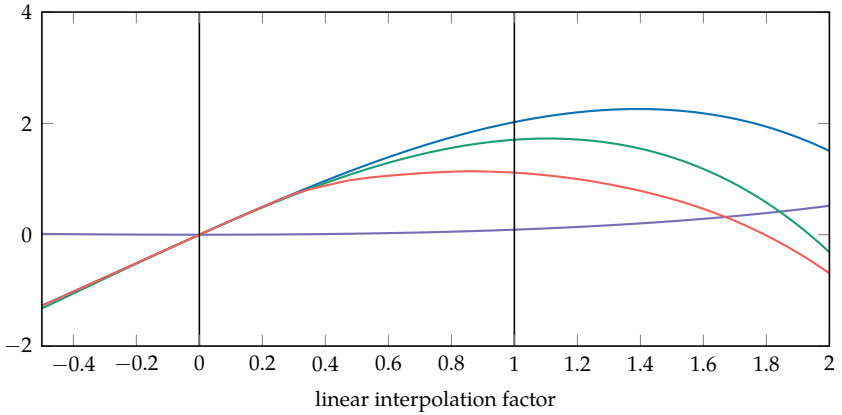


图12.7. 在使用线性二次调节器问题时，与钳制代理策略优化相关的代理目标比较。x轴展示了当我们在1处进行自然策略更新时，从θ在0处向θ'移动时的代理目标。通过减去θ的代理目标函数值，代理目标被中心化至0。我们看到，钳制代理目标的行为与无需约束的有效TRPO目标非常相似。注意，对于两种算法，ϵ和β均可调整，这将影响每种情况下最大值的位置。

12.6 Summary

- 梯度上升算法可以利用从上一章讨论的方法中获得的梯度估计来迭代地改进我们的策略。
- 通过缩放、裁剪或强制使改进步长的大小一致，梯度上升可以变得更加稳健。
- 自然梯度方法使用目标函数的一阶近似，并对每一步轨迹分布之间的散度施加约束，该散度通过Fisher信息矩阵的估计来近似。
- 信赖域策略优化通过在自然梯度法中加入线搜索，进一步改进策略，而无需额外的轨迹模拟。
- 我们可以使用TRPO目标的悲观下界来获得一个限幅替代目标，其表现相似但无需进行线搜索。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

12.7 Exercises

练习 12.1。TRPO 从一个由自然策略梯度更新给出的新参数化开始其线搜索。然而，TRPO 使用与自然策略梯度不同的目标函数进行线搜索。证明 TRPO 中使用的替代目标方程（12.18）的梯度实际上与奖励到目标的策略梯度方程（11.26）相同。

Solution: TRPO 替代目标的梯度是

$$\nabla_{\theta'} U_{\text{TRPO}} = \mathop{\mathbb{E}}_{s\sim b_{\gamma,\theta}}\left[\mathop{\mathbb{E}}_{a\sim\pi_\theta(\cdot|s)}\left[\frac{\nabla_{\theta'}\pi_{\theta'}(a \mid s)}{\pi_\theta(a \mid s)} Q_\theta(s,a)\right]\right]$$

在进行初始自然策略梯度更新时，搜索方向在 $\theta' = \theta$ 处进行评估。此外，动作价值通过奖励到当前步骤的累计值来近似：

$$\nabla_{\theta'} U_{\text{TRPO}} = \mathop{\mathbb{E}}_{s\sim b_{\gamma,\theta}}\left[\mathop{\mathbb{E}}_{a\sim\pi_\theta(\cdot|s)}\left[\frac{\nabla_{\theta}\pi_{\theta}(a \mid s)}{\pi_\theta(a \mid s)} r_{\text{to-go}}\right]\right]$$

R回想一下，$\log f(x)$ 的导数是 $f'(x)/f(x)$。因此 ws that

$$\nabla_{\theta'} U_{\text{TRPO}} = \mathop{\mathbb{E}}_{s\sim b_{\gamma,\theta}}\left[\mathop{\mathbb{E}}_{a\sim\pi_\theta(\cdot|s)}\left[\nabla_\theta \log \pi_\theta(a \mid s) r_{\text{to-go}}\right]\right]$$

其形式与奖励至目标策略梯度方程（11.26）相同。

练习12.2。执行例12.1中的计算。首先，计算费舍尔信息矩阵$\mathbf{F}_\theta^{-1}$的逆，计算u，并计算更新后的参数θ'。

Solution: 我们首先计算Fisher信息矩阵的逆：

$$\mathbf{F}_\theta^{-1} \approx \frac{1}{0.341(29.529) - 0.332(0.332)}\begin{bmatrix} 29.529 & -0.332 \\ -0.332 & 0.341 \end{bmatrix} \approx \begin{bmatrix} 0.501 & -0.017 \\ -0.017 & 0.034 \end{bmatrix}$$

现在，我们按如下方式更新 u：

$$\mathbf{u} = \mathbf{F}_\theta^{-1}\nabla U(\theta) \approx \begin{bmatrix} 0.501 & -0.017 \\ -0.017 & 0.034 \end{bmatrix}\begin{bmatrix} 2.030 \\ 1.020 \end{bmatrix} \approx \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

最后，我们估计更新后的参数 θ：

$$\begin{aligned}\theta' &= \theta + \mathbf{u}\sqrt{\frac{2\epsilon}{\nabla U(\theta)^\top \mathbf{u}}} \\ &\approx \begin{bmatrix}0\\1\end{bmatrix} + \begin{bmatrix}1\\0\end{bmatrix}\sqrt{\frac{2(0.1)}{\begin{bmatrix}2.030 & 1.020\end{bmatrix}\begin{bmatrix}1\\0\end{bmatrix}}} \\ &\approx \begin{bmatrix}0\\1\end{bmatrix} + \begin{bmatrix}1\\0\end{bmatrix}\sqrt{\frac{0.2}{2.030}} \\ &\approx \begin{bmatrix}0.314\\1\end{bmatrix}\end{aligned}$$

练习12.3。假设我们有以下表中所给的参数化策略π_θ和$\pi_{\theta'}$：

	a_1	a_2	a_3	a_4
$\pi_\theta(a \mid s_1)$	0.1	0.2	0.3	0.4
$\pi_{\theta'}(a \mid s_1)$	0.4	0.3	0.2	0.1
$\pi_\theta(a \mid s_2)$	0.1	0.1	0.6	0.2
$\pi_{\theta'}(a \mid s_2)$	0.1	0.1	0.5	0.3

给定我们对以下五个状态进行采样，s_1、s_2、s_1、s_1、s_2，使用定义来近似 $\mathbb{E}_s$ [$D_{\text{KL}}(\pi_\theta(\cdot \mid s) \mid\mid \pi_{\theta'}(\cdot \mid s))$]

$$D_{\text{KL}}(P \mid\mid Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$$

Solution: 首先，我们计算状态样本s_1的KL散度：

$$D_{\text{KL}}(\pi_\theta(\cdot \mid s_1) \mid\mid \pi_{\theta'}(\cdot \mid s_1)) = 0.1 \log\left(\tfrac{0.1}{0.4}\right) + 0.2 \log\left(\tfrac{0.2}{0.3}\right) + 0.3 \log\left(\tfrac{0.3}{0.3}\right) + 0.4 \log\left(\tfrac{0.4}{0.1}\right) \approx 0.456$$

现在，我们计算一个状态样本 s_2 的 KL 散度：

$$D_{\text{KL}}(\pi_\theta(\cdot \mid s_2) \mid\mid \pi_{\theta'}(\cdot \mid s_2)) = 0.1 \log\left(\tfrac{0.1}{0.1}\right) + 0.1 \log\left(\tfrac{0.1}{0.1}\right) + 0.6 \log\left(\tfrac{0.6}{0.5}\right) + 0.2 \log\left(\tfrac{0.2}{0.3}\right) \approx 0.0283$$

最后，我们计算期望的近似值，即参数化策略在n个状态样本上的平均KL散度：

$$\begin{aligned}\mathbb{E}_s[D_{\text{KL}}(\pi_\theta(\cdot \mid s) \mid\mid \pi_{\theta'}(\cdot \mid s))] &\approx \frac{1}{n}\sum_{i=1}^{n} D_{\text{KL}}\left(\pi_\theta(\cdot \mid s^{(i)}) \,\middle\|\, \pi_{\theta'}(\cdot \mid s^{(i)})\right) \\ &\approx \frac{1}{5}(0.456 + 0.0283 + 0.456 + 0.456 + 0.0283) \\ &\approx 0.285\end{aligned}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

13 Actor-Critic Methods

上一章讨论了通过从回合估计的梯度信息来改进参数化策略的方法。本章介绍*actor-critic methods*，它们使用价值函数的估计来帮助指导优化。在这种情况下，演员是策略，评论家是价值函数。两者并行训练。我们将讨论几种方法，它们的区别在于它们近似价值函数、优势函数还是动作价值函数。大多数关注随机策略，但我们也将讨论一种支持输出连续动作的确定性策略的方法。最后，我们将讨论一种方法，用于结合在线方法来生成更具信息性的轨迹，以训练演员和评论家。

13.1 Actor-Critic

在actor-critic方法中，我们有一个由策略π_θ表示的actor，由θ参数化，借助一个critic提供价值函数$U_\phi(s)$、$Q_\phi(s、a)$或$A_\phi(s、a)$的估计，参数化为ϕ。我们将从一种简单的actor-critic方法开始本章，其中π_θ的优化通过梯度上升完成，目标函数的梯度与方程(11.44)中相同：

$$\nabla U(\theta) = \mathbb{E}_\tau\left[\sum_{k=1}^{d} \nabla_\theta \log \pi_\theta(a^{(k)} \mid s^{(k)})\gamma^{k-1} A_\theta\left(s^{(k)}, a^{(k)}\right)\right] \tag{13.1}$$

当遵循由 θ 参数化的策略时，其优势可以通过一组从 s 到 s' 的观察到的转移及其奖励 r 来估计：

$$A_\theta(s,a) = \mathbb{E}_{r,s'}\left[r + \gamma U^{\pi_\theta}(s') - U^{\pi_\theta}(s)\right] \tag{13.2}$$

期望中的 $r + \gamma U^{\pi_\theta}(s') - U^{\pi_\theta}(s)$ 被称为 *temporal difference residual*。

评论家允许我们在遵循 π_θ 时估计真实价值函数 U^{π_θ}，从而得到演员的以下梯度：

$$\nabla U(\theta) \approx \mathbb{E}_\tau\left[\sum_{k=1}^{d} \nabla_\theta \log \pi_\theta(a^{(k)} \mid s^{(k)}) \gamma^{k-1}\left(r^{(k)} + \gamma U_\phi(s^{(k+1)}) - U_\phi(s^{(k)})\right)\right] \tag{13.3}$$

这种期望可以通过展开轨迹来估计，如第11章所述。

评论家也通过梯度优化进行更新。我们希望找到一个使损失函数最小化的ϕ：

$$\ell(\phi) = \frac{1}{2}\mathbb{E}_s\left[\left(U_\phi(s) - U^{\pi_\theta}(s)\right)^2\right] \tag{13.4}$$

为了最小化该目标，我们可以沿着梯度的反方向采取步骤：

$$\nabla \ell(\phi) = \mathbb{E}_s\left[\left(U_\phi(s) - U^{\pi_\theta}(s)\right)\nabla_\phi U_\phi(s)\right] \tag{13.5}$$

当然，我们并不确切知道U^{π_θ}，但可以通过沿轨迹奖励来估计，从而得出

$$\nabla \ell(\phi) = \mathbb{E}_\tau\left[\sum_{k=1}^{d}\left(U_\phi(s^{(k)}) - r_{\text{to-go}}^{(k)}\right)\nabla_\phi U_\phi(s^{(k)})\right] \tag{13.6}$$

其中 $r_{\text{to-go}}^{(k)}$ 是特定轨迹 τ 中步骤 k 的奖励-to-go。

算法13.1展示了如何从回滚中估计$\nabla U(\theta)$和$\nabla \ell(\phi)$。在每次迭代中，我们沿着$\nabla U(\theta)$的方向调整θ以最大化效用，并沿着$\nabla \ell(\phi)$的反方向调整ϕ以最小化损失。由于θ和ϕ的估计之间存在依赖关系，这种方法可能会变得不稳定，但它在多种问题上表现良好。为了提高稳定性，常见的做法是比价值函数更频繁地更新策略。本章的实现可以轻松地调整为仅在策略更新的部分迭代中更新价值函数。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct ActorCritic
    𝒫     # problem
    b     # initial state distribution
    d     # depth
    m     # number of samples
    ∇logπ # gradient of log likelihood ∇logπ(θ,a,s)
    U     # parameterized value function U(ϕ, s)
    ∇U    # gradient of value function ∇U(ϕ,s)
end

function gradient(M::ActorCritic, π, θ, ϕ)
    𝒫, b, d, m, ∇logπ = M.𝒫, M.b, M.d, M.m, M.∇logπ
    U, ∇U, γ = M.U, M.∇U, M.𝒫.γ
    πθ(s) = π(θ, s)
    R(τ,j) = sum(r*γ^(k-1) for (k,(s,a,r)) in enumerate(τ[j:end]))
    A(τ,j) = τ[j][3] + γ*U(ϕ,τ[j+1][1]) - U(ϕ,τ[j][1])
    ∇Uθ(τ) = sum(∇logπ(θ,a,s)*A(τ,j)*γ^(j-1) for (j, (s,a,r))
                 in enumerate(τ[1:end-1]))
    ∇ℓϕ(τ) = sum((U(ϕ,s) - R(τ,j))*∇U(ϕ,s) for (j, (s,a,r))
                 in enumerate(τ))
    trajs = [simulate(𝒫, rand(b), πθ, d) for i in 1:m]
    return mean(∇Uθ(τ) for τ in trajs), mean(∇ℓϕ(τ) for τ in trajs)
end
```

算法13.1. 一种基本的演员-评论家方法，用于计算MDP $\mathcal{P}$的策略梯度和值函数梯度，初始状态分布为`b`。策略π由θ参数化，并具有对数梯度`∇logπ`。值函数`U`由ϕ参数化，其目标函数的梯度为`∇U`。此方法运行`m`次回滚至深度`d`。结果用于更新θ和ϕ。策略参数化沿`∇θ`方向更新以最大化期望值，而值函数参数化则沿`∇ϕ`的负方向更新以最小化值损失。

13.2 *Generalized Advantage Estimation*

Generalized advantage estimation（算法13.2）是一种演员-评论家方法，它使用了公式(13.2)中所示优势估计的一个更通用的版本，使我们能够在偏差和方差之间进行平衡。[1]近似通过

时序差分残差具有低方差，但由于使用可能不准确的 U_ϕ 来近似 U^{π_θ}，它引入了偏差。另一种方法是将 $r + \gamma U^{\pi_\theta}(s')$ 替换为一系列展开奖励 $r_1, \ldots, r_d$：

1 J. Schulman, P. Moritz, S. Levine, M. Jordan, 和 P. Abbeel, “使用广义优势估计的高维连续控制”，载于 *International Conference on Learning Representations (ICLR)*，2016年。arXiv: 1506.02438v6。

$$A_\theta(s,a) = \mathbb{E}_{r_1,\ldots,r_d}\left[r_1 + \gamma r_2 + \gamma^2 r_3 + \cdots + \gamma^{d-1} r_d - U^{\pi_\theta}(s)\right] \tag{13.7}$$

$$= \mathbb{E}_{r_1,\ldots,r_d}\left[-U^{\pi_\theta}(s) + \sum_{\ell=1}^{d} \gamma^{\ell-1} r_\ell\right] \tag{13.8}$$

我们可以通过展开轨迹获得该期望的无偏估计，正如策略梯度估计方法（第11章）中所做的那样。然而，这种估计方差较大，意味着我们需要大量样本才能得到准确的估计。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

广义优势估计所采取的方法是在使用时间差分残差和完整展开这两种极端之间取得平衡。我们将$\hat{A}^{(k)}$定义为从k步展开中获得的优势估计，以及与所得状态s'相关的效用：

$$\hat{A}^{(k)}(s,a) = \mathbb{E}_{r_1,\ldots,r_k,s'}\left[r_1 + \gamma r_2 + \cdots + \gamma^{k-1} r_k + \gamma^k U^{\pi_\theta}(s') - U^{\pi_\theta}(s)\right] \tag{13.9}$$

$$= \mathbb{E}_{r_1,\ldots,r_k,s'}\left[-U^{\pi_\theta}(s) + \gamma^k U^{\pi_\theta}(s') + \sum_{\ell=1}^{k} \gamma^{\ell-1} r_\ell\right] \tag{13.10}$$

另一种写法$\hat{A}^{(k)}$是通过时间差分残差的期望来表示。我们可以定义

$$\delta_t = r_t + \gamma U(s_{t+1}) - U(s_t) \tag{13.11}$$

其中s_t、r_t和s_{t+1}分别是沿采样轨迹的状态、奖励和后续状态，而U是我们的价值函数估计。然后，

$$\hat{A}^{(k)}(s,a) = \mathbb{E}\left[\sum_{\ell=1}^{k} \gamma^{\ell-1} \delta_\ell\right] \tag{13.12}$$

广义优势估计不是为k确定一个特定值，而是引入一个参数$\lambda \in [0,1]$，它提供了一个*exponentially weighted average*的$\hat{A}^{(k)}$，用于k从1到d：[2]

2 序列$x_1, x_2, \ldots$的指数加权平均为$(1-\lambda)(x_1 + \lambda x_2 + \lambda^2 x_3 + \cdots)$。

$$\hat{A}^{\text{GAE}}(s,a)\,|_{d=1} = \hat{A}^{(1)} \tag{13.13}$$

$$\hat{A}^{\text{GAE}}(s,a)\,|_{d=2} = (1-\lambda)\hat{A}^{(1)} + \lambda \hat{A}^{(2)} \tag{13.14}$$

$$\hat{A}^{\text{GAE}}(s,a)\,|_{d=3} = (1-\lambda)\hat{A}^{(1)} + \lambda\left((1-\lambda)\hat{A}^{(2)} + \lambda \hat{A}^{(3)}\right) \tag{13.15}$$

$$= (1-\lambda)\hat{A}^{(1)} + \lambda(1-\lambda)\hat{A}^{(2)} + \lambda^2 \hat{A}^{(3)} \tag{13.16}$$

您是一位专业、真实的机器翻译引擎。仅输出翻译后的文本，不包含任何其他文本。将以下 M

$$\hat{A}^{\text{GAE}}(s,a) = (1-\lambda)\left(\hat{A}^{(1)} + \lambda \hat{A}^{(2)} + \lambda^2 \hat{A}^{(3)} + \cdots + \lambda^{d-2} \hat{A}^{(d-1)}\right) + \lambda^{d-1} \hat{A}^{(d)} \tag{13.17}$$

对于无限视界，广义优势估计简化为

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

$$\hat{A}^{\text{GAE}}(s,a) = (1-\lambda)\left(\hat{A}^{(1)} + \lambda\hat{A}^{(2)} + \lambda^2\hat{A}^{(3)} + \cdots\right) \tag{13.18}$$

$$= (1-\lambda)\left(\delta_1\left(1+\lambda+\lambda^2+\cdots\right) + \gamma\delta_2\left(\lambda+\lambda^2+\cdots\right) + \gamma^2\delta_3\left(\lambda^2+\cdots\right) + \cdots\right) \tag{13.19}$$

$$= (1-\lambda)\left(\delta_1\frac{1}{1-\lambda} + \gamma\delta_2\frac{\lambda}{1-\lambda} + \gamma^2\delta_3\frac{\lambda^2}{1-\lambda} + \cdots\right) \tag{13.20}$$

$$= \mathbb{E}\left[\sum_{k=1}^{\infty}(\gamma\lambda)^{k-1}\delta_k\right] \tag{13.21}$$

我们可以调整参数 λ 来平衡偏差和方差。如果 $\lambda = 0$，那么我们得到上一节中时间差分残差的高偏差、低方差估计。如果 $\lambda = 1$，我们得到无偏的完全展开，但方差增加。图13.1展示了不同 λ 值下的算法。

```
struct GeneralizedAdvantageEstimation
    𝒫     # problem
    b     # initial state distribution
    d     # depth
    m     # number of samples
    ∇logπ # gradient of log likelihood ∇logπ(θ,a,s)
    U     # parameterized value function U(ϕ, s)
    ∇U    # gradient of value function ∇U(ϕ,s)
    λ     # weight ∈ [0,1]
end

function gradient(M::GeneralizedAdvantageEstimation, π, θ, ϕ)
    𝒫, b, d, m, ∇logπ = M.𝒫, M.b, M.d, M.m, M.∇logπ
    U, ∇U, γ, λ = M.U, M.∇U, M.𝒫.γ, M.λ
    πθ(s) = π(θ, s)
    R(τ,j) = sum(r*γ^(k-1) for (k,(s,a,r)) in enumerate(τ[j:end]))
    δ(τ,j) = τ[j][3] + γ*U(ϕ,τ[j+1][1]) - U(ϕ,τ[j][1])
    A(τ,j) = sum((γ*λ)^(ℓ-1)*δ(τ, j+ℓ-1) for ℓ in 1:d-j)
    ∇Uθ(τ) = sum(∇logπ(θ,a,s)*A(τ,j)*γ^(j-1)
                 for (j, (s,a,r)) in enumerate(τ[1:end-1]))
    ∇ℓϕ(τ) = sum((U(ϕ,s) - R(τ,j))*∇U(ϕ,s)
                 for (j, (s,a,r)) in enumerate(τ))
    trajs = [simulate(𝒫, rand(b), πθ, d) for i in 1:m]
    return mean(∇Uθ(τ) for τ in trajs), mean(∇ℓϕ(τ) for τ in trajs)
end
```

算法13.2. 用于计算MDP $\mathcal{P}$的策略梯度和价值函数梯度的广义优势估计，初始状态分布为`b`。策略由`θ`参数化，并具有对数梯度`∇logπ`。价值函数`U`由`ϕ`参数化，并具有梯度`∇U`。此方法运行`m`次滚降至深度`d`。广义优势通过指数加权`λ`使用方程(13.21)在有限视野下计算。此处的实现是原始论文中所提出内容的简化版本，原论文在采取步骤时包含了信任区域的各个方面。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

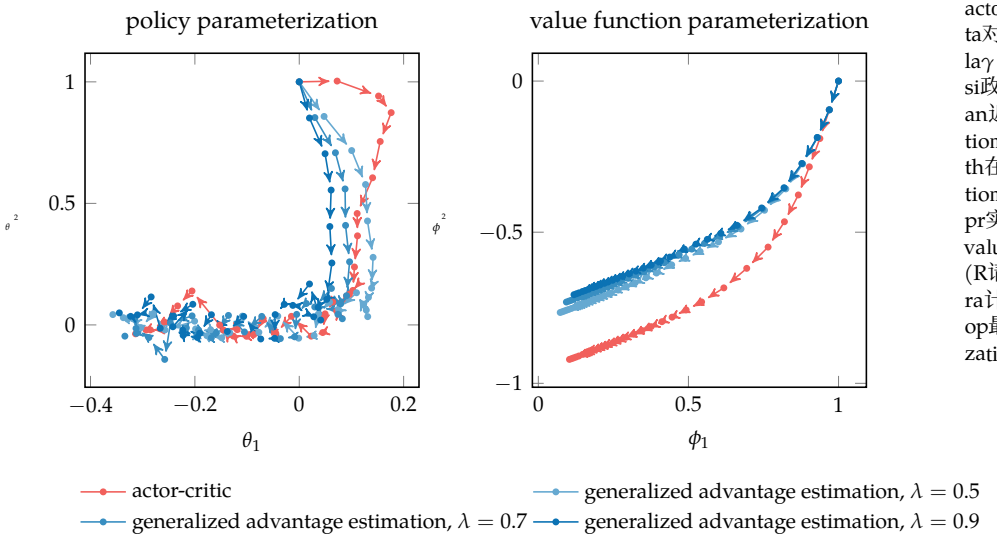


图13.1. 基本对比 actor-critic到广义优势-ta对简单回归的估计 la$\gamma = 0.9$ 的 tor 问题，一个 Gaus-si政策 $\pi_\theta(s) = \mathcal{N}(\theta_1 s、\theta_2^2)$，an近似值函数 tion $U_\phi(s) = \phi_1 s + \phi_2 s^2$。我们发现 th在广义优势估 tion能够更有效地ap-pr实现良好性能的策略以及 value 函数参数化。 (R请记住，最优策略pa-ra计量化是 $[-1, 0]$，并且 op最优值函数参数化—— zation 接近 $[0, -0.7]$。)

13.3 *Deterministic Policy Gradient*

*deterministic policy gradient*方法[3]涉及优化一个确定性策略冰冷的 $\pi_\theta(s)$，在参数化动作价值函数 $Q_\phi(s、a)$ 形式的批评者的帮助下产生连续动作。与迄今为止讨论的演员-评论家方法一样，我们针对参数化 ϕ 定义了一个损失函数：

$$\ell(\phi) = \frac{1}{2} \mathop{\mathbb{E}}_{s,a,r,s'} \left[\left(r + \gamma Q_\phi(s', \pi_\theta(s')) - Q_\phi(s, a) \right)^2 \right] \tag{13.22}$$

其中期望是对由 π_θ 的rollout生成的经验元组取的。这个损失函数试图最小化 Q_ϕ 的残差，类似于第一节中的actor-critic方法试图最小化 U_ϕ 的残差。

与其他方法类似，我们通过朝着梯度的相反方向迈出一步来更新 ϕ：

$$\nabla \ell(\phi) = \mathop{\mathbb{E}}_{s,a,r,s'} \left[\left(r + \gamma Q_\phi(s', \pi_\theta(s')) - Q_\phi(s, a) \right) \left(\gamma \nabla_\phi Q_\phi(s', \pi_\theta(s')) - \nabla_\phi Q_\phi(s, a) \right) \right] \tag{13.23}$$

因此，我们需要一个可微的参数化动作价值函数，从中我们可以计算 $\nabla_\phi Q_\phi(s、a)$，例如神经网络。

3 D. Silver、G. Lever、N. Heess、T. Degris、D. Wierstra 和 M. Riedmiller，“确定性策略梯度算法”，见 *International Conference on Machine Learning (ICML)*，2014年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

对于演员，我们想要找到使 θ 最大化的值

$$U(\theta) = \mathop{\mathbb{E}}_{s \sim b_{\gamma,\theta}}\left[Q_\phi(s, \pi_\theta(s))\right] \tag{13.24}$$

其中期望是对遵循 π_θ 时的折扣访问频率中的状态进行的。再次，我们可以使用梯度上升来优化 θ，其梯度由下式给出：

$$\nabla U(\theta) = \mathbb{E}_s\left[\nabla_\theta Q_\phi(s, \pi_\theta(s))\right] \tag{13.25}$$

$$= \mathbb{E}_s\left[\nabla_\theta \pi_\theta(s) \nabla_a Q_\phi(s, a)|_{a=\pi_\theta(s)}\right] \tag{13.26}$$

这里，$\nabla_\theta \pi_\theta(s)$ 是一个雅可比矩阵，其第 i 列是策略在参数化 θ 下关于第 i 个动作维度的梯度。该项的一个示例见例 13.1。梯度 $\nabla_a Q_\phi(s、a)|_{a=\pi_\theta(s)}$ 是一个向量，表示当我们扰动由策略在状态 s 下给出的动作时，我们估计的动作值变化了多少。除了雅可比矩阵之外，我们还需要提供这个梯度才能使用此方法。

例13.1。确定性策略梯度中雅可比矩阵的一个示例。

考虑以下针对二维动作空间和一维状态空间的确定性策略：

$$\pi_\theta(s) = \begin{bmatrix} \theta_1 + \theta_2 s + \theta_3 s^2 \\ \theta_1 + \sin(\theta_4 s) + \cos(\theta_5 s) \end{bmatrix}$$

矩阵 $\nabla_\theta \pi_\theta(s)$ 则采用以下形式：

$$\nabla_\theta \pi_\theta(s) = \begin{bmatrix} \nabla_\theta \pi_\theta(s)\,|_{a_1} & \nabla_\theta \pi_\theta(s)\,|_{a_2} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ s & 0 \\ s^2 & 0 \\ 0 & \cos(\theta_4 s) s \\ 0 & -\sin(\theta_5 s) s \end{bmatrix}$$

与其他演员-评论家方法一样，我们对 $\ell(\phi)$ 执行梯度下降，对 $U(\theta)$ 执行梯度上升。为了使这种方法在实践中有效，还需要一些额外的技巧。其中之一是从随机策略中生成经验，以便更好地探索。通常，只需添加零均值

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

向由我们的确定性策略 π_θ 生成的动作添加高斯噪声，如同算法 13.3 中所做。为了在学习 θ 和 ϕ 时促进稳定性，我们可以使用经验回放。[4]

izations

该方法的一个示例以及σ对性能的影响在例13.2中给出。

4 我们将在第17.7节中在强化学习的背景下讨论经验回放。其他稳定学习的技术包括使用 *target parameter-*，由T. P. Lillicrap、J. J. Hunt、A. Pritzel、N. Heess、T. Erez、Y. Tassa、D. Silver和D. Wierstra在《连续控制与深度强化学习》中描述，收录于*International Conference on Learning Representations* (*ICLR* 2016)。

```
struct DeterministicPolicyGradient
    𝒫     # problem
    b     # initial state distribution
    d     # depth
    m     # number of samples
    ∇π    # gradient of deterministic policy π(θ, s)
    Q     # parameterized value function Q(ϕ,s,a)
    ∇Qϕ   # gradient of value function with respect to ϕ
    ∇Qa   # gradient of value function with respect to a
    σ     # policy noise
end

function gradient(M::DeterministicPolicyGradient, π, θ, ϕ)
    𝒫, b, d, m, ∇π = M.𝒫, M.b, M.d, M.m, M.∇π
    Q, ∇Qϕ, ∇Qa, σ, γ = M.Q, M.∇Qϕ, M.∇Qa, M.σ, M.𝒫.γ
    π_rand(s) = π(θ, s) + σ*randn()*I
    ∇Uθ(τ) = sum(∇π(θ,s)*∇Qa(ϕ,s,π(θ,s))*γ^(j-1) for (j,(s,a,r))
                 in enumerate(τ))
    ∇ℓϕ(τ,j) = begin
        s, a, r = τ[j]
        s′ = τ[j+1][1]
        a′ = π(θ,s′)
        δ = r + γ*Q(ϕ,s′,a′) - Q(ϕ,s,a)
        return δ*(γ*∇Qϕ(ϕ,s′,a′) - ∇Qϕ(ϕ,s,a))
    end
    ∇ℓϕ(τ) = sum(∇ℓϕ(τ,j) for j in 1:length(τ)-1)
    trajs = [simulate(𝒫, rand(b), π_rand, d) for i in 1:m]
    return mean(∇Uθ(τ) for τ in trajs), mean(∇ℓϕ(τ) for τ in trajs)
end
```

算法13.3. 确定性策略梯度方法，用于计算确定性策略π的策略梯度$\nabla\theta$，以及连续动作MDP $\mathcal{P}$（初始状态分布为`b`）的值函数梯度$\nabla\phi$。策略由θ参数化，其梯度$\nabla\pi$生成一个矩阵，其中每一列是关于该连续动作分量的梯度。值函数`Q`由ϕ参数化，其关于参数化的梯度为$\nabla Q\phi$，关于动作的梯度为`∇Qa`。该方法运行`m`次rollout至深度`d`，并使用均值为0、标准差为σ的高斯噪声进行探索。

13.4 Actor-Critic with Monte Carlo Tree Search

我们可以将在线规划（第9章）中的概念扩展到演员-评论家设置中，其中我们改进参数化策略 $\pi_\theta(a \mid s)$ 和参数化价值函数 $U_\phi(s)$。[5] 本节讨论蒙特卡洛树搜索的应用

（第9.6节）到学习具有离散动作空间的随机策略。我们使用参数化策略和价值函数来引导蒙特卡洛树搜索，

5 确定性策略梯度使用了 Q_ϕ，但这种方法像本章讨论的其他演员-评论家方法一样使用了 U_ϕ。

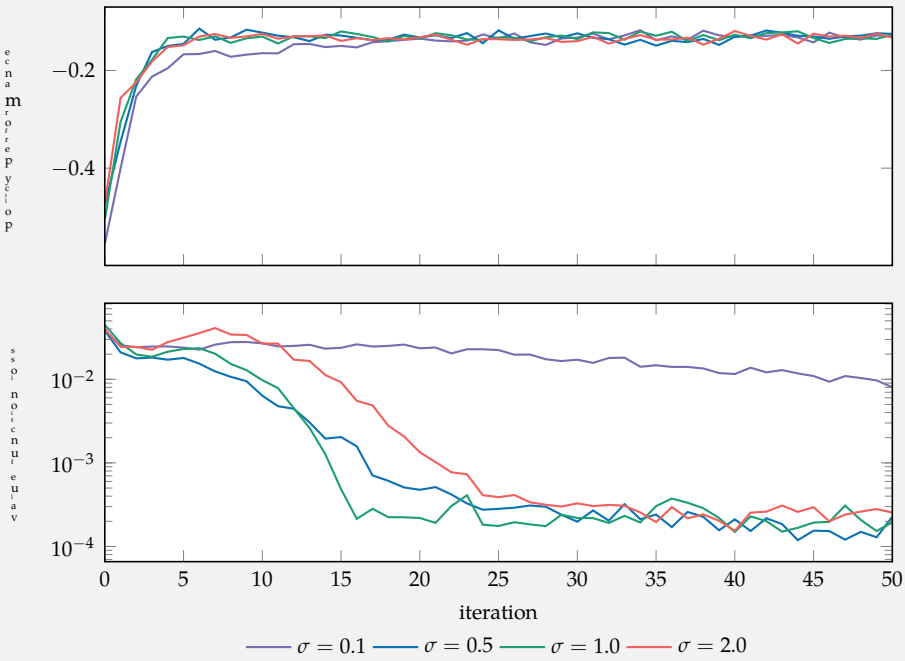
© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

例13.2. 将确定性策略梯度方法应用于简单调节器问题，并探究策略随机性参数σ的影响。

考虑将确定性策略梯度算法应用于简单的调节器问题。假设我们使用一个简单的参数化确定性策略 $\pi_\theta(s) = \theta_1$ 和参数化的状态-动作值函数：

$$Q_\Phi(s,a) = \phi_1 + \phi_2 s + \phi_3 s^2 + \phi_4 (s+a)^2$$

这里，我们绘制了确定性策略梯度算法的进展，从 $\theta = [0]$ 和 $\Phi = [0, 1, 0, -1]$ 开始，针对不同的 σ 值。每次迭代都运行了五次展开，深度为 10，$\gamma =$ 为 0.9。



对于这个简单问题，策略几乎不论 σ 如何都能迅速收敛至最优。然而，若 σ 过小或过大，价值函数的改进就会耗时更长。当 σ 非常小时，我们的策略探索不足，无法有效学习价值函数。而 σ 较大时，我们探索更多，但也更频繁地做出糟糕的移动选择。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们使用蒙特卡洛树搜索的结果来优化我们的参数化策略和价值函数。与其他演员-评论家方法一样，我们对θ和ϕ应用基于梯度的优化。[6]

在执行蒙特卡洛树搜索时，我们希望通过参数化策略$\{v^*\}$在一定程度上引导我们的探索。一种方法是使用一个最大化*probabilistic upper confidence bound*的动作：

$$a = \arg\max_a Q(s,a) + c\pi_\theta(a \mid s)\frac{\sqrt{N(s)}}{1+N(s,a)} \tag{13.27}$$

其中 $Q(s、a)$ 是通过树搜索估计的动作价值，$N(s、a)$ 是第 9.6 节中讨论的访问计数，且 $N(s) = \sum_a N(s、a)$。[7]

在运行树搜索之后，我们可以利用收集到的统计信息来获得$\pi_{\text{MCTS}}(a \mid s)$。一种定义方式是依据计数：[8]

$$\pi_{\text{MCTS}}(a \mid s) \propto N(s,a)^\eta \tag{13.28}$$

其中 $\eta \geq 0$ 是一个超参数，用于控制策略的贪心程度。如果 $\eta = 0$，那么 π_{MCTS} 将随机生成动作。当 $\eta \to \infty$ 时，它将选择在该状态下被选择次数最多的动作。

在我们对 θ 的优化中，我们希望我们的模型 π_θ 能够与通过蒙特卡洛树搜索得到的结果相匹配。我们可以定义的一个损失函数是 $\pi_\theta(\cdot \mid s)$ 相对于 $\pi_{\text{MCTS}}(\cdot \mid s)$ 的期望交叉熵：

$$\ell(\theta) = -\mathbb{E}_s\left[\sum_a \pi_{\text{MCTS}}(a \mid s)\log \pi_\theta(a \mid s)\right] \tag{13.29}$$

其中期望是对树探索过程中所经历状态的期望。梯度为

$$\nabla\ell(\theta) = -\mathbb{E}_s\left[\sum_a \frac{\pi_{\text{MCTS}}(a \mid s)}{\pi_\theta(a \mid s)}\nabla_\theta \pi_\theta(a \mid s)\right] \tag{13.30}$$

为了学习ϕ，我们依据树搜索过程中生成的价值函数来定义损失函数：

$$U_{\text{MCTS}}(s) = \max_a Q(s,a) \tag{13.31}$$

它至少在我们在树搜索过程中探索的状态下被定义。损失函数旨在使 U_ϕ 与树搜索的估计值一致：

$$\ell(\phi) = \frac{1}{2}\mathbb{E}_s\left[\left(U_\phi(s) - U_{\text{MCTS}}(s)\right)^2\right] \tag{13.32}$$

6 这种通用方法由 D. Silver、J. Schrittwieser、K. Simonyan、I. Antonoglou、A. Huang、A. Guez、T. Hubert、L. Baker、M. Lai、A. Bolton 等人提出，“Mastering the Game of Go Without Human Knowledge”，*Nature*，第 550 卷，第 354–359 页，2017 年。这里的讨论大致遵循他们的 *AlphaGo Zero* 算法，但并非试图解决围棋问题，而是试图解决一般 MDP。Alpha Zero 同时扮演双方围棋选手，以及游戏往往有赢家和输家这一事实，使原始方法能够强化获胜行为并惩罚失败行为。推广后的 MDP 形式在应用于类似问题时往往会受到稀疏奖励的影响。

7 与公式（9.1）中给出的上置信界相比，存在一些显著差异；例如，公式（13.27）中没有对数项，并且我们在分母上加1以遵循AlphaGo Zero所采用的形式。

8 在算法9.5中，我们选择相对于Q的贪婪动作。其他策略由C. B. Browne，E. Powley，D. White-house，S. M. Lucas，P. I. Cowling，P. Rohlfshagen，S. Tavener，D. Perez，S. Samothrakis和S. Colton在“蒙特卡洛树搜索方法综述”中进行了调查，*IEEE Transactions on Computational Intelligence and AI in Games*，第4卷，第1期，第43–57页，2012年。此处建议的方法遵循AlphaGo Zero。

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

梯度是

$$\nabla \ell(\boldsymbol{\phi}) = \mathbb{E}_s\left[\left(U_{\boldsymbol{\phi}}(s) - U_{\text{MCTS}}(s)\right)\nabla_{\boldsymbol{\phi}} U_{\boldsymbol{\phi}}(s)\right] \tag{13.33}$$

与第一部分的演员-评论家方法类似，我们需要能够计算参数化价值函数的梯度。

在进行若干次蒙特卡洛树搜索模拟后，我们通过朝 $\nabla \ell(\boldsymbol{\theta})$ 的反方向移动来更新 $\boldsymbol{\theta}$，并通过朝 $\nabla \ell(\boldsymbol{\phi})$ 的反方向移动来更新 $\boldsymbol{\phi}$。[9]

9 AlphaGo Zero实现使用单个神经网络来同时表示价值函数和策略，而非本节所讨论的独立参数化。用于更新网络参数的梯度是方程(13.30)和(13.33)的混合。这一改进显著减少了评估时间和特征学习时间。

13.5 Summary

- 在演员-评论家方法中，演员试图借助评论家来优化参数化的策略，评论家提供值函数的参数化估计。

- 一般来说，演员-评论家方法使用基于梯度的优化来学习策略和价值函数逼近的参数。
- 基本的演员-评论家方法使用策略梯度来优化演员，并最小化评论家的时间差分残差平方。
- 广义优势估计试图通过跨多个时间步累积时间差分残差，以减少其策略梯度的方差，但会带来一些偏差。

- 确定性策略梯度可以应用于具有连续动作空间的问题，并使用确定性策略演员和动作价值评论家。
- 在线方法，如蒙特卡洛树搜索，可用于指导策略和价值函数估计的优化。

13.6 Exercises

练习13.1。如第13.4节所述，采用蒙特卡洛树搜索的演员-评论家方法是否是解决倒立摆问题（附录F.3）的好方法?

Solution: 蒙特卡洛树搜索基于已访问状态扩展树。推车-杆问题具有连续的状态空间，导致搜索树具有无限的 branching factor。使用此算法将需要调整问题，例如离散化状态空间。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

练习13.2. 在以下优势函数的表达式中，判断哪些是正确的，并解释它们所指的内容：

(a) $\mathbb{E}_{r,s'}\left[r + \gamma U^{\pi_\theta}(s) - U^{\pi_\theta}(s')\right]$

(b) $\mathbb{E}_{r,s'}\left[r + \gamma U^{\pi_\theta}(s') - U^{\pi_\theta}(s)\right]$

(c) $\mathbb{E}_{r_{1:d},s'}\left[-U^{\pi_\theta}(s) + \gamma^k U^{\pi_\theta}(s') + \sum_{\ell=1}^{k} \gamma^{l-1} r_l\right]$

(d) $\mathbb{E}_{r_{1:d},s'}\left[-U^{\pi_\theta}(s) + \gamma U^{\pi_\theta}(s') + \sum_{\ell=1}^{k} \gamma^{l-1} r_l\right]$

(e) $\mathbb{E}\left[-U^{\pi_\theta}(s) + \sum_{\ell=1}^{d} \gamma^{l-1} r_l\right]$

(f) $\mathbb{E}\left[-\gamma U^{\pi_\theta}(s') + \sum_{\ell=1}^{d+1} \gamma^{l-1} r_l\right]$

(g) $\mathbb{E}\left[\sum_{\ell=1}^{k} \gamma^{l-1} \delta_{l-1}\right]$

(h) $\mathbb{E}\left[\sum_{\ell=1}^{k} \gamma^{l-1} \delta_l\right]$

(i) $\mathbb{E}\left[\sum_{k=1}^{\infty} (\gamma\lambda)^{k-1} \delta_k\right]$

(j) $\mathbb{E}\left[\sum_{k=1}^{\infty} (\lambda)^{k-1} \delta_k\right]$

Solution: 下表列出了正确的表达式：

(b) Advantage with temporal difference residual
(c) Advantage estimate after k-step rollouts
(e) Advantage with the sequence of rollout rewards
(h) Advantage estimate with temporal difference residuals
(i) Generalized advantage estimate

练习13.3。使用时间差分残差相对于一系列 rollout 奖励有哪些好处，反之亦然?

Solution: 使用时间差分残差的近似在计算上比使用一系列展开更高效。时间差分残差近似由于使用评价器价值函数U_ϕ作为真实价值函数U^{π_θ}的近似器，具有低方差但高偏差。另一方面，展开近似具有高方差

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

是无偏的。使用时间差分残差近似获得准确估计通常比使用展开近似所需的样本少得多，代价是给我们的估计引入了偏差。

练习13.4。考虑例13.2中给出的动作价值函数，$Q_{\boldsymbol{\phi}}(s、a) =、\phi_1 + \phi_2 s + \phi_3 s^2 + \phi_4 (s + a)^2$。计算确定性策略梯度方法所需的梯度。

Solution: 我们需要计算两个梯度。对于actor，我们需要计算$\nabla_{\boldsymbol{\phi}} Q_{\boldsymbol{\phi}}(s、a)$，而对于critic，我们需要计算$\nabla_a Q_{\boldsymbol{\phi}}(s、a)$。

$$\nabla_{\boldsymbol{\phi}} Q(s,a) = \left[1, s, s^2, (s+a)^2\right]$$
$$\nabla_a Q(s,a) = 2\phi_4 (s+a)$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

14 Policy Validation

前几章介绍的方法展示了如何根据动力学和奖励模型构建最优或近似最优解。然而，在将决策系统部署到现实世界之前，通常希望在仿真中验证所得到策略的行为是否与预期一致。本章讨论用于验证决策策略的各种分析工具。[1]我们将从讨论如何

关于如何评估性能指标。准确计算这些指标在计算上可能具有挑战性，尤其是当它们涉及罕见事件（如故障）时。我们将讨论有助于解决计算效率问题的方法。重要的是，我们的系统要能对分析所用模型与现实世界之间的差异保持稳健。本章提出了分析稳健性的方法。许多决策系统设计的根本在于多个目标之间的权衡，我们将概述分析这些权衡的方法。本章最后讨论对抗性分析，它可用于寻找最可能的故障轨迹。

1 更广泛的讨论由A. Corso、R. J. Moss、M. Koren、R. Lee和M. J. Kochen-derfer提供。*Jour-nal of Artificial Intelligence Research*，第72卷，第377–428页，2021年。“黑盒安全验证算法综述”

14.1 Performance Metric Evaluation

一旦我们有了策略，我们往往有兴趣针对各种*performance metrics*对其进行评估。例如，假设我们构建了一个防撞系统——无论是通过某种标量奖励函数的优化，还是如例14.1所讨论的仅凭启发式方法——并且我们希望通过计算遵循我们的策略时发生碰撞的概率来评估其安全性。[2]或者，如果我们创建了一个用于构建投资组合的策略，我们可能会

对理解我们的策略导致极端损失的概率或预期回报可能是什么感兴趣。

2 其他安全风险度量由 I. L. Johansen 和 M. Rausand 在“风险度量的基础与选择”中讨论，*Safety Sci-ence*，第 62 卷，第 386–399 页，2014 年。

目前，我们将考虑一个单一的度量 f，在策略 π 上进行评估。通常，这个度量被定义为轨迹度量 f_{traj} 的期望，在遵循该策略产生的轨迹 $\tau = (s_1, a_1, \ldots)$ 上进行评估：

$$f(\pi) = \mathbb{E}_\tau[f_{\text{traj}}(\tau)] \tag{14.1}$$

这个期望是基于轨迹分布的。为了定义与MDP相关的轨迹分布，我们需要指定一个*initial state distribution* b。生成轨迹τ的概率是

$$P(\tau) = P(s_1, a_1, \ldots) = b(s_1) \prod_t T(s_{t+1} \mid s_t, a_t) \tag{14.2}$$

在碰撞避免的语境中，如果轨迹导致了碰撞，f_{traj}可能为1，否则为0。期望值对应于碰撞概率。

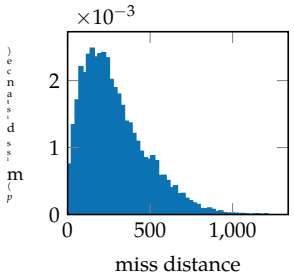


图14.1. 遵循简单避碰策略，从初始状态出发，经过10^4次模拟估计的脱靶距离分布

在某些情况下，我们感兴趣的是研究 f_{traj} 的输出的分布。图14.1展示了这种分布的一个例子。方程(14.1)中的期望只是将轨迹度量分布转换为单一值的众多方法之一。在我们的讨论中，我们将主要关注这个期望，但将分布转换为值的其他变换示例包括方差、第五百分位数以及低于第五百分位数的值的均值。[3]

$h \sim \mathcal{U}(-10, 10)$ (m)
$\dot{h} \sim \mathcal{U}(-200, 200)$ (m/s)
$a_{\text{prev}} = 0\,\text{m/s}$
$t_{\text{col}} = 40\,\text{s}$

轨迹度量有时可以写成这种形式：

$$f_{\text{traj}}(\tau) = f_{\text{traj}}(s_1, a_1, \ldots) = \sum_t f_{\text{step}}(s_t, a_t) \tag{14.3}$$

其中 f_{step} 是一个依赖于当前状态和动作的函数，很像 MDP 中的奖励函数。如果 $f(\pi)$ 定义为 f_{traj} 的期望，那么目标与求解 MDP 时相同，其中 f_{step} 就是奖励函数。因此，我们可以使用第 7.2 节中介绍的策略评估算法来评估我们的策略相对于公式 (14.3) 形式的任何性能指标。

[3] 文献中讨论了多种风险度量方法。A. Ruszczyński在“Risk-Averse Dynamic Programming for Markov Decision Processes”一文中概述了一些在MDP背景下使用的风险度量，*Mathematical Programming*，第125卷，第2期，第235-261页，2010年。

策略评估将输出一个值函数，该函数是状态的函数，[4] 对应于从该状态开始时期望达到的性能指标值。例14.2展示了碰撞避免问题中该价值函数的几个片段。整体性能由下式给出：

$$f(\pi) = \sum_s f_{\text{state}}(s) b(s) \tag{14.4}$$

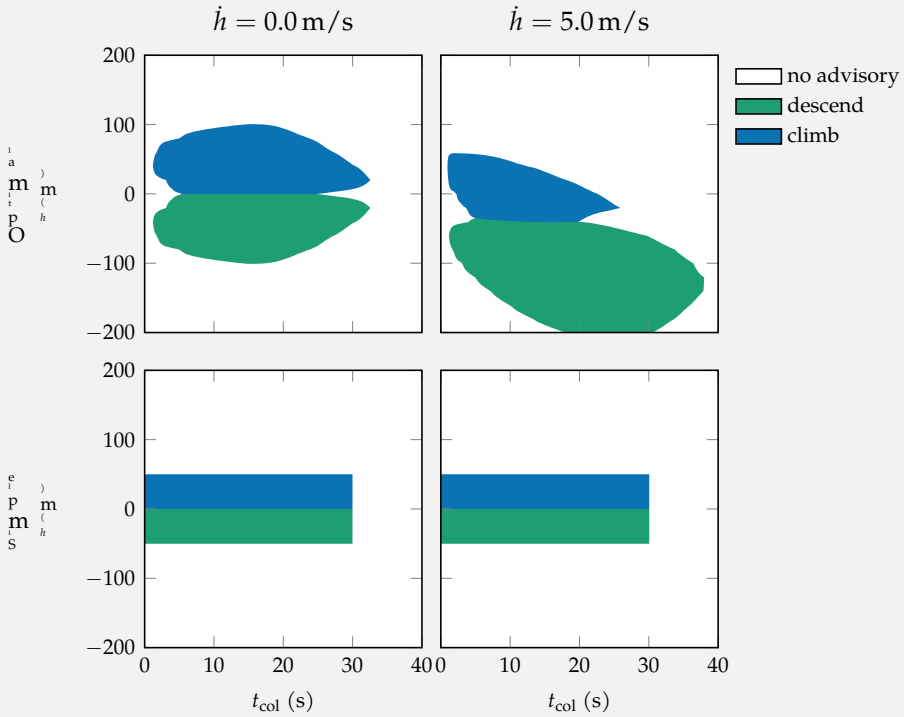
其中 f_{state} 是通过策略评估获得的价值函数。

[4] 在前几章中，我们使用 U^π 来表示与策略 π 相关的价值函数。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

在飞机防撞问题中，我们需要决定何时向我们的飞机发出爬升或下降建议，以避开入侵飞机。入侵者正以恒定的水平接近速度迎面逼近我们。状态由以下因素指定：相对于入侵飞机测量的我们飞机的高度 h、我们的垂直速率 $\dot{h}$、先前的动作 a_{prev} 以及潜在碰撞的时间 t_{col}。当发生碰撞时，即当入侵者在 $t_{\text{col}} = 0$ 时进入50米范围内，会有1的惩罚。此外，当 $a \neq a_{\text{prev}}$ 时会有0.01的惩罚，以阻止建议变更。

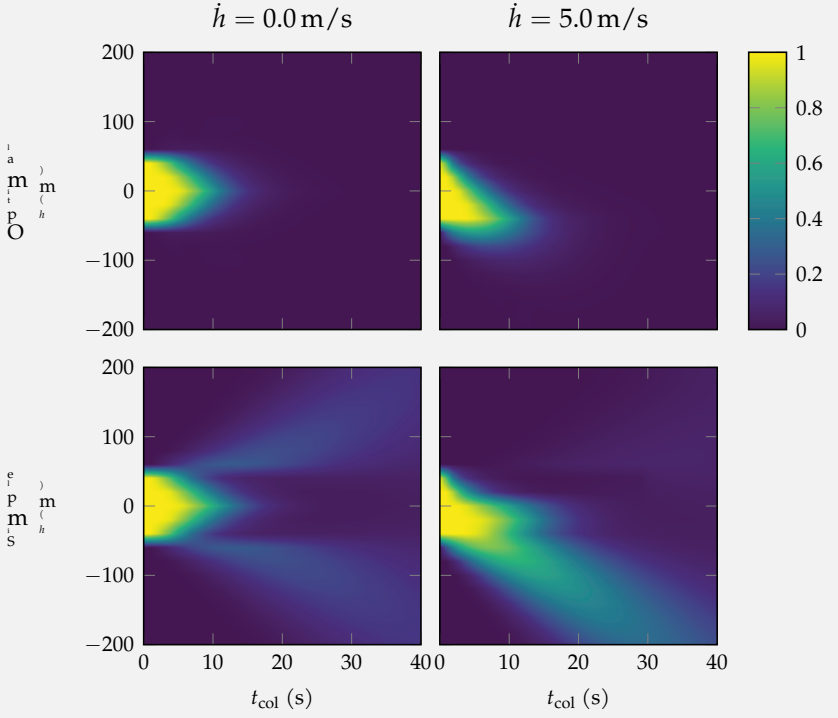
我们可以使用带有线性插值的动态规划（第8.4节）来推导最优策略。或者，我们可以定义一种简单的启发式策略，由t_{col}和h上的阈值参数化，其工作方式如下。如果$|h| < h_{\text{thresh}}$和$t_{\text{col}} < t_{\text{thresh}}$，则生成一条建议。该建议是在$h >$为0时爬升，否则下降。默认情况下，我们使用$h_{\text{thresh}} =$为50米和$t_{\text{thresh}} =$为30秒。以下是状态空间中两个切片的最优策略和简单策略的图：



示例14.1. 最优与简化的避碰策略。问题的额外细节见附录F.6。

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

这里展示了将策略评估应用于最优策略和示例14.1中引入的简单策略的结果。图中的每个点对应于从相关状态开始时的度量值。我们定义 $f_{\text{state}}(s$，如果 s 是碰撞，则 $a) =$ 为1，否则为0。该图显示了遵循该策略时，在状态空间中存在显著碰撞风险的位置，用“更热”的颜色表示。我们可以看到，最优策略相当安全，特别是当 $t_{\text{col}} >$ 为20秒时。当 t_{col} 较低时，由于车辆的物理加速度限制，即使是最优策略也无法避免碰撞。与最优策略相比，简单策略的风险水平要高得多，特别是当 $t_{\text{col}} >$ 为20秒、$\dot{h} =$ 为5米/秒且入侵者位于我们下方时——部分原因是简单策略中生成建议的选择没有考虑 $\dot{h}$。



示例14.2. 遵循最优及简单避碰策略时发生碰撞的概率。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

如果状态空间是离散的，则方程 (14.4) 可以解析计算。然而，如果状态空间很大或连续，我们可能希望通过采样来估计 $f(\pi)$。我们可以从初始状态分布中抽取一个样本，然后展开策略并计算轨迹度量。然后，我们可以从轨迹度量的均值来估计总体度量的值。估计的质量通常随着样本数量的增加而提高。例 14.3 说明了这一过程，用于估计与避碰策略相关的各种度量。

我们经常使用 *standard error* 来衡量估计的质量：

$$\text{SE} = \hat{\sigma}/\sqrt{n} \tag{14.5}$$

其中 $\hat{\sigma}$ 是样本的标准差，n 是样本数量。在示例14.3中，碰撞指标的标准差为0.0173，因此碰撞概率指标的标准误差为0.000173。

我们可以将标准误差转换为*confidence interval*。例如，一个95%置信区间将是$\hat{\mu} \pm 1.96$ SE，其中$\hat{\mu}$是我们样本的均值。对于我们的避碰示例，该区间为$(-3.94 \times 10^{-5}, 6.39 \times 10^{-4})$。或者，我们可以采用贝叶斯方法，将我们的后验表示为贝塔分布，如第4.2节所讨论的那样。

对于小概率，例如相对安全系统中的故障概率，我们通常关注的是 *relative standard error*，其由下式给出

$$\frac{\hat{\sigma}}{\hat{\mu}\sqrt{n}} \tag{14.6}$$

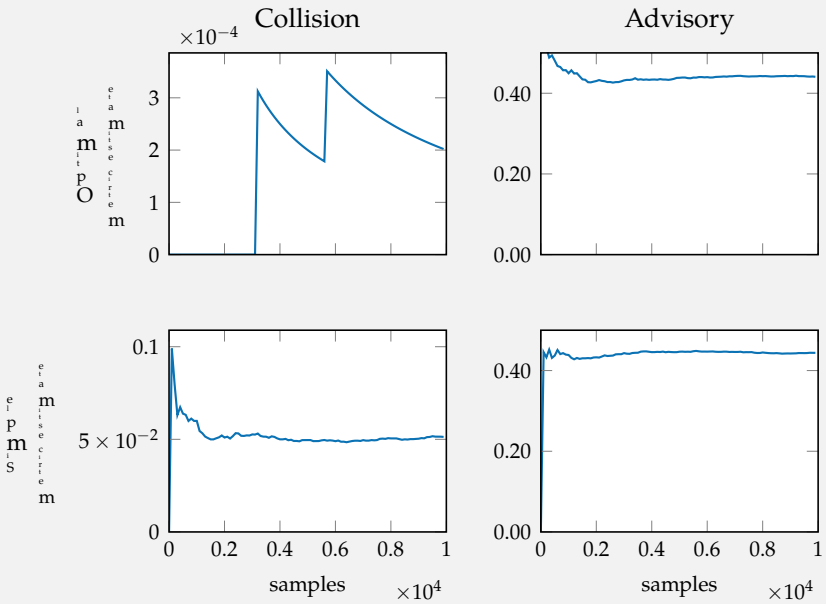
这相当于将标准误差除以均值。在我们的碰撞避免问题中，相对误差为0.578。尽管绝对误差可能很小，但由于我们试图估计一个较小的概率，相对误差相当高。

14.2 Rare Event Simulation

正如我们在例14.3中看到的，为了准确估计那些罕见事件影响很大的指标，我们可能需要大量样本，例如估计碰撞概率。在避免碰撞的例子中，我们的10^4个样本仅包含三次碰撞，正如图中三个尖峰所示。当我们为高风险系统设计算法时，比如涉及金钱交易或驾驶汽车的系统，

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们希望估计碰撞的概率以及生成建议的概率。这里，我们将考虑例14.1中介绍的最优策略和简单策略。为了评估这些指标，我们从图14.1中使用的初始状态分布中抽取10^4个样本，然后进行展开。这里的图表展示了收敛曲线：



我们可以看到，最优策略比简单策略安全得多，同时发布建议的频率大致相同。建议指标的估计收敛速度远快于碰撞估计。建议指标收敛更快的原因在于建议比碰撞更为常见。涉及最优策略的碰撞极为罕见，以至于即便有10^4个样本，似乎也不足以进行准确估计。曲线非常参差不齐，在涉及碰撞的样本处出现大的尖峰，随后随着无碰撞样本的模拟，碰撞概率估计逐渐衰减。

示例14.3。遵循最优和简单碰撞避免策略时的碰撞概率和建议。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

通过直接采样和模拟来准确估计故障概率在计算上可能具有挑战性。

一种提高效率的常见方法称为*importance sampling*，它涉及从替代分布中抽样，并对结果进行适当加权，以获得无偏估计。[5] 我们使用了同样的方在贝叶斯网络推断的背景下，这种方法被称为似然加权采样（第3.7节）。替代的采样分布通常被称为*proposal distribution*，我们将使用$P'(\tau)$来表示我们的提议分布赋予轨迹τ的概率。

关于重要性采样及其他稀有事件模拟技术的更详尽介绍，由J. A. Bucklew提供，*Introduction to Rare Event Simulation*施普林格，2004年。

我们将从P'推导出适当的样本加权方法。如果我们有从真实分布P中抽取的$\tau^{(1)}, \ldots, \tau^{(n)}$，那么我们有

$$f(\pi) = \mathbb{E}_\tau[f_{\text{traj}}(\tau)] \tag{14.7}$$

$$= \sum_\tau f_{\text{traj}}(\tau) P(\tau) \tag{14.8}$$

$$\approx \frac{1}{n} \sum_i f_{\text{traj}}(\tau^{(i)}) \text{ with } \tau^{(i)} \sim P \tag{14.9}$$

我们可以将方程（14.8）乘以 $P'(\tau)/P'(\tau)$ 并得到以下结果：

$$f(\pi) = \sum_\tau f_{\text{traj}}(\tau) P(\tau) \frac{P'(\tau)}{P'(\tau)} \tag{14.10}$$

$$= \sum_\tau f_{\text{traj}}(\tau) P'(\tau) \frac{P(\tau)}{P'(\tau)} \tag{14.11}$$

$$\approx \frac{1}{n} \sum_i f_{\text{traj}}(\tau^{(i)}) \frac{P(\tau^{(i)})}{P'(\tau^{(i)})} \text{ with } \tau^{(i)} \sim P' \tag{14.12}$$

换句话说，我们需要对来自提议分布的样本结果进行加权，其中样本i的权重[6]为$P(\tau^{(i)})/P'(\tau^{(i)})$。

6 重要的是，P' 不应对 P 赋予正似然的任何轨迹赋予零似然。

我们希望选择提议分布P'，使样本生成集中在那些“重要”的样本上，即更有可能对整体性能估计做出贡献的样本。在避免碰撞的情况下，我们希望这种提议分布鼓励碰撞，以便我们有不止少数碰撞情况来估计碰撞风险。然而，我们不希望所有样本都导致碰撞。一般来说，假设历史空间是离散的，最优提议分布是

$$P^*(\tau) = \frac{|f_{\text{traj}}(\tau)| P(\tau)}{\sum_{\tau'} |f_{\text{traj}}(\tau')| P(\tau')} \tag{14.13}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

如果 f_{traj} 是非负的，那么分母与我们在等式（14.1）中试图估计的度量完全相同。

虽然方程 (14.13) 通常不便于精确计算（这正是我们采用重要性抽样的原因），但它可以提供一些直观见解，帮助我们利用领域知识来构建提议分布。常见的做法是使初始状态分布或转移模型稍微偏向更重要的轨迹，例如偏向碰撞的情况。

为了说明重要性分布的构造，我们将使用例14.1中避碰问题的最优策略。我们从t_{col} =40秒开始，而是从更近的位置开始，即t_{col} =20秒，以使避碰问题更具挑战性。真实分布为$h \sim \mathcal{U}(-10, 10)$ (m)和$\dot{h} \sim \mathcal{U}(-200, 200)$ (m/s)。然而，h和$\dot{h}$的某些组合对最优策略来说更难以解决。我们使用了问题离散版本的动态规划来确定不同h和$\dot{h}$值下的碰撞概率。我们可以将这些结果归一化，将其转换为图14.2所示的提议分布。

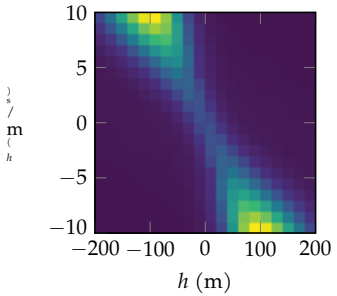


图14.2. 当遵循来自不同初始状态的最优防撞策略且t_{col} =为20秒、a_{prev} =为0米/秒时，基于碰撞概率生成的提议分布。黄色表示较高的概率密度。

使用图 14.2 中所示的提议分布，在相同样本数量下，比直接采样能得到更好的碰撞概率估计。图 14.3 展示了收敛曲线。到 5×10^4 个样本时，两种采样方法都收敛到相同的估计值。然而，重要性采样在 10^4 个样本内就紧密收敛到真实值。使用我们的提议分布，重要性采样生成了 939 次碰撞，而直接采样只生成了 246 次。如果我们同时对转移分布进行偏置，而不仅仅是偏置初始状态分布，还能生成更多碰撞。

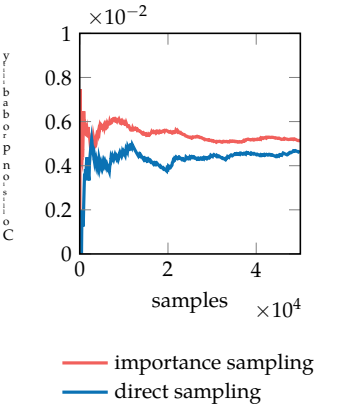


图14.3. 遵循最优策略时，通过重要性采样和直接采样估计的碰撞概率。

14.3 *Robustness Analysis*

在将系统部署到现实世界之前，研究其对建模误差的鲁棒性非常重要。我们可以使用前面几节提到的工具，例如策略评估和重要性采样，但在偏离优化策略时所假设模型的环境上评估我们的策略。图14.4展示了当真实模型偏离用于优化的模型时，性能如何变化。我们还可以研究我们的指标对状态空间上建模假设的敏感性（示例14.4）。如果在

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

环境模型，那么我们就可以更有信心地认为，我们的系统在部署时会按计划运行。

我们通常希望我们的*planning model*，即用于优化策略的模型，相对简单，以防止过度拟合那些可能不代表现实世界的错误建模假设。更简单的规划模型的一个附带好处是它们可以使规划在计算上更高效。然而，我们的*evaluation model*可以尽可能复杂，只要我们能证明其合理性。例如，在生成防撞策略时，我们可能使用一个简单、低维、离散的飞机动力学模型，但随后在连续、高保真的仿真中评估该策略。更简单的规划模型通常对评估模型中的扰动更具鲁棒性。

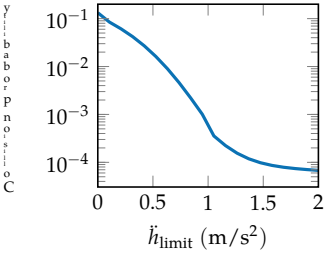


图14.4. 针对$\ddot{h}_{\text{limit}} = 1$米/秒^{2}优化的策略在具有不同$\ddot{h}_{\text{limit}}$值的环境中的鲁棒性分析。

在多种评估模型上评估我们的策略的过程有时被称为*stress testing*，尤其是当评估模型的范围包括相当极端的场景时。在防撞中，极端场景可能包括飞机以可能无法在物理上实现的极端爬升率相互接近的情况。了解哪些类别的场景可能导致系统故障在设计阶段可能是有用的，即使我们选择不针对这些场景优化系统行为，因为它们被认为是不现实的。

如果我们发现我们的策略对建模假设过于敏感，我们可能会考虑使用一种称为 *robust dynamic programming* 的方法。[7] 而不是承诺于一个特定的转移动态模型，我们拥有一系列转移动态模型$T_{1:n}$和奖励模型$R_{1:n}$。我们可以修改方程(7.16)中的贝尔曼更新方程，以如下方式提供对不同模型的鲁棒性：

$$U_{k+1}(s) = \max_a \min_i \left(R_i(s,a) + \gamma \sum_{s'} T_i(s' \mid s,a) U_k(s') \right) \tag{14.14}$$

更新使用在使用使我们的效用最小化的模型时最大化期望效用的动作。

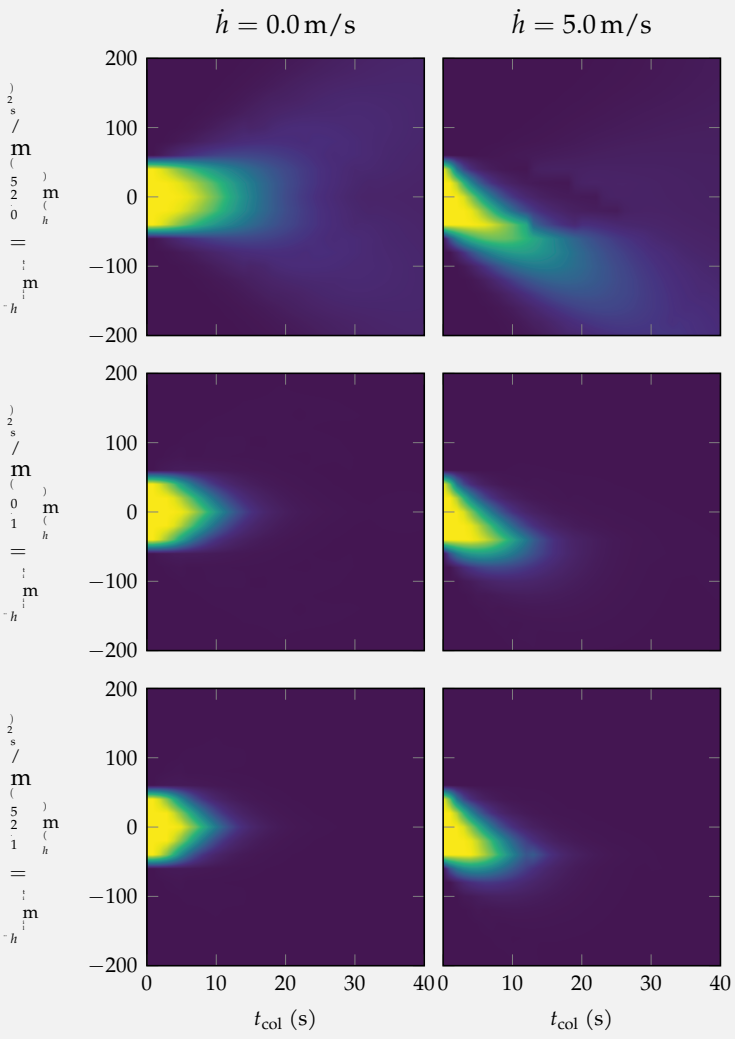
7 G. N. Iyengar，“鲁棒动态规划”，*Mathematics of Operations Research*，第30卷，第2期，第257–280页，2005年。这种方法可以在避免碰撞的背景下提高鲁棒性。M. J. Kochenderfer、J. P. Chryssanthacopoulos和P. Radecki，“优化避碰逻辑对建模误差的鲁棒性”，见 *Digital Avionics Systems Conference (DASC)*，2010年。

14.4 Trade Analysis

许多有趣的任务涉及多个、往往相互竞争的目标。对于自主系统而言，安全与效率之间常常存在权衡。在设计碰撞避免系统时，我们希望非常安全，同时又不使

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们可以绘制从不同初始状态开始时的碰撞概率，类似于示例14.2。这里，我们使用针对附录F.6中参数优化的策略，但在评估模型中我们会改变限制$\dot{h}_{\text{limit}}$。



我们用$\ddot{h}_{\text{limit}} = 1\ \text{m/s}^2$优化了策略。如果实际上是$0.25\ \text{m/s}^2$，那么该策略在某些状态下表现不佳，因为达到目标垂直速率需要更长时间。如果限制是$1.25\ \text{m/s}^2$，我们会更安全一些。

例14.4。当用于规划的模型与用于评估的模型之间存在不匹配时，遵循最优防撞策略发生碰撞的概率。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

过多的不必要规避动作。*trade analysis*研究了随着设计参数变化，各种性能指标如何权衡。

如果只考虑两个性能指标，我们可以绘制一个*trade-off curve*，如示例14.5中所讨论的那样。通过改变策略中的参数，我们获得两个指标的不同值。这些曲线在比较生成策略的不同方法时非常有用。例如，示例14.5中的曲线表明，生成策略的动态规划方法可能比简单的基于阈值的策略带来显著的好处——至少在我们定义它们的方式上是这样。

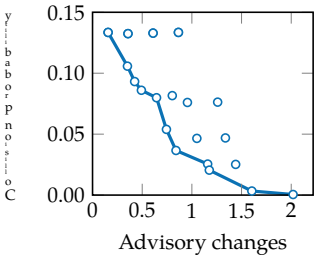


图14.5. 通过改变示例14.1中简单策略的参数生成的策略性能。近似的帕累托曲线以蓝色高亮显示。

对于例14.5中的每条曲线，我们一次只改变一个参数，但要得到满意的系统，我们可能需要研究改变多个参数的效果。当我们改变多个参数时，我们得到一个可能策略的空间。其中一些策略在所有性能指标上可能都比该空间中至少一个其他策略表现更差。我们通常可以从考虑中排除那些被其他策略*dominated*的策略。如果一个策略在该空间中不被任何其他策略支配，则称为*Pareto optimal*[8]或*Pareto efficient*。

以意大利经济学家维尔弗雷多·费德里科·达马索·帕累托（1848–1923）命名。

空间。帕累托最优策略集称为*Pareto frontier*，或（在二维情况下）称为*Pareto curve*。图14.5展示了帕累托曲线的一个示例。

14.5 Adversarial Analysis

研究策略的鲁棒性从 *adversarial analysis* 的角度可能很有用。在每个时间步，*adversary* 选择从当前状态应用策略指定的动作后产生的状态。对手有两个目标需要平衡：最小化我们的回报和最大化根据我们的转移模型得到的轨迹的可能性。我们可以将原始问题转化为对抗问题。对抗状态空间与原始问题相同，但对抗动作空间是原始问题的状态空间。对抗奖励是

$$R'(s,a) = -R(s,\pi(s)) + \lambda \log(T(a \mid s,\pi(s))) \tag{14.15}$$

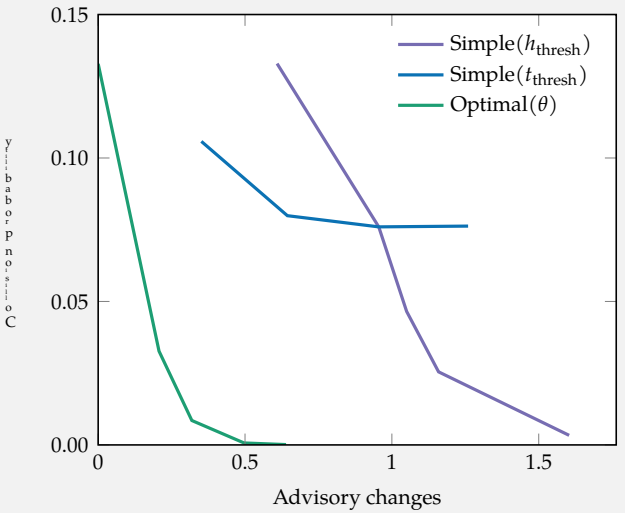
其中 π 是我们的策略，R 是我们原始的奖励函数，T 是我们原始的转移模型，而 $\lambda \geq 0$ 是一个控制最大化所得轨迹似然性重要程度的参数。由于对手试图最大化对抗奖励的总和，它实际上是在最大化我们的期望负回报加上 λ 乘以所生成轨迹的对数概率。[9] 对抗性转移

9 轨迹的对数概率等于各个状态转移概率的对数之和。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

在我们的飞机防撞问题中，我们必须在碰撞概率的安全性与其他指标之间取得平衡，例如预期咨询变更次数。这两个指标都可以通过按照步骤可加分解的轨迹度量来实现，正如方程（14.3）所做的那样，使我们能够使用精确策略评估来计算它们。

这里的图表展示了与简单策略和最优策略的不同参数化版本相关的三条曲线。第一条曲线显示了随着h_{thresh}参数（在示例14.1中定义）的变化，简单策略在两个指标上的表现。第二条曲线显示了随着t_{thresh}变化，简单策略的表现。第三条曲线显示了随着参数θ的变化，最优策略的表现，其中碰撞成本为$-\theta$，更改建议的成本为$-(1-\theta)$。



我们可以看到，最优策略优于参数化的简单策略生成的曲线。当θ接近1时，我们非常安全，但必须容忍更多建议变更。随着θ趋近于0，我们的安全性降低，但不产生建议。给定特定的安全阈值水平，我们能够创建一个优化策略，其在预期中的建议变更次数少于任一简单的参数策略。

示例 14.5. 对不同避碰系统参数变化时安全性与运行效率之间权衡的分析。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

模型是确定性的；状态恰好转移到对手指定为其动作的状态。

算法14.1实现了向对抗性问题的转换。它假设状态和动作空间是离散的，然后可以使用第7章中的一种动态规划算法来求解。解决方案是一个将状态映射到状态的对抗策略。给定一个初始状态，我们可以在某一概率水平下生成一条使我们的奖励最小化的轨迹。由于问题是确定性的，它实际上是一个搜索问题，可以使用附录E中的任何算法。如果我们的问题是高维的或连续的，我们可以使用第8章和第9章中讨论的一种近似求解技术。

```
function adversarial(𝒫::MDP, π, λ)
    𝒮, 𝒜, T, R, γ = 𝒫.𝒮, 𝒫.𝒜, 𝒫.T, 𝒫.R, 𝒫.γ
    𝒮′ = 𝒜′ = 𝒮
    R′ = zeros(length(𝒮′), length(𝒜′))
    T′ = zeros(length(𝒮′), length(𝒜′), length(𝒮′))
    for s in 𝒮′
        for a in 𝒜′
            R′[s,a] = -R(s, π(s)) + λ*log(T(s, π(s), a))
            T′[s,a,a] = 1
        end
    end
    return MDP(T′, R′, γ)
end
```

算法14.1. 转换为对抗问题，给定策略π。一个对抗代理试图改变我们策略动作的结果，以便在最小化我们原始效用和最大化轨迹的可能性之间取得平衡。参数λ控制着最大化所产生轨迹可能性的重要程度。它返回一个MDP，其转移和奖励模型以矩阵表示。

有时我们感兴趣的是找到与某一策略相关联的*most likely failure*，针对特定的失败定义。在某些问题中，失败可定义为进入特定状态。例如，在我们的碰撞避免问题中，碰撞可能被视为失败。其他问题可能需要更复杂的失败定义，这超出了仅进入状态空间子集的范围。例如，我们可能希望使用*temporal logic*来指定失败，这是一种表示和推理时间限定命题的方式。然而，在许多情况下，我们可以利用这些失败规格来创建一个扩展的状态空间，进而求解。[10]

10 M. Bouton, J. Tumova, and M. J. Kochenderfer, “基于点的方法在部分可观察马尔可夫决策过程中的模型检测”，载于 *AAAI Conference on Artificial Intelligence* (*AAAI*)，2020年

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

在定义了失败状态后，我们可以通过将式(14.15)中的奖励函数改为下式来求解最可能的失败轨迹：

$$R'(s,a) = \begin{cases} -\infty & \text{if } s \text{ is terminal and not a failure} \\ 0 & \text{if } s \text{ is terminal and a failure} \\ \log(T(a \mid s, \pi(s))) & \text{otherwise} \end{cases} \tag{14.16}$$

我们可以使用各种近似方法来找到这些最可能的故障。根据近似方法的不同，放松对终止时未达到故障的无限惩罚可能很重要，以便搜索可以被引导到故障。如果将蒙特卡洛树搜索应用于碰撞避免，惩罚可能与错过距离有关。[11]

我们可以回放最可能的故障轨迹，并评估该轨迹是否值得担忧。如果该轨迹被认为极不可能，那么我们就可以更有信心地认为我们的策略是安全的。然而，如果该故障轨迹确实值得担忧，那么我们可能有几个选择：

11 这一策略由 R. Lee、M. J. Kochenderfer、O. J. Meng-shoel、G. P. Brat 和 M. P. Owen 在 *Digital Avionics Systems*，2015 中提出，题为“机载防撞系统的自适应压力测试”。

1. *Change the action space.* 对于我们的避碰问题，我们可能会在动作集中添加更多极端机动动作。

2. *Change the reward function.* 我们可以降低更改咨询的成本，以降低碰撞风险，如例14.5中的权衡曲线所示。

3. *Change the transition function.* 我们可以提高加速度限制，以便在按照我们的策略指示时，飞机能更快地达到目标垂直速率。

4. *Improve the solver.* 我们可能采用了过于粗略的状态空间离散化，以至于无法捕捉最优策略的重要特征。为了获得更好的策略，我们或许愿意花费更多的计算时间，来细化离散化。或者，我们也可以采用不同的近似方法。

5. *Do not deploy the system.* 如果该策略不安全，最好不要将其部署在现实世界中。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

14.6 Summary

- 策略的性能指标可以使用前面章节讨论的动态规划技术或通过采样推演进行评估。
- 我们可以使用标准误差、置信区间或之前讨论的贝叶斯方法来评估我们对性能指标评估的信心。
- 使用一种称为重要性采样的方法可以更有效地估计稀有事件的概率。
- 重要性采样涉及从替代分布中抽样，并对结果进行适当的加权。
- 由于用于优化的模型可能无法准确反映现实世界，研究我们的策略对建模假设的敏感性非常重要。
- 鲁棒动态规划可以通过针对一组不同的转移和奖励模型进行优化，帮助提高对模型不确定性的鲁棒性。
- 交易分析可以帮助我们确定在优化策略时如何平衡多个绩效目标。
- 对抗分析涉及一个对手，其选择我们在每一步过渡到的状态，以便最小化我们的目标同时最大化轨迹的可能性。

14.7 Exercises

练习14.1。我们有一个轨迹τ，其中

s_1	a_1	s_2	a_2	s_3
6.0	2.2	1.4	0.7	6.0

我们的动态是线性高斯的，具有$T(s' \mid s、a) = \mathcal{N}(s' \mid 2s + a、5^2)$，且我们的初始状态分布由$\mathcal{N}(5、6^2)$给出。轨迹$\tau$的对数似然是多少？

Solution: 轨迹的对数似然为

$$\log \mathcal{N}(6.0 \mid 5, 6^2) + \log \mathcal{N}(1.4 \mid 2 \cdot 6.0 + 2.2, 5^2) + \log \mathcal{N}(6.0 \mid 2 \cdot 1.4 + 0.7, 5^2) \approx -11.183$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

练习14.2。我们进行了百万次模拟，发现我们的避撞系统导致了10次碰撞。我们的碰撞概率估计值及相对标准误差是多少?

Solution: 碰撞概率估计为

$$\hat{\mu} = 10/10^6 = 10^{-5}$$

第 i 个样本 x_i 如果发生碰撞则为 1，否则为 0。标准差为

$$\hat{\sigma} = \sqrt{\frac{1}{10^6 - 1}\sum_{i=1}^{n}(x_i - \hat{\mu})^2} = \sqrt{\frac{1}{10^6 - 1}\left(10(1-\hat{\mu})^2 + (10^6 - 10)\hat{\mu}^2\right)} \approx 0.00316$$

相对误差为

$$\frac{\hat{\sigma}}{\hat{\mu}\sqrt{n}} \approx \frac{0.00316}{10^{-5}\sqrt{10^6}} = 0.316$$

练习14.3。我们想要计算期望$\mathbb{E}_{x\sim\mathcal{U}(0,5)}[f(x)]$，其中如果$|x|$ ≤=1，则$f(x)$为-1，否则为0。最优提议分布是什么?

Solution: 最优提议分布是

$$p^*(x) = \frac{|f(x)|p(x)}{\int |f(x)|p(x)\,\mathrm{d}x}$$

这等价于$\mathcal{U}(0,1)$，因为$f(x)$仅在$x \in [-1, 1]$时非零，$\mathcal{U}(0,5)$仅在$x \in [0, 5]$上有支持，且$f(x)$和$p(x)$在非零时均产生恒定值。

练习 14.4。假设我们从先前练习中的提议分布中抽取样本 0.3，其权重是多少? $\mathbb{E}_{x\sim\mathcal{U}(0,5)}[f(x)]$ 的估计值是多少?

Solution: 权重为 $p(x)/p^*(x) = 0.2/1$。由于 $f(0.3) = -1$，估计值为 -0.2，这正是准确答案。

练习 14.5. 假设我们有以下四项策略，它们已在三个我们希望最大化的指标上进行了评估:

System	f_1	f_2	f_3
π_1	2.7	1.1	2.8
π_2	1.8	2.8	4.5
π_3	9.0	4.5	2.3
π_4	5.3	6.0	2.8

哪些策略位于帕累托前沿?

Solution: 只有π_1被其他策略支配。因此，π_2、π_3和π_4位于帕累托前沿。

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

第三部分

模型不确定性

在我们目前对序贯决策问题的讨论中，我们假设转移和奖励模型是已知的。然而，在许多问题中，这些模型并不完全已知，智能体必须通过经验学习行动。通过观察其行动的结果，以状态转移和奖励的形式，智能体应选择能最大化其长期奖励积累的行动。解决这类存在模型不确定性的问题是*reinforcement learning*领域的主题，这也是本书这一部分的重点。我们将讨论应对模型不确定性中的几个挑战。首先，智能体必须谨慎平衡对环境的探索与对通过经验获得的知识的利用。其次，奖励可能在重要决策做出很久之后才收到，因此后期奖励的功劳必须归因于早期的决策。第三，智能体必须从有限的经验中进行泛化。我们将回顾应对这些挑战的理论和一些关键算法。

15 Exploration and Exploitation

强化学习智能体[1]必须平衡环境的*exploration*与*exploitation*通过其交互获得的知识。[2] 纯粹探索将允许智能体构建一个全面的模型，但智能体很可能不得不牺牲奖励的收集。纯粹的利用让智能体持续选择它认为最能积累奖励的动作，但可能存在其他更好的动作可供采取。本章通过聚焦于一个单一状态的问题，介绍了与探索-利用权衡相关的挑战。最后，我们介绍了多状态MDPs中的探索。

1 强化学习领域的综述见M. Wiering和M. van Otterlo编，*Reinforcement Learning: State of the Art*。Springer，2012年。

2 在某些应用中，我们希望基于一组固定的轨迹来优化策略。这种场景称为*batch reinforcement learning*。本章假设我们必须通过交互来收集自己的数据，这使得选择合适的探索策略变得尤为重要。

15.1 Bandit Problems

对探索-利用权衡的早期分析主要聚焦于老虎机，也称为*one-armed bandits*。[3]这一名称来源于较早的老虎机有一个拉杆，以及机器倾向于拿走赌徒的钱。许多现实世界的问题可以被框定为*multiarmed bandit problems*,[4]，例如临床试验的分配和自适应网络路由。

文献中存在许多赌博机问题的表述，但本章将聚焦于所谓的*binary bandit*、*Bernoulli bandit*或*binomial bandit*。在这些问题中，臂a以概率θ_a支付1，否则支付0。拉动臂不产生任何成本，但我们仅有h次拉动机会。

一个多臂老虎机问题可以被构建为一个h步的MDP，具有单一状态、n个动作，以及一个未知的随机奖励函数$R(s、a)$，如图15.1所示。回想一下，$R(s、a)$是在s中采取动作a时的期望奖励，但环境中实现的个别奖励可能来自一个概率分布。

3 这些赌博机问题在二战期间得到了探索，并被证明异常难以解决。根据彼得·惠特尔的说法，“解决[赌博机问题]的努力极大地消耗了盟军分析人员的精力和心智，以至于有人提议将这个问题空投到德国，作为智力破坏的终极工具。” J. C. Gittins，“赌博机过程与动态分配指数”，*Journal of the Royal Statistical Society. Series B (Methodological)*，第41卷，第2期，第148–177页，1979年。

4 C. Szepesvári 和 T. Lattimore, *Bandit Algorithms*. 剑桥大学出版社，2020.

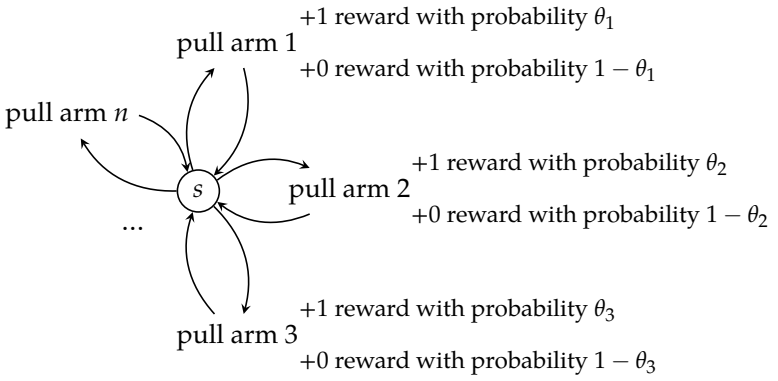


图15.1。多臂老虎机问题是一个单状态MDP，其中动作的不同仅在于它们产生奖励的可能性。

算法15.1定义了赌博机问题的模拟循环。在每一步，我们根据当前对收益概率的模型评估探索策略π，以生成动作a。下一节将讨论一种对收益概率建模的方法，本章其余部分将概述几种探索策略。在获得a之后，我们模拟拉动该臂，返回二元奖励r。然后使用观测到的a和r更新模型。该模拟循环重复至时间范围h。

```
struct BanditProblem
    θ # vector of payoff probabilities
    R # reward sampler
end

function BanditProblem(θ)
    R(a) = rand() < θ[a] ? 1 : 0
    return BanditProblem(θ, R)
end

function simulate(𝒫::BanditProblem, model, π, h)
    for i in 1:h
        a = π(model)
        r = 𝒫.R(a)
        update!(model, a, r)
    end
end
```

算法15.1. 赌博机问题的模拟。赌博机问题由一个收益概率向量`θ`定义，每个动作对应一个概率。我们还定义了一个函数`R`，用于模拟根据动作选择生成随机二元奖励的过程。模拟的每一步涉及从探索策略`π`生成一个动作`a`。探索策略在选择动作时通常会参考模型。该动作的选择产生一个随机生成的奖励，随后用于更新模型。模拟运行至时间范围`h`。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

15.2 Bayesian Model Estimation

我们希望跟踪我们对臂 a 的获胜概率 θ_a 的信念。贝塔分布（第 4.2 节）常用于表示这种信念。假设先验为均匀分布 Beta(1, 1)，则在 w_a 次获胜和 ℓ_a 次失败后，θ_a 的后验分布为 Beta($w_a + 1$, $\ell_a + 1$)。获胜的后验概率为

$$\rho_a = P(\text{win}_a \mid w_a, \ell_a) = \int_0^1 \theta \times \text{Beta}(\theta \mid w_a + 1, \ell_a + 1)\, \mathrm{d}\theta = \frac{w_a + 1}{w_a + \ell_a + 2} \tag{15.1}$$

算法15.2提供了此实现。例15.1说明了如何根据胜负次数计算这些后验分布。

```
struct BanditModel
    B # vector of beta distributions
end

function update!(model::BanditModel, a, r)
    α, β = StatsBase.params(model.B[a])
    model.B[a] = Beta(α + r, β + (1-r))
    return model
end
```

算法15.2. 对于 bandit 模型的贝叶斯更新函数。在采取动作 `a` 后观察到奖励 `r` 之后，我们通过递增相应的参数来更新与该动作相关的 beta 分布。

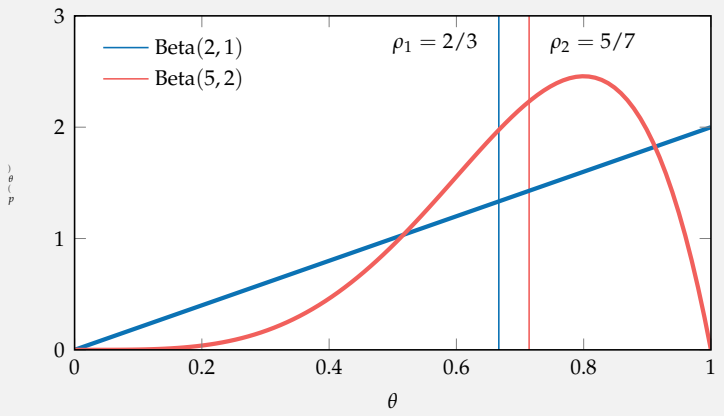
一个*greedy action*是最大化我们预期即时奖励的动作——换言之，在我们的二元老虎机问题背景下，即赢得比赛的后验概率最大的动作。可能存在多个贪婪动作。我们并不总是希望选择贪婪动作，因为可能会错过发现实际上可能提供更高预期奖励的另一个动作。我们可以利用与不同动作相关的贝塔分布中的信息来驱动我们对非贪婪动作的探索。

15.3 Undirected Exploration Strategies

有几种常用的*ad hoc exploration*策略来平衡探索与利用。本节讨论一种称为*undirected exploration*的特殊探索方式，在这种方式中，我们不利用先前结果的信息来引导对非贪婪动作的探索。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

假设我们有一个双臂老虎机，我们已经玩了六次。第一个臂有1次胜利和0次失败，另一个臂有4次胜利和1次失败。假设先验是均匀的，θ_1的后验分布是Beta(2, 1)，而θ_2的后验分布是Beta(5, 2)。



这些后验概率为0到1之间的获胜概率分配了非零的可能性。由于两个臂都至少获得了一次胜利，因此在0处的密度对于两个臂都是0。同样地，臂2在1处的密度为0，因为它至少遭受了一次失败。支付概率$\rho_1 = 2/3$和$\rho_2 = 5/7$以垂直线表示。我们认为第二个臂最有可能产生收益。

例15.1. 多臂老虎机中的后验概率分布与期望收益。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

最常见的无向探索策略之一是ϵ-*greedy explo- ration* (算法15.3)。该策略以概率ϵ选择一个随机臂。否则，我们选择一个贪婪臂，即$\arg\max_a \rho_a$。这个ρ_a是根据上一节给出的贝叶斯模型，采取动作a获胜的后验概率。或者，我们可以使用最大似然估计，但在足够的拉动下，两种方法之间的差异很小。ϵ值越大，探索越多，从而更快地识别最佳臂，但更多的拉动浪费在次优臂上。例15.2展示了这种探索策略以及我们信念的演变。

ϵ-贪心方法在探索量上保持恒定，尽管在与老虎机的交互初期，不确定性远高于后期。一个常见的调整是随时间衰减ϵ，例如采用如下的指数衰减计划进行更新：

$$\epsilon \leftarrow \alpha\epsilon \tag{15.2}$$

对于$\alpha \in (0, 1)$通常接近1。

```
mutable struct EpsilonGreedyExploration
    ϵ # probability of random arm
end

function (π::EpsilonGreedyExploration)(model::BanditModel)
    if rand() < π.ϵ
        return rand(eachindex(model.B))
    else
        return argmax(mean.(model.B))
    end
end
```

算法15.3. ϵ-贪婪探索策略。以概率ϵ，它将返回一个随机动作。否则，它将返回一个贪婪动作。

另一种策略是*explore-then-commit exploration* (算法15.4)，其中在前k个时间步我们均匀随机地选择动作。从那时起，我们选择一个贪心动作。[5]较大的k值降低了承诺于
一个次优动作，我们浪费了更多时间去探索潜在的次优动作。

5 A. Garivier、T. Lattimore 和 E. Kaufmann，“关于先探索后承诺策略”，载于 *Advances in Neural Information Processing Systems* (*NIPS*)，2016年。

15.4 Directed Exploration Strategies

Directed exploration 利用之前拉取所收集的信息来指导对非贪婪动作的探索。例如，*softmax strategy* (算法15.5)

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们希望将 ϵ-贪心探索策略应用于两臂老虎机。我们可以用均匀先验构建模型，并使用 $\epsilon = 0.3$ 构建探索策略：

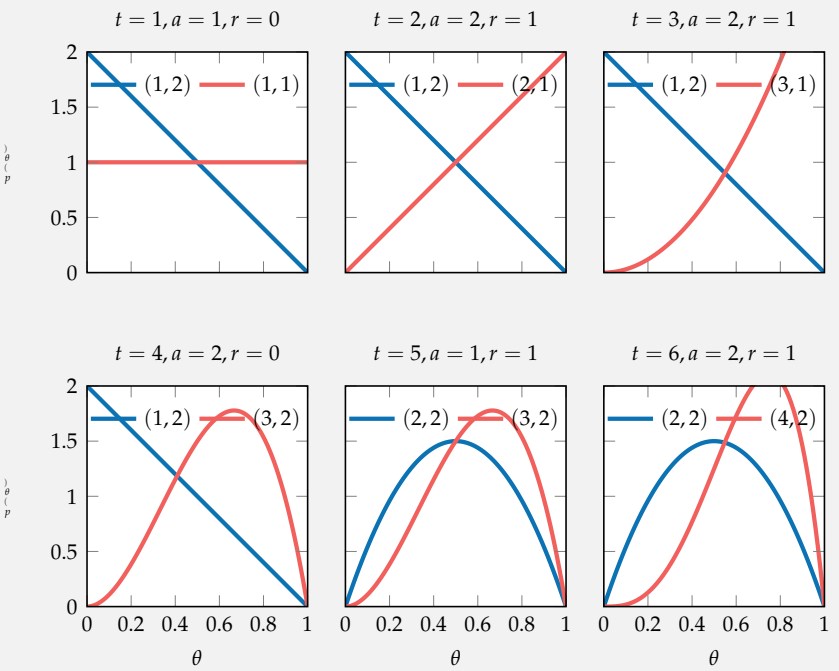
```
model(fill(Beta(),2))
π = EpsilonGreedyExploration(0.3)
```

为了获得我们的第一个动作，我们调用 `π(model)`，它根据随机数生成器的当前状态返回 1。我们观察到损失，$r =$ 为 0，然后调用

```
update!(model, 1, 0)
```

这会更新模型中的贝塔分布，以反映我们采取了行动1并获得了奖励0的情况。

此处的图表展示了使用我们的探索策略执行六步后，每次步骤中收益信念的演变情况。蓝色代表第一个臂，红色代表第二个臂：



示例15.2。ϵ-贪心探索策略在双摇臂老虎机问题中的应用。

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

```
mutable struct ExploreThenCommitExploration
    k # pulls remaining until commitment
end

function (π::ExploreThenCommitExploration)(model::BanditModel)
    if π.k > 0
        π.k -= 1
        return rand(eachindex(model.B))
    end
    return argmax(mean.(model.B))
end
```

算法15.4。先探索后承诺的探索策略。如果k严格为正，则在递减k后返回一个随机动作。否则，它将返回一个贪婪动作。

以与exp（$\lambda\rho_a$）成比例的概率拉动臂a，其中*precision parame- ter* $\lambda \geq 0$控制探索的程度。当$\lambda \to 0$时，我们进行均匀随机选择；而当$\lambda \to \infty$时，则采取贪婪选择。随着数据积累增多，我们可能希望通过乘以一个因子来增加λ，以减少探索。

```
mutable struct SoftmaxExploration
    λ # precision parameter
    α # precision factor
end

function (π::SoftmaxExploration)(model::BanditModel)
    weights = exp.(π.λ * mean.(model.B))
    π.λ *= π.α
    return rand(Categorical(normalize(weights, 1)))
end
```

算法15.5. 柔性最大探索策略。它以与exp($\lambda\rho_a$)成比例的概率选择动作a。精度参数λ在每一步乘以因子α进行缩放。

多种探索策略都基于*optimism under uncertainty*的思想。如果我们根据数据在统计上允许的程度，对行动结果持乐观态度，我们就会被隐式地驱动去平衡探索与利用。其中一种方法是*quantile exploration*（算法15.6），[6]

其中我们选择具有最高 α-*quantile* (第2.2.2节) 的臂以获取支付概率。$\alpha >$ 为0.5的值导致在不确定性下的乐观态度，激励探索那些尚未被频繁尝试的行动。α 的值越大，探索越多。例15.3展示了分位数估计，并将其与其他探索策略进行了比较。

另一种精确计算后验分布置信上限的替代方法是使用*UCB1 exploration*（算法15.7），该算法最初在9.6节中引入，用于蒙特卡洛树搜索中的探索。在此策略中，

6 这种通用策略与*upper confidence bound exploration*、*interval exploration*和*interval estimation*相关，指的是置信区间的上界。L. P. Kaelbling, *Learning in Embedded Systems*, MIT Press, 1993年。另见E. Kaufmann, "On Bayesian Index Policies for Sequential Resource Allocation," *Annals of Statis-*, 第46卷，第2期，第842–865页，2018年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
mutable struct QuantileExploration
    α # quantile (e.g., 0.95)
end

function (π::QuantileExploration)(model::BanditModel)
    return argmax([quantile(B, π.α) for B in model.B])
end
```

算法15.6。分位数探索，返回具有最高α分位数的动作。

我们选择使

$$\rho_a + c\sqrt{\frac{\log N}{N(a)}} \tag{15.3}$$

其中$N(a)$是我们采取行动a的次数，且$N = \sum_a N(a)$。参数$c \geq 0$控制通过第二项鼓励的探索量。较大的c值会导致更多的探索。这种策略通常与收益概率的最大似然估计一起使用，但我们可以通过让$N(a)$成为与a相关的beta分布参数之和，将其适应于贝叶斯上下文。

另一种通用的探索方法是使用 *posterior sampling*（算法 15.8），也称为 *randomized probability matching* 或 *Thompson sam- pling*。[7] 它易于实现，且不需要精细的参数调优。

其思想是从与各种动作相关的奖励的后验分布中进行采样。选择采样值最大的动作。

15.5 Optimal Exploration Strategies

与臂 a 相关的贝塔分布由计数 (w_a, ℓ_a) 参数化。这些计数 $w_1, \ell_1, \ldots, w_n, \ell_n$ 共同表示我们对收益的信念，因此表示一个 *belief state*。这 $2n$ 个数字可以描述 n 个关于可能收益概率的连续概率分布。

我们可以构建一个MDP，其状态为长度$2n$的向量，表示代理对n-臂老虎机问题的信念。动态规划可用于求解此MDP，以获得最优策略π^*，该策略规定了给定计数时应拉哪个臂。

7 W. R. Thompson，“关于根据两个样本的证据，一个未知概率超过另一个的可能性”，*Biometrika*，第 25 卷，第 3/4 期，第 285–294 页，1933 年。关于近期的教程，参见 D. Russo、B. V. Roy、A. Kazero uni、I. Osband 和 Z. Wen，“Thom pson 采样教程”，*Foundations and Trends in Machine Learning*，第 11 卷，第 1 期，第 1–96 页，2018 年。

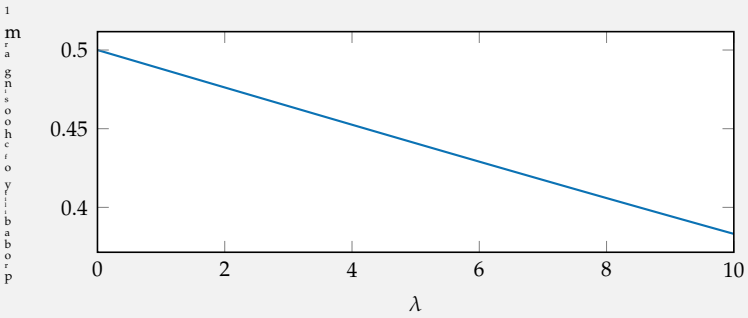
© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

示例 15.3. 用于示例 15.1 中双臂老虎机问题的探索策略。

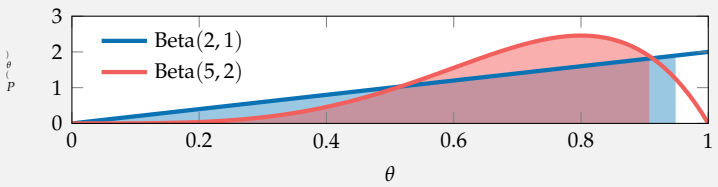
考虑在例15.1的双臂老虎机问题中，利用所获得的信息采用探索策略，其中θ_1的后验分布为Beta(2, 1)，而θ_2的后验分布为Beta(5, 2)。第二个臂具有更高的收益概率。

具有$\epsilon = 0.2$的ϵ-贪心策略有20%的概率在臂之间随机选择，有80%的概率选择第二个臂。因此，选择第一个臂的总体概率为0.1，选择第二个臂的概率为0.9。

具有 $\lambda = 1$ 的 softmax 策略为第一个臂分配权重 $\exp(\rho_1) = \exp(2/3) \approx 1.948$，为第二个臂分配权重 $\exp(\rho_2) = \exp(5/7) \approx 2.043$。选择第一个臂的概率为 $1.948/(1.948 + 2.043) \approx 0.488$，选择第二个臂的概率为 0.512。此处的图展示了选择第一个臂的概率如何随 λ 变化：



使用 $\alpha = 0.9$ 进行分位数探索，计算每个后验分布中大于 90% 概率质量的支付概率。θ_1 的 0.9 分位数为 0.949，θ_2 的为 0.907，如下所示。第一个臂（蓝色）具有更高的分位数，将下一个被拉动。



© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
mutable struct UCB1Exploration
    c # exploration constant
end

function bonus(π::UCB1Exploration, B, a)
    N = sum(b.α + b.β for b in B)
    Na = B[a].α + B[a].β
    return π.c * sqrt(log(N)/Na)
end

function (π::UCB1Exploration)(model::BanditModel)
    B = model.B
    ρ = mean.(B)
    u = ρ .+ [bonus(π, B, a) for a in eachindex(B)]
    return argmax(u)
end
```

算法15.7. 探索常数为c的UCB1探索策略。我们根据B中的伪计数参数计算每个动作的方程(15.3)。然后我们返回使该量最大化的动作。

```
struct PosteriorSamplingExploration end

(π::PosteriorSamplingExploration)(model::BanditModel) =
    argmax(rand.(model.B))
```

算法15.8。后验采样探索策略。它没有自由参数。它简单地从事先与每个动作关联的beta分布中采样，然后返回与最大样本关联的动作。

令 $Q^*(w_1、\ell_1、\ldots、w_n、\ell_n、a)$ 表示拉动摇臂 a 并在此后采取最优行动后的期望收益。最优效用函数和最优策略可以用 Q^* 表示为：

$$U^*(w_1, \ell_1, \ldots, w_n, \ell_n) = \max_a Q^*(w_1, \ell_1, \ldots, w_n, \ell_n, a) \tag{15.4}$$

$$\pi^*(w_1, \ell_1, \ldots, w_n, \ell_n) = \arg\max_a Q^*(w_1, \ell_1, \ldots, w_n, \ell_n, a) \tag{15.5}$$

我们可以将 Q^* 分解为两项：

$$\begin{aligned} Q^*(w_1, \ell_1, \ldots, w_n, \ell_n, a) = &\frac{w_a + 1}{w_a + \ell_a + 2}(1 + U^*(\ldots, w_a + 1, \ell_a, \ldots)) \\ &+ \left(1 - \frac{w_a + 1}{w_a + \ell_a + 2}\right) U^*(\ldots, w_a, \ell_a + 1, \ldots) \end{aligned} \tag{15.6}$$

第一项与臂a的获胜相关联，第二项与失败相关联。值$(w_a + 1)/(w_a + \ell_a + 2)$是获胜的后验概率，来自方程(15.1)。[8]方程(15.6)中的第一个U^*记录一场胜利，而第二个U^*记录了一场失败。

8 如果我们有非均匀先验，这个概率可以调整。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们可以为整个信念空间计算 Q^*，因为我们假设了一个有限时域 h。我们从所有终端信念状态开始，其中 $\sum_a (w_a + \ell_a) = h$，$U^* = 0$。然后我们可以反向推导到 $\sum_a (w_a + \ell_a) = h - 1$ 的状态，并应用方程（15.6）。重复这个过程，直到达到我们的初始状态。这样的最优策略在例15.4中计算。

虽然这个动态规划解决方案是最优的，但信念状态的数量为$O(h^{2n})$。我们可以将该问题表述为一个无限视界、折扣版本，使用*Gittins allocation index*，[9]能够高效求解，这

可以存储为一个查找表，该表根据与臂相关的拉动次数和胜利次数指定标量分配索引值。[10] 臂

具有最高分配索引的那一个应当是接下来要拉取的。

9 J. C. Gittins, "Bandit Processes and Dynamic Allocation Indices," *Journal of the Royal Statistical Society. Series B (Methodological)*, vol. 41, no. 2, pp. 148–177, 1979. J. Gittins, K. Glazebrook, and R. Weber, *Multi-Armed Bandit Allocation Indices*, 2nd ed. Wiley, 2011.

10 关于计算该查找表的算法综述见 J. Chakravorty 和 A. Mahajan，"多臂老虎机、Gittins 指数及其计算"，载于 *Methods and Applications of Statistics in Clinical Trials*，N. Balakrishnan 编，第2卷，Wiley，2014年，第416–435页。

15.6 Exploration with Multiple States

在具有多个状态的一般强化学习环境中，我们必须利用关于状态转移的观察来指导决策。我们可以修改算法15.1中的模拟过程，以适应状态转移并适当地更新我们的模型。算法15.9为此提供了一个实现。正如我们将在接下来的几章中讨论的，有许多方法可以对问题进行建模和进行探索，但模拟结构是完全相同的。

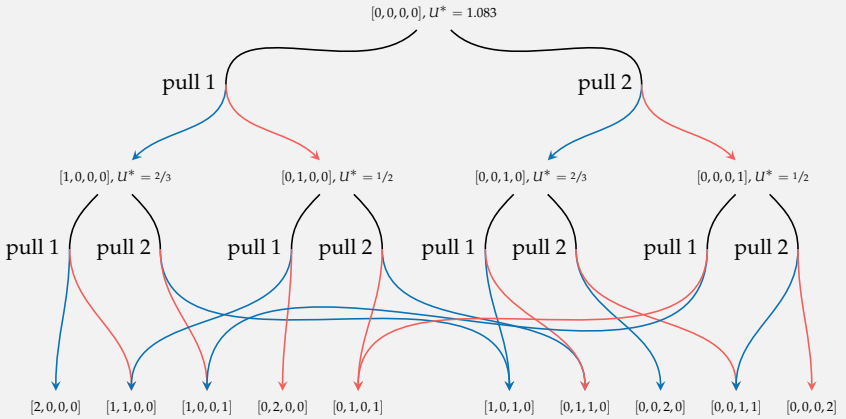
15.7 Summary

- 探索-利用权衡是在探索状态-动作空间以获取更高奖励与利用已知的有利状态动作之间的一种平衡。
- 多臂老虎机问题涉及一个单一状态，其中智能体因采取不同动作而获得随机奖励。
- 贝塔分布可用于维护对多臂老虎机奖励的信念。
- 无向探索策略，包括 ϵ-贪心算法和探索后承诺策略，实现起来简单，但它们不利用先前结果的信息来指导对非贪婪动作的探索。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

示例 15.4. 计算两臂、两步视界老虎机问题的最优策略。

接下来，我们构建了一个两臂老虎机问题的状态-行动树，具有两步决策周期。状态向量表示为 $[w_1, \ell_1, w_2, \ell_2]$；蓝色箭头表示胜利，红色箭头表示失败。



不出所料，策略对于臂1和臂2是对称的。我们发现第一臂无关紧要，最佳做法是连续两次拉动获胜的臂，而不要连续两次拉动失败的臂。

使用以下方法计算最优值函数：

$$
\begin{aligned}
Q^*([1,0,0,0],1) &= \frac{2}{3}(1+0) + \frac{1}{3}(0) = 2/3 \\
Q^*([1,0,0,0],2) &= \frac{1}{2}(1+0) + \frac{1}{2}(0) = 1/2 \\
Q^*([0,1,0,0],1) &= \frac{1}{3}(1+0) + \frac{2}{3}(0) = 1/3 \\
Q^*([0,1,0,0],2) &= \frac{1}{2}(1+0) + \frac{1}{2}(0) = 1/2 \\
Q^*([0,0,0,0],1) &= \frac{1}{2}(1+2/3) + \frac{1}{2}(1/2) = 1.083
\end{aligned}
$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
function simulate(𝒫::MDP, model, π, h, s)
    for i in 1:h
        a = π(model, s)
        s′, r = 𝒫.TR(s, a)
        update!(model, s, a, r, s′)
        s = s′
    end
end
```

算法 15.9. 强化学习问题的模拟循环。探索策略 π 根据模型中的信息和当前状态 s 生成下一个动作。MDP 问题 $\mathcal{P}$ 被视为真实情况，并用于采样下一个状态和奖励。状态转移和奖励用于更新模型。模拟运行至时间范围 h。

- 定向探索策略，包括softmax、分位数、UCB1和后验采样探索，利用过去动作的信息来更好地探索有前景的动作。

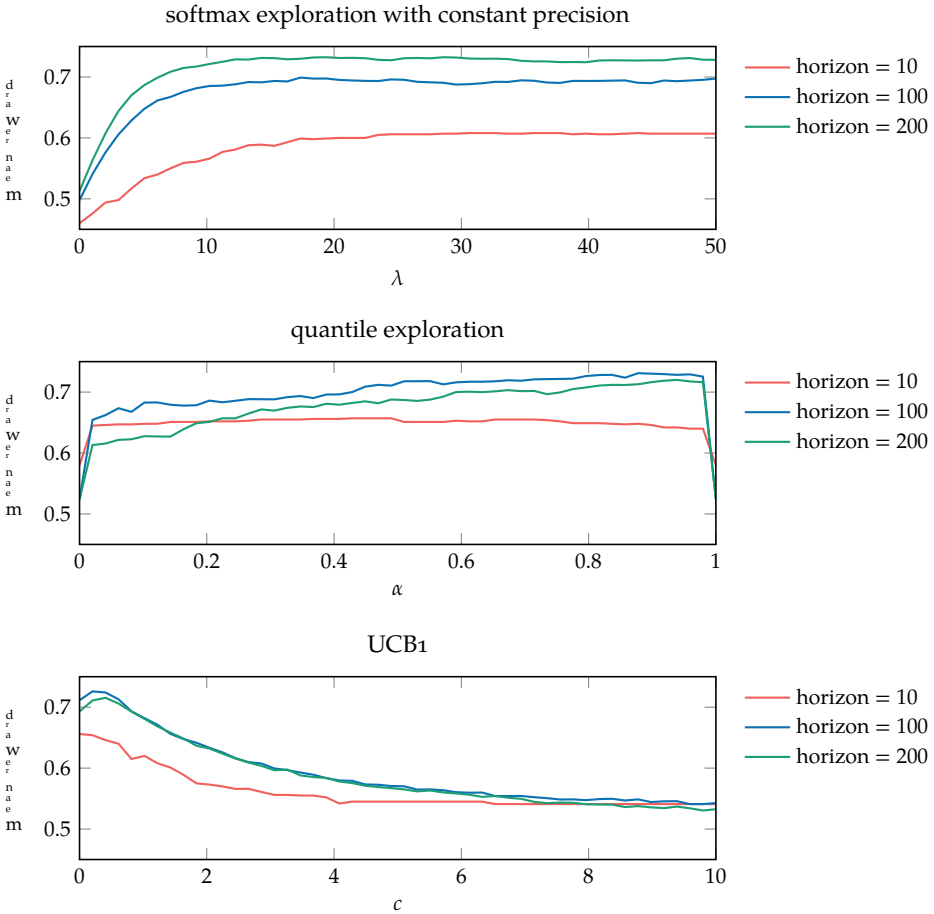
- 动态规划可用于推导有限时域下的最优探索策略，但这些策略的计算成本可能很高。

15.8 Exercises

练习15.1。再次考虑三臂老虎机问题，其中每个臂的获胜概率在0到1之间均匀抽取。比较softmax、分位数和UCB1探索策略。定性而言，对于随机生成的老虎机问题，λ、α和c取什么值能产生最高的期望奖励?

Solution: 这里我们绘制了三种策略各自每步的预期奖励。同样，参数化的有效性取决于问题的时间范围，因此也展示了几个不同的深度。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com



Softmax 策略在 λ 取较大值时表现最佳，因为此时优先拉取根据当前信念具有更高预期奖励的臂。分位数探索在更长的时间范围内表现更好，且与其参数化无关。置信界 α 的大小对性能没有显著影响，除非其值非常接近 0 或 1。UCB1 策略在探索标量 c 取较小正值时表现最佳。预期奖励随着 c 的增大而衰减。这三种策略都可以通过调参产生相似的最大预期奖励。

练习15.2。举一个多臂老虎机问题的实际应用例子。

Solution: 有许多多臂老虎机问题。例如，考虑一家新闻公司希望在其网站上的文章上最大化互动（点击）。该

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

公司可能有几篇文章需要展示，但在任何给定时间必须选择一篇文章进行展示。这个问题是一个多臂老虎机问题，因为用户要么以概率θ_i点击文章i，要么以概率$1-\theta_i$不点击。探索将包括在网站上展示文章并观察点击次数，而利用将包括展示可能导致最高点击次数的文章。这个问题与*A/B testing*相关，即公司测试网站的不同版本，以确定哪个版本产生最多的互动。

练习15.3。给定一个单臂老虎机，其先验为$\theta \sim \text{Beta}(7, 2)$，请给出在额外进行10次拉动后获胜的后验概率的界限。

Solution: 在假设所有抽取均导致失败（例如$\underline{\ell} =$ 为10且$\underline{w} =$ 为0）的情况下，可以计算出我们获胜后验概率$\underline{\rho}$的一个下界。同样地，我们可以计算出一个上界$\overline{\rho}$，假设所有抽取均导致胜利（例如$\overline{w} =$ 为10且$\ell =$ 为0）。因此，这些界为

$$\underline{\rho} = \frac{\underline{w}+7}{\underline{w}+\underline{\ell}+9} = \frac{0+7}{0+10+9} = \frac{7}{19}$$
$$\overline{\rho} = \frac{\overline{w}+7}{\overline{w}+\overline{\ell}+9} = \frac{10+7}{10+0+9} = \frac{17}{19}$$

练习 15.4。假设我们有一个包含臂 a 和 b 的 bandit，并且我们使用 ϵ-贪心探索策略，其中 $\epsilon =$ 为 0.3，探索衰减因子 $\alpha =$ 为 0.9。我们生成一个介于 0 和 1 之间的随机数 x 来确定我们是探索（$x < \epsilon$）还是利用（$x > \epsilon$）。假设我们有 $\rho_a > \rho_b$，如果在第一次迭代中 $x =$ 为 0.2914，则选择哪个臂？如果在第九次迭代中 $x =$ 为 0.1773，则选择哪个臂?

Solution: 由于在第一次迭代中 $x < \epsilon_1$，我们以概率 0.5 探索并选择 a，并以概率 0.5 选择 b。在第九次迭代时，$\epsilon_9 = \alpha^8\epsilon_1 \approx 0.129$。由于 $x > \epsilon_9$，我们利用并选择 a。

练习 15.5。我们有一个四臂老虎机，并希望使用精度参数 $\lambda = 2$ 和每个臂 a 的先验信念 $\theta_a \sim \text{Beta}(2, 2)$ 的 softmax 探索策略。假设我们拉动每个臂四次，结果是臂 1、2、3 和 4 分别支付 1、2、3 和 4 次。列出关于 θ_a 的后验分布，并计算我们选择臂 2 的概率。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: 每只臂的后验分布分别为：Beta(3, 5)、Beta(4, 4)、Beta(5, 3) 和 Beta(6, 2)。选择第2只臂的概率可以通过以下步骤计算：

$$
\begin{aligned}
P(a = i) &\propto \exp(\lambda \rho_i) \\
P(a = i) &= \frac{\exp(\lambda \rho_i)}{\sum_a \exp(\lambda \rho_a)} \\
P(a = 2) &= \frac{\exp(2 \times \frac{4}{8})}{\exp(2 \times \frac{3}{8}) + \exp(2 \times \frac{4}{8}) + \exp(2 \times \frac{5}{8}) + \exp(2 \times \frac{6}{8})} \\
P(a = 2) &\approx 0.2122
\end{aligned}
$$

练习15.6。针对任意的Beta(α, β)先验，重写方程(15.6)。

Solution: 我们可以将方程更一般地重写为：

$$
\begin{aligned}
Q^*(w_1, \ell_1, \ldots, w_n, \ell_n, a) = &\frac{w_a + \alpha}{w_a + \ell_a + \alpha + \beta}(1 + U^*(\ldots, w_a + 1, \ell_a, \ldots)) \\
&+ \left(1 - \frac{w_a + \alpha}{w_a + \ell_a + \alpha + \beta}\right) U^*(\ldots, w_a, \ell_a + 1, \ldots)
\end{aligned}
$$

练习 15.7。回忆例 15.4。假设不是每个臂的收益都是 1，而是臂 1 给出收益 1，而臂 2 给出收益 2。计算两个臂的新动作价值函数。

Solution: 对于臂1，我们有

$$
\begin{aligned}
Q^*([1,0,0,0], 1) &= \frac{2}{3}(1 + 0) + \frac{1}{3}(0) = 2/3 \\
Q^*([1,0,0,0], 2) &= \frac{1}{2}(2 + 0) + \frac{1}{2}(0) = 1 \\
Q^*([0,1,0,0], 1) &= \frac{1}{3}(1 + 0) + \frac{2}{3}(0) = 1/3 \\
Q^*([0,1,0,0], 2) &= \frac{1}{2}(2 + 0) + \frac{1}{2}(0) = 1 \\
Q^*([0,0,0,0], 1) &= \frac{1}{2}(1 + 1) + \frac{1}{2}(1) = 1.5
\end{aligned}
$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

对于臂2，我们有

$$Q^*([0,0,1,0],1) = \frac{1}{2}(1+0) + \frac{1}{2}(0) = 1/2$$
$$Q^*([0,0,1,0],2) = \frac{2}{3}(2+0) + \frac{1}{3}(0) = 4/3$$
$$Q^*([0,0,0,1],1) = \frac{1}{2}(1+0) + \frac{1}{2}(0) = 1/2$$
$$Q^*([0,0,0,1],2) = \frac{1}{3}(2+0) + \frac{2}{3}(0) = 2/3$$
$$Q^*([0,0,0,0],2) = \frac{1}{2}(2+4/3) + \frac{1}{2}(2/3) = 2$$

练习15.8。证明在具有h时间范围的n臂赌博机问题中，信念状态的数量为$O(h^{2n})$。

Solution: 我们首先计算 $w_1 + \ell_1 + \cdots + w_n + \ell_n = k$ 的解的数量，其中 $0 \leq k \leq h$。若 $n = 2$ 且 $k = 6$，则一个解为 $2 + 0 + 3 + 1 = 6$。在我们的计数论证中，我们将使用计数符号来表示整数。例如，我们可以将解写成 $2 + 0 + 3 + 1 =$ `||+++|||+|` $= 6$。对于 n 和 k 的一般取值，我们将有 k 个计数符号和 $2n - 1$ 个加号。给定如此多的计数符号和加号，我们可以按任意顺序排列它们。我们可以将一个解表示为由 $k + 2n - 1$ 个字符组成的字符串，其中字符要么是 `|`，要么是 `+`，且其中有 k 个字符为 `|`。为了得到解的数量，我们计算从 $k + 2n - 1$ 个位置中为 `|` 选择 k 个位置的方式数，结果为

$$\frac{(k+2n-1)!}{(2n-1)!k!} = O(h^{2n-1})$$

解。信念状态的数量是该表达式对 k 从 0 到 h 的求和，即 $O(h \times h^{2n-1}) = O(h^{2n})$。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

16 Model-Based Methods

本章讨论了通过与环境的交互来学习潜在动力学和奖励的最大似然方法和贝叶斯方法。最大似然方法涉及计数状态转移并记录所获得的奖励量，以估计模型参数。我们将讨论几种使用持续更新的模型进行规划的方法。即使我们精确地解决了估计问题，我们通常也必须依赖启发式探索策略来得到合适的解。贝叶斯方法涉及计算模型参数上的后验分布。求解最优探索策略通常是难解的，但我们往往可以通过后验采样获得一个合理的近似。

16.1 Maximum Likelihood Models

如第15.6节所述并在算法15.9中实现，强化学习涉及使用关于过去状态转移和奖励的信息来指导决策。本节描述如何获取潜在问题的*maximum likelihood estimate*。这种最大似然估计可用于生成价值函数估计，进而可与探索策略结合以生成行动。

我们记录转移计数 $N(s、a、s')$，表示在采取行动 a 时，从 s 到 s' 的转移被观测到的次数。给定这些转移计数，转移函数的最大似然估计为

$$T(s' \mid s, a) \approx N(s, a, s') / N(s, a) \tag{16.1}$$

其中 $N(s、a) = \sum_{s'} N(s、a、s')$。如果 $N(s、a) =$ 为 0，则我们没有可用于估计转移的信息。在这种情况下，我们可以默认采用均匀

分布，或者通过将转移概率设为零来假定该转移是不可能的。

奖励函数也可以被估计。当我们收到奖励时，我们更新$\rho(s、a)$，即在状态s采取动作a时获得的所有奖励之和。奖励函数的最大似然估计是平均奖励：

$$R(s,a) \approx \rho(s,a)/N(s,a) \tag{16.2}$$

如果 $N(s、a) =$ 为 0，那么我们对 $R(s、a)$ 的估计就是 0。如果我们对转移概率或奖励有先验知识，那么我们可以将 $N(s、a、s')$ 以及 $\rho(s、a)$ 初始化为非 0 的值。

算法16.1在观察到采取动作a并收到奖励r后从s转移到s'之后，更新了N和ρ。算法16.2将最大似然模型转换为MDP表示。例16.1说明了这一过程。我们可以利用这个最大似然模型在与环境互动并改进模型的同时选择行动。

16.2 Update Schemes

随着我们更新模型的最大似然估计，我们也需要更新我们的计划。本节讨论了几种更新方案，以应对不断变化的模型。一个主要考虑因素是计算效率，因为在与环境交互时，我们希望相当频繁地执行这些更新。

16.2.1 Full Updates

算法16.3使用第7.7节的线性规划公式来解决最大似然模型，尽管我们本可以使用值迭代或其他算法。每一步之后，我们获得一个新的模型估计并重新求解。

16.2.2 Randomized Updates

在每个状态转换时重新计算最优策略通常是计算成本高昂的。另一种方法是在先前访问过的状态以及一些随机选择的状态上对估计模型进行贝尔曼更新。[1]

算法16.4实现了这种方法。

1 该方法与R. S. Sutton在“Dyna，一种学习、规划与反应的集成架构，”*SIGART Bulletin*，第2卷，第4期，第160-163页，1991年提出的*Dyna*方法相关。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
mutable struct MaximumLikelihoodMDP
    𝒮 # state space (assumes 1:nstates)
    𝒜 # action space (assumes 1:nactions)
    N # transition count N(s,a,s′)
    ρ # reward sum ρ(s, a)
    γ # discount
    U # value function
    planner
end

function lookahead(model::MaximumLikelihoodMDP, s, a)
    𝒮, U, γ = model.𝒮, model.U, model.γ
    n = sum(model.N[s,a,:])
    if n == 0
        return 0.0
    end
    r = model.ρ[s, a] / n
    T(s,a,s′) = model.N[s,a,s′] / n
    return r + γ * sum(T(s,a,s′)*U[s′] for s′ in 𝒮)
end

function backup(model::MaximumLikelihoodMDP, U, s)
    return maximum(lookahead(model, s, a) for a in model.𝒜)
end

function update!(model::MaximumLikelihoodMDP, s, a, r, s′)
    model.N[s,a,s′] += 1
    model.ρ[s,a] += r
    update!(model.planner, model, s, a, r, s′)
    return model
end
```

算法16.1. 一种用于更新离散状态和动作空间下最大似然强化学习的转移和奖励模型的方法。在观察到采取动作`a`后从`s`转移到`s'`的情况后，我们将`N[s,a,s']`递增，并将`r`加到`ρ[s,a]`。该模型还包含价值函数`U`的估计以及一个规划器。此算法块还包括针对该模型执行备份和前瞻的方法。

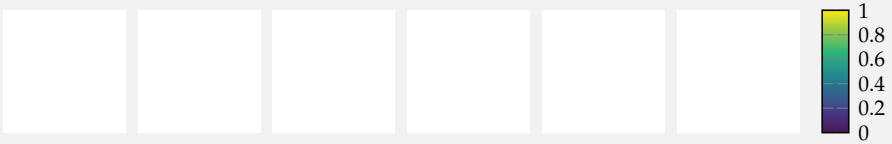
```
function MDP(model::MaximumLikelihoodMDP)
    N, ρ, 𝒮, 𝒜, γ = model.N, model.ρ, model.𝒮, model.𝒜, model.γ
    T, R = similar(N), similar(ρ)
    for s in 𝒮
        for a in 𝒜
            n = sum(N[s,a,:])
            if n == 0
                T[s,a,:] .= 0.0
                R[s,a] = 0.0
            else
                T[s,a,:] = N[s,a,:] / n
                R[s,a] = ρ[s,a] / n
            end
        end
    end
    return MDP(T, R, γ)
end
```

算法16.2. 一种将最大似然模型转换为`MDP`问题的方法。

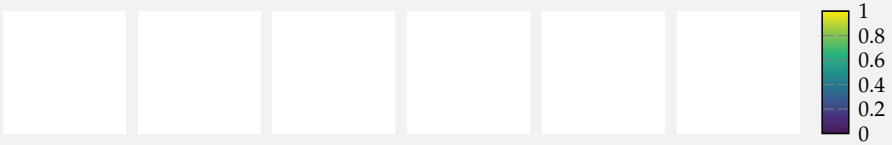
© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

例16.1. 将最大似然估计应用于六边形世界问题。

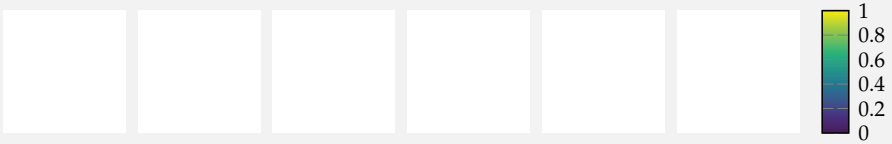
我们希望将最大似然模型估计应用于六边形世界问题。真实的转移矩阵如下所示：



有六个转移矩阵，每个动作对应一个。行对应当前状态，列对应下一个状态。共有26个状态。图像中的强度与进行相应转移的概率有关。在强化学习环境中，我们事先并不知道这些转移概率。然而，我们可以与环境交互并记录我们观察到的转移。在从随机初始状态进行10次模拟，每次10步之后，最大似然估计得到以下矩阵：



经过1000次模拟，我们的估计变为



© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct FullUpdate end

function update!(planner::FullUpdate, model, s, a, r, s′)
    𝒫 = MDP(model)
    U = solve(𝒫).U
    copy!(model.U, U)
    return planner
end
```

算法16.3. 一种使用第7.7节中的线性规划公式对U的值函数进行完全更新的方法。

```
struct RandomizedUpdate
    m # number of updates
end

function update!(planner::RandomizedUpdate, model, s, a, r, s′)
    U = model.U
    U[s] = backup(model, U, s)
    for i in 1:planner.m
        s = rand(model.𝒮)
        U[s] = backup(model, U, s)
    end
    return planner
end
```

算法16.4. 基于最大似然模型的强化学习，在随机状态下进行更新。此方法在先前访问过的状态以及随机选择的 m 个额外状态上执行贝尔曼更新。

16.2.3 *Prioritized Updates*

一种称为*prioritized sweeping*[2] (算法16.5)的方法使用优先队列来帮助确定哪些状态最需要更新。从s到s'的转移之后，会根据我们更新的转移和奖励模型对$U(s)$进行更新。然后，我们会遍历所有能够立即转移到s的状态-动作对（s^-，a^-）。任何此类s^-的优先级将提高到$T(s \mid s^-, a^-) \times |U(s) - u|$，其中$u$是更新前$U(s)$的值。因此，$U(s)$的变化越大，转移到$s$的可能性越高，导致$s$的状态的优先级就越高。队列中最高优先级状态的更新过程会持续固定次数的迭代，或者直到队列为空。

2 A. W. Moore 和 C. G. Atkeson, “优先扫描：用更少数据和更少时间的强化学习，” *Machine Learning*，第13卷，第1期，第103-130页，1993年。

16.3 *Exploration*

无论采用何种更新方案，通常都必须遵循某种形式的探索策略，以避免上一章提到的纯粹利用的陷阱。我们可以调整那一章中介绍的探索算法。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct PrioritizedUpdate
    m  # number of updates
    pq # priority queue
end

function update!(planner::PrioritizedUpdate, model, s)
    N, U, pq = model.N, model.U, planner.pq
    𝒮, 𝒜 = model.𝒮, model.𝒜
    u = U[s]
    U[s] = backup(model, U, s)
    for s⁻ in 𝒮
        for a⁻ in 𝒜
            n_sa = sum(N[s⁻,a⁻,s′] for s′ in 𝒮)
            if n_sa > 0
                T = N[s⁻,a⁻,s] / n_sa
                priority = T * abs(U[s] - u)
                if priority > 0
                    pq[s⁻] = max(get(pq, s⁻, 0.0), priority)
                end
            end
        end
    end
    return planner
end

function update!(planner::PrioritizedUpdate, model, s, a, r, s′)
    planner.pq[s] = Inf
    for i in 1:planner.m
        if isempty(planner.pq)
            break
        end
        update!(planner, model, dequeue!(planner.pq))
    end
    return planner
end
```

算法16.5。优先扫描算法维护一个状态优先队列`pq`，决定哪些状态需要更新。在每次更新时，我们将前一个状态的优先级设为无限。然后，对最高优先级状态的价值函数`U`执行`m`次贝尔曼更新。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

用于多状态问题。算法16.6提供了ϵ-贪婪探索策略的实现。

```
function (π::EpsilonGreedyExploration)(model, s)
    𝒜, ϵ = model.𝒜, π.ϵ
    if rand() < ϵ
        return rand(𝒜)
    end
    Q(s,a) = lookahead(model, s, a)
    return argmax(a→Q(s,a), 𝒜)
end
```

算法16.6。用于最大似然模型估计的ϵ-贪婪探索策略。它以概率ϵ选择一个随机动作；否则，它使用模型来提取贪婪动作。

上一章讨论的探索策略的一个局限是，它们不考虑从当前状态以外的状态探索动作。例如，我们可能希望采取行动，将自己带入尚未探索的状态空间区域。已经提出了几种算法来解决这个问题，这些算法也提供了在有限次交互后所得策略质量的概率界。[3]

其中一种这样的算法被称为*R-MAX* (算法16.7)。[4]它的名字来源于从将最大奖励分配给探索不足的状态-动作对。访问次数少于m的状态-动作对被认为探索不足。我们不使用奖励的最大似然估计（方程(16.2)），而是使用

$$R(s,a) = \begin{cases} r_{\max} & \text{if } N(s,a) < m \\ \rho(s,a)/N(s,a) & \text{otherwise} \end{cases} \tag{16.3}$$

其中 $r_{\max}$ 是最大可获得的奖励。

R-MAX中的转移模型也被修改，使得探索不足的状态-动作对导致状态保持不变：

$$T(s' \mid s,a) = \begin{cases} (s' = s) & \text{if } N(s,a) < m \\ N(s,a,s')/N(s,a) & \text{otherwise} \end{cases} \tag{16.4}$$

因此，未被充分探索的状态具有价值$r_{\max}/(1-\gamma)$，这为探索它们提供了激励。这种探索激励使我们无需单独的探索机制。我们只需根据由转移和奖励估计得出的价值函数，贪婪地选择行动。例16.2演示了ϵ-贪婪和R-MAX探索。

3 M. Kearns和S. Singh，“多项式时间内的近优强化学习”，*Machine Learning*，第49卷，第2/3期，第209-232页，2002年。

4 R. I. Brafman 和 M. Tennenholtz，“R-MAX——一种用于近最优强化学习的通用多项式时间算法”，*Journal of Machine Learning Research*，第3卷，第213–231页，2002年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
mutable struct RmaxMDP
    𝒮 # state space (assumes 1:nstates)
    𝒜 # action space (assumes 1:nactions)
    N # transition count N(s,a,s′)
    ρ # reward sum ρ(s, a)
    γ # discount
    U # value function
    planner
    m    # count threshold
    rmax # maximum reward
end

function lookahead(model::RmaxMDP, s, a)
    𝒮, U, γ = model.𝒮, model.U, model.γ
    n = sum(model.N[s,a,:])
    if n < model.m
        return model.rmax / (1-γ)
    end
    r = model.ρ[s, a] / n
    T(s,a,s′) = model.N[s,a,s′] / n
    return r + γ * sum(T(s,a,s′)*U[s′] for s′ in 𝒮)
end

function backup(model::RmaxMDP, U, s)
    return maximum(lookahead(model, s, a) for a in model.𝒜)
end

function update!(model::RmaxMDP, s, a, r, s′)
    model.N[s,a,s′] += 1
    model.ρ[s,a] += r
    update!(model.planner, model, s, a, r, s′)
    return model
end

function MDP(model::RmaxMDP)
    N, ρ, 𝒮, 𝒜, γ = model.N, model.ρ, model.𝒮, model.𝒜, model.γ
    T, R, m, rmax = similar(N), similar(ρ), model.m, model.rmax
    for s in 𝒮
        for a in 𝒜
            n = sum(N[s,a,:])
            if n < m
                T[s,a,:] .= 0.0
                T[s,a,s] = 1.0
                R[s,a] = rmax
            else
                T[s,a,:] = N[s,a,:] / n
                R[s,a] = ρ[s,a] / n
            end
        end
    end
    return MDP(T, R, γ)
end
```

算法16.7。R-MAX探索策略修改了最大似然估计中的转移和奖励模型。它将最大奖励`rmax`分配给任何探索不足的状态-动作对，定义为那些尝试次数少于`m`次的配对。此外，所有探索不足的状态-动作对都被模拟为转移到相同的状态。这个`RmaxMDP`可以作为算法16.1中引入的`MaximumLikelihoodMDP`的替代品。

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

示例 16.2. ϵ-贪心与R-MAX探索的演示。

我们可以将ϵ-贪婪探索应用于在与环境交互过程中构建的最大似然模型估计。接下来的代码将计数、奖励和效用初始化为零。它在每一步都对价值函数使用完全更新。为了进行探索，我们以概率0.1选择一个随机动作。最后一行在随机初始状态下对问题$\mathcal{P}$运行了100步的模拟（算法15.9）：

```
N = zeros(length(𝒮), length(𝒜), length(𝒮))
ρ = zeros(length(𝒮), length(𝒜))
U = zeros(length(𝒮))
planner = FullUpdate()
model = MaximumLikelihoodMDP(𝒮, 𝒜, N, ρ, γ, U, planner)
π = EpsilonGreedyExploration(0.1)
simulate(𝒫, model, π, 100, rand(𝒮))
```

或者，我们可以使用探索阈值为 $m = 3$ 的 R-MAX。我们可以针对 R-MAX 模型采取贪婪策略：

```
rmax = maximum(𝒫.R(s,a) for s in 𝒮, a in 𝒜)
m = 3
model = RmaxMDP(𝒮, 𝒜, N, ρ, γ, U, planner, m, rmax)
π = EpsilonGreedyExploration(0)
simulate(𝒫, model, π, 100, rand(𝒮))
```

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

16.4 Bayesian Methods

与目前讨论的最大似然方法相比，贝叶斯方法在探索与利用之间取得平衡，而无需依赖启发式探索策略。本节描述了第15.5节中涵盖的贝叶斯方法的推广。在*Bayesian reinforcement learning*中，我们为所有模型参数θ.[5]指定了一个先验分布。这些模型参数可能

包括支配即时奖励分布的参数，但本节重点讨论支配状态转移概率的参数。

该问题的结构可以用图16.1所示的动态决策网络表示，其中模型参数被明确表示。阴影节点表明状态是可观测的，而模型参数不是。我们通常假设模型参数在$\theta_{t+1} = \theta_t$下具有时不变性。然而，随着我们过渡到新的状态，我们对θ的信念随时间演变。

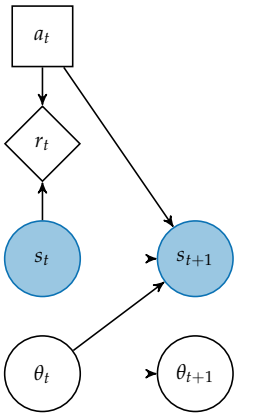


图16.1. 一个具有模型不确定性的MDP的动态决策网络。

关于转移概率的信念可以使用一组狄利克雷分布来表示，每个源状态和动作对应一个分布。每个狄利克雷分布表示给定 s 和 a 时对 s' 的分布。如果 $\theta_{(s,a)}$ 是一个 $|\mathcal{S}|$ 维向量，表示对下一状态的分布，那么先验分布由下式给出

$$\text{Dir}(\theta_{(s,a)} \mid \mathbf{N}(s,a)) \tag{16.5}$$

其中 $\mathbf{N}(s, a)$ 是与从状态 s 出发采取动作 a 的转移相关的计数向量。通常使用所有分量均设为 1 的均匀先验，但也可以利用关于转移动态的先验知识以不同方式初始化这些计数。例 16.3 展示了 Dirichlet 分布如何使用这些计数来表示可能转移概率上的分布。

θ 上的分布是狄利克雷分布乘积的结果：

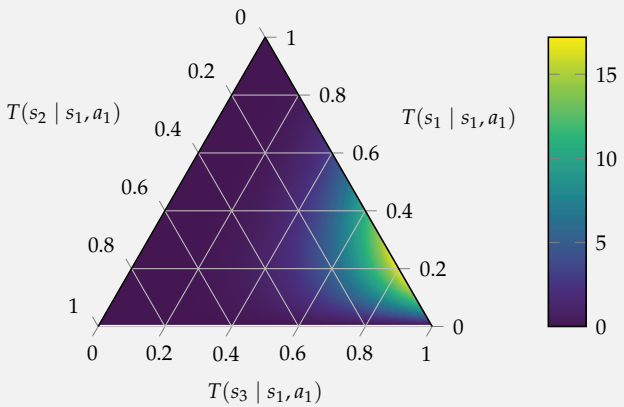
$$b(\theta) = \prod_s \prod_a \text{Dir}\left(\theta_{(s,a)} \mid \mathbf{N}(s,a)\right) \tag{16.6}$$

算法16.8提供了此类后验模型的贝叶斯更新实现。对于空间较大或连续的问题，我们可以使用其他后验表示方法。

5 关于该主题的综述由 M. Ghavamzadeh、S. Mannor、J. Pineau 和 A. Tamar 提供，“贝叶斯强化学习：综述”，*Foundations and Trends in Machine Learning*，第 8 卷，第 5–6 期，第 359–483 页，2015 年。它涵盖了将先验纳入奖励函数的方法，这些内容在此不予讨论。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

假设我们的智能体在一个具有三个状态的环境中随机探索。智能体从状态s_1采取动作a_1五次。它转移到s_3四次，并保持在s_1一次。与s_1和a_1相关的计数为$\text{N}(s_1, a_1) = [1, 0, 4]$。如果我们想假设结果状态上的均匀先验，我们会将计数加1，得到$\text{N}(s_1, a_1) = [2, 1, 5]$。从$s_1$采取动作$a_1$的转移函数是一个三值分类分布，因为存在三个可能的后继状态。每个后继状态都有一个未知的转移概率。可能的转移概率空间是三个元素向量之和为1的集合。狄利克雷分布表示这些可能的转移概率上的概率分布。下面是密度函数的图：



例16.3。在采取特定动作时，从特定状态转移概率的后验Dirichlet分布。在未知MDP中学习转移函数的智能体可能会选择在每个状态-动作对上维持这样的分布。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
mutable struct BayesianMDP
    𝒮 # state space (assumes 1:nstates)
    𝒜 # action space (assumes 1:nactions)
    D # Dirichlet distributions D[s,a]
    R # reward function as matrix (not estimated)
    γ # discount
    U # value function
    planner
end

function lookahead(model::BayesianMDP, s, a)
    𝒮, U, γ = model.𝒮, model.U, model.γ
    n = sum(model.D[s,a].alpha)
    if n == 0
        return 0.0
    end
    r = model.R(s,a)
    T(s,a,s′) = model.D[s,a].alpha[s′] / n
    return r + γ * sum(T(s,a,s′)*U[s′] for s′ in 𝒮)
end

function update!(model::BayesianMDP, s, a, r, s′)
    α = model.D[s,a].alpha
    α[s′] += 1
    model.D[s,a] = Dirichlet(α)
    update!(model.planner, model, s, a, r, s′)
    return model
end
```

算法16.8. 当过渡模型的后验分布表示为狄利克雷分布的乘积时，一种贝叶斯更新方法。在此实现中，我们假设奖励模型R是已知的，尽管我们可以使用贝叶斯方法从经验中估计期望奖励。矩阵D将狄利克雷分布与每个状态-动作对关联起来，以建模向其后继状态过渡中的不确定性。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

16.5 Bayes-Adaptive Markov Decision Processes

我们可以将未知模型的MDP中采取最优行动的问题形式化为具有已知模型的更高维MDP。这个MDP被称为*Bayes-adaptive Markov decision process*，它与第四部分讨论的部分可观测马尔可夫决策过程相关。

贝叶斯自适应MDP中的状态空间是笛卡尔积 $\mathcal{S} \times \mathcal{B}$，其中 $\mathcal{B}$ 是关于模型参数 θ 的可能信念空间。尽管 $\mathcal{S}$ 是离散的，但 $\mathcal{B}$ 通常是高维连续状态空间。[6] 一个状态在贝叶斯自适应MDP是一个由基础MDP中的当前状态s和信念状态b组成的对(s, b)。其动作空间和奖励函数与基础MDP相同。

6 在转移概率上的Dirichlet分布情形下，它是连续的，如例16.3所示。

贝叶斯自适应MDP中的转移函数为$T(s'、b' \mid s、b、a)$，即在智能体从s出发、信念为b并采取动作a的情况下，转移到具有信念状态b'的状态s'的概率。新的信念状态b'可以根据贝叶斯规则确定性地计算出来。如果我们用τ表示这个确定性函数，使得$b' = \tau(s、b、a、s')$，那么我们可以将贝叶斯自适应MDP的转移函数分解为

$$T(s', b' \mid s, b, a) = \delta_{\tau(s,b,a,s')}(b')\, P(s' \mid s, b, a) \tag{16.7}$$

其中 $\delta_x(y)$ 是 *Kronecker delta function*[7]，使得当 $x = y$ 时，$\delta_x(y) =$ 为 1，否则为 0 否则。

7 此函数以德国数学家利奥波德·克罗内克（1823–1891）命名。

第二项可以通过积分来计算：

$$P(s' \mid s, b, a) = \int_\theta b(\theta) P(s' \mid s, \theta, a)\, \mathrm{d}\theta = \int_\theta b(\theta) \theta_{(s,a,s')}\, \mathrm{d}\theta \tag{16.8}$$

这个方程可以类似于方程(15.1)的方式解析求解。当我们的信念b由方程(16.6)中的因子狄利克雷表示时，我们有

$$P(s' \mid s, b, a) = N(s, a, s') / \sum_{s''} N(s, a, s'') \tag{16.9}$$

我们可以将贝尔曼最优方程（方程(7.16)）针对已知模型的MDP推广到模型未知的情况：

$$U^*(s, b) = \max_a \left(R(s, a) + \gamma \sum_{s'} P(s' \mid s, b, a) U^*\left(s', \tau\left(s, b, a, s'\right)\right) \right) \tag{16.10}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

不幸的是，我们不能直接应用策略迭代或值迭代，因为b是连续的。不过，我们可以使用第8章的近似方法或第9章的在线方法。第四部分介绍能更好地利用贝叶斯自适应MDP结构的方法。

16.6 Posterior Sampling

在信念空间上求解最优价值函数的一种替代方法是使用*posterior sampling*、[8]，这最初是在以下背景下引入的

在 15.4.[9] 节中关于 bandit 问题的探索。在这里，我们抽取一个样本 θ 从当前信念 b，然后假设 θ 是真实模型，求解最佳行动。接着我们更新信念，抽取新样本，并求解相应的 MDP。例 16.4 提供了一个这样的实例。

8 M. J. A. Strens，“强化学习的贝叶斯框架”，载于 *International Conference on Machine Learning* (*ICML*)，2000年。

9 在该部分，我们从关于收益概率的后验分布中抽样，然后假设所选动作时抽样的概率是正确的。

后验抽样的一个优势在于，我们无需确定启发式探索参数。然而，每一步解决MDP可能成本高昂。算法16.9中实现了一种从后验中抽样离散MDP的方法。

```
struct PosteriorSamplingUpdate end

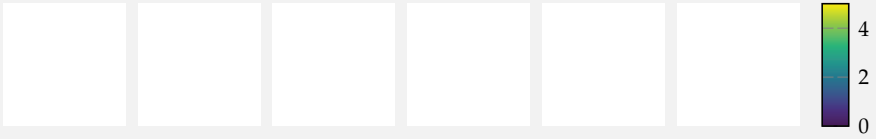
function Base.rand(model::BayesianMDP)
    𝒮, 𝒜 = model.𝒮, model.𝒜
    T = zeros(length(𝒮), length(𝒜), length(𝒮))
    for s in 𝒮
        for a in 𝒜
            T[s,a,:] = rand(model.D[s,a])
        end
    end
    return MDP(T, model.R, model.γ)
end

function update!(planner::PosteriorSamplingUpdate, model, s, a, r, s′)
    𝒫 = rand(model)
    U = solve(𝒫).U
    copy!(model.U, U)
end
```

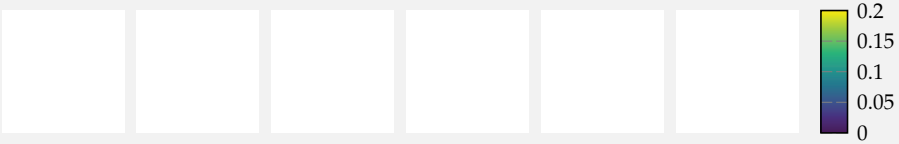
算法16.9。后验采样更新方法。在更新贝叶斯后验参数之后，我们根据该后验采样一个MDP问题。此实现假设状态和动作空间是离散的，并使用Dirichlet分布来建模从每个状态-动作对的转移概率中的不确定性。为了生成转移模型，我们遍历每个状态和动作，并从相关的Dirichlet分布中采样。一旦我们得到了一个采样问题 $\mathcal{P}$，我们便使用线性规划形式对其进行求解，并存储由此得到的价值函数 `U`。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们希望将贝叶斯模型估计应用于六边形世界。我们首先为每个状态-动作对关联均匀的Dirichlet先验。在进行100次长度为10的模拟并将我们的转移计数加到先验中的伪计数之后，我们关于后继状态的后验分布的参数如下所示：



我们可以从该分布中采样，以生成这里展示的模型。注意，它比示例16.1中展示的最大似然模型拥有多得多的非零转移概率。



示例 16.4. 将贝叶斯模型估计和后验采样应用于六边形世界问题。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

16.7 Summary

- 基于模型的方法通过与环境的交互来学习状态转移模型和奖励模型。
- 最大似然模型使用转移计数来维持对后继状态转移概率的估计，并追踪与状态-动作对相关的平均奖励。
- 最大似然模型必须与探索策略配对，例如上一章在赌博机背景下介绍的那些策略。
- 虽然我们可以根据每一步经验重新规划，但精确地这样做可能代价高昂。
- 优先扫描可以通过更新在环境动态模型中看起来最需要更新的状态的值来集中重规划。
- 基于贝叶斯模型的方法对可能的问题维持一个概率分布，从而能够进行有原则的探索推理。
- 在贝叶斯自适应MDP中，其状态通过可能MDP模型上的概率分布增强了原始MDP。
- 后验采样通过求解从信念状态中采样的 MDP，而不是对所有可能的 MDP 进行推理，从而降低了求解贝叶斯自适应 MDP 的高计算复杂度。

16.8 Exercises

练习 16.1。假设有一个智能体在一个具有三个状态和两个动作的环境中交互，其转移和奖励模型未知。我们与环境进行了一个直接交互序列。表 16.1 列出了状态、动作、奖励以及结果状态。请使用最大似然估计从这些数据中估计转移函数和奖励函数。

表16.1. 转移数据。

s	a	r	s'
s_2	a_1	2	s_1
s_1	a_2	1	s_2
s_2	a_2	1	s_1
s_1	a_2	1	s_2
s_2	a_2	1	s_3
s_3	a_2	2	s_2
s_2	a_2	1	s_3
s_3	a_2	2	s_3
s_3	a_1	2	s_2
s_2	a_1	2	s_3

Solution: 我们首先将每个状态和动作的转移次数$N(s、a)$，收到的奖励$\rho(s、a)$，以及奖励函数的最大似然估计$\hat{R}(s、a) = \rho(s、a)/N(s、a)$制表如下：

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

s	a	$N(s,a)$	$\rho(s,a)$	$\hat{R}(s,a) = \frac{\rho(s,a)}{N(s,a)}$
s_1	a_1	0	0	0
s_1	a_2	2	2	1
s_2	a_1	2	4	2
s_2	a_2	3	3	1
s_3	a_1	1	2	2
s_3	a_2	2	4	2

在下一组表格中，我们计算观测到的转移次数 $N(s、a、s')$ 以及转移模型的最大似然估计 $\hat{T}(s' \mid s、a) = N(s、a、s')/N(s、a)$。当 $N(s、a) =$ 为0时，我们在得到的状态上使用均匀分布。

s	a	s'	$N(s,a,s')$	$\hat{T}(s' \mid s,a) = \frac{N(s,a,s')}{N(s,a)}$
s_1	a_1	s_1	0	1/3
s_1	a_1	s_2	0	1/3
s_1	a_1	s_3	0	1/3
s_1	a_2	s_1	0	0
s_1	a_2	s_2	2	1
s_1	a_2	s_3	0	0
s_2	a_1	s_1	1	1/2
s_2	a_1	s_2	0	0
s_2	a_1	s_3	1	1/2
s_2	a_2	s_1	1	1/3
s_2	a_2	s_2	0	0
s_2	a_2	s_3	2	2/3
s_3	a_1	s_1	0	0
s_3	a_1	s_2	1	1
s_3	a_1	s_3	0	0
s_3	a_2	s_1	0	0
s_3	a_2	s_2	1	1/2
s_3	a_2	s_3	1	1/2

练习16.2。给出在优先扫描的一次迭代过程中可能执行的更新次数的一个下界和一个上界。

Solution: 在优先扫描的一次迭代中，执行的更新次数下界为1。这可能在首次迭代使用最大似然模型时发生，此时转移模型中的唯一非零项是$T(s' \mid s, a)$。由于没有

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

状态-动作对（s^-，a^-）转换到s，我们的优先队列将为空，因此唯一执行的更新将是针对$U(s)$。

在优先扫描的一次迭代中，执行更新次数的上界为$|\mathcal{S}|$。假设我们刚刚转移到s'，并且对于所有s和a，$\hat{T}(s' \mid s、a) >$均为0。如果我们不提供最大更新次数，我们将执行$|\mathcal{S}|$次更新。如果我们提供最大更新次数$m < |\mathcal{S}|$，则上界减少到m。

练习16.3。在对状态空间为$\mathcal{S}$、动作空间为$\mathcal{A}$的离散MDP进行贝叶斯强化学习以估计转移模型参数时，如果使用Dirichlet分布来表示转移模型的不确定性，那么有多少个独立参数？

Solution: 对于每个状态和动作，我们指定一个关于转移概率参数的狄利克雷分布，因此将有$|\mathcal{S}||\mathcal{A}|$个狄利克雷分布。每个狄利克雷分布由$|\mathcal{S}|$个独立参数来指定。总计，我们有$|\mathcal{S}|^2|\mathcal{A}|$个独立参数。

练习 16.4。考虑练习 16.1 中的问题陈述，但这次我们希望使用贝叶斯强化学习，其中先验分布由狄利克雷分布表示。假设先验是均匀的，给定我们处于状态 s_2 并采取动作 a_1，下一个状态的后验分布是什么？

Solution: $\text{Dir}(\theta_{(s_2,a_1)} \mid [2,1,2])$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

17 Model-Free Methods

与基于模型的方法相比，*model-free reinforcement learning* 不需要构建显式的转移和奖励模型表示。[1]

本章讨论的无模型方法直接对动作价值函数进行建模。避免显式表示很有吸引力，尤其是在问题维度较高时。本章首先介绍分布均值的增量估计，这在估计回报均值中起着重要作用。然后我们讨论一些常见的无模型算法以及更高效地处理延迟奖励的方法。最后，我们讨论如何使用函数逼近从我们的经验中进行泛化。[2]

1 本章中的许多主题由 R. S. Sutton 和 A. G. Barto 在 *Reinforcement Learning: An Introduction* 第2版中更深入地探讨，MIT Press，2018年。另见 D. P. Bertsekas, *Reinforcement Learning and Optimal Control*。Athena Scientific，2019年。

2 尽管本书的这一部分一直聚焦于环境模型未知的问题，但强化学习常用于模型已知的问题。本章讨论的无模型方法作为近似动态规划的一种形式，在复杂环境中尤其有用。它们可以用于离线生成策略，或作为在在线上下文中生成下一个动作的手段。

17.1 Incremental Estimation of the Mean

许多无模型方法*incrementally estimate*动作价值函数$Q(s, a)$从样本中。暂时假设我们只关心来自m个样本的单个变量X的期望：

$$\hat{x}_m = \frac{1}{m}\sum_{i=1}^{m} x^{(i)} \tag{17.1}$$

其中 $x^{(1)}$、……$x^{(m)}$ 是样本。我们可以推导出一个增量更新：

$$\begin{aligned}\hat{x}_m &= \frac{1}{m}\left(x^{(m)} + \sum_{i=1}^{m-1} x^{(i)}\right) && (17.2)\\ &= \frac{1}{m}\left(x^{(m)} + (m-1)\hat{x}_{m-1}\right) && (17.3)\\ &= \hat{x}_{m-1} + \frac{1}{m}\left(x^{(m)} - \hat{x}_{m-1}\right) && (17.4)\end{aligned}$$

我们可以通过引入一个 *learning rate* 函数 $\alpha(m)$ 来重写这个方程：

$$\hat{x}_m = \hat{x}_{m-1} + \alpha(m)\left(x^{(m)} - \hat{x}_{m-1}\right) \tag{17.5}$$

学习率可以是除1/m以外的函数。为确保收敛，我们通常选择$\alpha(m)$，使得我们有$\sum_{m=1}^{\infty} \alpha(m) = \infty$和$\sum_{m=1}^{\infty} \alpha^2(m) < \infty$。第一个条件确保步长足够大，第二个条件确保步长足够小。[3]

3 关于收敛性的讨论及其在本章讨论的其他一些算法中的应用，参见 T. Jaakkola、M. I. Jordan和S. P. Singh，“On the Convergence of Stochastic Iterative Dynamic Programming Algorithms”，*Neural Computation*，第6卷，第6期，第1185–1201页，1994年。

如果学习率是恒定的，这在强化学习应用中很常见，那么较旧样本的权重会以速率$(1 - \alpha)$呈指数衰减。在学习率恒定的情况下，我们可以在观察到x后，使用以下规则来更新我们的估计：

$$\hat{x} \leftarrow \hat{x} + \alpha(x - \hat{x}) \tag{17.6}$$

算法17.1提供了这一实现。例17.1展示了几个学习率的例子。

这里讨论的更新规则将在后续章节中再次出现，并与随机梯度下降有关。更新幅度与样本和先前估计之间的差成正比。样本与先前估计之间的差称为 *temporal difference error*。

17.2 Q-Learning

Q-learning (算法17.2)涉及应用动作价值函数$Q(s, a)$的增量估计。[4]该更新源自于动作价值形式的贝尔曼期望方程：

4 C. J. C. H. Watkins，“Learning from Delayed Rewards”，博士论文，剑桥大学，1989年。

$$Q(s, a) = R(s, a) + \gamma \sum_{s'} T(s' \mid s, a) U(s') \tag{17.7}$$

$$= R(s, a) + \gamma \sum_{s'} T(s' \mid s, a) \max_{a'} Q(s', a') \tag{17.8}$$

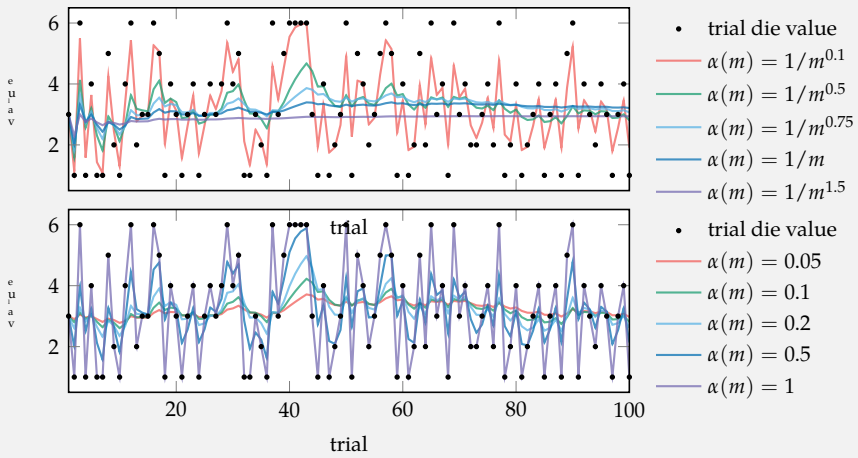
我们可以根据奖励 r 和下一状态 s' 的样本期望来重写上述方程，而不是使用 T 和 R：

$$Q(s, a) = \mathbb{E}_{r,s'}[r + \gamma \max_{a'} Q(s', a')] \tag{17.9}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

考虑估计掷一枚公平的六面骰子时获得的期望值。以下是*learning curves*，展示了在100次试验中与不同学习率函数相关的增量估计。正如我们所看到的，如果$\alpha(m)$衰减得太快，则无法保证收敛；如果$\alpha(m)$衰减得不够快，则收敛速度会很慢。

对于恒定值 $\alpha \in (0, 1]$，均值估计将继续波动。较大的恒定值 α 波动剧烈，而较低的值则需要更长时间才能收敛。



例17.1。以不同的函数对{v7}衰减学习率的效果。

```
mutable struct IncrementalEstimate
    μ # mean estimate
    α # learning rate function
    m # number of updates
end

function update!(model::IncrementalEstimate, x)
    model.m += 1
    model.μ += model.α(model.m) * (x - model.μ)
    return model
end
```

算法17.1. 一种用于维护随机变量均值增量估计的类型。相关类型维护当前平均值 `μ`、学习率函数 `α` 和迭代计数 `m`。使用新值 `x` 调用 `update!` 会更新估计值。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们可以使用方程（17.6）来生成一个增量更新规则，以估计动作价值函数：[5]

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left(r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right) \tag{17.10}$$

我们的行为选择会影响我们最终所处的状态，从而影响我们准确估计$Q(s、a)$的能力。为了保证动作价值函数的收敛性，我们需要采用某种形式的探索策略，例如ϵ-贪心或softmax，正如我们在上一章中对基于模型的方法所做的那样。示例17.2展示了如何使用Q-学习更新规则和探索策略来运行模拟。图17.1在六边形世界问题上展示了这一过程。

5 该方程中的最大化可能会引入偏差。像*double Q-learning*这样的算法试图校正这种偏差，从而可能带来更好的性能。H. van Hasselt，“Double Q-Learning”，载于*Advances in Neural Information Processing Systems (NIPS)*，2 010年。

```
mutable struct QLearning
    𝒮 # state space (assumes 1:nstates)
    𝒜 # action space (assumes 1:nactions)
    γ # discount
    Q # action value function
    α # learning rate
end

lookahead(model::QLearning, s, a) = model.Q[s,a]

function update!(model::QLearning, s, a, r, s′)
    γ, Q, α = model.γ, model.Q, model.α
    Q[s,a] += α*(r + γ*maximum(Q[s′,:]) - Q[s,a])
    return model
end
```

算法 17.2。用于无模型强化学习的Q-学习更新，可应用于转移和奖励函数未知的问题。该更新修改 `Q`，这是一个状态-动作值矩阵。此更新函数可与探索策略（例如 ϵ-贪心）一起用于算法 15.9 的模拟函数中。该模拟函数调用更新函数时传入 `s′`，尽管这个 Q-学习实现并不使用它。

17.3 *Sarsa*

Sarsa (算法17.3)是Q学习的一种替代方法。[6]它的名字来源于使用 (s, a, r, s', a') 在每一步更新 Q 函数。它使用实际的下一个动作 a' 来更新 Q，而不是对所有可能的动作取最大值：

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left(r + \gamma Q(s',a') - Q(s,a)\right) \tag{17.11}$$

采用适当的探索策略，a'将收敛到$\arg\max_{a'} Q(s',a')$，这正是Q-学习更新中所使用的。

6 这种方法在 G. A. Rummery 和 M. Niranjan 的“使用连接主义系统的在线Q学习”，剑桥大学，技术报告 CUED/F-INFENG/TR 166, 1994 年中以不同的名称提出。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

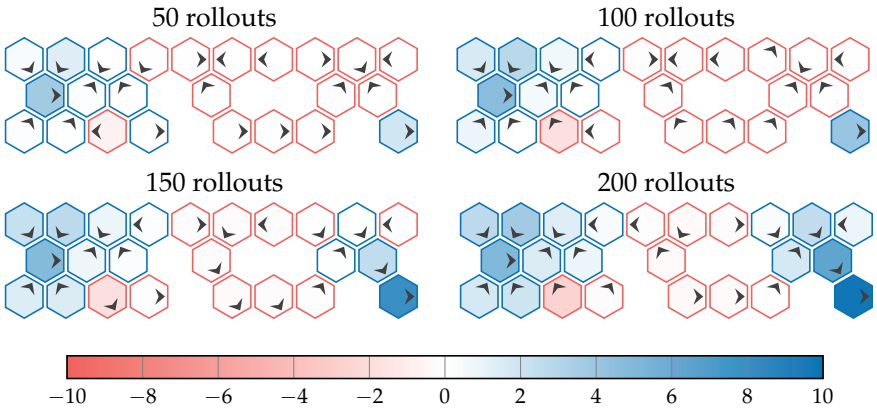


图17.1. Q-学习用于迭代学习六边形世界问题的动作价值函数。根据Q，每个状态根据该状态下最佳动作的期望值着色。动作同样是最佳期望动作。Q-学习以$\alpha = 0.1$和每次试验10步运行。

假设我们想将Q-学习应用于一个MDP问题$\mathcal{P}$。我们可以构建一个探索策略，比如上一章算法16.6中实现的ϵ-贪婪策略。Q-学习模型来自算法17.2，而模拟函数则在算法15.9中实现。

```
Q = zeros(length(𝒫.𝒮), length(𝒫.𝒜))
α = 0.2 # learning rate
model = QLearning(𝒫.𝒮, 𝒫.𝒜, 𝒫.γ, Q, α)
ϵ = 0.1 # probability of random action
π = EpsilonGreedyExploration(ϵ)
k = 20 # number of steps to simulate
s = 1  # initial state
simulate(𝒫, model, π, k, s)
```

示例 17.2. 如何在模拟中使用带有 Q 学习的探索策略。参数设置为假设值。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Sarsa被称为一种*on-policy*强化学习方法，因为它试图在遵循探索策略的同时直接估计其价值。相比之下，Q-学习是一种*off-policy*方法，因为它试图在遵循探索策略的同时找到最优策略的价值。尽管Q-学习和Sarsa都能收敛到最优策略，但收敛速度取决于应用场景。Sarsa在图17.2所示的六边形世界问题上运行。

```
mutable struct Sarsa
    𝒮 # state space (assumes 1:nstates)
    𝒜 # action space (assumes 1:nactions)
    γ # discount
    Q # action value function
    α # learning rate
    ℓ # most recent experience tuple (s,a,r)
end

lookahead(model::Sarsa, s, a) = model.Q[s,a]

function update!(model::Sarsa, s, a, r, s′)
    if model.ℓ != nothing
        γ, Q, α, ℓ = model.γ, model.Q, model.α,  model.ℓ
        model.Q[ℓ.s,ℓ.a] += α*(ℓ.r + γ*Q[s,a] - Q[ℓ.s,ℓ.a])
    end
    model.ℓ = (s=s, a=a, r=r)
    return model
end
```

算法17.3. 无模型强化学习中的Sarsa更新。我们更新包含状态-动作值的矩阵`Q`，其中`α`是恒定学习率，而ℓ是最新的经验元组。与Q-学习实现一样，更新函数可以在算法15.9中的模拟器中使用。

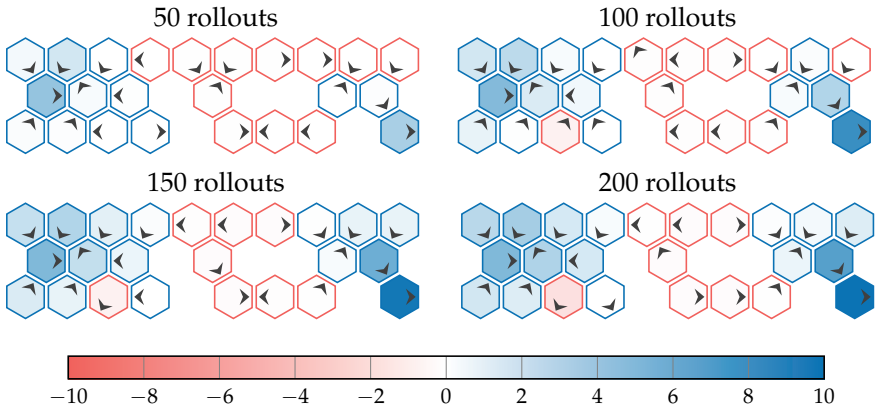


图17.2. Sarsa用于迭代学习六边形世界问题的动作价值函数，其方式与图17.1相同。我们发现Sarsa收敛到真实动作价值函数的速度较慢。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

17.4 Eligibility Traces

Q学习和Sarsa的缺点之一是学习可能非常缓慢，尤其是在*sparse rewards*的情况下。例如，假设环境中有一个单一的目标状态，它提供了很大的奖励，而在所有其他状态下奖励为零。在环境中进行了一定数量的随机探索后，我们到达了目标状态。无论我们使用Q学习还是Sarsa，我们仅更新紧邻目标状态之前的状态的动作值。所有其他通向目标状态的状态的值仍然为零。为了缓慢地将非零值传播到状态空间的其余部分，需要大量的探索。

Q-学习和Sarsa可以通过使用*eligibility traces*将奖励反向传播到导致奖励来源的状态和动作来进行修改。[7] 该信用以指数方式衰减，使得离奖励更近的状态被赋予更大的值。通常使用$0 < \lambda < 1$作为指数衰减参数。带有资格迹的Q学习和Sarsa版本通常被称为Q(λ)和Sarsa(λ)。[8]

Learning

一种Sarsa(λ)版本在算法17.4中实现，该算法为所有状态-动作对维护一个指数衰减的访问计数$N(s, a)$。当在状态s采取动作a时，$N(s, a)$增加1。然后，根据这个衰减的访问计数，Sarsa的时间差分更新被部分应用于每个状态-动作对。

让 δ 表示 Sarsa 时间差分更新：

Learning

$$\delta = r + \gamma Q(s', a') - Q(s, a) \tag{17.12}$$

动作价值函数中的每一项随后根据以下公式进行更新

$$Q(s, a) \leftarrow Q(s, a) + \alpha \delta N(s, a) \tag{17.13}$$

访问计数随后使用折扣因子和指数衰减参数进行衰减：

$$N(s, a) \leftarrow \gamma \lambda N(s, a) \tag{17.14}$$

尽管资格迹的影响在稀疏奖励环境中尤为显著，但该算法在奖励分布更广的一般环境中也能加速学习。

7 资格迹是由 R. Sutton 在时间差分学习的背景下提出的，“通过学习时间差分的方法进行预测”，*Machine*，第 3 卷，第 1 期，第 9-44 页，1988 年。

8 这些算法由 C. J. C. H. Watkins 在“Learning from Delayed Rewards”中提出，博士论文，剑桥大学，1989年。以及 J. Peng 和 R. J. Williams，“Incremental Multi-Step Q-Learning,” *Machine*，第22卷，第1–3期，第283–290页，1996年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
mutable struct SarsaLambda
    𝒮 # state space (assumes 1:nstates)
    𝒜 # action space (assumes 1:nactions)
    γ # discount
    Q # action value function
    N # trace
    α # learning rate
    λ # trace decay rate
    ℓ # most recent experience tuple (s,a,r)
end

lookahead(model::SarsaLambda, s, a) = model.Q[s,a]

function update!(model::SarsaLambda, s, a, r, s′)
    if model.ℓ != nothing
        γ, λ, Q, α, ℓ = model.γ, model.λ, model.Q, model.α, model.ℓ
        model.N[ℓ.s,ℓ.a] += 1
        δ = ℓ.r + γ*Q[s,a] - Q[ℓ.s,ℓ.a]
        for s in model.𝒮
            for a in model.𝒜
                model.Q[s,a] += α*δ*model.N[s,a]
                model.N[s,a] *= γ*λ
            end
        end
    else
        model.N[:,:] .= 0.0
    end
    model.ℓ = (s=s, a=a, r=r)
    return model
end
```

算法17.4. Sarsa(λ)更新，使用资格迹将奖励反向传播以加速稀疏奖励的学习。矩阵`Q`包含状态-动作值，矩阵`N`包含指数衰减的状态-动作访问计数，`α`是常数学习率，`λ`是指数衰减参数，ℓ是最近的经历元组。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

将资格迹应用于像Q学习这样的off-policy算法时需特别小心，该算法试图学习最优策略的价值。[9]
资格迹会反向传播从探索策略中获得的值。这种不匹配可能导致学习不稳定。

9 关于此问题的概述及潜在解决方案，参见A. Harutyunyan、M. G. Bellemare、T. Stepleton和R. Munos，“Q(λ) 与离策略修正”，载于 *International Conference on Algorithmic Learning Theory (ALT)*，2016年。

17.5 Reward Shaping

奖励函数增强也可以改善学习，特别是在奖励稀疏的问题中。例如，如果我们试图达到一个单一的目标状态，我们可以通过一个与到目标的距离成反比的量来补充奖励函数。或者，我们可以根据离目标的距离增加另一个惩罚。例如，如果我们正在下棋，当我们失去一个棋子时，我们可能会在奖励函数中增加一个惩罚，尽管我们只关心游戏最终的胜负，而不关心单个棋子的得失。

在训练过程中通过融入领域知识来修改奖励函数以加速训练被称为*reward shaping*。假设我们问题中的奖励是根据$R(s、a、s')$生成的，允许奖励依赖于结果状态。我们将使用$F(s、a、s')$来表示我们的*shaping function*。在训练期间，我们不使用$R(s、a、s')$作为奖励，而是使用$R(s、a、s') + F(s、a、s')$。

将$F(s、a、s')$加入我们的奖励当然可以改变最优策略。我们通常感兴趣的是在不改变最优策略的情况下塑形奖励。事实证明，一个在原始奖励下最优的策略在塑形奖励下仍然最优，当且仅当

$$F(s, a, s') = \gamma\beta(s') - \beta(s) \tag{17.15}$$

对于某个势函数 $\beta(s)$。[10]

10 A. Y. Ng, D. Harada, 和 S. Russell, “奖励变换下的策略不变性：理论及其在奖励塑形中的应用”，载于 *International Conference on Machine Learning (ICML)*，1999年。

17.6 Action Value Function Approximation

本章到目前为止讨论的算法都假设状态和动作空间是离散的，其中动作价值函数可以存储在查找表中。我们可以调整算法以使用价值函数逼近，从而能够将它们应用于大空间或连续空间的问题，并从有限的经验中进行泛化。与第8章中在

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

已知模型，我们将使用 $Q_\theta(s、a)$ 来表示当模型未知时动作价值函数的参数化近似。[11]

为了说明这一概念，我们将推导出一个使用参数近似的 Q 学习版本。我们希望最小化我们的近似与最优动作价值函数 $Q^*(s,a)$ 之间的损失，我们将[12]定义为

$$\ell(\theta) = \frac{1}{2} \mathop{\mathbb{E}}_{(s,a)\sim\pi^*} \left[\left(Q^*(s,a) - Q_\theta(s,a) \right)^2 \right] \tag{17.16}$$

期望是在遵循最优策略π^*时所经历的状态-动作对上。

最小化这种损失的常见方法是使用某种形式的梯度下降。损失的梯度是

$$\nabla \ell(\theta) = - \mathop{\mathbb{E}}_{(s,a)\sim\pi^*} [(Q^*(s,a) - Q_\theta(s,a)) \nabla_\theta Q_\theta(s,a)] \tag{17.17}$$

我们通常选择动作价值函数的参数化表示，这些表示是可微的，并且 $\nabla_\theta Q_\theta(s、a)$ 易于计算，例如线性表示或神经网络表示。如果我们应用梯度下降，[13] 我们的更新规则
是

$$\theta \leftarrow \theta + \alpha \mathop{\mathbb{E}}_{(s,a)\sim\pi^*} [(Q^*(s,a) - Q_\theta(s,a)) \nabla_\theta Q_\theta(s,a)] \tag{17.18}$$

其中 α 是我们的步长因子或学习率。我们可以利用经历过的状态-动作对样本 (s, a) 来近似上述更新规则：

$$\theta \leftarrow \theta + \alpha (Q^*(s,a) - Q_\theta(s,a)) \nabla_\theta Q_\theta(s,a) \tag{17.19}$$

当然，我们无法直接计算方程（17.19），因为那将需要知道最优策略，而这正是我们试图寻找的。相反，我们尝试从观察到的转换和动作价值近似中估计它：

$$Q^*(s,a) \approx r + \gamma \max_{a'} Q_\theta(s',a') \tag{17.20}$$

从而得到以下更新规则：

$$\theta \leftarrow \theta + \alpha (r + \gamma \max_{a'} Q_\theta(s',a') - Q_\theta(s,a)) \nabla_\theta Q_\theta(s,a) \tag{17.21}$$

近年来，一个主要的焦点是 *deep reinforcement learning*，其中深度神经网络被用于这种参数近似。实际实施的讨论由L. Graesser和W. Keng提供 *Foundations of Deep Reinforcement Learning* 。Addison Wesley，2020年。

12 前面的1/2是为了方便，因为后面我们将计算这个二次型的导数。

13 我们希望下降而不是上升，因为我们试图最小化我们的损失。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

此更新在算法17.5中实现，加入了缩放梯度步骤（算法12.2），通常需要确保梯度步骤不会变得过大。例17.3展示了如何结合线性动作值近似使用此更新。图17.3以山地车问题演示了该算法。

```
struct GradientQLearning
    𝒜  # action space (assumes 1:nactions)
    γ  # discount
    Q  # parameterized action value function Q(θ,s,a)
    ∇Q # gradient of action value function
    θ  # action value function parameter
    α  # learning rate
end

function lookahead(model::GradientQLearning, s, a)
    return model.Q(model.θ, s,a)
end

function update!(model::GradientQLearning, s, a, r, s′)
    𝒜, γ, Q, θ, α = model.𝒜, model.γ, model.Q, model.θ, model.α
    u = maximum(Q(θ,s′,a′) for a′ in 𝒜)
    Δ = (r + γ*u - Q(θ,s,a))*model.∇Q(θ,s,a)
    θ[:] += α*scale_gradient(Δ, 1)
    return model
end
```

算法17.5. 带有动作价值函数近似的Q-学习更新。对于每个新的经验元组`s`、`a`、`r`、`s′`，我们使用恒定学习率`α`更新向量`θ`。我们的参数化动作价值函数由`Q(θ,s,a)`给出，其梯度为`∇Q(θ,s,a)`。

17.7 Experience Replay

在强化学习中使用全局函数逼近的一个主要挑战是*catastrophic forgetting*。例如，我们可能最初发现我们的特定策略将我们带到状态空间中的低奖励区域。然后我们优化我们的策略以避开该区域。然而，经过一段时间后，我们可能会忘记为什么避开状态空间的该区域很重要，并且我们可能会冒险回到表现不佳的策略。

可以使用 *experience replay*、[14] 缓解灾难性遗忘，其中固定最近的经验元组的数量在训练迭代中存储。从这个*batch*中均匀采样一个*replay memory*的元组，以提醒我们避免已经发现效果不佳的策略。[15] 更新方程为从方程（17.21）修改为

14 经验回放 在 V. Mnih、K. Kavukcuoglu、D. Silver、A. Graves、I. Antonoglou、D. Wierstra 和 M. Riedmiller 的 “Playing Atari with Deep Reinforcement Learning,” 2013. arXiv: 1312 . 5602v1 工作中发挥了重要作用。这个概念早先由 L.-J. Lin 在 “Reinforcement Learning for Robots Using Neural Networks,” 博士论文，卡内基梅隆大学，1993 中探索过。

15 该方法的变体包括优先经验回放。T. Schaul, J. Quan, I. Antonoglou, and D. Silver, “Prioritized Experience Replay,” in *International Conference on Learning Representations (ICLR)*, 2016.

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们感兴趣的是将具有线性动作价值近似的Q学习应用于具有$\gamma = 1$的简单调节器问题。我们的动作价值近似为$Q_\theta(s, a) = \theta^\top \beta(s, a)$，其中我们的基函数是

$$\beta(s, a) = [s, s^2, a, a^2, 1]$$

在这个线性模型中，

$$\nabla_\theta Q_\theta(s, a) = \beta(s, a)$$

我们可以按如下方式为问题$\mathcal{P}$实现这一点：

```
β(s,a) = [s,s^2,a,a^2,1]
Q(θ,s,a) = dot(θ,β(s,a))
∇Q(θ,s,a) = β(s,a)
θ = [0.1,0.2,0.3,0.4,0.5] # initial parameter vector
α = 0.5 # learning rate
model = GradientQLearning(𝒫.𝒜, 𝒫.γ, Q, ∇Q, θ, α)
ϵ = 0.1 # probability of random action
π = EpsilonGreedyExploration(ϵ)
k = 20  # number of steps to simulate
s = 0.0 # initial state
simulate(𝒫, model, π, k, s)
```

示例17.3。如何在模拟中结合Q-学习与动作价值函数近似的探索策略。参数设置为假设值。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

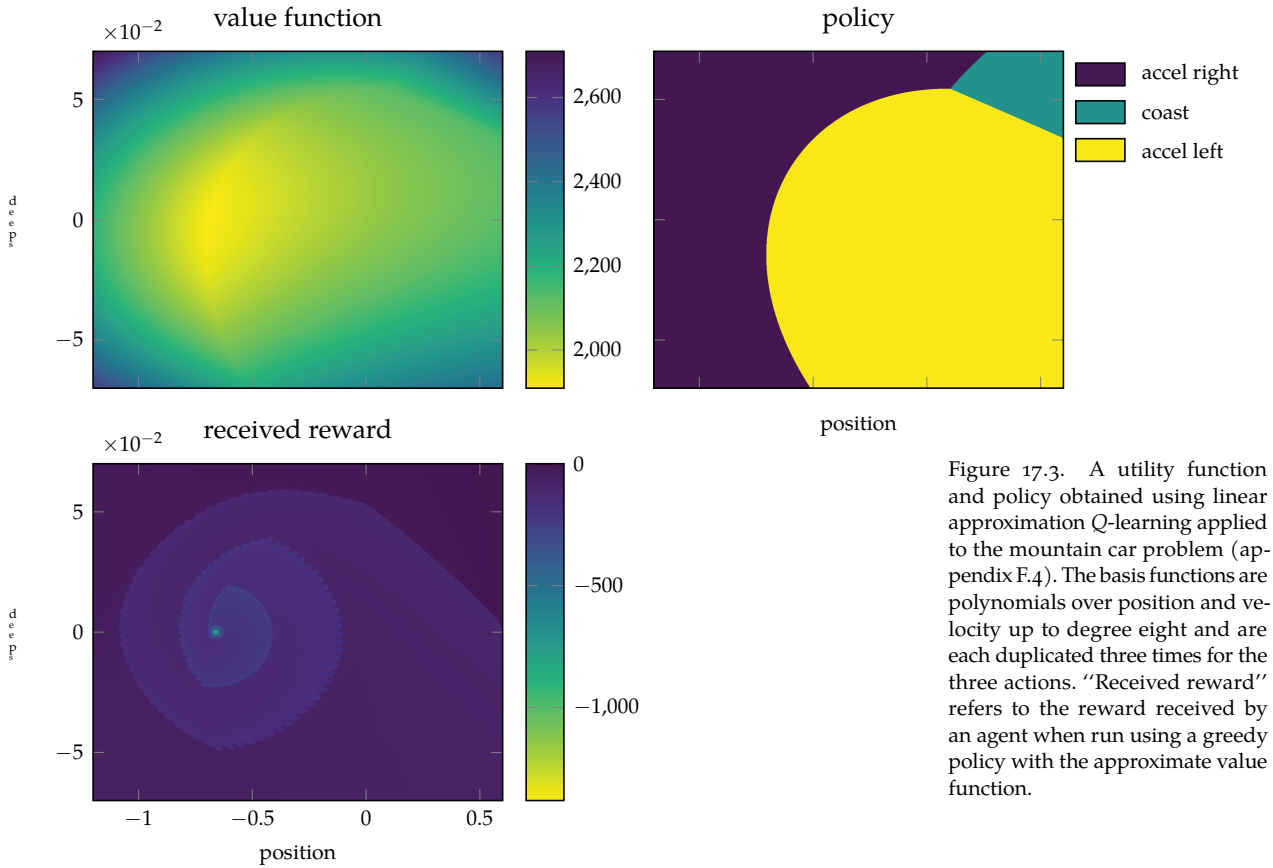


Figure 17.3. A utility function and policy obtained using linear approximation Q-learning applied to the mountain car problem (appendix F.4). The basis functions are polynomials over position and velocity up to degree eight and are each duplicated three times for the three actions. ''Received reward'' refers to the reward received by an agent when run using a greedy policy with the approximate value function.

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

$$\theta \leftarrow \theta + \alpha \frac{1}{m_{\text{grad}}} \sum_i (r^{(i)} + \gamma \max_{a'} Q_\theta(s'^{(i)}, a') - Q_\theta(s^{(i)}, a^{(i)})) \nabla_\theta Q_\theta(s^{(i)}, a^{(i)}) \tag{17.22}$$

其中 $s^{(i)}$、$a^{(i)}$、$r^{(i)}$ 和 $s'^{(i)}$ 是大小为 m_{grad} 的随机批次中的第 i 个经验元组。

经验回放使经验元组能够多次为学习做出贡献，从而提高数据效率。此外，从回放记忆中均匀随机采样打破了原本从轨迹中获取的相关序列，从而降低了梯度估计的方差。经验回放通过保留先前策略参数化的信息来稳定学习过程。

算法17.6展示了如何将经验回放融入带有动作价值函数近似的Q-学习。示例17.4说明了如何将此方法应用于简单的调节器问题。

17.8 Summary

- 无模型方法旨在直接学习动作价值函数，而不是学习状态转移和奖励模型。
- 简单的技术可用于从连续的更新中逐步学习均值。
- Q-学习算法使用贝尔曼方程的近似来增量地学习动作价值函数。
- 与Q-学习相比，Sarsa在其更新中使用的是探索策略所采取的动作，而不是对所有后续动作进行最大化。
- 资格迹可以通过状态-动作空间传播稀疏奖励，从而加速学习。
- Q-学习可以通过随机梯度下降来近似值函数。
- Q-学习和Sarsa所经历的灾难性遗忘可通过经验回放得到缓解，该回放重用了过去的经验元组。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct ReplayGradientQLearning
    𝒜      # action space (assumes 1:nactions)
    γ      # discount
    Q      # parameterized action value function Q(θ,s,a)
    ∇Q     # gradient of action value function
    θ      # action value function parameter
    α      # learning rate
    buffer # circular memory buffer
    m      # number of steps between gradient updates
    m_grad # batch size
end

function lookahead(model::ReplayGradientQLearning, s, a)
    return model.Q(model.θ, s,a)
end

function update!(model::ReplayGradientQLearning, s, a, r, s′)
    𝒜, γ, Q, θ, α = model.𝒜, model.γ, model.Q, model.θ, model.α
    buffer, m, m_grad = model.buffer, model.m, model.m_grad
    if isfull(buffer)
        U(s) = maximum(Q(θ,s,a) for a in 𝒜)
        ∇Q(s,a,r,s′) = (r + γ*U(s′) - Q(θ,s,a))*model.∇Q(θ,s,a)
        Δ = mean(∇Q(s,a,r,s′) for (s,a,r,s′) in rand(buffer, m_grad))
        θ[:] += α*scale_gradient(Δ, 1)
        for i in 1:m # discard oldest experiences
            popfirst!(buffer)
        end
    else
        push!(buffer, (s,a,r,s′))
    end
    return model
end
```

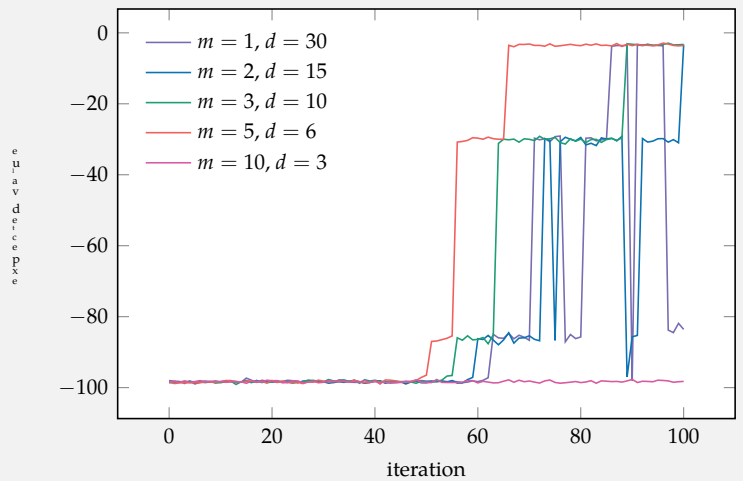
算法17.6. 使用函数逼近和经验回放的Q学习。更新依赖于参数化策略`Q(θ,s,a)`和梯度`∇Q(θ,s,a)`。它更新参数向量`θ`以及由`DataStructures.jl`提供的循环记忆缓冲区。它每`m`步使用从缓冲区中的`m_grad`个样本估计的梯度来更新`θ`。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

假设我们想为示例17.3添加经验回放。在构建模型时，我们需要提供一个具有所需容量的回放缓冲区：

```
capacity = 100 # maximum size of the replay buffer
ExperienceTuple = Tuple{Float64,Float64,Float64,Float64}
M = CircularBuffer{ExperienceTuple}(capacity) # replay buffer
m_grad = 20 # batch size
model = ReplayGradientQLearning(𝒫.𝒜, 𝒫.γ, Q, ∇Q, θ, α, M, m, m_grad)
```

我们可以改变梯度更新之间的步数m以及每次模拟的深度d。在所示图表中，我们将所有训练运行限制为每次迭代$md = 30$个经验元组。这表明，足够深度的回放对于训练成功是必要的。此外，极少次数的过度深度的回放表现不如适度次数的适度深度的回放。



示例17.4。将经验回放应用于带有Q-学习和动作值近似的简单调节器问题。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

17.9 Exercises

练习 17.1。给定以下样本集，对均值进行两次增量估计：一次使用学习率 $\alpha = 0.1$，一次使用学习率 $\alpha = 0.5$。在两种情况下，均使用等于第一个样本的初始均值：

$$x^{(1:5)} = \{1.0, 1.8, 2.0, 1.6, 2.2\}$$

Solution: 我们在第一次迭代时将均值设为第一个样本，并使用方程（17.6）逐步估计均值：

$$\begin{aligned} \hat{x}_1 &= 1.0 & \hat{x}_1 &= 1.0 \\ \hat{x}_2 &= 1.0 + 0.1(1.8 - 1.0) = 1.08 & \hat{x}_2 &= 1.0 + 0.5(1.8 - 1.0) = 1.4 \\ \hat{x}_3 &= 1.08 + 0.1(2.0 - 1.08) = 1.172 & \hat{x}_3 &= 1.4 + 0.5(2.0 - 1.4) = 1.7 \\ \hat{x}_4 &= 1.172 + 0.1(1.6 - 1.172) \approx 1.215 & \hat{x}_4 &= 1.7 + 0.5(1.6 - 1.7) = 1.65 \\ \hat{x}_5 &= 1.215 + 0.1(2.2 - 1.215) \approx 1.313 & \hat{x}_5 &= 1.65 + 0.5(2.2 - 1.65) = 1.925 \end{aligned}$$

练习 17.2. 继续之前的练习，假设在两种方法各自使用五个样本估计出均值后，我们又获得了一个额外的样本 $x^{(6)}$，用作估计均值的最终样本。在两种增量估计方法（即 $\alpha = 0.1$ 或 $\alpha = 0.5$）中，哪一种更可取?

Solution: 虽然我们不知道样本会是什么，或者过程的潜在均值是多少，但我们可能更倾向于使用$\alpha = 0.5$的第二个增量估计均值。由于我们只剩下一个样本，第一个学习率太小，无法显著改变均值，而第二个学习率足够大，能够做出响应，同时不忽视过去的样本。考虑两种情况：

1. 如果我们假设下一个样本大约等于所有先前样本的*incremental*平均值，那么我们有 $x^{(6)} \approx \hat{x}_5$。因此，对均值进行增量更新不会改变我们的估计。对于学习率0.1，我们有 $\hat{x}_6 \approx 1.313$，对于学习率0.5，我们有$\hat{x}_6 = 1.925$。

2. 如果我们假设下一个样本近似等于所有先前样本的 *exact* 均值，那么我们得到 $x^{(6)} \approx 1.72$。使用学习率 0.1 的更新得到$\hat{x}_6 \approx 1.354$，而使用学习率 0.5 的更新得到$\hat{x}_6 \approx 1.823$。

在这两种情况下，假设下一个样本等于所有先前样本的平均值，那么学习率为0.5的估计会更准确。

练习17.3。考虑通过离散化动作空间，将带有函数逼近的Q-学习应用于连续动作空间的问题。假设连续动作空间位于$\mathbb{R}^n$中，比如一个拥有n个执行器的机器人，并且每个维度被离散化为m个区间。那么，在得到的离散动作空间中有多少个动作？对于具有许多维度的连续问题，带有函数逼近的Q-学习是否非常适合?

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: 一个动作空间具有n个维度，每个维度有m个区间，这导致m^n个离散动作。离散动作的数量以n为指数增长。即使m很小，较大的n值也可能迅速导致非常高的动作计数。因此，采用函数逼近的Q学习并不适合处理具有许多动作维度的连续问题。

练习 17.4. 如果我们与环境交互 d 个时间步，Q 学习的复杂度是多少？如果我们与环境交互 d 个时间步，Sarsa 的复杂度是多少？

Solution: 对于 Q-学习，我们的更新规则是

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left(r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right)$$

在每个时间步，我们必须对动作进行最大化操作，因此对于 d 个时间步，Q 学习的复杂度为 $O(d|\mathcal{A}|)$。对于 Sarsa，我们的更新规则是

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left(r + \gamma Q(s',a') - Q(s,a)\right) \tag{17.23}$$

在每个时间步，与 Q 学习不同，我们不必对动作执行最大化操作，因此对于 d 个时间步，Sarsa 的复杂度简单地是 $O(d)$。

练习17.5。Sarsa每个经验元组的计算复杂度（s_t，a_t，r_t，s_{t+1}）与Sarsa(λ)相比，是更高还是更低?

Solution: 对于 Sarsa，我们的更新规则是

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left(r + \gamma Q(s',a') - Q(s,a)\right) \tag{17.24}$$

因此，对于每个经验元组，我们具有$O(1)$的复杂度。对于Sarsa(λ)，我们的更新规则是

$$\begin{aligned}
\delta &\leftarrow r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \\
N(s_t, a_t) &\leftarrow N(s_t, a_t) + 1 \\
Q(s,a) &\leftarrow Q(s,a) + \alpha\delta N(s,a) \qquad \text{for all } s, a \\
N(s,a) &\leftarrow \gamma\lambda N(s,a) \qquad \text{for all } s, a
\end{aligned}$$

对于每个经验元组，我们需要计算 δ 并在 (s_t, a_t) 处增加访问计数，这两者均为 $O(1)$。然而，我们需要更新所有状态和动作的动作价值函数和访问计数，这两者也均为 $O(|\mathcal{S}||\mathcal{A}|)$。因此，对于 Sarsa($\lambda$)，每个经验元组的计算复杂度更高。但 Sarsa(λ) 通常使用较少的经验元组就能收敛。

练习 17.6. 当 $\lambda \to$ 趋于 0 时，$Q(\lambda)$ 的极限行为是什么？当 $\lambda \to$ 趋于 1 时，$Q(\lambda)$ 的极限行为是什么?

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: 对于 $Q(\lambda)$，我们执行以下更新规则：

$$\begin{aligned}\delta &\leftarrow r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\\ N(s_t, a_t) &\leftarrow N(s_t, a_t) + 1\\ Q(s, a) &\leftarrow Q(s, a) + \alpha\delta N(s, a) \qquad \text{for all } s, a\\ N(s, a) &\leftarrow \gamma\lambda N(s, a) \qquad \text{for all } s, a\end{aligned}$$

在$\lambda \to$趋于0的极限情况下，对于我们的第一次迭代，我们计算时间差分误差δ，并增加访问计数$N(s_t, a_t)$。在动作价值函数更新中，唯一非零的$N(s, a)$位于$N(s_t, a_t)$，因此我们执行$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\delta N(s_t, a_t)$。最后，我们将所有访问计数重置为零。由此可以看出，在$\lambda \to$趋于0的极限情况下，我们没有资格迹，并且正在执行一个直接的Q-学习更新。

在$\lambda \to$趋近于1的极限情况下，我们的访问计数会累积，并且我们拥有完整的资格轨迹，这将会把奖励传播到所有先前访问过的状态-动作对上。

练习 17.7。按照轨迹后，使用 Sarsa(λ) 计算 $Q(s、a)$

$$(s_1, a_R, 0, s_2, a_R, 0, s_3, a_L, 10, s_2, a_R, 4, s_1, a_R)$$

使用$\alpha = 0.5$、$\lambda = 1$、$\gamma = 0.9$，并将所有位置的初始动作价值函数和访问计数设为零。假设$\mathcal{S} = \{s_1、s_2、s_3、s_4\}$以及$\mathcal{A} = \{a_L、a_R\}$。

Solution: Sarsa(λ) 更新规则是：

$$\begin{aligned}\delta &\leftarrow r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)\\ N(s_t, a_t) &\leftarrow N(s_t, a_t) + 1\\ Q(s, a) &\leftarrow Q(s, a) + \alpha\delta N(s, a) \qquad \text{for all } s, a\\ N(s, a) &\leftarrow \gamma\lambda N(s, a) \qquad \text{for all } s, a\end{aligned}$$

对于第一个经验元组，我们有 $\delta = 0 + 0.9 \times 0 - 0 = 0$，我们在 $N(s_1、a_R)$ 处增加访问计数，由于 $\delta = 0$，动作价值函数不变，并且我们更新计数。在此之后，我们有

$Q(s,a)$	s_1	s_2	s_3	s_4
a_L	0	0	0	0
a_R	0	0	0	0

$N(s,a)$	s_1	s_2	s_3	s_4
a_L	0	0	0	0
a_R	0.9	0	0	0

对于第二个经验元组，我们有 $\delta = 0$，我们在 $N(s_2、a_R)$ 处增加访问计数，由于 $\delta = 0$，动作价值函数不变，并且我们更新计数。在此之后，我们有

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

$Q(s,a)$	s_1	s_2	s_3	s_4
a_L	0	0	0	0
a_R	0	0	0	0

$N(s,a)$	s_1	s_2	s_3	s_4
a_L	0	0	0	0
a_R	0.81	0.9	0	0

对于第三个经验元组，我们有 $\delta = 10$，我们在 $N(s_3, a_L)$ 处增加访问计数，我们更新动作价值函数，并更新我们的计数。在此之后，我们有

$Q(s,a)$	s_1	s_2	s_3	s_4
a_L	0	0	5	0
a_R	4.05	4.5	0	0

$N(s,a)$	s_1	s_2	s_3	s_4
a_L	0	0	0.9	0
a_R	0.729	0.81	0	0

对于第四个经验元组，我们有 $\delta = 4 + 0.9 \times 4.05 - 4.5 = 3.145$，我们在 $N(s_2$ 处增加访问计数，$a_R) = 0.81 + 1 = 1.81$，我们更新动作价值函数，并更新我们的计数。在此之后，我们有

$Q(s,a)$	s_1	s_2	s_3	s_4
a_L	0	0	6.415	0
a_R	5.196	7.346	0	0

$N(s,a)$	s_1	s_2	s_3	s_4
a_L	0	0	0.81	0
a_R	0.656	1.629	0	0

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

18 Imitation Learning

前几章假设了奖励函数已知，或者在与环境交互时接收到奖励。对于某些应用，专家可能更容易演示期望的行为，而不是指定奖励函数。本章讨论*imitation learning*的算法，其中期望的行为是从专家演示中学习得到的。我们将涵盖多种方法，从非常简单的似然最大化方法到涉及强化学习的更复杂的迭代方法。[1]

1 其他方法和应用由A. Hussein、M. M. Gaber、E. Elyan和C. Jayne在“模仿学习：学习方法综述”中进行了综述，*ACM Comput-ing Surveys*，第50卷，第2期，第1–35页，2017年。

18.1 Behavioral Cloning

一种简单的模仿学习形式是将其视为监督学习问题。这种方法称为*behavioral cloning*，[2]，它训练一个随机策略π_θ，其参数化通过 θ 来最大化来自专家状态-动作对数据集 $\mathcal{D}$ 的动作似然：

2 D. A. Pomerleau，“用于自主导航的人工神经网络高效训练”，*Neu-ral Computation*，第3卷，第1期，第88–97页，1991年。

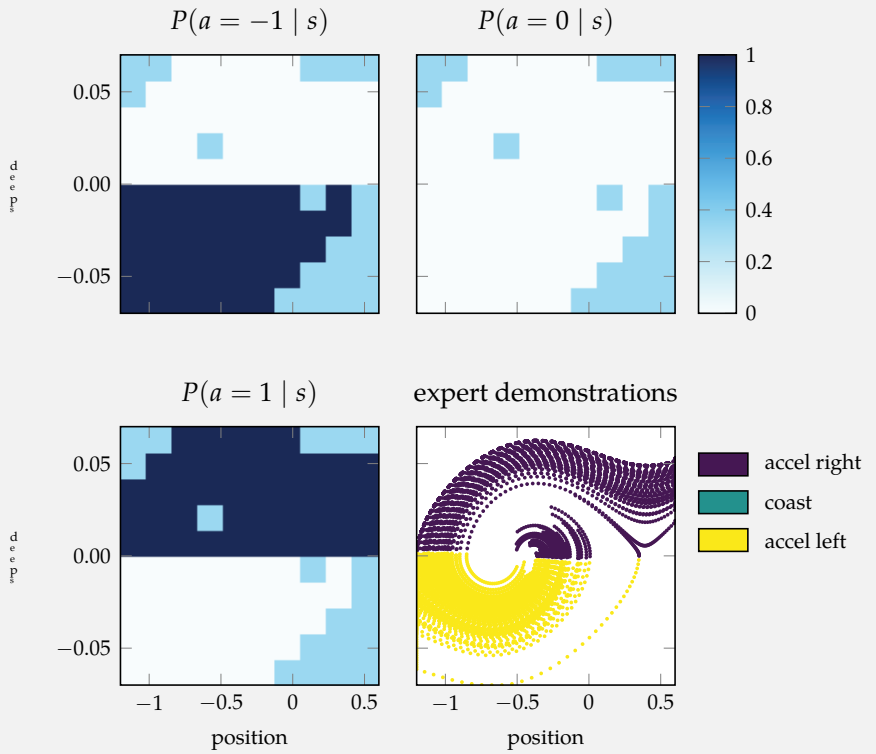
$$\underset{\theta}{\text{maximize}} \prod_{(s,a)\in\mathcal{D}} \pi_\theta(a \mid s) \tag{18.1}$$

正如前面章节所做的那样，我们可以将关于 $\pi_\theta(a \mid s)$ 的乘积形式的最大化转换为关于 $\log \pi_\theta(a \mid s)$ 的求和形式。

根据我们想要如何表示条件分布 $\pi_\theta(a \mid s)$，我们可以解析地计算 θ 的最大似然估计。例如，如果我们使用离散条件模型（第2.4节），θ 将由来自 $\mathcal{D}$ 和 $\pi_\theta(a \mid s) = N(s \text{ 的计数 } N(s、a)，\text{ 以及 } a)/\sum_a N(s、a)$ 组成。例18.1将离散条件模型应用于山地车问题的数据。

如果我们的策略具有因子化表示，我们可以使用贝叶斯网络来表示状态和动作变量的联合分布。图18.1展示了一个示例。我们可以同时学习结构和

考虑对山地车问题（附录F.4）中的专家演示使用行为克隆。我们获得了专家策略的10次滚动。我们拟合了一个条件分布并绘制了结果。连续轨迹被离散化，位置和速度各分为10个区间。



示例18.1. 山区汽车问题中行为克隆的应用演示。浅蓝色区域表示缺乏训练数据的区域，当代理遇到这些状态时，导致策略表现不佳。

状态空间并未被专家演示完全覆盖，这是模仿学习问题的典型特征。由此产生的策略在有覆盖的区域使用时可能表现良好，但在无覆盖的区域会对动作分配均匀分布。即使我们从有覆盖的区域开始，由于环境中的随机性，也可能转移到无覆盖的区域。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

参数（第4章）从数据$\mathcal{D}$中得出。给定当前状态，我们可以使用前面讨论的推断算法之一（第3章）来推断动作的分布。

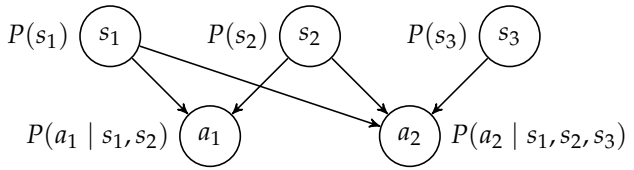


图18.1。贝叶斯网络可用于表示状态和动作变量上的联合分布。给定状态变量的当前值，我们可以应用推断算法生成动作上的分布。

我们可以为π_θ使用许多其他表示。例如，我们可能希望使用神经网络，其中输入对应于状态变量的值，输出对应于动作空间上分布的参数。如果我们的表示是可微的，就像神经网络那样，我们可以尝试使用梯度上升来优化方程(18.1)。这种方法在算法18.1中实现。

```
struct BehavioralCloning
    α     # step size
    k_max # number of iterations
    ∇logπ # log likelihood gradient
end

function optimize(M::BehavioralCloning, D, θ)
    α, k_max, ∇logπ = M.α, M.k_max, M.∇logπ
    for k in 1:k_max
        ∇ = mean(∇logπ(θ, a, s) for (s,a) in D)
        θ += α*∇
    end
    return θ
end
```

算法18.1. 一种从专家演示中学习参数化随机策略的方法，这些演示以一组状态-动作元组 `D` 的形式给出。策略参数化向量 `θ` 通过最大化给定状态下动作的对数似然来迭代改进。行为克隆需要步长 `α`、迭代次数 `k_max` 和对数似然梯度 `∇logπ`。

专家演示越接近最优，由此得到的行为克隆策略表现就越好。[3] 然而，行为克隆存在

从*cascading errors*开始。正如例18.2所讨论的，在模拟过程中，微小的不准确性会累积，最终导致训练数据中表示不佳的状态，从而导致更差的决策，最终导致无效或未见过的状况。尽管行为克隆因其简单性而颇具吸引力，但级联错误导致该方法在许多问题上表现不佳，特别是在策略需要应用于长时间范围的情况下。

3 U. Syed 和 R. E. Schapire，“从学徒学习到分类的归约”，载于 *Advances in Neural Information Processing Systems (NIPS)*，2010年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

考虑将行为克隆应用于训练自动驾驶赛车的策略。一位人类赛车手提供了专家演示。作为专家，驾驶员从不会偏离到草地上或过于靠近栏杆。使用行为克隆训练的模型在靠近栏杆或偏离到草地上时没有可用的信息，因此它不知道如何恢复。

示例18.2。行为克隆方法中固有泛化问题的简要示例。

18.2 Data Set Aggregation

解决级联错误问题的一种方法是使用额外的专家输入来修正已训练的策略。*Sequential interactive demonstration* 方法在从专家收集由已训练策略生成的情境数据与使用这些数据改进该策略之间交替进行。

一种顺序交互演示方法称为 *data set aggregation* (*DAgger*) (算法 18.2)。[4] 它首先训练一个随机策略

使用行为克隆。然后使用该策略从初始状态分布b运行若干次推演，然后将这些推演提供给专家，为每个状态提供正确的动作。新数据与先前的数据集聚合，并训练新的策略。例18.3说明了这一过程。

4 S. Ross、G. J. Gordon 和 J. A. Bagnell，“将模仿学习和结构化预测简化为无遗憾在线学习”，载于 *International Conference on Artificial Intelligence and Statistics* (*AISTATS*)，2011年。

这些交互式演示基于先前的学习迭代，迭代地构建一个数据集，涵盖智能体可能遇到的状态空间区域。随着每次迭代，新添加的示例在数据集中的占比越来越小，从而导致策略变化较小。虽然顺序交互式演示在实践中效果良好，但不能保证收敛。可以证明，混合专家策略的影响可以保证收敛，这是下一节的主题。

18.3 Stochastic Mixing Iterative Learning

顺序交互方法也可以通过随机混合新训练的策略来迭代构建策略。其中一种方法是*stochastic mixingiterative learning* (*SMILe*) (算法18.3)。[5]它在每个

迭代，但将新训练的策略与之前的策略混合。

我们从专家策略 $\pi^{(1)} = \pi_E$ 开始。[6] 在每次迭代中，我们执行

5 S. Ross 和 J. A. Bagnell，“模仿学习的高效简化”，载于 *International Conference on Artificial Intelligence and Statistics* (*AISTATS*)，2010年。

6 我们没有π_E的显式表示。评估π_E需要像上一节那样交互式地查询专家。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct DataSetAggregation
    𝒫     # problem with unknown reward function
    bc    # behavioral cloning struct
    k_max # number of iterations
    m     # number of rollouts per iteration
    d     # rollout depth
    b     # initial state distribution
    πE    # expert
    πθ    # parameterized policy
end

function optimize(M::DataSetAggregation, D, θ)
    𝒫, bc, k_max, m = M.𝒫, M.bc, M.k_max, M.m
    d, b, πE, πθ = M.d, M.b, M.πE, M.πθ
    θ = optimize(bc, D, θ)
    for k in 2:k_max
        for i in 1:m
            s = rand(b)
            for j in 1:d
                push!(D, (s, πE(s)))
                a = rand(πθ(θ, s))
                s = rand(𝒫.T(s, a))
            end
        end
        θ = optimize(bc, D, θ)
    end
    return θ
end
```

算法18.2. DAgger方法，即数据集聚合方法，用于从专家演示中学习随机参数化策略。该方法采用初始状态-动作元组数据集`D`、随机参数化策略`πθ(θ, s)`、定义了转移函数的MDP $\mathcal{P}$以及初始状态分布`b`。在每次迭代中使用行为克隆（算法18.1）来改进策略。

一个专家策略 `πE` 为从最新学习策略中采样的轨迹打标签，以扩充数据集。原始论文通过随机混合专家策略来生成轨迹。因此，此实现即为混合值为零的原始 DAg-ger。

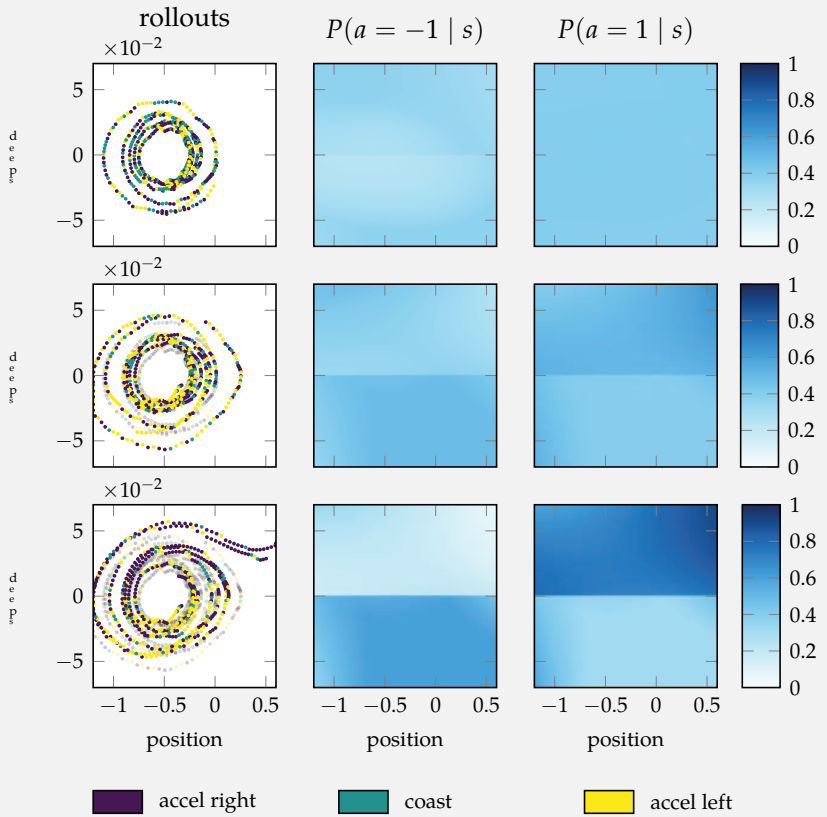
在实践中，专家策略并不存在，对该策略的调用将被替换为向人类专家的查询。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

考虑使用DAgger在山地车问题上训练策略，其中奖励未被观测到。我们使用一个在行进方向上加速的专家策略。在此示例中，我们使用以下特征来训练策略：

$$\mathbf{f}(s) = [1[v > 0], 1[v < 0], x, x^2, v, v^2, xv]$$

其中 x 和 v 分别是汽车的位置和速度。



例18.3. 将DAgger应用于山地车问题，迭代自上而下进行。随着时间推移，数据集中积累的轨迹逐渐增多。代理的行为在每次迭代后都有所改善。

轨迹根据动作进行着色。在第一次迭代中，智能体随机行动，无法朝着目标取得进展（$x \geq 0.6$）。随着迭代次数增加，智能体学会模仿沿行进方向加速的专家策略。这种行为在新轨迹中表现明显，轨迹呈螺旋状向外扩展，而策略在$v > 0$时给$a = 1$分配高似然，在$v < 0$时给$a = -1$分配高似然。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

使用最新的策略 $\pi^{(k)}$ 生成新的数据集，并咨询专家以提供正确的动作。仅对这个新数据集应用行为克隆来训练一个新的 *component policy* $\hat{\pi}^{(k)}$。这个组件策略与之前迭代的组件策略混合，生成新的策略 $\pi^{(k+1)}$。

组件策略的混合以生成$\pi^{(k+1)}$由混合标量$\beta \in (0, 1)$控制。依据专家策略行动的概率为$(1-\beta)^k$，而依据$\hat{\pi}^{(i)}$行动的概率为$\beta(1-\beta)^{i-1}$。该方案在假设较旧策略组件是在最可能遇到的状态上训练的前提下，给予较旧策略更多权重。[7]随着每次迭代，

按照原始专家策略行动的概率衰减为零。混合标量通常很小，以便代理不会过快地放弃专家的策略。例18.4通过山地车问题演示了这种方法。

7 在SMILe中，我们依据最近学到的策略行动。我们期望这一学习策略能与专家策略相当吻合，并且主要在我们偏离专家策略时出现误预测。学习到的组件策略通常只需在每次迭代中贡献越来越小的部分，以弥补尚未学到的差异。

18.4 Maximum Margin Inverse Reinforcement Learning

在许多应用场景中，我们无法交互式地咨询专家；相反，我们拥有一批专家演示轨迹。我们将假设专家演示数据 $\mathcal{D}$ 由 m 条轨迹组成。$\mathcal{D}$ 中的每条轨迹 τ 都涉及深度为 d 的展开。在 *inverse reinforcement learning* 中，我们假设专家正在优化一个未知的奖励函数。根据 $\mathcal{D}$，我们尝试推导出该奖励函数。有了这个奖励函数，我们可以使用前面章节讨论的方法来推导出最优策略。

逆强化学习有多种方法。我们通常需要定义奖励函数的参数化。一个常见的假设是该参数化是线性的，使用$R_{\boldsymbol{\phi}}(s、a) = \boldsymbol{\phi}^\top \boldsymbol{\beta}(s、a)$，其中$\boldsymbol{\beta}(s、a)$是特征向量，$\boldsymbol{\phi}$是权重向量。本节我们将聚焦于一种称为 *maximum margin inverse reinforcement learning*、[8]的方法，其中

假设特征是二进制的。由于最优策略在奖励函数正比例缩放时仍保持最优，该方法还额外约束权重向量，使得 $\|\boldsymbol{\phi}\|_2 \leq 1$。专家数据以不同频率激活每个二进制特征，可能是追求某些特征而避免其他特征。这种方法试图学习这种激活模式，并训练智能体模仿这些激活频率。

8 P. Abbeel 和 A. Y. Ng，“通过逆向强化学习的学徒学习”，载于 *International Conference on Machine Learning (ICML)*，2004年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

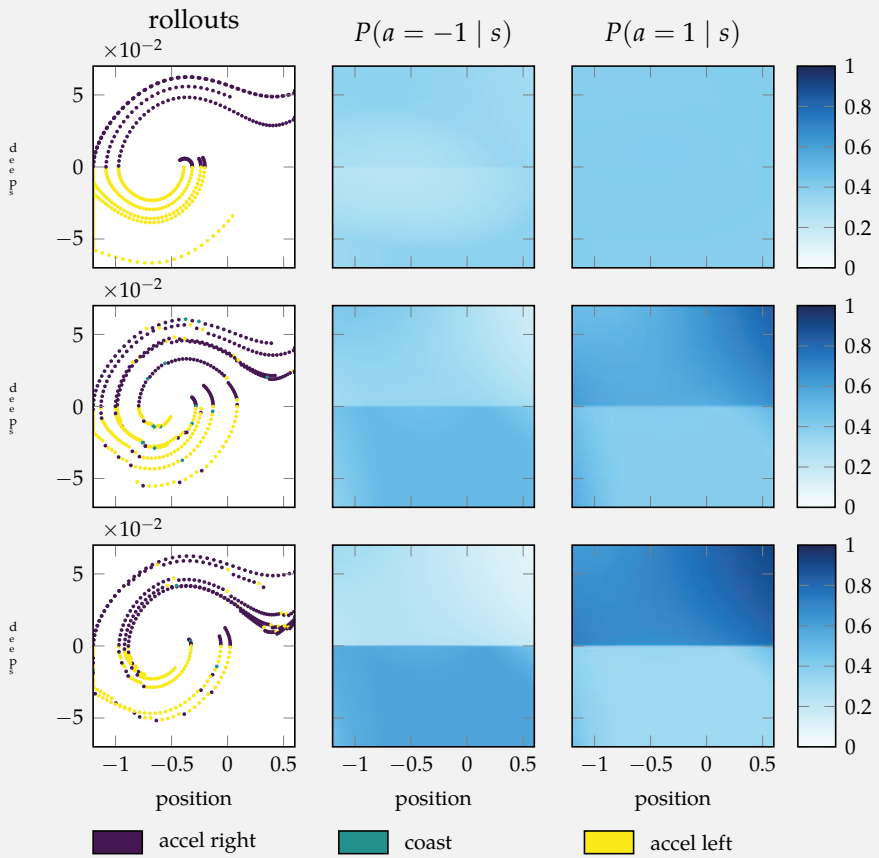
```
struct SMILe
    𝒫     # problem with unknown reward
    bc    # Behavioral cloning struct
    k_max # number of iterations
    m     # number of rollouts per iteration
    d     # rollout depth
    b     # initial state distribution
    β     # mixing scalar (e.g., d^-3)
    πE    # expert policy
    πθ    # parameterized policy
end

function optimize(M::SMILe, θ)
    𝒫, bc, k_max, m = M.𝒫, M.bc, M.k_max, M.m
    d, b, β, πE, πθ = M.d, M.b, M.β, M.πE, M.πθ
    𝒜, T = 𝒫.𝒜, 𝒫.T
    θs = []
    π = s -> πE(s)
    for k in 1:k_max
        # execute latest π to get new data set D
        D = []
        for i in 1:m
            s = rand(b)
            for j in 1:d
                push!(D, (s, πE(s)))
                a = π(s)
                s = rand(T(s, a))
            end
        end
        # train new policy classifier
        θ = optimize(bc, D, θ)
        push!(θs, θ)
        # compute a new policy mixture
        Pπ = Categorical(normalize([(1-β)^(i-1) for i in 1:k],1))
        π = s -> begin
            if rand() < (1-β)^(k-1)
                return πE(s)
            else
                return rand(Categorical(πθ(θs[rand(Pπ)], s)))
            end
        end
    end
    Ps = normalize([(1-β)^(i-1) for i in 1:k_max],1)
    return Ps, θs
end
```

算法 18.3. SMILe 算法用于从专家演示中为 MDP $\mathcal{P}$ 训练随机参数化策略。它依次以越来越小的权重混合新的组件策略，同时降低按照专家策略行事的概率。该方法返回组件策略的概率 `Ps` 和参数化 `θs`。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

考虑使用SMILe在未观察到奖励的山地车问题中训练策略。我们使用与例18.3中DAgger相同的特征。DAgger和SMILe在每次迭代中都会收到一个新的专家标注数据集。SMILe不是累积更大的专家标注数据集，而是仅使用最新数据训练一个新的策略组件，并将新策略组件与先前的策略组件混合。



示例18.4。使用SMILe学习山地车问题的策略。与例18.3中的DAgger不同，SMILe在rollouts过程中将专家融入策略中。这个专家组件的影响力随着每次迭代而减弱，使得初始的rollouts能更好地向目标推进。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

该算法的一个重要部分涉及在策略 π、权重 $\boldsymbol{\phi}$ 和初始状态分布 b 下对期望回报进行推理：

$$\begin{aligned}
\mathop{\mathbb{E}}_{s\sim b}[U(s)] &= \mathbb{E}_\tau\left[\sum_{k=1}^{d}\gamma^{k-1}R_{\boldsymbol{\phi}}(s^{(k)},a^{(k)})\right] && (18.2)\\
&= \mathbb{E}_\tau\left[\sum_{k=1}^{d}\gamma^{k-1}\boldsymbol{\phi}^\top\boldsymbol{\beta}(s^{(k)},a^{(k)})\right] && (18.3)\\
&= \boldsymbol{\phi}^\top\left(\mathbb{E}_\tau\left[\sum_{k=1}^{d}\gamma^{k-1}\boldsymbol{\beta}(s^{(k)},a^{(k)})\right]\right) && (18.4)\\
&= \boldsymbol{\phi}^\top\boldsymbol{\mu}_\pi && (18.5)
\end{aligned}$$

其中 τ 对应于由 π 生成到深度 d 的轨迹。在这里，我们引入 *feature expectations* 向量 $\boldsymbol{\mu}_\pi$，它是预期的折现累积特征值。这些特征期望可以从 m 滚动中估计，如算法18.4中所实现。

```
struct InverseReinforcementLearning
    𝒫  # problem
    b  # initial state distribution
    d  # depth
    m  # number of samples
    π  # parameterized policy
    β  # binary feature mapping
    μE # expert feature expectations
    RL # reinforcement learning method
    ϵ  # tolerance
end

function feature_expectations(M::InverseReinforcementLearning, π)
    𝒫, b, m, d, β, γ = M.𝒫, M.b, M.m, M.d, M.β, M.𝒫.γ
    μ(τ) = sum(γ^(k-1)*β(s, a) for (k,(s,a)) in enumerate(τ))
    τs = [simulate(𝒫, rand(b), π, d) for i in 1:m]
    return mean(μ(τ) for τ in τs)
end
```

算法18.4. 逆向强化学习的结构及从轨迹中估计特征期望向量的方法。

我们可以使用专家演示来估计专家特征期望 $\boldsymbol{\mu}_E$，并且我们希望找到一个尽可能接近这些特征期望的策略。在第一次迭代中，我们从随机策略 $\pi^{(1)}$ 开始，并估计其特征期望，记为 $\boldsymbol{\mu}^{(1)}$。在迭代 k 时，我们找到一个新的 $\boldsymbol{\phi}^{(k)}$，对应于奖励函数 $R_{\boldsymbol{\phi}^{(k)}}(s、a) = \boldsymbol{\phi}^{(k)\top}\boldsymbol{\beta}(s、a)$，使得

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

专家超越所有之前找到的策略，以最大的*margin* t：

$$\begin{aligned} \underset{t,\boldsymbol{\phi}}{\text{maximize}} \quad & t \\ \text{subject to} \quad & \boldsymbol{\phi}^\top \boldsymbol{\mu}_E \geq \boldsymbol{\phi}^\top \boldsymbol{\mu}^{(i)} + t \text{ for } i = 1, \ldots, k-1 \\ & \|\boldsymbol{\phi}\|_2 \leq 1 \end{aligned} \tag{18.6}$$

方程（18.6）是一个易于求解的二次规划问题。接着，我们利用奖励函数$R(s、a) = \boldsymbol{\phi}^{(k)\top}\boldsymbol{\beta}(s、a)$求解出新的策略$\pi^{(k)}$，并生成新的特征期望向量。图18.2展示了这一边际最大化过程。

我们迭代直到边际足够小，其中$t \leq \epsilon$。在收敛时，我们可以求解一个混合策略，该策略试图使特征期望尽可能接近专家策略的特征期望：

$$\begin{aligned} \underset{\boldsymbol{\lambda}}{\text{minimize}} \quad & \|\boldsymbol{\mu}_E - \boldsymbol{\mu}_{\boldsymbol{\lambda}}\|_2 \\ \text{subject to} \quad & \boldsymbol{\lambda} \geq 0 \\ & \|\boldsymbol{\lambda}\|_1 = 1 \end{aligned} \tag{18.7}$$

其中 $\boldsymbol{\mu}_{\boldsymbol{\lambda}} = \sum_i \lambda_i \boldsymbol{\mu}^{(i)}$。混合权重 $\boldsymbol{\lambda}$ 结合了每次迭代中找到的策略。以概率 λ_i，我们遵循策略 $\pi^{(i)}$。最大间隔逆强化学习在算法18.5中实现。

18.5 Maximum Entropy Inverse Reinforcement Learning

上一节的逆强化学习方法存在欠定问题，这意味着通常会有多个策略能够产生与专家演示相同的特征期望。本节介绍
maximum entropy inverse reinforcement learning，它通过优先选择使轨迹分布具有最大 *entropy* (附录 A.8) 的策略来避免这种歧义。[9] 该问题可以转化为

在给定专家数据 $\mathcal{D}$ 的情况下，在最大似然估计问题中找到最佳奖励函数参数 $\boldsymbol{\phi}$。

9 B. D. Ziebart, A. Maas, J. A. Bagnell, 和 A. K. Dey, “最大熵逆强化学习”，载于 *AAAI Conference on Artificial Intelligence (AAAI)*，2008 年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

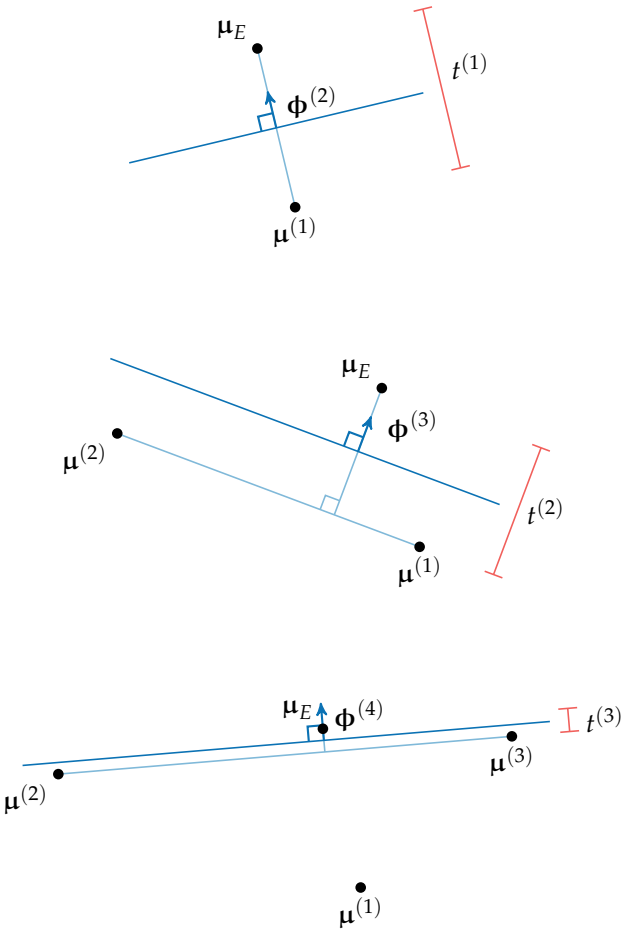


图18.2。最大间隔逆强化学习算法三次示例迭代的几何可视化，从上到下。在每次迭代中，新的权重向量指向垂直于超平面的方向，该超平面以最大可能间隔将专家特征期望向量与前一策略的特征期望向量分开。间隔随着每次迭代而减小。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
function calc_weighting(M::InverseReinforcementLearning, μs)
    μE = M.μE
    k = length(μE)
    model = Model(Ipopt.Optimizer)
    @variable(model, t)
    @variable(model, ϕ[1:k] ≥ 0)
    @objective(model, Max, t)
    for μ in μs
        @constraint(model, ϕ⋅μE ≥ ϕ⋅μ + t)
    end
    @constraint(model, ϕ⋅ϕ ≤ 1)
    optimize!(model)
    return (value(t), value.(ϕ))
end

function calc_policy_mixture(M::InverseReinforcementLearning, μs)
    μE = M.μE
    k = length(μs)
    model = Model(Ipopt.Optimizer)
    @variable(model, λ[1:k] ≥ 0)
    @objective(model, Min, (μE - sum(λ[i]*μs[i] for i in 1:k))⋅
                           (μE - sum(λ[i]*μs[i] for i in 1:k)))
    @constraint(model, sum(λ) == 1)
    optimize!(model)
    return value.(λ)
end

function optimize(M::InverseReinforcementLearning, θ)
    π, ϵ, RL = M.π, M.ϵ, M.RL
    θs = [θ]
    μs = [feature_expectations(M, s→π(θ,s))]
    while true
        t, ϕ = calc_weighting(M, μs)
        if t ≤ ϵ
            break
        end
        copyto!(RL.ϕ, ϕ) # R(s,a) = ϕ⋅β(s,a)
        θ = optimize(RL, π, θ)
        push!(θs, θ)
        push!(μs, feature_expectations(M, s→π(θ,s)))
    end
    λ = calc_policy_mixture(M, μs)
    return λ, θs
end
```

算法18.5。最大间隔逆强化学习，该方法计算一个混合策略，使其特征期望与给定专家演示的特征期望相匹配。我们使用`JuMP.jl`来求解约束优化问题。该实现要求所提供的强化学习结构具有一个权重向量`ϕ`，该权重向量可以用新值更新。该方法返回组件策略的随机权重`λ`和参数化`θs`。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

任何策略π都会在轨迹[10] $P_\pi(\tau)$上诱导出一个分布。不同的策略产生不同的轨迹分布。我们可以自由选择任何符合专家特征期望的轨迹分布。*principle of maximum entropy*选择的是最不具有信息量的分布，即熵最大的一种。[11] 可以证明，最不信息丰富的轨迹分布采用以下形式：

$$P_{\boldsymbol{\phi}}(\tau) = \frac{1}{Z(\boldsymbol{\phi})} \exp(R_{\boldsymbol{\phi}}(\tau)) \tag{18.8}$$

其中 $P_{\boldsymbol{\phi}}(\tau)$ 是给定奖励参数 $\boldsymbol{\phi}$ 时轨迹 τ 的似然，并且

$$R_{\boldsymbol{\phi}}(\tau) = \sum_{k=1}^{d} \gamma^{k-1} R_{\boldsymbol{\phi}}(s^{(k)}, a^{(k)}) \tag{18.9}$$

是折扣轨迹奖励。除了可微性之外，我们对 $R_{\boldsymbol{\phi}}(s^{(k)}、a^{(k)})$ 的参数化不做任何假设，允许神经网络等表示形式。归一化标量 $Z(\boldsymbol{\phi})$ 确保概率之和为1：

$$Z(\boldsymbol{\phi}) = \sum_{\tau} \exp(R_{\boldsymbol{\phi}}(\tau)) \tag{18.10}$$

对所有可能的轨迹求和。

我们为策略选择了一类特定的轨迹分布。现在，我们使用最大似然估计来使该类分布拟合我们的轨迹数据，从而获得最能描述我们数据的参数：

$$\max_{\boldsymbol{\phi}} f(\boldsymbol{\phi}) = \max_{\boldsymbol{\phi}} \sum_{\tau \in \mathcal{D}} \log P_{\boldsymbol{\phi}}(\tau) \tag{18.11}$$

我们可以重写方程（18.11）中的目标函数$f(\boldsymbol{\phi})$：

$$f(\boldsymbol{\phi}) = \sum_{\tau \in \mathcal{D}} \log \frac{1}{Z(\boldsymbol{\phi})} \exp(R_{\boldsymbol{\phi}}(\tau)) \tag{18.12}$$

$$= \left(\sum_{\tau \in \mathcal{D}} R_{\boldsymbol{\phi}}(\tau) \right) - |\mathcal{D}| \log Z(\boldsymbol{\phi}) \tag{18.13}$$

$$= \left(\sum_{\tau \in \mathcal{D}} R_{\boldsymbol{\phi}}(\tau) \right) - |\mathcal{D}| \log \sum_{\tau} \exp(R_{\boldsymbol{\phi}}(\tau)) \tag{18.14}$$

10 为简化起见，本节假设有限视界且状态和动作空间离散，使得 {v*} 成为概率质量。为了将最大熵逆强化学习扩展到动力学可能未知的连续状态和动作空间问题，请考虑引导成本学习。C. Finn, S. Levine, and P. Abbeel, “Guided Cost Learning: Deep Inverse Optimal Control via Policy Optimization,” in *International Conference on Machine Learning (ICML)*, 2016.

11 关于这一原理的介绍，参见E. T. Jaynes, “信息论与统计力学”, *Physical Review*, 第106卷, 第4期, 第620–630页, 1957年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们可以尝试通过梯度上升来优化这个目标函数。f的梯度是

$$\nabla_{\phi} f = \left(\sum_{\tau \in \mathcal{D}} \nabla_{\phi} R_{\phi}(\tau)\right) - \frac{|\mathcal{D}|}{\sum_{\tau} \exp(R_{\phi}(\tau))} \sum_{\tau} \exp(R_{\phi}(\tau)) \nabla_{\phi} R_{\phi}(\tau) \tag{18.15}$$

$$= \left(\sum_{\tau \in \mathcal{D}} \nabla_{\phi} R_{\phi}(\tau)\right) - |\mathcal{D}| \sum_{\tau} P_{\phi}(\tau) \nabla_{\phi} R_{\phi}(\tau) \tag{18.16}$$

$$= \left(\sum_{\tau \in \mathcal{D}} \nabla_{\phi} R_{\phi}(\tau)\right) - |\mathcal{D}| \sum_{s} b_{\gamma,\phi}(s) \sum_{a} \pi_{\phi}(a \mid s) \nabla_{\phi} R_{\phi}(s, a) \tag{18.17}$$

如果奖励函数是线性的，其中 $R_{\phi}(s、a) = \phi^{\top} \beta(s、a)$ 如上一节所述，那么 $\nabla_{\phi} R_{\phi}(s、a)$ 就是 $\beta(s、a)$。

更新参数向量 ϕ 因此需要折现状态访问频率 $b_{\gamma,\phi}$ 和当前参数向量下的最优策略 $\pi_{\phi}(a \mid s)$。我们可以通过运行强化学习来获得最优策略。为了计算折现状态访问频率，我们可以采用模拟轨迹或动态规划方法。

如果我们采用动态规划方法来计算折扣状态访问频率，我们可以从初始状态分布$b^{(1)}_{\gamma\phi} = b(s)$开始，并随时间迭代向前推进：

$$b^{(k+1)}_{\gamma,\phi}(s) = \gamma \sum_{a} \sum_{s'} b^{(k)}_{\gamma,\phi}(s) \pi(a \mid s) T(s' \mid s, a) \tag{18.18}$$

这个版本的最大熵逆强化学习在算法18.6中实现。

18.6 Generative Adversarial Imitation Learning

在*generative adversarial imitation learning* (*GAIL*)、[12]中，我们优化一个可微分的参数化策略π_{θ}，通常由神经网络表示。我们不提供奖励函数，而是使用*adversarial learning* (附录D.7)。我们还训练一个*discriminator* $C_{\phi}(s、a)$，通常也是神经网络，来返回它赋予来自学习策略的状态-动作对的概率。该过程涉及交替训练这个判别器，使其变得更好

12 J. Ho 和 S. Ermon, “生成对抗模仿学习”，载于 *Advances in Neural Information Processing Systems* (*NIPS*)，2016年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct MaximumEntropyIRL
    𝒫     # problem
    b     # initial state distribution
    d     # depth
    π     # parameterized policy π(θ,s)
    Pπ    # parameterized policy likelihood π(θ, a, s)
    ∇R    # reward function gradient
    RL    # reinforcement learning method
    α     # step size
    k_max # number of iterations
end

function discounted_state_visitations(M::MaximumEntropyIRL, θ)
    𝒫, b, d, Pπ = M.𝒫, M.b, M.d, M.Pπ
    𝒮, 𝒜, T, γ = 𝒫.𝒮, 𝒫.𝒜, 𝒫.T, 𝒫.γ
    b_sk = zeros(length(𝒫.𝒮), d)
    b_sk[:,1] = [pdf(b, s) for s in 𝒮]
    for k in 2:d
        for (si′, s′) in enumerate(𝒮)
            b_sk[si′,k] = γ*sum(sum(b_sk[si,k-1]*Pπ(θ, a, s)*T(s, a, s′)
                    for (si,s) in enumerate(𝒮))
                for a in 𝒜)
        end
    end
    return normalize!(vec(mean(b_sk, dims=2)),1)
end

function optimize(M::MaximumEntropyIRL, D, ϕ, θ)
    𝒫, π, Pπ, ∇R, RL, α, k_max = M.𝒫, M.π, M.Pπ, M.∇R, M.RL, M.α, M.k_max
    𝒮, 𝒜, γ, nD = 𝒫.𝒮, 𝒫.𝒜, 𝒫.γ, length(D)
    for k in 1:k_max
        copyto!(RL.ϕ, ϕ) # update parameters
        θ = optimize(RL, π, θ)
        b = discounted_state_visitations(M, θ)
        ∇Rτ = τ -> sum(γ^(i-1)*∇R(ϕ,s,a) for (i,(s,a)) in enumerate(τ))
        ∇f = sum(∇Rτ(τ) for τ in D) - nD*sum(b[si]*sum(Pπ(θ,a,s)*∇R(ϕ,s,a)
                    for (ai,a) in enumerate(𝒜))
                for (si, s) in enumerate(𝒮))
        ϕ += α*∇f
    end
    return ϕ, θ
end
```

算法18.6. 最大熵逆强化学习，该算法寻找一种随机策略，以在最大熵轨迹分布下最大化专家示范的似然度。此实现通过在所有状态上进行动态规划来计算预期访问次数，这要求问题是离散的。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

在于区分模拟状态-动作对与专家状态-动作对，并训练策略使其看起来与专家演示无法区分。该过程概述于图18.3。

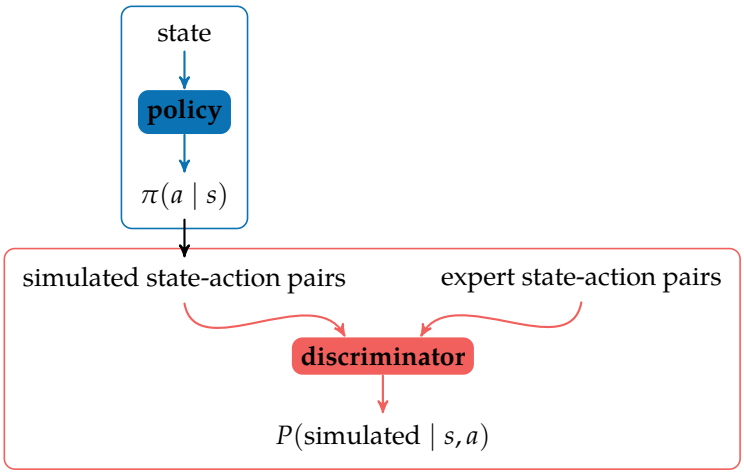


图18.3。生成对抗模仿学习不是推断奖励函数，而是优化判别器以区分模拟和专家的状态-动作对，并优化策略以使其在判别器看来无法区分。目标是最终产生一个类似专家的策略。

判别器和策略具有相反的目标。GAIL 寻求找到判别器二分类问题负对数损失的鞍点（θ，ϕ）：[13]

$$\max_{\phi} \min_{\theta} \mathbb{E}_{(s,a)\sim\pi_\theta}\left[\log(C_\phi(s,a))\right] + \mathbb{E}_{(s,a)\sim\mathcal{D}}\left[\log(1 - C_\phi(s,a))\right] \tag{18.19}$$

其中我们使用 $(s, a) \sim \mathcal{D}$ 来表示从专家数据集 $\mathcal{D}$ 所代表的分布中抽取的样本。我们可以在对 ϕ 进行梯度上升以增加目标与对 θ 进行信赖域策略优化（第12.4节）以减少目标之间交替进行，从而从策略中生成必要的轨迹样本以执行这些步骤。判别器为策略提供了一种类似于已知奖励信号的学习信号。

13 原始论文还包含以下熵项：

$$-\lambda\, \mathbb{E}_{(s,\ a)\sim\mathcal{D}}[-\text{对数}\ \pi_\theta(a \mid s)]$$

18.7 Summary

- 模仿学习涉及从专家演示中学习期望行为，而不使用奖励函数。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

- 一种模仿学习类型是行为克隆，它产生一个随机策略，最大化数据集中动作的条件似然。

- 当专家可以被多次咨询时，我们可以使用诸如数据集合聚或随机混合迭代学习等迭代方法。
- 逆强化学习涉及从专家数据中推断奖励函数，然后使用传统方法寻找最优策略。
- 最大边际逆强化学习试图找到一种策略，使其匹配专家数据集中发现的二元特征频率。
- 最大熵逆强化学习将寻找最佳奖励参数的问题视为最大似然估计问题，并尝试使用梯度上升法来求解。

- 生成对抗模仿学习通过迭代优化判别器和策略进行；判别器试图区分策略做出的决策和专家做出的决策，而策略则试图欺骗判别器。

18.8 Exercises

练习 18.1. 考虑在一个离散问题上应用行为克隆，其中我们已获得专家演示。我们可以定义一个特征函数 $\beta(s)$，并用 softmax 分布表示策略：

$$\pi(a \mid s) \propto \exp(\theta_a^\top \beta(s))$$

然后，我们将从专家数据中学习每个动作的参数θ_a。为什么我们可能想要使用这种方法，而不是为每个状态直接估计一个离散分布，其中每个状态-动作对一个参数?

Solution: 在模仿学习中，我们通常受限于相对较少的专家演示。分布$P(a \mid s)$具有$(|\mathcal{A}| - 1)|\mathcal{S}|$个必须学习的独立参数，这往往大得令人望而却步。专家演示通常仅覆盖状态空间的一小部分。即使$P(a \mid s)$能够针对所提供数据集中覆盖的状态进行可靠训练，所得策略在其他状态下将未经过训练。使用特征函数能够泛化到未见过的状态。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

练习 18.2。第 18.1 节建议使用最大似然方法从专家数据中训练策略。该方法试图找到使分配给训练示例的似然最大化的策略参数。然而，在某些问题中，我们知道将一个错误动作赋予高概率并不像将另一个错误动作赋予高概率那样糟糕。例如，在山地车问题中，当专家指定的加速度为 1 时，预测加速度为 -1 比预测加速度为 0 更糟糕。行为克隆应如何修改，以便对不同的误分类给予不同的惩罚？

Solution: 我们可以改为提供一个代价函数$C(s、a_{\text{true}}、a_{\text{pred}})$，该函数定义了当专家的动作为$a_{\text{true}}$时，针对状态$s$预测动作$a_{\text{pred}}$的代价。例如，在山地车问题中，我们可能会使用

$$C(s, a_{\text{true}}, a_{\text{pred}}) = -|a_{\text{true}} - a_{\text{pred}}|$$

这会比小偏差更严厉地惩罚大偏差。与专家动作相关的成本通常为零。

如果我们有一个随机策略 {v*}，我们则寻求在数据集上最小化成本：

$$\underset{\theta}{\text{minimize}} \quad \sum_{(s,a_{\text{true}})\in\mathcal{D}} \sum_{a_{\text{pred}}} C\left(s, a_{\text{true}}, a_{\text{pred}}\right) \pi\left(a_{\text{pred}} \mid s\right)$$

这种技术称为*cost-sensitive classification*。[14] 成本敏感分类的一个好处是分类的优点在于，我们可以使用各种各样的现成分类模型，如k-最近邻、支持向量机或决策树，来训练策略。

14 C. Elkan，“成本敏感学习的基础”，载于 *Inter-*，2001年。*national Joint Conference on Artificial Intelligence* (*IJCAI*)

练习18.3。举一个例子，说明最大边际逆强化学习不能唯一确定最优策略的情况。

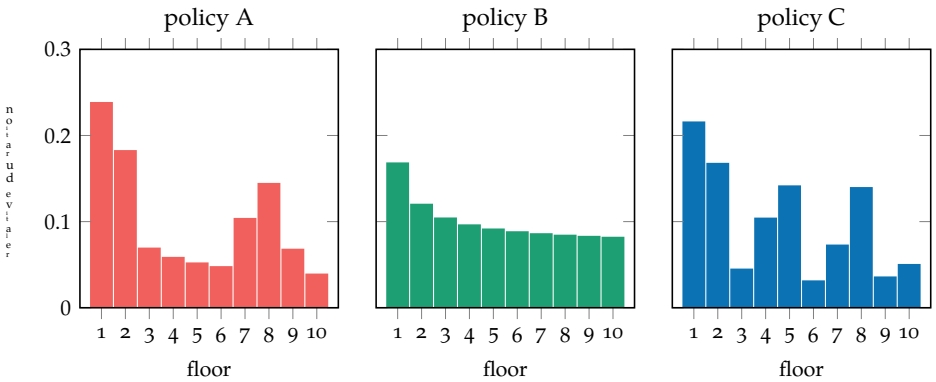
Solution: 最大边际逆强化学习从专家数据中提取二值特征，并寻求一个奖励函数，使其最优策略产生的轨迹与这些二值特征具有相同的频率。无法保证多个策略不会产生相同的特征期望。例如，一辆只进行左车道变换的自动驾驶汽车可能与一辆只进行右车道变换的自动驾驶汽车具有相同的车道变换频率。

练习18.4。最大边际逆强化学习使用特征期望来衡量策略与专家演示的相似度。如果使用非二元特征，这种相似度度量会受到什么影响？

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: 如果我们使用非二元特征，那么某些特征可能会比其他特征更大，从而激励智能体去匹配那些较大的特征，而不是那些倾向于较小的特征。尺度并非唯一的问题。即使所有特征都被限制在[0, 1]范围内，一个始终产生$\phi(s、a)_1$ =为0.5的策略，与一个一半时间产生$\phi(s、a)_1$ =为0而另一半时间产生$\phi(s、a)_1$ =为1的策略，会有相同的特征期望。根据特征所编码的内容，这可能导致非常不同的策略。任何一组连续特征都可以被离散化，从而由一组二元特征来近似。

练习 18.5。假设我们正在一栋高层建筑中构建一个系统，该系统必须选择将电梯发送到哪一层。我们已经训练了若干策略，以匹配专家演示的特征期望，例如顾客等待电梯的时间，或他们到达目的地所需等待的时间。我们对每个策略运行多次 rollout，并绘制其在每一层所花费的相对时长。根据最大熵原理，假设每个策略在特征期望上匹配程度相同，我们应该更偏好哪个策略?



Solution: 这些相对时长的分布类似于该电梯问题中轨迹的分布。在应用最大熵原理时，我们偏好熵最大的分布。因此，我们会选择策略B，因为它最为均匀，具有最大的熵。

练习18.6。考虑生成对抗模仿学习中的策略优化步骤。将目标函数重写为奖励函数的形式，以便可以应用传统的强化学习技术。

Solution: 我们重写方程(18.19)，去掉依赖于专家数据集的项，并将符号翻转，从对{v11}的最小化变为对奖励{v12}的最大化，从而得到代理奖励函数:

$$\tilde{R}_{\phi}(s,a) = -\log C_{\phi}(s,a)$$

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

尽管$\tilde{R}_{\phi}(s, a)$可能与未知的真实奖励函数有较大差异，但它们可用于驱动学习策略进入与专家所覆盖的状态-动作空间相似的区域。

练习 18.7. 解释如何修改生成对抗模仿学习，使得判别器接收轨迹而非状态-动作对。这样做为何可能有用？

Solution: 改变生成对抗模仿学习，使判别器接收轨迹是直接了当的，尤其是当轨迹长度固定时。专家数据集被分割成轨迹，而学习策略则用于生成轨迹，与之前一样。判别器不再针对状态-动作对操作，而是通过诸如循环神经网络（附录D.5）这样的表示来接收轨迹，并产生分类概率。目标函数大体上保持不变：

$$\max_{\phi} \min_{\theta} \mathbb{E}_{\tau \sim \pi_\theta}\left[\log(C_\phi(\tau))\right] + \mathbb{E}_{\tau \sim \mathcal{D}}\left[\log(1 - C_\phi(\tau))\right]$$

在整个轨迹上运行判别器的优势在于，它可以帮判别器捕捉到单个状态-动作对中不明显的一些特征，从而可以产生更好的策略。例如，对于自动驾驶策略，当观察单个加速度和转向速率时，判别器几乎没有什么可学习的。一个经过训练以观察更长轨迹的判别器可以看到更多车辆行为，如换道侵略性和平滑度，以更好地匹配专家驾驶示范。[15]

15 这种方法在 A. Kuefler, J. Morton, T. A. Wheeler, 和 M. J. Kochenderfer 的 “Imitating Driver Behavior with Generative Adversarial Networks,” in *IEEE Intelligent Vehicles Symposium (IV)*, 2017年 被使用。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

第四部分

状态不确定性

前几章已讨论了转移函数中的不确定性，包括结果状态和模型的不确定性。在本部分中，我们将不确定性扩展到状态本身。我们不是精确观测状态，而是接收仅与状态存在概率关系的观测。此类问题可建模为*partially observable Markov decision process*（*POMDP*）。求解POMDP的常见方法包括推断当前时间步下潜在状态的信念分布，然后应用将信念映射到动作的策略。我们将展示如何在给定过去的观测和动作序列的情况下更新信念分布。这使我们能够设计精确的求解方法来优化这些基于信念的策略。遗憾的是，除最小规模的问题外，POMDP在最优求解上都是难解的。我们回顾了多种离线近似方法，这些方法在较大问题上往往比精确方法具有更好的可扩展性。我们还展示了如何扩展本书前面讨论的一些在线近似方法，以适应部分可观测性。最后，我们引入有限状态控制器作为另一种策略表示，并讨论优化它们以求解POMDP的方法。

19 Beliefs

POMDP 是一种具有状态不确定性的 MDP。智能体接收的是当前状态的潜在不完美 *observation*，而非真实状态。根据过去的观测和动作序列，智能体形成对世界的理解。本章讨论智能体的 *belief* 如何通过底层状态上的概率分布来表示。我们介绍了多种算法，用于根据智能体所采取的观测和动作来更新我们的信念。[1] 如果状态空间是离散的，或者如果

满足特定的线性高斯假设。在这些假设不成立的情况下，我们可以使用基于线性化或采样的近似方法。

1 在机器人应用的背景下，S. Thrun、W. Burgard 和 D. Fox 讨论了不同的信念更新方法，*Probabilis-tic Robotics*。麻省理工学院出版社，2006年。

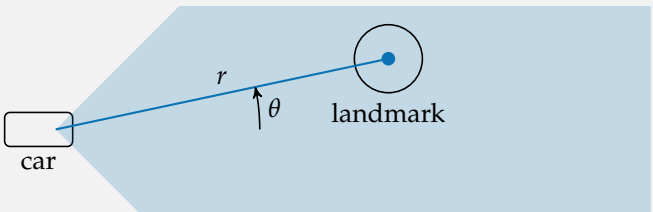
19.1 Belief Initialization

有几种不同的方式来表示我们的信念。在本章中，我们将讨论{v*}表示法，其中信念分布由固定分布族（如分类分布或多元正态分布）的一组参数来代表。我们还将讨论{v*}表示法，其中信念分布由粒子或从状态空间采样的点来表示。与这些不同的表示法相关联的是根据代理所采取的动作和观察来更新信念的不同程序。

在智能体采取任何行动或进行任何观察之前，我们以一个初始信念分布作为起点。如果我们对智能体可能处于状态空间中的位置有一些先验信息，我们可以将其编码到初始信念中。在缺乏信息的情况下，我们通常希望使用分散的初始信念，以避免过度自信地认为智能体处于状态空间中它实际上可能不在的区域。强烈的初始信念如果集中在远离真实状态的状态上，即使经过许多次观察，也可能导致较差的状态估计。

弥漫的初始信念可能导致困难，特别是在信念的非参数表示中，状态空间可能只能非常稀疏地采样。在某些情况下，等到获得有信息量的观测后再初始化信念可能是有用的。例如，在机器人导航问题中，我们可能希望等到传感器检测到一个已知的*landmark*，然后适当地初始化信念。地标可以帮助缩小状态空间的相关区域，以便我们可以将空间的采样集中在与地标观测一致的区域。例19.1说明了这个概念。

考虑一辆自动驾驶汽车，它装备有利用摄像头、雷达和激光雷达数据来追踪其位置的定位系统。该车能够从其当前位姿识别出一个距离为r、方位角为θ的独特地标：

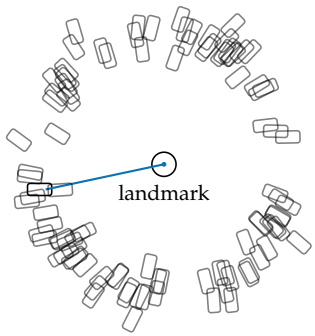


距离和方位测量具有零均值高斯噪声，其方差分别为 ν_r 和 ν_θ，并且已知地标位于 (x, y)。给定测量值 r 和 θ，我们可以生成关于车辆位置 $(\hat{x}, \hat{y})$ 和方向$\hat{\psi}$ 的分布：

$$\hat{r} \sim \mathcal{N}(r, \nu_r) \qquad \hat{\theta} \sim \mathcal{N}(\theta, \nu_\theta) \qquad \hat{\phi} \sim \mathcal{U}(0, 2\pi)$$
$$\hat{x} \leftarrow x + \hat{r}\cos\hat{\phi} \qquad \hat{y} \leftarrow y + \hat{r}\sin\hat{\phi} \qquad \hat{\psi} \leftarrow \hat{\phi} - \hat{\theta} - \pi$$

其中 $\hat{\phi}$ 是车辆相对于地标在全球坐标系中的角度 1 帧。

示例19.1. 基于地标观测生成初始非参数化信念。在此情形下，自动驾驶汽车可能位于地标周围环形区域内的任意位置：



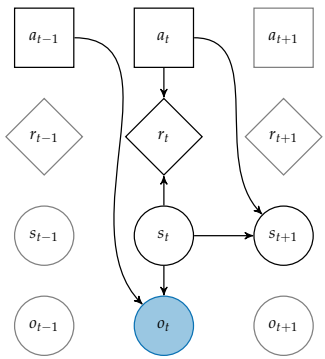


图19.1. POMDP问题表述的动态决策网络。与图7.1一样，进入动作节点的信息边未显示。

19.2 *Discrete State Filter*

在POMDP中，代理不直接观察环境的底层状态。相反，代理在每个时间步接收到一个观察，该观察属于某个 *observation space* $\mathcal{O}$。给定代理采取动作 a 并转移到状态 s' 时，观察到 o 的概率由 $O(o \mid a, s')$ 给出。如果 $\mathcal{O}$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

是连续的，则 $O(o \mid a、s')$ 是概率密度。图 19.1 显示了与 POMDP 相关的动态决策网络。算法 19.1 提供了 POMDP 数据结构的实现。

```
struct POMDP
    γ   # discount factor
    𝒮   # state space
    𝒜   # action space
    𝒪   # observation space
    T   # transition function
    R   # reward function
    O   # observation function
    TRO # sample transition, reward, and observation
end
```

算法19.1. POMDP的数据结构。我们将使用`TRO`字段来根据当前状态和动作采样下一个状态、奖励和观测：`s', r, o = TRO(s, a)`。M. Egorov、Z. N. Sunberg、E. Balaban、T. A. Wheeler、J. K. Gupta和M. J. Kochenderfer提供了一个用于指定和求解POMDP的综合软件包，“POMDPs.jl: A Framework for Sequential Decision Making Under Uncertainty”，*Journal of Machine Learning Research*，第18卷，第26期，第1–5页，2017年。在数学写作中，POMDP有时用由MDP各组成部分构成的元组来定义，写作$(\mathcal{S}, \mathcal{A}, \mathcal{O}, T, R, O, \gamma)$。

一种称为*recursive Bayesian estimation*的推理方法可用于根据最近的动作和观测更新我们对当前状态的信念分布。我们用$b(s)$表示赋予状态s的概率（或连续状态空间中的概率密度）。特定的信念b属于一个*belief space* $\mathcal{B}$，它包含所有可能的信念。

当状态空间和观测空间有限时，我们可以使用*discrete state filter*来精确地进行这种推断。对于具有离散状态空间的问题，信念可以通过分类分布来表示，其中每个状态都分配有一个概率质量。这种分类分布可以表示为一个长度为$|\mathcal{S}|$的向量，通常被称为*belief vector*。在b可以被视为向量的情况下，我们将使用$\mathbf{b}$。在这种情况下，$\mathcal{B} \subset \mathbb{R}^{|\mathcal{S}|}$。有时$\mathcal{B}$被称为*probability simplex*或*belief simplex*。

由于信念向量表示概率分布，其元素必须严格非负且总和为1：

$$b(s) \geq 0 \text{ for all } s \in \mathcal{S} \qquad \sum_s b(s) = 1 \tag{19.1}$$

以向量形式，我们有

$$\mathbf{b} \geq \mathbf{0} \qquad \mathbf{1}^\top \mathbf{b} = 1 \tag{19.2}$$

三态POMDP的信念空间如图19.2所示。离散POMDP问题见例19.2。

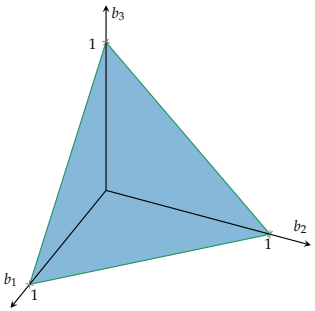


图19.2. 针对具有三个状态的问题的有效信念向量集合。尽管状态空间是离散的，但信念空间是连续的。

如果一个具有信念 b 的代理采取行动 a 并接收到观测 o，由于独立性假设，新信念 b' 可以按如下方式计算：

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

*crying baby problem*是一个简单的POMDP，包含两个状态、三个动作和两个观察。我们的目标是照顾婴儿，通过在每个时间步选择喂婴儿、给婴儿唱歌或忽略婴儿来实现。

婴儿会随着时间变得饥饿。人们无法直接观察到婴儿是否饥饿，而是通过婴儿是否哭泣这一带有噪声的观察来间接判断。饥饿的婴儿有80%的时间会哭泣，而吃饱的婴儿有10%的时间会哭泣。给婴儿唱歌会得到完美的观察。状态空间、动作空间和观察空间分别为：

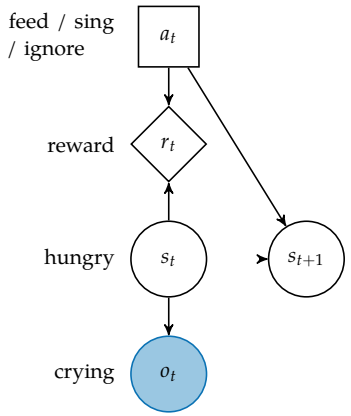
$$\begin{aligned}\mathcal{S} &= \{\text{sated}, \text{hungry}\}\\ \mathcal{A} &= \{\text{feed}, \text{sing}, \text{ignore}\}\\ \mathcal{O} &= \{\text{crying}, \text{quiet}\}\end{aligned}$$

转移动态为：

$$\begin{aligned}T(\text{sated} \mid \text{hungry}, \text{feed}) &= 100\,\%\\ T(\text{hungry} \mid \text{hungry}, \text{sing}) &= 100\,\%\\ T(\text{hungry} \mid \text{hungry}, \text{ignore}) &= 100\,\%\\ T(\text{sated} \mid \text{sated}, \text{feed}) &= 100\,\%\\ T(\text{hungry} \mid \text{sated}, \text{sing}) &= 10\,\%\\ T(\text{hungry} \mid \text{sated}, \text{ignore}) &= 10\,\%\end{aligned}$$

如果婴儿饥饿，奖励函数分配-10奖励，并因所需努力为喂养婴儿额外分配-5奖励。因此，喂养饥饿的婴儿会得到-15奖励。给婴儿唱歌需要额外努力，并产生额外的-0.5奖励。作为婴儿看护者，我们寻求折扣因子为$\gamma = 0.9$的最优无限视界策略。

例19.2. 哭泣婴儿问题是一个简单的部分可观测马尔可夫决策过程（POMDP），用于演示在状态不确定情况下的决策制定。



© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

在图19.1中：

$$
\begin{aligned}
b'(s') &= P(s' \mid b, a, o) && (19.3)\\
&\propto P(o \mid b, a, s')P(s' \mid b, a) && (19.4)\\
&= O(o \mid a, s')P(s' \mid b, a) && (19.5)\\
&= O(o \mid a, s')\sum_s P(s' \mid a, b, s)P(s \mid b, a) && (19.6)\\
&= O(o \mid a, s')\sum_s T(s' \mid s, a)b(s) && (19.7)
\end{aligned}
$$

更新离散信念的一个实例见例19.3，而信念更新则在算法19.2中实现。信念更新的成功依赖于准确的观测模型和转移模型。当这些模型不甚明了时，通常建议采用更分散分布的简化模型，以防过度自信，因为过度自信会导致状态估计变得脆弱。

19.3 Kalman Filter

我们可以按照以下方式调整方程（19.7）以处理连续状态空间：

$$b'(s') \propto O(o \mid a, s') \int T(s' \mid s, a)b(s)\,\mathrm{d}s \tag{19.8}$$

除非我们对T、O和b的形式做出一些假设，否则上述积分可能具有挑战性。一种特殊类型的滤波器，称为*Kalman filter* (算法 19.3),[2]，在假设T和O的情况下提供了精确更新。

是线性高斯，且 b 是高斯：[3]

$$
\begin{aligned}
T(\mathbf{s}' \mid \mathbf{s}, \mathbf{a}) &= \mathcal{N}(\mathbf{s}' \mid \mathbf{T}_s\mathbf{s} + \mathbf{T}_a\mathbf{a}, \boldsymbol{\Sigma}_s) && (19.9)\\
O(\mathbf{o} \mid \mathbf{s}') &= \mathcal{N}(\mathbf{o} \mid \mathbf{O}_s\mathbf{s}', \boldsymbol{\Sigma}_o) && (19.10)\\
b(\mathbf{s}) &= \mathcal{N}(\mathbf{s} \mid \boldsymbol{\mu}_b, \boldsymbol{\Sigma}_b) && (19.11)
\end{aligned}
$$

卡尔曼滤波器首先进行 *predict step*，利用转移动态得到具有以下均值和协方差的预测分布：

$$
\begin{aligned}
\boldsymbol{\mu}_p &\leftarrow \mathbf{T}_s\boldsymbol{\mu}_b + \mathbf{T}_a\mathbf{a} && (19.12)\\
\boldsymbol{\Sigma}_p &\leftarrow \mathbf{T}_s\boldsymbol{\Sigma}_b\mathbf{T}_s^\top + \boldsymbol{\Sigma}_s && (19.13)
\end{aligned}
$$

2 以匈牙利裔美国电气工程师鲁道夫·E·卡尔曼（1930–2016 年）命名，他参与了该滤波器的早期开发。

3 R. E. Kálmán，“线性滤波与预测问题的新方法”，*ASME Journal of Ba-sic Engineering*，第82卷，第35–45页，1960年。Y. Bar-Shalom、X. R. Li 和 T. Kirubarajan 在 *Estima-tion with Applications to Tracking and Navigation* 中提供了卡尔曼滤波器及其变种的全面概述。Wiley，2001年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

例19.3. 哭泣婴儿问题中的离散信念更新。

哭泣婴儿问题（例19.2）假设初始信念状态均匀分布：[b(满足), b(饥饿)] = [0.5, 0.5]。

假设我们忽略婴儿，婴儿就会哭。我们根据公式(19.7)更新我们的信念如下：

$$\begin{aligned} b'(\text{sated}) &\propto O(\text{crying} \mid \text{ignore}, \text{sated}) \sum_s T(\text{sated} \mid s, \text{ignore}) b(s) \\ &\propto 0.1(0.0 \cdot 0.5 + 0.9 \cdot 0.5) \\ &\propto 0.045 \\ b'(\text{hungry}) &\propto O(\text{crying} \mid \text{ignore}, \text{hungry}) \sum_s T(\text{hungry} \mid s, \text{ignore}) b(s) \\ &\propto 0.8(1.0 \cdot 0.5 + 0.1 \cdot 0.5) \\ &\propto 0.440 \end{aligned}$$

归一化后，我们的新信念大约为 [0.0928, 0.9072]。一个哭泣的婴儿很可能饿了。

假设我们随后喂婴儿，哭声停止。喂食确定性地使婴儿感到饱足，因此新的信念是[1, 0]。

最后，我们给宝宝唱歌，宝宝安静下来。再次使用公式（19.7）更新信念，得到[0.9890, 0.0110]。一个吃饱的宝宝只有10%的时间会变得饥饿，而这个百分比由于没有观察到任何哭泣而进一步降低。

算法19.2. 一种基于方程(19.7)更新离散信念的方法，其中b是向量，$\mathcal{P}$是POMDP模型。如果给定的观测具有零似然，则返回均匀分布。

```
function update(b::Vector{Float64}, 𝒫, a, o)
    𝒮, T, O = 𝒫.𝒮, 𝒫.T, 𝒫.O
    b′ = similar(b)
    for (i′, s′) in enumerate(𝒮)
        po = O(a, s′, o)
        b′[i′] = po * sum(T(s, a, s′) * b[i] for (i, s) in enumerate(𝒮))
    end
    if sum(b′) ≈ 0.0
        fill!(b′, 1)
    end
    return normalize!(b′, 1)
end
```

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

在*update step*中，我们利用这一预测分布与当前观测来更新我们的信念：

$$\mathbf{K} \leftarrow \boldsymbol{\Sigma}_p \mathbf{O}_s^\top \left(\mathbf{O}_s \boldsymbol{\Sigma}_p \mathbf{O}_s^\top + \boldsymbol{\Sigma}_o \right)^{-1} \tag{19.14}$$

$$\boldsymbol{\mu}_b \leftarrow \boldsymbol{\mu}_p + \mathbf{K}\left(\mathbf{o} - \mathbf{O}_s \boldsymbol{\mu}_p \right) \tag{19.15}$$

$$\boldsymbol{\Sigma}_b \leftarrow (\mathbf{I} - \mathbf{K}\mathbf{O}_s)\boldsymbol{\Sigma}_p \tag{19.16}$$

其中K被称为*Kalman gain*。

```
struct KalmanFilter
    μb # mean vector
    Σb # covariance matrix
end

function update(b::KalmanFilter, 𝒫, a, o)
    μb, Σb = b.μb, b.Σb
    Ts, Ta, Os = 𝒫.Ts, 𝒫.Ta, 𝒫.Os
    Σs, Σo = 𝒫.Σs, 𝒫.Σo
    # predict
    μp = Ts*μb + Ta*a
    Σp = Ts*Σb*Ts' + Σs
    # update
    Σpo = Σp*Os'
    K = Σpo/(Os*Σp*Os' + Σo)
    μb′ = μp + K*(o - Os*μp)
    Σb′ = (I - K*Os)*Σp
    return KalmanFilter(μb′, Σb′)
end
```

算法19.3. 卡尔曼滤波器，以高斯分布的形式更新信念。当前的信念由`μb`和`Σb`表示，而`𝒫`包含定义线性高斯动态和观测模型的矩阵。这个`𝒫`可以使用复合类型或命名元组来定义。

卡尔曼滤波器常被应用于实际上并不具有线性高斯动态和观测的系统。人们提出了对基本卡尔曼滤波器的各种修改，以更好地适应此类系统。[4]

4 S. Thrun、W. Burgard 和 D. Fox，*Probabilistic Robotics*。MIT 出版社，2006年。

19.4 Extended Kalman Filter

extended Kalman filter (*EKF*)是卡尔曼滤波器向具有高斯噪声的非线性动力学问题的简单扩展：

$$T(\mathbf{s}' \mid \mathbf{s}, \mathbf{a}) = \mathcal{N}(\mathbf{s}' \mid \mathbf{f}_T(\mathbf{s}, \mathbf{a}), \boldsymbol{\Sigma}_s) \tag{19.17}$$

$$O(\mathbf{o} \mid \mathbf{s}') = \mathcal{N}(\mathbf{o} \mid \mathbf{f}_O(\mathbf{s}'), \boldsymbol{\Sigma}_o) \tag{19.18}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

其中 $\mathrm{f}_T(\mathrm{s}、\mathrm{a})$ 和 $\mathrm{f}_O(\mathrm{s}')$ 是可微函数。

通过非线性动力学进行的精确信念更新不保证产生新的高斯信念，如图19.3所示。扩展卡尔曼滤波器使用非线性动力学的局部线性近似，从而产生一个近似真实更新信念的新高斯信念。我们可以使用与卡尔曼滤波器相似的更新方程，但必须在每次迭代时基于当前信念计算矩阵T_s和O_s。

动力学的局部线性近似，即{v*}，由雅可比矩阵形式的一阶泰勒展开给出。[5] 对于状态矩阵，泰勒

在$\boldsymbol{\mu}_b$和当前动作处进行扩展，而对于观察矩阵，则在预测均值$\boldsymbol{\mu}_p = \mathrm{f}_T(\boldsymbol{\mu}_b)$处计算。

扩展卡尔曼滤波器在第19.4节算法中实现。尽管它是一种近似方法，但在各种实际问题中表现出快速且良好的性能。扩展卡尔曼滤波器通常不保持后验分布的真实均值和方差，也不建模多模态后验分布。

5 多变量函数 f 的雅可比矩阵，若其有 n 个输入和 m 个输出，则是一个 $m \times n$ 矩阵，其中第 (i, j) 项为 $\partial f_i / \partial x_j$。

```
struct ExtendedKalmanFilter
    μb # mean vector
    Σb # covariance matrix
end

import ForwardDiff: jacobian
function update(b::ExtendedKalmanFilter, 𝒫, a, o)
    μb, Σb = b.μb, b.Σb
    fT, fO = 𝒫.fT, 𝒫.fO
    Σs, Σo = 𝒫.Σs, 𝒫.Σo
    # predict
    μp = fT(μb, a)
    Ts = jacobian(s->fT(s, a), μb)
    Os = jacobian(fO, μp)
    Σp = Ts*Σb*Ts' + Σs
    # update
    Σpo = Σp*Os'
    K = Σpo/(Os*Σp*Os' + Σo)
    μb′ = μp + K*(o - fO(μp))
    Σb′ = (I - K*Os)*Σp
    return ExtendedKalmanFilter(μb′, Σb′)
end
```

算法19.4. 扩展卡尔曼滤波器，是对卡尔曼滤波器的一种扩展，适用于具有非线性高斯动态的问题。当前信念由均值`μb`和协方差`Σb`表示。问题`𝒫`通过均值转移动力学函数`fT`和均值观测动力学函数`fO`指定非线性动力学。雅可比矩阵通过`ForwardDiff.jl`包获得。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

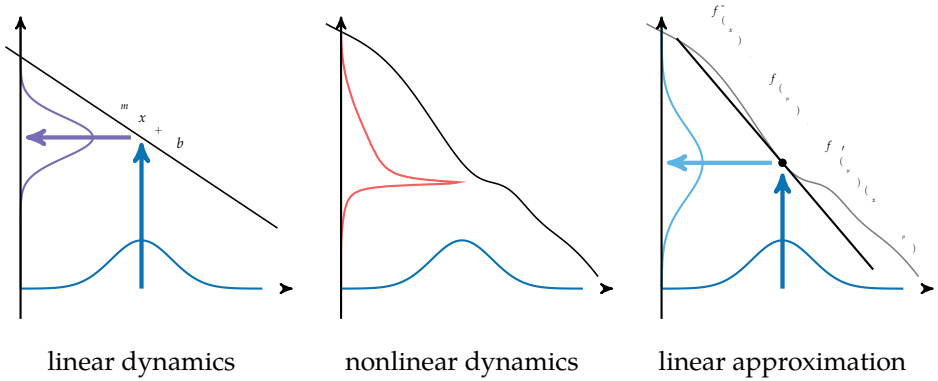


图19.3。使用线性变换更新高斯信念（左侧）产生另一个高斯分布。使用非线性变换更新高斯信念（中心）通常不会产生高斯分布。扩展卡尔曼滤波器使用变换的线性近似（右侧），从而产生另一个近似后验的高斯分布。

19.5 Unscented Kalman Filter

unscented Kalman filter (*UKF*)[6] 是对卡尔曼滤波的另一种扩展，用以具有高斯噪声的非线性问题。[7] 与扩展卡尔曼滤波器，无迹卡尔曼滤波器无需导数，并依赖于确定性采样策略来近似分布经过（通常是非线性）变换后的效果。

无迹卡尔曼滤波器是为了估计以下效果而开发的：用非线性函数 $\mathrm{f}(\mathrm{x})$ 对 x 上的分布进行变换，从而生成 x' 上的分布。我们希望估计 x' 上分布的均值 $\boldsymbol{\mu}'$ 和协方差 $\boldsymbol{\Sigma}'$。与 x[8] 上分布的均值 $\boldsymbol{\mu}$ 和协方差 $\boldsymbol{\Sigma}$ 相比，无迹变换允许使用更多关于 $p(\mathrm{x})$ 的信息。

一个*unscented transform*通过 f 传递一组*sigma points* S，并使用变换后的点来近似变换后的均值$\boldsymbol{\mu}'$和协方差$\boldsymbol{\Sigma}'$。原始均值和协方差使用 sigma 点和权重向量 w 构造：

$$\boldsymbol{\mu} = \sum_i w_i \mathbf{s}_i \tag{19.19}$$

$$\boldsymbol{\Sigma} = \sum_i w_i (\mathbf{s}_i - \boldsymbol{\mu})(\mathbf{s}_i - \boldsymbol{\mu})^\top \tag{19.20}$$

6 S. J. Julier 和 J. K. Uhlmann，“无迹滤波与非线性估计”，*Proceedings of the IEEE*，第92卷，第3期，第401-422页，2004年。

7 据 Jeffrey K. Uhlmann 所言，“无迹”一词源自他在某人桌上看到的一瓶除臭剂容器上的标签。他用这个词来避免将其称为“Uhlmann 滤波器”。IEEE 历史中心工作人员，“IEEE 会刊百年历程：2000-2009，”*Proceedings of the IEEE*，第100卷，第11期，第3131-3145页，2012年。

8 我们不一定需要假设先验分布是高斯分布。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

其中第 i 个 sigma 点 $\mathbf{s}_i$ 的权重为 w_i。为了提供无偏估计，这些权重之和必须为 1，但它们不必全为正数。

通过 f 的无迹变换给出的更新后的均值和协方差矩阵因此为：

$$\boldsymbol{\mu}' = \sum_i w_i \mathbf{f}(\mathbf{s}_i) \tag{19.21}$$

$$\boldsymbol{\Sigma}' = \sum_i w_i \left(\mathbf{f}(\mathbf{s}_i) - \boldsymbol{\mu}'\right)\left(\mathbf{f}(\mathbf{s}_i) - \boldsymbol{\mu}'\right)^\top \tag{19.22}$$

一组常见的 sigma 点包括均值 $\boldsymbol{\mu} \in \mathbb{R}^n$ 和额外的 $2n$ 个点，这些点由 $\boldsymbol{\mu}$ 在由协方差矩阵 $\boldsymbol{\Sigma}$ 确定的方向上的扰动形成：[9]

$$\mathbf{s}_1 = \boldsymbol{\mu} \tag{19.23}$$

$$\mathbf{s}_{2i} = \boldsymbol{\mu} + \left(\sqrt{(n+\lambda)\boldsymbol{\Sigma}}\right)_i \text{ for } i \text{ in } 1:n \tag{19.24}$$

$$\mathbf{s}_{2i+1} = \boldsymbol{\mu} - \left(\sqrt{(n+\lambda)\boldsymbol{\Sigma}}\right)_i \text{ for } i \text{ in } 1:n \tag{19.25}$$

这些 sigma 点与以下权重相关联：

$$w_i = \begin{cases} \frac{\lambda}{n+\lambda} & \text{for } i = 1 \\ \frac{1}{2(n+\lambda)} & \text{otherwise} \end{cases} \tag{19.26}$$

标量*spread parameter* λ决定了sigma点从均值扩散的距离。[10]不同λ值的几个sigma点集显示在图 19.4。

9 矩阵 A 的平方根是满足 $BB^\top = A$ 的矩阵 B。在 Julia 中，`sqrt` 方法生成一个矩阵 C，使得 $CC = A$，这并不相同。一种常见的平方根矩阵可以从 Cholesky 分解中获得。

10 使用 $\lambda = 2$ 是很常见的，这对于匹配高斯分布的四阶矩是最优的。选择这种形式的 sigma 点集的动机在练习 19.13 和练习 19.14 中提供。

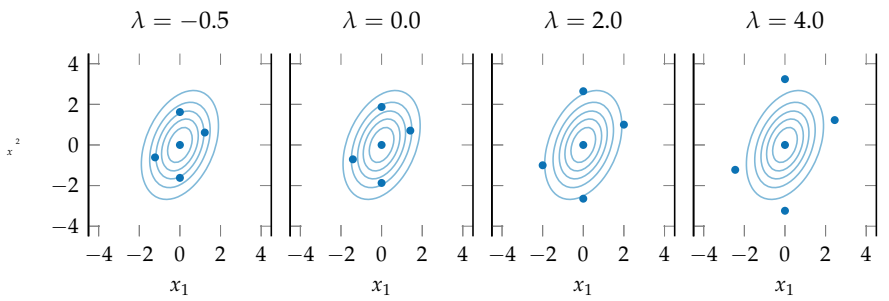


图19.4. 变化{\(v22\)}对方差为零均值和协方差的高斯分布生成的等式(19.23)中 sigma 点的影响：

`Σ = [1 1/2; 1/2 2]`。

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

无迹卡尔曼滤波器执行两次无迹变换：一次用于预测步骤，一次用于观测更新。算法19.5提供了这一过程的实现。

```
struct UnscentedKalmanFilter
    μb # mean vector
    Σb # covariance matrix
    λ  # spread parameter
end

function unscented_transform(μ, Σ, f, λ, ws)
    n = length(μ)
    Δ = cholesky((n + λ) * Σ).L
    S = [μ]
    for i in 1:n
        push!(S, μ + Δ[:,i])
        push!(S, μ - Δ[:,i])
    end
    S′ = f.(S)
    μ′ = sum(w*s for (w,s) in zip(ws, S′))
    Σ′ = sum(w*(s - μ′)*(s - μ′)' for (w,s) in zip(ws, S′))
    return (μ′, Σ′, S, S′)
end

function update(b::UnscentedKalmanFilter, 𝒫, a, o)
    μb, Σb, λ = b.μb, b.Σb, b.λ
    fT, fO = 𝒫.fT, 𝒫.fO
    n = length(μb)
    ws = [λ / (n + λ); fill(1/(2(n + λ)), 2n)]
    # predict
    μp, Σp, Sp, Sp′ = unscented_transform(μb, Σb, s->fT(s,a), λ, ws)
    Σp += 𝒫.Σs
    # update
    μo, Σo, So, So′ = unscented_transform(μp, Σp, fO, λ, ws)
    Σo += 𝒫.Σo
    Σpo = sum(w*(s - μp)*(s′ - μo)' for (w,s,s′) in zip(ws, So, So′))
    K = Σpo / Σo
    μb′ = μp + K*(o - μo)
    Σb′ = Σp - K*Σo*K'
    return UnscentedKalmanFilter(μb′, Σb′, λ)
end
```

算法19.5. 无迹卡尔曼滤波，是卡尔曼滤波在非线性高斯动力学问题上的扩展。当前信念由均值`μb`和协方差`Σb`表示。问题$\mathcal{P}$通过均值转移动力学函数`fT`和均值观测动力学函数`fO`指定了非线性动力学。在无迹变换中使用的sigma点由散布参数`λ`控制。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

19.6 Particle Filter

具有大状态空间的离散问题或动态无法被卡尔曼滤波器的线性高斯假设很好地近似的连续问题，通常必须借助近似技术来表示信念并执行信念更新。一种常见的方法是使用 *particle filter*，它将信念状态表示为一组状态。[11] 近似中的每个状态信念被称为*particle*。

粒子滤波器通过选择或随机采样一组代表初始信念的粒子来初始化。具有m个粒子的粒子滤波器的信念更新首先从转移分布中采样，对每个状态s_i进行传播，以概率$T(s_i' \mid s_i, a)$获得新状态s_i'。新信念的构建方式是根据观测函数$O(o_i \mid s', a, s_i')$对传播后的状态进行加权，从中抽取m个粒子。该过程见算法19.6。例19.4展示了粒子滤波器的一个应用。

11 粒子滤波器的教程由 M. S. Arulampalam、S. Maskell、N. Gordon 和 T. Clapp 提供，“在线非线性/非高斯贝叶斯跟踪的粒子滤波器教程”，*IEEE Transac-tions on Signal Processing*，第 50 卷，第 2 期，第 174–188 页，2002 年。

在离散观测问题中，我们也可以进行带有拒绝的粒子信念更新。我们重复以下过程 m 次以生成下一状态样本集。首先，我们随机选择滤波器中的某个状态 s_i，然后根据我们的转移模型采样一个下一状态 s_i'。其次，我们根据观测模型生成一个随机观测 o_i。如果 o_i 不等于真实观测 o，则将其拒绝，并重新生成新的 s_i' 和 o_i，直到观测匹配。这一过程 *particle filter with rejection* 在算法19.7中实现。

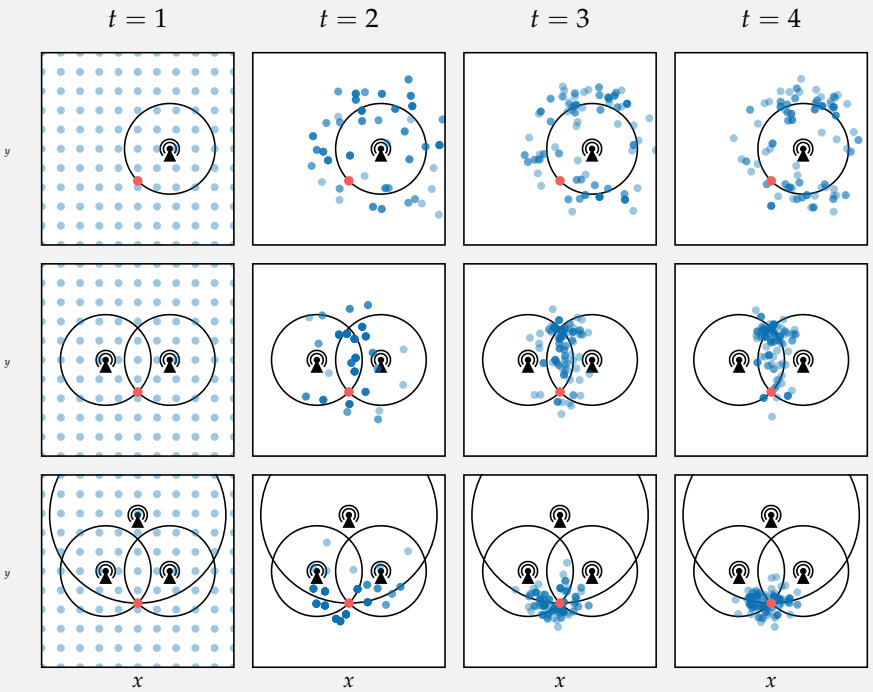
随着粒子滤波器中粒子数量的增加，粒子所表示的分布趋近于真实的后验分布。遗憾的是，粒子滤波器在实际应用中可能失效。粒子覆盖不足及重采样过程的随机性可能导致真实状态附近没有粒子。*particle deprivation*这一问题可以通过几种策略在一定程度上缓解。例19.5给出了一个激励性的例子。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

假设我们想要根据对已知位置的无线电 beacon 的不完美距离测量来确定我们的位置。我们保持大致静止几步，以收集独立测量值。粒子滤波状态是我们可能的位置。我们可以将每个粒子预期会测量到的距离与观测到的距离进行比较。

我们假设每个信标的单个距离观测值均受到零均值高斯噪声的影响。由于我们仅近似保持静止，我们的粒子转移函数会加入零均值高斯噪声。

这里的图像展示了粒子滤波器的演化过程。各行对应不同数量的信标。红点表示我们的真实位置，蓝点代表粒子。圆圈表示与每个传感器的无噪声距离测量结果一致的位置。



需要三个信标才能准确定位我们的位置。粒子滤波器的一个优点是它能够表示多峰分布，尤其是在只有一个或两个信标时尤为明显。

示例19.4. 粒子滤波器应用于不同的信标配置。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct ParticleFilter
    states # vector of state samples
end

function update(b::ParticleFilter, 𝒫, a, o)
    T, O = 𝒫.T, 𝒫.O
    states = [rand(T(s, a)) for s in b.states]
    weights = [O(a, s′, o) for s′ in states]
    D = SetCategorical(states, weights)
    return ParticleFilter(rand(D, length(states)))
end
```

算法19.6. 粒子滤波器的信念更新器，它根据代理的动作`a`和观测`o`更新代表信念的状态向量。附录G.5提供了`SetCategorical`的实现，用于定义离散集上的分布。

```
struct RejectionParticleFilter
    states # vector of state samples
end

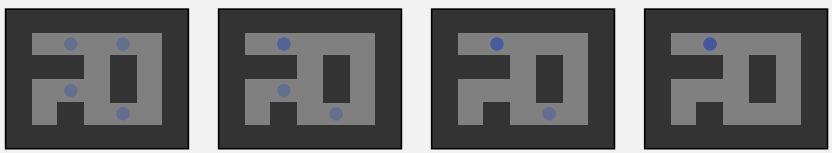
function update(b::RejectionParticleFilter, 𝒫, a, o)
    T, O = 𝒫.T, 𝒫.O
    states = similar(b.states)
    i = 1
    while i ≤ length(states)
        s = rand(b.states)
        s′ = rand(T(s,a))
        if rand(O(a,s′)) == o
            states[i] = s′
            i += 1
        end
    end
    return RejectionParticleFilter(states)
end
```

算法19.7. 使用拒绝法更新粒子滤波器，强制采样状态与输入观测值`o`匹配。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

洞穴探险者乔迷失在一个网格迷宫中。他丢失了提灯，因此只能通过触摸来观察周围环境。在任何时刻，乔都能判断出他在迷宫中的位置在每个基本方向上是否有墙壁。乔对自己感知墙壁的能力相当自信，因此他假设自己的观察是完美的。

乔使用粒子滤波器来追踪他的信念随时间的变化。在某个时刻，他停下来休息。他继续运行粒子滤波器以更新他的信念。下面的图表展示了他随时间变化的信念，其中点表示他的粒子滤波器中对应迷宫这些位置的信念粒子。



初始信念在每个网格位置都有一个粒子，这些位置与他对北面和南面墙的当前观察相匹配。探险家乔不移动，也不获得新信息，因此他的信念不应随时间改变。由于重采样的随机性质，后续信念可能不包含所有初始状态。随着时间的推移，他的信念将继续丢失状态，直到只包含一个状态。这个状态有可能不是探险家乔所在的位置。

示例19.5。由于重采样的随机性质，粒子滤波器运行足够长时间后，可能会在状态空间的相关区域丢失粒子。当粒子数量较少或粒子分布在较大的状态空间时，该问题更为显著。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

19.7 Particle Injection

Particle injection 涉及注入随机粒子以防止粒子匮乏。算法19.8从更广泛的分布中注入固定数量的粒子，例如状态空间上的均匀分布。[12] 当粒子

注入有助于防止粒子匮乏，但同时也降低了粒子滤波器所表示后验信念的准确性。

12 对于机器人定位问题，常见的做法是从所有可能的机器人位姿上的均匀分布中注入粒子，并根据当前观测进行加权。

```
struct InjectionParticleFilter
    states # vector of state samples
    m_inject # number of samples to inject
    D_inject # injection distribution
end

function update(b::InjectionParticleFilter, 𝒫, a, o)
    T, O, m_inject, D_inject = 𝒫.T, 𝒫.O, b.m_inject, b.D_inject
    states = [rand(T(s, a)) for s in b.states]
    weights = [O(a, s′, o) for s′ in states]
    D = SetCategorical(states, weights)
    m = length(states)
    states = vcat(rand(D, m - m_inject), rand(D_inject, m_inject))
    return InjectionParticleFilter(states, m_inject, D_inject)
end
```

算法19.8. 带有注入的粒子滤波更新，其中从注入分布`D_inject`中采样`m_inject`个粒子，以减少粒子耗尽的的风险。

在每个更新步骤中，与其固定注入粒子数量，我们可以采取更为自适应的策略。当所有粒子都被赋予极低的权重时，通常意味着我们需要注入更多粒子。仅根据当前粒子集的平均权重来决定注入粒子的数量或许颇具吸引力，但这样做可能导致滤波器在初期收敛阶段或是遇到高传感器噪声时，因自然较低的观测概率而敏感，影响其成功运行。[13]

13 S. Thrun、W. Burgard 和 D. Fox，*Probabilistic Robotics*。MIT Press，2006年。

算法19.9提出了一种*adaptive injection*算法，该算法追踪平均粒子权重的两个指数移动平均值，并根据它们的比例决定注入次数。[14] 如果w_{mean}是当前平均粒子权重，则这两个

移动平均值的更新依据是

14 D. E. Goldberg 和 J. Richard- son，“局部化方法的实验比较”，载于 *International Conference on Genetic Algorithms*，1987年。

$$w_{\text{fast}} \leftarrow w_{\text{fast}} + \alpha_{\text{fast}}(w_{\text{mean}} - w_{\text{fast}}) \tag{19.27}$$

$$w_{\text{slow}} \leftarrow w_{\text{slow}} + \alpha_{\text{slow}}(w_{\text{mean}} - w_{\text{slow}}) \tag{19.28}$$

其中 $0 \leq \alpha_{\text{slow}} < \alpha_{\text{fast}} \leq 1$。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

在给定迭代中注入样本的数量是通过比较快和慢平均粒子权重获得的：[15]

$$m_{\text{inject}} = \left\lfloor m \max\left(0, 1 - \nu \frac{w_{\text{fast}}}{w_{\text{slow}}}\right)\right\rceil \tag{19.29}$$

标量 $\nu \geq 1$ 允许我们调整注入率。

15 注意 $\lfloor x \rceil$ 表示最接近 x 的整数。

```
mutable struct AdaptiveInjectionParticleFilter
    states   # vector of state samples
    w_slow   # slow moving average
    w_fast   # fast moving average
    α_slow   # slow moving average parameter
    α_fast   # fast moving average parameter
    ν        # injection parameter
    D_inject # injection distribution
end

function update(b::AdaptiveInjectionParticleFilter, 𝒫, a, o)
    T, O = 𝒫.T, 𝒫.O
    w_slow, w_fast, α_slow, α_fast, ν, D_inject =
        b.w_slow, b.w_fast, b.α_slow, b.α_fast, b.ν, b.D_inject
    states = [rand(T(s, a)) for s in b.states]
    weights = [O(a, s′, o) for s′ in states]
    w_mean = mean(weights)
    w_slow += α_slow*(w_mean - w_slow)
    w_fast += α_fast*(w_mean - w_fast)
    m = length(states)
    m_inject = round(Int, m * max(0, 1.0 - ν*w_fast / w_slow))
    D = SetCategorical(states, weights)
    states = vcat(rand(D, m - m_inject), rand(D_inject, m_inject))
    b.w_slow, b.w_fast = w_slow, w_fast
    return AdaptiveInjectionParticleFilter(states,
        w_slow, w_fast, α_slow, α_fast, ν, D_inject)
end
```

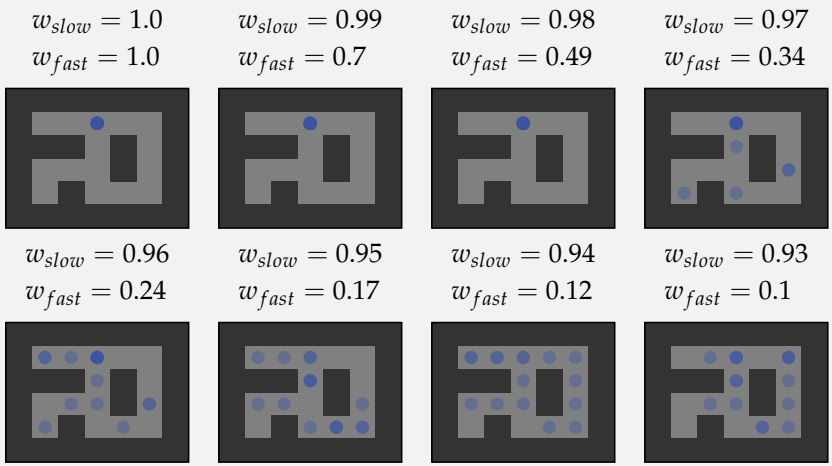
算法19.9。一种具有自适应注入的粒子滤波器，它分别维护平均粒子权重的快、慢指数移动平均`w_fast`和`w_slow`，平滑因子分别为`α_fast`和`α_slow`。仅当平均粒子权重的快速移动平均小于慢速移动平均的$1/\nu$时，才注入粒子。原论文推荐值为`α_fast = 0.1`、`α_slow = 0.001`和`ν = 2`。

19.8 Summary

- 部分可观测马尔可夫决策过程（POMDP）将MDP扩展为包含状态不确定性。
- 不确定性要求POMDP中的代理维持对其状态的信念。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

来自例19.6的探险家乔现在向东移动一格，并且将他的粒子滤波器中的所有粒子也向东移动一格。他现在仅感知到北面和东面有墙，遗憾的是，这一观测结果与他滤波器中的任何更新粒子都不吻合。他决定采用自适应注入法来解决粒子匮乏问题。在此，我们看到他如何从均匀随机分布中注入粒子，以及快速和慢速滤波器的值：



迭代从左到右、从上到下进行。每个蓝点代表粒子滤波器中的一个粒子，对应于位于网格该位置的部分信念。

示例 19.6. 一个带有自适应注入 `α_slow = 0.01`、`α_fast = 0.3` 和 `ν = 2.0` 的粒子滤波，从一个由 16 个相同粒子构成的匮乏状态开始。移动平均被初始化为 1，以反映长期观测完美匹配滤波器中的每一个粒子。在接下来的迭代中，这些移动平均根据与观测匹配的粒子数量以不同速率变化。迭代按从左到右、从上到下的顺序进行。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

- 对于具有离散状态空间的POMDP，其信念可以通过类别分布来表示，并且能够进行解析更新。
- 线性高斯POMDP的信念可以使用高斯分布表示，并且也可以解析地进行更新。
- 对于非线性、连续的POMDP，其信念也可以使用高斯分布来表示，但通常无法通过解析方法进行更新。在这种情况下，可以使用扩展卡尔曼滤波和无迹卡尔曼滤波。
- 连续问题有时可以在假设其为线性高斯的情况下进行建模。
- 粒子滤波器用大量状态粒子集合来近似信念。

19.9 Exercises

练习 19.1. 每个 MDP 都能被构建为 POMDP 吗?

Solution: 是的。POMDP公式通过以观测分布的形式引入状态不确定性，扩展了MDP公式。任何MDP都可以通过 $\mathcal{O} = \mathcal{S}$ 和 $O(o \mid a, s') = (o = s')$ 构建为POMDP。

练习19.2. 对于没有观测的离散POMDP，信念更新是什么？对于具有线性高斯动态且无观测的POMDP，信念更新是什么?

Solution: 如果 POMDP 中的一个 agent 在没有观测的情况下，信念为 b 并采取动作 a，那么新的信念 b' 可以按如下方式计算：

$$b'(s') = P(s' \mid b, a) = \sum_s P(s' \mid a, b, s) P(s \mid b, a) = \sum_s T(s' \mid s, a) b(s)$$

这种信念更新相当于具有均匀观测分布。一个具有线性高斯动态且没有观测的POMDP将仅使用方程（19.12）中的卡尔曼滤波预测步骤来更新其信念。

练习 19.3。一辆自动驾驶汽车使用多元正态分布来表示其位置的信念。当它在一个红绿灯前停下时，信念更新器在它静止时继续运行。随着时间的推移，信念集中，并对某一特定位置变得极其自信。为什么这可能成为一个问题？如何避免这种极端的自信?

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: 对信念的过度自信在模型或信念更新未能完美反映现实时可能成为问题。过于自信的信念可能已收敛到一个与真实状态不符的状态。一旦车辆再次移动，新的观测结果可能与信念不一致，导致估计效果不佳。为帮助解决这一问题，我们可以要求协方差矩阵对角线元素的值高于阈值。

练习 19.4。考虑追踪我们对工厂生产的部件的次品率的信念。我们使用泊松分布来建模在工厂次品率为 λ 的情况下，一天工厂运营中生产出 k 个次品的概率：

$$P(k \mid \lambda) = \frac{1}{k!}\lambda^k e^{-\lambda}$$

假设我们最初对次品率的信念服从伽马分布：

$$p(\lambda \mid \alpha, \beta) = \frac{\beta^\alpha}{\Gamma(\alpha)}\lambda^{\alpha-1}e^{-\beta\lambda}$$

其中 $\lambda \in (0, \infty)$，并且信念由形状 $\alpha > 0$ 和速率 $\beta > 0$ 参数化。经过一天的工厂运营，我们观察到生产了 $d \geq 0$ 个次品。证明我们对次品率的更新信念也是伽玛分布。[16]

16 Gamma 分布是泊松分布的共轭先验。*conjugate prior* 是一类概率分布族，当用观测数据进行更新时，仍保持在同一分布族内。共轭先验对于建模信念很有用，因为其形式保持不变。

Solution: 我们寻求后验分布 $p(\lambda \mid d、\alpha、\beta)$，这可以通过贝叶斯规则得到：

$$\begin{aligned} p(\lambda \mid d, \alpha, \beta) &\propto p(d \mid \lambda)p(\lambda \mid \alpha, \beta) \\ &\propto \frac{1}{d!}\lambda^d e^{-\lambda} \ \frac{\beta^\alpha}{\Gamma(\alpha)}\lambda^{\alpha-1}e^{-\beta\lambda} \\ &\propto \lambda^{\alpha+d-1}e^{-(\beta+1)\lambda} \end{aligned}$$

这是一个伽马分布：

$$\begin{aligned} p(\lambda \mid \alpha + d, \beta + 1) &= \frac{(\beta+1)^{\alpha+d}}{\Gamma(\alpha+d)}\lambda^{\alpha+d-1}e^{-(\beta+1)\lambda} \\ &\propto \lambda^{\alpha+d-1}e^{-(\beta+1)\lambda} \end{aligned}$$

练习19.5。为什么在具有连续观测的POMDP中，不使用带拒绝的粒子滤波器来更新信念？

Solution: 拒绝采样需要反复对转移函数和观测函数进行采样，直到采样得到的观测值与真实观测值匹配。在连续概率分布中，采样到任何特定值的概率为零，这使得拒绝采样会永远运行下去。在实践中，我们会为连续值使用有限表示，例如64位浮点数，但拒绝采样对每个粒子可能都会运行极长时间。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

练习19.6。解释为什么在示例19.5中，Spelunker Joe不会从切换到具有自适应注入$\nu \geq 1$的粒子滤波器中获益。

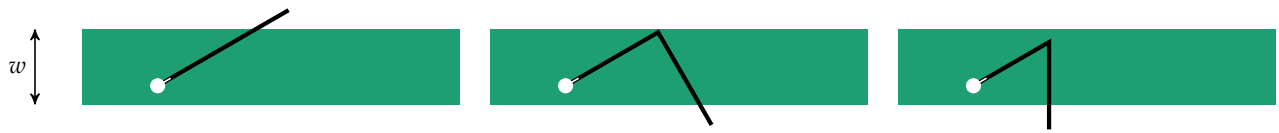
Solution: 自适应注入在 $\nu w_{\text{fast}}/w_{\text{slow}} < 1$ 时注入新粒子。Spelunker Joe 假设观测完美，并且其信念中的粒子与当前观测相匹配。因此，每个粒子的权重均为 1，并且 w_{fast} 和 w_{slow} 均为 1。由此可得，$w_{\text{fast}}/w_{\text{slow}}$ 始终为 1，因此不会产生新粒子。

练习19.7。为什么在带有自适应注入的粒子滤波器中，注入率标量ν通常不设置为小于1的值?

Solution: 粒子注入的设计目的是在当前观测的似然度低于观测似然度的历史趋势时注入粒子。因此，注入通常仅当平均粒子权重的短期估计w_{fast}小于平均粒子权重的长期估计w_{slow}时发生。如果$\nu < 1$，那么即使$w_{\text{fast}} \geq w_{\text{slow}}$，粒子仍然可以被生成，尽管这表明当前观测的似然度高于过去平均水平。

练习19.8。假设我们被随机投放至一片矩形森林中，初始位置均匀随机选定。我们不清楚自己面向哪个方向。幸运的是，我们确实知道森林的尺寸（宽度为w，长度为$\ell \gg w$）。[17] 我们可以沿着连续路径移动，不断观察我们是否仍在森林中。我们如何将信念更新应用于此问题?这里有三种可能的策略，每种定义了一条不同的路径。这些策略中哪些能保证逃离森林?哪种策略最佳?

17 这个问题源于理查德·贝尔曼的“迷失森林问题”，在这个问题中，我们从一个已知几何形状的森林中的随机位置和方向出发，必须找到一种策略，以最小化离开森林的平均（或最大）时间。R. Bellman，“最小化问题”，*Bulletin of the American Mathematical Society*，第62卷，第3期，第270页，1956年。



A straight path of length $2w$

Two perpendicular segments, each of length $\sqrt{2}w$

Two legs of an equilateral triangle, each of length $\frac{2\sqrt{3}}{3}w$

Solution: 我们最初的信念是在森林中所有二维位置和方向（状态）上的均匀分布。我们可以用迄今为止所走过的路径来表示更新后的信念。如果我们仍在森林中，我们的信念由所有状态组成，这些状态可以通过沿着我们的路径从森林内的某个状态到达，同时始终完全位于森林中。一旦我们离开森林，我们的信念就由所有通过沿着我们的路径到达边缘、同时始终完全位于森林中的状态组成。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com



在给定的策略中，只有最后两个能确保逃离森林。由两条垂直线段和等边三角形的两条边形成的路径总是会与森林的边界相交。然而，直线段可能无法离开森林。我们倾向于两种逃脱策略中较短的那个，即等边三角形。

练习 19.9. 算法 19.2 检查更新后的信念是否为零向量。在什么情况下，信念更新会产生零向量？为什么这在现实世界应用中可能出现？

Solution: 零信念向量可能由被认为不可能的观测o产生。这种情况可能在从信念b采取动作a后出现，当根据b和我们的转移模型，对于所有可能的下一状态s'，$O(o \mid a, s')$均为0时。算法19.2通过返回均匀信念来处理这种情况。在实际应用中，模型与真实世界之间可能存在不匹配。我们通常希望谨慎避免将零概率分配给观测，以防我们的信念、转移或观测模型不正确。

练习 19.10. 假设我们正在对一架飞机进行飞行中监测。飞机要么处于正常运行状态 s^0，要么处于故障状态 s^1。我们通过无警告 w^0 或有警告 w^1 来接收观测。我们可以选择允许飞机继续飞行 m^0 或送飞机进行维修 m^1。我们有以下转移和观测动态，其中我们假设警告与动作无关，给定飞机的状态：

$$T(s^0 \mid s^0, m^0) = 0.95 \qquad O(w^0 \mid s^0) = 0.99$$
$$T(s^0 \mid s^0, m^1) = 1 \qquad O(w^1 \mid s^1) = 0.7$$
$$T(s^1 \mid s^1, m^0) = 1$$
$$T(s^0 \mid s^1, m^1) = 0.98$$

给定初始信念 $\mathbf{b} = [0.95, 0.05]$，计算更新后的信念 $\mathbf{b}'$，假设我们允许飞机继续飞行并观察到一个警告。

Solution: 使用公式 (19.7)，我们更新对于 s^0 的信念：

$$b'(s^0) \propto O(w^1 \mid s^0) \sum_s T(s^0 \mid s, m^0) b(s)$$
$$b'(s^0) \propto O(w^1 \mid s^0)(T(s^0 \mid s^0, m^0) b(s^0) + T(s^0 \mid s^1, m^0) b(s^1))$$
$$b'(s^0) \propto (1 - 0.99)(0.95 \times 0.95 + (1 - 1) \times 0.05) = 0.009025$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们对 s^1 重复更新：

$$b'(s^1) \propto O(w^1 \mid s^1) \sum_s T(s^1 \mid s, m^0) b(s)$$
$$b'(s^1) \propto O(w^1 \mid s^1)(T(s^1 \mid s^0, m^0) b(s^0) + T(s^1 \mid s^1, m^0) b(s^1))$$
$$b'(s^1) \propto 0.7((1 - 0.95) \times 0.95 + 1 \times 0.05) = 0.06825$$

归一化后，我们得到以下更新后的信念：

$$b'(s^0) = \frac{b'(s^0)}{b'(s^0) + b'(s^1)} \approx 0.117$$
$$b'(s^1) = \frac{b'(s^1)}{b'(s^0) + b'(s^1)} \approx 0.883$$
$$b' \approx [0.117, 0.883]$$

练习19.11。考虑一个机器人沿直线运动，其位置为x，速度为v，加速度为a。在每个时间步，我们直接控制加速度并观察速度。机器人的运动方程为：

$$x' = x + v\Delta t + \tfrac{1}{2} a \Delta t^2$$
$$v' = v + a\Delta t$$

其中 Δt 是每一步的时长。假设我们希望实现一个卡尔曼滤波器来更新我们的信念。状态向量为 $\mathrm{s} = [x, v]$。确定 T_s、T_a 和 O_s。

Solution: 转移和观测动态可以写成如下线性形式：

$$\begin{bmatrix} x' \\ v' \end{bmatrix} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ v \end{bmatrix} + \begin{bmatrix} \frac{1}{2}\Delta t^2 \\ \Delta t \end{bmatrix} a$$
$$o = \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} x' \\ v' \end{bmatrix}$$

通过这些方程，我们可以确定T_s、T_a和O_s：

$$\mathbf{T}_s = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \qquad \mathbf{T}_a = \begin{bmatrix} \frac{1}{2}\Delta t^2 \\ \Delta t \end{bmatrix} \qquad \mathbf{O}_s = \begin{bmatrix} 0 & 1 \end{bmatrix}$$

练习 19.12。考虑一个以恒定速度 v 在二维空间移动的差速驱动机器人。机器人的状态是其位置（x，y）和其朝向 θ。在每个时间步，我们控制机器人的转向速率 ω。机器人的运动方程为

$$x' = x + v\cos(\theta)\Delta t$$
$$y' = y + v\sin(\theta)\Delta t$$
$$\theta' = \theta + \omega\Delta t$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

该转移函数是非线性的。其线性化 $\mathrm{T}_{\mathbf{s}}$ 作为状态 $\mathrm{s} = [x, y, \theta]$ 的函数是什么?

Solution: 线性化由以下雅可比矩阵给出:

$$\mathbf{T}_{\mathbf{s}} = \begin{bmatrix} \frac{\partial x'}{\partial x} & \frac{\partial x'}{\partial y} & \frac{\partial x'}{\partial \theta} \\ \frac{\partial y'}{\partial x} & \frac{\partial y'}{\partial y} & \frac{\partial y'}{\partial \theta} \\ \frac{\partial \theta'}{\partial x} & \frac{\partial \theta'}{\partial y} & \frac{\partial \theta'}{\partial \theta} \end{bmatrix} = \begin{bmatrix} 1 & 0 & -v\sin(\theta)\Delta t \\ 0 & 1 & v\cos(\theta)\Delta t \\ 0 & 0 & 1 \end{bmatrix}$$

这种线性化可以用于扩展卡尔曼滤波器中，以维持一个信念状态。

练习 19.13. 假设我们为 n 维分布选择以下 $2n$ 个 sigma 点:

$$\mathbf{s}_{2i} = \boldsymbol{\mu} + \sqrt{n\boldsymbol{\Sigma}_i} \text{ for } i \text{ in } 1:n$$
$$\mathbf{s}_{2i-1} = \boldsymbol{\mu} - \sqrt{n\boldsymbol{\Sigma}_i} \text{ for } i \text{ in } 1:n$$

证明我们可以使用权重 $w_i = 1/(2n)$ 从这些 sigma 点重构均值和协方差。

Solution: 如果我们使用权值 $w_i = 1/(2n)$，则重建均值为

$$\sum_i w_i \mathbf{s}_i = \sum_{i=1}^{n} \frac{1}{2n}\left(\boldsymbol{\mu} + \sqrt{n\boldsymbol{\Sigma}_i}\right) + \frac{1}{2n}\left(\boldsymbol{\mu} - \sqrt{n\boldsymbol{\Sigma}_i}\right) = \sum_{i=1}^{n} \frac{1}{n}\boldsymbol{\mu} = \boldsymbol{\mu}$$

重建后的协方差为

$$\begin{aligned} \sum_i w_i(\mathbf{s}_i - \boldsymbol{\mu}')(\mathbf{s}_i - \boldsymbol{\mu}')^\top &= 2\sum_{i=1}^{n} \frac{1}{2n}\left(\sqrt{n\boldsymbol{\Sigma}_i}\right)\left(\sqrt{n\boldsymbol{\Sigma}_i}\right)^\top \\ &= \frac{1}{n}\sum_{i=1}^{n}\left(\sqrt{n\boldsymbol{\Sigma}_i}\right)\left(\sqrt{n\boldsymbol{\Sigma}_i}\right)^\top \\ &= \sqrt{\boldsymbol{\Sigma}}\sqrt{\boldsymbol{\Sigma}}^\top \\ &= \boldsymbol{\Sigma} \end{aligned}$$

练习 19.14。回顾上一个问题中表示均值 $\boldsymbol{\mu}$ 和协方差 $\boldsymbol{\Sigma}$ 的 $2n$ 个 sigma 点及权重。我们希望参数化 sigma 点和权重，以便控制这些点围绕均值的集中程度。证明我们可以通过均匀降低原始 sigma 点的权重，然后将均值 $\boldsymbol{\mu}$ 作为一个额外的 sigma 点包含进来，从而构造出一组新的 sigma 点。证明这组新的 $2n + 1$ 个 sigma 点与式 (19.23) 中的形式相匹配。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: 我们可以在习题19.13的sigma点中包含均值$\boldsymbol{\mu}$，以得到一组新的$2n+1$个sigma点：

$$\begin{aligned}
\mathbf{s}_1 &= \boldsymbol{\mu} \\
\mathbf{s}_{2i} &= \boldsymbol{\mu} + \left(\sqrt{\frac{n}{1-w_1}\boldsymbol{\Sigma}}\right)_i \text{ for } i \text{ in } 1:n \\
\mathbf{s}_{2i+1} &= \boldsymbol{\mu} - \left(\sqrt{\frac{n}{1-w_1}\boldsymbol{\Sigma}}\right)_i \text{ for } i \text{ in } 1:n
\end{aligned}$$

其中 w_1 是第一个 sigma 点的权重。其余 sigma 点的权重从 $1/(2n)$ 均匀减少到 $(1-w_1)/(2n)$。重构均值仍为 $\boldsymbol{\mu}$，重构协方差仍为 $\boldsymbol{\Sigma}$。

我们可以改变 w_1 来生成不同的 sigma 点集。设置 $w_1 > 0$ 会使 sigma 点远离均值；设置 $w_1 < 0$ 会使 sigma 点更接近均值。这会产生一组缩放后的 sigma 点，具有不同的高阶矩，但它保留了相同的均值和协方差。

我们可以通过代入 $w_1 = \lambda/(n+\lambda)$ 来匹配方程 (19.23)。由此可得 $(1-w_1)/2n = 1/(2(n+\lambda))$ 以及 $n/(1-w_1) = n+\lambda$。

练习19.15. 计算具有λ =2的多元高斯分布下的sigma点集及权重，其中

$$\boldsymbol{\mu} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \qquad \boldsymbol{\Sigma} = \begin{bmatrix} 4 & 0 \\ 0 & 2.25 \end{bmatrix}$$

Solution: 由于我们有一个二维高斯分布，并且给定了 $\lambda = 2$，我们需要计算 $2n+1 = 5$ 个 sigma 点。我们需要计算平方根矩阵 $\mathrm{B} = \sqrt{(n+\lambda)\boldsymbol{\Sigma}}$，使得 $\mathrm{B}\mathrm{B}^\top = (n+\lambda)\boldsymbol{\Sigma}$。由于缩放后的协方差矩阵是对角矩阵，平方根矩阵就是 $(n+\lambda)\boldsymbol{\Sigma}$ 的逐元素平方根：

$$\sqrt{(n+\lambda)\boldsymbol{\Sigma}} = \sqrt{(2+2)\begin{bmatrix} 4 & 0 \\ 0 & 2.25 \end{bmatrix}} = \begin{bmatrix} 4 & 0 \\ 0 & 3 \end{bmatrix}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

现在，我们可以计算sigma点和权重：

$$\mathbf{s}_1 = \begin{bmatrix}1\\2\end{bmatrix} \qquad w_1 = \frac{2}{2+2} = \frac{1}{2}$$
$$\mathbf{s}_2 = \begin{bmatrix}1\\2\end{bmatrix} + \begin{bmatrix}4\\0\end{bmatrix} = \begin{bmatrix}5\\2\end{bmatrix} \qquad w_2 = \frac{1}{2(2+2)} = \frac{1}{8}$$
$$\mathbf{s}_3 = \begin{bmatrix}1\\2\end{bmatrix} - \begin{bmatrix}4\\0\end{bmatrix} = \begin{bmatrix}-3\\2\end{bmatrix} \qquad w_3 = \frac{1}{2(2+2)} = \frac{1}{8}$$
$$\mathbf{s}_4 = \begin{bmatrix}1\\2\end{bmatrix} + \begin{bmatrix}0\\3\end{bmatrix} = \begin{bmatrix}1\\5\end{bmatrix} \qquad w_4 = \frac{1}{2(2+2)} = \frac{1}{8}$$
$$\mathbf{s}_5 = \begin{bmatrix}1\\2\end{bmatrix} - \begin{bmatrix}0\\3\end{bmatrix} = \begin{bmatrix}1\\-1\end{bmatrix} \qquad w_5 = \frac{1}{2(2+2)} = \frac{1}{8}$$

练习 19.16. 使用前一个练习中的 sigma 点和权重，通过 $\text{f(x)} = [2x_1, x_1x_2]$ 计算由无迹变换给出的更新均值和协方差。

Solution: 变换后的 sigma 点为

$$\mathbf{f}(\mathbf{s}_1) = \begin{bmatrix}2\\2\end{bmatrix} \qquad \mathbf{f}(\mathbf{s}_2) = \begin{bmatrix}10\\10\end{bmatrix} \qquad \mathbf{f}(\mathbf{s}_3) = \begin{bmatrix}-6\\-6\end{bmatrix} \qquad \mathbf{f}(\mathbf{s}_4) = \begin{bmatrix}2\\5\end{bmatrix} \qquad \mathbf{f}(\mathbf{s}_5) = \begin{bmatrix}2\\-1\end{bmatrix}$$

我们可以将均值重构为变换后 sigma 点的加权和：

$$\boldsymbol{\mu}' = \sum_i w_i \mathbf{f}(\mathbf{s}_i)$$
$$\boldsymbol{\mu}' = \frac{1}{2}\begin{bmatrix}2\\2\end{bmatrix} + \frac{1}{8}\begin{bmatrix}10\\10\end{bmatrix} + \frac{1}{8}\begin{bmatrix}-6\\-6\end{bmatrix} + \frac{1}{8}\begin{bmatrix}2\\5\end{bmatrix} + \frac{1}{8}\begin{bmatrix}2\\-1\end{bmatrix} = \begin{bmatrix}2\\2\end{bmatrix}$$

协方差矩阵可以从逐点协方差矩阵的加权和中重建：

$$\boldsymbol{\Sigma}' = \sum_i w_i \left(\mathbf{f}(\mathbf{s}_i) - \boldsymbol{\mu}'\right)\left(\mathbf{f}(\mathbf{s}_i) - \boldsymbol{\mu}'\right)^\top$$
$$\boldsymbol{\Sigma}' = \frac{1}{2}\begin{bmatrix}0 & 0\\0 & 0\end{bmatrix} + \frac{1}{8}\begin{bmatrix}64 & 64\\64 & 64\end{bmatrix} + \frac{1}{8}\begin{bmatrix}64 & 64\\64 & 64\end{bmatrix} + \frac{1}{8}\begin{bmatrix}0 & 0\\0 & 9\end{bmatrix} + \frac{1}{8}\begin{bmatrix}0 & 0\\0 & 9\end{bmatrix} = \begin{bmatrix}16 & 16\\16 & 18.25\end{bmatrix}$$

练习 19.17。卡尔曼滤波器和扩展卡尔曼滤波器都使用观测协方差 O_s 来计算互协方差矩阵 $\boldsymbol{\Sigma}_{po}$。无迹卡尔曼滤波器不直接计算这个观测矩阵，而是直接计算 $\boldsymbol{\Sigma}_{po}$。证明无迹卡尔曼滤波器的协方差更新 $\boldsymbol{\Sigma}_{b'} \leftarrow \boldsymbol{\Sigma}_p - \text{K}\boldsymbol{\Sigma}_o\text{K}^\top$，与卡尔曼滤波器和扩展卡尔曼滤波器的协方差更新 $\boldsymbol{\Sigma}_{b'} \leftarrow (\text{I} - \text{KO}_s)\boldsymbol{\Sigma}_p$ 相匹配。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: 我们可以利用关系 $\mathrm{K} = \boldsymbol{\Sigma}_{po}\boldsymbol{\Sigma}_o^{-1}$ 和 $\boldsymbol{\Sigma}_{po} = \boldsymbol{\Sigma}_p \mathrm{O}_s^\top$ 来证明这两个更新是等价的。另请注意，对称矩阵的转置等于其自身，且协方差矩阵是对称的。

$$\begin{aligned}
\boldsymbol{\Sigma}_{b'} &= \boldsymbol{\Sigma}_p - \mathbf{K}\boldsymbol{\Sigma}_o\mathbf{K}^\top \\
&= \boldsymbol{\Sigma}_p - \mathbf{K}\boldsymbol{\Sigma}_o\left(\boldsymbol{\Sigma}_{po}\boldsymbol{\Sigma}_o^{-1}\right)^\top \\
&= \boldsymbol{\Sigma}_p - \mathbf{K}\boldsymbol{\Sigma}_o\left(\boldsymbol{\Sigma}_o^{-1}\right)^\top\boldsymbol{\Sigma}_{po}^\top \\
&= \boldsymbol{\Sigma}_p - \mathbf{K}\boldsymbol{\Sigma}_{po}^\top \\
&= \boldsymbol{\Sigma}_p - \mathbf{K}\left(\Sigma_p\mathbf{O}_s^\top\right)^\top \\
&= \boldsymbol{\Sigma}_p - \mathbf{K}\mathbf{O}_s\Sigma_p^\top \\
&= \boldsymbol{\Sigma}_p - \mathbf{K}\mathbf{O}_s\Sigma_p \\
&= (\mathbf{I} - \mathbf{K}\mathbf{O}_s)\Sigma_p
\end{aligned}$$

练习 19.18. 使用粒子滤波器代替卡尔曼滤波器有哪些优点和缺点?

Solution: 当系统为线性高斯时，卡尔曼滤波器能够提供精确的信念更新。当系统非线性且不确定性呈多峰分布时，粒子滤波器可能效果更好。粒子滤波器通常计算成本更高，并且可能遭受粒子退化问题。

练习19.19。考虑在一个观测非常可靠的问题中使用粒子滤波器来维持信念，其中观测具有高或低的似然度。例如，在Spelunker Joe问题中，我们能够可靠地确定四面墙中的哪些存在，从而使我们能够立即排除任何与观测不符的状态。对于这类问题，为什么带有拒绝机制的粒子滤波器可能比传统粒子滤波器更为合适?

Solution: 传统的粒子滤波器会生成一组粒子，并根据其观测似然度赋予权重。在像Spelunker Joe遇到的问题中，许多粒子可能最终几乎没有权重。拥有众多低权重的粒子会使信念容易受到粒子匮乏的影响。带有拒绝机制的粒子滤波器确保每个粒子的后继状态与观测兼容，从而缓解了粒子匮乏的问题。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

20 Exact Belief State Planning

POMDP 的目标是选择在与环境交互时最大化奖励累积的动作。与 MDP 不同，状态不可直接观测，要求智能体利用其过去的动作和观测历史来形成信念。如前一章所述，信念可以表示为状态上的概率分布。给定转移、观测和奖励模型，有多种方法可以计算将信念映射到动作的最优策略。[1] 一种方法是转换将POMDP转换为MDP并应用动态规划。其他方法包括将策略表示为条件计划或在信念空间上的分段线性价值函数。本章以一个计算最优策略的算法作为结束，该算法类似于MDP中的值迭代。

1 关于精确解法的讨论由 L. P. Kaelbling、M. L. Littman 和 A. R. Cassandra 提供，“Planning and Acting in Partially Observable Stochastic Domains,” *Artificial Intelligence*, vol. 101, no. 1–2, pp. 99–134, 1998。

20.1 Belief-State Markov Decision Processes

任何POMDP都可以被视为一种使用信念作为状态的MDP，也称为 *belief-state MDP*。[2]信念状态MDP的状态空间是所有信念的集合$\mathcal{B}$。动作空间与POMDP相同。

信念状态MDP的奖励函数取决于信念和所采取的行动。它简单来说就是奖励的期望值。对于离散状态空间，其表达式为

$$R(b, a) = \sum_{s} R(s, a) b(s) \tag{20.1}$$

2 K. J. Åström，“不完全状态信息的马尔可夫过程的最优控制”，*Journal of Mathematical Analysis and Applications*，第10卷，第1期，第174–205页，1965年。

如果状态空间和观测空间是离散的，则信念状态MDP的信念状态转移函数由下式给出：

$$
\begin{aligned}
T(b' \mid b, a) &= P(b' \mid b, a) && (20.2)\\
&= \sum_o P(b' \mid b, a, o) P(o \mid b, a) && (20.3)\\
&= \sum_o P(b' \mid b, a, o) \sum_s P(o \mid b, a, s) P(s \mid b, a) && (20.4)\\
&= \sum_o P(b' \mid b, a, o) \sum_s P(o \mid b, a, s) b(s) && (20.5)\\
&= \sum_o P(b' \mid b, a, o) \sum_{s'} \sum_s P(o \mid b, a, s, s') P(s' \mid b, s, a) b(s) && (20.6)\\
&= \sum_o (b' = \text{Update}(b, a, o)) \sum_{s'} O(o \mid a, s') \sum_s T(s' \mid s, a) b(s) && (20.7)
\end{aligned}
$$

在方程（20.7）中，Update(b, a, o) 使用上一章讨论的确定性过程返回更新后的信念。[3] 对于连续问题，我们将求和替换为积分。

3 提醒一下，我们使用的约定是将括号中的逻辑陈述按数值处理，当其为真时取1，为假时取0。

求解信念状态MDP颇具挑战，因其状态空间为连续。我们可以采用前几章介绍的近似动态规划技术，但通过利用信念状态MDP的结构，往往能做得更好，这一点将在本章后续部分详细讨论。

20.2 *Conditional Plans*

有多种方法可以表示POMDP的策略。一种方法是使用以树形结构表示的 *conditional plan*。图20.1展示了一个在二元动作和观察空间中的三步条件计划示例。节点对应于信念状态。边用观察标注，节点用动作标注。如果我们有一个计划π，与根关联的动作表示为$\pi()$，与观察o关联的子计划表示为$\pi(o)$。算法20.1提供了对此的实现。

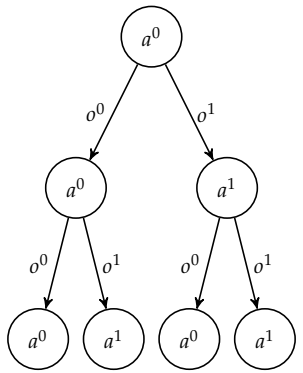


图20.1. 一个三步条件计划。

条件规划告诉我们，在面对截止到树所代表的视界为止的观测结果时该采取什么行动。执行条件规划时，我们从根节点开始，执行与之关联的动作。我们依据观测结果沿着树向下推进，采取经过节点所关联的动作。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct ConditionalPlan
    a        # action to take at root
    subplans # dictionary mapping observations to subplans
end

ConditionalPlan(a) = ConditionalPlan(a, Dict())

(π::ConditionalPlan)() = π.a
(π::ConditionalPlan)(o) = π.subplans[o]
```

算法20.1。条件计划数据结构，由动作和从观察到子计划的映射组成。`subplans`字段是从观察到条件计划的`Dict`。为方便起见，我们创建了一个特殊构造器，用于仅包含单个节点的计划。

假设我们有一个条件计划 π，并且我们想要计算从状态 s 开始时的期望效用。这个计算可以通过递归进行：

$$U^\pi(s) = R(s, \pi()) + \gamma \left[\sum_{s'} T\left(s' \mid s, \pi()\right) \sum_o O\left(o \mid \pi(), s'\right) U^{\pi(o)}(s') \right] \tag{20.8}$$

该过程的实现见算法20.2。

```
function lookahead(𝒫::POMDP, U, s, a)
    𝒮, 𝒪, T, O, R, γ = 𝒫.𝒮, 𝒫.𝒪, 𝒫.T, 𝒫.O, 𝒫.R, 𝒫.γ
    u′ = sum(T(s,a,s′)*sum(O(a,s′,o)*U(o,s′) for o in 𝒪) for s′ in 𝒮)
    return R(s,a) + γ*u′
end

function evaluate_plan(𝒫::POMDP, π::ConditionalPlan, s)
    U(o,s′) = evaluate_plan(𝒫, π(o), s′)
    return isempty(π.subplans) ? 𝒫.R(s,π()) : lookahead(𝒫, U, s, π())
end
```

算法20.2. 一种评估MDP $\mathcal{P}$中，从状态 `s` 出发的条件计划 `π` 的方法。计划表示为包含一个动作和一个将观察结果映射到子计划的字典的元组。

我们可以按如下方式计算信念b的效用：

$$U^\pi(b) = \sum_s b(s) U^\pi(s) \tag{20.9}$$

示例20.1展示了如何计算与三步条件计划相关的效用。

既然我们已经有了一种在视界h内评估条件计划的方法，我们现在可以计算最优的h-步价值函数：

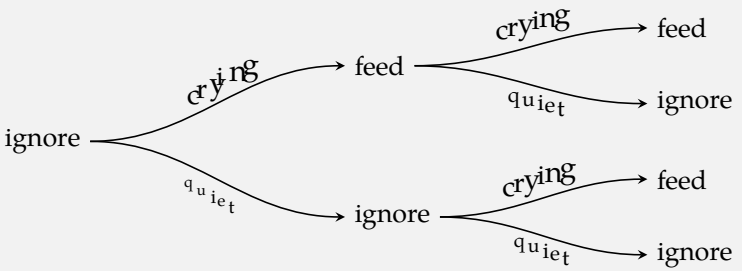
$$U^*(b) = \max_\pi U^\pi(b) \tag{20.10}$$

从与最大化的π的根相关联的动作中可以生成一个最优动作。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

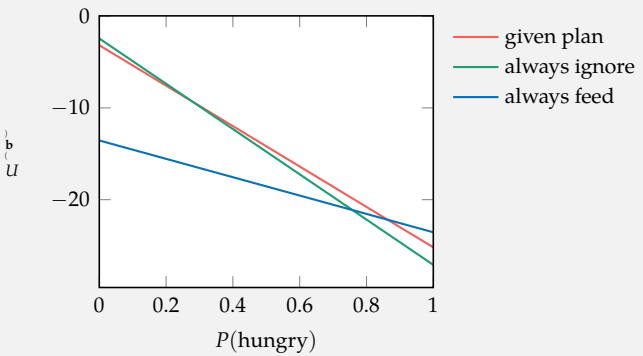
例20.1. 三步哭泣婴儿问题的条件计划（附录F.7），与两个更简单的条件计划进行评估和比较。

考虑以下针对婴儿哭泣问题的三步条件计划：



在这个计划中，我们首先忽略婴儿。如果我们观察到任何哭泣，我们就喂养婴儿。如果我们没有观察到任何哭泣，我们就忽略婴儿。我们的第三个行动同样是在有哭泣时进行喂养。

在这个信念空间中，该计划的预期效用与总是喂养婴儿的三步计划和总是忽略婴儿的三步计划并列绘制。



我们发现，给定的计划并不比总是忽略或总是喂养婴儿普遍更好。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

通过直接枚举所有h步条件计划来求解一个h步POMDP通常是计算上不可行的，如图20.2所示。在一个h步计划中有$(|\mathcal{O}|^h - 1)/(|\mathcal{O}| - 1)$个节点。一般来说，任何动作都可以插入任何节点，从而产生$|\mathcal{A}|^{(|\mathcal{O}|^h-1)/(|\mathcal{O}|-1)}$个可能的$h$步计划。这种指数增长意味着，即使对于适中的$h$值，枚举所有计划也是不可行的。正如本章后面将讨论的那样，除了显式枚举所有可能的计划之外，还有其他选择。

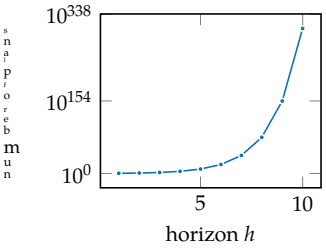


图 20.2. 即使对于只有两个动作和两个观测的小型 POMDP，可能的计划数量也会随着规划周期的延长而极快地增长。我们通常可以在每次迭代时显著地修剪 alpha 向量集合，并仅考虑远少得多的计划。

20.3 Alpha Vectors

我们可以将方程（20.9）重写为向量形式：

$$U^\pi(b) = \sum_s b(s)U^\pi(s) = \boldsymbol{\alpha}_\pi^\top \mathbf{b} \tag{20.11}$$

向量 $\boldsymbol{\alpha}_\pi$ 称为 *alpha vector*，包含了在计划 π 下每个状态的期望效用。与信念向量一样，alpha 向量的维度为 $|\mathcal{S}|$。与信念不同，alpha 向量中的分量表示效用，而不是概率质量。算法 20.3 展示了如何计算 alpha 向量。

```
function alphavector(𝒫::POMDP, π::ConditionalPlan)
    return [evaluate_plan(𝒫, π, s) for s in 𝒫.𝒮]
end
```

算法20.3。我们可以通过从所有可能的初始状态调用`evaluate_plan`，从条件规划中生成一个alpha向量。

每个alpha向量在信念空间中定义一个超平面。方程(20.11)中给出的最优价值函数是这些超平面中的最大值：

$$U^*(\mathbf{b}) = \max_\pi \boldsymbol{\alpha}_\pi^\top \mathbf{b} \tag{20.12}$$

使价值函数分段线性且凸。[4]

使用条件计划来表示策略的另一种方法是使用一组alpha向量 Γ，每个向量都标注有一个动作。尽管不切实际，但生成集合 Γ 的一种方法是枚举 h 步条件计划的集合，然后计算它们的alpha向量。与alpha向量相关联的动作是相关条件计划根部的动作。我们通过更新信念状态并在新信念b处执行与主导alpha向量相关联的动作来执行由 Γ 表示的策略。在b处的主导alpha向量 $\boldsymbol{\alpha}$ 是使 $\boldsymbol{\alpha}^\top$b最大化的那个。这种策略可用于选择动作

4 连续状态POMDP的最优值函数也是凸的，这可以通过将POMDP通过状态空间离散化进行近似，并取离散状态数趋于无穷大时的极限来看出。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

超出原始条件计划的范围。算法20.4提供了一种实现。

```
struct AlphaVectorPolicy
    𝒫 # POMDP problem
    Γ # alpha vectors
    a # actions associated with alpha vectors
end

function utility(π::AlphaVectorPolicy, b)
    return maximum(α⋅b for α in π.Γ)
end

function (π::AlphaVectorPolicy)(b)
    i = argmax([α⋅b for α in π.Γ])
    return π.a[i]
end
```

算法20.4。一个alpha向量策略是根据一组alpha向量Γ和一个关联动作数组a定义的。给定当前信念b，它将找到在该信念点上给出最高值的alpha向量。它将返回关联的动作。

如果我们使用*one-step lookahead*，就不必追踪与Γ中的alpha向量相关的动作。从信念b出发，使用由Γ表示的值函数进行的一步前瞻动作，记为U^Γ，是

$$\pi^\Gamma(b) = \arg\max_a \left[R(b,a) + \gamma \sum_o P(o \mid b,a) U^\Gamma(\text{Update}(b,a,o)) \right] \tag{20.13}$$

其中

$$P(o \mid b,a) = \sum_s P(o \mid s,a) b(s) \tag{20.14}$$

$$P(o \mid s,a) = \sum_{s'} T(s' \mid s,a) O(o \mid s',a) \tag{20.15}$$

算法20.5提供了该方法的实现。例20.2展示了在哭泣婴儿问题上使用一步前瞻法。

20.4 Pruning

如果我们有一组 alpha 向量 Γ，我们可能希望 *prune* 那些对我们表示价值函数没有贡献的 alpha 向量，或者对任何信念都不是最优的计划。移除这些 alpha 向量或计划可以提高计算效率。我们可以通过以下方式检查一个 alpha 向量 $\boldsymbol{\alpha}$ 是否是 *dominated*：

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
function lookahead(𝒫::POMDP, U, b::Vector, a)
    𝒮, 𝒪, T, O, R, γ = 𝒫.𝒮, 𝒫.𝒪, 𝒫.T, 𝒫.O, 𝒫.R, 𝒫.γ
    r = sum(R(s,a)*b[i] for (i,s) in enumerate(𝒮))
    Posa(o,s,a) = sum(O(a,s′,o)*T(s,a,s′) for s′ in 𝒮)
    Poba(o,b,a) = sum(b[i]*Posa(o,s,a) for (i,s) in enumerate(𝒮))
    return r + γ*sum(Poba(o,b,a)*U(update(b, 𝒫, a, o)) for o in 𝒪)
end

function greedy(𝒫::POMDP, U, b::Vector)
    u, a = findmax(a->lookahead(𝒫, U, b, a), 𝒫.𝒜)
    return (a=a, u=u)
end

struct LookaheadAlphaVectorPolicy
    𝒫 # POMDP problem
    Γ # alpha vectors
end

function utility(π::LookaheadAlphaVectorPolicy, b)
    return maximum(α⋅b for α in π.Γ)
end

function greedy(π, b)
    U(b) = utility(π, b)
    return greedy(π.𝒫, U, b)
end

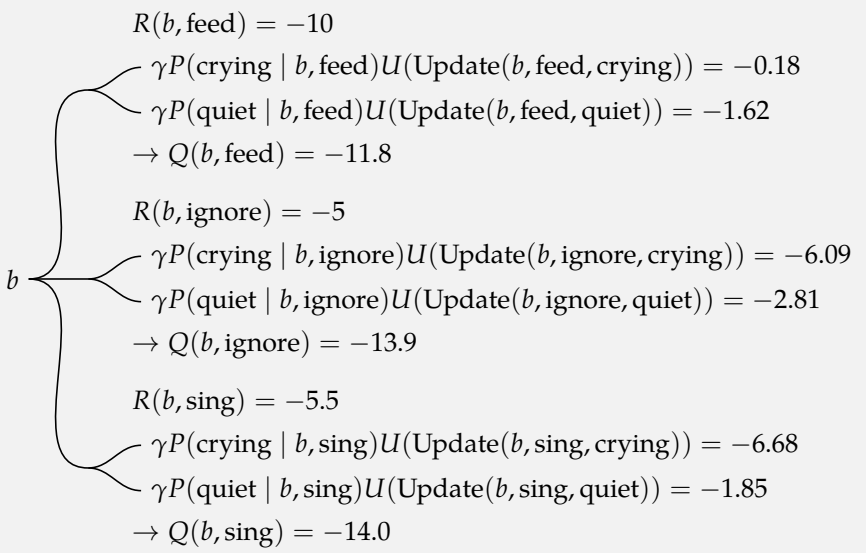
(π::LookaheadAlphaVectorPolicy)(b) = greedy(π, b).a
```

算法20.5. 由一组alpha向量{v*}表示的策略。它通过一步前瞻来生成最优动作及相关的效用。公式(20.13)用于计算前瞻。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

示例 20.2. 将前瞻策略应用于哭泣婴儿问题。

考虑在哭泣婴儿问题上使用一步前瞻，值函数由alpha向量$[-3.7, -15]$和$[-2, -21]$给出。假设我们当前的信念是$b = [0.5, 0.5]$，意味着我们认为婴儿饿与不饿的可能性相同。我们应用方程（20.13）



我们使用 $Q(b、a)$ 来表示信念状态下的动作价值函数。策略预测喂婴儿将带来最高期望效用，因此采取该行动。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

通过求解一个线性规划问题，最大化向量δ相较于所有其他向量所实现的效用差距[5]，来确定集合Γ中的alpha向量。

[5] 形式为 $\mathbf{a} \geq \mathbf{b}$ 的约束是逐元素的。也就是说，我们指的是对所有 i 满足 $a_i \geq$。b_i

$$\begin{aligned} \underset{\delta, \mathbf{b}}{\text{maximize}} \quad & \delta \\ \text{subject to} \quad & \mathbf{b} \geq \mathbf{0} \\ & \mathbf{1}^\top \mathbf{b} = 1 \\ & \boldsymbol{\alpha}^\top \mathbf{b} \geq \boldsymbol{\alpha}'^\top \mathbf{b} + \delta, \qquad \boldsymbol{\alpha}' \in \Gamma \end{aligned} \tag{20.16}$$

前两个约束确保 b 是一个分类分布，最后一组约束确保我们找到一个信念向量，使得 $\boldsymbol{\alpha}$ 的期望奖励高于 Γ 中的所有 alpha 向量。如果在求解线性规划后，效用差距 δ 为负，则 $\boldsymbol{\alpha}$ 被支配。如果 δ 为正，则 $\boldsymbol{\alpha}$ 不被支配，且 b 是一个 $\boldsymbol{\alpha}$ 不被支配的信念。算法 20.6 提供了求解方程（20.16）的实现，以确定一个信念（如果存在），其中 δ 最为正。

```
function find_maximal_belief(α, Γ)
    m = length(α)
    if isempty(Γ)
        return fill(1/m, m) # arbitrary belief
    end
    model = Model(GLPK.Optimizer)
    @variable(model, δ)
    @variable(model, b[i=1:m] ≥ 0)
    @constraint(model, sum(b) == 1.0)
    for a in Γ
        @constraint(model, (α-a)⋅b ≥ δ)
    end
    @objective(model, Max, δ)
    optimize!(model)
    return value(δ) > 0 ? value.(b) : nothing
end
```

算法20.6. 一种寻找信念向量b的方法，使得 alpha 向量α相比于 alpha 向量集合Γ改进最大。如果不存在这样的信念，则不返回任何结果。包`JuMP.jl`和`GLPK.jl`分别提供了数学优化框架和线性规划的求解器。

算法20.7展示了一个过程，该过程使用算法20.6来找到集合Γ中的主导α向量。最初，所有α向量都是成为主导的候选者。接着，我们从中选择一个候选者，并确定信念b，在此信念下，该候选者相较于主导集合中的所有其他α向量能带来最大的价值提升。如果候选者未能带来提升，我们将其从集合中移除。若确实带来了提升，我们将从候选集合中移出一个能带来最大改进的α向量。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

在 b 处加入支配集。此过程持续进行，直到不再有候选者为止。我们可以修剪掉任何在任何信念点上都不占支配地位的 alpha 向量及相关的条件计划。例 20.3 展示了在哭泣婴儿问题上的修剪过程。

```
function find_dominating(Γ)
    n = length(Γ)
    candidates, dominating = trues(n), falses(n)
    while any(candidates)
        i = findfirst(candidates)
        b = find_maximal_belief(Γ[i], Γ[dominating])
        if b === nothing
            candidates[i] = false
        else
            k = argmax([candidates[j] ? b⋅Γ[j] : -Inf for j in 1:n])
            candidates[k], dominating[k] = false, true
        end
    end
    return dominating
end

function prune(plans, Γ)
    d = find_dominating(Γ)
    return (plans[d], Γ[d])
end
```

算法20.7. 一种用于修剪被支配的alpha向量及相关计划的方法。`find_dominating`函数识别集合Γ中所有支配性的alpha向量。它使用二进制向量`candidates`和`dominating`分别追踪哪些alpha向量是支配集的候选以及哪些当前位于支配集中。

20.5 *Value Iteration*

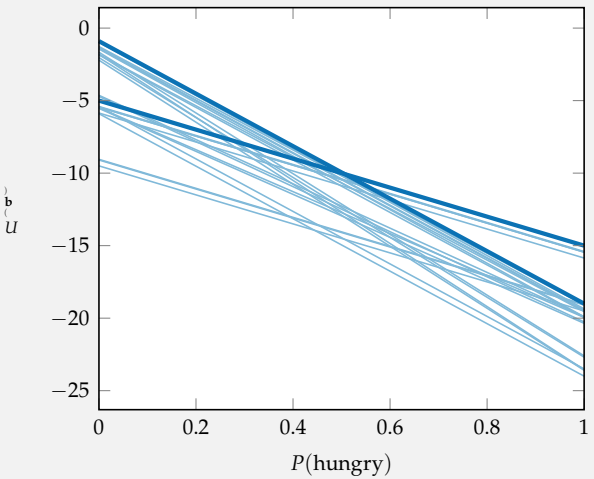
MDP 的值迭代算法可以适用于 POMDP。[6] POMDP *value iteration*（算法20.8）首先构建所有单步计划。我们剔除任何对于任何初始信念都不是最优的计划。然后，我们将所有单步计划的组合扩展为两步计划。再次，我们排除任何次优计划。这种交替进行扩展和剔除的过程一直重复，直到达到所需的时间范围。图20.3展示了在哭泣婴儿问题上的值迭代。

6 本节从条件计划和 alpha 向量的角度描述了一种值迭代版本。若仅使用 alpha 向量的版本，请参见A. R. Cassandra、M. L. Littman 和 N. L. Zhang 的“增量剪枝：一种简单、快速、精确的部分可观测马尔可夫决策过程方法”，收录于 *Conference on Uncertainty in Artificial Intelligence* (*UAI*)，1997。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们可以为哭泣婴儿问题构建所有两步计划。共有$3^3 = 27$个这样的计划。

在信念空间中，每个计划的期望效用如下图所示。我们发现有两个计划优于所有其他计划。这些占优的计划是作为最优三步计划的子计划唯一需要考虑的。



示例20.3。哭泣婴儿问题（附录F.7）所有两步计划的信念空间上的期望效用。粗线对某些信念是最优的，而细线是被支配的。

```
function value_iteration(𝒫::POMDP, k_max)
    𝒮, 𝒜, R = 𝒫.𝒮, 𝒫.𝒜, 𝒫.R
    plans = [ConditionalPlan(a) for a in 𝒜]
    Γ = [[R(s,a) for s in 𝒮] for a in 𝒜]
    plans, Γ = prune(plans, Γ)
    for k in 2:k_max
        plans, Γ = expand(plans, Γ, 𝒫)
        plans, Γ = prune(plans, Γ)
    end
    return (plans, Γ)
end

function solve(M::ValueIteration, 𝒫::POMDP)
    plans, Γ = value_iteration(𝒫, M.k_max)
    return LookaheadAlphaVectorPolicy(𝒫, Γ)
end
```

算法20.8. 用于POMDP的值迭代，通过迭代构造最优子计划，为规划周期为`k_max`的有限周期POMDP找到占主导地位的h步计划。`ValueIteration`结构与在MDP背景下算法7.8中定义的结构相同。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

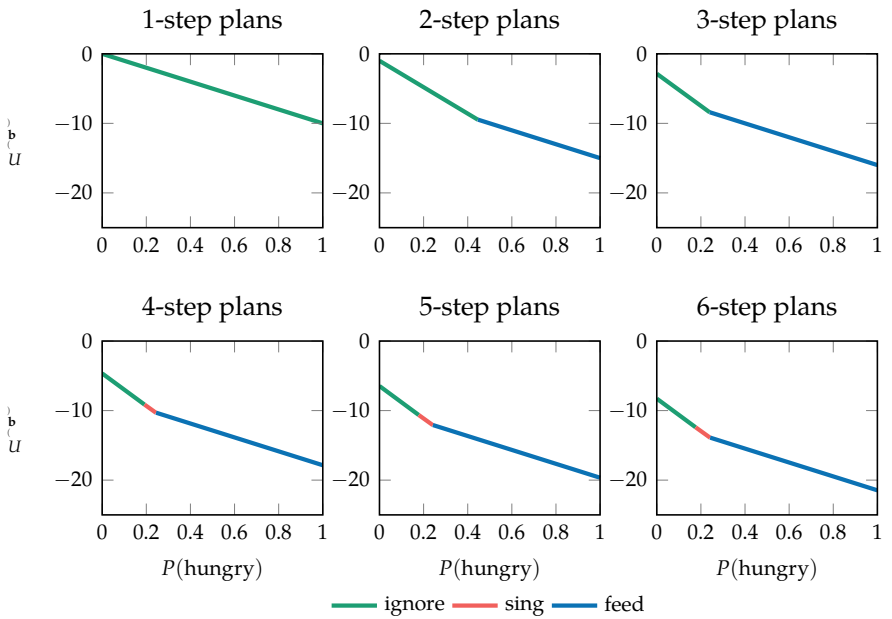


图20.3. 用于寻找哭泣婴儿问题在不同时间范围内最优价值函数的POMDP价值迭代法。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

此过程中的扩展步骤（算法20.9）从一组k步计划构建所有可能的（$k+1$）步计划。如图20.4所示，可以使用新的首动作和作为子计划的所有可能的k步计划组合来构建新计划。虽然计划也可以通过将动作添加到子计划末尾来扩展，但顶层扩展允许为k步计划构建的alpha向量被用来高效地为（$k+1$）步计划构建alpha向量。

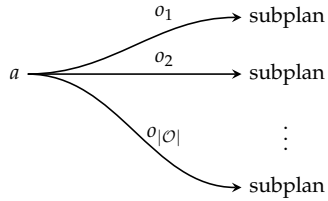


图20.4. 可以使用一个新的初始动作来构建一个$(k+1)$-步计划，该动作导致k-步子计划的任何组合。

从与一个计划的子计划相关联的α向量集合计算与该计划π相关联的α向量，可以按照以下方式进行。我们使用$\boldsymbol{\alpha}_o$来表示与子计划$\pi(o)$相关联的α向量。那么，与π相关联的α向量是

$$\alpha(s) = R(s, \pi()) + \gamma \sum_{s'} T(s' \mid s, \pi()) \sum_{o} O(o \mid \pi(), s') \alpha_o(s') \tag{20.17}$$

即使对于相对简单的问题和较浅的深度，以这种方式从子计划计算alpha向量比从头计算（如算法20.2）要高效得多。

20.6 Linear Policies

如19.3节所述，线性高斯动态问题中的信念状态可以由高斯分布表示，$\mathcal{N}(\boldsymbol{\mu}_b, \boldsymbol{\Sigma}_b)$。如果奖励函数是二次的，那么可以证明最优策略可以通过一个通常称为*linear quadratic Gaussian* (*LQG*)控制的过程精确地离线计算。最优动作的获取方式与7.8节相同，但使用线性高斯滤波器计算出的$\boldsymbol{\mu}_b$被视为真实状态。[7]

对于每一次观察，我们仅需使用滤波器来更新我们的$\boldsymbol{\mu}_b$，并通过将$\boldsymbol{\mu}_b$与算法7.11中的策略矩阵相乘来获得最优动作。例20.4演示了这一过程。

7 我们能够简单地使用分布的均值，这是*certainty equivalence*原理的又一例证，该原理最初在7.8节中引入。

20.7 Summary

- POMDP的精确解通常只能针对有限时域离散POMDP获得。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
function ConditionalPlan(𝒫::POMDP, a, plans)
    subplans = Dict(o⇒π for (o, π) in zip(𝒫.𝒪, plans))
    return ConditionalPlan(a, subplans)
end

function combine_lookahead(𝒫::POMDP, s, a, Γo)
    𝒮, 𝒪, T, O, R, γ = 𝒫.𝒮, 𝒫.𝒪, 𝒫.T, 𝒫.O, 𝒫.R, 𝒫.γ
    U′(s′,i) = sum(O(a,s′,o)*α[i] for (o,α) in zip(𝒪,Γo))
    return R(s,a) + γ*sum(T(s,a,s′)*U′(s′,i) for (i,s′) in enumerate(𝒮))
end

function combine_alphavector(𝒫::POMDP, a, Γo)
    return [combine_lookahead(𝒫, s, a, Γo) for s in 𝒫.𝒮]
end

function expand(plans, Γ, 𝒫)
    𝒮, 𝒜, 𝒪, T, O, R = 𝒫.𝒮, 𝒫.𝒜, 𝒫.𝒪, 𝒫.T, 𝒫.O, 𝒫.R
    plans′, Γ′ = [], []
    for a in 𝒜
        # iterate over all possible mappings from observations to plans
        for inds in product([eachindex(plans) for o in 𝒪]...)
            πo = plans[[inds...]]
            Γo = Γ[[inds...]]
            π = ConditionalPlan(𝒫, a, πo)
            α = combine_alphavector(𝒫, a, Γo)
            push!(plans′, π)
            push!(Γ′, α)
        end
    end
    return (plans′, Γ′)
end
```

算法20.9. 值迭代中的扩展步骤，它从一组k步条件计划及alpha向量构建所有$(k+1)$步条件计划及关联的alpha向量。我们合并子计划的alpha向量的方式遵循方程(20.17)。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

例20.4。用于具有线性高斯动态和二次奖励的POMDP的最优策略。

考虑一颗在二维空间中导航的卫星，忽略重力、阻力和其他外力。卫星可以利用其推进器以线性动力学在任何方向上加速：

$$\begin{bmatrix} x \\ y \\ \dot{x} \\ \dot{y} \end{bmatrix} \leftarrow \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ \dot{x} \\ \dot{y} \end{bmatrix} + \begin{bmatrix} \frac{1}{2}\Delta t^2 & 0 \\ 0 & \frac{1}{2}\Delta t^2 \\ \Delta t & 0 \\ 0 & \Delta t \end{bmatrix} \begin{bmatrix} \dot{x} \\ \ddot{y} \end{bmatrix} + \boldsymbol{\epsilon}$$

其中 Δt 是时间步的持续时间，$\boldsymbol{\epsilon}$ 是均值为零、协方差为 $\Delta t/20\mathrm{I}$ 的高斯噪声。

我们寻求将卫星放置在其位于原点的轨道位置上，同时尽量减少燃料使用。我们的二次奖励函数是：

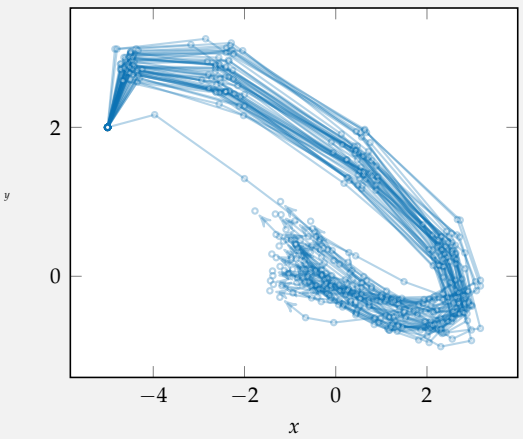
$$R(\mathbf{s}, \mathbf{a}) = -\mathbf{s}^\top \begin{bmatrix} \mathbf{I}_{2\times 2} & \mathbf{0}_{2\times 2} \\ \mathbf{0}_{2\times 2} & \mathbf{0}_{2\times 2} \end{bmatrix} \mathbf{s} - 2\mathbf{a}^\top \mathbf{a}$$

卫星的传感器根据以下方式测量其位置：

$$\mathbf{o} = \begin{bmatrix} \mathbf{I}_{2\times 2} & \mathbf{0}_{2\times 2} \end{bmatrix} \mathbf{s} + \boldsymbol{\varepsilon}$$

wh 其中 $\boldsymbol{\varepsilon}$ 是均值为零、协方差为 $\Delta t/1$ 的高斯噪声 0I.

以下是使用$\Delta t = 1$的最优策略和卡尔曼滤波器来跟踪信念的10步展开的50条轨迹。在每种情况下，卫星从$\mathbf{s} = \boldsymbol{\mu}_b = [-5, 2, 0, 1]$开始，其中$\boldsymbol{\Sigma}_b = [\mathrm{I}\ 0; 0\ 0.25\mathrm{I}]$。



© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

- 这些问题的策略可以表示为条件计划，即根据观测结果描述应采取行动的树。
- Alpha 向量包含从不同状态出发并遵循特定条件计划时的期望效用。
- Alpha 向量也可以作为 POMDP 策略的一种替代表示。
- POMDP值迭代可以通过迭代计算子计划并修剪次优子计划，避免枚举所有条件计划的计算负担。
- 具有二次奖励的线性高斯问题可以使用与完全可观测情况推导的方法非常相似的方法精确求解。

20.8 Exercises

练习 20.1。是否每个 POMDP 都可以被表述为一个 MDP?

Solution: 是的。任何POMDP都可以等价地视为一个信念状态MDP，其状态空间是POMDP中的信念空间，其动作空间与POMDP的动作空间相同，其转移函数由方程(20.2)给出。

练习20.2。一步哭泣婴儿问题（附录F.7）的alpha向量是什么？所有可用的动作都是占优的吗?

Solution: 有三个一步条件计划，分别对应每个动作，从而产生三个alpha向量。在给定当前信念下，最优一步策略必须在这些动作之间做出选择。POMDP的一步alpha向量可从最优一步信念价值函数获得:

$$U^*(b) = \max_a \sum_s b(s)R(s,a)$$

喂养婴儿产生的预期奖励为:

$$\begin{aligned} &R(\text{hungry},\text{feed})P(\text{hungry}) + R(\text{sated},\text{feed})P(\text{sated}) \\ &\quad = -15P(\text{hungry}) - 5(1 - P(\text{hungry})) \\ &\quad = -10P(\text{hungry}) - 5 \end{aligned}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

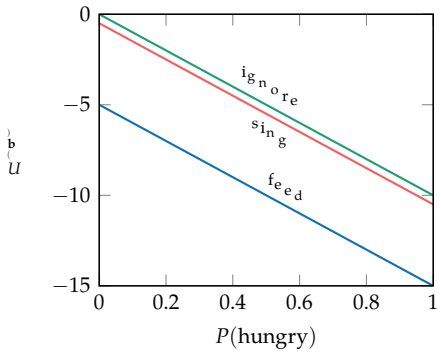
对着婴儿唱歌会产生预期的奖励：

$$\begin{aligned} &R(\text{hungry}, \text{sing})P(\text{hungry}) + R(\text{sated}, \text{sing})P(\text{sated}) \\ &\quad = -10.5P(\text{hungry}) - 0.5(1 - P(\text{hungry})) \\ &\quad = -10P(\text{hungry}) - 0.5 \end{aligned}$$

忽略婴儿会产生预期奖励：

$$\begin{aligned} &R(\text{hungry}, \text{ignore})P(\text{hungry}) + R(\text{sated}, \text{ignore})P(\text{sated}) \\ &\quad = -10P(\text{hungry}) \end{aligned}$$

每个动作的预期奖励在信念空间上的分布如下所示：



我们发现在一步决策时间范围内，喂食或对婴儿唱歌从不是最优选择。忽视这一行为占主导地位。

练习 20.3. 为什么在算法20.8中实现值迭代时，会调用算法20.9中的`expand`，而不是评估算法20.2中的计划来为每个新条件计划获取alpha向量?

Solution: 规划评估方法递归地应用公式(20.8)来评估条件规划的期望效用。随着规划视野的增加，条件规划会变得非常庞大。POMDP值迭代可以通过使用上一轮迭代中子规划的alpha向量来节省计算：

$$U^{\pi}(s) = R(s, \pi()) + \gamma \left[\sum_{s'} T(s' \mid s, \pi()) \sum_{o} O(o \mid \pi(), s') \alpha_{s'}^{\pi(o)} \right]$$

练习 20.4. 条件计划的数量是随动作数量的增加而增长更快，还是随观察数量的增加而增长更快?

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: 回想一下，有$|\mathcal{A}|^{(|\mathcal{O}|^h-1)/(|\mathcal{O}|-1)}$种可能的$h$步计划。指数增长（$n^x$）比多项式增长（$x^n$）更快，而我们在$|\mathcal{O}|$中具有优于指数的增长，在$|\mathcal{A}|$中则呈多项式增长。因此，计划的数量相对于观察次数的增加而增长得更快。为演示这一点，我们以$|\mathcal{A}|$ =3、$|\mathcal{O}|$ =3和h =3为基准。基准情况下有1，594，323个计划。增加动作数量会导致67，108，864个计划，而增加观察次数则会导致10，460，353，203个计划。

练习 20.5. 假设我们有一个病人，我们不确定他们是否患有某种特定疾病。我们确实有三种诊断测试，每种测试正确指示疾病是否存在的概率各不相同。当病人在我们的办公室时，我们可以选择按顺序进行多种诊断测试。我们立即观察每项诊断测试的结果。此外，我们可以多次重复任何诊断测试，鉴于疾病的存在与否，所有测试的结果彼此条件独立。完成测试后，我们决定是治疗疾病还是让病人未接受治疗就回家。解释您如何定义POMDP表述中的各个组成部分。

Solution: 我们有三种状态：

1. $s_{\text{no-disease}}$：患者没有该疾病
2. s_{disease}：患者患有该疾病
3. s_{terminal}：交互结束（终止状态）

我们有五个动作：

1. a_1：管理测试 1
2. a_2：管理测试 2
3. a_3：执行测试 3
4. a_{treat}：实施治疗并让患者回家
5. a_{stop}：让患者未经治疗回家

我们有三个观察结果：

1. $o_{\text{no-disease}}$：检测结果（如果进行了检测）表明患者没有该疾病
2. o_{disease}：检测结果（如果进行了检测）表明患者患有该疾病
3. o_{terminal}：未进行测试

转移模型将是确定性的，其中

$$T(s' \mid s, a) = \begin{cases} 1 & \text{if } a \in \{a_{\text{treat}}, a_{\text{stop}}\} \wedge s' = s_{\text{terminal}} \\ 1 & \text{if } s = s' \\ 0 & \text{otherwise} \end{cases}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

奖励函数将是治疗和每次检测费用的函数，以及如果疾病确实存在而不进行治疗的成本的函数。从s_{terminal}可获得的奖励为0。观察模型根据来自某个非终端的诊断测试结果，为疾病状态的正确和错误观察分配概率。初始信念将赋予我们对患者是否患有该疾病的先验概率，而终态被赋予零概率。

练习20.6。为什么在之前的练习中我们可能需要多次执行相同的测试?

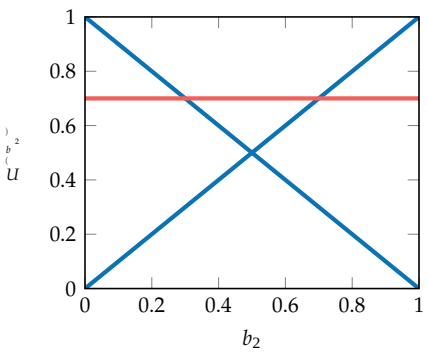
Solution: 根据错误结果的概率，我们可能希望多次进行相同的测试，以提高我们对患者是否患有该疾病的信心。在给定疾病状态的情况下，测试结果是独立的。

练习20.7。假设我们有三个alpha向量，[1, 0]、[0, 1]和[θ, θ]，对于常数θ。在θ满足什么条件时，我们可以修剪alpha向量?

Solution: 如果 $\theta <$ 为0.5或 $\theta >$ 为1，我们可以修剪alpha向量。如果 $\theta <$ 为0.5，那么[θ, θ]被其他两个alpha向量支配。如果 $\theta >$ 为1，则[θ, θ]支配其他两个alpha向量。

练习 20.8。我们有 $\Gamma = \{[1, 0]、[0, 1]\}$ 和 $\boldsymbol{\alpha} = [0.7, 0.7]$。由方程（20.16）中的线性规划所定义的效用差距 δ，使哪个信念 b 最大化?

Solution: Γ 中的 alpha 向量以蓝色显示，alpha 向量 $\boldsymbol{\alpha}$ 以红色显示。我们只关心 $0.3 \leq b_2 \leq 0.7$ 的区域，在该区域中 $\boldsymbol{\alpha}$ 支配 Γ 中的 alpha 向量；换句话说，红线高于蓝线的区域。红线与蓝线最大值之间的差距出现在 $b_2 = 0.5$ 处，差距为 $\delta = 0.2$。因此，使该差距最大化的信念是 $\text{b} = [0.5, 0.5]$。



© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

21 Offline Belief State Planning

在最坏情况下，一般有限视野 POMDP 的精确解是 *PSPACE- complete*，这是一个包含 NP 完全问题、并被认为可能包含更难问题的复杂性类。[1] 一般无限-

Operation Research

无限视界POMDP已被证明是不可计算的。[2]因此，存在一个
最近有大量关于近似方法的研究。本章讨论各种离线POMDP求解方法，这些方法涉及在执行之前进行全部或大部分计算。我们专注于将价值函数表示为alpha向量和不同形式的插值的方法。

21.1 Fully Observable Value Approximation

最简单的离线近似技术之一是 *QMDP*，其名称来源于与完全可观测 MDP 相关的动作价值函数。[3] 这

tional Conference on Machine Learning（*ICML*）

该方法以及本章讨论的其他几种方法，都涉及迭代更新一组 Γ 的 alpha 向量，如算法 21.1 所示。所得到的一组 Γ 定义了一个价值函数和一个策略，可以直接使用，也可以如前一章所述进行一步前瞻使用，尽管所得策略只是最优解的近似。

ligence Research

1 C. Papadimitriou 和 J. Tsitsiklis，“马尔可夫决策过程的复杂性”，*Mathematics of*，第12卷，第3期，第441-450页，1987年。

2 O. Madani, S. Hanks, and A. Condon，“关于概率规划及相关随机优化问题的不可判定性”，*Artificial Intelligence*，第147卷，第1–2期，第5–34页，2003年。

3 M. L. Littman、A. R. Cassandra 和 L. P. Kaelbling，“部分可观测环境中的策略学习：扩展”，载于 *Interna-*，1995年。QMDP 提供最优价值函数上界的证明由 M. Hauskrecht 给出，“部分可观测马尔可夫决策过程的价值函数近似”，*Journal of Artificial Intel-*，第13卷，第33–94页，2000年。

```
function alphavector_iteration(𝒫::POMDP, M, Γ)
    for k in 1:M.k_max
        Γ = update(𝒫, M, Γ)
    end
    return Γ
end
```

算法21.1。用于更新本章中若干方法所使用的alpha向量集合Γ的迭代结构。包括QMDP在内的各种方法在`update`的实现上有所不同。经过`k_max`次迭代后，该函数返回由Γ中的alpha向量表示的策略。

QMDP（算法21.2）使用值迭代为每个动作a构建一个单一的alpha向量$\boldsymbol{\alpha}_a$。每个alpha向量初始化为零，然后我们迭代：

$$\alpha_a^{(k+1)}(s) = R(s,a) + \gamma \sum_{s'} T(s' \mid s, a) \max_{a'} \alpha_{a'}^{(k)}(s') \tag{21.1}$$

每次迭代需要$O(|\mathcal{A}|^2|\mathcal{S}|^2)$次操作。图21.1展示了该过程。

```
struct QMDP
    k_max # maximum number of iterations
end

function update(𝒫::POMDP, M::QMDP, Γ)
    𝒮, 𝒜, R, T, γ = 𝒫.𝒮, 𝒫.𝒜, 𝒫.R, 𝒫.T, 𝒫.γ
    Γ′ = [[R(s,a) + γ*sum(T(s,a,s′)*maximum(α′[j] for α′ in Γ)
        for (j,s′) in enumerate(𝒮)) for s in 𝒮] for a in 𝒜]
    return Γ′
end

function solve(M::QMDP, 𝒫::POMDP)
    Γ = [zeros(length(𝒫.𝒮)) for a in 𝒫.𝒜]
    Γ = alphavector_iteration(𝒫, M, Γ)
    return AlphaVectorPolicy(𝒫, Γ, 𝒫.𝒜)
end
```

算法21.2。QMDP算法，它为具有离散状态和动作空间的无限视界POMDP寻找近似最优策略，其中`k_max`是迭代次数。QMDP假设完全可观测性。

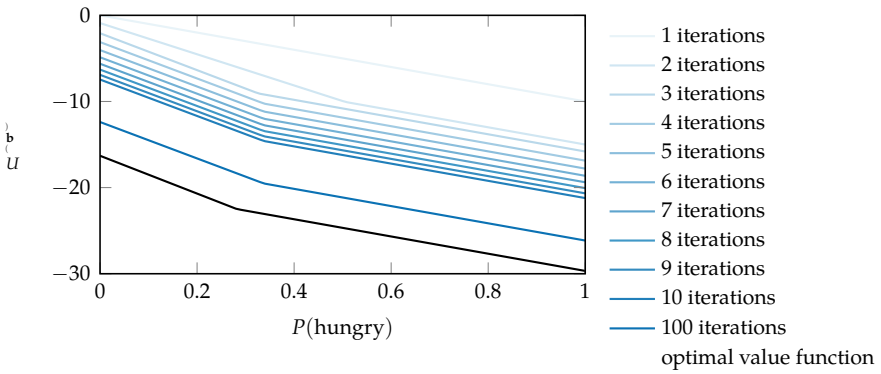


图21.1. 使用QMDP解决哭泣婴儿问题（附录F.7）获得的值函数。在第一次迭代中，单个alpha向量占据主导。在后续迭代中，两个alpha向量占据主导。

在有限时域问题中，当QMDP运行至时域尽头；对于无限时域问题，则运行至收敛时，所得策略等同于假设采取第一步后将实现完全可观测性。因为我们能做得更好

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

只有当具有完全可观测性时，QMDP 才会对真正的最优价值函数 $U^*(\mathbf{b})$ 给出一个上界。换句话说，对于所有 b，都有 $\max_a \boldsymbol{\alpha}_a^\top \mathbf{b} \geq U^*(\mathbf{b})$。[4]

如果QMDP在无限地平线问题上没有运行到收敛，它可能不会提供上界。保证QMDP在有限次迭代后提供上界的一种方法是将值函数初始化为某个上界。一个相当宽松的上界是*best-action best-state upper bound*，即从最佳状态永远采取最佳动作所获得的效用：

$$\overline{U}(b) = \max_{s,a} \frac{R(s,a)}{1-\gamma} \tag{21.2}$$

第一步之后完全可观测性的假设可能导致QMDP对*information-gathering*动作的价值近似不佳，这些动作能显著减少状态中的不确定性。例如，在驾驶时变换车道前回头看是一种信息收集动作。QMDP在最优策略不包含昂贵信息收集的问题上可以表现良好。

我们可以将QMDP方法推广到可能不具有小型离散状态空间的问题。在这类问题中，方程(21.1)中的迭代可能不可行，但我们可以使用前面章节中讨论的众多方法之一来获得近似动作值函数$Q(s、a)$。该值函数可能定义在高维连续状态空间上，例如使用神经网络表示。然后，在信念点上评估的值函数为，

$$U(b) = \max_a \int Q(s,a)b(s)\,\mathrm{d}s \tag{21.3}$$

上述积分可以通过采样来近似。

21.2 Fast Informed Bound

与QMDP一样，*fast informed bound*为每个动作计算一个α向量。然而，快速信息界在一定程度上考虑了观测模型。[5]迭代为：

$$\alpha_a^{(k+1)}(s) = R(s,a) + \gamma \sum_o \max_{a'} \sum_{s'} O(o \mid a, s') T(s' \mid s, a) \alpha_{a'}^{(k)}(s') \tag{21.4}$$

这需要每次迭代进行 $O(|\mathcal{A}|^2|\mathcal{S}|^2|\mathcal{O}|)$ 次操作。

4 尽管由QMDP alpha向量表示的值函数上界了最优值函数，但当然，QMDP策略实现的期望效用不会超过最优策略的期望效用。

5 QMDP与快速信息界之间的关系，连同实证结果，由M. Hauskrecht在“部分可观察马尔可夫决策过程的价值函数近似”一文中讨论，*Journal of Artificial Intelligence Research*，第13卷，第33-94页，2000年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

快速信息界提供了最优值函数的上界。该上界保证不劣于QMDP提供的上界，并且往往更紧。快速信息界在算法21.3中实现，并在图21.2中用于计算最优值函数。

```
struct FastInformedBound
    k_max # maximum number of iterations
end

function update(𝒫::POMDP, M::FastInformedBound, Γ)
    𝒮, 𝒜, 𝒪, R, T, O, γ = 𝒫.𝒮, 𝒫.𝒜, 𝒫.𝒪, 𝒫.R, 𝒫.T, 𝒫.O, 𝒫.γ
    Γ′ = [[R(s, a) + γ*sum(maximum(sum(O(a,s′,o)*T(s,a,s′)*α′[j]
        for (j,s′) in enumerate(𝒮)) for α′ in Γ) for o in 𝒪)
        for s in 𝒮] for a in 𝒜]
    return Γ′
end

function solve(M::FastInformedBound, 𝒫::POMDP)
    Γ = [zeros(length(𝒫.𝒮)) for a in 𝒫.𝒜]
    Γ = alphavector_iteration(𝒫, M, Γ)
    return AlphaVectorPolicy(𝒫, Γ, 𝒫.𝒜)
end
```

算法21.3。快速信息界算法，该算法为具有离散状态、动作和观察空间的无限 horizon POMDP 找到近似最优策略，其中 `k_max` 是迭代次数。

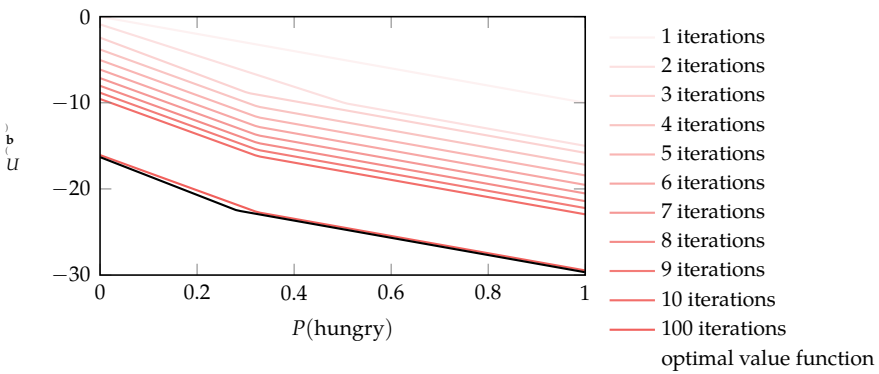


图21.2. 使用快速信息界限为哭泣婴儿问题获得的值函数。经过10次迭代后的值函数明显低于QMDP算法的值函数。

21.3 *Fast Lower Bounds*

前两节介绍了可用于生成表示为alpha向量的值函数上界的方法。本节将介绍

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

几种无需在信念空间中进行规划就能快速生成表示为 alpha 向量的下界的方法。尽管上界方法通常可以直接用于生成合理的策略，但本节讨论的下界通常仅用于为其他规划算法提供初始值。图21.3 绘制了本节讨论的两种下界方法。

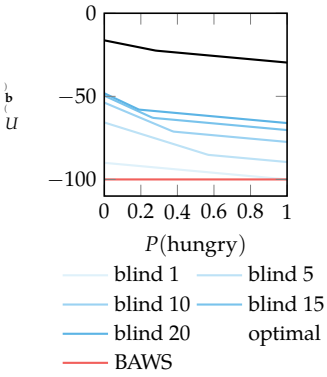


图21.3. 不同迭代次数下的盲下界及BAWS下界在哭泣婴儿问题中的应用。

一个常见的下界是*best-action worst-state* (*BAWS*) *lower bound* (算法21.4)。它表示永远在最坏状态下采取最佳动作所获得的折扣奖励：

$$r_{\text{baws}} = \max_a \sum_{k=1}^{\infty} \gamma^{k-1} \min_s R(s,a) = \frac{1}{1-\gamma} \max_a \min_s R(s,a) \tag{21.5}$$

这个下界由一个单一的alpha向量表示。这个界限通常非常宽松，但它可以用来启动其他能收紧这一界限的算法，我们很快会讨论到这一点。

```
function baws_lowerbound(𝒫::POMDP)
    𝒮, 𝒜, R, γ = 𝒫.𝒮, 𝒫.𝒜, 𝒫.R, 𝒫.γ
    r = maximum(minimum(R(s, a) for s in 𝒮) for a in 𝒜) / (1-γ)
    α = fill(r, length(𝒮))
    return α
end
```

算法21.4. 方程(21.5)中的最佳行动最差状态下界实现，表示为α向量。

blind lower bound (算法 21.5)表示一个下界，其中每个动作对应一个 alpha 向量。其假设是我们被迫永远坚持单一动作，对未来观察到的情况视而不见。为了计算这些 alpha 向量，我们从另一个下界（通常是最佳动作最差状态下界）开始，然后执行若干次迭代：

$$\alpha_a^{(k+1)}(s) = R(s,a) + \gamma \sum_{s'} T(s' \mid s,a) \alpha_a^{(k)}(s') \tag{21.6}$$

这次迭代与公式(21.1)中的QMDP更新类似，只是右侧没有对{alpha}向量进行最大化处理。

21.4 Point-Based Value Iteration

QMDP和快速信息边界为每个动作生成一个alpha向量，但最优价值函数通常可以通过更多向量来更好地近似。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
function blind_lowerbound(𝒫, k_max)
    𝒮, 𝒜, T, R, γ = 𝒫.𝒮, 𝒫.𝒜, 𝒫.T, 𝒫.R, 𝒫.γ
    Q(s,a,α) = R(s,a) + γ*sum(T(s,a,s′)*α[j] for (j,s′) in enumerate(𝒮))
    Γ = [baws_lowerbound(𝒫) for a in 𝒜]
    for k in 1:k_max
        Γ = [[Q(s,a,α) for s in 𝒮] for (α,a) in zip(Γ, 𝒜)]
    end
    return Γ
end
```

算法21.5. 以一组alpha向量表示的盲下界的实现。

alpha 向量。*Point-based value iteration*[6] 计算 m 个不同的 alpha 向量 $\Gamma = \{\boldsymbol{\alpha}_1, \ldots, \boldsymbol{\alpha}_m\}$，每个都与不同的信念点 $B = \{\mathbf{b}_1, \ldots, \mathbf{b}_m\}$ 相关联。选择这些信念的方法将在21.7节讨论。和之前一样，这些alpha向量定义了一个近似最优值函数：

$$U^{\Gamma}(\mathbf{b}) = \max_{\boldsymbol{\alpha} \in \Gamma} \boldsymbol{\alpha}^{\top} \mathbf{b} \tag{21.7}$$

6 一项关于基于点的值迭代方法的综述由 G. Shani、J. Pineau 和 R. Kaplow 提供，“基于点的 POMDP 求解器综述”，*Autonomous Agents and Multi-Agent Systems*，第 27 卷，第 1–51 页，2012 年。该参考文献提供了一种略有不同的计算信念备份的方法，尽管结果相同。

该算法维护最优值函数的下界，$U^{\Gamma}(\mathbf{b}) \leq U^{*}(\mathbf{b})$ 对所有 b 成立。我们将 alpha 向量初始化为下界，然后执行一次 *backup* 来更新 B 中每个点处的 alpha 向量。备份操作（算法 21.6）接收一个信念 b 和一组 alpha 向量 Γ，并构造一个新的 alpha 向量。该算法遍历每个可能的动作 a 和观测 o，并从 Γ 中提取在所得信念状态下最大化的 alpha 向量：

$$\boldsymbol{\alpha}_{a,o} = \arg\max_{\boldsymbol{\alpha} \in \Gamma} \boldsymbol{\alpha}^{\top} \text{Update}(\mathbf{b}, a, o) \tag{21.8}$$

然后，对于每个可用的动作 a，我们基于这些 $\boldsymbol{\alpha}_{a,o}$ 向量构造一个新的 alpha 向量：

$$\alpha_a(s) = R(s,a) + \gamma \sum_{s',o} O(o \mid a, s') T(s' \mid s, a) \alpha_{a,o}(s') \tag{21.9}$$

最终通过备份操作产生的 alpha 向量是 {v*}

$$\boldsymbol{\alpha} = \arg\max_{\boldsymbol{\alpha}_a} \boldsymbol{\alpha}_a^{\top} \mathbf{b} \tag{21.10}$$

如果{v*}是一个下界，那么备份操作将仅产生同时也是下界的alpha向量。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

对 B 中的信念反复应用备份操作，会逐步提高由 alpha 向量表示的价值函数的下界，直至收敛。收敛后的价值函数不一定是最优的，因为 B 通常不包含从初始信念可达的所有信念。然而，只要 B 中的信念在可达信念空间中分布良好，该近似可能是可接受的。无论如何，所得到的价值函数保证提供一个下界，可与其他算法（可能在线）结合使用，以进一步改进策略。

基于点的价值迭代在算法21.7中实现。图21.4展示了在一个示例问题上的几次迭代。

```
function backup(𝒫::POMDP, Γ, b)
    𝒮, 𝒜, 𝒪, γ = 𝒫.𝒮, 𝒫.𝒜, 𝒫.𝒪, 𝒫.γ
    R, T, O = 𝒫.R, 𝒫.T, 𝒫.O
    Γa = []
    for a in 𝒜
        Γao = []
        for o in 𝒪
            b′ = update(b, 𝒫, a, o)
            push!(Γao, argmax(α→α⋅b′, Γ))
        end
        α = [R(s, a) + γ*sum(sum(T(s, a, s′)*O(a, s′, o)*Γao[i][j]
            for (j,s′) in enumerate(𝒮)) for (i,o) in enumerate(𝒪))
            for s in 𝒮]
        push!(Γa, α)
    end
    return argmax(α→α⋅b, Γa)
end
```

算法21.6。一种为具有离散状态和动作空间的POMDP备份信念的方法，其中Γ是alpha向量组成的向量，b是要在其上应用备份的信念向量。用于向量信念的update方法在算法19.2中定义。

21.5 Randomized Point-Based Value Iteration

Randomized point-based value iteration (算法21.8)是上一节基于点的值迭代方法的一种变体。[7]主要区别

事实是，我们不必在 B 中的每个信念处都维护一个 alpha 向量。我们用 Γ 中的单个 alpha 向量初始化算法，然后在每次迭代时更新 Γ，根据需要可能增加或减少 Γ 中的 alpha 向量数量。对更新步骤的这种修改可以提高效率。

7 M. T. J. Spaan 和 N. A. Vlassis, "Perseus：基于随机点的POMDP值迭代," *Journal of Artificial Intelligence Research*, 第24卷, 第195–220页, 2005年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct PointBasedValueIteration
    B     # set of belief points
    k_max # maximum number of iterations
end

function update(𝒫::POMDP, M::PointBasedValueIteration, Γ)
    return [backup(𝒫, Γ, b) for b in M.B]
end

function solve(M::PointBasedValueIteration, 𝒫)
    Γ = fill(baws_lowerbound(𝒫), length(𝒫.𝒜))
    Γ = alphavector_iteration(𝒫, M, Γ)
    return LookaheadAlphaVectorPolicy(𝒫, Γ)
end
```

算法21.7. 基于点的值迭代，该算法为具有离散状态、动作和观察空间的无限 horizon POMDP 寻找近似最优策略，其中 `B` 是信念向量，`k_max` 是迭代次数。

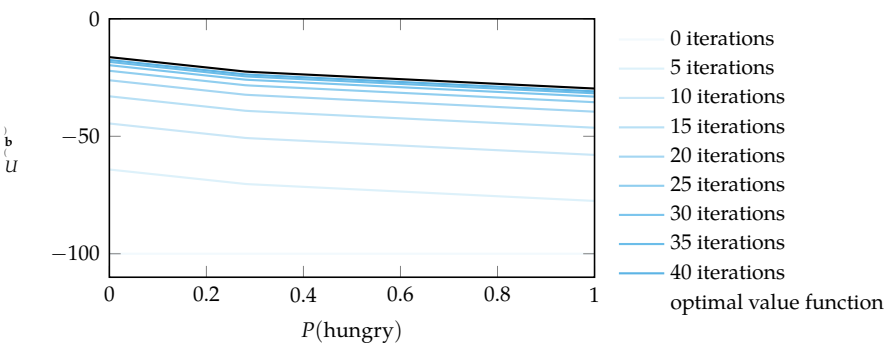


图21.4。在哭泣婴儿问题上使用基于点的值迭代得到的近似价值函数，信念向量为[1/4, 3/4]和[3/4, 1/4]。与QMDP和快速信息界限不同，基于点的值迭代的价值函数始终是真实价值函数的下界。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

每次更新以一组 alpha 向量 Γ 为输入，并输出一组 alpha 向量 Γ'，这些向量改进了由 Γ 表示的值函数在 B 中的信念点处的值。换句话说，它输出 Γ'，使得对于所有 $\mathrm{b} \in B$，有 $U^{\Gamma'}(\mathrm{b}) \geq U^{\Gamma}(\mathrm{b})$。我们首先将 Γ' 初始化为空集，并将 B' 初始化为 B。然后我们从 B' 中随机移除一个点 b，并对 b 执行信念备份（算法 21.6），使用 Γ 得到一个新的 alpha 向量 $\boldsymbol{\alpha}$。接着我们找到 $\Gamma \cup \{\boldsymbol{\alpha}\}$ 中在 b 处占优的 alpha 向量，并将其加入 Γ'。所有在 B' 中因该 alpha 向量而值得到改进的信念点随后从 B' 中移除。随着算法的推进，B' 变得更小，并包含尚未被 Γ' 改进的点集。当 B' 为空时，更新结束。图 21.5 用哭泣婴儿问题说明了这一过程。

```
struct RandomizedPointBasedValueIteration
    B     # set of belief points
    k_max # maximum number of iterations
end

function update(𝒫::POMDP, M::RandomizedPointBasedValueIteration, Γ)
    Γ′, B′ = [], copy(M.B)
    while !isempty(B′)
        b = rand(B′)
        α = argmax(α→α⋅b, Γ)
        α′ = backup(𝒫, Γ, b)
        if α′⋅b ≥ α⋅b
            push!(Γ′, α′)
        else
            push!(Γ′, α)
        end
        filter!(b→maximum(α⋅b for α in Γ′) <
            maximum(α⋅b for α in Γ), B′)
    end
    return Γ′
end

function solve(M::RandomizedPointBasedValueIteration, 𝒫)
    Γ = [baws_lowerbound(𝒫)]
    Γ = alphavector_iteration(𝒫, M, Γ)
    return LookaheadAlphaVectorPolicy(𝒫, Γ)
end
```

算法21.8。随机化基于点的值迭代，该算法为具有离散状态、动作和观察空间的无限时域POMDP寻找近似最优策略，其中`B`是信念向量，`k_max`是迭代次数。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

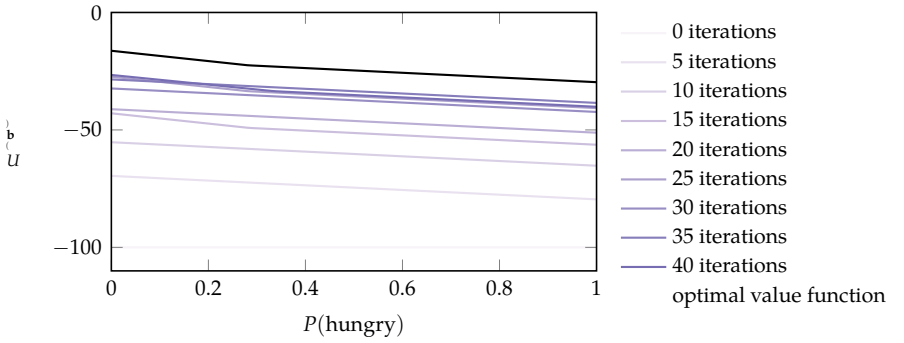


图21.5. 在哭泣婴儿问题上，使用基于点的随机值迭代方法，在信念点[1/4, 3/4]和[3/4, 1/4]处获得的近似价值函数。

21.6 Sawtooth Upper Bound

sawtooth upper bound 是表示价值函数的另一种方式。我们不是存储一组 alpha 向量 Γ，而是存储一组信念-效用对：

$$V = \{(\mathbf{b}_1, \mathbf{u}_1), \ldots, (\mathbf{b}_m, \mathbf{u}_m)\} \tag{21.11}$$

其中 $u_i = U(\mathrm{b}_i)$。这些对必须包含所有标准基信念：

$$E = \{\mathbf{e}_1 = [1, 0, \ldots, 0], \ldots, \mathbf{e}_n = [0, 0, \ldots, 1]\} \tag{21.12}$$

换句话说，

$$\{(\mathbf{e}_1, U(\mathbf{e}_1)), \ldots, (\mathbf{e}_n, U(\mathbf{e}_n))\} \subseteq V \tag{21.13}$$

如果这些效用是上界（例如，从快速知情界获得的），那么我们使用V来估计任意信念b处的$U(\mathrm{b})$的方式将导致上界。[8]

“锯齿”这个名字来源于我们通过在 V 中插值点来估计 $U(\mathrm{b})$ 的方式。对于 V 中的每个信念-效用对 $(\mathrm{b}, U(\mathrm{b}))$，我们形成一个单一的、尖的“齿”。当考虑多个对时，它形成“锯齿”形状。如果信念空间是 n 维的，每个齿是一个倒置的 n 维金字塔。每个金字塔的底部由标准基信念 $(\mathrm{e}_i, U(\mathrm{e}_i))$ 形成。每个金字塔的顶点对应于一个信念-效用对 $(\mathrm{b}, U(\mathrm{b})) \in V$。每个金字塔的壁可以由超平面定义。多个金字塔的组合形成 n 维锯齿。在任何信念处的锯齿上界是这些金字塔在该信念处的最小值。

8 锯齿波与其他边界之间的关系由 M. Hauskrecht 在“部分可观测马尔可夫决策过程的价值函数近似”中讨论，*Journal of Artificial Intelligence Research*，第13卷，第33–94页，2000年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

考虑两状态POMDP中的锯齿表示，例如图21.6所示的哭泣婴儿问题。每个齿的角是对应于每个标准基信念$\mathbf{e}_i$的值$U(\mathbf{e}_1)$和$U(\mathbf{e}_2)$。每个齿的尖锐低点是值$U(\mathbf{b})$，因为每个齿是一个点集对$(\mathbf{b}, U(\mathbf{b}))$。从$U(\mathbf{e}_1)$到$U(\mathbf{b})$的线性插值，以及再次从$U(\mathbf{b})$到$U(\mathbf{e}_2)$，构成了齿。为了组合多个齿并形成上界，我们在任意信念处取最小插值，从而形成独特的锯齿形状。如图21.6所示，锯齿上界是一个分段线性函数，但它不是凸的，这与本章前面讨论的alpha向量表示形成对比。

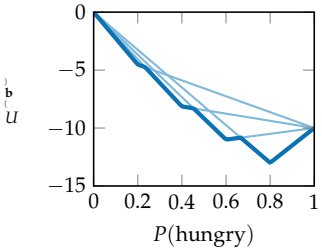


图21.6. 应用于哭闹婴儿问题的锯齿上界表示。

我们可以将双状态情况下的直观理解扩展到任意数量状态，从而计算集合V中的配对对于任意信念$\mathbf{b}$的锯齿形上界$U^V(\mathbf{b})$。首先，我们将定义如何计算与V中的非基配对$(\mathbf{b}', u')$相关联的单个齿的上界。为了定义这个上界$U_{(\mathbf{b}',u')}(b)$，我们将使用$\mathbf{u}_E = [U(\mathbf{e}_1), \ldots, U(\mathbf{e}_n)]$，这是一个包含所有基信念处效用值的向量。我们还将使用所谓的最小比率：

$$\rho(\mathbf{b}, \mathbf{b}') = \min_i \frac{b_i}{b'_i} \tag{21.14}$$

事实证明，上界可以写成

$$U^{(\mathbf{b}',u')}(\mathbf{b}) = \mathbf{u}_E^\top \mathbf{b} + \rho(\mathbf{b}, \mathbf{b}')(\mathbf{u}' - \mathbf{u}_E^\top \mathbf{b}') \tag{21.15}$$

要计算由V定义的所有齿的锯齿上界，我们对所有齿取最小值：

$$U^V(\mathbf{b}) = \min_{(\mathbf{b}',u') \in V | \mathbf{b}' \notin E} U^{(\mathbf{b}',u')}(\mathbf{b}) \tag{21.16}$$

算法21.9提供了实现。我们也可以通过贪心一步前瞻来推导策略。

我们可以在一组信念B上迭代应用贪婪一步前瞻，以收紧对上限的估计。B中的信念可以是V中信念的超集。算法21.10提供了这一实现。例21.1展示了锯齿近似在哭泣婴儿问题上的多次迭代效果。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct SawtoothPolicy
    𝒫 # POMDP problem
    V # dictionary mapping beliefs to utilities
end

function basis(𝒫)
    n = length(𝒫.𝒮)
    e(i) = [j == i ? 1.0 : 0.0 for j in 1:n]
    return [e(i) for i in 1:n]
end

function utility(π::SawtoothPolicy, b)
    𝒫, V = π.𝒫, π.V
    if haskey(V, b)
        return V[b]
    end
    uE = [V[e] for e in E]
    ρ(b′) = minimum(b[i] / b′[i] for i in eachindex(b) if b′[i] > 0)
    U(b′, u′) = uE⋅b + ρ(b′)*(u′ - uE⋅b′)
    return minimize(U(b′, u′) for (b′, u′) in V if b′ ∉ E)
end

(π::SawtoothPolicy)(b) = greedy(π, b).a
```

算法21.9. 值函数和策略的锯齿状上界表示。它通过一个字典`V`定义，该字典将信念向量映射到其获得的效用的上界，例如来自快速信息界。这种表示的一个要求是`V`包含标准基信念下的信念-效用对，这些可以从`basis`函数获得。我们可以使用一步前瞻从任意信念`b`获得贪婪动作-效用对。

```
struct SawtoothIteration
    V     # initial mapping from beliefs to utilities
    B     # beliefs to compute values including those in V map
    k_max # maximum number of iterations
end

function solve(M::SawtoothIteration, 𝒫::POMDP)
    E = basis(𝒫)
    π = SawtoothPolicy(𝒫, M.V)
    for k in 1:M.k_max
        V = Dict(b ⇒ (b ∈ E ? M.V[b] : greedy(π, b).u) for b in M.B)
        π = SawtoothPolicy(𝒫, V)
    end
    return π
end
```

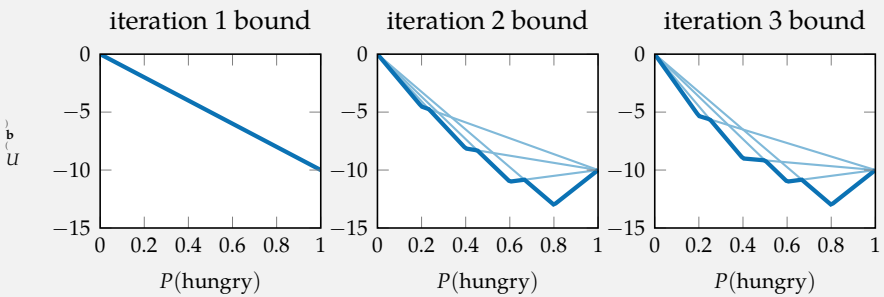
算法21.10. 锯齿迭代在`B`中的点上迭代地应用一步前瞻，以改进`V`中点的效用估计。`B`中的信念是`V`中包含的信念的超集。为了在每次迭代中保持上界，不更新存储在`E`中的标准基信念。我们运行`k_max`次迭代。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

假设我们要为哭泣的婴儿问题维持一个上界值，其中信念点以0.2的步长均匀分布。为了获得初始上界，我们采用快速信息上界。接着，我们可以如下进行三步锯齿迭代：

```
n = length(𝒫.𝒮)
πfib = solve(FastInformedBound(1), 𝒫)
V = Dict(e ⇒ utility(πfib, e) for e in basis(𝒫))
B = [[p, 1 - p] for p in 0.0:0.2:1.0]
π = solve(SawtoothIteration(V, B, 2), 𝒫)
```

锯齿上界改善如下：



例21.1. 一个示例，说明锯齿波在哭泣婴儿问题中，能够在定期分布的信念上保持一个上限的能力。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

21.7 Point Selection

像基于点的值迭代和锯齿迭代这样的算法需要一组信念 B。我们希望选择 B，以便在信念空间的相关区域中有更多的点；我们不想在那些在我们（希望是近似最优的）策略下不太可能到达的信念上浪费计算。探索潜在可达空间的一种方法是在信念空间中进行步进（算法21.11）。步进的结果将是随机的，因为观测是根据我们的概率模型生成的。

```
function randstep(𝒫::POMDP, b, a)
    s = rand(SetCategorical(𝒫.𝒮, b))
    s′, r, o = 𝒫.TRO(s, a)
    b′ = update(b, 𝒫, a, o)
    return b′, r
end
```

算法21.11. 在问题$\mathcal{P}$中，给定当前信念b和行动a，随机抽样下一个信念b′和奖励r的函数。

我们可以从某个初始信念状态出发，在随机策略下可到达的信念状态创建B。这种*random belief expansion*过程（算法21.12）可能会探索比必要范围大得多的信念空间；随机策略可到达的信念空间可能远大于最优策略可到达的空间。当然，计算最优策略可到达的信念空间通常需要知道最优策略，而这正是我们最初想要计算的。可以采取的一种方法是使用最优策略的逐次逼近来迭代生成B。[9]

[9] 这就是被称为*Successive Approximations of the Reachable Space under Optimal Policies*（*SARSOP*）的算法背后的直觉。H. Kurniawati、D. Hsu和W. S. Lee，“SARSOP：通过近似最优可达信念空间进行高效基于点的POMDP规划”，载于*Robotics: Science*，2008年。

除了希望我们的信念点集中在可达信念空间上，我们还希望这些点分散开来，以便更好地近似价值函数。与B中的点相关联的alpha向量所提供的近似质量，在我们评估距离B更远的点时下降。我们可以采用*exploratory belief expansion*方法（算法21.13），其中我们尝试B中每个信念的每个动作，并添加与集合中已有信念距离最远的所得到的信念状态。信念空间中的距离可以用不同方式衡量。该算法使用L_1范数。[10] 图21.7展示了

[10] b 与 b′ 之间的 L_1 距离为 $\sum_s |b(s) - b'(s)|$，记为 $\|\text{b} - \text{b}'\|_1$。参见附录 A.4。

使用此方法向B添加信念点的一个示例。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
function random_belief_expansion(𝒫, B)
    B′ = copy(B)
    for b in B
        a = rand(𝒫.𝒜)
        b′, r = randstep(𝒫, b, a)
        push!(B′, b′)
    end
    return unique!(B′)
end
```

算法21.12。基于可达信念，随机扩展用于基于点的值迭代的有限信念集 {v*}。

```
function exploratory_belief_expansion(𝒫, B)
    B′ = copy(B)
    for b in B
        best = (b=copy(b), d=0.0)
        for a in 𝒫.𝒜
            b′, r = randstep(𝒫, b, a)
            d = minimum(norm(b - b′, 1) for b in B′)
            if d > best.d
                best = (b=b′, d=d)
            end
        end
        push!(B′, best.b)
    end
    return unique!(B′)
end
```

算法21.13。通过探索可达信念并添加与当前信念距离最远的信念，扩展用于基于点的值迭代的有限信念集{v*}。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

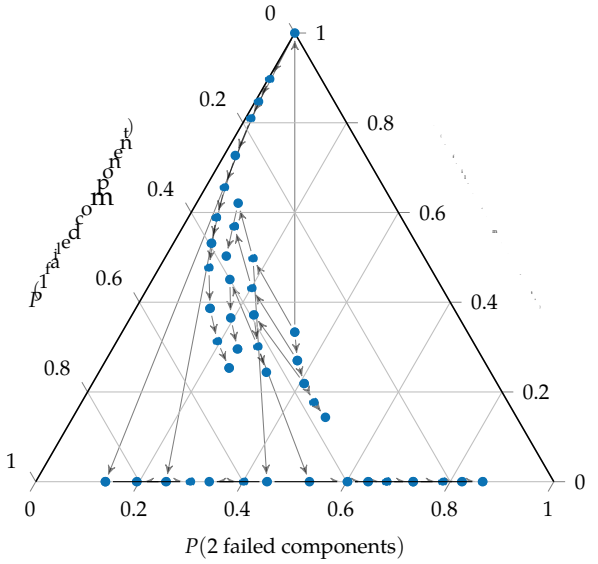


图21.7. 在三状态机器替换问题上进行的探索性信念扩展运行，从初始均匀信念b = [1/3, 1/3, 1/3]开始。如果与任何先前信念的距离至少为0.05，则添加新信念。

21.8 Sawtooth Heuristic Search

第9章在全可观测背景下介绍了启发式搜索作为一种在线方法的概念。本节讨论*sawtooth heuristic search*（算法21.14）作为一种离线方法，它生成一组可用于表示离线策略的alpha向量。然而，与下一章讨论的在线POMDP方法一样，计算工作集中在从某个指定初始信念可达的信念上。驱动可达信念空间探索的启发式方法是价值函数上界与下界之间的差距。[11]

Uncertainty in Artificial Intelligence (UAI)

该算法初始化时，使用一组锯齿状信念-效用对V表示值函数的上界，同时利用一组alpha向量Γ表示值函数的下界。定义锯齿状上界的信念-效用对可通过快速信息界获得。下界则可从最佳行动最差状态界获得，如算法21.14所示，或采用其他方法，如基于点的值迭代。

Systems

11 *heuristic search value iteration*算法（HSVI）引入了基于锯齿波的动作启发式和基于间隙的观察启发式的概念。T. Smith和R. G. Simmons，“用于POMDP的启发式搜索值迭代”，收录于*Conference on*，2004年。SARSOP算法在此基础上进一步发展。H. Kurniawati、D. Hsu和W. S. Lee，“SARSOP：通过近似最优可达信念空间进行高效基于点的POMDP规划”，收录于*Robotics: Science and*，2008年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct SawtoothHeuristicSearch
    b     # initial belief
    δ     # gap threshold
    d     # depth
    k_max # maximum number of iterations
    k_fib # number of iterations for fast informed bound
end

function explore!(M::SawtoothHeuristicSearch, 𝒫, πhi, πlo, b, d=0)
    𝒮, 𝒜, 𝒪, γ = 𝒫.𝒮, 𝒫.𝒜, 𝒫.𝒪, 𝒫.γ
    ϵ(b′) = utility(πhi, b′) - utility(πlo, b′)
    if d ≥ M.d || ϵ(b) ≤ M.δ / γ^d
        return
    end
    a = πhi(b)
    o = argmax(o → ϵ(update(b, 𝒫, a, o)), 𝒪)
    b′ = update(b, 𝒫, a, o)
    explore!(M, 𝒫, πhi, πlo, b′, d+1)
    if b′ ∉ basis(𝒫)
        πhi.V[b′] = greedy(πhi, b′).u
    end
    push!(πlo.Γ, backup(𝒫, πlo.Γ, b′))
end

function solve(M::SawtoothHeuristicSearch, 𝒫::POMDP)
    πfib = solve(FastInformedBound(M.k_fib), 𝒫)
    Vhi = Dict(e ⇒ utility(πfib, e) for e in basis(𝒫))
    πhi = SawtoothPolicy(𝒫, Vhi)
    πlo = LookaheadAlphaVectorPolicy(𝒫, [baws_lowerbound(𝒫)])
    for i in 1:M.k_max
        explore!(M, 𝒫, πhi, πlo, M.b)
        if utility(πhi, M.b) - utility(πlo, M.b) < M.δ
            break
        end
    end
    return πlo
end
```

算法21.14. 锯齿启发式搜索策略。求解器从信念`b`开始，探索深度至`d`，迭代次数不超过`k_max`。它使用通过`k_fib`次迭代计算的快速信息上界获得的上界。下界来自最佳行动最差状态界。差距阈值为`δ`。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

在每次迭代中，我们探索从初始信念可达的信念，直至最大深度。在探索过程中，我们更新构成锯齿上界的信念-动作对集合以及构成下界的alpha向量集合。我们在一定次数的迭代后停止探索，或者直到初始状态处的差距低于阈值$\delta > 0$。

当我们在探索过程中从初始节点出发遇到信念 b 时，我们检查 b 处的差距是否低于阈值 δ/γ^d，其中 d 是我们当前的深度。如果低于该阈值，那么我们可以停止沿该分支探索。我们希望阈值随着 d 的增加而增加，因为更新后 b 处的差距至多是立即可达信念处差距加权平均的 γ 倍。

如果b处的差距高于阈值，且我们尚未达到最大深度，那么我们可以探索下一个信念，b'。首先，我们根据锯齿策略确定推荐的动作a。然后，我们选择使结果信念处差距最大化的观测o。[12]我们递归地向下探索树。

在探索了b'的后代之后，我们将(b', u)添加到V中，其中u是b'的一步前瞻值。我们将b'处备份得到的alpha向量添加到Γ中。图21.8展示了边界的收紧。

12 有些变体仅对下一个观察结果进行采样。其他变体则选择那些因其可能性而加权最大化的观察结果。

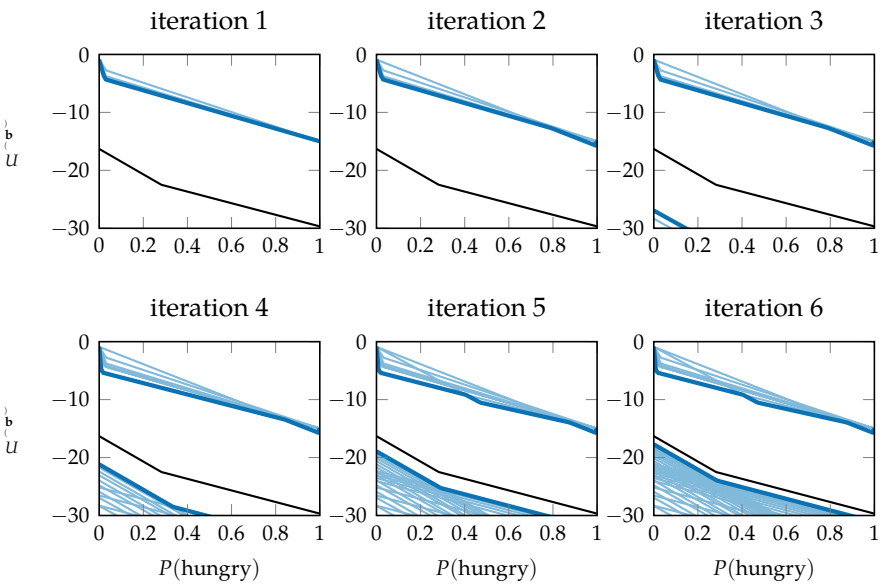


图21.8。上界（由锯齿对表示）和下界（由婴儿哭泣问题的alpha向量表示）的演变过程。最优价值函数以黑色显示。

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

21.9 *Triangulated Value Functions*

如第20.1节所述，POMDP可以转换为信念状态MDP。该信念状态MDP中的状态空间是连续的，对应于原始POMDP中可能的信念空间。我们可以采用类似第8章所述的方法来近似值函数，然后对近似结果应用诸如值迭代之类的动态规划算法。本节讨论一种特定类型的局部值函数近似，涉及在离散信念点集B上的*Freudenthal triangulation*[13]。这

三角剖分允许我们在信念空间中的任意点插值价值函数。与锯齿表示一样，我们使用一组信念-效用对 $V = \{(b, U(b)) \mid b \in B\}$ 来表示我们的价值函数。这种方法可用于获得价值函数的上界。

信念空间中的弗赖登塔尔插值涉及将信念点均匀分布在B空间中，如图21.9所示。B中的信念数量取决于弗赖登塔尔三角剖分的维度n和粒度m：[14]

$$|B| = \frac{(m+n-1)!}{m!(n-1)!} \tag{21.17}$$

我们可以通过在B中的离散点处插值来估计任意点b处的$U(\mathrm{b})$。与第8.5节介绍的单形插值类似，我们找到B中构成包含b的单形的信念点集合，并将它们的值加权合并。在n维信念空间中，最多有$n+$个1顶点，其值需要加权合并。如果$b^{(1)}$，。。。，$b^{(n+1)}$是包围点，λ_1，。。。，λ_{n+1}是它们的权重，那么在b处的值的估计为

$$U(b) = \sum_i \lambda_i U(b^{(i)}) \tag{21.18}$$

算法21.15从V中的配对里提取出该效用函数和策略。

算法21.16将近似值迭代（在算法8.1中引入）的一种变体应用于我们的三角化策略表示。我们仅需使用单步前瞻与值函数插值，在B中对信念进行迭代备份。如果U初始化为一个上界，则即使经过有限次迭代，值迭代也将得到一个上界。这一性质成立，是因为值函数是凸的，而值函数上顶点之间的线性插值必须位于底层凸函数之上或之上。[15]

图21.10展示了一个策略和效用函数的示例。

13 H. Freudenthal，“Simplizialzerlegungen von Beschränkter Flachheit，” *Annals of Mathematics*，第43卷，第580–582页，1942年。这种三角剖分方法被应用于POMDPs中，见W. S. Lovejoy，“Computationally Feasible Bounds for Partially Observed Markov Decision Processes，” *Operations Re-*，第39卷，第1期，第162–175页，1991年。

14 `FreudenthalTriangulations.jl`提供了生成这些信念的实现。

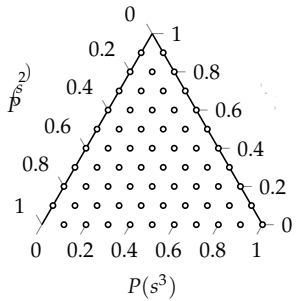


图21.9. 使用Freudenthal三角化在$n=$维信念空间中进行信念状态离散化，粒度为$m=$10。

15 参见 W. S. Lovejoy 的引理4，“部分可观测马尔可夫决策过程的计算可行边界”，*Operations Re-*，第39卷，第1期，第162–175页，1991年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct TriangulatedPolicy
    𝒫 # POMDP problem
    V # dictionary mapping beliefs to utilities
    B # beliefs
    T # Freudenthal triangulation
end

function TriangulatedPolicy(𝒫::POMDP, m)
    T = FreudenthalTriangulation(length(𝒫.𝒮), m)
    B = belief_vertices(T)
    V = Dict(b ⇒ 0.0 for b in B)
    return TriangulatedPolicy(𝒫, V, B, T)
end

function utility(π::TriangulatedPolicy, b)
    B, λ = belief_simplex(π.T, b)
    return sum(λi*π.V[b] for (λi, b) in zip(λ, B))
end

(π::TriangulatedPolicy)(b) = greedy(π, b).a
```

算法21.15。使用粒度m的Freudenthal三角剖分的策略表示。与锯齿方法一样，我们维护一个将信念向量映射到效用的字典。此实现将效用初始化为0，但如果我们想要表示上界，则需要适当地初始化这些效用。我们定义一个函数，通过插值来估计给定信念的效用。我们可以使用贪婪前瞻提取策略。Freudenthal三角剖分结构在构造时接收维度和粒度。

FreudenthalTriangulations.jl

该包提供了函数 belief_vertices，给定特定的三角剖分时返回 B。它还提供了 belief_simplex，返回一个信念的包围点集和权重。

```
struct TriangulatedIteration
    m     # granularity
    k_max # maximum number of iterations
end

function solve(M::TriangulatedIteration, 𝒫)
    π = TriangulatedPolicy(𝒫, M.m)
    U(b) = utility(π, b)
    for k in 1:M.k_max
        U′ = [greedy(𝒫, U, b).u for b in π.B]
        for (b, u′) in zip(π.B, U′)
            π.V[b] = u′
        end
    end
    return π
end
```

算法21.16. 使用粒度m的三角化策略进行k_max次迭代的近似值迭代。在每次迭代中，我们使用三角化效用进行贪婪一步前瞻，更新与B中的信念相关的效用。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

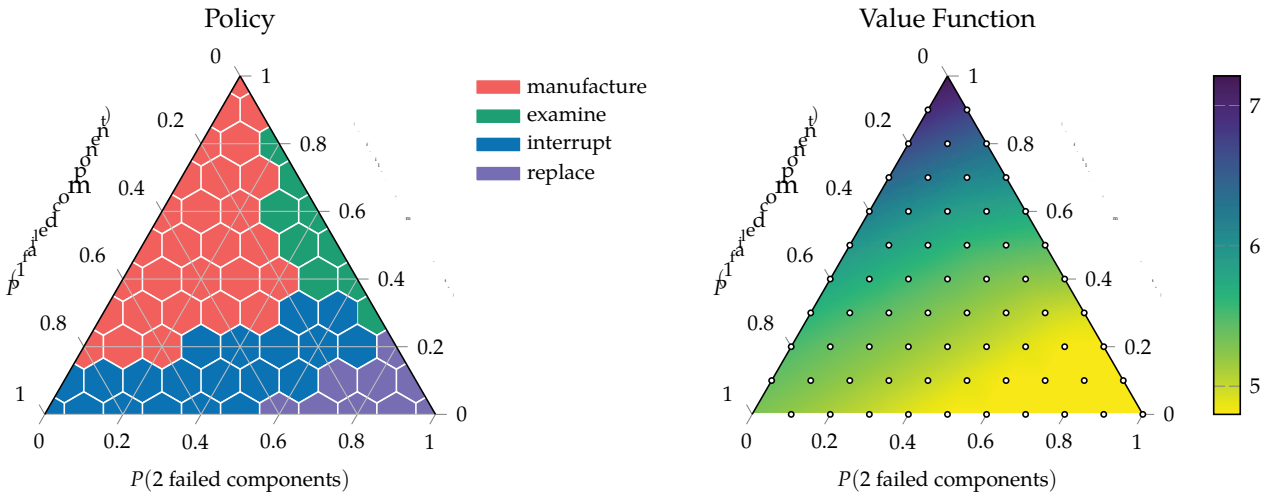


图21.10. 维护问题的策略与价值函数，在11次迭代后，粒度参数为 $m = 10$。价值函数图以白点展示离散信念点。该策略近似于附录F.8中给出的确切策略。

21.10 *Summary*

- QMDP算法假设第一步之后具有完全可观测性，从而得到真实价值函数的上界{v*}。
- 快速知情界通过考虑观测模型，提供了比QMDP更紧的值函数上界。
- 基于点的值迭代通过在有限信念集上的alpha向量提供了值函数的下界。
- 基于随机点的值迭代在信念集中的随机选择点上执行更新，直到集合中所有点的值都被改进。
- 锯齿上界允许使用高效的点集表示对快速信息界进行迭代改进。
- 在基于点的值迭代中，精心选择要使用的信念点可以提高所产生策略的质量。
- 锯齿启发式搜索试图分别收紧由锯齿对和阿尔法向量表示的值函数的上下界。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

- 近似求解POMDP的一种方法是将信念空间离散化，然后应用动态规划来提取价值函数的上界和策略。

21.11 Exercises

练习21.1. 假设我们处于直线六边形世界问题的一个变体中（附录F.1），该问题由四个单元组成，对应于状态 $s_{1:4}$。有两种动作：向左移动（ℓ）和向右移动（r）。这些动作的效果是确定性的。在 s_1 中向左移动或在 s_4 中向右移动会获得100的奖励并结束游戏。折扣因子为0.9时，使用QMDP计算alpha向量。然后，利用这些alpha向量，在给定信念 b = [0.3, 0.1, 0.5, 0.1] 的情况下，计算近似最优动作。

Solution: 我们将与向左移动相关联的alpha向量表示为α_ℓ，将与向右移动相关联的alpha向量表示为α_r。我们将alpha向量初始化为零：

$$\boldsymbol{\alpha}_\ell^{(1)} = [R(s_1,\ell), R(s_2,\ell), R(s_3,\ell), R(s_4,\ell)] = [0,0,0,0]$$
$$\boldsymbol{\alpha}_r^{(1)} = [R(s_1,r), R(s_2,r), R(s_3,r), R(s_4,r)] = [0,0,0,0]$$

在第一次迭代中，由于alpha向量中的所有条目均为零，只有奖励项对QMDP更新（方程(21.1)）有贡献：

$$\boldsymbol{\alpha}_\ell^{(2)} = [100,0,0,0]$$
$$\boldsymbol{\alpha}_r^{(2)} = [0,0,0,100]$$

在下一次迭代中，我们应用更新，这导致左侧alpha向量的s_2和右侧alpha向量的s_3得到新值。左侧alpha向量的更新如下（右侧alpha向量的更新是对称的）：

$$\begin{aligned}
\alpha_\ell^{(3)}(s_1) &= 100 \quad \text{(terminal state)} \\
\alpha_\ell^{(3)}(s_2) &= 0 + 0.9 \times \max(\alpha_\ell^{(2)}(s_1), \alpha_r^{(2)}(s_1)) = 90 \\
\alpha_\ell^{(3)}(s_3) &= 0 + 0.9 \times \max(\alpha_\ell^{(2)}(s_2), \alpha_r^{(2)}(s_2)) = 0 \\
\alpha_\ell^{(3)}(s_4) &= 0 + 0.9 \times \max(\alpha_\ell^{(2)}(s_3), \alpha_r^{(2)}(s_3)) = 0
\end{aligned}$$

这导致了以下结果：

$$\boldsymbol{\alpha}_\ell^{(3)} = [100,90,0,0]$$
$$\boldsymbol{\alpha}_r^{(3)} = [0,0,90,100]$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

在第三次迭代中，左侧alpha向量的更新为

$$\begin{aligned}
\alpha_\ell^{(4)}(s_1) &= 100 \quad \text{(terminal state)} \\
\alpha_\ell^{(4)}(s_2) &= 0 + 0.9 \times \max(\alpha_\ell^{(3)}(s_1), \alpha_r^{(3)}(s_1)) = 90 \\
\alpha_\ell^{(4)}(s_3) &= 0 + 0.9 \times \max(\alpha_\ell^{(3)}(s_2), \alpha_r^{(3)}(s_2)) = 81 \\
\alpha_\ell^{(4)}(s_4) &= 0 + 0.9 \times \max(\alpha_\ell^{(3)}(s_3), \alpha_r^{(3)}(s_3)) = 81
\end{aligned}$$

那么，我们的alpha向量是，

$$\begin{aligned}
\boldsymbol{\alpha}_\ell^{(4)} &= [100, 90, 81, 81] \\
\boldsymbol{\alpha}_r^{(4)} &= [81, 81, 90, 100]
\end{aligned}$$

此时，我们的alpha向量估计已收敛。现在，我们通过最大化与信念相关的效用来确定最优行动：

$$\begin{aligned}
\boldsymbol{\alpha}_\ell^\top \mathbf{b} &= 100 \times 0.3 + 90 \times 0.1 + 81 \times 0.5 + 81 \times 0.1 = 87.6 \\
\boldsymbol{\alpha}_r^\top \mathbf{b} &= 81 \times 0.3 + 81 \times 0.1 + 90 \times 0.5 + 100 \times 0.1 = 87.4
\end{aligned}$$

因此，我们发现在这种信念状态下，向左移动是最佳行动，尽管位于网格世界右半部分的概率较高。这是由于我们赋予状态 s_1 的相对较高可能性，在该状态下，通过向左移动，我们将获得丰厚且即时的奖励。

练习21.2。回顾练习21.1中的简化六边形世界问题。使用盲下界计算每个动作的alpha向量。然后，利用这些alpha向量，计算在信念b = [0.3, 0.1, 0.5, 0.1]下的价值。

Solution: 盲下界，如方程 (21.6) 所示，类似于 QMDP 更新，但缺少最大化步骤。我们将 alpha 向量的分量初始化为零，并按如下方式运行至收敛：

$$\begin{aligned}
\boldsymbol{\alpha}_\ell^{(2)} &= [100, 0, 0, 0] \\
\boldsymbol{\alpha}_r^{(2)} &= [0, 0, 0, 100]
\end{aligned}$$

$$\begin{aligned}
\boldsymbol{\alpha}_\ell^{(3)} &= [100, 90, 0, 0] \\
\boldsymbol{\alpha}_r^{(3)} &= [0, 0, 90, 100]
\end{aligned}$$

$$\begin{aligned}
\boldsymbol{\alpha}_\ell^{(4)} &= [100, 90, 81, 0] \\
\boldsymbol{\alpha}_r^{(4)} &= [0, 81, 90, 100]
\end{aligned}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

$$\boldsymbol{\alpha}_\ell^{(5)} = [100, 90, 81, 72.9]$$
$$\boldsymbol{\alpha}_r^{(5)} = [72.9, 81, 90, 100]$$

此时，我们的alpha向量估计已经收敛。接下来，我们通过在所有动作中最大化与我们的信念相关的效用来确定价值：

$$\boldsymbol{\alpha}_\ell^\top \mathbf{b} = 100 \times 0.3 + 90 \times 0.1 + 81 \times 0.5 + 72.9 \times 0.1 = 86.79$$
$$\boldsymbol{\alpha}_r^\top \mathbf{b} = 72.9 \times 0.3 + 81 \times 0.1 + 90 \times 0.5 + 100 \times 0.1 = 84.97$$

因此，b 的下界是 86.79。

练习21.3。在基于点的值迭代中，假设$|\Gamma| > |S|$，单个信念点的备份复杂度是多少?

Solution: 在进行备份的过程中，我们为每个动作 a 和观测 o 计算一个 $\boldsymbol{\alpha}_{a,o}$。计算方程 (21.8) 中的 $\boldsymbol{\alpha}_{a,o}$ 需要在 Γ 中找到使 $\boldsymbol{\alpha}^\top$Update(b, a, o) 最大化的 alpha 向量 $\boldsymbol{\alpha}$。如方程 (19.7) 所示，信念更新是 $O(|\mathcal{S}|^2)$，因为它遍历所有初始状态和后继状态。因此，对于特定的 a 和 o，计算 $\boldsymbol{\alpha}_{a,o}$ 需要 $O(|\Gamma||S| + |S|^2) = O(|\Gamma||S|)$ 次操作，总共需要 $O(|\Gamma||S||A||O|)$ 次操作。然后，我们使用这些 $\boldsymbol{\alpha}_{a,o}$ 的值，为每个动作 a 计算方程 (21.9) 中的 $\boldsymbol{\alpha}_a$，总共需要 $O(|S|^2|A||O|)$。一旦我们有了 $\boldsymbol{\alpha}_a$ 的值，找到使 $\boldsymbol{\alpha}_a^\top$ b 最大化的 alpha 向量 $\boldsymbol{\alpha}_a$ 需要 $O(|S||A|)$ 次操作。综合起来，在信念 b 处进行一次备份需要 $O(|\Gamma||S||A||O|)$ 次操作。

练习21.4。考虑由以下给出的信念-效用对集合

$$V = \{([1, 0], 0), ([0, 1], -10), ([0.8, 0.2], -4), ([0.4, 0.6], -6)\}$$

使用权重 $w_i = 0.5$ 对所有 i，使用锯齿上界确定信念 b = [0.5, 0.5] 的效用。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: 我们用信念-效用对进行插值。对于每一个非基信念，我们首先找到最远的基信念，即$\mathbf{e}_i$。从$\mathbf{b}_3$开始，我们进行如下计算：

$$
\begin{aligned}
i_3 &= \arg\max_j \left\|\mathbf{b} - \mathbf{e}_j\right\|_1 - \left\|\mathbf{b}_3 - \mathbf{e}_j\right\|_1 \\
\|\mathbf{b} - \mathbf{e}_1\|_1 - \|\mathbf{b}_3 - \mathbf{e}_1\|_1 &= \left\|\begin{bmatrix}0.5\\0.5\end{bmatrix} - \begin{bmatrix}1\\0\end{bmatrix}\right\|_1 - \left\|\begin{bmatrix}0.8\\0.2\end{bmatrix} - \begin{bmatrix}1\\0\end{bmatrix}\right\|_1 \\
&= \left\|\begin{bmatrix}-0.5\\0.5\end{bmatrix}\right\|_1 - \left\|\begin{bmatrix}-0.2\\0.2\end{bmatrix}\right\|_1 \\
&= 0.6 \\
\|\mathbf{b} - \mathbf{e}_2\|_1 - \|\mathbf{b}_3 - \mathbf{e}_2\|_1 &= \left\|\begin{bmatrix}0.5\\0.5\end{bmatrix} - \begin{bmatrix}0\\1\end{bmatrix}\right\|_1 - \left\|\begin{bmatrix}0.8\\0.2\end{bmatrix} - \begin{bmatrix}0\\1\end{bmatrix}\right\|_1 \\
&= \left\|\begin{bmatrix}0.5\\-0.5\end{bmatrix}\right\|_1 - \left\|\begin{bmatrix}0.8\\-0.8\end{bmatrix}\right\|_1 \\
&= -0.6 \\
i_3 &= 1
\end{aligned}
$$

因此，$\mathbf{e}_1$ 是距离 $\mathbf{b}_3$ 最远的基信念。

对于 $\mathbf{b}_4$，我们计算如下：

$$
\begin{aligned}
i_4 &= \arg\max_j \left\|\mathbf{b} - \mathbf{e}_j\right\|_1 - \left\|\mathbf{b}_4 - \mathbf{e}_j\right\|_1 \\
\|\mathbf{b} - \mathbf{e}_1\|_1 - \|\mathbf{b}_3 - \mathbf{e}_1\|_1 &= \left\|\begin{bmatrix}0.5\\0.5\end{bmatrix} - \begin{bmatrix}1\\0\end{bmatrix}\right\|_1 - \left\|\begin{bmatrix}0.4\\0.6\end{bmatrix} - \begin{bmatrix}1\\0\end{bmatrix}\right\|_1 \\
&= \left\|\begin{bmatrix}-0.5\\0.5\end{bmatrix}\right\|_1 - \left\|\begin{bmatrix}-0.6\\0.6\end{bmatrix}\right\|_1 \\
&= -0.2 \\
\|\mathbf{b} - \mathbf{e}_2\|_1 - \|\mathbf{b}_3 - \mathbf{e}_2\|_1 &= \left\|\begin{bmatrix}0.5\\0.5\end{bmatrix} - \begin{bmatrix}0\\1\end{bmatrix}\right\|_1 - \left\|\begin{bmatrix}0.4\\0.6\end{bmatrix} - \begin{bmatrix}0\\1\end{bmatrix}\right\|_1 \\
&= \left\|\begin{bmatrix}0.5\\-0.5\end{bmatrix}\right\|_1 - \left\|\begin{bmatrix}0.4\\-0.4\end{bmatrix}\right\|_1 \\
&= 0.2 \\
i_4 &= 2
\end{aligned}
$$

因此，$\mathbf{e}_2$是离$\mathbf{b}_4$最远的基信念。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们可以使用我们的权重，以及相应的角点和效用对 (e_2, b_3) 和 (e_1, b_4) 来计算 $U(b)$：

$$U_3(\mathbf{b}) = 0.5 \times -4 + 0.5 \times (-10) = -7$$
$$U_4(\mathbf{b}) = 0.5 \times -6 + 0.5 \times 0 = -3$$

最后，我们通过取 $U_3(b)$ 和 $U_4(b)$ 的最小值来计算 $U(b)$。因此，$U(b) = -7$。

练习 21.5。假设我们有一个有效的下界，表示为 alpha 向量集合 Γ。在信念状态 b 处进行备份是否可能产生一个 alpha 向量 $\boldsymbol{\alpha}'$，使得 $\boldsymbol{\alpha}'^{\top}b$ 低于由 Γ 表示的效用函数？换句话说，在信念 b 处进行备份是否可能产生一个 alpha 向量，其对 b 的效用赋值低于由 Γ 表示的价值函数？

Solution: 这是可能的。假设我们只有一个动作，观测完美，没有折扣，状态空间为$\{s^0, s^1\}$。对于所有i，奖励为$R(s^i) = i$，并且状态确定性地转移到s^0。我们从一个有效的下界开始，$\Gamma = \{[-1, +1]\}$，如图21.11中的红色所示。我们为进行备份的信念选择b = [0.5, 0.5]。使用方程（21.9），我们得到

$$\alpha(s^0) = R(s^0) + U^{\Gamma}(s^0) = 0 + (-1) = -1$$
$$\alpha(s^1) = R(s^1) + U^{\Gamma}(s^0) = 1 + (-1) = 0$$

因此，我们备份后得到的 alpha 向量是 $[-1, 0]$，如图 21.11 中蓝色所示。在 b 处使用该 alpha 向量的效用为 -0.5。然而，$U^{\Gamma}(b) = 0$，表明对一个信念进行备份可能会得到一个在该信念处表示较低效用的 alpha 向量。这一事实促使我们在随机化基于点的价值迭代（算法 21.8）中使用 if 语句。该 if 语句将使用来自备份的 alpha 向量或 Γ 中在信念 b 处占支配地位的 alpha 向量，以给出最大效用估计者为准。

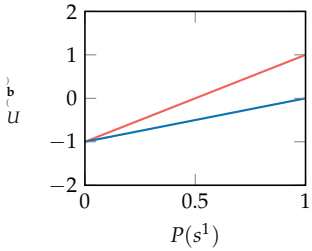


图21.11. 一个示例，说明在信念处的备份如何导致一个alpha向量，该向量单独作用时会降低该信念处的值，相较于原始值函数。我们进行更新的信念b对应于$P(s^1) = 0.5$。原始值函数由Γ表示，以红色显示。在b处进行备份得到的alpha向量以蓝色显示。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

22 Online Belief State Planning

在线方法通过从当前信念状态进行规划来确定最优策略。从当前状态可达的信念空间通常比完整信念空间小。正如在完全可观测背景下所介绍的，许多在线方法使用基于树的搜索变体，达到某个时间范围。[1] 各种

可以使用策略来尝试避免随树深度呈指数增长的计算量。尽管在线方法在执行期间每个决策步骤所需的计算量比离线方法更多，但在线方法有时更容易应用于高维问题。

1 一项调查由 S. Ross、J. Pineau、S. Paquet 和 B. Chaib-draa 提供。“Online Planning Algorithms for POMDPs”，*Journal of Artificial Intelligence Research*，第 32 卷，第 663–704 页，2008 年。

22.1 Lookahead with Rollouts

算法9.1将带有rollout的前瞻作为完全可观测问题中的在线方法引入。该算法可直接用于部分可观测问题。它使用一个函数来随机采样下一状态，这在部分可观测性的语境中对应于信念状态。该函数已在算法21.11中引入。由于我们可以使用生成模型而非显式模型来处理转移、奖励和观测，因此能够适应具有高维状态和观测空间的问题。

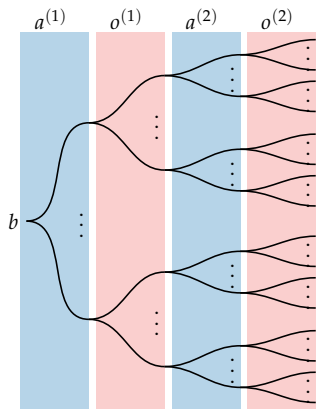


图22.1. 前向搜索在动作-观察-信念图中任意有限深度内进行搜索，以选择能够产生最高预期奖励的动作。此图展示了深度为2的搜索过程。

22.2 Forward Search

我们可以将算法9.2中的前向搜索策略直接应用于部分可观察问题，无需修改。MDP与POMDP之间的差异由一个步骤的前瞻所体现，它根据动作和观察进行分支，如图22.1所示。从信念b采取动作a的价值可以定义为

递归到深度 d:

$$Q_d(b,a) = \begin{cases} R(b,a) + \gamma \sum_o P(o \mid b,a) U_{d-1}(\text{Update}(b,a,o)) & \text{if } d > 0 \\ U(b) & \text{otherwise} \end{cases} \tag{22.1}$$

在$U_d(b)$ =为$\max_a Q_d(b、a)$时。当d =为0时，我们已达到最大深度，并使用近似价值函数$U(b)$返回效用，该函数可能来自前一章讨论的某一种方法、启发式选择，或从一个或多个rollout中估计得到。当d >为0时，我们继续向更深处搜索，递归到下一层。例22.1展示了如何将QMDP与前向搜索结合用于机器替换问题。例22.2演示了在哭泣婴儿问题上的前向搜索。

示例22.1. 将前向搜索应用于机器替换问题（附录F.8）。

考虑将前向搜索应用于机器更换问题。我们可以首先通过QMDP（算法21.2）获得一个近似价值函数。然后，我们可以构建一个`ForwardSearch`对象，该对象最初定义在算法9.2中。该函数内对`lookahead`的调用将使用算法20.5中为POMDP定义的那个。以下代码将前向搜索应用于问题$\mathcal{P}$，从信念状态[0.5, 0.2, 0.3]到深度5，使用我们从叶节点处QMDP获得的效用估计：

```
k_max = 10 # maximum number of iterations of QMDP
πQMDP = solve(QMDP(k_max), 𝒫)
d = 5 # depth
U(b) = utility(πQMDP, b)
π = ForwardSearch(𝒫, d, U)
π([0.5,0.2,0.3])
```

与方程（22.1）中递归相关的计算随深度呈指数增长，$O(|\mathcal{A}|^d|\mathcal{O}|^d)$。因此，前向搜索通常局限于相对较浅的深度。为了深入探索，我们可以限制动作或观测分支。例如，若具备一定的领域知识，我们可以在根节点或树的下层限制动作集。对于观测分支，我们可以将考虑范围限制在一小组可能的观测上——甚至仅考虑最可能的观测，[2]。通过采用

2 R. Platt Jr.、R. Tedrake、L. P. Kaelbling 和 T. Lozano-Pérez，“假设最大似然观测的信念空间规划”，载于 *Robotics: Science and Systems*，2010年。

第9.9.3节中描述的开环或事后优化方法，其状态从当前信念中采样。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

例22.2。前向搜索应用于哭泣婴儿问题（附录F.7）。

考虑带有哭泣婴儿问题的前向搜索，其近似值函数由 alpha 向量 $[-3.7, -15]$ 和 $[-2, -21]$ 给出。从初始信念 $b = [0.5, 0.5]$ 出发，运行深度为 2 的前向搜索如下所示：

$$\begin{aligned}
Q_2(b, a_{\text{feed}}) &= R(b, a_{\text{feed}}) + \gamma(P(\text{crying} \mid b, \text{feed})U_1([1.0, 0.0]) \\
&\qquad + P(\text{quiet} \mid b, \text{feed})U_1([1.0, 0.0])) \\
&= -10 + 0.9(0.1 \times -3.2157 + 0.9 \times -3.2157) \\
&= -12.894 \\
Q_2(b, a_{\text{ignore}}) &= R(b, a_{\text{ignore}}) + \gamma(P(\text{crying} \mid b, \text{ignore})U_1([0.093, 0.907]) \\
&\qquad + P(\text{quiet} \mid b, \text{ignore})U_1([0.786, 0.214])) \\
&= -5 + 0.9(0.485 \times -15.872 + 0.515 \times -7.779) \\
&= -15.534 \\
Q_2(b, a_{\text{sing}}) &= R(b, a_{\text{sing}}) + \gamma(P(\text{crying} \mid b, \text{sing})U_1([0.0, 1.0]) \\
&\qquad + P(\text{quiet} \mid b, \text{sing})U_1([0.891, 0.109])) \\
&= -5.5 + 0.9(0.495 \times -16.8 + 0.505 \times -5.543) \\
&= -15.503
\end{aligned}$$

回想一下，喂养婴儿总是会让婴儿感到饱足（$b = [1, 0]$），而给婴儿唱歌则确保它只有在饥饿时才会哭泣（$b = [0, 1]$）。每个 U_1 值都是通过在方程（22.1）中递归深入一层来评估的，使用 $U_d(b) = \max_a Q_d(b, a)$。在最大深度处，我们使用由 alpha 向量给出的近似价值函数，$Q_0(b, a) = \max\left(b^\top[-3.7, -15], b^\top[-2, -21]\right)$。

策略预测，喂养婴儿将产生最高的预期效用，因此推荐采取该行动。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

22.3 Branch and Bound

*branch and bound*技术最初是在MDP背景下引入的，它同样可以扩展到POMDP中。第9.4节中的相同算法无需修改即可使用（见例22.3），依赖于适当的前瞻实现来更新信念并考虑观测结果。算法的效率仍然取决于用于修剪的上界和下界的质量。

虽然我们可以像在完全观测情况下那样，对上限和下限使用特定领域的启发式方法，但我们也可以采用前一章介绍的针对离散状态空间的方法之一。例如，我们可以使用快速信息边界作为上限，并基于点的值迭代作为下限。只要下界 $\underline{U}$ 和上界 Q 是真实的下界和上界，分支定界算法的结果将与以 $\underline{U}$ 作为近似值函数的前向搜索算法相同。

例22.3. 分支定界法在哭闹婴儿问题中的应用。

在这个例子中，我们对深度为5的哭泣婴儿问题应用分支定界法。上界来自于快速信息边界，下界则来自基于点的价值迭代。我们根据信念[0.4, 0.6]计算动作如下：

```
k_max = 10 # maximum number of iterations for bounds
πFIB = solve(FastInformedBound(k_max), 𝒫)
d = 5 # depth
Uhi(b) = utility(πFIB, b)
Qhi(b,a) = lookahead(𝒫, Uhi, b, a)
B = [[p, 1 - p] for p in 0.0:0.2:1.0]
πPBVI = solve(PointBasedValueIteration(B, k_max), 𝒫)
Ulo(b) = utility(πPBVI, b)
π = BranchAndBound(𝒫, d, Ulo, Qhi)
π([0.4,0.6])
```

22.4 Sparse Sampling

前向搜索对所有可能的观测进行求和，导致在$|\mathcal{O}|$中运行时呈指数级增长。如第9.5节所述，我们可以使用采样来避免穷举求和。我们可以为每个动作生成m个观测，然后

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

计算

$$Q_d(b,a) = \begin{cases} \frac{1}{m}\sum_{i=1}^{m}\left(r_a^{(i)} + \gamma U_{d-1}\left(\text{Update}(b,a,o_a^{(i)})\right)\right) & \text{if } d > 0 \\ U(b) & \text{otherwise} \end{cases} \tag{22.2}$$

其中 $r_a^{(i)}$ 和 $o_a^{(i)}$ 是与来自信念 b 的动作 a 相关的第 i 次采样的观测和奖励，而 $U(b)$ 是最大深度处的值函数估计。我们可以不加修改地使用算法 9.4。由此产生的复杂度为 $O(|\mathcal{A}|^d m^d)$。

22.5 Monte Carlo Tree Search

用于MDP的*Monte Carlo tree search*方法可以扩展到POMDP，尽管我们不能使用完全相同的实现。[3]算法的输入-

算法是一个信念状态 b、深度 d、探索因子 c 和展开策略 π。[4] 该 POMDP 算法（算法 22.1）与 MDP 算法的主要区别在于，计数和值是与 *histories* 关联，而不是与状态关联。历史是过去动作和观察的序列。例如，如果我们有两个动作 a_1 和 a_2 以及两个观察 o_1 和 o_2，那么一个可能的历史可以是序列 $h = a_1 o_2 a_2 o_2 a_1 o_1$。在算法执行过程中，我们为一组历史-动作对更新值估计 $Q(h、a)$ 和计数 $N(h、a)$。[5]

与Q和N相关联的历史可以组织成一棵树，类似于图22.2中的树。根节点表示从初始信念状态b开始的空历史。在算法执行过程中，树结构会扩展。树的层在动作节点和观察节点之间交替。与每个动作节点相关联的是值$Q(h、a)$和$N(h、a)$，其中历史由从根节点到该节点的路径确定。与MDP版本一样，当沿着树向下搜索时，算法采取使下式最大化的动作

$$Q(h,a) + c\sqrt{\frac{\log N(h)}{N(h,a)}} \tag{22.3}$$

其中 $N(h) = \sum_a N(h、a)$ 是历史 h 的总访问次数，而 c 是探索参数。重要的是，c 提升了未探索和探索不足动作的价值，从而代表了探索与利用之间的相对权衡。

3 Silver和Veness提出了一种针对POMDP的蒙特卡洛树搜索算法，称为*Partially Observable Monte Carlo Planning* (*POMCP*)，并展示了其收敛性。D. Silver和J. Veness，“大型POMDP中的蒙特卡洛规划”，载于*Advances in Neural Information Processing Systems* (*NIPS*)，2010年。

4 蒙特卡洛树搜索可以通过在信念上操作的POMDP展开策略，或在状态上操作的MDP展开策略来实现。随机策略通常被使用。

5 这里介绍的基本算法有许多变体，包括一些结合了第9.6节讨论的双渐进拓宽的方面。Z. N. Sunberg和M. J. Kochenderfer，“连续状态、动作和观察空间的POMDP在线算法”，载于*International Conference on Automated Planning and Scheduling* (*ICAPS*)，2018年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct HistoryMonteCarloTreeSearch
    𝒫 # problem
    N # visit counts
    Q # action value estimates
    d # depth
    m # number of simulations
    c # exploration constant
    U # value function estimate
end

function explore(π::HistoryMonteCarloTreeSearch, h)
    𝒜, N, Q, c = π.𝒫.𝒜, π.N, π.Q, π.c
    Nh = sum(get(N, (h,a), 0) for a in 𝒜)
    return argmax(a->Q[(h,a)] + c*bonus(N[(h,a)], Nh), 𝒜)
end

function simulate(π::HistoryMonteCarloTreeSearch, s, h, d)
    if d ≤ 0
        return π.U(s)
    end
    𝒫, N, Q, c = π.𝒫, π.N, π.Q, π.c
    𝒮, 𝒜, TRO, γ = 𝒫.𝒮, 𝒫.𝒜, 𝒫.TRO, 𝒫.γ
    if !haskey(N, (h, first(𝒜)))
        for a in 𝒜
            N[(h,a)] = 0
            Q[(h,a)] = 0.0
        end
        return π.U(s)
    end
    a = explore(π, h)
    s′, r, o = TRO(s,a)
    q = r + γ*simulate(π, s′, vcat(h, (a,o)), d-1)
    N[(h,a)] += 1
    Q[(h,a)] += (q-Q[(h,a)])/N[(h,a)]
    return q
end

function (π::HistoryMonteCarloTreeSearch)(b, h=[])
    for i in 1:π.m
        s = rand(SetCategorical(π.𝒫.𝒮, b))
        simulate(π, s, h, π.d)
    end
    return argmax(a->π.Q[(h,a)], π.𝒫.𝒜)
end
```

算法22.1. 基于信念b的POMDP蒙特卡洛树搜索。初始历史h是可选的。此实现与算法9.5中的类似。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

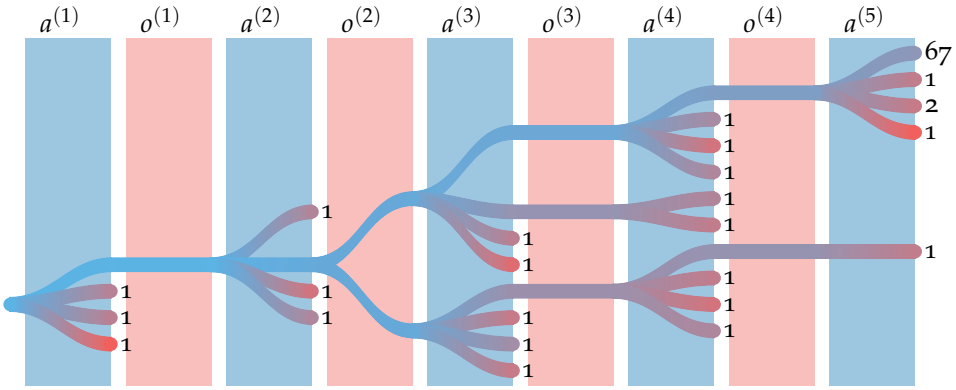


图22.2. 在机器更换问题上运行100个样本的蒙特卡洛树搜索时，包含所有历史记录的搜索树。每个动作节点下方给出了访问次数，颜色表示节点值，高值用蓝色表示，低值用红色表示。未显示访问次数为零的扩展节点。该搜索使用了探索常数$c = 0.5$、最大深度$d = 5$以及均匀随机模拟策略。初始信念是完全正常工作的系统。蒙特卡洛树搜索能够避免某些动作，转而将样本集中在更有希望的路径上。

与MDP版本一样，蒙特卡洛树搜索算法是一种随时算法。算法22.1中的循环可以在任何时候终止，而已找到的最佳解将会返回。经过足够数量的迭代，算法会收敛到最优动作。

先验知识可以通过我们如何初始化N和Q的方式融入蒙特卡洛树搜索中。我们的实现采用了零初始化，但也可以有其他选择，包括使动作值的初始化成为历史的函数。价值估计同样可以通过模拟 rollout 策略来获得。

算法不需要在每次决策时重新初始化。*history tree*以及相关的计数和值估计可以在调用之间保持。与所选动作和实际观察相关联的观察节点在下一个时间步成为根节点。

22.6 Determinized Sparse Tree Search

Determinized sparse tree search 通过使执行动作所得到的观测结果具有确定性，努力减少稀疏采样和蒙特卡洛树搜索中的总体采样量。[6] 它通过构建一个

determinized belief tree 从一种特殊的粒子信念表示出发，形成对真实信念树的稀疏近似。每个粒子指的是m *scenarios*之一，每个粒子的深度为d。一个情景表示粒子在任意给定动作序列$a^{(1)}$、$a^{(2)}$、...、$a^{(d)}$下将遵循的固定历史。每个不同的动作序列

6 Ye、Somani、Hsu 和 Lee 提出了一种针对 POMDP 的确定化稀疏树搜索算法，称为 *De-terminized Sparse Partially Observable Tree* (*DESPOT*)。N. Ye, A. Somani, D. Hsu, and W. S. Lee, "DESPOT: Online POMDP Plan- ning with Regularization," *Journal of Artificial Intelligence Research* , vol. 58, pp. 231–266, 2017。此外，该算法还包括分支定界、启发式搜索和正则化技术。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

在特定场景[7]下产生一个独特的历史记录。这种确定化将搜索树的大小减小到 $O(|\mathcal{A}|^d m)$。历史的一个示例见例22.4。确定化树如图22.3所示。

7 在第9.9.3节中讨论了类似的想法，并且与第11.1节提到的PEGASUS算法相关。

例22.4。在确定性稀疏树搜索的上下文中的历史与场景。

假设我们有两个状态 s_1 和 s_2，两个动作 a_1 和 a_2，以及两个观测 o_1 和 o_2。对于初始状态为 s_2 的粒子，一个可能的深度为 $d = 2$ 的历史是序列 $h = s_2 a_1 o_2 s_1 a_2 o_1$。如果将此历史用作场景，那么每次从初始状态以动作序列 $a^{(1)} = a_1$ 和 $a^{(2)} = a_2$ 遍历信念树时，都会返回此历史。

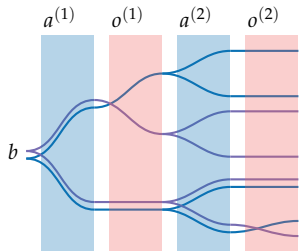


图22.3. 一个确定化的稀疏搜索树，包含两种情景，用蓝色和紫色表示。线条轨迹展示了在不同动作序列下每种情景可能经过的路径。

一棵包含 m 个场景、深度达到 d 的搜索树，可以通过一个紧凑的包含概率质量的 $m \times d$ *determinizing matrix* $\mathbf{\Phi}$ 来完整指定。元素 Φ_{ij} 包含了在深度 j 处遵循第 i 个场景的粒子识别其后继状态和观测所需的信息。具体来说，Φ_{ij} 是一个均匀分布的随机数，可以按照分布 $P(s', o \mid s, a) = T(s' \mid s, a)O(o \mid a, s')$，从状态-动作对 (s, a) 生成后继对 (s', o)。我们可以通过用0到1之间均匀采样的值填充它来生成一个确定性矩阵。

信念以信念粒子向量的形式表示。每个信念粒子 ϕ 包含一个状态 s，以及对应于场景 i 和当前深度 j 的确定性矩阵 $\mathbf{\Phi}$ 中的索引 i 和 j。给定特定动作 a，使用 Φ_{ij} 确定性地转移到后继状态 s' 和观测 o。后继粒子 $\phi' = (s', i, j + 1)$ 接收 s' 作为其状态，并将 j 增加 1。例 22.5 演示了此树遍历过程。粒子信念表示在算法 22.2 中实现，并在算法 22.3 的前向搜索中使用。

22.7 *Gap Heuristic Search*

与第21.8节中介绍的离线启发式搜索类似，*gap heuristic search*利用上界和下界之间的差距来引导我们的搜索朝向那些在其关联价值中存在不确定性的信念，并作为我们何时可以停止探索的指示。信念b处的差距是上界值与下界值之间的差：$U(b) - \underline{U}(b)$。采用差距启发式的搜索算法选择使差距最大化的观测，因为它们更有可能从信念备份中获益。动作通常根据

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

示例 22.5. 确定化稀疏树搜索使用矩阵来使给定粒子的树遍历具有确定性。

假设我们为一个3状态、2观测问题生成一个确定化矩阵$\mathbf{\Phi}$，其中包含四个深度达到3的历史：

$$\mathbf{\Phi} = \begin{bmatrix} 0.393 & 0.056 & 0.369 \\ 0.313 & 0.749 & 0.273 \\ 0.078 & 0.262 & 0.009 \\ 0.969 & 0.598 & 0.095 \end{bmatrix}$$

假设我们在深度为2时，遵循历史3，在状态s_2采取了行动a_3。相应的信念粒子为$\phi = (2, 3, 2)$，而在$\mathbf{\Phi}$中的确定性值为$\Phi_{3,2} = 0.262$。

确定性的后继动作和观测值通过遍历所有后继状态-观测对并累加它们的转移概率来得到。我们从 $p = 0$ 开始并评估 $s' = s_1$、$o = o_1$。假设我们得到 $T(s_1 \mid s_2、a_3)O(o_1 \mid a_3、s_1) = 0.1$。我们将 p 增加到 0.1，这小于 $\Phi_{3,2}$，因此我们继续。

接下来，我们评估$s' = s_2$、$o = o_1$。假设我们得到$T(s_2 \mid s_2、a_3)O(o_1 \mid a_3、s_2) = 0.17$。我们将$p$增加到0.27，这大于$\Phi_{3,2}$。因此，我们确定性地进入$s' = s_2$、$o = o_1$作为我们的后继状态，从而产生一个新的粒子$\phi' = (1, 3, 3)$。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct DeterminizedParticle
    s # state
    i # scenario index
    j # depth index
end

function successor(𝒫, Φ, ϕ, a)
    𝒮, 𝒪, T, O = 𝒫.𝒮, 𝒫.𝒪, 𝒫.T, 𝒫.O
    p = 0.0
    for (s′, o) in product(𝒮, 𝒪)
        p += T(ϕ.s, a, s′) * O(a, s′, o)
        if p ≥ Φ[ϕ.i, ϕ.j]
            return (s′, o)
        end
    end
    return last(𝒮), last(𝒪)
end

function possible_observations(𝒫, Φ, b, a)
    𝒪 = []
    for ϕ in b
        s′, o = successor(𝒫, Φ, ϕ, a)
        push!(𝒪, o)
    end
    return unique(𝒪)
end

function update(b, Φ, 𝒫, a, o)
    b′ = []
    for ϕ in b
        s′, o′ = successor(𝒫, Φ, ϕ, a)
        if o == o′
            push!(b′, DeterminizedParticle(s′, ϕ.i, ϕ.j + 1))
        end
    end
    return b′
end
```

算法 22.2. 用于POMDP $\mathcal{P}$的确定化稀疏树搜索中采用的确定化粒子信念更新。每个信念 `b` 由粒子 `ϕ` 组成，每个粒子编码一个特定场景及沿该场景的深度。其场景的轨迹通过一个包含[0, 1]内随机值的矩阵 `Φ` 进行确定化。每个粒子 `ϕ` 表示特定深度 `j` 下的特定场景 `i`，指向 `Φ` 的第 `i` 行和第 `j` 列。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct DeterminizedSparseTreeSearch
    𝒫 # problem
    d # depth
    Φ # m×d determinizing matrix
    U # value function to use at leaf nodes
end

function determinized_sparse_tree_search(𝒫, b, d, Φ, U)
    𝒮, 𝒜, 𝒪, T, R, O, γ = 𝒫.𝒮, 𝒫.𝒜, 𝒫.𝒪, 𝒫.T, 𝒫.R, 𝒫.O, 𝒫.γ
    if d == 0
        return (a=nothing, u=U(b))
    end
    best = (a=nothing, u=-Inf)
    for a in 𝒜
        u = sum(R(ϕ.s, a) for ϕ in b) / length(b)
        for o in possible_observations(𝒫, Φ, b, a)
            Poba = sum(sum(O(a,s′,o)*T(ϕ.s,a,s′) for s′ in 𝒮)
                       for ϕ in b) / length(b)
            b′ = update(b, Φ, 𝒫, a, o)
            u′ = determinized_sparse_tree_search(𝒫,b′,d-1,Φ,U).u
            u += γ*Poba*u′
        end
        if u > best.u
            best = (a=a, u=u)
        end
    end
    return best
end

function determinized_belief(b, 𝒫, m)
    particles = []
    for i in 1:m
        s = rand(SetCategorical(𝒫.𝒮, b))
        push!(particles, DeterminizedParticle(s, i, 1))
    end
    return particles
end

function (π::DeterminizedSparseTreeSearch)(b)
    particles = determinized_belief(b, π.𝒫, size(π.Φ,1))
    return determinized_sparse_tree_search(π.𝒫,particles,π.d,π.Φ,π.U).a
end
```

算法22.3. 确定化稀疏树搜索的一种实现，是对前向搜索的修改，用于POMDPs。该策略接收一个信念`b`，其形式为概率向量，该向量通过由`determinized_belief`确定的粒子向量近似表示。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

使用近似值函数的前瞻。算法22.4提供了一种实现。[8]

启发式搜索中使用的初始下界值和上界值对算法性能起着重要作用。例22.6使用随机rollout策略作为下界$\underline{U}(b)$。当然，rollout并不能保证产生下界，因为它基于固定深度的一次试验。随着样本数量的增加，它将收敛到真正的下界。该示例使用了公式(21.2)中的最佳动作最佳状态上界。存在许多其他形式的上界和下界，可以提供更快的收敛速度，但代价是运行时间和实现复杂性。例如，使用快速信息上界（算法21.3）作为上界可以改善探索并帮助缩小差距。对于下界，我们可以使用问题特定的rollout策略来更好地指导搜索。

8 对于POMDPs，存在多种不同的启发式搜索算法，旨在缩小差距。例如，参见S. Ross和B. Chaib-draa的“AEMS：一种用于大型POMDPs中近似策略改进的随时在线搜索算法”，2007年发表于*In-*。此实现*ternational Joint Conference on Artificial Intelligence (IJCAI)*与上一种在提到的DESPOT所使用的相似。

22.8 Summary

- 一种简单的在线策略是执行一步前瞻，该策略考虑从当前信念中采取的每个动作，并使用近似值函数来估计其期望值。
- 前向搜索是将前瞻推广到任意步长的一种方法，这可能导致更好的策略，但其计算复杂度随步长呈指数增长。
- 分支定界法是前向搜索的一种更高效版本，它可以通过在价值函数上设置上界和下界来避免搜索某些路径。
- 稀疏采样是一种近似方法，可以减少遍历所有可能观测空间的计算负担。
- 蒙特卡洛树搜索可以通过在历史而非状态上进行操作来适应部分可观测马尔可夫决策过程。
- 确定化稀疏树搜索使用一种特殊形式的粒子信念，确保观测被确定化，从而极大地减少了搜索树。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct GapHeuristicSearch
    𝒫     # problem
    Ulo   # lower bound on value function
    Uhi   # upper bound on value function
    δ     # gap threshold
    k_max # maximum number of simulations
    d_max # maximum depth
end

function heuristic_search(π::GapHeuristicSearch, Ulo, Uhi, b, d)
    𝒫, δ = π.𝒫, π.δ
    𝒮, 𝒜, 𝒪, R, γ = 𝒫.𝒮, 𝒫.𝒜, 𝒫.𝒪, 𝒫.R, 𝒫.γ
    B = Dict((a,o)⇒update(b,𝒫,a,o) for (a,o) in product(𝒜,𝒪))
    B = merge(B, Dict(()⇒copy(b)))
    for (ao, b′) in B
        if !haskey(Uhi, b′)
            Ulo[b′], Uhi[b′] = π.Ulo(b′), π.Uhi(b′)
        end
    end
    if d == 0 || Uhi[b] - Ulo[b] ≤ δ
        return
    end
    a = argmax(a → lookahead(𝒫,b′→Uhi[b′],b,a), 𝒜)
    o = argmax(o → Uhi[B[(a, o)]] - Ulo[B[(a, o)]], 𝒪)
    b′ = update(b,𝒫,a,o)
    heuristic_search(π,Ulo,Uhi,b′,d-1)
    Ulo[b] = maximum(lookahead(𝒫,b′→Ulo[b′],b,a) for a in 𝒜)
    Uhi[b] = maximum(lookahead(𝒫,b′→Uhi[b′],b,a) for a in 𝒜)
end

function (π::GapHeuristicSearch)(b)
    𝒫, k_max, d_max, δ = π.𝒫, π.k_max, π.d_max, π.δ
    Ulo = Dict{Vector{Float64}, Float64}()
    Uhi = Dict{Vector{Float64}, Float64}()
    for i in 1:k_max
        heuristic_search(π, Ulo, Uhi, b, d_max)
        if Uhi[b] - Ulo[b] < δ
            break
        end
    end
    return argmax(a → lookahead(𝒫,b′→Ulo[b′],b,a), 𝒫.𝒜)
end
```

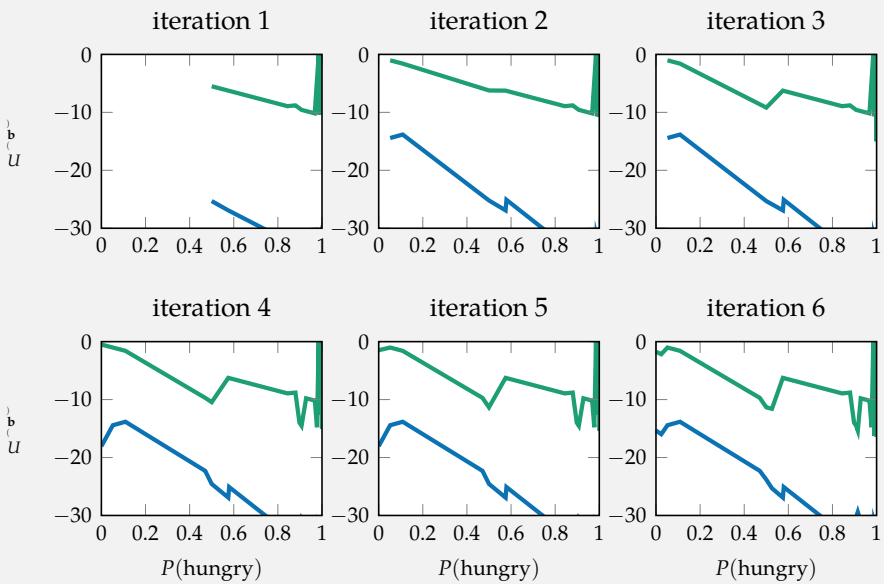
算法22.4。一种启发式搜索的实现，它使用边界、间隙准则以及值函数的初始下界和上界。我们更新字典 `Ulo` 和 `Uhi` 以将值函数的下界和上界保持为特定的信念。在信念 `b` 处，间隙为 `Uhi[b] - Ulo[b]`。当间隙小于阈值 `δ` 或达到最大深度 `d_max` 时，探索停止。搜索被分配最大迭代次数 `k_max`。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

以下代码展示了如何将间隙启发式搜索应用于哭泣婴儿问题。

```
δ = 0.001  # gap threshold
k_max = 5  # maximum number of iterations
d_max = 10 # maximum depth
πrollout(b) = rand(𝒜) # random rollout policy
Ulo(b) = rollout(𝒫, b, πrollout, d_max) # initial lower bound
Rmax = maximum(R(s,a) for (s,a) in product(𝒮,𝒜)) # max reward
Uhi(b) = Rmax / (1.0 - 𝒫.γ) # best action best state upper bound
π = GapHeuristicSearch(𝒫, Ulo, Uhi, δ, k_max, d_max)
π([0.5, 0.5]) # evaluate at initial belief point
```

这里，我们展示了以初始信念$\mathbf{b}$为[0.5, 0.5]进行启发式搜索的六次迭代。在每次迭代中，上界用绿色表示，下界用蓝色表示。



锯齿状边界是由于某些信念没有根据动作和观察选择被重新探索。在底部一行中，我们看到它已经探索了许多信念一次，但边界仍然宽松。启发式搜索旨在减少最大差距。

例22.6。在启发式搜索迭代过程中，针对哭泣婴儿问题使用间隙启发式搜索的上下界。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

- 启发式搜索智能地选择动作-观察对，以探索其维持的价值函数上下界之间差距较大的区域。

22.9 *Exercises*

练习 22.1。假设我们有 $\mathcal{A} = \{a^1、a^2\}$ 以及一个信念 $\mathrm{b} = [0.5, 0.5]$。奖励始终为 1。观测函数由 $P(o^1 \mid a^1) = 0.8$ 和 $P(o^1 \mid a^2) = 0.4$ 给出。我们有一个由 alpha 向量 $\boldsymbol{\alpha} = [-3, 4]$ 表示的近似值函数。取 $\gamma = 0.9$，使用深度为 1 的前向搜索来计算 $U(\mathrm{b})$。在计算中使用以下更新后的信念：

a	o	$\text{Update}(\mathbf{b}, a, o)$
a^1	o^1	$[0.3, 0.7]$
a^2	o^1	$[0.2, 0.8]$
a^1	o^2	$[0.5, 0.5]$
a^2	o^2	$[0.8, 0.2]$

Solution: 我们需要根据公式（22.1）计算深度为1时的动作价值函数：

$$Q_d(\mathbf{b}, a) = R(\mathbf{b}, a) + \gamma \sum_o P(o \mid b, a) U_{d-1}(\text{Update}(\mathbf{b}, a, o))$$

首先，我们计算更新后的信念的效用：

$$\begin{aligned}
U_0(\text{Update}(\mathbf{b}, a^1, o^1)) &= \boldsymbol{\alpha}^\top \mathbf{b}' = 0.3 \times -3 + 0.7 \times 4 = 1.9 \\
U_0(\text{Update}(\mathbf{b}, a^2, o^1)) &= 0.2 \times -3 + 0.8 \times 4 = 2.6 \\
U_0(\text{Update}(\mathbf{b}, a^1, o^2)) &= 0.5 \times -3 + 0.5 \times 4 = 0.5 \\
U_0(\text{Update}(\mathbf{b}, a^2, o^2)) &= 0.8 \times -3 + 0.2 \times 4 = -1.6
\end{aligned}$$

其次，我们计算两种动作的动作价值函数：

$$\begin{aligned}
Q_1(\mathbf{b}, a^1) &= 1 + 0.9((P(o^1 \mid \mathbf{b}, a^1) U_0(\text{Update}(\mathbf{b}, a^1, o^1)) + (P(o^2 \mid \mathbf{b}, a^1) U_0(\text{Update}(\mathbf{b}, a^1, o^2))) \\
&= 1 + 0.9(0.8 \times 1.9 + 0.2 \times 0.5) = 2.458 \\
Q_1(\mathbf{b}, a^2) &= 1 + 0.9((P(o^1 \mid \mathbf{b}, a^2) U_0(\text{Update}(\mathbf{b}, a^2, o^1)) + (P(o^2 \mid \mathbf{b}, a^2) U_0(\text{Update}(\mathbf{b}, a^2, o^2))) \\
&= 1 + 0.9(0.4 \times 2.6 + 0.6 \times -1.6) = 1.072
\end{aligned}$$

最后，我们得到 $U_1(\mathrm{b}) = \max_a Q_1(\mathrm{b},\ a) = 2.458$。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

练习 22.2。使用以下轨迹样本，基于深度为 1 的稀疏采样，计算信念 b 以及动作 a^1 和 a^2 的动作价值函数。使用以下更新后的信念、折扣因子 $\gamma = 0.9$，以及由 alpha 向量 $\boldsymbol{\alpha} = [10, 1]$ 表示的近似价值函数。

a	o	r	Update$(\mathbf{b}, a, o)$
1	1	0	$[0.47, 0.53]$
2	1	1	$[0.22, 0.78]$
1	2	1	$[0.49, 0.51]$
2	1	1	$[0.22, 0.78]$
2	2	1	$[0.32, 0.68]$
1	2	1	$[0.49, 0.51]$

Solution: 我们首先计算更新后信念的效用：

a	o	r	Update$(\mathbf{b}, a, o_a)$	U_0(Update$(\mathbf{b}, a, o)$)
1	1	0	$[0.47, 0.53]$	5.23
2	1	1	$[0.22, 0.78]$	2.98
1	2	1	$[0.49, 0.51]$	5.41
2	1	1	$[0.22, 0.78]$	2.98
2	2	1	$[0.32, 0.68]$	3.88
1	2	1	$[0.49, 0.51]$	5.41

然后，我们可以使用公式计算所有动作的动作价值函数 n (22.2)：

$$Q_1(\mathbf{b}, a^1) = \frac{1}{3}(0 + 1 + 1 + 0.9(5.23 + 5.41 + 5.41)) = 5.48$$
$$Q_1(\mathbf{b}, a^2) = \frac{1}{3}(1 + 1 + 1 + 0.9(2.98 + 2.98 + 3.88)) = 3.95$$

练习22.3。康 sider 例22.5。假设我们有以下转换f 函数：

$$T(s_2 \mid s_1, a_3) = 0.4 \qquad O(o_1 \mid s_2, a_3) = 0.6 \qquad O(o_1 \mid s_3, a_3) = 0.4$$
$$T(s_3 \mid s_1, a_3) = 0.45 \qquad O(o_2 \mid s_2, a_3) = 0.5 \qquad O(o_2 \mid s_3, a_3) = 0.1$$

与$\phi = (1, 4, 2)$相关联的粒子，如果采取行动a_3，其路径是什么?

Solution: 从确定化矩阵中，我们的确定化值为 $\Phi_{4,2} = 0.598$，且我们处于状态 s_1。然后，我们计算 p 如下：

$$p \leftarrow T(s_2 \mid s_1, a_3)O(o_1 \mid s_2, a_3) = 0.4 \times 0.6 = 0.24$$
$$p \leftarrow p + T(s_3 \mid s_1, a_3)O(o_1 \mid s_3, a_3) = 0.24 + 0.45 \times 0.4 = 0.42$$
$$p \leftarrow p + T(s_2 \mid s_1, a_3)O(o_2 \mid s_2, a_3) = 0.42 + 0.4 \times 0.5 = 0.67$$

我们停止迭代，因为 $p > 0.598$。因此，从最后一次迭代，我们继续到 (s_2, o_2)。

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 b
ugs@algorithmsbook.com

练习22.4。总结本章所涵盖的减少基于动作分支的技术。

Solution: 分支定界可以通过使用值函数的上界来减少动作分支。它会跳过那些无法改善先前探索动作所得值的动作。间隙启发式搜索和蒙特卡洛树搜索使用动作值的近似来指导探索过程中动作的选择。

练习22.5. 总结本章涵盖的减少对观察分支的技术。

Solution: 稀疏采样通过仅采样少量观测来减少观测分支。观测从$P(o \mid b, a)$中采样，这意味着概率较大的观测更有可能被采样。确定性稀疏树搜索使用类似的方法，但采样只进行一次然后固定。观测分支也可以基于前瞻值$U(b')$来减少。间隙启发式搜索评估间隙，并避免对值函数有高置信度的观测进行分支。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

23 Controller Abstractions

本章介绍了 POMDP 策略的控制器表示，这些表示允许策略维护其自身的内部状态。相比于之前枚举信念点的方法，这些表示可以提高可扩展性。本章介绍了使用策略迭代、非线性规划和梯度上升来构建控制器的算法。

23.1 Controllers

controller 是一种策略表示，它维护自身的内部状态。它被表示为一个由有限节点集 X 组成的图。[1] 活动的 *node*
随着采取行动和进行新的观察，这些变化会发生。拥有有限的节点集使得这些控制器在计算上比必须考虑可达信念空间的信念点方法更为可行。

1 这样的策略表示也称为 *finite state controller*。我们将控制器状态称为“节点”而非“状态”，以减少与环境状态的歧义。

根据当前节点所依赖的 *action distribution* $\psi(a \mid x)$ 来选择动作。在选择动作时，除了转换到一个未观察到的 s' 并接收观测 o 之外，控制状态还根据 *successor distribution* $\eta(x' \mid x、a、o)$ 推进。图 23.1 展示了在遵循控制器策略时如何使用这些分布。算法 23.1 提供了一种实现，例 23.1 展示了一个针对哭泣婴儿问题的控制器。

控制器推广了第20.2节中介绍的条件计划。条件计划将策略表示为树，每个节点确定性地分配一个动作，每条边指定一个唯一的后继节点。控制器将策略表示为有向图，动作可能具有到多个后继节点的随机转移。例23.2比较了这两种表示。

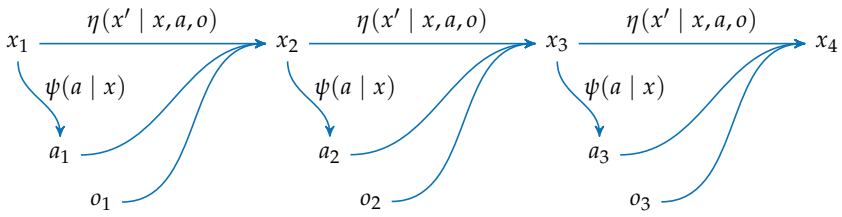
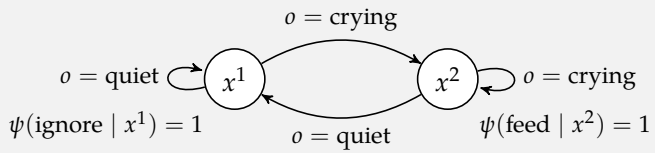


图23.1。在控制器表示中，动作是从动作选择分布中采样得到的。这个动作及其产生的后续观察结果，与前一节点x一起用于生成后继节点x'。

我们可以为哭闹婴儿问题（附录 F.7）构造一个简单的控制器。这里将该示例展示为一个包含两个节点 x^1 和 x^2 的图。当处于 x^1 时，控制器总是忽略婴儿。当处于 x^2 时，控制器总是喂养婴儿。如果婴儿哭闹，我们总是转移到 x^2；如果婴儿安静，我们总是转移到 x^1。



示例23.1. 针对哭泣婴儿问题的双节点控制器。这种紧凑的表示形式捕捉到了解决哭泣婴儿问题的一个直接方法（即，对最近的观察立即作出反应）。

```
mutable struct ControllerPolicy
    𝒫 # problem
    X # set of controller nodes
    ψ # action selection distribution
    η # successor selection distribution
end

function (π::ControllerPolicy)(x)
    𝒜, ψ = π.𝒫.𝒜, π.ψ
    dist = [ψ[x, a] for a in 𝒜]
    return rand(SetCategorical(𝒜, dist))
end

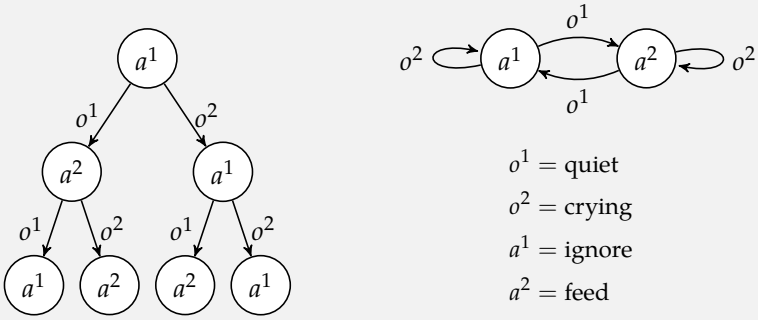
function update(π::ControllerPolicy, x, a, o)
    X, η = π.X, π.η
    dist = [η[x, a, o, x′] for x′ in X]
    return rand(SetCategorical(X, dist))
end
```

算法23.1. POMDP $\mathcal{P}$ 的有限状态控制器策略表示。`X` 中的节点是可到达信念的抽象表示。动作和控制器后继节点以随机方式选择。给定节点 `x`，动作按照分布 `ψ` 选择。函数 `π(x)` 实现此机制以随机选择动作。在节点 `x` 中执行动作 `a` 并观察到观测 `o` 后，后继节点按照分布 `η` 选择。函数 `update` 实现此机制以随机选择后继节点。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

示例 23.2. 一个简单条件计划与一个简单确定性控制器的比较。

考虑一个三步条件计划（左）与例23.1中更一般的两节点有限状态控制器（右）进行比较。在这种情况下，动作和后继是确定性地选择的。确定性的动作标记在节点的中心，而出边表示确定性的后继节点。此问题有两个动作（a^1和a^2）和两个观测（o^1和o^2）。



条件计划首先执行动作 {v*}，如果观察到 o^1，则切换先前选择的动作；如果观察到 o^2，则保留先前选择的动作。控制器执行相同的逻辑，但控制器节点少五个。此外，控制器仅用两个节点（相比之下需要七个）就完美地表示了所描述的无限时域策略。条件计划无法捕捉这种无限时域策略，因为它需要一棵无限深度的树。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

控制器相较于条件规划有几个优势。首先，控制器能够提供更为紧凑的表示形式。条件规划中的节点数量随深度呈指数增长，而有限状态控制器则未必如此。前几章中的近似方法可能也不那么高效，因为它们必须维护一个庞大的信念集及相应的alpha向量。控制器则能更加紧凑，即使面对无限多的可达信念，也仅需少量有限节点即可表示。控制器的另一个优势在于，它不需要维护一个信念。每个控制器节点对应信念空间的一个子集，这些子集不一定是互斥的。控制器在这些共同覆盖可达信念空间的子集之间进行转换。控制器自身根据每次观察选择新的节点，而不依赖于信念更新，后者在某些领域可能代价高昂。

遵循控制器策略的效用可通过构建状态空间为 $X \times \mathcal{S}$ 的乘积 MDP 来计算。处于状态 s 且节点 x 激活时的价值为

$$U(x,s) = \sum_a \psi(a \mid x) \left(R(s,a) + \gamma \sum_{s'} T(s' \mid s,a) \sum_o O(o \mid a,s') \sum_{x'} \eta(x' \mid x,a,o) U(x',s') \right) \tag{23.1}$$

策略评估涉及求解方程（23.1）中给出的线性方程组。或者，我们可以应用如算法23.2所示的迭代策略评估。

如果信念已知，则当前值为

$$U(x,b) = \sum_s b(s) U(x,s) \tag{23.2}$$

我们可以将 $U(x、s)$ 视为定义了一组 alpha 向量，对于 X 中的每个节点 x 各有一个。每个 alpha 向量 $\boldsymbol{\alpha}_x$ 由 $\alpha_x(s) = U(x、s)$ 定义。给定 alpha 向量的当前值为 $U(x、b) = \mathrm{b}^\top \boldsymbol{\alpha}_x$。

给定一个控制器和初始信念，我们可以通过最大化以下内容来选择初始节点：

$$x^* = \arg\max_x U(x,b) = \arg\max_x \mathbf{b}^\top \boldsymbol{\alpha}_x \tag{23.3}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
function utility(π::ControllerPolicy, U, x, s)
    𝒮, 𝒜, 𝒪 = π.𝒫.𝒮, π.𝒫.𝒜, π.𝒫.𝒪
    T, O, R, γ = π.𝒫.T, π.𝒫.O, π.𝒫.R, π.𝒫.γ
    X, ψ, η = π.X, π.ψ, π.η
    U′(a,s′,o) = sum(η[x,a,o,x′]*U[x′,s′] for x′ in X)
    U′(a,s′) = T(s,a,s′)*sum(O(a,s′,o)*U′(a,s′,o) for o in 𝒪)
    U′(a) = R(s,a) + γ*sum(U′(a,s′) for s′ in 𝒮)
    return sum(ψ[x,a]*U′(a) for a in 𝒜)
end

function iterative_policy_evaluation(π::ControllerPolicy, k_max)
    𝒮, X = π.𝒫.𝒮, π.X
    U = Dict((x, s) ⇒ 0.0 for x in X, s in 𝒮)
    for k in 1:k_max
        U = Dict((x, s) ⇒ utility(π, U, x, s) for x in X, s in 𝒮)
    end
    return U
end
```

算法23.2. 一种执行迭代策略评估以计算有限状态控制器`π`经过`k_max`次迭代后的效用的算法。该效用函数根据方程(23.1)，对当前控制器节点`x`和状态`s`进行单步评估。此算法改编自算法7.3，后者将迭代策略评估应用于MDPs。

23.2 Policy Iteration

第20.5节展示了如何在条件计划中增量添加节点，以达到最优有限时域策略（算法20.8）。本节展示如何向控制器增量添加节点，以优化无限时域问题。尽管策略表示不同，但本节[2]中介绍的部分可观察问题的策略迭代版本有一些相似之处

2 这里给出的策略迭代方法由 E. A. Hansen 在 *Conference on Uncertainty in Artificial Intelligence* (*UAI*) 中的“通过策略空间搜索求解 POMDP”一文中给出，1998年。

使用用于完全观测问题的策略迭代算法（第7.4节）。

策略迭代（算法23.3）从任意初始控制器开始，然后在策略评估和策略改进之间交替进行。在策略评估中，我们通过求解方程（23.1）来评估效用值 $U(x、s)$。在策略改进中，我们向控制器中引入新的节点。具体而言，我们为每一个确定性动作赋值 $\psi(a_i \mid x') = 1$ 与确定性后继选择分布 $\eta(x \mid x'、a、o)$ 的组合引入一个新节点 x'。该过程在第 k 次迭代时向节点集合 $X^{(k)}$ 中添加 $|\mathcal{A}||X^{(k)}|^{|\mathcal{O}|}$ 个新的控制器节点。[3] 一个

3 添加所有可能的组合通常是不可行的。一种称为 *bounded policy iteration* 的替代算法仅添加一个节点。P. Poupart 和 C. Boutilier，“有界有限状态控制器”，载于 *Advances in Neural Information Processing Systems* (*NIPS*)，2003 中。算法也可以添加为节点者之间的数量。例如，*Monte Carlo value iteration* 在每次迭代 k 时添加 $O(n|\mathcal{A}||X^{(k)}|)$ 个新节点，其中 n 是一个参数。H. Bai、D. Hsu、W. S. Lee 和 V. A. Ngo，“连续状态POMDP的蒙特卡洛值迭代”，载于 *International Workshop on the Algorithmic Foundations of Robotics* (*WAFR*)，2010年。

改进步骤在示例 23.3 中演示。

策略改进不会降低控制器策略的期望值。$X^{(k)}$中任何节点的值保持不变，因为这些节点及其可达的后继节点均未改变。可以保证，如果$X^{(k)}$不是最优控制器，则策略改进中引入的至少一个新节点

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct ControllerPolicyIteration
    k_max    # number of iterations
    eval_max # number of evaluation iterations
end

function solve(M::ControllerPolicyIteration, 𝒫::POMDP)
    𝒜, 𝒪, k_max, eval_max = 𝒫.𝒜, 𝒫.𝒪, M.k_max, M.eval_max
    X = [1]
    ψ = Dict((x, a) ⇒ 1.0 / length(𝒜) for x in X, a in 𝒜)
    η = Dict((x, a, o, x′) ⇒ 1.0 for x in X, a in 𝒜, o in 𝒪, x′ in X)
    π = ControllerPolicy(𝒫, X, ψ, η)
    for i in 1:k_max
        prevX = copy(π.X)
        U = iterative_policy_evaluation(π, eval_max)
        policy_improvement!(π, U, prevX)
        prune!(π, U, prevX)
    end
    return π
end

function policy_improvement!(π::ControllerPolicy, U, prevX)
    𝒮, 𝒜, 𝒪 = π.𝒫.𝒮, π.𝒫.𝒜, π.𝒫.𝒪
    X, ψ, η = π.X, π.ψ, π.η
    repeatX𝒪 = fill(X, length(𝒪))
    assign𝒜X′ = vec(collect(product(𝒜, repeatX𝒪...)))
    for ax′ in assign𝒜X′
        x, a = maximum(X) + 1, ax′[1]
        push!(X, x)
        successor(o) = ax′[findfirst(isequal(o), 𝒪) + 1]
        U′(o,s′) = U[successor(o), s′]
        for s in 𝒮
            U[x, s] = lookahead(π.𝒫, U′, s, a)
        end
        for a′ in 𝒜
            ψ[x, a′] = a′ == a ? 1.0 : 0.0
            for (o, x′) in product(𝒪, prevX)
                η[x, a′, o, x′] = x′ == successor(o) ? 1.0 : 0.0
            end
        end
    end
    for (x, a, o, x′) in product(X, 𝒜, 𝒪, X)
        if !haskey(η, (x, a, o, x′))
            η[x, a, o, x′] = 0.0
        end
    end
end
```

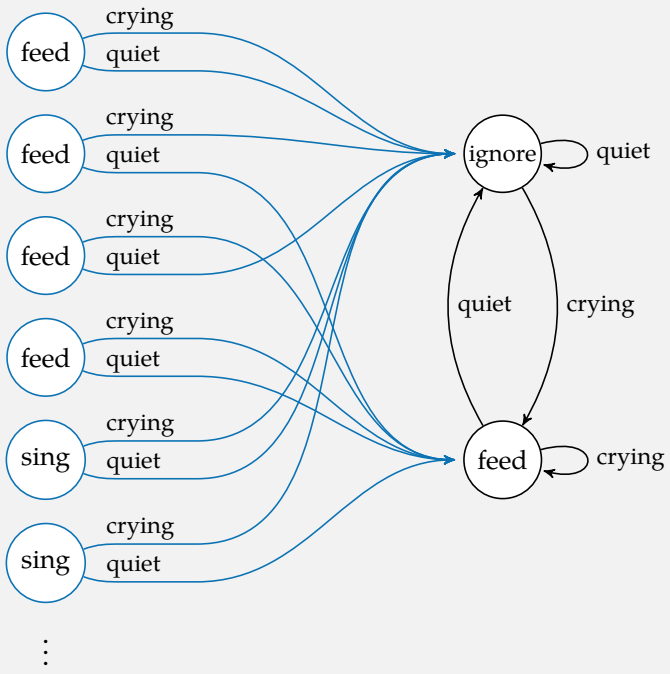
算法23.3. 针对具有固定迭代次数{v*=unchanged}和策略评估迭代次数{v*=unchanged}的POMDP的策略迭代。该算法迭代应用策略评估（算法23.2）和策略改进。剪枝在算法23.4中实现。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们可以对示例23.1中的哭泣婴儿控制器应用策略改进。动作是$\mathcal{A} = \{$喂食、唱歌、忽略$\}$，观测是$\mathcal{O} = \{$哭泣、安静$\}$。策略改进备份步骤产生$|\mathcal{A}||X^{(1)}|^{|\mathcal{O}|} = 3 \times 2^2 = 12$个新节点。新的控制器策略具有节点$\{x^1$、...、$x^{14}\}$，其分布如下：

Node	Action	Successors (for all a below)
x^3	$\psi(\text{feed} \mid x^3) = 1$	$\eta(x^1 \mid x^3, a, \text{crying}) = \eta(x^1 \mid x^3, a, \text{quiet}) = 1$
x^4	$\psi(\text{feed} \mid x^4) = 1$	$\eta(x^1 \mid x^4, a, \text{crying}) = \eta(x^2 \mid x^4, a, \text{quiet}) = 1$
x^5	$\psi(\text{feed} \mid x^5) = 1$	$\eta(x^2 \mid x^5, a, \text{crying}) = \eta(x^1 \mid x^5, a, \text{quiet}) = 1$
x^6	$\psi(\text{feed} \mid x^6) = 1$	$\eta(x^2 \mid x^6, a, \text{crying}) = \eta(x^2 \mid x^6, a, \text{quiet}) = 1$
x^7	$\psi(\text{sing} \mid x^7) = 1$	$\eta(x^1 \mid x^7, a, \text{crying}) = \eta(x^1 \mid x^7, a, \text{quiet}) = 1$
x^8	$\psi(\text{sing} \mid x^8) = 1$	$\eta(x^1 \mid x^8, a, \text{crying}) = \eta(x^2 \mid x^8, a, \text{quiet}) = 1$
⋮	⋮	⋮

我们有以下控制器，其中新节点为蓝色，原来的两个节点为黑色：



示例23.3。在带有控制器策略表示的哭泣婴儿问题上，策略迭代中改进步骤的说明。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

对于某些状态将具有更好的期望值，因此整体控制器必须得到改进。

在策略改进过程中添加的许多节点往往不会改进策略。剪枝在策略评估之后进行，以消除不必要的节点。这样做不会降低控制器的最优价值函数。剪枝方法有助于减少改进步骤带来的节点指数级增长。在某些情况下，剪枝可以促使循环形成，从而得到紧凑的控制器。

我们会剪除任何与现有节点相同的新节点。我们还会剪除任何被其他节点 *dominated* 的新节点。当某节点 x 被另一个节点 x' 支配时，有

$$U(x, s) \leq U(x', s) \quad \text{for all } s \tag{23.4}$$

现有节点也可以被修剪。每当一个新节点支配了一个现有节点时，我们就从控制器中修剪掉这个现有节点。所有指向被删除节点的转移都会被重新路由到支配节点。这一过程与改为修剪新节点，并将被支配节点的动作和后继链接更新为新节点的动作和后继链接是完全相同的。例23.4展示了在哭泣婴儿问题上的评估、扩展和修剪过程。

23.3 *Nonlinear Programming*

策略改进问题可以被构建为一个单一的、大型的*nonlinear pro-gramming*公式化问题（算法23.5），该问题涉及到在所有节点上同时优化ψ和η。[4]这种公式化允许通用求解器

应用。非线性规划方法直接在控制器空间中搜索，以最大化给定初始信念的效用，同时满足贝尔曼期望方程，即方程（23.1）。它并不在策略评估和策略改进步骤之间交替，并且控制器节点数保持不变。

4 C. Amato, D. S. Bernstein, 和 S. Zilberstein, “优化固定大小随机控制器以应对POMDP和分散式POMDP，” *Autonomous Agents and Multi-Agent Systems*, 第21卷, 第3期, 第293–320页, 2010年。

我们使用 x^1 来表示与给定初始信念 b 相对应的初始节点。于是，优化问题为

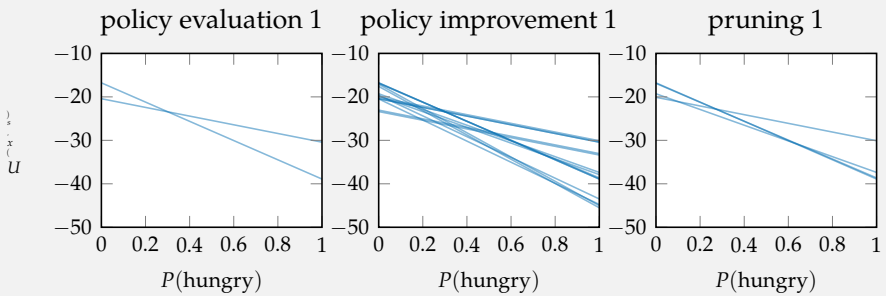
© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
function prune!(π::ControllerPolicy, U, prevX)
    𝒮, 𝒜, 𝒪, X, ψ, η = π.𝒫.𝒮, π.𝒫.𝒜, π.𝒫.𝒪, π.X, π.ψ, π.η
    newX, removeX = setdiff(X, prevX), []
    # prune dominated from previous nodes
    dominated(x,x′) = all(U[x,s] ≤ U[x′,s] for s in 𝒮)
    for (x,x′) in product(prevX, newX)
        if x′ ∉ removeX && dominated(x, x′)
            for s in 𝒮
                U[x,s] = U[x′,s]
            end
            for a in 𝒜
                ψ[x,a] = ψ[x′,a]
                for (o,x′′) in product(𝒪, X)
                    η[x,a,o,x′′] = η[x′,a,o,x′′]
                end
            end
            push!(removeX, x′)
        end
    end
    # prune identical from new nodes
    identical_action(x,x′) = all(ψ[x,a] ≈ ψ[x′,a] for a in 𝒜)
    identical_successor(x,x′) = all(η[x,a,o,x′′] ≈ η[x′,a,o,x′′]
            for a in 𝒜, o in 𝒪, x′′ in X)
    identical(x,x′) = identical_action(x,x′) && identical_successor(x,x′)
    for (x,x′) in product(prevX, newX)
        if x′ ∉ removeX && identical(x,x′)
            push!(removeX, x′)
        end
    end
    # prune dominated from new nodes
    for (x,x′) in product(X, newX)
        if x′ ∉ removeX && dominated(x′,x) && x ≠ x′
            push!(removeX, x′)
        end
    end
    # update controller
    π.X = setdiff(X, removeX)
    π.ψ = Dict(k ⇒ v for (k,v) in ψ if k[1] ∉ removeX)
    π.η = Dict(k ⇒ v for (k,v) in η if k[1] ∉ removeX)
end
```

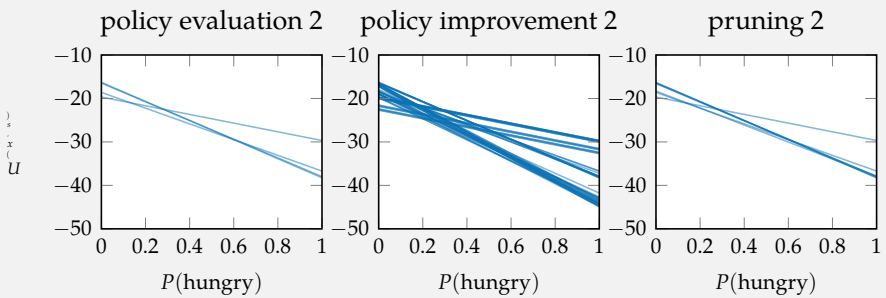
算法23.4。策略迭代的剪枝步骤。它使用策略评估计算得到的效用U以及先前的节点列表prevX，减少当前策略π中的节点数量。其第一步是将任何逐点被支配的先前节点替换为其改进后的节点，并将冗余节点标记为现已支配。第二步标记任何与先前节点相同的新添加节点。第三步标记任何逐点被支配的新节点。最后，所有被标记的节点均被剪除。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

回忆示例 23.3。此处，我们展示了使用相同初始控制器的策略迭代的首次迭代过程。它包括两个主要步骤：策略评估（左）和策略改进（中），以及可选的修剪步骤（右）。



策略迭代的第二次迭代遵循相同的模式：



在第二次迭代之后，效用得到了极大提升，接近最优值。我们发现，修剪步骤不仅移除了前几次迭代中的支配节点和重复节点，还去除了当前迭代新产生的节点。

示例23.4。策略迭代，展示了在哭泣婴儿领域中使用控制器策略表示的评估、改进和修剪步骤。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

$$
\begin{aligned}
\underset{U,\psi,\eta}{\text{maximize}} \quad & \sum_s b(s)U(x^1,s) \\
\text{subject to} \quad & U(x,s) = \sum_a \psi(a \mid x)\left(R(s,a) + \gamma \sum_{s'} T(s' \mid s,a) \sum_o O(o \mid a,s') \sum_{x'} \eta(x' \mid x,a,o)U(x',s')\right) \\
& \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad \text{for all } x,s \\
& \psi(a \mid x) \geq 0 \quad \text{for all } x,a \\
& \sum_a \psi(a \mid x) = 1 \quad \text{for all } x \\
& \eta(x' \mid x,a,o) \geq 0 \quad \text{for all } x,a,o,x' \\
& \sum_{x'} \eta(x' \mid x,a,o) = 1 \quad \text{for all } x,a,o
\end{aligned}
\tag{23.5}
$$

这个问题可以写成一个 *quadratically constrained linear program* (*QCLP*)，可以使用专用求解器有效求解。[5] 示例23.5演示了展示了这种方法。

求解一般的QCLP问题是NP难的，但专用求解器提供了有效的近似方法。

23.4 Gradient Ascent

固定大小的控制器策略可以使用梯度上升（见附录 A.11）进行迭代改进。[6] 尽管梯度计算具有挑战性，这为控制器优化开辟了多种基于梯度的优化技术。算法23.6利用算法23.7实现控制器梯度上升。

6 N. Meuleau, K.-E. Kim, L. P. Kaelbling 和 A. R. Cassandra, “通过搜索有限策略空间求解 POMDP”，载于 *Confer-*，1999 年。*ence on Uncertainty in Artificial Intel-*

考虑第23.3节中非线性问题的显式描述。对于初始信念b和任意初始控制器节点x^1，我们寻求最大化如下：

$$\sum_s b(s)U(x^1,s) \tag{23.6}$$

在效用 $U(x、s)$ 由对所有 x 和 s 的贝尔曼最优方程定义的情况下：

$$U(x,s) = \sum_a \psi(a \mid x)\left(R(s,a) + \gamma \sum_{s'} T(s' \mid s,a) \sum_o O(o \mid a,s') \sum_{x'} \eta(x' \mid x,a,o)U(x',s')\right) \tag{23.7}$$

此外，ψ 和 η 必须是适当的概率分布。为了应用梯度上升时，使用线性代数重写这个问题更为方便。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct NonlinearProgramming
    b # initial belief
    ℓ # number of nodes
end

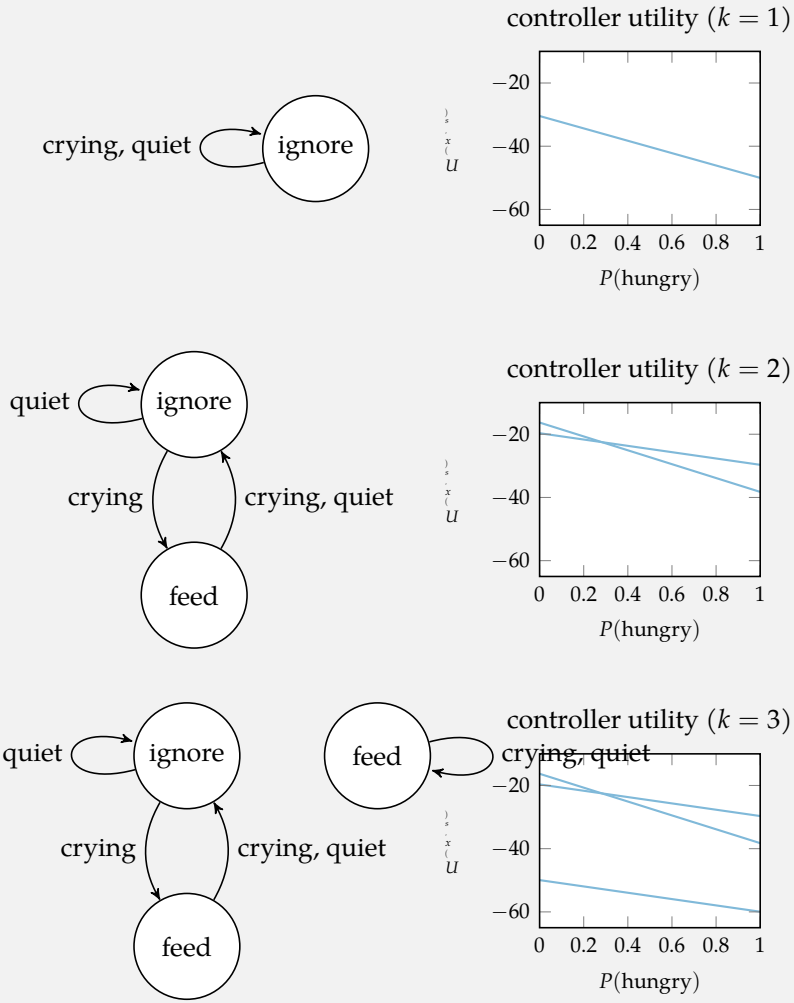
function tensorform(𝒫::POMDP)
    𝒮, 𝒜, 𝒪, R, T, O = 𝒫.𝒮, 𝒫.𝒜, 𝒫.𝒪, 𝒫.R, 𝒫.T, 𝒫.O
    𝒮′ = eachindex(𝒮)
    𝒜′ = eachindex(𝒜)
    𝒪′ = eachindex(𝒪)
    R′ = [R(s,a) for s in 𝒮, a in 𝒜]
    T′ = [T(s,a,s′) for s in 𝒮, a in 𝒜, s′ in 𝒮]
    O′ = [O(a,s′,o) for a in 𝒜, s′ in 𝒮, o in 𝒪]
    return 𝒮′, 𝒜′, 𝒪′, R′, T′, O′
end

function solve(M::NonlinearProgramming, 𝒫::POMDP)
    x1, X = 1, collect(1:M.ℓ)
    𝒫, γ, b = 𝒫, 𝒫.γ, M.b
    𝒮, 𝒜, 𝒪, R, T, O = tensorform(𝒫)
    model = Model(Ipopt.Optimizer)
    @variable(model, U[X,𝒮])
    @variable(model, ψ[X,𝒜] ≥ 0)
    @variable(model, η[X,𝒜,𝒪,X] ≥ 0)
    @objective(model, Max, b⋅U[x1,:])
    @NLconstraint(model, [x=X,s=𝒮],
        U[x,s] == (sum(ψ[x,a]*(R[s,a] + γ*sum(T[s,a,s′]*sum(O[a,s′,o]
        *sum(η[x,a,o,x′]*U[x′,s′] for x′ in X)
        for o in 𝒪) for s′ in 𝒮)) for a in 𝒜)))
    @constraint(model, [x=X], sum(ψ[x,:]) == 1)
    @constraint(model, [x=X,a=𝒜,o=𝒪], sum(η[x,a,o,:]) == 1)
    optimize!(model)
    ψ′, η′ = value.(ψ), value.(η)
    return ControllerPolicy(𝒫, X,
        Dict((x, 𝒫.𝒜[a]) ⇒ ψ′[x, a] for x in X, a in 𝒜),
        Dict((x, 𝒫.𝒜[a], 𝒫.𝒪[o], x′) ⇒ η′[x, a, o, x′]
             for x in X, a in 𝒜, o in 𝒪, x′ in X))
end
```

算法23.5. 一种非线性规划方法，用于计算从初始信念b开始的POMDP $\mathcal{P}$的最优固定大小控制器策略。有限状态控制器的大小由节点数ℓ指定。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

以下是针对哭泣婴儿问题使用非线性规划计算得到的最优固定大小控制器，其中 $b_0 = [0.5, 0.5]$。顶部节点是 x_1。



当 $k = 1$ 时，最优策略是永远简单地忽略。当 $k = 2$ 时，最优策略是忽略直到观察到哭泣，此时最佳行动是喂养婴儿，然后返回忽略。对于无限时域的哭泣婴儿POMDP，该策略接近最优。当 $k = 3$ 时，最优策略基本上与 $k = 2$ 时保持不变。

例23.5. 针对固定大小为k且分别设置为1、2和3的控制器的非线性规划算法。每一行分别展示了策略及其对应的效用（alpha向量），左、右两侧各一组。随机控制器以圆圈表示，中间标明最可能的动作。输出边显示了给定观察时后继节点的选择。节点动作和后继中的随机性通过透明度表示（越不透明表示概率越高，越透明表示概率越低）。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们定义带有控制器的转移函数，其状态空间为$X \times \mathcal{S}$。对于任何由$\theta = (\psi, \eta)$参数化的固定大小控制器策略，转移矩阵$\mathrm{T}_\theta \in \mathbb{R}^{|X\times\mathcal{S}|\times|X\times\mathcal{S}|}$为

$$\mathbf{T}_\theta((x,s),(x',s')) = \sum_a \psi(x,a) T(s,a,s') \sum_o O(a,s',o)\eta(x,a,o,x') \tag{23.8}$$

参数化策略的奖励表示为向量$\mathrm{r}_\theta \in \mathbb{R}^{|X\times\mathcal{S}|}$:

$$\mathbf{r}_\theta((x,s)) = \sum_a \psi(x,a) R(s,a) \tag{23.9}$$

然后，效用 $\mathrm{u}_\theta \in \mathbb{R}^{|X\times\mathcal{S}|}$ 的贝尔曼期望方程为

$$\mathbf{u}_\theta = \mathbf{r}_\theta + \gamma \mathbf{T}_\theta \mathbf{u}_\theta \tag{23.10}$$

我们可以考虑一个初始节点信念向量 $\boldsymbol{\beta} \in \mathbb{R}^{|X\times\mathcal{S}|}$，其中如果 $x = x^1$ 则 $\boldsymbol{\beta}_{xs} = b(s)$，否则 $\boldsymbol{\beta}_{xs} =$ 为 0。效用向量 $\mathrm{u}_\theta \in \mathbb{R}^{|X\times\mathcal{S}|}$ 也定义在节点 X 和状态 $\mathcal{S}$ 上，适用于任何这些固定大小的参数化控制器策略 $\theta = (\psi, \eta)$。我们现在寻求如下最大化:

$$\boldsymbol{\beta}^\top \mathbf{u}_\theta \tag{23.11}$$

我们首先重写方程(23.10):

$$\begin{aligned}
\mathbf{u}_\theta &= \mathbf{r}_\theta + \gamma \mathbf{T}_\theta \mathbf{u}_\theta && (23.12)\\
(\mathbf{I} - \gamma \mathbf{T}_\theta)\mathbf{u}_\theta &= \mathbf{r}_\theta && (23.13)\\
\mathbf{u}_\theta &= (\mathbf{I} - \gamma \mathbf{T}_\theta)^{-1}\mathbf{r}_\theta && (23.14)\\
\mathbf{u}_\theta &= \mathbf{Z}^{-1}\mathbf{r}_\theta && (23.15)
\end{aligned}$$

为方便起见，记 $\mathrm{Z} = \mathrm{I} - \gamma \mathrm{T}_\theta$。为了执行梯度上升，我们需要知道方程 (23.15) 关于策略参数的偏导数:

$$\begin{aligned}
\frac{\partial \mathbf{u}_\theta}{\partial \theta} &= \frac{\partial \mathbf{Z}^{-1}}{\partial \theta}\mathbf{r}_\theta + \mathbf{Z}^{-1}\frac{\partial \mathbf{r}_\theta}{\partial \theta} && (23.16)\\
&= -\mathbf{Z}^{-1}\frac{\partial \mathbf{Z}}{\partial \theta}\mathbf{Z}^{-1}\mathbf{r}_\theta + \mathbf{Z}^{-1}\frac{\partial \mathbf{r}_\theta}{\partial \theta} && (23.17)\\
&= \mathbf{Z}^{-1}\left(\frac{\partial \mathbf{r}_\theta}{\partial \theta} - \frac{\partial \mathbf{Z}}{\partial \theta}\mathbf{Z}^{-1}\mathbf{r}_\theta\right) && (23.18)
\end{aligned}$$

为了方便起见，$\partial\theta$ 同时指代 $\partial\psi(\hat{x}, \hat{a})$ 以及 $\partial\eta(\hat{x}, \hat{a}, \hat{o}, \hat{x}')$。

计算 Z 和 r_θ 的偏导数，得到四个方程:

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

$$\frac{\partial \mathbf{r}_\theta((x,s))}{\partial \psi(\hat{x},\hat{a})} = \begin{cases} R(s,a) & \text{if } x = \hat{x} \\ 0 & \text{otherwise} \end{cases} \tag{23.19}$$

$$\frac{\partial \mathbf{r}_\theta((x,s))}{\partial \eta(\hat{x},\hat{a},\hat{o},\hat{x}')} = 0 \tag{23.20}$$

$$\frac{\partial \mathbf{Z}((x,s),(x',s'))}{\partial \psi(\hat{x},\hat{a})} = \begin{cases} -\gamma T(s,\hat{a},s') \sum_o O(\hat{a},s',o)\eta(\hat{x},\hat{a},o,x') & \text{if } x = \hat{x} \\ 0 & \text{otherwise} \end{cases} \tag{23.21}$$

$$\frac{\partial \mathbf{Z}((x,s),(x',s'))}{\partial \eta(\hat{x},\hat{a},\hat{o},\hat{x}')} = \begin{cases} -\gamma \psi(\hat{x},\hat{a}) T(s,\hat{a},s') O(\hat{a},s',\hat{o})\eta(\hat{x},\hat{a},\hat{o},x') & \text{if } x = \hat{x} \text{ and } x' = \hat{x}' \\ 0 & \text{otherwise} \end{cases} \tag{23.22}$$

最后，将这四个梯度代入方程（23.18）如下：

$$\frac{\partial \mathbf{u}_\theta}{\partial \psi(\hat{x},\hat{a})} = \mathbf{Z}^{-1}\left(\frac{\partial \mathbf{r}_\theta}{\partial \psi(\hat{x},\hat{a})} - \frac{\partial \mathbf{Z}}{\partial \psi(\hat{x},\hat{a})}\mathbf{Z}^{-1}\mathbf{r}_\theta\right) \tag{23.23}$$

$$\frac{\partial \mathbf{u}_\theta}{\partial \eta(\hat{x},\hat{a},\hat{o},\hat{x}')} = \mathbf{Z}^{-1}\left(\frac{\partial \mathbf{r}_\theta}{\partial \eta(\hat{x},\hat{a},\hat{o},\hat{x}')} - \frac{\partial \mathbf{Z}}{\partial \eta(\hat{x},\hat{a},\hat{o},\hat{x}')}\mathbf{Z}^{-1}\mathbf{r}_\theta\right) \tag{23.24}$$

我们终于可以回到式(23.11)中的原始目标。控制器梯度上升从固定数量的节点在X中开始，以及任意策略ψ和η。在第k次迭代时，它按如下方式更新这些参数：

$$\psi^{k+1}(x,a) = \psi^k(x,a) + \alpha \boldsymbol{\beta}^\top \frac{\partial \mathbf{u}_{\theta^k}}{\partial \psi^k(\hat{x},\hat{a})} \tag{23.25}$$

$$\eta^{k+1}(x,a,o,x') = \eta^k(x,a,o,x') + \alpha \boldsymbol{\beta}^\top \frac{\partial \mathbf{u}_{\theta^k}}{\partial \eta^k(\hat{x},\hat{a},\hat{o},\hat{x}')} \tag{23.26}$$

使用梯度步长 $\alpha >$ 为 0。在此更新之后，ψ^{k+1} 和 η^{k+1} 可能不再是有效的分布。为了使它们有效，我们将它们投影到概率单纯形上。将向量 y 投影到概率单纯形上的一种方法是根据 L_2 范数找到最近的分布：

$$\begin{aligned} \underset{\mathbf{b}}{\text{minimize}} \quad & \frac{1}{2}\|\mathbf{y} - \mathbf{b}\|_2^2 \\ \text{subject to} \quad & \mathbf{b} \geq 0 \\ & \mathbf{1}^\top \mathbf{b} = 1 \end{aligned} \tag{23.27}$$

这种优化可以通过算法23.6中包含的一个简单算法精确求解。[7] 示例23.6展示了更新con-的过程。

控制器。

[7] J. Duchi, S. Shalev-Shwartz, Y. Singer, 和 T. Chandra, “高效投影到ℓ_1球体以进行高维学习”，载于 *International Conference on Machine Learning (ICML)*, 2008。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

公式（23.6）中的优化目标不一定是凸的。[8] 因此，普通梯度上升可能会根据初始控制器收敛到局部最优解。可以应用自适应梯度算法来帮助平滑和加速收敛。

8 该目标与效用 $U(x、b) = \sum_s b(s)U(x、s)$ 不同，如第20.3节所述，效用 $U(x、b) = \sum_s b(s)U(x、s)$ 相对于信念状态 b 被保证是分段线性和凸的。

23.5 *Summary*

- 控制器是不依赖于探索或维护信念的策略表示。
- 控制器由节点、动作选择函数和后继选择函数组成。
- 节点和控制器图是抽象的；然而，它们可以被解释为可数无限可达信念的集合。
- 控制器节点的价值函数可以解释为一个alpha向量。
- 策略迭代在策略评估（计算每个节点的效用）和策略改进（添加新节点）之间交替进行。
- 在策略迭代过程中进行剪枝有助于减少每个改进步骤中节点数量的指数增长。
- 非线性规划将寻找最优固定尺寸控制器的问题重新表述为一个通用优化问题，从而允许使用现成的求解器和技术。
- 控制器梯度上升在策略空间中攀升，以直接改进价值函数，得益于基于 POMDP 的显式梯度步骤。

23.6 *Exercises*

练习23.1。列出控制器策略表示相对于基于树的条件计划和基于信念的表示所具有的任何优势。

Solution: 与基于树的条件计划不同，控制器可以表示能够无限期执行的策略。它们不必随着时间跨度的增加而呈指数级增长。

与基于信念的表示相比，控制器表示中的参数数量在较大问题中往往远少于alpha向量的数量。我们还可以在固定内存量的情况下更容易地优化控制器。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct ControllerGradient
    b       # initial belief
    ℓ       # number of nodes
    α       # gradient step
    k_max   # maximum iterations
end

function solve(M::ControllerGradient, 𝒫::POMDP)
    𝒜, 𝒪, ℓ, k_max = 𝒫.𝒜, 𝒫.𝒪, M.ℓ, M.k_max
    X = collect(1:ℓ)
    ψ = Dict((x, a) ⇒ rand() for x in X, a in 𝒜)
    η = Dict((x, a, o, x′) ⇒ rand() for x in X, a in 𝒜, o in 𝒪, x′ in X)
    π = ControllerPolicy(𝒫, X, ψ, η)
    for i in 1:k_max
        improve!(π, M, 𝒫)
    end
    return π
end

function improve!(π::ControllerPolicy, M::ControllerGradient, 𝒫::POMDP)
    𝒮, 𝒜, 𝒪, X, x1, ψ, η = 𝒫.𝒮, 𝒫.𝒜, 𝒫.𝒪, π.X, 1, π.ψ, π.η
    n, m, z, b, ℓ, α = length(𝒮), length(𝒜), length(𝒪), M.b, M.ℓ, M.α
    ∂U′∂ψ, ∂U′∂η = gradient(π, M, 𝒫)
    UIndex(x, s) = (s - 1) * ℓ + (x - 1) + 1
    E(U, x1, b) = sum(b[s]*U[UIndex(x1,s)] for s in 1:n)
    ψ′ = Dict((x, a) ⇒ 0.0 for x in X, a in 𝒜)
    η′ = Dict((x, a, o, x′) ⇒ 0.0 for x in X, a in 𝒜, o in 𝒪, x′ in X)
    for x in X
        ψ′x = [ψ[x, a] + α * E(∂U′∂ψ(x, a), x1, b) for a in 𝒜]
        ψ′x = project_to_simplex(ψ′x)
        for (aIndex, a) in enumerate(𝒜)
            ψ′[x, a] = ψ′x[aIndex]
        end
        for (a, o) in product(𝒜, 𝒪)
            η′x = [(η[x, a, o, x′] +
                    α * E(∂U′∂η(x, a, o, x′), x1, b)) for x′ in X]
            η′x = project_to_simplex(η′x)
            for (x′Index, x′) in enumerate(X)
                η′[x, a, o, x′] = η′x[x′Index]
            end
        end
    end
    π.ψ, π.η = ψ′, η′
end

function project_to_simplex(y)
    u = sort(copy(y), rev=true)
    i = maximum([j for j in eachindex(u)
                 if u[j] + (1 - sum(u[1:j])) / j > 0.0])
    δ = (1 - sum(u[j] for j = 1:i)) / i
    return [max(y[j] + δ, 0.0) for j in eachindex(u)]
end
```

算法23.6. 针对初始信念b的POMDP $\mathcal{P}$控制器梯度上升算法的一种实现。控制器本身具有固定大小，包含ℓ个节点。通过沿控制器的梯度方向，以步长α进行k_max次迭代来改进，以最大化提升初始信念的价值。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
function gradient(π::ControllerPolicy, M::ControllerGradient, 𝒫::POMDP)
    𝒮, 𝒜, 𝒪, T, O, R, γ = 𝒫.𝒮, 𝒫.𝒜, 𝒫.𝒪, 𝒫.T, 𝒫.O, 𝒫.R, 𝒫.γ
    X, x1, ψ, η = π.X, 1, π.ψ, π.η
    n, m, z = length(𝒮), length(𝒜), length(𝒪)
    X𝒮 = vec(collect(product(X, 𝒮)))
    T′ = [sum(ψ[x, a] * T(s, a, s′) * sum(O(a, s′, o) * η[x, a, o, x′]
          for o in 𝒪) for a in 𝒜) for (x, s) in X𝒮, (x′, s′) in X𝒮]
    R′ = [sum(ψ[x, a] * R(s, a) for a in 𝒜) for (x, s) in X𝒮]
    Z = 1.0I(length(X𝒮)) - γ * T′
    invZ = inv(Z)
    ∂Z∂ψ(hx, ha) = [x == hx ? (-γ * T(s, ha, s′)
                    * sum(O(ha, s′, o) * η[hx, ha, o, x′]
                          for o in 𝒪)) : 0.0
                    for (x, s) in X𝒮, (x′, s′) in X𝒮]
    ∂Z∂η(hx, ha, ho, hx′) = [x == hx && x′ == hx′ ? (-γ * ψ[hx, ha]
                    * T(s, ha, s′) * O(ha, s′, ho)) : 0.0
                  for (x, s) in X𝒮, (x′, s′) in X𝒮]
    ∂R′∂ψ(hx, ha) = [x == hx ? R(s, ha) : 0.0 for (x, s) in X𝒮]
    ∂R′∂η(hx, ha, ho, hx′) = [0.0 for (x, s) in X𝒮]
    ∂U′∂ψ(hx, ha) = invZ * (∂R′∂ψ(hx, ha) - ∂Z∂ψ(hx, ha) * invZ * R′)
    ∂U′∂η(hx, ha, ho, hx′) = invZ * (∂R′∂η(hx, ha, ho, hx′)
                               - ∂Z∂η(hx, ha, ho, hx′) * invZ * R′)
    return ∂U′∂ψ, ∂U′∂η
end
```

算法23.7。控制器梯度上升法的`gradient`步骤。它构建了效用U关于策略梯度$\partial U'\partial\psi$和$\partial U'\partial\eta$的梯度。

在执行过程中，控制器绝不会像基于信念的策略那样出现除以零的情况。基于信念的方法需要维护一个信念。当遇到不可能的观测时，公式 (19.7) 中的离散状态滤波器会出现除以零的现象。这种情况可能发生在传感器返回的含噪观测无法被 $T(s、a、s')$ 以及 $O(o \mid a、s')$ 的模型准确捕捉时。

练习 23.2。控制器策略迭代仅添加具有确定性动作选择函数和后继分布的节点。这是否意味着所得到的控制器必然是次优的?

Solution: 控制器策略迭代在极限情况下保证收敛到最优策略。然而，该方法无法找到可能需要随机节点的最优控制器策略的更紧凑表示。

练习23.3。证明策略迭代中的节点剪枝不会影响效用。

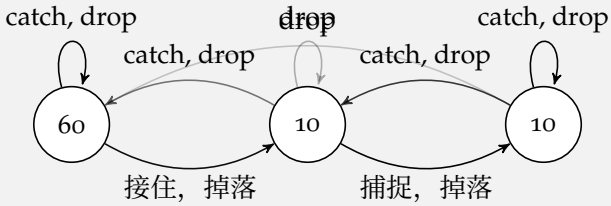
Solution: 令 x' 为来自某次迭代 i 的新节点，而 x 为来自迭代 $i-1$ 的前一节点。

按构造，$\eta(x'、a、o、x)$ 定义了所有新节点 x' 仅有一个来自前一次迭代的后继节点 x。因此，对于每个状态 s，$U^{(i)}(x'、s)$ 在方程 (23.1) 中仅对后继节点 $U^{(i-1)}(x、s')$ 求和。这意味着迭代 i 中的其他效用，包括

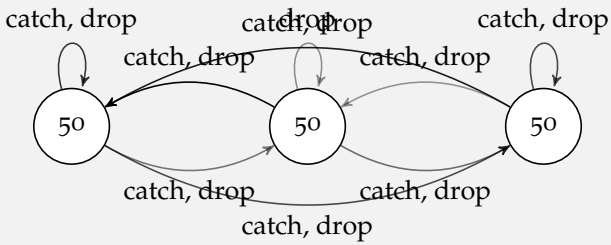
© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

考虑接球问题（附录F.9），初始信念均匀为b_1。此处图表展示了在具有$k =$ 个节点的接球问题上应用梯度上升迭代时策略的效用。左侧节点为x_1。

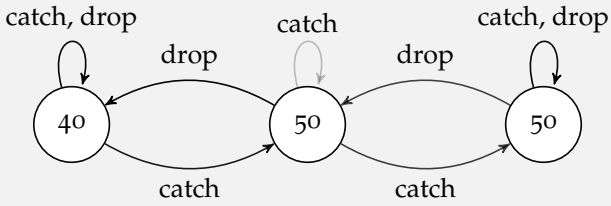
在第1次迭代时，策略在动作选择和后继选择上基本上是随机的：



在第50次迭代时，智能体已确定了一个合理的投球距离（50），但仍未利用其三个节点记住任何有用信息：



在第500次迭代时，鉴于其固定的三个内存节点，策略已构建了一个合理的计划：



它首先尝试在距离40的位置扔球。如果孩子接住了球，那么它会将距离增加到50。它使用最终节点来记住孩子接住球的次数（最多两次），以选择距离。

示例 23.6. 针对固定大小为$\ell = 3$的控制器的控制器梯度算法演示。该策略显示为在算法的迭代过程中自我优化。代理逐步确定如何最佳利用其固定数量的节点，从而在收敛时形成合理且可解释的策略。随机控制器以圆圈表示，最可能的动作位于中间。输出边显示了给定观察时的后继节点选择。节点动作和后继的随机性以不透明度表示（更不透明表示概率更高，更透明表示概率更低）。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

一个自环到x本身，不影响效用$U^{(i)}(x', s)$。由于初始节点是由方程(23.3)选择的，我们必须确保在有和没有修剪节点的情况下，所有信念的效用是相同的。节点修剪有两种方式之一。

首先，x'在所有状态上比其修剪后的后继x获得更高的效用。形式上，$U^{(i)}(x, s) \leq U^{(i-1)}(x', s)$对于所有$s$。修剪步骤将$x$替换为$x'$，包括$U$、$\psi$和$\eta$。通过构造，$U$在任何状态$s$上都没有减少。

其次，x与现有的先前节点x'相同。注意，这意味着过渡$\eta(x, a, o, x') = \eta(x', a, o, x')$。这意味着效用是相同的，只是$x$被$\gamma$减少；换句话说，根据方程(23.1)，$\gamma U^{(i)}(x, s) = U^{(i-1)}(x, s)$。修剪$x$不影响最终效用。

练习23.4. 设计一种算法，利用非线性规划算法寻找最小固定尺寸控制器，以达到尺寸为ℓ的大型固定尺寸控制器的最优性。可以假设在此情况下，非线性优化器返回最优策略。

Solution: 这个想法是创建一个外层循环，在得知大型固定尺寸控制器的效用后，递增控制器的固定尺寸。首先，我们必须计算大型固定尺寸控制器在初始节点x_1和初始信念b_1处的效用$U^* = \sum_s b_1(s) U(x_1, s)$。接下来，我们创建一个循环，递增控制器的大小$\ell$。在每一步中，我们评估策略并计算效用$U^\ell$。根据我们的假设，返回的控制器为固定大小$\ell$产生全局最优效用。一旦我们达到效用$U^\ell$，如果我们看到$U^\ell = U^*$，那么我们就停止并返回策略。

练习23.5。分析控制器梯度上升算法的梯度步骤。假设$|\mathcal{S}|$大于$|\mathcal{A}|$和$|\mathcal{O}|$。梯度步骤中计算量最大的部分是什么？如何改进这一点？

Solution: 计算逆$\mathrm{Z}^{-1} = (\mathrm{I} - \gamma \mathrm{T}_\theta)$是梯度步骤中计算成本最高的部分，也是整个梯度算法中成本最高的部分。矩阵Z的大小为$|X \times \mathcal{S}|$。高斯-约旦消元法需要$O(|X \times \mathcal{S}|^3)$次运算，尽管通过使用最先进的矩阵求逆算法，指数中的3可以减少到2.3728639。[9]

临时矩阵T_θ的创建也需要$O(|X \times \mathcal{S}|^2 |\mathcal{A} \times \mathcal{O}|)$次操作来支持计算逆。所有其他循环和其他临时数组的创建所需的运算次数要少得多。这可以通过近似逆技术来改进。

[9] 参见F. L. Gall，“快速的幂与快速矩阵乘法”，*International Symposium on Symbolic and Algebraic Computation*（ISSAC），2014年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

第五部分

多智能体系统

到目前为止，我们一直聚焦于单个智能体的决策视角。现在，我们将迄今讨论的核心概念扩展至涉及多个智能体的问题。在多智能体系统中，我们可以将其他智能体模型化为潜在的盟友或对手，并随时间相应调整策略。这类问题本质上极具挑战性，原因在于智能体间交互的复杂性，以及智能体需要对其他智能体进行推理，而这些智能体又在对当前智能体进行推理，如此循环往复。我们首先介绍游戏中的多智能体推理，并概述如何从简单互动中计算均衡点。随后，我们讨论如何为随时间交互的多个智能体设计算法，描述倾向于理性适应而非仅收敛至均衡的学习算法。引入状态不确定性显著增加了问题的复杂度，本部分着重强调了由此带来的挑战。最后一章聚焦于协作智能体的各种模型与算法，这些智能体致力于围绕共同目标协同工作。

24 Multiagent Reasoning

到目前为止，我们专注于为单个智能体做出理性决策。这些模型自然可以扩展到多个智能体。随着智能体之间的交互，新的挑战出现了；智能体可以互相帮助或追求自身的最佳利益。多智能体推理是*game theory*.[1]的主题。本章基于这些概念构建。

之前介绍的，将它们扩展到多智能体场景。我们将讨论计算决策策略和多智能体均衡的基础博弈论方法。

1 博弈论是一个广泛的领域。几本标准的入门书籍包括 D. Fudenberg 和 J. Tirole，*Game Theory*。MIT Press，1991年。R. B. Myerson，*Game Theory: Analysis of*。哈佛大学出版社，1997年。Y. Shoham 和 K. Leyton-Brown，*Multiagent Systems: Algo-*。剑桥大学出版社，2009年。*rithmic, Game-Theoretic, and Logical Foundations*

24.1 Simple Games

一个*simple game* (算法24.1)是多智能体推理的基本模型。[2]每个智能体 $i \in \mathcal{I}$ 选择一个动作 a^i 以最大化自身奖励 r^i 的累积。*joint action space* $\mathcal{A} = \mathcal{A}^1 \times \cdots \times \mathcal{A}^k$ 由每个智能体可用的动作 $\mathcal{A}^i$ 的所有可能组合构成。跨智能体同时选择的动作可以组合，从该联合动作空间中形成 *joint action* 一个 $= (a^1, \ldots, a^k)$。[3] *joint reward function* $\mathrm{R(a)} = (R^1(\mathrm{a}), \ldots, R^k(\mathrm{a}))$ 表示奖励由联合动作 a 产生。*joint reward*被写为$\mathrm{r} = (r^1, \ldots, r^k)$。简单游戏不包含状态或转移函数。例24.1介绍了一个简单游戏。

form games

joint action

tion profile

2 简单博弈包括 *normal*，也称为 *standard* 或 *matrix games*)、有限期 *repeated games* 以及无限期折扣重复博弈。Y. Shoham 和 K. Leyton-Brown，*Multiagent Systems:*。剑桥大学出版社，2009年。*Algorithmic, Game-Theoretic, and Logical Foundations*

3 联合行动也称为 *ac-*。

```
struct SimpleGame
    γ  # discount factor
    ℐ  # agents
    𝒜  # joint action space
    R  # joint reward function
end
```

算法24.1. 简单游戏的数据结构。

*prisoner's dilemma*是一个涉及两名受审囚犯的双智能体、双动作博弈。他们可以选择*cooperate*并对其共同犯罪保持沉默，或者*defect*并指责对方犯罪。如果他们都合作，则各服刑一年。如果智能体i合作而另一个智能体背叛，则i服刑四年，另一个不服刑。如果双方都背叛，则各服刑三年。

双代理简单游戏可以通过表格来表示。行代表代理1的动作。列代表代理2的动作。每个单元格中显示了代理1和2的奖励。

例24.1. 一个简单的游戏，被称为囚徒困境。更多细节参见附录F.10。

agent 1 \ agent 2	cooperate	defect
cooperate	$-1, -1$	$-4, 0$
defect	$0, -4$	$-3, -3$

一个 *joint policy* $\boldsymbol{\pi}$ 指定了智能体所采取的联合动作上的概率分布。联合策略可以分解为各个智能体的策略。智能体 i 选择动作 a 的概率由 $\pi^i(a)$ 给出。在博弈论中，确定性策略称为 *pure strategy*，随机性策略称为 *mixed strategy*。从智能体 i 的角度看，联合策略 $\boldsymbol{\pi}$ 的效用为

$$U^i(\boldsymbol{\pi}) = \sum_{\mathbf{a} \in \mathcal{A}} R^i(\mathbf{a}) \prod_{j \in \mathcal{I}} \pi^j(a^j) \tag{24.1}$$

算法24.2实现了用于表示策略并计算其效用的例程。

一个*zero-sum game*是一种简单游戏，其中各代理的奖励总和为零。在这里，任何代理的收益都是其他代理的损失。一个包含两个代理$\mathcal{I} = \{1、2\}$的零和游戏具有相反的奖励函数$R^1(\mathrm{a}) = -R^2(\mathrm{a})$。它们通常使用针对这种奖励结构专门设计的算法来解决。例24.2描述了这样一个游戏。

24.2 Response Models

在探讨求解联合策略的不同概念之前，我们将首先讨论如何在给定其他代理固定策略的情况下，建模单个代理*response*的i。我们将使用符号$-i$作为$(1, \ldots, i-1, i+1, \ldots, k)$的简写。利用这一符号，联合动作表示为$= (a^i, \mathrm{a}^{-i})$，联合奖励表示为$\mathrm{R}(a^i, \mathrm{a}^{-i})$，而联合策略则表示为$\boldsymbol{\pi} = (\pi^i, \boldsymbol{\pi}^{-i})$。本节将讨论针对已知$\boldsymbol{\pi}^{-i}$计算响应的多种方法。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Rock-paper-scissors 是一个双人零和博弈。每个agent选择*rock*、*paper*或*scissors*。石头胜剪刀，布胜石头，剪刀胜布，胜者获得1的奖励，负者获得−1的奖励。如果agent选择相同的动作，双方都获得0奖励。通常，双人重复博弈可以表示为一系列收益矩阵，如下所示：

$t = 1$

agent 1 \ agent 2	rock	paper	scissors
rock	0, 0	−1, 1	1, −1
paper	1, −1	0, 0	−1, 1
scissors	−1, 1	1, −1	0, 0

$t = 2$

agent 1 \ agent 2	rock	paper	scissors
rock	0, 0	−1, 1	1, −1
paper	1, −1	0, 0	−1, 1
scissors	−1, 1	1, −1	0, 0

$\cdots$

$\cdots$

示例24.2.众所周知的石头-剪刀-布游戏是一个零和游戏的例子。附录F.11提供了额外的细节。

24.2.1 Best Response

代理 i 对其它代理 $\boldsymbol{\pi}^{-i}$ 的策略的 *best response* 是一个满足以下条件的策略 π^i：

$$U^i(\pi^i, \boldsymbol{\pi}^{-i}) \geq U^i(\pi^{i\prime}, \boldsymbol{\pi}^{-i}) \tag{24.2}$$

对于所有其他策略 $\pi^{i\prime} \neq \pi^i$。换句话说，在给定其他代理固定策略集的情况下，代理的最佳响应是一种没有动机改变其策略的策略。可能存在多个最佳响应。

如果我们将自己限制在确定性策略上，从*deterministic best response*到对手策略 $\boldsymbol{\pi}^{-i}$的计算就非常直接。我们只需遍历代理i的所有行动，并返回最大化效用的那个行动，如下所示：

$$\arg\max_{a^i \in \mathcal{A}^i} U^i(a^i, \boldsymbol{\pi}^{-i}) \tag{24.3}$$

算法24.3提供了这一实现。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct SimpleGamePolicy
    p # dictionary mapping actions to probabilities

    function SimpleGamePolicy(p::Base.Generator)
        return SimpleGamePolicy(Dict(p))
    end

    function SimpleGamePolicy(p::Dict)
        vs = collect(values(p))
        vs ./= sum(vs)
        return new(Dict(k ⇒ v for (k,v) in zip(keys(p), vs)))
    end

    SimpleGamePolicy(ai) = new(Dict(ai ⇒ 1.0))
end

(πi::SimpleGamePolicy)(ai) = get(πi.p, ai, 0.0)

function (πi::SimpleGamePolicy)()
    D = SetCategorical(collect(keys(πi.p)), collect(values(πi.p)))
    return rand(D)
end

joint(X) = vec(collect(product(X...)))

joint(π, πi, i) = [i == j ? πi : πj for (j, πj) in enumerate(π)]

function utility(𝒫::SimpleGame, π, i)
    𝒜, R = 𝒫.𝒜, 𝒫.R
    p(a) = prod(πj(aj) for (πj, aj) in zip(π, a))
    return sum(R(a)[i]*p(a) for a in joint(𝒜))
end
```

算法24.2。与智能体相关联的策略由一个将动作映射到概率的字典表示。构造策略有多种方式。一种方式是传入一个字典目录，在这种情况下概率会被归一化。另一种方式是传入一个生成该字典的生成器。我们还可以通过传入一个动作来构造策略，在这种情况下，它为该动作分配概率1。如果我们有一个个体策略`πi`，我们可以调用`πi(ai)`来计算该策略与动作`ai`相关联的概率。如果我们调用`πi()`，那么它将根据该策略返回一个随机动作。我们可以使用`joint(𝒜)`从$\mathcal{A}$构造联合动作空间。我们可以使用`utility(𝒫, π, i)`从智能体`i`的角度计算在博弈$\mathcal{P}$中执行联合策略`π`所关联的效用。

```
function best_response(𝒫::SimpleGame, π, i)
    U(ai) = utility(𝒫, joint(π, SimpleGamePolicy(ai), i), i)
    ai = argmax(U, 𝒫.𝒜[i])
    return SimpleGamePolicy(ai)
end
```

算法24.3。对于简单博弈$\mathcal{P}$，我们可以计算在给定其他代理采用策略`π`的情况下，代理`i`的确定性最佳响应。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

24.2.2 *Softmax Response*

我们可以使用*softmax response*来建模智能体i将如何选择他们的动作。[4] 如同在第6.7节中讨论过，人类往往并非完全理性的期望效用优化者。柔性最大值响应模型的基本原理是，（通常指人类）agent在优化过程中，当错误代价较小时更可能犯错。给定一个 *precision parameter* $\lambda \geq 0$，该模型根据以下方式选择动作 a^i

$$\pi^i(a^i) \propto \exp(\lambda U^i(a^i, \boldsymbol{\pi}^{-i})) \tag{24.4}$$

当 $\lambda \to$ 为 0 时，代理对效用差异不敏感，并均匀随机地选择动作。当 $\lambda \to \infty$ 时，策略收敛到确定性的最佳响应。我们可以将 λ 视为一个参数，可以使用例如最大似然估计（第 4.1 节）从数据中学习。这种基于学习的方法旨在预测行为，而不是规定行为，尽管拥有其他人类代理的预测模型对于构建规定最优行为的系统可能是有用的。算法 24.4 提供了 softmax 响应的实现。

[4] 这类模型有时被称为 *logit response* 或 *quantal response*。我们在本书前面关于强化学习定向探索策略的章节中（第15.4节）介绍过类似的softmax模型。

```
function softmax_response(𝒫::SimpleGame, π, i, λ)
    𝒜i = 𝒫.𝒜[i]
    U(ai) = utility(𝒫, joint(π, SimpleGamePolicy(ai), i), i)
    return SimpleGamePolicy(ai ⇒ exp(λ*U(ai)) for ai in 𝒜i)
end
```

算法24.4。对于一个简单游戏$\mathcal{P}$和一个特定代理`i`，我们可以计算softmax响应策略`πi`，假设其他代理正在使用`π`中的策略。此计算需要指定精度参数`λ`。

24.3 *Dominant Strategy Equilibrium*

在一些博弈中，一个代理人拥有*dominant strategy*，这是一种针对所有其他可能的代理人策略的最佳回应策略。例如，在囚徒困境（例24.1）中，无论代理人2采取何种策略，代理人1的最佳回应都是背叛，这使得背叛成为代理人1的占优策略。所有代理人都采用占优策略的联合策略被称为*dominant strategy equilibrium*。在囚徒困境中，双方代理人都背叛的联合策略是一个占优策略均衡。[5] 许多博弈并没有占优策略

均衡。例如，在石头剪刀布游戏（例24.2）中，代理1的最佳响应取决于代理2的策略。

[5] 有趣的是，如果两个代理都各自按照自己的效用函数贪婪行动，结果对双方都不利。倘若他们合作，本应只被判一年而非三年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

24.4 Nash Equilibrium

与占优策略均衡概念相比，纳什均衡[6]对于具有有限动作空间的博弈，[7]总是存在。*Nash equilibrium*是一个联合策略 $\boldsymbol{\pi}$，其中所有智能体都遵循最佳响应。换句话说，纳什均衡是一种联合策略，其中没有任何智能体有动机单方面改变其策略。

单个博弈中可能存在多个纳什均衡（练习 24.2）。有时纳什均衡可能涉及确定性策略，但并非总是如此（见例 24.3）。计算纳什均衡是 *PPAD-complete*，这一类别不同于 NP 完全问题（附录 C.2），但同样没有已知的多项式时间算法。[8]

寻找纳什均衡的问题可以被形式化为一个优化问题：

$$\begin{aligned} \underset{\boldsymbol{\pi}, U}{\text{minimize}} \quad & \sum_i \left(U^i - U^i(\boldsymbol{\pi}) \right) \\ \text{subject to} \quad & U^i \geq U^i(a^i, \boldsymbol{\pi}^{-i}) \text{ for all } i, a^i \\ & \sum_{a^i} \pi^i(a^i) = 1 \text{ for all } i \\ & \pi^i(a^i) \geq 0 \text{ for all } i, a^i \end{aligned} \tag{24.5}$$

优化变量对应于$\boldsymbol{\pi}$和U的参数。在收敛时，目标将为0，其中U^i与策略$\boldsymbol{\pi}$相关的效用相匹配，如方程(24.1)中对每个代理i所计算的那样。第一个约束确保没有代理能通过单方面改变其行动而获得更好的结果。与目标一样，这第一个约束是非线性的，因为它涉及优化变量$\boldsymbol{\pi}$中参数的乘积。最后两个约束是线性的，确保$\boldsymbol{\pi}$表示动作上有效的概率分布集。算法24.5实现了这一优化过程。

24.5 Correlated Equilibrium

*correlated equilibrium*通过放宽代理独立行动的假设，推广了纳什均衡概念。在这种情况下，联合行动源自一个完整的联合分布。*correlated joint policy* $\pi(\mathrm{a})$是关于所有代理联合行动的单一分布。因此，各个代理的行动

6 以美国数学家小约翰·福布斯·纳什（1928–2015）命名，他将这一概念形式化。J. Nash, “非合作博弈”, *Annals of Mathematics*, 第286–295页, 1951年。

7 练习24.1探讨了动作空间无限的情况。

8 C. Daskalakis, P. W. Goldberg, and C. H. Papadimitriou, “计算纳什均衡的复杂性”, *Communications of the ACM*, 第52卷, 第2期, 第89–97页, 2009年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

例24.3. 确定性和随机纳什均衡。

假设我们希望为示例24.1中的囚徒困境找到一个纳什均衡。如果两个代理总是背叛，双方都将获得−3的奖励。任何代理的任何偏离都会导致该代理获得−4的奖励；因此，没有偏离的动机。两个代理都背叛因此是囚徒困境的一个纳什均衡。

假设我们现在希望为例24.2中的石头-剪刀-布场景找到一个纳什均衡。一个代理的任何确定性策略都很容易被另一个代理所 counter。例如，如果代理1出石头，那么代理2的最佳响应是布。由于石头-剪刀-布不存在确定性纳什均衡，我们知道必定存在一个涉及随机策略的均衡。假设每个代理从动作中均匀随机选择。这个解为两个代理产生的期望效用都是0：

$$\begin{aligned} U^i(\boldsymbol{\pi}) &= 0\frac{1}{3}\frac{1}{3} - 1\frac{1}{3}\frac{1}{3} + 1\frac{1}{3}\frac{1}{3} \\ &\quad + 1\frac{1}{3}\frac{1}{3} + 0\frac{1}{3}\frac{1}{3} - 1\frac{1}{3}\frac{1}{3} \\ &\quad - 1\frac{1}{3}\frac{1}{3} + 1\frac{1}{3}\frac{1}{3} + 0\frac{1}{3}\frac{1}{3} \\ &= 0 \end{aligned}$$

任何代理的偏离都会降低其预期收益，这意味着我们已经找到了纳什均衡。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct NashEquilibrium end

function tensorform(𝒫::SimpleGame)
    ℐ, 𝒜, R = 𝒫.ℐ, 𝒫.𝒜, 𝒫.R
    ℐ′ = eachindex(ℐ)
    𝒜′ = [eachindex(𝒜[i]) for i in ℐ]
    R′ = [R(a) for a in joint(𝒜)]
    return ℐ′, 𝒜′, R′
end

function solve(M::NashEquilibrium, 𝒫::SimpleGame)
    ℐ, 𝒜, R = tensorform(𝒫)
    model = Model(Ipopt.Optimizer)
    @variable(model, U[ℐ])
    @variable(model, π[i=ℐ, 𝒜[i]] ≥ 0)
    @NLobjective(model, Min,
        sum(U[i] - sum(prod(π[j,a[j]] for j in ℐ) * R[y][i]
            for (y,a) in enumerate(joint(𝒜))) for i in ℐ))
    @NLconstraint(model, [i=ℐ, ai=𝒜[i]],
        U[i] ≥ sum(
            prod(j==i ? (a[j]==ai ? 1.0 : 0.0) : π[j,a[j]] for j in ℐ)
            * R[y][i] for (y,a) in enumerate(joint(𝒜))))
    @constraint(model, [i=ℐ], sum(π[i,ai] for ai in 𝒜[i]) == 1)
    optimize!(model)
    πi′(i) = SimpleGamePolicy(𝒫.𝒜[i][ai] ⇒ value(π[i,ai]) for ai in 𝒜[i])
    return [πi′(i) for i in ℐ]
end
```

算法24.5。该非线性程序计算简单博弈$\mathcal{P}$的纳什均衡。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

可能是相关的，从而阻止了策略被拆分为单个策略 $\pi^i(a^i)$。算法24.6展示了如何表示这样的策略。

```
mutable struct JointCorrelatedPolicy
    p # dictionary mapping from joint actions to probabilities
    JointCorrelatedPolicy(p::Base.Generator) = new(Dict(p))
end

(π::JointCorrelatedPolicy)(a) = get(π.p, a, 0.0)

function (π::JointCorrelatedPolicy)()
    D = SetCategorical(collect(keys(π.p)), collect(values(π.p)))
    return rand(D)
end
```

算法24.6。联合相关策略由一个将联合动作映射到概率的字典表示。如果π是一个联合相关策略，那么评估π(a)将返回与联合动作a相关的概率。

一个 *correlated equilibrium* 是一个相关联合策略，其中没有任何代理 i 可以通过从当前动作 a^i 偏离到另一个动作 $a^{i\prime}$ 来提高其预期效用：

$$\sum_{\mathbf{a}^{-i}} R^i(a^i, \mathbf{a}^{-i})\pi(a^i, \mathbf{a}^{-i}) \geq \sum_{\mathbf{a}^{-i}} R^i(a^{i\prime}, \mathbf{a}^{-i})\pi(a^i, \mathbf{a}^{-i}) \tag{24.6}$$

示例 24.4 演示了这一概念。

每个纳什均衡都是相关均衡，因为我们总能从独立策略形成联合策略：

$$\pi(\mathbf{a}) = \prod_{i=1}^{k} \pi^i(a^i) \tag{24.7}$$

如果各个策略满足方程(24.2)，那么联合策略将满足方程(24.6)。然而，并非所有相关均衡都是纳什均衡。

可以使用线性规划（算法24.7）计算相关均衡：

$$\begin{aligned}
\underset{\boldsymbol{\pi}}{\text{maximize}} \quad & \sum_i \sum_{\mathbf{a}} R^i(\mathbf{a})\pi(\mathbf{a}) \\
\text{subject to} \quad & \sum_{\mathbf{a}^{-i}} R^i(a^i, \mathbf{a}^{-i})\pi(a^i, \mathbf{a}^{-i}) \geq \sum_{\mathbf{a}^{-i}} R^i(a^{i\prime}, \mathbf{a}^{-i})\pi(a^i, \mathbf{a}^{-i}) \quad \text{for all } i, a^i, a^{i\prime} \\
& \sum_{\mathbf{a}} \pi(\mathbf{a}) = 1 \\
& \pi(\mathbf{a}) \geq 0 \quad \text{for all } \mathbf{a}
\end{aligned} \tag{24.8}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

例24.4。计算石头-剪刀-布中的相关均衡。

再次考虑例24.2中的石头-剪刀-布场景。在例24.3中，我们发现纳什均衡涉及两个代理随机均匀选择其行动。在相关均衡中，我们使用相关联合策略$\pi(a)$，这意味着我们需要找到一个分布，覆盖（石头，石头）、（石头，布）、（石头，剪刀）、（布，石头）等。共有九种可能的联合行动。

首先，考虑智能体1选择石头而智能体2选择剪刀的联合策略。效用为

$$U^1(\pi) = 0\frac{0}{9} - 1\frac{0}{9} + 1\frac{9}{9} + 1\frac{0}{9} + \cdots = 1$$
$$U^2(\pi) = 0\frac{0}{9} + 1\frac{0}{9} - 1\frac{9}{9} - 1\frac{0}{9} + \cdots = -1$$

如果代理人2切换到纸，其效用将为1。因此，这并不是一个相关均衡。

相反，考虑一种相关的联合策略，其中联合动作是均匀随机选择的，$\pi(a) =$为1/9：

$$U^1(\pi) = 0\frac{1}{9} - 1\frac{1}{9} + 1\frac{1}{9} + 1\frac{1}{9} + \cdots = 0$$
$$U^2(\pi) = 0\frac{1}{9} + 1\frac{1}{9} - 1\frac{1}{9} - 1\frac{1}{9} + \cdots = 0$$

任何偏离此结果的情况都会导致一个代理获得效用，而另一个代理失去效用。这是石头剪刀布博弈中的相关均衡。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

虽然线性规划可以在多项式时间内求解，但联合动作空间的大小随着代理数量的增加而呈指数增长。约束条件强制实现相关均衡。然而，目标函数可用于在多个有效相关均衡中做出选择。表24.1列出了几种常见的目标函数选择。

```
struct CorrelatedEquilibrium end

function solve(M::CorrelatedEquilibrium, 𝒫::SimpleGame)
    ℐ, 𝒜, R = 𝒫.ℐ, 𝒫.𝒜, 𝒫.R
    model = Model(Ipopt.Optimizer)
    @variable(model, π[joint(𝒜)] ≥ 0)
    @objective(model, Max, sum(sum(π[a]*R(a) for a in joint(𝒜))))
    @constraint(model, [i=ℐ, ai=𝒜[i], ai′=𝒜[i]],
        sum(R(a)[i]*π[a] for a in joint(𝒜) if a[i]==ai)
        ≥ sum(R(joint(a,ai′,i))[i]*π[a] for a in joint(𝒜) if a[i]==ai))
    @constraint(model, sum(π) == 1)
    optimize!(model)
    return JointCorrelatedPolicy(a ⇒ value(π[a]) for a in joint(𝒜))
end
```

算法24.7。对于简单博弈$\mathcal{P}$而言，相关均衡是一种比纳什均衡更为普遍的最优性概念。它们可以通过线性规划来计算。由此产生的策略是相关的，这意味着代理随机地选择它们的联合行动。

Name	Description	Objective Function
Utilitarian	Maximize the net utility.	$\text{maximize}_\pi \sum_i \sum_{\mathbf{a}} R^i(\mathbf{a})\pi(\mathbf{a})$
Egalitarian	Maximize the minimum of all agents' utilities.	$\text{maximize}_\pi \text{minimize}_i \sum_{\mathbf{a}} R^i(\mathbf{a})\pi(\mathbf{a})$
Plutocratic	Maximize the maximum of all agents' utilities.	$\text{maximize}_\pi \text{maximize}_i \sum_{\mathbf{a}} R^i(\mathbf{a})\pi(\mathbf{a})$
Dictatorial	Maximize agent i's utility.	$\text{maximize}_\pi \sum_{\mathbf{a}} R^i(\mathbf{a})\pi(\mathbf{a})$

表24.1. 方程(24.8)的替代目标函数，这些函数为各种相关均衡选择。这些描述改编自A. Greenwald和K. Hall，“相关Q学习”，收录于*International Conference on Machine Learning*（*ICML*），2003年。

24.6 Iterated Best Response

由于计算纳什均衡可能计算成本高昂，一种替代方法是在一系列重复博弈中迭代应用最佳响应。在*iterated best response*（算法24.8）中，我们在智能体之间随机循环，依次求解每个智能体的最佳响应策略。这个过程可能会收敛到纳什均衡，但仅对某些类别的博弈有保证。[9]在许多问题中，通常会观察到循环现象。

9 迭代最佳响应将会收敛，例如，对于一类被称为 *potential games* 的情况，正如 N. Nisan, T. Roughgarden, É. Tardos, 和 V. V. Vazirani 编著的教科书 *Algorithmic Game Theory* 中定理 19.12 所讨论的那样。剑桥大学出版社，2007年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct IteratedBestResponse
    k_max # number of iterations
    π     # initial policy
end

function IteratedBestResponse(𝒫::SimpleGame, k_max)
    π = [SimpleGamePolicy(ai ⇒ 1.0 for ai in 𝒜i) for 𝒜i in 𝒫.𝒜]
    return IteratedBestResponse(k_max, π)
end

function solve(M::IteratedBestResponse, 𝒫)
    π = M.π
    for k in 1:M.k_max
        π = [best_response(𝒫, π, i) for i in 𝒫.ℐ]
    end
    return π
end
```

算法24.8。迭代最佳响应涉及循环遍历各个智能体，并将它们的最佳响应应用于其他智能体。该算法从某个初始策略开始，并在`k_max`次迭代后停止。为方便起见，我们提供了一个构造函数，它接受一个简单博弈作为输入，并创建一个初始策略，使每个智能体以均匀随机的方式选择动作。同一个求解函数将在下一章中，在更复杂形式的博弈背景下被复用。

24.7 Hierarchical Softmax

一个被称为*behavioral game theory*的领域旨在模拟人类代理。在构建必须与人类互动的决策系统时，计算纳什均衡并不总是有帮助的。人类往往不会采取纳什均衡策略。首先，如果博弈中存在许多不同的均衡，那么采用哪一个均衡可能就不清楚。对于只有一个均衡的博弈，由于认知限制，人类可能难以计算纳什均衡。即使人类代理能够计算纳什均衡，他们也可能怀疑对手能否执行这种计算。

文献中有许多行为模型，[10]但一种方法是将上一节中的迭代方法与softmax模型相结合。这种*hierarchical softmax*方法（算法24.9）[11]对*depth of rationality*建模由$k \geq$级别为0的代理。级别为0的代理随机均匀地采取行动。级别为1的代理假设其他玩家采用级别为0的策略，并根据精度为λ的softmax响应选择行动。级别为k的代理根据其他玩家采用级别为$k-1$的softmax模型选择行动。图24.1展示了这一方法在一个简单游戏中的应用。

10 C. F. Camerer, *Behavioral Game Theory: Experiments in Strategic Interaction*. Princeton University Press, 2003.

11 这种方法有时被称为*quantal-level-k*或*logit-level-k*。D. O. Stahl和P. W. Wilson，“关于参与者对其他参与者模型的实验证据”，*Journal of Economic Behavior & Organization*，第25卷，第3期，第309-327页，1994年。

我们可以从数据中学习这种行为模型的k和λ参数。如果我们有不同代理执行的一系列联合动作，我们可以计算给定k和λ的相关似然。然后，我们可以使用优化算法来尝试找到使似然最大化的k和λ值。这

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct HierarchicalSoftmax
    λ # precision parameter
    k # level
    π # initial policy
end

function HierarchicalSoftmax(𝒫::SimpleGame, λ, k)
    π = [SimpleGamePolicy(ai ⇒ 1.0 for ai in 𝒜i) for 𝒜i in 𝒫.𝒜]
    return HierarchicalSoftmax(λ, k, π)
end

function solve(M::HierarchicalSoftmax, 𝒫)
    π = M.π
    for k in 1:M.k
        π = [softmax_response(𝒫, π, i, M.λ) for i in 𝒫.ℐ]
    end
    return π
end
```

算法24.9. 精确参数为λ、层级为k的层次softmax模型。默认情况下，它以对所有单个动作分配均匀概率的初始联合策略开始。

优化通常无法通过解析方法完成，但我们可以使用数值方法来进行这种优化。[12] 或者，我们可以使用贝叶斯方法
参数学习方法。[13]

12 J. R. Wright 和 K. Leyton-Brown，“超越均衡：预测正常形式博弈中的人类行为”，载于 *AAAI*，2016。

13 J. R. Wright 和 K. Leyton-Brown，“行为博弈论模型：参数分析的贝叶斯框架”，载于 *International Conference on*，2012年。

24.8 Fictitious Play

计算不同智能体策略的另一种方法是让它们在模拟中相互对战，并学习如何最佳应对。算法24.10提供了模拟循环的一种实现。在每次迭代中，我们评估各种策略以获得一个联合动作，然后智能体使用这个联合动作来更新它们的策略。我们可以使用多种方法根据观察到的联合动作来更新策略。本节重点讨论*fictitious play*，其中智能体使用对其他智能体所遵循策略的最大似然估计（如第16.1节所述）。每个智能体遵循自己的最佳应对，假设其他智能体根据这些估计行动。[14]

14 G. W. Brown，“通过虚拟博弈迭代求解博弈”，*Activ-*，第13卷第1期，第374-376页，1951年。J. Robinson，“求解博弈的迭代方法”，*Annals of Math-*，第296-301页，1951年。

为了计算最大似然估计，agent i 跟踪 agent j 执行动作 a^j 的次数，并将其存储在表 $N^i(j, a^j)$ 中。这些计数可以初始化为任何值，但通常初始化为 1 以创建初始的均匀不确定性。Agent i 计算其最佳响应，假设每个 agent j 遵循随机策略：

$$\pi^j(a^j) \propto N^i(j, a^j) \tag{24.9}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

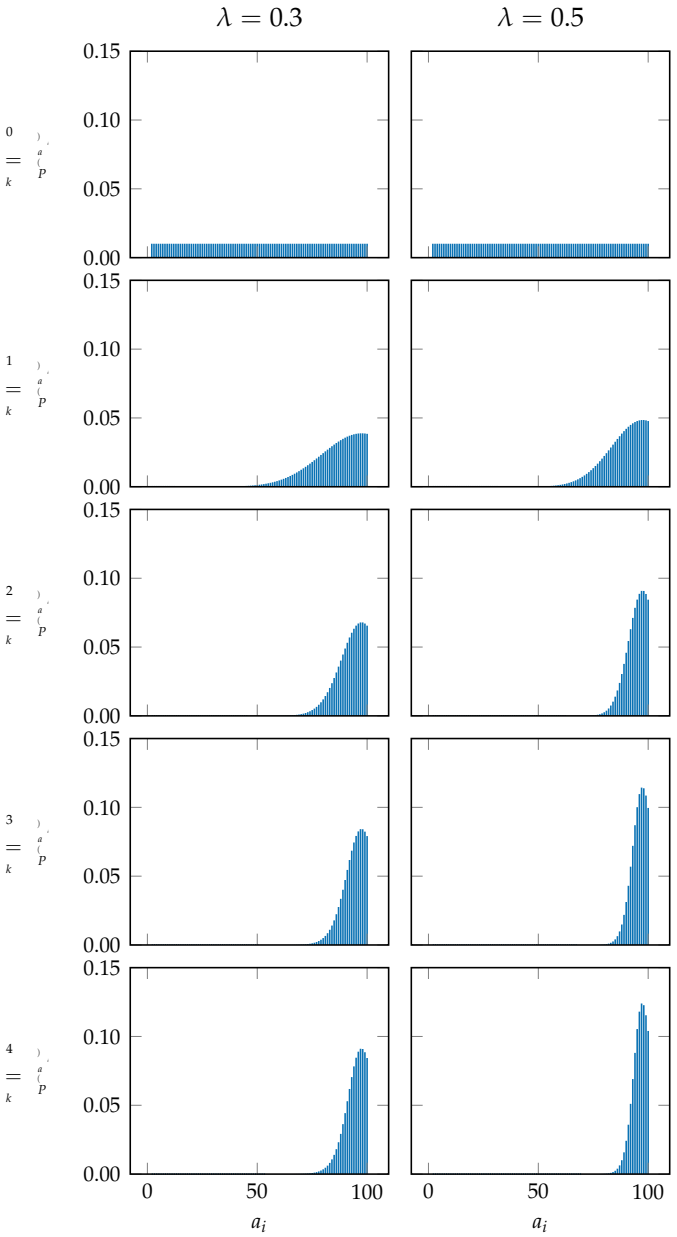


图24.1. 应用于旅行者困境（见附录F.12所述）的分层softmax模型，展示了不同理性深度k和精度参数λ下的情况。人们倾向于选择97到100美元之间的行动，尽管纳什均衡仅为2美元。

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

```
function simulate(𝒫::SimpleGame, π, k_max)
    for k = 1:k_max
        a = [πi() for πi in π]
        for πi in π
            update!(πi, a)
        end
    end
    return π
end
```

算法24.10。在简单游戏$\mathcal{P}$中模拟联合策略，迭代`k_max`次。联合策略`π`是一个策略向量，可以通过调用`update!(πi, a)`单独更新。

在每次迭代中，我们让每个智能体根据最佳响应行动，假设其他智能体采用这些基于随机计数的策略。然后，我们更新所采取动作的动作计数。算法24.11实现了这个简单的自适应过程。图24.2和图24.3展示了使用虚拟博弈时策略如何随时间演变。虚拟博弈并不保证收敛到纳什均衡。[15]

15 U. Berger 在“Brown’s Original Fictitious Play,” *Journal of Economic Theory*, 第135卷, 第1期, 第572–578页, 2007年中提供了简洁的背景介绍。

```
mutable struct FictitiousPlay
    𝒫  # simple game
    i  # agent index
    N  # array of action count dictionaries
    πi # current policy
end

function FictitiousPlay(𝒫::SimpleGame, i)
    N = [Dict(aj ⇒ 1 for aj in 𝒫.𝒜[j]) for j in 𝒫.ℐ]
    πi = SimpleGamePolicy(ai ⇒ 1.0 for ai in 𝒫.𝒜[i])
    return FictitiousPlay(𝒫, i, N, πi)
end

(πi::FictitiousPlay)() = πi.πi()

(πi::FictitiousPlay)(ai) = πi.πi(ai)

function update!(πi::FictitiousPlay, a)
    N, 𝒫, ℐ, i = πi.N, πi.𝒫, πi.𝒫.ℐ, πi.i
    for (j, aj) in enumerate(a)
        N[j][aj] += 1
    end
    p(j) = SimpleGamePolicy(aj ⇒ u/sum(values(N[j])) for (aj, u) in N[j])
    π = [p(j) for j in ℐ]
    πi.πi = best_response(𝒫, π, i)
end
```

算法24.11. 虚构游戏是一种简单的学习算法，适用于简单游戏$\mathcal{P}$中的代理`i`，它随时间维护并平均其他代理动作选择的`counts`，假设这是他们的随机策略。然后，它计算对该策略的最佳响应，并执行相应的效用最大化行动。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

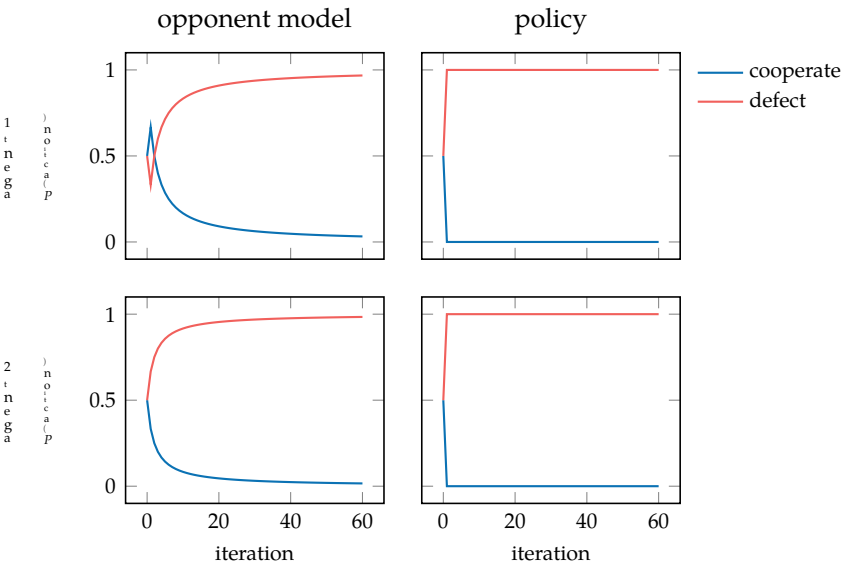


图24.2。在囚徒困境博弈中，两个虚拟博弈代理相互学习并适应对方。第一行展示了代理1在迭代过程中学习到的关于代理2的模型（左）和代理1的策略（右）。第二行遵循相同的模式，但针对代理2。为了说明学习行为的变化，每个代理关于另一个代理动作的模型的初始计数被随机赋予1到10之间的数值。

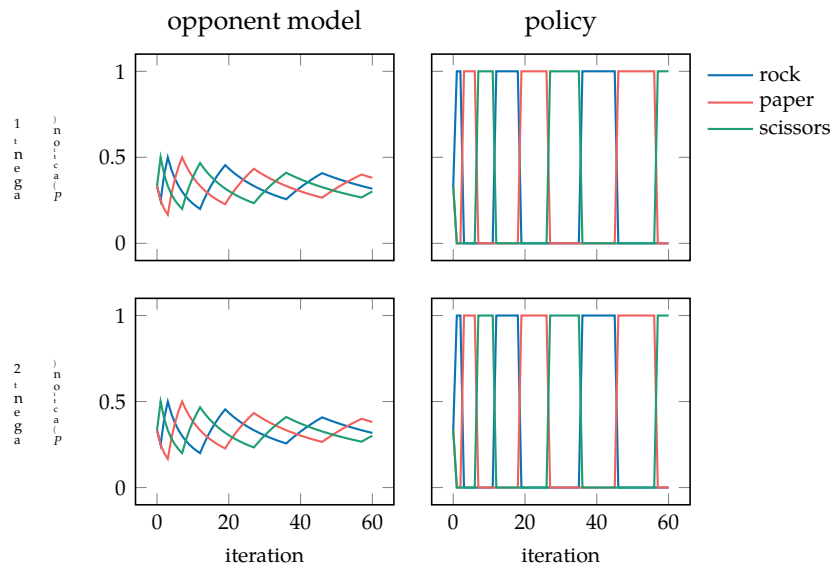


图24.3. 两个虚拟博弈代理在石头剪刀布游戏中相互学习并适应的可视化图。第一行展示了代理1随时间学习的关于代理2的模型（左侧）以及代理1的策略（右侧）。第二行遵循相同的模式，但针对代理2。为了展示学习行为的变化，每个代理关于另一个代理动作的初始模型计数被赋值为1到10之间的随机数。在这个零和博弈中，虚拟博弈代理趋近于其随机策略纳什均衡的收敛。

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

有许多虚构博弈的变体。一种变体称为*smooth fictitiousplay*，[16]，它使用期望效用加平滑函数来选择最佳反应，
例如策略的熵。另一种变体称为 *rational learning* 或 *Bayesian learning*。理性学习将虚拟对局模型扩展为对其他代理行为的任意信念，表述为贝叶斯先验。随后，根据联合行动的历史，使用贝叶斯规则更新这些信念。传统的虚拟对局可以被视为使用狄利克雷先验的理性学习（第4.2.2节）。

16 D. Fudenberg 和 D. Levine，“一致性与谨慎虚拟博弈”，*Journal of Economic Dy-*，第19卷，第5–7期，第1065–1089页，1995年。 *namics and Control*

24.9 Gradient Ascent

Gradient ascent (算法24.12)根据其效用相对于梯度的方向逐步调整代理的策略。在时刻t，代理i的梯度为

$$\frac{\partial U^i(\boldsymbol{\pi}_t)}{\partial \pi_t^i(a^i)} = \frac{\partial}{\partial \pi_t^i}\left(\sum_{\mathbf{a}} R^i(\mathbf{a}) \prod_j \pi_t^j(a^j)\right) = \sum_{\mathbf{a}^{-i}} R^i(a^i, \mathbf{a}^{-i}) \prod_{j \neq i} \pi_t^j(a^j) \tag{24.10}$$

然后我们可以使用标准的梯度上升法，其中 {v*} 保持不变。

$$\pi_{t+1}^i(a^i) = \pi_t^i(a^i) + \alpha_t^i \frac{\partial U^i(\boldsymbol{\pi}_t)}{\partial \pi_t^i(a^i)} \tag{24.11}$$

学习率 α_t^i.[17] 这个 π_{t+1}^i 可能需要投影回有效的概率分布概率分布，就像第23.4节中POMDP策略那样。

然而在实践中，智能体 i 只知道自己的策略 π_t^i，而不知道其他智能体的策略，这使得梯度的计算变得困难。但智能体确实能观察到所执行的联合动作 $\mathbf{a}_t$。尽管我们可以尝试像虚拟博弈中那样估计他们的策略，但一个简单的方法是假设其他智能体的策略是重复他们最近一次的动作。[18] 梯度

然后简化为

$$\frac{\partial U^i(\boldsymbol{\pi}_t)}{\partial \pi_t^i(a^i)} = R^i(a^i, \mathbf{a}^{-i}) \tag{24.12}$$

图24.4展示了这种用于简单石头剪刀布游戏的方法。

17 *infinitesimal gradient ascent* 方法采用 $\alpha_t^i = 1/\sqrt{t}$ 的逆平方根学习率。它之所以被称为无穷小，是因为 $\alpha_t^i \to 0$ 当 $t \to \infty$。我们在实现中使用这种学习率。S. Singh, M. K earns, and Y. Man- sour，“Nash Co nvergence of Gra- dient Dynamics in General-Sum Games”，载于 *Conference on Uncer-* *tainty in Artificial Intelligence (UAI)*，2000。

18 这种方法在*gener-*中使用。M. Zi *alized infinitesimal gradient ascent* *(GIGA)* *nkevich*，“在线凸规划与广义无穷小梯度上升”，载于 *International Conference on Machine Learning (ICML)*，2003年。M. Bowling在2005年的 *Advances in Neu-* *ral Information Pro-* *(NIPS)* 中提出了鼓励收敛的梯度更新规则的一种变体，“多智能体学习中的收敛与无悔”。

24.10 Summary

- 在简单的博弈中，多个智能体相互竞争以最大化预期奖励。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
mutable struct GradientAscent
    𝒫  # simple game
    i  # agent index
    t  # time step
    πi # current policy
end

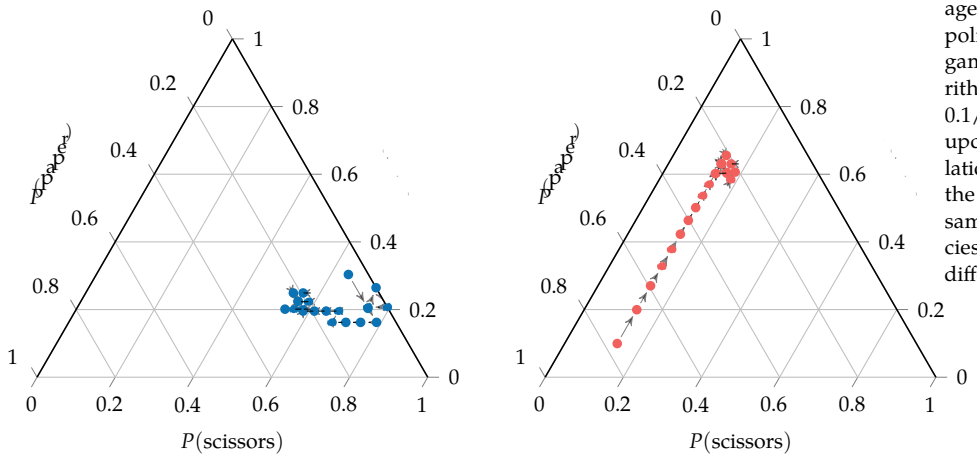
function GradientAscent(𝒫::SimpleGame, i)
    uniform() = SimpleGamePolicy(ai ⇒ 1.0 for ai in 𝒫.𝒜[i])
    return GradientAscent(𝒫, i, 1, uniform())
end

(πi::GradientAscent)() = πi.πi()

(πi::GradientAscent)(ai) = πi.πi(ai)

function update!(πi::GradientAscent, a)
    𝒫, 𝒯, 𝒜i, i, t = πi.𝒫, πi.𝒫.𝒯, πi.𝒫.𝒜[πi.i], πi.i, πi.t
    jointπ(ai) = [SimpleGamePolicy(j == i ? ai : a[j]) for j in 𝒯]
    r = [utility(𝒫, jointπ(ai), i) for ai in 𝒜i]
    π′ = [πi.πi(ai) for ai in 𝒜i]
    π = project_to_simplex(π′ + r / sqrt(t))
    πi.t = t + 1
    πi.πi = SimpleGamePolicy(ai ⇒ p for (ai, p) in zip(𝒜i, π))
end
```

算法 24.12。针对简单博弈 $\mathcal{P}$ 的智能体 i 的梯度上升实现。该算法按照梯度上升逐步更新其动作分布，以提高期望效用。使用算法 23.6 中的投影函数来确保所得策略仍然是有效的概率分布。



Figu关于第24.4节。两种梯度上升法agen随机初始化的ts poli石头剪刀布中的选择game. 我们使用一种算法的变体——rithm 24.12，学习率为 $0.1/\sqrt{t}$这里展示了20项策略updates。尽管不同的模拟lation 条迹将会收敛，因为the 步长趋于0，不同sam来自随机策略的示例cies 可能导致收敛到 {v*} diffe租金政策。

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

- 在多智能体环境中，最优性并非如此直接明了，因为从奖励规范中提取策略存在多种可能的解概念。
- 针对其他代理固定策略集，一个代理的最佳响应是指没有动机偏离的策略。

- 纳什均衡是一种联合策略，其中所有代理都遵循最佳响应。
- 相关均衡与纳什均衡相同，只是所有代理都遵循一个允许代理间相关性的联合行动分布。

- 迭代最佳响应可以通过迭代应用最佳响应来快速优化联合策略，但不存在收敛的一般保证。
- 分层 softmax 试图根据其理性深度和精确度对智能体进行建模，这些可以从过去的联合行动中学习得到。
- 虚拟博弈是一种学习算法，它利用对其他代理的最大似然动作模型来寻找最佳响应策略，并有可能收敛到纳什均衡。

- 梯度上升，随后投影到概率单纯形上，可用于学习策略。

24.11 Exercises

练习 24.1。给出一个包含两个参与者和无限个动作的博弈示例，使得纳什均衡不存在。

Solution: 假设每个智能体的动作空间由负实数组成，且其奖励等于其动作。由于不存在最大的负数，纳什均衡无法存在。

练习 24.2。给出一个包含两个智能体、两个动作以及两个涉及确定性策略的纳什均衡的博弈示例。

Solution: 这是一个示例。[19] 假设我们有两架飞机处于碰撞航线上，每架飞机的飞行员必须在爬升或下降之间做出选择以避免相撞。如果两名飞行员都选择相同的操作，那么就会发生碰撞，两名飞行员的效用均为-4。由于爬升比下降需要更多燃料，任何决定爬升的飞行员都会额外承受-1的惩罚。

19 这个例子来自 M. J. Kochenderfer, *Decision Making Under Uncertainty: Theory and Application*. MIT Press, 2015.

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

		agent 2: climb	agent 2: descend
agent 1	climbing	$-5, -5$	$-1, 0$
agent 1	descending	$0, -1$	$-4, -4$

练习24.3。给定一个平稳联合策略$\boldsymbol{\pi}$，它是一个视界为1的简单游戏的纳什均衡，证明它也是同一个简单游戏重复到任何有限或无限视界的纳什均衡。

Solution: 根据纳什均衡的定义，所有智能体 i 都在按照公式 (24.2) 对所有其他策略 $\pi^{i\prime} \neq \pi^i$ 做出最佳响应 π^i：

$$U^i(\pi^i, \boldsymbol{\pi}^{-i}) \geq U^i(\pi^{i\prime}, \boldsymbol{\pi}^{-i})$$

根据 U^i 的定义，我们有

$$U^i(\boldsymbol{\pi}) = \sum_{\mathbf{a} \in \mathbf{A}} R^i(\mathbf{a}) \prod_{j=1}^{k} \pi^j(a^j)$$

联合策略对所有智能体随时间保持不变。应用任何视界 n，以及任何折扣因子（对于 $n < \infty$，$\gamma = 1$；对于 $n \to \infty$，$\gamma < 1$）。智能体 i 在 n 步之后的效用为

$$\begin{aligned} U^{i,n}(\boldsymbol{\pi}) &= \sum_{t=1}^{n} \gamma^{t-1} \sum_{\mathbf{a} \in \mathbf{A}} R^i(\mathbf{a}) \prod_{j=1}^{k} \pi^j(a^j) \\ &= \sum_{\mathbf{a} \in \mathbf{A}} R^i(\mathbf{a}) \prod_{j=1}^{k} \pi^j(a^j) \sum_{t=1}^{n} \gamma^{t-1} \\ &= U^i(\boldsymbol{\pi}) \sum_{t=1}^{n} \gamma^{t-1} \\ &= U^i(\boldsymbol{\pi}) c \end{aligned}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

折扣因子变为常数乘数 $c > 0$。因此，在等式(24.2)两边同时乘以任意常数都会得到相同的不等式，从而完成证明：

$$\begin{aligned}
U^i(\pi^i, \boldsymbol{\pi}^{-i}) &\geq U^i(\pi^{i\prime}, \boldsymbol{\pi}^{-i}) \\
U^i(\pi^i, \boldsymbol{\pi}^{-i})c &\geq U^i(\pi^{i\prime}, \boldsymbol{\pi}^{-i})c \\
U^i(\pi^i, \boldsymbol{\pi}^{-i}) \sum_{t=1}^{n} \gamma^{t-1} &\geq U^i(\pi^{i\prime}, \boldsymbol{\pi}^{-i}) \sum_{t=1}^{n} \gamma^{t-1} \\
\sum_{t=1}^{n} \gamma^{t-1} U^i(\pi^i, \boldsymbol{\pi}^{-i}) &\geq \sum_{t=1}^{n} \gamma^{t-1} U^i(\pi^{i\prime}, \boldsymbol{\pi}^{-i}) \\
U^{i,n}(\pi^i, \boldsymbol{\pi}^{-i}) &\geq U^{i,n}(\pi^{i\prime}, \boldsymbol{\pi}^{-i})
\end{aligned}$$

练习24.4。证明纳什均衡是一种相关均衡。

Solution: 考虑任何不相关的联合策略 $\pi(\mathrm{a})$ 。对于任何 a 根 i:

$$\pi(\mathbf{a}) = \prod_{j=1}^{k} \pi^j(a^j) = \pi^i(a^i) \prod_{j \neq i} \pi^j(a^j) \tag{24.13}$$

只需证明该约束下的相关均衡恰好构成纳什均衡的定义。首先将方程(24.13)应用于相关均衡的定义。对于所有i，任何在π中具有非零概率[20]的a^i，以及所有$a^{i\prime}$：

[20] 即 $\sum_{\mathbf{a}^{-i}} \pi(a^i, \mathrm{a}^{-i}) > 0$。如果它为零，那么不等式显然变为 $0 \geq 0$ 成立。

$$\begin{aligned}
\sum_{\mathbf{a}^{-i}} R^i(a^i, \mathbf{a}^{-i}) \pi(a^i, \mathbf{a}^{-i}) &\geq \sum_{\mathbf{a}^{-i}} R^i(a^{i\prime}, \mathbf{a}^{-i}) \pi(a^i, \mathbf{a}^{-i}) \\
\sum_{\mathbf{a}^{-i}} R^i(a^i, \mathbf{a}^{-i}) \pi^i(a^i) \prod_{j \neq i} \pi^j(a^j) &\geq \sum_{\mathbf{a}^{-i}} R^i(a^{i\prime}, \mathbf{a}^{-i}) \pi^i(a^i) \prod_{j \neq i} \pi^j(a^j) \\
\sum_{\mathbf{a}^{-i}} R^i(a^i, \mathbf{a}^{-i}) \prod_{j \neq i} \pi^j(a^j) &\geq \sum_{\mathbf{a}^{-i}} R^i(a^{i\prime}, \mathbf{a}^{-i}) \prod_{j \neq i} \pi^j(a^j)
\end{aligned} \tag{24.14}$$

现在考虑效用的定义：

$$U^i(\pi^i, \boldsymbol{\pi}^{-i}) = \sum_{\mathbf{a}} R^i(a^i, \mathbf{a}^{-i}) \prod_{j=1}^{k} \pi^j(a^j) = \sum_{a^i} \pi^i(a^i) \left(\sum_{\mathbf{a}^{-i}} R^i(a^i, \mathbf{a}^{-i}) \prod_{j \neq i} \pi^j(a^j) \right)$$

接下来将方程（24.14）应用于括号内的项：

$$U^i(\pi^i, \boldsymbol{\pi}^{-i}) \geq \sum_{a^i} \pi^i(a^i) \left(\sum_{\mathbf{a}^{-i}} R^i(a^{i\prime}, \mathbf{a}^{-i}) \prod_{j \neq i} \pi^j(a^j) \right) = \left(\sum_{\mathbf{a}^{-i}} R^i(a^{i\prime}, \mathbf{a}^{-i}) \prod_{j \neq i} \pi^j(a^j) \right) \sum_{a^i} \pi^i(a^i) = \sum_{\mathbf{a}^{-i}} R^i(a^{i\prime}, \mathbf{a}^{-i}) \prod_{j \neq i} \pi^j(a^j)$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

这个方程对任何动作 $a^{i\prime}$ 都成立。因此，应用任何概率加权都会保持这个不等式右侧不变。考虑任何其他策略 $\pi^{i\prime}$ 作为加权：

$$U^i(\pi^i, \boldsymbol{\pi}^{-i}) \geq \sum_{a^i} \pi^{i\prime}(a^i) \sum_{\mathbf{a}^{-i}} R^i(a^i, \mathbf{a}^{-i}) \prod_{j \neq i} \pi^j(a^j) = U^i(\pi^{i\prime}, \boldsymbol{\pi}^{-i})$$

这个不等式是最佳响应的定义。它必须对所有代理人i成立，因此构成了纳什均衡的定义。总结来说，纳什均衡是一种特殊的相关均衡，它被限制在无关联的联合策略上。

练习 24.5。给出一个两人博弈的例子，每人有两个行动，使得其相关均衡无法表示为纳什均衡。

Solution: 考虑以下博弈：两个人想要约会，但在约会的类型（在这种情况下，是晚餐还是电影）上存在冲突的偏好：

		agent 2	
		dinner	movie
agent 1	dinner	2, 1	0, 0
	movie	0, 0	1, 2

存在一个随机纳什均衡。智能体1遵循$\pi^1(\text{晚餐}) = 2/3$和$\pi^1(\text{电影}) = 1/3$。智能体2遵循$\pi^2(\text{晚餐}) = 1/3$和$\pi^2(\text{电影}) = 2/3$。效用为：

$$U^1(\boldsymbol{\pi}) = \frac{2}{3} \cdot \frac{1}{3} \cdot 2 + \frac{2}{3} \cdot \frac{2}{3} \cdot 0 + \frac{1}{3} \cdot \frac{1}{3} \cdot 0 + \frac{1}{3} \cdot \frac{2}{3} \cdot 1 = \frac{2}{9} \cdot 2 + \frac{2}{9} \cdot 1 = \frac{2}{3}$$
$$U^2(\boldsymbol{\pi}) = \frac{2}{3} \cdot \frac{1}{3} \cdot 1 + \frac{2}{3} \cdot \frac{2}{3} \cdot 0 + \frac{1}{3} \cdot \frac{1}{3} \cdot 0 + \frac{1}{3} \cdot \frac{2}{3} \cdot 2 = \frac{2}{9} \cdot 1 + \frac{2}{9} \cdot 2 = \frac{2}{3}$$

然而，如果两个代理人通过公平的抛硬币来协调他们的行动$\pi(\text{电影，电影}) = \pi(\text{晚餐，晚餐}) = 0.5$，那么他们可以协调要么都去晚餐，要么都去看电影。效用为：

$$U^1(\boldsymbol{\pi}) = 0.5 \cdot 2 + 0.0 \cdot 0 + 0.0 \cdot 0 + 0.5 \cdot 1 = 0.5 \cdot 2 + 0.5 \cdot 1 = \frac{3}{2}$$
$$U^2(\boldsymbol{\pi}) = 0.5 \cdot 1 + 0.0 \cdot 0 + 0.0 \cdot 0 + 0.5 \cdot 2 = 0.5 \cdot 1 + 0.5 \cdot 2 = \frac{3}{2}$$

这对于纳什均衡是不可能的。直观上，在这个例子中，这是因为概率权重是独立地为每个参与者在每一行上分布的。相反，相关均衡可以针对特定的单元格（在这种情况下，具有更高的收益）。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

练习 24.6。诸如迭代最佳响应和虚拟博弈等算法并非在每种博弈中都能收敛。构建一个展示这种非收敛性的博弈。

Solution: 迭代最佳响应在石头剪刀布中发散。以下是随机初始化下前10次迭代的示例：

Iteration	Agent 1's Action	Agent 2's Action	Rewards
1	paper	rock	1.0, −1.0
2	paper	scissors	−1.0, 1.0
3	rock	scissors	1.0, −1.0
4	rock	paper	−1.0, 1.0
5	scissors	paper	1.0, −1.0
6	scissors	rock	−1.0, 1.0
7	paper	rock	1.0, −1.0
8	paper	scissors	−1.0, 1.0
9	rock	scissors	1.0, −1.0
10	rock	paper	−1.0, 1.0

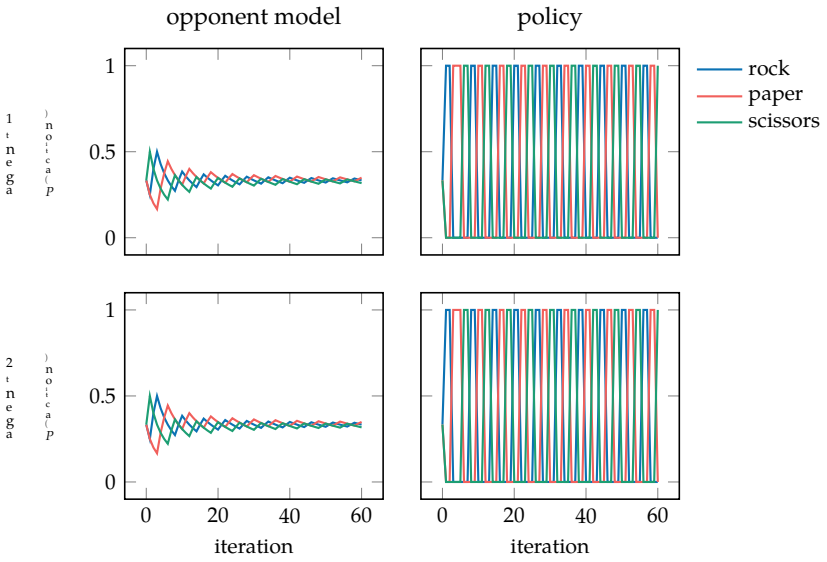
虚拟博弈在 *almost-rock-paper-scissors*:[21] 中也不会收敛

agent 1 \ agent 2	rock	paper	scissors
rock	0, 0	0, 1	1, 0
paper	1, 0	0, 0	0, 1
scissors	0, 1	1, 0	0, 0

21 这个游戏以及其他许多游戏在Y. Shoham和K. Leyton-Brown的著作中有更详细的讨论，*Multiagent Systems: Algo-rithmic, Game-Theoretic, and Logical Foundations*。剑桥大学出版社，2009年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

这里是虚拟博弈代理在此游戏中进行了60次迭代的示例：



练习 24.7. 在旅行者困境（附录 F.12）中，迭代最佳响应会收敛到什么？

Solution: 它收敛于 $2 的纳什均衡。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

25 Sequential Problems

本章将简单博弈扩展到具有多个状态的序贯情境。*Markov game* (*MG*) 可以被视为一个涉及多个智能体的马尔可夫决策过程，这些智能体各自拥有自己的奖励函数。[1] 在这种表述中，转移

依赖于联合动作，且所有智能体均追求自身奖励的最大化。我们将响应模型与纳什均衡解的概念从简单博弈推广至考虑状态转移模型的情形。本章最后一部分探讨基于学习的模型，其中智能体依据观察到的交互信息以及对奖励和转移函数的了解来调整其策略。

1 MG（也称为 *stochastic games*）最初是在20世纪50年代与MDP大约同一时期被研究的。L. S. Shapley，“Stochastic Games,” *Proceedings of the National Academy of Sciences*，第39卷，第10期，第1095–1100页，1953年。几十年后，它们被引入多智能体人工智能领域。M. L. Littman，“Markov Games as a Framework for Multi-Agent Reinforcement Learning,” 载于 *International Conference on Machine Learning* (*ICML*)，1994年

25.1 Markov Games

一个MG（算法25.1）将简单博弈扩展为包含共享状态$s \in \mathcal{S}$。在联合动作a下，从状态s转移到状态s'的可能性由转移分布$T(s' \mid s, \mathrm{a})$给出。每个智能体i根据其自身的奖励函数$R^i(s, \mathrm{a})$获得奖励，该函数现在也依赖于状态。示例25.1概述了如何将交通路由构建为一个MG。

```
struct MG
    γ  # discount factor
    ℐ  # agents
    𝒮  # state space
    𝒜  # joint action space
    T  # transition function
    R  # joint reward function
end
```

算法25.1. MG的数据结构。

考虑开车上班的通勤者。每辆车都有一个起始位置和一个目的地。每辆车可以选择几条可用道路中的任意一条到达目的地，但这些道路的行驶时间各不相同。某条道路上行驶的车辆越多，所有车辆的速度就越慢。

这个问题是一个MG。代理是驾驶汽车的通勤者，状态是所有汽车在道路上的位置，而动作对应于选择下一条道路的决定。状态转移根据它们的联合动作使所有汽车代理前进。负奖励与在道路上行驶所花费的时间成正比。

示例 25.1. 作为 MG 的交通路由。该问题无法使用像 MDP 这样的单一智能体模型来建模，因为我们不知道其他智能体的行为，只知道它们的奖励。我们可以尝试寻找均衡，或通过交互学习策略，类似于我们在简单博弈中所做的。

MG 中的联合策略 $\boldsymbol{\pi}$ 指定了给定当前状态下联合动作的概率分布。与 MDP 一样，我们将关注依赖于当前状态而非过去历史的策略，因为给定当前状态时，未来状态和奖励与历史条件独立。此外，我们将关注不依赖于时间的平稳策略。智能体 i 在状态 s 下选择动作 a 的概率由 $\pi^i(a \mid s)$ 给出。我们经常使用 $\boldsymbol{\pi}(s)$ 来表示联合动作上的分布。

从智能体i的角度来看，联合策略$\boldsymbol{\pi}$的效用可以通过第7.2节中为MDP引入的策略评估变体来计算。在遵循联合策略$\boldsymbol{\pi}$时，智能体i从状态s获得的奖励是

$$R^i(s, \boldsymbol{\pi}(s)) = \sum_{\mathbf{a}} R^i(s, \mathbf{a}) \prod_{j \in \mathcal{I}} \pi^j(a^j \mid s) \tag{25.1}$$

在遵循$\boldsymbol{\pi}$时，从状态s转移到s'的概率是

$$T(s' \mid s, \boldsymbol{\pi}(s)) = \sum_{\mathbf{a}} T(s' \mid s, \mathbf{a}) \prod_{j \in \mathcal{I}} \pi^j(a^j \mid s) \tag{25.2}$$

在无限期折现博弈中，代理i从状态s获得的效用为

$$U^{\boldsymbol{\pi},i}(s) = R^i(s, \boldsymbol{\pi}(s)) + \gamma \sum_{s'} T(s' \mid s, \boldsymbol{\pi}(s)) U^{\boldsymbol{\pi},i}(s') \tag{25.3}$$

可以精确求解（算法25.2）。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct MGPolicy
    p # dictionary mapping states to simple game policies
    MGPolicy(p::Base.Generator) = new(Dict(p))
end

(πi::MGPolicy)(s, ai) = πi.p[s](ai)
(πi::SimpleGamePolicy)(s, ai) = πi(ai)

probability(𝒫::MG, s, π, a) = prod(πj(s, aj) for (πj, aj) in zip(π, a))
reward(𝒫::MG, s, π, i) =
    sum(𝒫.R(s,a)[i]*probability(𝒫,s,π,a) for a in joint(𝒫.𝒜))
transition(𝒫::MG, s, π, s′) =
    sum(𝒫.T(s,a,s′)*probability(𝒫,s,π,a) for a in joint(𝒫.𝒜))

function policy_evaluation(𝒫::MG, π, i)
    𝒮, 𝒜, R, T, γ = 𝒫.𝒮, 𝒫.𝒜, 𝒫.R, 𝒫.T, 𝒫.γ
    p(s,a) = prod(πj(s, aj) for (πj, aj) in zip(π, a))
    R′ = [sum(R(s,a)[i]*p(s,a) for a in joint(𝒜)) for s in 𝒮]
    T′ = [sum(T(s,a,s′)*p(s,a) for a in joint(𝒜)) for s in 𝒮, s′ in 𝒮]
    return (I - γ*T′)\R′
end
```

算法25.2。MG策略是从状态到简单博弈策略的映射，这一概念在上一章中已介绍。我们可以通过传入一个生成器来构建它，以构造字典。一个策略（无论是MG还是简单博弈）从状态`s`采取行动`ai`的概率是`πi(s, ai)`。还提供了计算$R^i(s, \boldsymbol{\pi}(s))$以及$T(s' \mid s, \boldsymbol{\pi}(s))$的函数。策略评估函数将计算一个代表$U^{\boldsymbol{\pi},i}$的向量。

25.2 Response Models

我们可以将上一章中介绍的反应模型推广到MG。这样做需要考虑状态转移模型。

25.2.1 Best Response

*response policy*对于智能体i是一个策略π^i，该策略在其他智能体$\boldsymbol{\pi}^{-i}$的固定策略下最大化期望效用。如果其他智能体的策略是固定的，那么问题就简化为一个MDP。这个MDP具有状态空间$\mathcal{S}$和动作空间$\mathcal{A}^i$。我们可以如下定义转移函数和奖励函数：

$$T'(s' \mid s, a^i) = T(s' \mid s, a^i, \boldsymbol{\pi}^{-i}(s)) \tag{25.4}$$

$$R'(s, a^i) = R^i(s, a^i, \boldsymbol{\pi}^{-i}(s)) \tag{25.5}$$

因为这是智能体 i 的最佳响应，MDP 只使用奖励 R^i。解决这个 MDP 会得到智能体 i 的最佳响应策略。算法 25.3 提供了对此的实现。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
function best_response(𝒫::MG, π, i)
    𝒮, 𝒜, R, T, γ = 𝒫.𝒮, 𝒫.𝒜, 𝒫.R, 𝒫.T, 𝒫.γ
    T′(s,ai,s′) = transition(𝒫, s, joint(π, SimpleGamePolicy(ai), i), s′)
    R′(s,ai) = reward(𝒫, s, joint(π, SimpleGamePolicy(ai), i), i)
    πi = solve(MDP(γ, 𝒮, 𝒜[i], T′, R′))
    return MGPolicy(s ⇒ SimpleGamePolicy(πi(s)) for s in 𝒮)
end
```

算法25.3。对于一个MG $\mathcal{P}$，我们可以计算代理i的确定性最佳响应策略，假设其他代理正在使用π中的策略。我们可以使用第7章中的一种方法精确求解该MDP。

25.2.2 *Softmax Response*

与上一章的做法类似，我们可以定义一个 *softmax response policy*，它在每个状态下为其他智能体的策略分配一个随机响应。正如我们在构建确定性最佳响应策略时所做的那样，我们求解一个MDP，其中具有固定策略 π^{-i} 的智能体被纳入环境中。然后，我们使用一步前瞻提取动作价值函数 $Q(s、a)$。softmax响应为

$$\pi^i(a^i \mid s) \propto \exp(\lambda Q(s, a^i)) \tag{25.6}$$

使用精度参数 $\lambda \geq 0$。算法25.4提供了实现。这种方法可用于生成层次softmax解决方案（第24.7节）。实际上，我们可以直接使用算法24.9。

```
function softmax_response(𝒫::MG, π, i, λ)
    𝒮, 𝒜, R, T, γ = 𝒫.𝒮, 𝒫.𝒜, 𝒫.R, 𝒫.T, 𝒫.γ
    T′(s,ai,s′) = transition(𝒫, s, joint(π, SimpleGamePolicy(ai), i), s′)
    R′(s,ai) = reward(𝒫, s, joint(π, SimpleGamePolicy(ai), i), i)
    mdp = MDP(γ, 𝒮, joint(𝒜), T′, R′)
    πi = solve(mdp)
    Q(s,a) = lookahead(mdp, πi.U, s, a)
    p(s) = SimpleGamePolicy(a ⇒ exp(λ*Q(s,a)) for a in 𝒜[i])
    return MGPolicy(s ⇒ p(s) for s in 𝒮)
end
```

算法25.4。智能体i在精度参数λ下对联合策略π的softmax响应。

25.3 *Nash Equilibrium*

纳什均衡概念可以推广到马尔可夫博弈。[2]与简单博弈一样，所有智能体都对彼此采取最佳响应，并且没有偏离的动机。所有具有折扣无限视界的有限MGs都有一个纳什均衡。[3]

2 因为我们假设策略是*stationary*，即它们不随时间变化，所以这里涉及的纳什均衡是*stationary Markov perfect equilibria*

3 A. M. Fink，“随机n人博弈中的均衡”，*Journal of Science of the Hiroshima University*，第28卷，第1期，第89–93页，1964年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们可以通过求解一个类似于我们在简单博弈背景下求解的非线性优化问题来找到一个纳什均衡。该问题最小化前瞻效用偏差之和，并约束策略为有效分布：

$$\begin{aligned} \underset{\boldsymbol{\pi}, U}{\text{minimize}} \quad & \sum_{i \in \mathcal{I}} \sum_{s} \left(U^i(s) - Q^i(s, \boldsymbol{\pi}(s)) \right) \\ \text{subject to} \quad & U^i(s) \geq Q^i(s, a^i, \boldsymbol{\pi}^{-i}(s)) \text{ for all } i, s, a^i \\ & \sum_{a^i} \pi^i(a^i \mid s) = 1 \text{ for all } i, s \\ & \pi^i(a^i \mid s) \geq 0 \text{ for all } i, s, a^i \end{aligned} \tag{25.7}$$

其中

$$Q^i(s, \boldsymbol{\pi}(s)) = R^i(s, \boldsymbol{\pi}(s)) + \gamma \sum_{s'} T(s' \mid s, \boldsymbol{\pi}(s)) U^i(s') \tag{25.8}$$

该非线性优化问题在算法25.5.[4]中实现并求解。

4 J. A. Filar, T. A. Schultz, F. Thuijsman 和 O. Vrieze，“随机博弈中的非线性规划与平稳均衡”，*Mathematical Programming*，第 50 卷，第 1–3 期，第 227–237 页，1991 年。

25.4 Fictitious Play

如同我们在简单游戏的背景下所做的那样，我们可以采取基于学习的方法，通过在模拟中运行代理来达成联合策略。算法25.6将前章引入的模拟循环推广，以处理状态转移。在模拟中运行的各种策略根据状态转移及各代理采取的行动进行自我更新。

更新策略的一种方法是使用上一章中 *fictitious play* (算法 25.7) 的推广，[5]这涉及维护

5 W. Uther 和 M. Veloso，“对抗性强化学习”，卡内基梅隆大学，技术报告 CMU-CS-03-107，1997年。M. Bowling 和 M. Veloso，“多智能体强化学习的随机博弈论分析”，卡内基梅隆大学，技术报告 CMU-CS-00-165，2000年。

对其他智能体策略的最大似然模型。该最大似然模型除了跟踪每个智能体所采取的动作外，还跟踪状态。我们跟踪智能体 j 在状态 s 下采取动作 a^j 的次数，将其存储在表 $N(j、a^j、s)$ 中，通常初始化为1。然后，假设每个智能体 j 遵循状态依赖的随机策略，我们可以计算最佳响应：

$$\pi^j(a^j \mid s) \propto N(j, a^j, s) \tag{25.9}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
function tensorform(𝒫::MG)
    ℐ, 𝒮, 𝒜, R, T = 𝒫.ℐ, 𝒫.𝒮, 𝒫.𝒜, 𝒫.R, 𝒫.T
    ℐ′ = eachindex(ℐ)
    𝒮′ = eachindex(𝒮)
    𝒜′ = [eachindex(𝒜[i]) for i in ℐ]
    R′ = [R(s,a) for s in 𝒮, a in joint(𝒜)]
    T′ = [T(s,a,s′) for s in 𝒮, a in joint(𝒜), s′ in 𝒮]
    return ℐ′, 𝒮′, 𝒜′, R′, T′
end

function solve(M::NashEquilibrium, 𝒫::MG)
    ℐ, 𝒮, 𝒜, R, T = tensorform(𝒫)
    𝒮′, 𝒜′, γ = 𝒫.𝒮, 𝒫.𝒜, 𝒫.γ
    model = Model(Ipopt.Optimizer)
    @variable(model, U[ℐ, 𝒮])
    @variable(model, π[i=ℐ, 𝒮, ai=𝒜[i]] ≥ 0)
    @NLobjective(model, Min,
        sum(U[i,s] - sum(prod(π[j,s,a[j]] for j in ℐ)
            * (R[s,y][i] + γ*sum(T[s,y,s′]*U[i,s′] for s′ in 𝒮))
            for (y,a) in enumerate(joint(𝒜))) for i in ℐ, s in 𝒮))
    @NLconstraint(model, [i=ℐ, s=𝒮, ai=𝒜[i]],
        U[i,s] ≥ sum(
            prod(j==i ? (a[j]==ai ? 1.0 : 0.0) : π[j,s,a[j]] for j in ℐ)
            * (R[s,y][i] + γ*sum(T[s,y,s′]*U[i,s′] for s′ in 𝒮))
            for (y,a) in enumerate(joint(𝒜))))
    @constraint(model, [i=ℐ, s=𝒮], sum(π[i,s,ai] for ai in 𝒜[i]) == 1)
    optimize!(model)
    π′ = value.(π)
    πi′(i,s) = SimpleGamePolicy(𝒜′[i][ai] ⇒ π′[i,s,ai] for ai in 𝒜[i])
    πi′(i) = MGPolicy(𝒮′[s] ⇒ πi′(i,s) for s in 𝒮)
    return [πi′(i) for i in ℐ]
end
```

算法25.5。该非线性程序计算MG $\mathcal{P}$ 的纳什均衡。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
function randstep(𝒫::MG, s, a)
    s′ = rand(SetCategorical(𝒫.𝒮, [𝒫.T(s, a, s′) for s′ in 𝒫.𝒮]))
    r = 𝒫.R(s,a)
    return s′, r
end

function simulate(𝒫::MG, π, k_max, b)
    s = rand(b)
    for k = 1:k_max
        a = Tuple(πi(s)() for πi in π)
        s′, r = randstep(𝒫, s, a)
        for πi in π
            update!(πi, s, a, s′)
        end
        s = s′
    end
    return π
end
```

算法25.6. 在MGs中采取随机步骤并运行完整模拟的函数。simulate函数将从`b`中随机抽取的状态开始，模拟联合策略`π`进行`k_max`步。

在状态s中观察到联合动作a后，我们更新

$$N(j, a^j, s) \leftarrow N(j, a^j, s) + 1 \tag{25.10}$$

对于每个代理j。

随着其他智能体动作分布的变化，我们必须更新效用。由于状态依赖性，马尔可夫博弈中的效用计算比简单博弈要困难得多。如第25.2.1节所述，任何固定其他智能体策略的分配$\boldsymbol{\pi}^{-i}$都会诱导一个MDP。在虚拟博弈中，$\boldsymbol{\pi}^{-i}$由方程（25.9）确定。无需在每次更新时求解MDP，通常的做法是定期应用更新，这是一种源自异步值迭代的策略。虚拟博弈的一个例子见例25.2。

我们对状态s的策略$\pi^i(s)$是从给定的对手模型$\boldsymbol{\pi}^{-i}$和计算出的效用U^i推导出来的。然后我们选择最佳响应：

$$\arg\max_a Q^i(s, a, \boldsymbol{\pi}^{-i}) \tag{25.11}$$

在实现中，我们利用了MG策略的每个状态都是一个简单游戏策略这一特性，其奖励对应于相应的{v*}。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
mutable struct MGFictitiousPlay
    𝒫  # Markov game
    i  # agent index
    Qi # state-action value estimates
    Ni # state-action counts
end

function MGFictitiousPlay(𝒫::MG, i)
    ℐ, 𝒮, 𝒜, R = 𝒫.ℐ, 𝒫.𝒮, 𝒫.𝒜, 𝒫.R
    Qi = Dict((s, a) ⇒ R(s, a)[i] for s in 𝒮 for a in joint(𝒜))
    Ni = Dict((j, s, aj) ⇒ 1.0 for j in ℐ for s in 𝒮 for aj in 𝒜[j])
    return MGFictitiousPlay(𝒫, i, Qi, Ni)
end

function (πi::MGFictitiousPlay)(s)
    𝒫, i, Qi = πi.𝒫, πi.i, πi.Qi
    ℐ, 𝒮, 𝒜, T, R, γ = 𝒫.ℐ, 𝒫.𝒮, 𝒫.𝒜, 𝒫.T, 𝒫.R, 𝒫.γ
    πi′(i,s) = SimpleGamePolicy(ai ⇒ πi.Ni[i,s,ai] for ai in 𝒜[i])
    πi′(i) = MGPolicy(s ⇒ πi′(i,s) for s in 𝒮)
    π = [πi′(i) for i in ℐ]
    U(s,π) = sum(πi.Qi[s,a]*probability(𝒫,s,π,a) for a in joint(𝒜))
    Q(s,π) = reward(𝒫,s,π,i) + γ*sum(transition(𝒫,s,π,s′)*U(s′,π)
                                     for s′ in 𝒮)
    Q(ai) = Q(s, joint(π, SimpleGamePolicy(ai), i))
    ai = argmax(Q, 𝒫.𝒜[πi.i])
    return SimpleGamePolicy(ai)
end

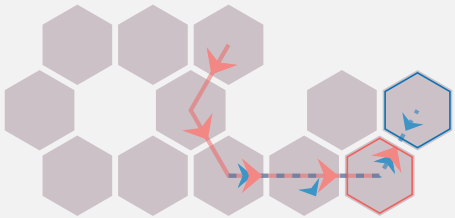
function update!(πi::MGFictitiousPlay, s, a, s′)
    𝒫, i, Qi = πi.𝒫, πi.i, πi.Qi
    ℐ, 𝒮, 𝒜, T, R, γ = 𝒫.ℐ, 𝒫.𝒮, 𝒫.𝒜, 𝒫.T, 𝒫.R, 𝒫.γ
    for (j,aj) in enumerate(a)
        πi.Ni[j,s,aj] += 1
    end
    πi′(i,s) = SimpleGamePolicy(ai ⇒ πi.Ni[i,s,ai] for ai in 𝒜[i])
    πi′(i) = MGPolicy(s ⇒ πi′(i,s) for s in 𝒮)
    π = [πi′(i) for i in ℐ]
    U(π,s) = sum(πi.Qi[s,a]*probability(𝒫,s,π,a) for a in joint(𝒜))
    Q(s,a) = R(s,a)[i] + γ*sum(T(s,a,s′)*U(π,s′) for s′ in 𝒮)
    for a in joint(𝒜)
        πi.Qi[s,a] = Q(s,a)
    end
end
```

算法25.7. 在MG $\mathcal{P}$中，对代理i进行虚拟对弈，该过程维护其他代理随时间在每个状态下动作选择的计数Ni并取平均值，假设这是它们的随机策略。然后，它计算对该策略的最佳响应，并执行相应的最大化效用的动作。

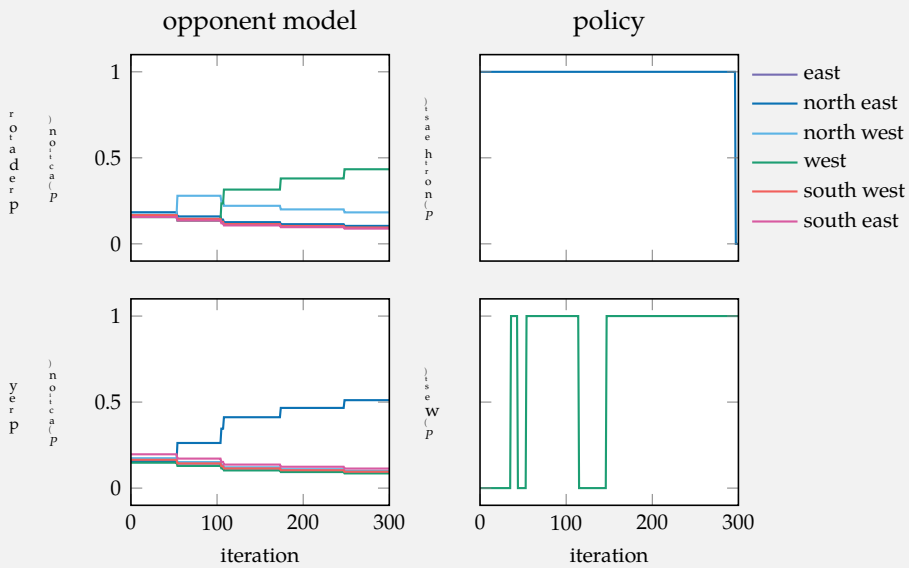
© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

捕食者-猎物六边形世界MG（附录F.13）有一个捕食者（红色）和一个猎物（蓝色）。如果捕食者抓住猎物，它会获得10的奖励，而猎物则会获得−10的奖励。否则，双方代理均获得−1的奖励。代理们同时移动。我们应用虚构 play，并每10步重置至初始状态。

我们观察到捕食者学会追逐猎物，而猎物学会逃跑。有趣的是，捕食者还了解到猎物会跑到东角并等待。猎物了解到，如果它在这个角落等待，当捕食者向它扑来时，它可以立即逃离捕食者。在这里，当捕食者向东北移动时，猎物通过向西移动来躲避捕食者。



以下是针对捕食者和猎物（两者都包括捕食者和猎物的六边形位置）高亮状态的学习对手模型的图表：



示例25.2。在捕食者-猎物六边形世界问题中的虚拟博弈。在初始化策略时引入了随机性，以更好地展示学习趋势。

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

25.5 Gradient Ascent

我们可以使用*gradient ascent*（算法25.8）以类似于前一章简单博弈中的方式来学习策略。现在必须考虑状态，并且需要学习动作价值函数。在每个时间步t，所有智能体在状态s_t下执行联合动作a_t。与简单博弈中的梯度上升一样，智能体i假设智能体的策略$\boldsymbol{\pi}_t^{-i}$是观察到的动作a_t^{-i}。梯度为

$$\frac{\partial U^{\boldsymbol{\pi}_t,i}(s_t)}{\partial \pi_t^i(a^i \mid s_t)} = \frac{\partial}{\partial \pi^i(a^i \mid s_t)} \left(\sum_{\mathbf{a}} \prod_j \pi^j(a^j \mid s_t) Q^{\boldsymbol{\pi}_t,i}(s_t, \mathbf{a}_t) \right) \tag{25.12}$$

$$= Q^{\boldsymbol{\pi}_t,i}(s_t, a^i, \mathbf{a}_t^{-i}) \tag{25.13}$$

梯度步骤遵循与前一章类似的模式，只是包含了状态s并使用了预期效用估计Q_t^i：

$$\pi_{t+1}^i(a^i \mid s_t) = \pi_t^i(a^i \mid s_t) + \alpha_t^i Q^i(s_t, a^i, \mathbf{a}^{-i}) \tag{25.14}$$

同样，此更新可能需要投影以确保在s_t处的策略π_{t+1}^i是有效的概率分布。

与上一节中的虚拟博弈一样，我们必须估计Q_t^i。我们可以使用Q学习：

$$Q_{t+1}^i(s_t, \mathbf{a}_t) = Q_t^i(s_t, \mathbf{a}_t) + \alpha_t \left(R^i(s_t, \mathbf{a}_t) + \gamma \max_{a^{i\prime}} Q_t^i(s_{t+1}, a^{i\prime}, \mathbf{a}_t^{-i}) - Q_t^i(s_t, \mathbf{a}_t) \right) \tag{25.15}$$

我们可以使用逆平方根学习率 $\alpha_t = 1/\sqrt{t}$。探索也是必要的。我们可以使用 ϵ-贪心策略，也许还可以使用 $\epsilon_t = 1/\sqrt{t}$。

25.6 Nash Q-Learning

另一种基于学习的方法是 *Nash Q-learning*（算法 25.9），它借鉴了 Q-学习（第17.2节）的灵感。[6] 该方法维护一个对动作价值函数的估计，随着智能体对彼此变化策略的反应而调整。在更新动作价值函数的过程中，它计算纳什均衡来模拟其他智能体的行为。

6 J. Hu 和 M. P. Wellman，“一般和随机博弈的纳什Q学习”，*Journal of Machine Learning Research*，第4卷，第1039–1069页，2003年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
mutable struct MGGradientAscent
    𝒫  # Markov game
    i  # agent index
    t  # time step
    Qi # state-action value estimates
    πi # current policy
end

function MGGradientAscent(𝒫::MG, i)
    ℐ, 𝒮, 𝒜 = 𝒫.ℐ, 𝒫.𝒮, 𝒫.𝒜
    Qi = Dict((s, a) ⇒ 0.0 for s in 𝒮, a in joint(𝒜))
    uniform() = Dict(s ⇒ SimpleGamePolicy(ai ⇒ 1.0 for ai in 𝒫.𝒜[i])
                     for s in 𝒮)
    return MGGradientAscent(𝒫, i, 1, Qi, uniform())
end

function (πi::MGGradientAscent)(s)
    𝒜i, t = πi.𝒫.𝒜[πi.i], πi.t
    ϵ = 1 / sqrt(t)
    πi′(ai) = ϵ/length(𝒜i) + (1-ϵ)*πi.πi[s](ai)
    return SimpleGamePolicy(ai ⇒ πi′(ai) for ai in 𝒜i)
end

function update!(πi::MGGradientAscent, s, a, s′)
    𝒫, i, t, Qi = πi.𝒫, πi.i, πi.t, πi.Qi
    ℐ, 𝒮, 𝒜i, R, γ = 𝒫.ℐ, 𝒫.𝒮, 𝒫.𝒜[πi.i], 𝒫.R, 𝒫.γ
    jointπ(ai) = Tuple(j == i ? ai : a[j] for j in ℐ)
    α = 1 / sqrt(t)
    Qmax = maximum(Qi[s′, jointπ(ai)] for ai in 𝒜i)
    πi.Qi[s, a] += α * (R(s, a)[i] + γ * Qmax - Qi[s, a])
    u = [Qi[s, jointπ(ai)] for ai in 𝒜i]
    π′ = [πi.πi[s](ai) for ai in 𝒜i]
    π = project_to_simplex(π′ + u / sqrt(t))
    πi.t = t + 1
    πi.πi[s] = SimpleGamePolicy(ai ⇒ p for (ai, p) in zip(𝒜i, π))
end
```

算法 25.8. 一个 MG $\mathcal{P}$ 中智能体 i 的梯度上升。该算法根据梯度上升，在所访问状态处逐步更新其动作分布，以提高期望效用。算法 23.6 中的投影函数用于确保所得策略仍然是有效的概率分布。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

遵循Nash Q-学习的智能体维护一个联合动作价值函数Q(s, a)的估计。每当状态转换后，该动作价值函数会根据由该价值函数构建的简单博弈计算出的纳什均衡进行更新。在遵循联合动作a从s转移到s'之后，我们构建一个具有相同数量智能体和相同联合动作空间的简单博弈，但奖励函数等于s'的估计值，使得R(a′) = Q(s', a′)。智能体计算关于下一个动作a′的纳什均衡策略$\boldsymbol{\pi}'$。根据推导出的策略，后继状态的预期效用是

$$\mathbf{U}(s') = \sum_{\mathbf{a}'} \mathbf{Q}(s', \mathbf{a}') \prod_{j \in \mathcal{I}} \pi^{j\prime}(a^{j\prime}) \tag{25.16}$$

然后，代理更新其价值函数：

$$\mathbf{Q}(s, \mathbf{a}) \leftarrow \mathbf{Q}(s, \mathbf{a}) + \alpha\big(\mathbf{R}(s, \mathbf{a}) + \gamma \mathbf{U}(s') - \mathbf{Q}(s, \mathbf{a})\big) \tag{25.17}$$

其中学习率α通常是状态-动作计数$\alpha = 1/\sqrt{N(s、\ \mathrm{a})}$的函数。

与常规的 Q-学习一样，我们需要采用探索策略，以确保所有状态和动作都被足够频繁地尝试。在算法 25.9 中，智能体遵循 ϵ-贪心策略。以概率 $\epsilon = 1/\sum_{\mathbf{a}}(N(s，\ 即))$，它均匀随机地选择一个动作。否则，它将使用纳什均衡的结果。

25.7 *Summary*

- 多智能体博弈（MGs）是将马尔可夫决策过程（MDPs）扩展到多个智能体的结果，或者是将简单博弈扩展到序列问题的结果。在这类问题中，多个智能体相互竞争，并随时间各自获得奖励。
- 纳什均衡可针对马尔可夫博弈进行表述，但此时必须考虑所有状态下所有智能体的所有动作。
- 寻找纳什均衡的问题可以表述为一个非线性优化问题。
- 我们可以通过使用已知的转移函数并结合对动作价值的估计，将虚构博弈推广到MGs。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
mutable struct NashQLearning
    𝒫 # Markov game
    i # agent index
    Q # state-action value estimates
    N # history of actions performed
end

function NashQLearning(𝒫::MG, i)
    ℐ, 𝒮, 𝒜 = 𝒫.ℐ, 𝒫.𝒮, 𝒫.𝒜
    Q = Dict((j, s, a) ⇒ 0.0 for j in ℐ, s in 𝒮, a in joint(𝒜))
    N = Dict((s, a) ⇒ 1.0 for s in 𝒮, a in joint(𝒜))
    return NashQLearning(𝒫, i, Q, N)
end

function (πi::NashQLearning)(s)
    𝒫, i, Q, N = πi.𝒫, πi.i, πi.Q, πi.N
    ℐ, 𝒮, 𝒜, 𝒜i, γ = 𝒫.ℐ, 𝒫.𝒮, 𝒫.𝒜, 𝒫.𝒜[πi.i], 𝒫.γ
    M = NashEquilibrium()
    𝒢 = SimpleGame(γ, ℐ, 𝒜, a -> [Q[j, s, a] for j in ℐ])
    π = solve(M, 𝒢)
    ϵ = 1 / sum(N[s, a] for a in joint(𝒜))
    πi′(ai) = ϵ/length(𝒜i) + (1-ϵ)*π[i](ai)
    return SimpleGamePolicy(ai ⇒ πi′(ai) for ai in 𝒜i)
end

function update!(πi::NashQLearning, s, a, s′)
    𝒫, ℐ, 𝒮, 𝒜, R, γ = πi.𝒫, πi.𝒫.ℐ, πi.𝒫.𝒮, πi.𝒫.𝒜, πi.𝒫.R, πi.𝒫.γ
    i, Q, N = πi.i, πi.Q, πi.N
    M = NashEquilibrium()
    𝒢 = SimpleGame(γ, ℐ, 𝒜, a′ -> [Q[j, s′, a′] for j in ℐ])
    π = solve(M, 𝒢)
    πi.N[s, a] += 1
    α = 1 / sqrt(N[s, a])
    for j in ℐ
        πi.Q[j,s,a] += α*(R(s,a)[j] + γ*utility(𝒢,π,j) - Q[j,s,a])
    end
end
```

算法25.9. 多智能体博弈中单个智能体的纳什Q-学习。该算法通过联合动作Q-学习，为所有智能体学习状态-动作价值函数。利用Q构建一个简单博弈，并采用算法24.5计算纳什均衡。随后，利用该均衡更新价值函数。此实现还采用了一个可变学习率，它与状态-联合动作对访问次数成比例，该次数存储在N中。此外，还使用ϵ-贪心探索策略，确保所有状态和动作都能被探索。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

- 梯度上升方法迭代地改进随机策略，且无需假设模型。

- 纳什Q-学习将传统的Q-学习适应于多智能体问题，并涉及求解一个由其他参与者模型构建的简单博弈的纳什均衡。

25.8 Exercises

练习25.1。证明马尔可夫博弈（MGs）是马尔可夫决策过程（MDPs）和简单博弈的扩展。通过将MDP表述为MG，并将简单博弈表述为MG来证明这一点。

Solution: MG 泛化了简单博弈。对于任何具有 $\mathcal{I}$、$\mathcal{A}$ 和 R 的简单博弈，我们可以通过仅有一个自循环状态来构造一个 MG。换句话说，这个 MG 具有 $\mathcal{S} = \{s^1\}$、$T(s^1 \mid s^1$、a$) = 1$，并且 R$(s^1,$ a$)$ = R(a)。

MG 泛化了 MDP。对于任何具有 $\mathcal{S}$、$\mathcal{A}$、T 和 R 的 MDP，我们只需将该智能体指定为这个单一智能体，即可构造出一个 MG。换句话说，这个 MG 具有 $\mathcal{I} = \{1\}$、$\mathcal{A}^1 = \mathcal{A}$、$T(s' \mid s$、a$) = T(s' \mid s'$、$a)$，以及 R$(s, a) = R(s$、$a)$。

练习25.2。对于代理i，在其他代理$\boldsymbol{\pi}^{-i}$的固定策略下，是否存在一种随机最佳响应能比确定性最佳响应产生更大的效用？我们为什么在纳什均衡中考虑随机策略？

Solution: 不，如果给定其他智能体的固定策略$\boldsymbol{\pi}^{-i}$，确定性最佳响应足以获得最高效用。最佳响应可以被形式化为求解一个MDP，如第25.2节所述。已经证明，确定性策略足以提供最优的效用最大化。因此，对于MG中的最佳响应也是如此。

在纳什均衡中，最佳响应必须对所有代理成立。尽管确定性最佳响应在效用上可能与随机响应相等，但均衡可能需要随机响应，以防止其他代理想要偏离。

练习25.3. 本章仅讨论了平稳马尔可夫策略。还有哪些其他类型的策略？

Solution: 所谓*behavioral policy* $\pi^i(\mathrm{h}_t)$是指依赖于完整历史$\mathrm{h}_t = (s_{1:t}$，$\mathrm{a}_{1:t-1})$的策略。此类策略依赖于其他智能体的博弈历史。*nonstationary Markov policy* $\pi^i(s$，$t)$则是指依赖于时间步t，但不依赖于完整历史的策略。例如，在捕食者-猎物六边形世界领域，在前10个时间步中，六边形上的动作可能是向东，而在10个时间步之后，则是向西。

可能存在位于非平稳、非马尔可夫联合策略空间中的纳什均衡；平稳、非马尔可夫联合策略；等等。然而，已经证明每个平稳马尔可夫博弈都有一个平稳马尔可夫纳什均衡。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

练习25.4。在MGs中，虚拟对局要求估算效用。列举计算效用的不同方法，并说明它们的优缺点。

Solution: 算法25.7对访问状态s和所有联合动作a执行单次备份。这种方法的优点是相对高效，因为它是一次单独的备份。在该状态下更新所有联合动作会导致探索未观察到的动作。这种方法的缺点是，为了获得合适的策略，我们可能需要在所有状态下多次执行这种更新。

另一种替代方案是仅更新已访问状态和实际执行的联合动作，这使得更新步骤更快。缺点是它需要更多的步骤来探索完整的联合动作范围。

另一种替代方法是在所有状态 s 上执行值迭代，直至每次更新步骤收敛。回想一下，对手的模型在每次更新时都会改变。这会诱导出一个新的 MDP，如第 25.2.1 节中针对确定性最佳响应所描述的那样。因此，我们需要在每次更新后重新运行值迭代。这种方法的好处是，它可以在每一步产生最明智的决策，因为效用 Q^i 考虑了随时间变化的所有状态。缺点是更新步骤在计算上非常昂贵。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

26 State Uncertainty

本书这一部分迄今为止讨论的多智能体模型都假设所有智能体能够观测到真实状态。正如MDP可以扩展为包含部分可观测性，MG也可以扩展为产生*partially observable Markov game* (*POMG*)。[1]事实上，POMG推广了前面所有其他问题——本章所介绍的内容。这些复杂问题可用于表示多个智能体对环境获得部分或带有噪声的观测的领域。这种一般性使得对POMG进行建模和求解在计算上具有挑战性。本章定义了POMG，概述了策略表示，并介绍了求解方法。

partially observable Markov game (*POMG*)

[1] POMG也称为*partially observable stochastic game* (*POSG*)。POMG与具有不完全信息的扩展式博弈密切相关。H. Kuhn，“扩展式博弈与信息问题”，载于*Contributions to the Theory of Games II*，H. Kuhn 和 A. Tucker 编，Princeton University Press，1953年，第193–216页。该模型后来被引入人工智能领域。E. A. Hansen、D. S. Bernstein 和 S. Zilberstein，“部分可观测随机博弈的动态规划”，载于*AAAI Conference on Artificial Intelligence* (*AAAI*)，2004年。

26.1 Partially Observable Markov Games

一个POMG（算法26.1）可以视为MGs向部分可观测性的扩展，或是POMDPs向多智能体的扩展。每个智能体$i \in \mathcal{I}$仅根据对共享状态s的局部观察o^i来选择行动$a^i \in \mathcal{A}^i$。系统真实状态$s \in \mathcal{S}$为所有智能体共享，但不一定完全被观测到。初始状态从已知的初始状态分布b中抽取。在联合行动a下，从状态s转移到状态s'的可能性遵循$T(s' \mid s, \text{a})$。如同MGs中一样，根据$R^i(s, \text{a})$生成联合奖励r。每个智能体力求最大化自身累积奖励。所有智能体执行联合行动a后，环境依据*joint observation space* $\mathcal{O} = \mathcal{O}^1 \times \cdots \times \mathcal{O}^k$发出一个联合观察$\text{o} = (o^1, \ldots, o^k)$，其中包括*joint observation*。随后，每个智能体从这一联合观察中获得个体观察o^i。哭泣婴儿问题在例26.1中被扩展到了多智能体场景。

在POMDPs中，如第19章所述，我们能够维持一个信念状态，但这种方法在POMGs中不可行。个体代理无法执行。

与POMDPs中相同类型的信念更新，因为联合动作和联合观察并未被观测到。推断联合动作的概率分布要求每个代理推理其他代理对彼此的推理，而这些代理又反过来推理彼此，如此循环。推断其他观察的分布同样复杂，因为这些观察取决于其他代理的行为。{v*}

由于在POMGs中显式建模信念的困难性，我们将重点放在不需要信念来确定动作的策略表示上。我们可以使用前面关于POMDPs章节中介绍的基于树的条件计划表示和基于图的控制器表示。与MGs中一样，POMG中的每个智能体根据策略π^i行动，或者等价地，智能体们共同根据联合策略$\boldsymbol{\pi} = (\pi^1, \ldots, \pi^k)$行动。

2 *Interactive POMDP*（*I-POMDP*）试图捕捉这种无限回归。P. J. Gmytrasiewicz 和 P. Doshi，“多智能体环境中的序列规划框架”，*Journal of Artificial Intelligence Research*，第 24 卷，第 1 期，第 49–79 页，2005 年。虽然由于它在时间和深度上进行推理，这是一个计算复杂的框架，但此类模型的算法已朝着实用用例的目标取得了巨大进步。E. Sonu、Y. Chen 和 P. Doshi，“智能体群体匿名性下的决策论规划”，*Journal of Artificial Intelligence Research*，第 59 卷，第 725–770 页，2017 年。

```
struct POMG
    γ  # discount factor
    ℐ  # agents
    𝒮  # state space
    𝒜  # joint action space
    𝒪  # joint observation space
    T  # transition function
    O  # joint observation function
    R  # joint reward function
end
```

算法26.1. POMG的数据结构。

考虑一个多智能体POMG对哭泣婴儿问题的泛化。我们有两个照顾者照顾一个婴儿。如同POMDP版本中那样，状态是婴儿饥饿或饱足。每个照顾者的动作是喂养、唱歌或忽略婴儿。如果两个照顾者选择执行相同的动作，成本减半。例如，如果两个照顾者都喂养婴儿，那么奖励仅为-2.5而不是-5。然而，照顾者并不完美地观察婴儿的状态。相反，他们依赖于婴儿哭泣的嘈杂观察，两者具有相同的观察。作为奖励结构的结果，在互相帮助和贪婪地选择一个成本较低的动作之间存在权衡。

示例26.1。作为POMG的多重看护者哭泣婴儿问题。附录F.14提供了额外的细节。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

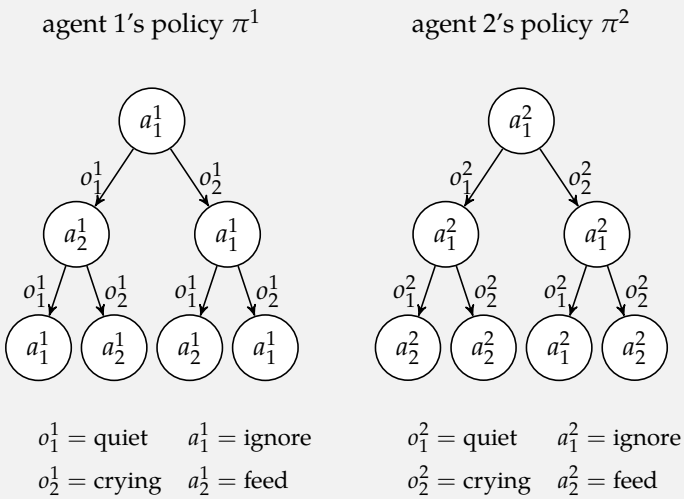
26.2 Policy Evaluation

本节讨论如何评估以基于树的条件计划或基于图的控制器表示的联合策略。正如在 POMDP 的语境中，我们使用条件计划来表示确定性策略，使用控制器来表示随机策略。

26.2.1 Evaluating Conditional Plans

回想一下，条件计划（第20.2节）是一棵树，其中动作与节点相关联，观察结果与边相关联。每个智能体都有自己的树，并且最初选择与其根节点关联的动作。在做出观察后，每个智能体沿着树向下行进，走与其观察结果相关联的边。基于观察结果采取动作和选择边的过程一直持续到树的末端。例26.2展示了一个由每个智能体的条件计划组成的联合策略。

这里是联合策略 $\boldsymbol{\pi} = (\pi^1、\pi^2)$，表示为多照料者哭泣婴儿问题的两步条件计划：



例26.2。针对多护理者哭泣婴儿问题，采用条件计划的两人两步联合策略。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们可以递归地计算联合效用函数U^{π}，类似于在状态s开始时POMDPs中方程(20.8)所做的那样：

$$\mathbf{U}^{\boldsymbol{\pi}}(s) = \mathbf{R}(s, \boldsymbol{\pi}()) + \gamma \left[\sum_{s'} T(s' \mid s, \boldsymbol{\pi}()) \sum_{\mathbf{o}} O(\mathbf{o} \mid \boldsymbol{\pi}(), s') \mathbf{U}^{\boldsymbol{\pi}(\mathbf{o})}(s') \right] \tag{26.1}$$

其中 $\boldsymbol{\pi}()$ 是与 $\boldsymbol{\pi}$ 相关联的树根处的动作向量，而 $\boldsymbol{\pi}(\mathbf{o})$ 是与观察联合观察 o 各分量的不同代理相关联的子规划向量。

与初始状态分布 b 中策略 $\boldsymbol{\pi}$ 相关的效用由下式给出

$$\mathbf{U}^{\boldsymbol{\pi}}(b) = \sum_{s} b(s) \mathbf{U}^{\boldsymbol{\pi}}(s) \tag{26.2}$$

算法26.2提供了这一实现。

```
function lookahead(𝒫::POMG, U, s, a)
    𝒮, 𝒪, T, O, R, γ = 𝒫.𝒮, joint(𝒫.𝒪), 𝒫.T, 𝒫.O, 𝒫.R, 𝒫.γ
    u′ = sum(T(s,a,s′)*sum(O(a,s′,o)*U(o,s′) for o in 𝒪) for s′ in 𝒮)
    return R(s,a) + γ*u′
end

function evaluate_plan(𝒫::POMG, π, s)
    a = Tuple(πi() for πi in π)
    U(o,s′) = evaluate_plan(𝒫, [πi(oi) for (πi, oi) in zip(π,o)], s′)
    return isempty(first(π).subplans) ? 𝒫.R(s,a) : lookahead(𝒫, U, s, a)
end

function utility(𝒫::POMG, b, π)
    u = [evaluate_plan(𝒫, π, s) for s in 𝒫.𝒮]
    return sum(bs * us for (bs, us) in zip(b, u))
end
```

算法26.2。条件计划表示有限时域POMG中的策略。它们是在算法20.1中为单个代理定义的。当从状态 s 开始时，我们可以计算执行由条件计划表示的联合策略 π 所关联的效用。从初始状态分布 b 计算效用涉及在从不同状态开始时对效用进行加权平均。

26.2.2 *Evaluating Stochastic Controllers*

控制器（第 23.1 节）被表示为随机图。与智能体 i 关联的控制器由动作分布 $\psi^i(a^i \mid x^i)$ 和后继分布 $\eta^i(x^{i\prime} \mid x^i、a^i、o^i)$ 定义。在联合节点 x 活跃且遵循联合策略 $\boldsymbol{\pi}$ 的情况下，处于状态 s 的效用为

$$\mathbf{U}^{\boldsymbol{\pi}}(\mathbf{x}, s) = \sum_{\mathbf{a}} \prod_{i} \psi^i(a^i \mid x^i) \left(\mathbf{R}(s, \mathbf{a}) + \gamma \sum_{s'} T(s' \mid s, \mathbf{a}) \sum_{\mathbf{o}} O(\mathbf{o} \mid \mathbf{a}, s') \sum_{\mathbf{x}'} \prod_{i} \eta^i(x^{i\prime} \mid x^i, a^i, o^i) \mathbf{U}^{\boldsymbol{\pi}}(\mathbf{x}', s') \right) \tag{26.3}$$

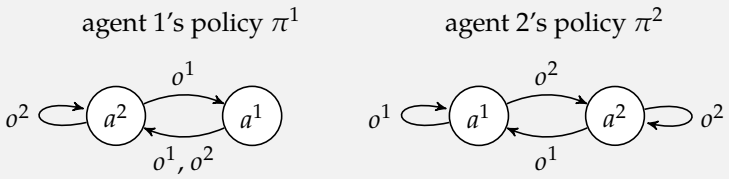
© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

在此背景下，策略评估涉及求解这一线性方程组。或者，我们可以采用类似于算法23.2针对POMDP的迭代策略评估方法。从初始状态分布b和联合控制器状态x开始时的效用为

$$\mathbf{U}^{\boldsymbol{\pi}}(\mathbf{x}, b) = \sum_{s} b(s)\mathbf{U}(\mathbf{x}, s) \tag{26.4}$$

例26.3展示了一个联合随机控制器。

这是一个针对哭泣婴儿问题中两位照顾者的联合控制器策略 $\boldsymbol{\pi} = (\pi^1、\pi^2)$ 。每个控制器有两个节点，$X^i = \{x_1^i、x_2^i\}$：



$o^1 =$ quiet　　$a^1 =$ ignore
$o^1 =$ hungry　　$a^1 =$ feed

示例26.3。针对多照顾者哭闹婴儿问题，采用控制器的双智能体联合策略。

26.3 Nash Equilibrium

与简单博弈和MGs一样，POMG的*Nash equilibrium*是指所有智能体都按照彼此的最佳响应策略行动，使得任何智能体都没有动机偏离其策略。POMG的纳什均衡通常在计算上极其难以求解。算法26.3为POMG计算一个d步纳什均衡。它枚举所有可能的d步联合条件计划来构造一个简单博弈，如例26.4所示。这个简单博弈的纳什均衡也是POMG的纳什均衡。

简单博弈与POMG具有相同的代理。对于POMG中的每一个联合条件计划，简单博弈中都存在一个联合行动。每个行动所获得的奖励等于POMG中联合条件计划下的效用。这个构造出的简单博弈的纳什均衡可以直接作为POMG的纳什均衡应用。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct POMGNashEquilibrium
    b # initial belief
    d # depth of conditional plans
end

function create_conditional_plans(𝒫, d)
    ℐ, 𝒜, 𝒪 = 𝒫.ℐ, 𝒫.𝒜, 𝒫.𝒪
    Π = [[ConditionalPlan(ai) for ai in 𝒜[i]] for i in ℐ]
    for t in 1:d
        Π = expand_conditional_plans(𝒫, Π)
    end
    return Π
end

function expand_conditional_plans(𝒫, Π)
    ℐ, 𝒜, 𝒪 = 𝒫.ℐ, 𝒫.𝒜, 𝒫.𝒪
    return [[ConditionalPlan(ai, Dict(oi ⇒ πi for oi in 𝒪[i]))
        for πi in Π[i] for ai in 𝒜[i]] for i in ℐ]
end

function solve(M::POMGNashEquilibrium, 𝒫::POMG)
    ℐ, γ, b, d = 𝒫.ℐ, 𝒫.γ, M.b, M.d
    Π = create_conditional_plans(𝒫, d)
    U = Dict(π ⇒ utility(𝒫, b, π) for π in joint(Π))
    𝒢 = SimpleGame(γ, ℐ, Π, π → U[π])
    π = solve(NashEquilibrium(), 𝒢)
    return Tuple(argmax(πi.p) for πi in π)
end
```

算法26.3。对于具有初始状态分布b的POMG $\mathcal{P}$，通过创建到某一深度d的所有条件计划的简单博弈来计算纳什均衡。我们使用算法24.5求解此简单博弈中的纳什均衡。为简便起见，我们选择最可能的联合策略。或者，我们可以在执行开始时随机选择联合策略。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

考虑具有两步规划期的多照护者哭泣婴儿问题。回想一下，对于每个主体i，有三种行动

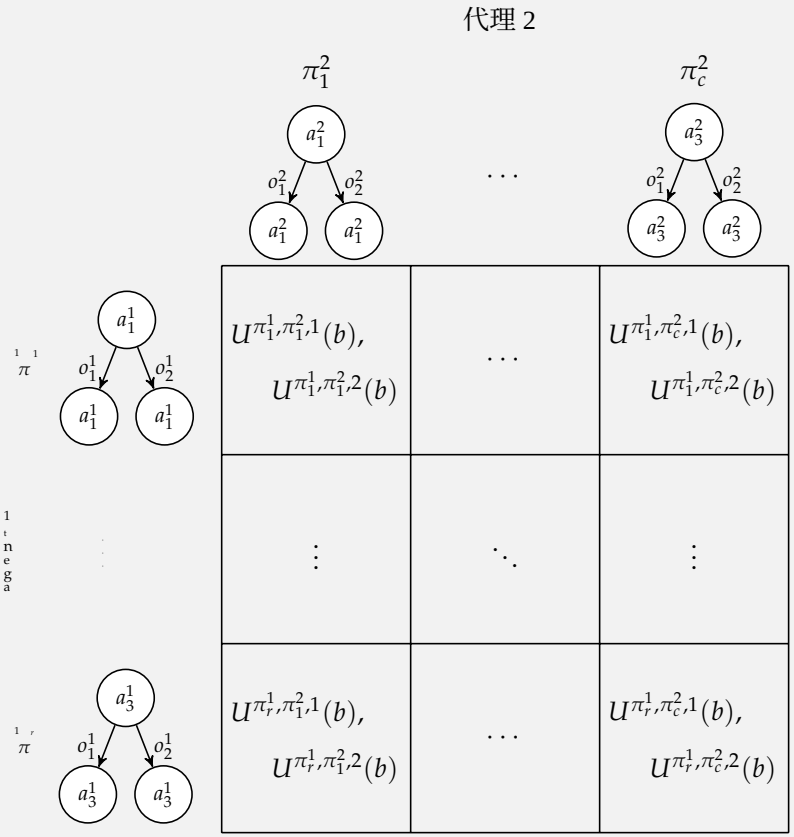
$$\mathcal{A}^i = \{a_1^i, a_2^i, a_3^i\} = \{\text{feed}, \text{sing}, \text{ignore}\}$$

以及两个观测值

$$\mathcal{O}^i = \{o_1^i, o_2^i\} = \{\text{cry}, \text{silent}\}$$

将这个POMG转换为简单博弈后，得到如下博弈表。每个照护者选择对应于完整条件计划的简单博弈动作。每个代理的简单博弈奖励是与联合策略相关的效用。

示例26.4。通过将多照护者哭泣婴儿问题转换为一个简单博弈来计算其纳什均衡，其中行动对应于条件计划。



© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

26.4 Dynamic Programming

上一节中计算纳什均衡所采用的方法通常在计算上极为昂贵，因为动作对应于达到某一深度的所有可能的条件计划。我们可以将值迭代方法调整用于POMDP（第20.5节），在那里我们在扩展所考虑的条件计划集合的深度和修剪次优计划之间进行迭代。虽然最坏情况下的计算复杂度与完全展开所有策略树相同，但这种增量方法可以带来显著的节省。

算法26.4实现了这种动态规划方法。它首先构建所有一步计划。我们剔除任何被其他计划支配的计划，然后扩展所有一步计划的组合以生成两步计划。这种交替进行扩展和剔除的过程会一直重复，直到达到所需的时间范围。

剪枝步骤会消除所有被支配的策略。如果存在另一个策略 $\pi^{i\prime}$ 总是至少和 π^i 一样好，那么属于一个 i 的策略 π^i 就可以被剪枝。尽管计算成本高昂，但这一条件可以通过求解一个线性规划来检查。这个过程与 POMDPs 中的控制器节点剪枝有关（算法 23.4）。

为其他agent的策略$\boldsymbol{\pi}^{-i}$的每一种可能组合求解一个单独的线性规划在计算上是不可行的。相反，我们可以采用一种更高效的方法，该方法绝不会剪除最优策略，但可能无法剪除所有次优策略。如果不存在其他联合策略$\boldsymbol{\pi}^{-i}$和状态s之间的$b(\boldsymbol{\pi}^{-i}、s)$使得

$$\sum_{\boldsymbol{\pi}^{-i}} \sum_{s} b(\boldsymbol{\pi}^{-i}, s) U^{\pi^{i\prime}, \boldsymbol{\pi}^{-i}, i}(s) \geq \sum_{\boldsymbol{\pi}^{-i}} \sum_{s} b(\boldsymbol{\pi}^{-i}, s) U^{\pi^{i}, \boldsymbol{\pi}^{-i}, i}(s) \tag{26.5}$$

这里，b是其他智能体策略与状态的联合分布。正如本章开头所提到的，计算信念状态通常是不可行的，但方程(26.5)检查了单个策略支配的信念空间。

我们可以构建一个单一的线性规划来检查方程（26.5）。[3] 如果线性

[3] A similar linear program was created to prune alpha vectors in POMDPs in equation (20.16).

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct POMGDynamicProgramming
    b   # initial belief
    d   # depth of conditional plans
end

function solve(M::POMGDynamicProgramming, 𝒫::POMG)
    ℐ, 𝒮, 𝒜, R, γ, b, d = 𝒫.ℐ, 𝒫.𝒮, 𝒫.𝒜, 𝒫.R, 𝒫.γ, M.b, M.d
    Π = [[ConditionalPlan(ai) for ai in 𝒜[i]] for i in ℐ]
    for t in 1:d
        Π = expand_conditional_plans(𝒫, Π)
        prune_dominated!(Π, 𝒫)
    end
    𝒢 = SimpleGame(γ, ℐ, Π, π -> utility(𝒫, b, π))
    π = solve(NashEquilibrium(), 𝒢)
    return Tuple(argmax(πi.p) for πi in π)
end

function prune_dominated!(Π, 𝒫::POMG)
    done = false
    while !done
        done = true
        for i in shuffle(𝒫.ℐ)
            for πi in shuffle(Π[i])
                if length(Π[i]) > 1 && is_dominated(𝒫, Π, i, πi)
                    filter!(πi′ -> πi′ ≠ πi, Π[i])
                    done = false
                    break
                end
            end
        end
    end
end

function is_dominated(𝒫::POMG, Π, i, πi)
    ℐ, 𝒮 = 𝒫.ℐ, 𝒫.𝒮
    jointΠnoti = joint([Π[j] for j in ℐ if j ≠ i])
    π(πi′, πnoti) = [j==i ? πi′ : πnoti[j>i ? j-1 : j] for j in ℐ]
    Ui = Dict((πi′, πnoti, s) => evaluate_plan(𝒫, π(πi′, πnoti), s)[i]
              for πi′ in Π[i], πnoti in jointΠnoti, s in 𝒮)
    model = Model(Ipopt.Optimizer)
    @variable(model, δ)
    @variable(model, b[jointΠnoti, 𝒮] ≥ 0)
    @objective(model, Max, δ)
    @constraint(model, [πi′=Π[i]],
        sum(b[πnoti, s] * (Ui[πi′, πnoti, s] - Ui[πi, πnoti, s])
        for πnoti in jointΠnoti for s in 𝒮) ≥ δ)
    @constraint(model, sum(b) == 1)
    optimize!(model)
    return value(δ) ≥ 0
end
```

算法26.4。动态规划计算POMG $\mathcal{P}$ 的纳什均衡 π，给定初始信念 b 和视野深度 d。它在每一步迭代计算策略树及其预期效用。每次迭代中的修剪阶段移除被支配的策略，即那些导致比至少一个其他可用策略树更低的预期效用的策略树。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

如果程序可行，那么这意味着π^i不被任何其他$\pi^{i\prime}$所支配：

$$\begin{aligned}
&\underset{\delta, b}{\text{最大化}} \quad \delta \\
&\text{主题ect to} \quad b(\boldsymbol{\pi}^{-i}, s) \geq 0 \text{ for all } \boldsymbol{\pi}^{-i}, s \\
&\qquad \sum_{\boldsymbol{\pi}^{-i}} \sum_{s} b(\boldsymbol{\pi}^{-i}, s) = 1 \\
&\qquad \sum_{\boldsymbol{\pi}^{-i}} \sum_{s} b(\boldsymbol{\pi}^{-i}, s) \left(U^{\pi^{i\prime}, \boldsymbol{\pi}^{-i}, i}(s) - U^{\pi^i, \boldsymbol{\pi}^{-i}, i}(s) \right) \geq \delta \text{ for all } \pi^{i\prime}
\end{aligned} \tag{26.6}$$

剪枝步骤通过随机选择一个智能体 i 并检查其每个策略是否被支配来移除受支配策略。此过程重复进行，直到对所有智能体的一轮遍历未能发现任何受支配策略。例26.5展示了在多照料者哭泣婴儿问题上的这一过程。

26.5 Summary

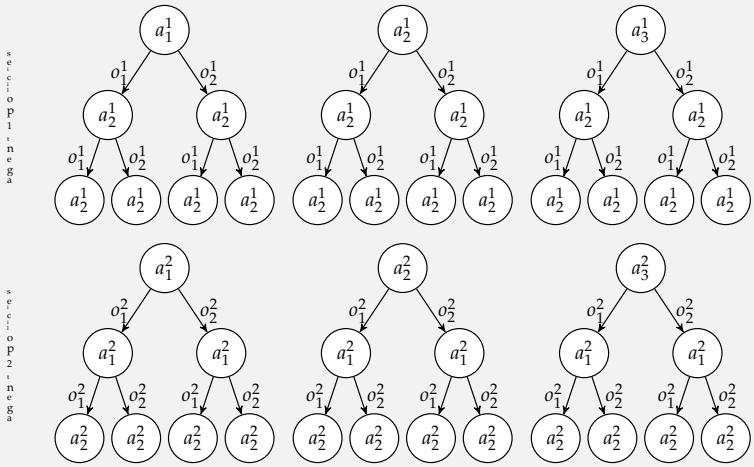
- POMG将POMDP推广到多智能体，将MG推广到部分可观测性。
- 由于智能体通常无法在POMGs中保持信念，策略通常采取条件计划或有限状态控制器的形式。
- 纳什均衡，以POMGs的d步条件计划形式，可以通过寻找简单博弈的纳什均衡来获得，这些简单博弈的联合行动包括所有可能的POMG联合策略。
- 动态规划方法可以通过迭代构建更深层的条件计划集，同时剪枝被支配的计划以限制搜索空间，从而更高效地计算纳什均衡。

26.6 Exercises

练习26.1。证明POMG泛化了POMDP和MG。

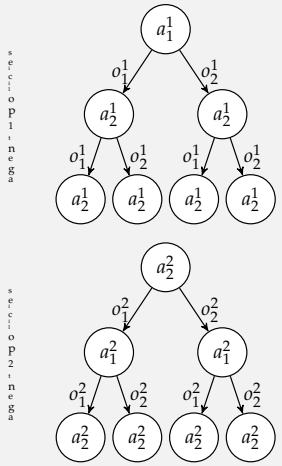
© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

考虑由动态规划解决的多重照顾者哭泣婴儿问题。初始时，深度为$d = 2$ 的策略是



例26.5。多照顾者哭泣婴儿问题的动态规划和单次剪枝步骤。

剪枝步骤之后，智能体策略为



在这种情况下，剪枝步骤找到了最佳联合策略。这种方法显著减少了算法下一次迭代需要考虑的可能联合策略的数量。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: 对于任何POMDP，我们可以定义一个有单个代理$\mathcal{I} = \{1\}$的POMG。状态$\mathcal{S}$相同，动作A $= (\mathcal{A}^1)$和观察O $= (\mathcal{O}^1)$也相同。因此，POMG的状态转移、观察函数和奖励直接随之确定。纳什均衡优化只有一个代理，因此它导致简单的期望值最大化，这与POMDP相同。

对于任何MG，我们可以定义一个具有相同智能体$\mathcal{I}$、状态$\mathcal{S}$、联合动作A、转移T和联合奖励R的POMG。个体观测被分配为状态$\mathcal{O}^i = \mathcal{S}$。然后，观测函数确定性地为每个智能体提供真实状态$O(\text{o} \mid \text{a}, s') = 1$如果$\text{o} = (s', \ldots, s')$，否则为0。

练习26.2。我们如何将智能体之间的通信融入POMG框架中?

Solution: 可以为智能体的动作空间增加通信动作。其他智能体可以根据其观察模型观察到这些通信动作。

练习26.3。代理总是有沟通的激励吗?

Solution: POMG 中的智能体通常具有竞争性，在这种情况下，它们没有动机与他人通信。如果它们的奖励在一定程度上是一致的，它们可能会倾向于通信。

练习26.4。深度为d的可能联合条件计划有多少种?

Solution: 回想一下，有$|\mathcal{A}|^{(|\mathcal{O}|^d-1)/(|\mathcal{O}|-1)}$种可能的$d$步单智能体条件计划。我们可以通过跨智能体组合这些单智能体条件计划的每一种组合来构建一个联合策略。d步多智能体条件计划的数量是

$$\prod_{i \in \mathcal{I}} |\mathcal{A}^i|^{(|\mathcal{O}^i|^d-1)/(|\mathcal{O}^i|-1)}$$

练习 26.5. 以代理 i 的效用 $U^{\boldsymbol{\pi},i}$ 定义 POMG 的最佳响应。提出 POMG 的迭代最佳响应。

Solution: 智能体 i 对其他智能体策略 $\boldsymbol{\pi}^{-i}$ 的最佳响应 π^i 根据方程 (24.2) 针对初始信念 b 定义如下：

$$U^{\pi^i,\boldsymbol{\pi}^{-i},i}(b) \geq U^{\pi^{i\prime},\boldsymbol{\pi}^{-i},i}(b)$$

与任何其他策略 $\pi^{i\prime}$。对于条件计划，$U^{\boldsymbol{\pi},i}$ 由方程 (26.1) 和 (26.2) 定义。

迭代最佳响应的实现遵循第24.2.1节。首先，可以按照算法26.3创建条件计划和简单游戏。然后，我们可以使用算法24.8迭代最佳响应。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

27 Collaborative Agents

许多多智能体领域是协作性的，在其中所有智能体在环境中独立行动，同时致力于一个共同的目标。应用范围从机器人搜索与救援到星际探索漫游车。*decentralized partially observable Markov decision process* (*Dec-POMDP*) 在关注此类协作智能体设置的同时，捕捉了POMG的普遍性。[1]

1 D. S. Bernstein、R. Givan、N. Immerman 和 S. Zilberstein，“马尔可夫决策过程分散控制的复杂性”，*Mathematics of Operation*，第 27 卷，第 4 期，第 819–840 页，2002 年。更全面的介绍由 F. A. Oliehoek 和 C. Amato 提供，*A*。Springer，2016 年。

Research

该模型更易于采用可扩展的近似算法，因为它只有一个共享目标，而不是在多个个体代理目标之间寻找均衡。本章介绍Dec-POMDP模型，突出其子类，并描述能最优和近似解决这些子类的算法。

Concise Introduction to Decentralized POMDPs

27.1 Decentralized Partially Observable Markov Decision Processes

Dec-POMDP（算法27.1）是一种POMG，其中所有智能体共享相同的目标。每个智能体$i \in \mathcal{I}$根据局部观测历史$o^i \in \mathcal{O}^i$选择局部动作$a^i \in \mathcal{A}^i$。系统真实状态$s \in \mathcal{S}$由所有智能体共享。单次奖励由$R(s$、基于状态s和联合动作a的)生成。所有智能体的目标是在局部部分可观测性下，最大化随时间共享的期望奖励。例27.1描述了捕食者-猎物问题的Dec-POMDP版本。

考虑一个捕食者-猎物六边形世界问题，其中一组捕食者$\mathcal{I}$努力捕获一只逃跑的猎物。捕食者独立移动。猎物随机移动到一个未被捕食者占据的相邻单元格。捕食者必须合作才能捕获猎物。

例27.1. 合作捕食者-猎物问题作为一个Dec-POMDP。更多细节见附录F.15。

POMG 中的许多相同挑战在 Dec-POMDP 中依然存在，例如智能体通常无法维持信念状态。我们关注以条件计划或控制器表示的政策。前一章介绍的相同算法可用于评估政策。唯一需要做的是为每个智能体 i 创建一个 POMG，其 $R^i(s、a)$ 等于 Dec-POMDP 中的 $R(s、a)$。

```
struct DecPOMDP
    γ  # discount factor
    ℐ  # agents
    𝒮  # state space
    𝒜  # joint action space
    𝒪  # joint observation space
    T  # transition function
    O  # joint observation function
    R  # reward function
end
```

算法27.1. 用于Dec-POMDP的数据结构。算法24.2中的`joint`函数允许创建给定集合的所有组合，如$\mathcal{A}$或$\mathcal{O}$。`tensorform`函数将Dec-POMDP $\mathcal{P}$转换为张量表示。

27.2 *Subclasses*

Dec-POMDP 有许多值得注意的子类。在设计利用其特定结构的算法时，对这些子类进行分类是有用的。

一个值得关注的属性是 *joint full observability*，即当每个智能体观察到状态的一个方面时，如果将它们各自的观察结果结合起来，就能唯一揭示真实状态。然而，智能体之间并不共享各自的观察结果。这一性质确保如果 $O(o \mid a, s') > 0$，则 $P(s' \mid o) = 1$。具有联合完全可观测性的 Dec-POMDP 被称为 *decentralized Markov decision process* (*Dec-MDP*)。当视界中的步数少于状态数时，Dec-POMDP 和 Dec-MDP 问题都是 *NEXP-complete*。[2]

在许多设置中，Dec-POMDP的状态空间是因子化的，每个智能体一个因子，环境一个因子。这被称为*factored Dec-POMDP*。我们有$\mathcal{S} = \mathcal{S}^0 \times \cdots \times \mathcal{S}^k$，其中$\mathcal{S}^i$是与智能体$i$相关的因子化状态分量，$\mathcal{S}^0$是与一般环境相关的因子化状态分量。例如，在协作捕食者-猎物问题中，每个智能体对其位置都有自己的状态因子，而猎物的位置则与状态空间的环境分量相关联。

2 与复杂性类 NP 和 PSPACE 不同，已知 NEXP 不属于 P。因此，我们可以证明 Dec-MDP 和 Dec-POMDP 不允许多项式时间算法。D. S. Bernstein, R. Givan, N. Immerman, and S. Zilberstein, "The Complexity of Decentralized Control of Markov Decision Processes," *Mathematics of Operation Research*, vol. 27, no. 4, pp. 819–840, 2002.

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

在一些问题中，一个因子化的Dec-POMDP可能具有以下一个或多个特性：

- *Transition independence*，其中智能体可能不会相互影响彼此的状态：

$$T(\mathbf{s}' \mid \mathbf{s}, \mathbf{a}) = T^0(s^{0\prime} \mid s^0) \prod_i T^i(s^{i\prime} \mid s^i, a^i) \tag{27.1}$$

- *Observation independence*，其中智能体的观测仅依赖于其局部状态和动作：

$$O(\mathbf{o} \mid \mathbf{a}, \mathbf{s}') = \prod_i O^i(o^i \mid a^i, s^{i\prime}) \tag{27.2}$$

- *Reward independence*，其中奖励可以分解为多个独立的组成部分：[3]

$$R(\mathbf{s}, \mathbf{a}) = R^0(s^0) + \sum_i R^i(s^i, a^i) \tag{27.3}$$

3 在这里，我们将奖励组件的组合展示为求和形式，但也可以使用任何单调非递减函数来代替，同时保持奖励独立性。 {v*}

计算复杂度会根据满足哪些独立性属性而有显著差异，如表27.1所总结。在建模问题时，考虑这些独立性以提高可扩展性是非常重要的。

表27.1. 不同独立性假设下的因式分解Dec-POMDP的复杂度。

Independence	Complexity
Transitions, observations, and rewards	P-complete
Transitions and observations	NP-complete
Any other subset	NEXP-complete

一个
network distributed partially observable Markov decision process (*ND-POMDP*) 是一个具有转移和观测独立性以及特殊奖励结构的 Dec-POMDP。奖励结构由协调图表示。与本书前面使用的图相比，*coordination graph* 是一种超图，它允许边连接任意数量的节点。ND-POMDP 超图中的节点对应于各个智能体。边与奖励函数中智能体之间的交互相关。一个 ND-POMDP 将超图中的每条边 j 与一个奖励分量 R_j 相关联，该分量取决于边所连接的状态和动作分量。奖励函数在

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

一个ND-POMDP就是与边相关的奖励分量之和。图27.1展示了一个协调图，其产生的奖励函数可以分解为：

$$R_{123}(s_1, s_2, s_3, a_1, a_2, a_3) + R_{34}(s_3, s_4, a_3, a_4) + R_5(s_5, a_5) \tag{27.4}$$

传感器网络和目标跟踪问题通常被构建为ND-POMDP。

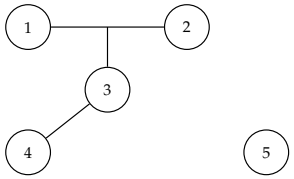


图27.1. 一个包含五个智能体的ND-POMDP结构。存在三个超边：一个涉及智能体1、2和3；另一个涉及智能体3和4；还有一个仅涉及智能体5。

ND-POMDP模型类似于转移和观测独立的Dec-MDP模型，但它不做出联合完全可观测性假设。即使所有观测都被共享，世界的真实状态也可能未知。此外，即使有因子化的转移和观测，ND-POMDP中的策略也是从观测历史到动作的映射，这与转移和观测Dec-MDP情况不同，后者中的策略是从局部状态到动作的映射。最坏情况复杂度仍与Dec-POMDP相同，但ND-POMDP的算法通常在智能体数量方面更具可扩展性。随着协调图变得不那么连通，可扩展性可以提高。

如果智能体能够无惩罚地完美交流它们的动作和观察，那么它们就能维持一个集体信念状态。这种模型被称为 *multiagent MDP* (*MMDP*) 或 *multiagent POMDP* (*MPOMDP*)。当存在转移、观察和奖励独立性时，也可以得到 MMDP 和 MPOMDP。之前章节中讨论的任何 MDP 或 POMDP 算法都可以用来解决这些问题。

表27.2总结了其中一些子类。图27.2展示了本书讨论的模型之间的关系。

Agents	Observability	Communication	Model
Single	Full	—	MDP
Single	Partial	—	POMDP
Multiple	Full	Free	MMDP
Multiple	Full	General	MMDP
Multiple	Joint full	Free	MMDP
Multiple	Joint full	General	Dec-MDP
Multiple	Partial	Free	MPOMDP
Multiple	Partial	General	Dec-POMDP

表27.2. Dec-POMDP子类按类型和计算复杂度分类。“可观测性”指共享状态的可观测程度。“通信”指合作智能体是否能自由地彼此共享所有观测。自由通信发生在模型之外（例如机器人中的高速无线连接）。一般通信是指智能体无法使用这种通信，必须通过其动作进行（通常是不完美的）通信。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

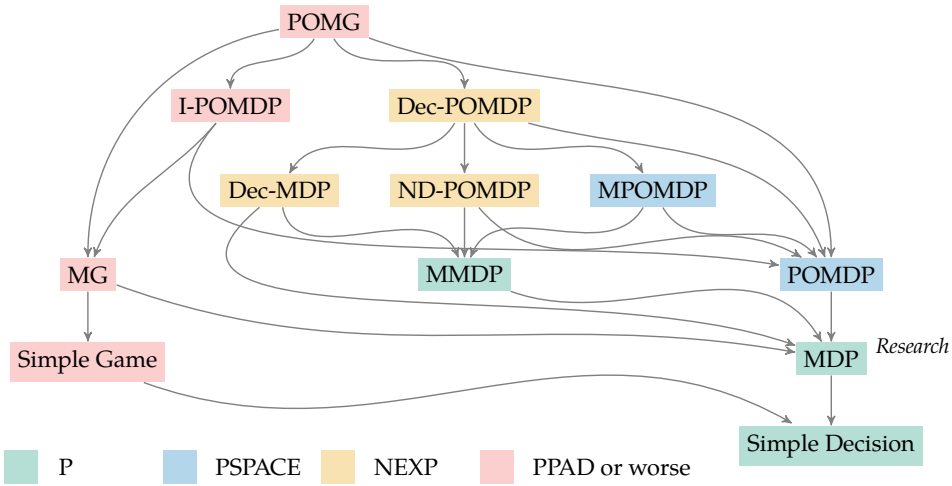


图 27.2. 本书所讨论模型的分类体系。在此图中，父节点泛化其子节点。例如，Dec-POMDPs 通过支持多智能体泛化了POMDPs。节点颜色表示计算复杂度，如图左下角的图例所示。此处列出的复杂度针对本书中介绍的常见模型、策略和目标公式。更详细的处理，请参见 C. Papadimitriou 和 J. Tsitsiklis 的“马尔可夫决策过程的复杂性”，*Mathematics of Operation Research*，第12卷，第3期，第441-450页，1987年。同时，参见 S. Seuken 和 S. Zilberstein 的“不确定下的去中心化决策的形式模型与算法”，*Autonomous Agents and Multi-Agent Systems*，第17卷，第2期，第190-250页，2008年。

27.3 *Dynamic Programming*

Dec-POMDPs的*dynamic programming*算法在每一步应用贝尔曼备份并剪枝被支配的策略。此过程与POMGs的动态规划相同，只是每个代理共享相同的奖励。算法27.2实现了这一过程。

```
struct DecPOMDPDynamicProgramming
    b   # initial belief
    d   # depth of conditional plans
end

function solve(M::DecPOMDPDynamicProgramming, 𝒫::DecPOMDP)
    ℐ, 𝒮, 𝒜, 𝒪, T, O, R, γ = 𝒫.ℐ, 𝒫.𝒮, 𝒫.𝒜, 𝒫.𝒪, 𝒫.T, 𝒫.O, 𝒫.R, 𝒫.γ
    R′(s, a) = [R(s, a) for i in ℐ]
    𝒫′ = POMG(γ, ℐ, 𝒮, 𝒜, 𝒪, T, O, R′)
    M′ = POMGDynamicProgramming(M.b, M.d)
    return solve(M′, 𝒫′)
end
```

算法27.2。动态规划为Dec-POMDP $\mathcal{P}$计算最优联合策略π，给定初始信念b和视野深度d。我们可以直接使用POMG算法，因为Dec-POMDP是POMG的一种特殊协作类。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

27.4 Iterated Best Response

与直接探索联合策略不同，我们可以采用一种*iterated best response*（算法27.3）的形式。在这种方法中，我们迭代地选择一个智能体，并假设其他智能体遵循固定策略，计算最佳响应策略。[4]这种近似算法通常很快，因为它是在选择每次仅针对一个智能体的最优策略。此外，由于所有智能体共享相同的奖励，它往往在相对较少的迭代后终止。

迭代最佳响应从随机初始联合策略 $\boldsymbol{\pi}_1$ 开始。该过程随机遍历各个智能体。如果选择智能体 i，则其策略 π^i 会被更新为对其他智能体固定策略 $\boldsymbol{\pi}^{-i}$ 的最佳响应，并使用初始信念分布 b：

$$\pi^i \leftarrow \arg\max_{\pi^{i\prime}} U^{\pi^{i\prime},\boldsymbol{\pi}^{-i}}(b) \tag{27.5}$$

在平局时倾向于当前策略。当代理停止改变其策略时，该过程可以终止。

虽然该算法速度快且保证收敛，但它并不总能找到最佳的联合策略。它依赖于迭代最佳响应来寻找纳什均衡，然而可能存在多个纳什均衡，且它们关联的效用各不相同。此方法只能找到其中一个。

4 这类算法也称为*joint equilibrium-based search*。R. Nair、M. Tambe、M. Yokoo、D. Pynadath 和 S. Marsella，“驯服去中心化POMDP：面向多智能体环境的高效策略计算”，载于*International Joint Conference on Artificial Intelligence (IJCAI)*，2003年。通过执行动态规划，它可以进一步改进。

27.5 Heuristic Search

不是扩展所有联合策略，*heuristic search*（算法27.4）探索固定数量的策略，[5]这些策略在迭代中存储，防止指数级增长。启发式探索通过仅尝试将最佳联合策略扩展至深度 d 来指导搜索。

算法的每次迭代 k 都保留一组联合策略 $\boldsymbol{\Pi}_k$。这个集合最初由所有单步条件计划组成。后续迭代开始时，会完全展开这些条件计划。目标是为此后的迭代添加固定数量的条件计划。

我们优先选择那些在决定要添加到集合中的条件计划时更有可能最大化效用的策略。然而，由于我们是从底层向上扩展条件计划，我们不能简单地从未初始信念状态b评估策略。相反，我们需要对未来$d-k$步的信念进行估计，我们通过采取随机动作并模拟状态来计算这一估计。

5 这种方法也被称为*memory-bounded dynamic programming (MBDP)*。S. Seuken和S. Zilberstein，“用于Dec-POMDPs的内存受限动态规划”，出自*International Joint Conference on Artificial Intelligence (IJCAI)*，2007年。还有其他启发式搜索算法，例如*multiagent A* (MMA*)*。D. Szer、F. Charpillet和S. Zilberstein，“MAA*：一种求解分散式POMDPs的启发式搜索算法”，出自*Conference on Uncertainty in Artificial Intelligence (UAI)*，2005年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct DecPOMDPIteratedBestResponse
    b     # initial belief
    d     # depth of conditional plans
    k_max # number of iterations
end

function solve(M::DecPOMDPIteratedBestResponse, 𝒫::DecPOMDP)
    ℐ, 𝒮, 𝒜, 𝒪, T, O, R, γ = 𝒫.ℐ, 𝒫.𝒮, 𝒫.𝒜, 𝒫.𝒪, 𝒫.T, 𝒫.O, 𝒫.R, 𝒫.γ
    b, d, k_max = M.b, M.d, M.k_max
    R′(s, a) = [R(s, a) for i in ℐ]
    𝒫′ = POMG(γ, ℐ, 𝒮, 𝒜, 𝒪, T, O, R′)
    Π = create_conditional_plans(𝒫, d)
    π = [rand(Π[i]) for i in ℐ]
    for k in 1:k_max
        for i in shuffle(ℐ)
            π′(πi) = Tuple(j == i ? πi : π[j] for j in ℐ)
            Ui(πi) = utility(𝒫′, b, π′(πi))[i]
            π[i] = argmax(Ui, Π[i])
        end
    end
    return Tuple(π)
end
```

算法27.3。协作Dec-POMDP {v*} 的迭代最佳响应 $\mathcal{P}$ 对每个智能体执行确定性最佳响应，以快速搜索条件计划策略空间。`solve` 函数对深度为 `d` 的条件计划执行此过程，最多 `k_max` 步，最大化初始信念 `b` 处的价值。

转换和观察，沿途更新信念。在第 k 次迭代中的这一信念表示为 b_k。对于每个可用的联合策略 $\boldsymbol{\pi} \in \boldsymbol{\Pi}_k$，检查效用 $U^{\boldsymbol{\pi}}(b_k)$ 以找到一个可添加的最大化效用的联合策略。例27.2演示了这一过程。

27.6 Nonlinear Programming

我们可以使用*nonlinear programming* (*NLP*) (算法27.5)来找到一个固定大小的最优联合控制器策略表示。[6]该方法推广了NLP

来自23.3节的POMDPs方法。

6 C. Amato、D. S. Bernstein 和 S. Zilberstein，“针对POMDP和分散式POMDP优化固定尺寸随机控制器”，*Autonomous Agents and Multi-Agent Systems*，第21卷，第3期，第293-320页，2010年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
struct DecPOMDPHeuristicSearch
    b     # initial belief
    d     # depth of conditional plans
    π_max # number of policies
end

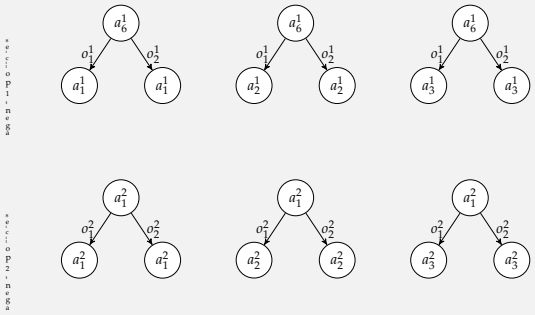
function solve(M::DecPOMDPHeuristicSearch, 𝒫::DecPOMDP)
    ℐ, 𝒮, 𝒜, 𝒪, T, O, R, γ = 𝒫.ℐ, 𝒫.𝒮, 𝒫.𝒜, 𝒫.𝒪, 𝒫.T, 𝒫.O, 𝒫.R, 𝒫.γ
    b, d, π_max = M.b, M.d, M.π_max
    R′(s, a) = [R(s, a) for i in ℐ]
    𝒫′ = POMG(γ, ℐ, 𝒮, 𝒜, 𝒪, T, O, R′)
    Π = [[ConditionalPlan(ai) for ai in 𝒜[i]] for i in ℐ]
    for t in 1:d
        allΠ = expand_conditional_plans(𝒫, Π)
        Π = [[] for i in ℐ]
        for z in 1:π_max
            b′ = explore(M, 𝒫, t)
            π = argmax(π -> first(utility(𝒫′, b′, π)), joint(allΠ))
            for i in ℐ
                push!(Π[i], π[i])
                filter!(πi -> πi != π[i], allΠ[i])
            end
        end
    end
    return argmax(π -> first(utility(𝒫′, b, π)), joint(Π))
end

function explore(M::DecPOMDPHeuristicSearch, 𝒫::DecPOMDP, t)
    ℐ, 𝒮, 𝒜, 𝒪, T, O, R, γ = 𝒫.ℐ, 𝒫.𝒮, 𝒫.𝒜, 𝒫.𝒪, 𝒫.T, 𝒫.O, 𝒫.R, 𝒫.γ
    b = copy(M.b)
    b′ = similar(b)
    s = rand(SetCategorical(𝒮, b))
    for τ in 1:t
        a = Tuple(rand(𝒜i) for 𝒜i in 𝒜)
        s′ = rand(SetCategorical(𝒮, [T(s,a,s′) for s′ in 𝒮]))
        o = rand(SetCategorical(joint(𝒪), [O(a,s′,o) for o in joint(𝒪)]))
        for (i′, s′) in enumerate(𝒮)
            po = O(a, s′, o)
            b′[i′] = po*sum(T(s,a,s′)*b[i] for (i,s) in enumerate(𝒮))
        end
        normalize!(b′, 1)
        b, s = b′, s′
    end
    return b′
end
```

算法27.4。内存受限启发式搜索使用启发式函数来搜索Dec-POMDP $\mathcal{P}$的条件计划空间。`solve`函数试图最大化初始信念`b`下深度为`d`的联合条件计划的值。`explore`函数通过采取随机行动并模拟行动和观察，生成未来`t`步的信念。该算法受内存限制，每个代理仅保留`π_max`个条件计划。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

考虑右侧所示的协作捕食者-猎物问题。我们将启发式搜索应用于深度为$d = 3$，每次迭代保留三个策略。在迭代$k = 1$之后，策略是



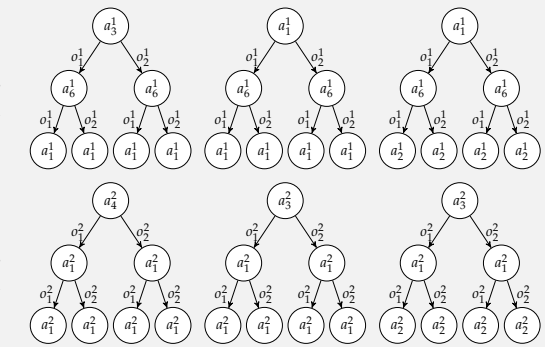
在下一次迭代$k = 2$中，启发式搜索再次从初始信念开始，并按照启发式探索采取$d - k = 3 - 2 = 1$步。用于选择接下来的三个条件计划的已探索信念是

$$b_1 = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.17, 0.0, 0.03, 0.01, 0.0, 0.0, 0.05, 0.0, 0.01, 0.23, 0.0, 0.08, 0.01, 0.0, 0.0, 0.14, 0.0, 0.03, 0.22, 0.0, 0.01]$$

$$b_2 = [0.0, 0.21, 0.03, 0.0, 0.04, 0.01, 0.0, 0.05, 0.01, 0.0, 0.08, 0.03, 0.0, 0.0, 0.01, 0.0, 0.0, 0.01, 0.08, 0.34, 0.03, 0.02, 0.05, 0.01, 0.0, 0.01, 0.0]$$

$$b_3 = [0.0, 0.03, 0.01, 0.0, 0.03, 0.01, 0.0, 0.15, 0.05, 0.0, 0.01, 0.0, 0.0, 0.0, 0.0, 0.0, 0.03, 0.06, 0.11, 0.32, 0.06, 0.03, 0.01, 0.01, 0.04, 0.06]$$

迭代$k = 2$之后的策略是



这些信念被用来确定根节点的动作以及其下的两个子树。这些子树是根据前一次迭代的树构建的。

示例27.2。此处展示的协同捕食者-猎物六边形世界问题的启发式搜索探索与条件计划扩展。捕食者为红色和绿色。猎物为蓝色。



© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

给定每个智能体 i 的固定节点集 X^i，初始信念 b 和初始联合节点 x_1，优化问题为

$$\begin{aligned}
\underset{U,\psi,\eta}{\text{maximize}} \quad & \sum_s b(s)U(\mathbf{x}_1, s) \\
\text{subject to} \quad & U(\mathbf{x}, s) = \sum_{\mathbf{a}} \prod_i \psi^i(a^i \mid x^i) \left(R(s, \mathbf{a}) + \gamma \sum_{s'} T(s' \mid s, \mathbf{a}) \sum_{\mathbf{o}} O(\mathbf{o} \mid \mathbf{a}, s') \sum_{\mathbf{x}'} \prod_i \eta^i(x^{i\prime} \mid x^i, a^i, o^i) U(\mathbf{x}', s') \right) \\
& \qquad \text{for all } \mathbf{x}, s \\
& \psi^i(a^i \mid x^i) \geq 0 \quad \text{for all } i, x^i, a^i \\
& \sum_a \psi^i(a^i \mid x^i) = 1 \quad \text{for all } i, x^i \\
& \eta^i(x^{i\prime} \mid x^i, a^i, o^i) \geq 0 \quad \text{for all } i, x^i, a^i, o^i, x^{i\prime} \\
& \sum_{x^{i\prime}} \eta^i(x^{i\prime} \mid x^i, a^i, o^i) = 1 \quad \text{for all } i, x^i, a^i, o^i
\end{aligned} \tag{27.6}$$

27.7 Summary

- Dec-POMDP 是完全合作的 POMG，用于建模一个智能体团队为共同目标协同工作，每个智能体仅使用本地信息独立行动。
- 由于确定信念状态是不可行的，正如在POMGs中那样，策略通常被表示为条件计划或控制器，使每个代理能够将单个观测序列映射到单个动作。
- Dec-POMDP 存在许多子类，具有不同程度的计算复杂度。
- 动态规划通过迭代计算价值函数，在迭代过程中使用线性规划修剪被支配的策略。
- 迭代最佳响应每次为单个智能体计算最佳效用最大化响应策略，迭代收敛至联合均衡。
- 启发式搜索在每次迭代中，依据启发式方法搜索一个固定的策略子集。
- 非线性规划可用于生成固定大小的控制器。

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

```
struct DecPOMDPNonlinearProgramming
    b # initial belief
    ℓ # number of nodes for each agent
end

function tensorform(𝒫::DecPOMDP)
    ℐ, 𝒮, 𝒜, 𝒪, R, T, O = 𝒫.ℐ, 𝒫.𝒮, 𝒫.𝒜, 𝒫.𝒪, 𝒫.R, 𝒫.T, 𝒫.O
    ℐ′ = eachindex(ℐ)
    𝒮′ = eachindex(𝒮)
    𝒜′ = [eachindex(𝒜i) for 𝒜i in 𝒜]
    𝒪′ = [eachindex(𝒪i) for 𝒪i in 𝒪]
    R′ = [R(s,a) for s in 𝒮, a in joint(𝒜)]
    T′ = [T(s,a,s′) for s in 𝒮, a in joint(𝒜), s′ in 𝒮]
    O′ = [O(a,s′,o) for a in joint(𝒜), s′ in 𝒮, o in joint(𝒪)]
    return ℐ′, 𝒮′, 𝒜′, 𝒪′, R′, T′, O′
end

function solve(M::DecPOMDPNonlinearProgramming, 𝒫::DecPOMDP)
    𝒫, γ, b = 𝒫, 𝒫.γ, M.b
    ℐ, 𝒮, 𝒜, 𝒪, R, T, O = tensorform(𝒫)
    X = [collect(1:M.ℓ) for i in ℐ]
    jointX, joint𝒜, joint𝒪 = joint(X), joint(𝒜), joint(𝒪)
    x1 = jointX[1]
    model = Model(Ipopt.Optimizer)
    @variable(model, U[jointX,𝒮])
    @variable(model, ψ[i=ℐ,X[i],𝒜[i]] ≥ 0)
    @variable(model, η[i=ℐ,X[i],𝒜[i],𝒪[i],X[i]] ≥ 0)
    @objective(model, Max, b⋅U[x1,:])
    @NLconstraint(model, [x=jointX,s=𝒮],
        U[x,s] == (sum(prod(ψ[i,x[i],a[i]] for i in ℐ)
                   *(R[s,y] + γ*sum(T[s,y,s′]*sum(O[y,s′,z]
                       *sum(prod(η[i,x[i],a[i],o[i],x′[i]] for i in ℐ)
                               *U[x′,s′] for x′ in jointX)
                       for (z, o) in enumerate(joint𝒪)) for s′ in 𝒮))
                    for (y, a) in enumerate(joint𝒜))))
    @constraint(model, [i=ℐ,xi=X[i]],
                sum(ψ[i,xi,ai] for ai in 𝒜[i]) == 1)
    @constraint(model, [i=ℐ,xi=X[i],ai=𝒜[i],oi=𝒪[i]],
                sum(η[i,xi,ai,oi,xi′] for xi′ in X[i]) == 1)
    optimize!(model)
    ψ′, η′ = value.(ψ), value.(η)
    return [ControllerPolicy(𝒫, X[i],
            Dict((xi,𝒫.𝒜[i][ai]) => ψ′[i,xi,ai]
                 for xi in X[i], ai in 𝒜[i]),
            Dict((xi,𝒫.𝒜[i][ai],𝒫.𝒪[i][oi],xi′) => η′[i,xi,ai,oi,xi′]
                 for xi in X[i], ai in 𝒜[i], oi in 𝒪[i], xi′ in X[i]))
        for i in ℐ]
end
```

算法27.5。NLP计算给定初始信念b和每个智能体的控制器节点数ℓ下，针对Dec-POMDP $\mathcal{P}$的最优联合控制器策略π。这推广了算法23.5中的NLP解法。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

27.8 Exercises

练习 27.1。为什么具有联合完全可观测性的 Dec-MDP 与智能体了解状态有所不同?

Solution: 完全联合可观测性意味着如果智能体共享各自的观测，那么团队将知道真实状态。这可以在规划期间离线完成。因此在Dec-MDP中，真实状态在规划期间本质上是已知的。问题在于它要求智能体共享各自的观测，而这在执行期间无法在线完成。因此，规划仍然需要对其他智能体所做的不确定观测进行推理。

练习27.2。为具有转移、观测和奖励独立性的Dec-MDP提出一种快速算法，并证明其正确性。

Solution: 如果一个因子化的Dec-MDP满足所有三个独立性假设，那么我们可以将其作为$|\mathcal{I}|$个独立的MDP来求解。每个智能体i的MDP所得到的策略π^i随后可以组合起来形成最优联合策略。为了证明这一事实，考虑每个智能体各自的MDP的效用：

$$U^{\pi^i}(s^i) = R\left(s^i, \pi^i()\right) + \gamma\left[\sum_{s^{i\prime}} T^i\left(s^{i\prime} \mid s^i, \pi^i()\right) \sum_{o^i} O^i\left(o^i \mid \pi^i(), s^{i\prime}\right) U^{\pi^i(o^i)}(s^{i\prime})\right]$$

如同公式 (26.1) 所示，$\pi^i()$ 指的是 i 的条件计划的根动作，而 $\pi^i(o^i)$ 指的是 i 在做出观察 o^i 后的子计划。我们将它们各自的贡献汇总如下：

$$\sum_i U^{\pi^i}(s) = \sum_i \left[R\left(s^i, \pi^i()\right) + \gamma\left[\sum_{s^{i\prime}} T^i\left(s^{i\prime} \mid s^i, \pi^i()\right) \sum_{o^i} O^i\left(o^i \mid \pi^i(), s^{i\prime}\right) U^{\pi^i(o^i)}(s^{i\prime})\right]\right]$$

我们可以将 T^i 和 O^i 合并为一个概率分布 P，移动求和符号，并应用奖励独立性的定义：

$$\begin{aligned}
\sum_i U^{\pi^i}(s) &= \sum_i \left[R\left(s^i, \pi^i()\right) + \gamma\left[\sum_{s^{i\prime}} P\left(s^{i\prime} \mid s^i, \pi^i()\right) \sum_{o^i} P\left(o^i \mid \pi^i(), s^{i\prime}\right) U^{\pi^i(o^i)}(s^{i\prime})\right]\right] \\
&= \sum_i R\left(s^i, \pi^i()\right) + \sum_i \left[\gamma\left[\sum_{s^{i\prime}} P\left(s^{i\prime} \mid s^i, \pi^i()\right) \sum_{o^i} P\left(o^i \mid \pi^i(), s^{i\prime}\right) U^{\pi^i(o^i)}(s^{i\prime})\right]\right] \\
&= R(s, \pi()) + \sum_i \left[\gamma\left[\sum_{s^{i\prime}} P\left(s^{i\prime} \mid s^i, \pi^i()\right) \sum_{o^i} P\left(o^i \mid \pi^i(), s^{i\prime}\right) U^{\pi^i(o^i)}(s^{i\prime})\right]\right]
\end{aligned}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

现在，我们对所有后继状态 s 和观测 o 进行边缘化。由于状态转移和观测的独立性，我们可以自由地将这些分布以其他非 i 状态和观测因子为条件，这等价于以 s 和 o 为条件。然后，我们可以应用状态转移和观测独立性的定义。最后，我们可以将求和移入并识别出 $U^{\boldsymbol{\pi}}(s)$ 结果：

$$\begin{aligned}
\sum_i U^{\pi^i}(s) &= R(s, \boldsymbol{\pi}()) + \sum_i \left[\gamma \left[\sum_{s'} P\left(s' \mid s^i, \pi^i()\right) \sum_o P\left(o \mid \pi^i(), s^{i\prime}\right) U^{\pi^i(o^i)}(s^{i\prime}) \right] \right] \\
&= R(s, \boldsymbol{\pi}()) + \sum_i \left[\gamma \left[\sum_{s'} P\left(s^{0\prime} \mid s^0\right) \prod_j P\left(s^{j\prime} \mid s^i, \pi^i()\right) \sum_o \prod_j P\left(o^j \mid \pi^i(), s^{i\prime}\right) U^{\pi^i(o^i)}(s^{i\prime}) \right] \right] \\
&= R(s, \boldsymbol{\pi}()) + \sum_i \left[\gamma \left[\sum_{s'} P\left(s^{0\prime} \mid s^0\right) \prod_j P\left(s^{j\prime} \mid s, \pi()\right) \sum_o \prod_j P\left(o^j \mid \pi(), s'\right) U^{\pi^i(o^i)}(s^{i\prime}) \right] \right] \\
&= R(s, \boldsymbol{\pi}()) + \sum_i \left[\gamma \left[\sum_{s'} T\left(s' \mid s, \boldsymbol{\pi}()\right) \sum_o O\left(o \mid \boldsymbol{\pi}(), s'\right) U^{\pi^i(o^i)}(s^{i\prime}) \right] \right] \\
&= R(s, \boldsymbol{\pi}()) + \gamma \left[\sum_{s'} T\left(s' \mid s, \boldsymbol{\pi}()\right) \sum_o O\left(o \mid \boldsymbol{\pi}(), s'\right) \left[\sum_i U^{\pi^i(o^i)}(s^{i\prime}) \right] \right] \\
&= R(s, \boldsymbol{\pi}()) + \gamma \left[\sum_{s'} T\left(s' \mid s, \boldsymbol{\pi}()\right) \sum_o O\left(o \mid \boldsymbol{\pi}(), s'\right) U^{\boldsymbol{\pi}(o)}(s') \right] \\
&= U^{\boldsymbol{\pi}}(s)
\end{aligned}$$

这是从公式(26.1)推导出的Dec-MDP效用函数，完成了证明。

练习 27.3. 在Dec-POMDP启发式搜索中，我们如何利用MMDP或MPOMDP作为启发式方法?

Solution: 我们可以假设规划过程中存在自由通信。在每个时间步 t，所有智能体都知道 a_t 和 o_t，这使我们能够维护一个多智能体信念 b_t，从而得到一个 MPOMDP。这个 MPOMDP 解可以作为启发式方法来指导策略树的搜索。或者，我们创建一个启发式方法，假设真实状态和联合动作是已知的。这导致一个 MMDP，它也可以用作启发式方法。这些假设仅用于规划。执行仍然是一个 Dec-POMDP，其中智能体在没有自由通信的情况下接收个体观测。两种启发式方法都会产生一个联合策略 $\hat{\boldsymbol{\pi}}$ 用于启发式探索。

练习27.4。我们如何计算最佳响应控制器？描述一下这如何在迭代最佳响应中使用。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Solution: 对于代理 i，最佳响应控制器 X^i、ψ^i 和 η^i 可以通过求解一个非线性程序来计算。该程序与第27.6节中给出的类似，不同之处在于 $\mathbf{X}^{-i}$、$\boldsymbol{\psi}^{-i}$ 和 $\boldsymbol{\eta}^{-i}$ 现在是给定的，不再作为变量：

$$\begin{aligned}
\underset{U,\psi^i,\eta^i}{\text{maximize}} \quad & \sum_s b(s) U(\mathbf{x}_1, s) \\
\text{subject to} \quad & U(\mathbf{x}, s) = \sum_{\mathbf{a}} \prod_i \psi^i(a^i \mid x^i) \left(R(s, \mathbf{a}) + \gamma \sum_{s'} T(s' \mid s, \mathbf{a}) \sum_{\mathbf{o}} O(\mathbf{o} \mid \mathbf{a}, s') \sum_{\mathbf{x}'} \prod_i \eta^i(x^{i\prime} \mid x^i, a^i, o^i) U(\mathbf{x}', s') \right) \\
& \qquad \text{for all } \mathbf{x}, s \\
& \psi^i(a^i \mid x^i) \geq 0 \quad \text{for all } x^i, a^i \\
& \sum_a \psi^i(a^i \mid x^i) = 1 \quad \text{for all } x^i \\
& \eta^i(x^{i\prime} \mid x^i, a^i, o^i) \geq 0 \quad \text{for all } x^i, a^i, o^i, x^{i\prime} \\
& \sum_{x^{i\prime}} \eta^i(x^{i\prime} \mid x^i, a^i, o^i) = 1 \quad \text{for all } x^i, a^i, o^i
\end{aligned}$$

将算法27.3适应于控制器策略，此程序替换了内部最佳响应操作。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

APPENDICES

A Mathematical Concepts

本附录简要概述了本书中使用的一些数学概念。

A.1 Measure Spaces

在引入测度空间的定义之前，我们将首先讨论集合 Ω 上的σ-代数的概念。σ-代数是一个集合 Σ，由 Ω 的子集组成，满足

1. $\Omega \in \Sigma$。

2. 如果 $E \in \Sigma$，那么 $\Omega \setminus E \in \Sigma$ (*closed under complementation*)。

3. 如果 E_1、E_2、E_3、...、$\in \Sigma$，那么 $E_1 \cup E_2 \cup E_3 \ldots \in \Sigma$ (*closed under countable unions*)。

一个元素 $E \in \Sigma$ 被称为 *measurable set*。

一个 *measure space* 由一个集合 Ω、一个 sigma-代数 Σ 以及一个 *measure* μ：$\Omega \to \mathbb{R} \cup \{\infty\}$ 定义。为了使 μ 成为一个测度，以下性质必须成立：

1. 如果$E \in \Sigma$，则$\mu(E) \geq 0$（*nonnegativity*）。

2. $\mu(\emptyset) = 0$.

3. 如果 E_1、E_2、E_3、... $\in \Sigma$ 两两不相交，则 $\mu(E_1 \cup E_2 \cup E_3 \ldots) = \mu(E_1) + \mu(E_2) + \mu(E_3) + \cdots$ (*countable additivity*)。

A.2 Probability Spaces

一个 *probability space* 是一个测度空间 (Ω, Σ, μ)，并要求 $\mu(\Omega) = 1$。在概率空间的语境中，Ω 被称为 *sample space*，Σ 被称为 *event space*，而 μ（或者更常见的是 P）被称为 *probability measure*。*probability axioms*[1] 指的是测度的非负性和可数可加性性质。

空格，连同要求 $\mu(\Omega) = 1$。

1 这些公理有时被称为 *Kolmorogov axioms*。A. 柯尔莫哥洛夫，*Foundations of the Theory of Probability*，第2版，切尔西出版社，1956年。

A.3 Metric Spaces

一个带有 *metric* 的集合被称为 *metric space*。一个度量 d，有时称为 *distance metric*，是一个将 X 中的元素对映射到非负实数的函数，使得对于所有 x、y、$z \in X$：

1. $d(x, y) =$ 为 0 当且仅当 $x = y$ (*identity of indiscernibles*)。
2. $d(x, y) = d(y, x)$ (*symmetry*)。
3. $d(x, y) \leq d(x, z) + d(z, y)$ (*triangle inequality*)。

A.4 Normed Vector Spaces

一个 *normed vector space* 由一个 *vector space* X 和一个范数 $\|\cdot\|$ 组成，该范数将 X 的元素映射到非负实数，使得对于所有标量 α 和向量 $\mathrm{x}, \mathrm{y} \in X$：

1. 当且仅当$\mathrm{x} = 0$时，$\|\mathrm{x}\| = 0$。
2. $\|\alpha \mathrm{x}\| = |\alpha| \|\mathrm{x}\|$ (*absolutely homogeneous*).
3. $\|\mathrm{x} + \mathrm{y}\| \leq \|x\| + \|y\|$ (*triangle inequality*)。

L_p 范数是一组常用的范数，由标量 $p \geq 1$ 参数化。向量 x 的 L_p 范数是

$$\|\mathbf{x}\|_p = \lim_{\rho \to p} (|x_1|^\rho + |x_2|^\rho + \cdots + |x_n|^\rho)^{\frac{1}{\rho}} \tag{A.1}$$

其中该极限对于定义无穷范数是必要的，L_∞。图 A.1 中展示了几个 L_p 范数。

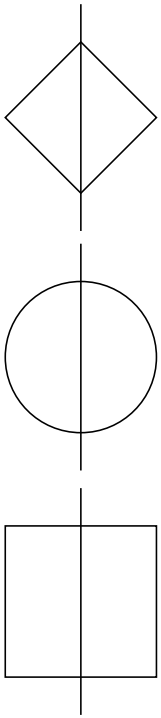
可以通过定义度量 $d(\mathrm{x}, \mathrm{y}) = \|\mathrm{x} - \mathrm{y}\|$，利用范数在向量空间中引入距离度量。例如，我们可以使用L_p范数来定义距离。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

L_1: $\|\mathrm{x}\|_1 = |x_1| + |x_2| + \cdots + |x_n|$ 这一指标通常被称为 *taxicab norm*。

L_2: $\|\mathrm{x}\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$ 该指标通常被称为 *Euclidean norm*。

L_∞: $\|\mathrm{x}\|_\infty = \max(|x_1|, |x_2|, \cdots, |x_n|)$ 这个度量通常被称为 *max norm*、*Chebyshev norm* 或 *chessboard norm*。后一个名称来源于国际象棋中国王在两个方格之间移动所需的最少步数。



图A.1. 常见L_p范数。图示展示了二维空间中范数轮廓的形状。在该范数下，轮廓上的所有点到原点的距离相等。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

A.5 Positive Definiteness

一个对称矩阵 A 是 *positive definite*，如果对于除原点以外的所有点，$x^\top Ax$ 都是正的。换句话说，对于所有 $x \neq 0$，都有 $x^\top Ax > 0$。一个对称矩阵 A 是 *positive semidefinite*，如果 $x^\top Ax$ 始终非负。换句话说，对于所有 x，都有 $x^\top Ax \geq 0$。

A.6 Convexity

两个向量 x 和 y 的 *convex combination* 是结果 的

$$\alpha \mathbf{x} + (1 - \alpha)\mathbf{y} \tag{A.2}$$

对于某个 $\alpha \in [0, 1]$，凸组合可以由 m 个向量构成：

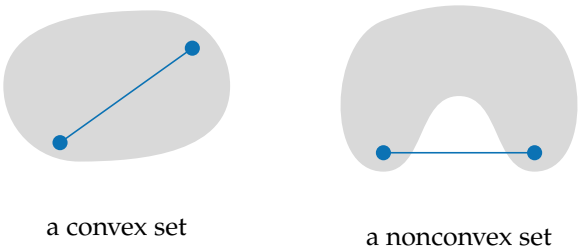
$$w_1 \mathbf{v}^{(1)} + w_2 \mathbf{v}^{(2)} + \cdots + w_m \mathbf{v}^{(m)} \tag{A.3}$$

其非负权重 w 之和为 1。

一个 *convex set* 是一个集合，其中集合内任意两点之间的连线完全位于该集合内。在数学上，如果满足以下条件，集合 $\mathcal{S}$ 是凸的：

$$\alpha \mathbf{x} + (1 - \alpha)\mathbf{y} \in \mathcal{S} \tag{A.4}$$

对于所有x、y属于$\mathcal{S}$，以及对于所有α属于[0, 1]。图A.2展示了一个凸集和一个非凸集。

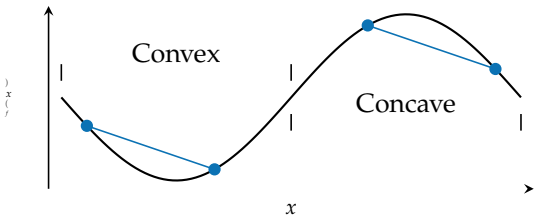


图A.2. 凸集与非凸集。

一个 *convex function* 是一个定义域为凸集的 *bowl-shaped* 函数。所谓“碗状”，我们指的是这样一个函数：在其定义域内任意两点间画的直线都不会位于函数图像下方。一个函数 f 在凸集 $\mathcal{S}$ 上是凸的，如果对于 $\mathcal{S}$ 中的所有 x、y 以及 [0, 1] 中的所有 α，

$$f(\alpha \mathbf{x} + (1 - \alpha)\mathbf{y}) \leq \alpha f(\mathbf{x}) + (1 - \alpha) f(\mathbf{y}) \tag{A.5}$$

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com



图A.3. 函数的凸和非凸部分。

函数的凸区域和凹区域如图A.3所示。

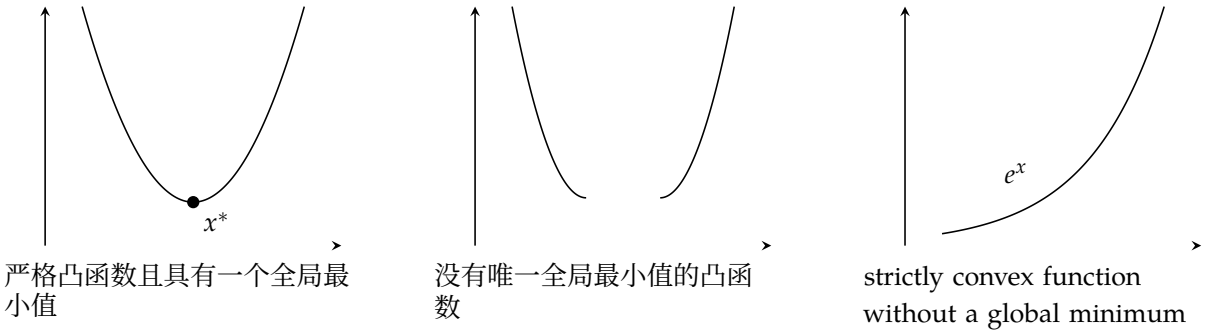
函数 f 在凸集 $\mathcal{S}$ 上是 *strictly convex* 的，如果对于 $\mathcal{S}$ 中的所有 x、y 以及 (0, 1) 中的 α，

$$f(\alpha\mathbf{x} + (1-\alpha)\mathbf{y}) < \alpha f(\mathbf{x}) + (1-\alpha)f(\mathbf{y}) \tag{A.6}$$

严格凸函数至多有一个最小值，而凸函数可能有平坦区域。[2] 严格凸性和非严格凸性的例子如图所示

图 A.4。

vex Optimization



图A.4。并非所有凸函数都有单个全局最小值。

一个函数 f 是 *concave*，如果 $-f$ 是凸的。此外，如果 $-f$ 是严格凸的，那么 f 是 *strictly concave*。

A.7 Information Content

如果我们有一个离散分布，它将概率 $P(x)$ 分配给值 x，则观察到 x 的 *information content*[3] 由下式给出

$$I(x) = -\log P(x) \tag{A.7}$$

2 凸函数的优化是S. Boyd和L. Vandenberghe所著教科书《凸优化》的主题，*Con-*。剑桥大学出版社，2004年。

3 有时信息内容被称为 *Shannon information*，以纪念信息论领域的创始人克劳德·香农。C. E. Shannon，“A Mathematical Theory of Communication，”*Bell System Technical Journal*，第27卷，第4期，第623–656页，1948年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

信息内容的单位取决于对数的底数。我们通常假设自然对数（以 e 为底），此时单位为 *nat*，它是 *natural* 的缩写。在信息论语境中，底数通常为 2，此时单位为 *bit*。我们可以将这一量视为：当消息分布服从指定分布时，按照最优消息编码传输值 x 所需的比特数。

A.8 Entropy

Entropy 是一种信息论中的不确定性度量。与离散随机变量 X 相关联的熵是期望信息量：

$$H(X) = \mathbb{E}_x[I(x)] = \sum_x P(x)I(x) = -\sum_x P(x)\log P(x) \tag{A.8}$$

其中 $P(x)$ 是分配给 x 的质量。

对于连续分布，其中 $p(x)$ 是分配给 x 的密度，*differential entropy* (也称为 *continuous entropy*)，被定义为

$$h(X) = \int p(x)I(x)\,dx = -\int p(x)\log p(x)\,\mathrm{d}x \tag{A.9}$$

A.9 Cross Entropy

一个分布相对于另一个分布的*cross entropy*可以通过期望信息内容来定义。如果我们有一个质量函数为$P(x)$的离散分布和另一个质量函数为$Q(x)$的分布，那么P相对于Q的交叉熵为

$$H(P,Q) = -\mathbb{E}_{x\sim P}[\log Q(x)] = -\sum_x P(x)\log Q(x) \tag{A.10}$$

对于具有密度函数 $p(x)$ 和 $q(x)$ 的连续分布，我们有

$$H(p,q) = -\int p(x)\log q(x)\,\mathrm{d}x \tag{A.11}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

A.10 Relative Entropy

Relative entropy，也称为*Kullback-Leibler* (*KL*) *divergence*，是衡量一个概率分布与参考分布差异程度的指标。[4] 如果 $P(x)$ 并且 $Q(x)$ 是质量函数时，从 Q 到 P 的 KL 散度是对数差的期望，其中期望使用 P:

$$D_{\text{KL}}(P \mid\mid Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)} = -\sum_x P(x) \log \frac{Q(x)}{P(x)} \tag{A.12}$$

该量仅当 P 的支撑集是 Q 的支撑集的子集时才有定义。求和是在 P 的支撑集上进行的，以避免除以零。

对于具有密度函数 $p(x)$ 和 $q(x)$ 的连续分布，我们有

$$D_{\text{KL}}(p \mid\mid q) = \int p(x) \log \frac{p(x)}{q(x)} \, \mathrm{d}x = -\int p(x) \log \frac{q(x)}{p(x)} \, \mathrm{d}x \tag{A.13}$$

类似地，仅当p的支撑集是q支撑集的子集时，该量才有定义。积分是在p的支撑集上进行的，以避免除以零。

4 以引入该度量的两位美国数学家所罗门·库尔贝克（1907—1994）和理查德·A·莱布勒（1914—2003）命名。S. Kullback and R. A. Leibler, "On Information and Sufficiency," *Annals of Mathematical Statistics*, vol. 22, no. 1, pp. 79–86, 1951. S. Kullback, *Information Theory and Statistics*. Wiley, 1959.

A.11 Gradient Ascent

Gradient ascent 是一种用于尝试最大化函数 $f(\mathrm{x})$ 的通用方法，其中 f 是可微函数。我们从点 x 开始，迭代地应用以下更新规则：

$$\mathbf{x} \leftarrow \mathbf{x} + \alpha \nabla f(\mathbf{x}) \tag{A.14}$$

其中 $\alpha > 0$ 被称为 *step factor*。这种优化方法的思想是，我们沿着梯度方向逐步前进，直到达到局部最大值。使用这种方法无法保证找到全局最大值。较小的 α 值通常需要更多迭代才能接近局部最大值。较大的 α 值往往会导致在局部最优附近震荡而无法真正达到。如果 α 在迭代过程中保持不变，有时被称为 *learning rate*。许多应用涉及 *decaying step factor*，其中除了在每次迭代时更新 x 外，我们还根据以下公式更新 α

$$\alpha \leftarrow \gamma \alpha \tag{A.15}$$

其中 $0 < \gamma < 1$ 是 *decay factor*。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

A.12 Taylor Expansion

Taylor expansion、[5]（也称为*Taylor series*），对于函数来说很重要本书中使用的许多近似值。从*first fundamental theorem of calculus*、[6] 我们知道

$$f(x+h) = f(x) + \int_0^h f'(x+a)\,\mathrm{d}a \tag{A.16}$$

将此定义嵌套会产生关于x的f的泰勒展开：

$$f(x+h) = f(x) + \int_0^h \left(f'(x) + \int_0^a f''(x+b)\,\mathrm{d}b \right) \mathrm{d}a \tag{A.17}$$

$$= f(x) + f'(x)h + \int_0^h \int_0^a f''(x+b)\,\mathrm{d}b\,\mathrm{d}a \tag{A.18}$$

$$= f(x) + f'(x)h + \int_0^h \int_0^a \left(f''(x) + \int_0^b f'''(x+c)\,\mathrm{d}c \right) \mathrm{d}b\,\mathrm{d}a \tag{A.19}$$

$$= f(x) + f'(x)h + \frac{f''(x)}{2!}h^2 + \int_0^h \int_0^a \int_0^b f'''(x+c)\,\mathrm{d}c\,\mathrm{d}b\,\mathrm{d}a \tag{A.20}$$

您是一位专业、真实的机器翻译引擎。仅输出翻译后的文本，不包含任何其他文本。将以下 Markdow

$$\tag{A.21}$$

$$= f(x) + \frac{f'(x)}{1!}h + \frac{f''(x)}{2!}h^2 + \frac{f'''(x)}{3!}h^3 + \ldots \tag{A.22}$$

$$= \sum_{n=0}^{\infty} \frac{f^{(n)}(x)}{n!} h^n \tag{A.23}$$

在这里给出的公式中，x通常是固定的，函数根据h来评估。将$f(x)$在点a处进行泰勒展开，使其保持为x的函数，往往更为方便：

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!}(x-a)^n \tag{A.24}$$

泰勒展开将函数表示为基于单点重复导数的多项式项的无穷和。任何解析函数都可以在其局部邻域内由其泰勒展开表示。

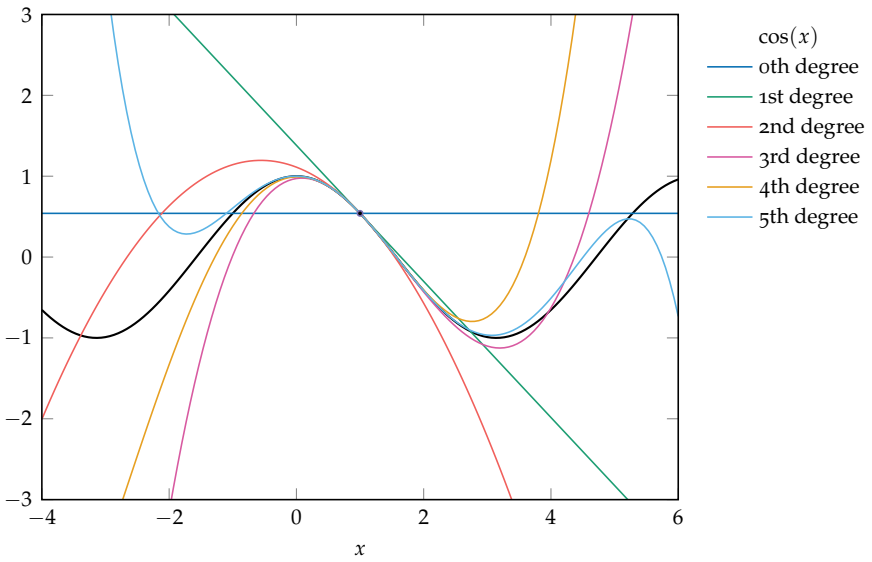
函数可以通过使用泰勒展开的前几项来进行局部近似。图A.5展示了关于x ==1的cos(x)的越来越好的近似。包含更多的项可以提高局部近似的精度，但随着远离展开点，误差仍然会累积。

5 以英国数学家布鲁克·泰勒（1685–1731）命名，他引入了这一概念。

6 微积分第一基本定理将函数与其导数的积分联系起来：

$$f(b) - f(a) = \int_a^b f'(x)\,\mathrm{d}x$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com



图A.5. 基于泰勒展开的前n项，cos(x)在1附近的逐次逼近。

一个线性 *Taylor approximation* 使用泰勒展开的前两项：

$$f(x) \approx f(a) + f'(a)(x - a) \tag{A.25}$$

A 二次泰勒近似使用前三项 毫秒：

$$f(x) \approx f(a) + f'(a)(x - a) + \frac{1}{2}f''(a)(x - a)^2 \tag{A.26}$$

等等。

在多维情况下，关于a的泰勒展开推广为

$$f(\mathbf{x}) = f(\mathbf{a}) + \nabla f(\mathbf{a})^\top (\mathbf{x} - \mathbf{a}) + \frac{1}{2}(\mathbf{x} - \mathbf{a})^\top \nabla^2 f(\mathbf{a})(\mathbf{x} - \mathbf{a}) + \ldots \tag{A.27}$$

前两项构成点a处的切平面。第三项包含了局部曲率。本书仅会使用此处展示的前三项。

A.13 Monte Carlo Estimation

Monte Carlo estimation 允许我们在输入 x 服从概率密度函数 p 时，计算函数 f 的期望值：

$$\mathbb{E}_{x \sim p}[f(x)] = \int f(x)p(x)\,\mathrm{d}x \approx \frac{1}{n}\sum_i f(x^{(i)}) \tag{A.28}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

其中 $x^{(1)}, \ldots, x^{(n)}$ 从 p 中抽取。估计的方差等于 $\text{Var}_{x\sim p}[f(x)]/n$。

A.14 Importance Sampling

Importance sampling 允许我们从从不同分布 q 中抽取的样本计算 $\mathbb{E}_{x\sim p}[f(x)]$：

$$\begin{aligned}\mathbb{E}_{x\sim p}[f(x)] &= \int f(x)p(x)\,\mathrm{d}x && \text{(A.29)}\\ &= \int f(x)p(x)\frac{q(x)}{q(x)}\,\mathrm{d}x && \text{(A.30)}\\ &= \int f(x)\frac{p(x)}{q(x)}q(x)\,\mathrm{d}x && \text{(A.31)}\\ &= \mathbb{E}_{x\sim q}\left[f(x)\frac{p(x)}{q(x)}\right] && \text{(A.32)}\end{aligned}$$

以上方程可以使用从 q 中抽取的样本 $x^{(1)}$、...、$x^{(n)}$ 来近似：

$$\mathbb{E}_{x\sim p}[f(x)] = \mathbb{E}_{x\sim q}\left[f(x)\frac{p(x)}{q(x)}\right] \approx \frac{1}{n}\sum_i f(x^{(i)})\frac{p(x^{(i)})}{q(x^{(i)})} \qquad \text{(A.33)}$$

A.15 Contraction Mappings

一个 *contraction mapping* f 是相对于度量空间上的函数来定义的，使得

$$d(f(x), f(y)) \leq \alpha d(x, y) \qquad \text{(A.34)}$$

其中 d 是与度量空间相关的距离度量，且 $0 \leq \alpha < 1$。因此，压缩映射会减少集合中任意两个成员之间的距离。这样的函数有时被称为 *contraction* 或 *contractor*。

重复应用压缩映射的一个结果是集合中任意两个成员之间的距离趋近于0。*contraction mapping theorem*或*Banach fixed-point theorem*[7]表明每个压缩映射

在一个完备的、[8]非空度量空间上具有唯一不动点。此外，
对于该集合中的任意元素 x，对该元素重复应用压缩映射会收敛到该不动点。

7 以首次陈述该定理的波兰数学家斯特凡·巴拿赫（1892–1945）命名。

8 完备度量空间是指其中每个柯西序列都收敛到该空间中的一点的度量空间。一个序列 x_1，x_2，... 是柯西的，如果对于每个正实数 $\epsilon > 0$，存在一个正整数 n，使得对于所有正整数 i，$j > n$，我们有 $d(x_i, x_j) < \epsilon$。

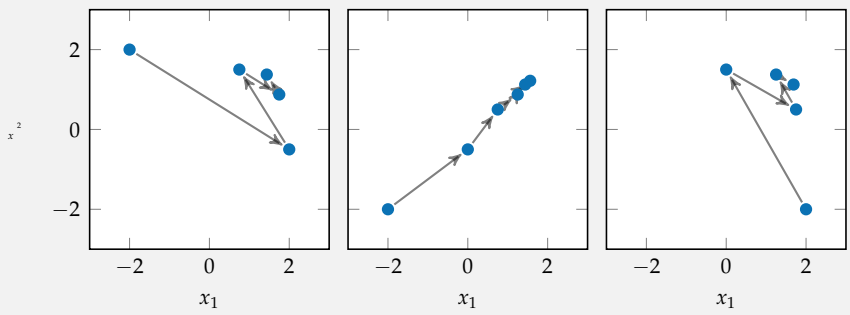
© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

证明函数 f 在度量空间上是压缩映射，在与前面介绍的概念相关的各种收敛证明中都很有用。例如，我们可以证明贝尔曼算子在值函数空间上关于最大范数是压缩映射。压缩映射定理的应用使我们能够证明，贝尔曼算子的重复应用会导致收敛到唯一的值函数。例 A.1 展示了一个简单的压缩映射。

考虑函数 $\mathrm{f}(\mathrm{x}) = [x_2/2 + 1, x_1/2 + 1/2]$。我们可以证明，对于集合 $\mathbb{R}^2$ 和欧几里得距离函数，f 是一个压缩映射：

$$\begin{aligned} d(\mathbf{f}(\mathbf{x}), \mathbf{f}(\mathbf{y})) &= \|\mathbf{f}(\mathbf{x}) - \mathbf{f}(\mathbf{y})\|_2 \\ &= \|[x_2/2 + 1, x_1/2 + 1/2] - [y_2/2 + 1, y_1/2 + 1/2]\|_2 \\ &= \|[\frac{1}{2}(x_2 - y_2), \frac{1}{2}(x_1 - y_1)]\|_2 \\ &= \frac{1}{2}\|[(x_2 - y_2), (x_1 - y_1)]\|_2 \\ &= \frac{1}{2} d(\mathbf{x}, \mathbf{y}) \end{aligned}$$

我们可以绘制 f 重复作用于 $\mathbb{R}^2$ 中的点的效果，并展示它们如何收敛到 [5/3, 4/3]：

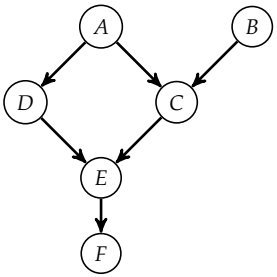


示例 A.1. 对于 $\mathbb{R}^2$ 的压缩映射。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

A.16 Graphs

一个 *graph* $G = (V, E)$ 由一组 *nodes* (定义，也称为 *vertices*) V 和 *edges* E。图 A.6 展示了一个图的示例。一条边 $e \in E$ 是一对节点（v_i、v_j）。我们主要关注 *directed graphs*，其中边是有向的并定义了父子关系。一条边 $e = (v_i, v_j)$ 通常以图形方式表示为从 v_i 指向 v_j 的箭头，其中 v_i 为 *parent*，v_j 为 *child*。如果存在一条连接 v_i 和 v_j 的边，那么我们说 v_i 和 v_j 是 *neighbors*。节点 v_i 的所有父节点的集合记作 $\text{Pa}(v_i)$。



图A.6。一个图的示例。在此，Pa(*C*) = {*A*，*B*}。序列(*A*，*C*，*E*，*F*)是一条有向路径，而(*A*，*C*，*B*)则是一条无向路径。节点*A*是*C*和*D*的父节点。节点*E*是*B*的后代。*C*的邻居包括*A*、*B*和*E*。

从节点v_i到节点v_j的*path*是连接v_i到v_j的一系列边。如果这条路径可以沿着边的方向从一个节点到另一个节点，那么我们称之为*directed path*。*undirected path*是不考虑边方向的路径。如果存在从v_i到v_j的有向路径，则节点v_j是v_i的后代。*cycle*是从一个节点到其自身的有向路径。如果一个图不包含任何环，则它是*acyclic*。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

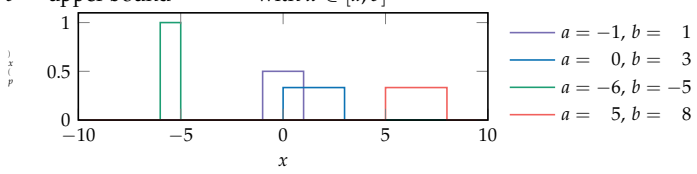
B Probability Distributions

本附录总结了与本书所介绍主题相关的几类概率分布。[1]这些分布通过以下方式之一表示：

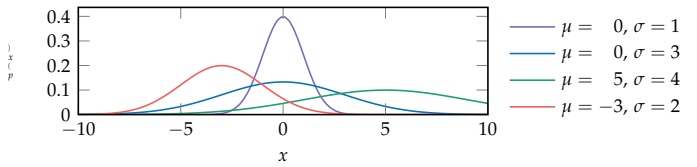
概率质量函数或概率密度函数，以及相关函数均被提供，还包括控制每种分布的参数。图表展示了各种参数如何影响分布。索引中包含这些分布在书中正文使用处的页码引用。有些分布是 *univariate*，意味着它们是标量变量上的分布；其他的是 *multivariate*，意味着它们是多个变量上的分布。

1 这些分布已在 `Distributions.jl` 中实现。M. Besançon, T. Papamarkou, D. Anthoff, A. Arslan, S. Byrne, D. Lin, 和 J. Pearson, "Distributions.jl: JuliaStats生态系统中的概率分布定义与建模," 2019. arXiv: 1907.08611v1。

Name	Parameters		Distribution Function
Uniform $\mathcal{U}(a,b)$	a b	lower bound upper bound	$p(x) = \frac{1}{b-a}$ with $x \in [a,b]$



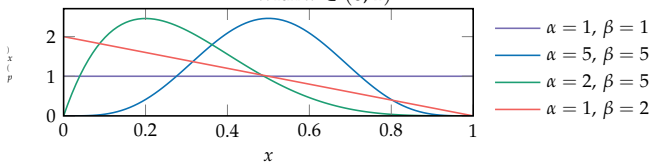
Gaussian **(univariate)** $\mathcal{N}(\mu,\sigma^2)$	μ σ^2	mean variance	$p(x) = \frac{1}{\sigma}\phi\left(\frac{x-\mu}{\sigma}\right)$ where $\phi(x) = \frac{1}{\sqrt{2\pi}}\exp(-x^2/2)$ with $x \in \mathbb{R}$



Beta
Beta(α, β)

$\alpha > 0$ shape
$\beta > 0$ shape

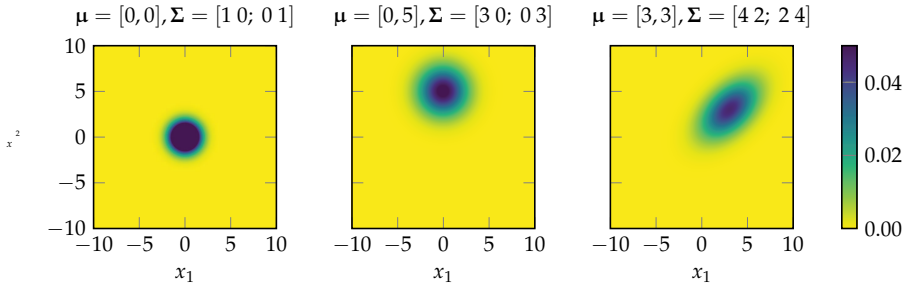
$p(x) = \frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} x^{\alpha-1}(1-x)^{\beta-1}$
with $x \in (0, 1)$



Gaussian (multivariate)
$\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$

$\boldsymbol{\mu}$ mean
$\boldsymbol{\Sigma}$ covariance

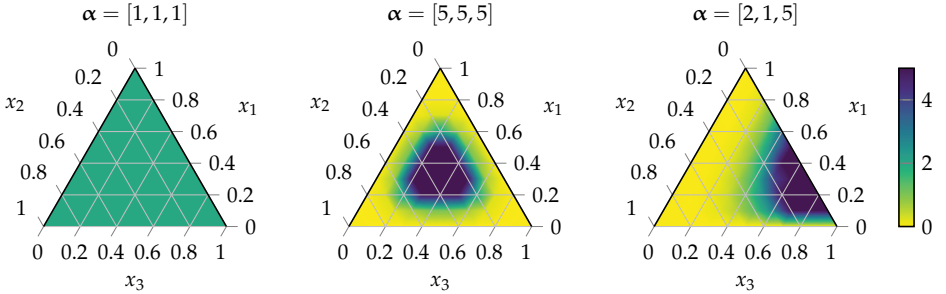
$p(\mathbf{x}) = \frac{1}{(2\pi)^{n/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right)$ where $n = \dim(\mathbf{x})$
with $\mathbf{x} \in \mathbb{R}^n$



Dirichlet
Dir$(\boldsymbol{\alpha})$

$\boldsymbol{\alpha} > 0$ concentration

$p(\mathbf{x}) = \frac{\Gamma(\alpha_0)}{\prod_{i=1}^n \Gamma(\alpha_i)} \prod_{i=1}^n x_i^{\alpha_i - 1}$ where $\alpha_0 = \sum_i \alpha_i$
with $x_i \in (0, 1)$ and $\sum_i x_i = 1$



© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

C Computational Complexity

在讨论各种算法时，分析它们的*computational complexity*是有用的，这指的是将它们运行至完成所需的资源。[1]我们通常关注的是时间或空间复杂度。本附录回顾了渐近记号，这是通常用于刻画复杂度的方法。随后，我们回顾了与书中算法相关的几个复杂度类，并讨论了可判定性问题。

1 算法分析代表了计算机科学中的一个广阔领域。对于入门教科书，参见O. Goldreich，*Computational Complexity: A Conceptual Perspective*，剑桥大学出版社，2008年。严格的论述需要引入概念和计算模型，例如*Turing machines*，我们在此讨论中将略过这些内容。

C.1 Asymptotic Notation

Asymptotic notation 常用于刻画函数的增长。这种记号有时被称为*big-Oh notation*，之所以使用字母 O，是因为函数的增长率通常被称为其 *order*。这种记号可用于描述与数值方法相关的误差，或算法的时间或空间复杂度。当自变量趋向某个值时，这种记号给出了函数的一个上界。

数学上，如果 $f(x) = O(g(x))$ 作为 $x \to a$，那么对于足够接近 a 的 x 值，$f(x)$ 的绝对值被 $g(x)$ 的绝对值乘以某个正有限 c 所界定：

$$|f(x)| \leq c|g(x)| \quad \text{for } x \to a \tag{C.1}$$

将 $f(x) = O(g(x))$ 写为等于号是一种常见的滥用。例如，$x^2 = O(x^2)$ 和 $2x^2 = O(x^2)$，但当然，$x^2 \neq 2x^2$。在一些数学文本中，$O(g(x))$ 表示所有增长速度不超过 $g(x)$ 的函数集合。例如，$5x^2 \in O(x^2)$。例 C.1 演示了渐近记号。

如果 $f(x)$ 是项的 *linear combination*[2]，那么 $O(f)$ 对应于增长最快的项。示例 C.2 比较了几个项的阶。

2 线性组合是项的加权和。如果这些项在向量 x 中，那么线性组合为 $w_1x_1 + w_2x_2 + \cdots = \mathbf{w}^\top \mathbf{x}$。

示例 C.1. 常数乘以函数的渐近符号表示。

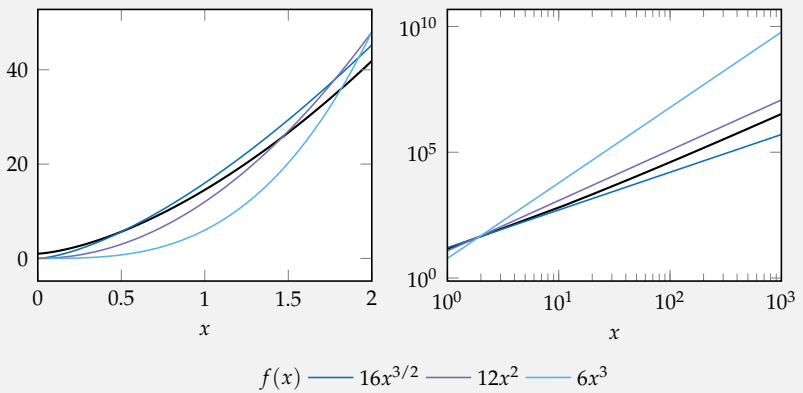
将 $f(x) = 10^6 e^x$ 视为 $x \to \infty$。这里，f 是常数 10^6 与 e^x 的乘积。该常数可以简单地并入界定常数 c 中，如下所示：

$$\begin{aligned} |f(x)| &\leq c|g(x)| \\ 10^6|e^x| &\leq c|g(x)| \\ |e^x| &\leq c|g(x)| \end{aligned}$$

因此，$f = O(e^x)$ 作为 $x \to \infty$。

示例 C.2。求项线性组合的阶数。

考虑 $f(x) = \cos(x) + x + 10x^{3/2} + 3x^2$。这里，$f$ 是各项的线性组合。当 x 趋近于无穷大时，各项 $\cos(x)$、x、$x^{3/2}$、x^2 按值递增的顺序排列。我们绘制 $f(x)$ 以及 $c|g(x)|$，其中对于每一项，c 已被选定，使得 $c|g(x = 2)|$ 超过 $f(x = 2)$。



不存在常数 c 使得对于足够大的 x，$f(x)$ 始终小于 $c|x^{3/2}|$。对于 $\cos(x)$ 和 x 也是如此。

我们发现 $f(x) = O(x^3)$，一般而言，对于 $m \geq 2$ 有 $f(x) = O(x^m)$，以及其他函数类如 $f(x) = e^x$。我们通常讨论能提供最紧上界的阶。因此，当 $x \to \infty$ 时，$f = O(x^2)$。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

C.2 Time Complexity Classes

解决某些问题的难度可以归入不同的时间复杂度类别。本书中频繁出现的重要类别包括:

- *P*: 可以在多项式时间内解决的问题,
- *NP*: 可以在多项式时间内验证解的问题,
- *NP-hard*: 至少在NP中最难问题一样难的问题，并且
- *NP-complete*: 既NP难又在NP中的问题。

这些复杂性类的正式定义相当复杂。普遍认为P $\neq$ NP，但这尚未得到证明，并且仍然是数学领域最重要的未解决问题之一。事实上，现代密码学依赖于这样一个事实：目前没有已知的高效（即多项式时间）算法来解决NP难问题。图C.1在假设P $\neq$ NP的情况下，展示了这些复杂性类之间的关系。

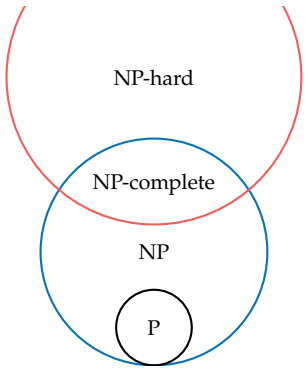


图 C.1. 复杂度类。

证明某个特定问题 Q 是否为 NP-hard 的常用方法是从一个已知的 *NP-complete* 问题 [3] 构造一个多项式变换。

Q' 到一个Q的实例。*3SAT*问题是首个已知的NP完全问题，并在例C.3中讨论。

C.3 Space Complexity Classes

另一组复杂度类与空间有关，指的是执行算法直至完成所需的内存量。复杂度类 *PSPACE* 包含所有可以在多项式空间量内解决的问题的集合，而不考虑时间。时间复杂度和空间复杂度之间存在根本区别，即时间不能被重用，但空间可以。我们知道 P 和 NP 是 PSPACE 的子集。目前尚不清楚，但有人怀疑，PSPACE 包含不在 NP 中的问题。通过多项式时间变换，我们可以定义 *PSPACE-hard* 和 *PSPACE-complete* 类，就像我们对 NP-hard 和 NP-complete 类所做的那样。

3 有许多著名的NP完全问题，正如R. M. Karp在《组合问题间的可归约性》中所综述的，该文收录于 *Complexity of Computer Computations*，由R. E. Miller和J. W. Thatcher编辑，Plenum出版社，1972年，第85-103页。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

例 C.3。3SAT 问题，这是第一个已知的 NP 完全问题。

Boolean satisfiability 问题涉及确定布尔公式是否为 *satisfiable*。布尔公式由涉及 n 个布尔变量 $x_1, \ldots x_n$ 的合取（$\wedge$）、析取（$\vee$）和否定（$\neg$）组成。文字是变量 x_i 或其否定 $\neg x_i$。3SAT 子句是最多三个文字（例如 $x_3 \vee \neg x_5 \vee x_6$）的析取。3SAT 公式是 3SAT 子句的合取，如

$$\begin{array}{rccccccc} & (& x_1 & \vee & x_2 & \vee & x_3 &) & \wedge \\ F(x_1, x_2, x_3, x_4) = & (& \neg x_1 & \vee & \neg x_2 & \vee & x_3 &) & \wedge \\ & (& x_2 & \vee & \neg x_3 & \vee & x_4 &) & \end{array}$$

3SAT 中的挑战在于确定是否存在一种可能的变量真值赋值，使得公式为真。在上面的公式中，

$$F(\text{true}, \text{false}, \text{false}, \text{true}) = \text{true}$$

因此，该公式是可满足的。尽管对于某些3SAT问题，通过快速检查就能轻松找到一个满足条件的赋值，但一般来说它们难以求解。判断是否能做出满足条件的赋值的一种方法是枚举所有变量的2^n种可能的真值。尽管确定是否存在满足条件的真值赋值很困难，但验证一个真值赋值是否导致满足可以在线性时间内完成。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

C.4 Decidability

一个 *undecidable* 问题并不总能在有限时间内解决。也许最著名的不可判定问题之一是 *halting problem*，它涉及将任何用足够表达力的语言编写的程序[4]作为输入，并且
判断它是否会终止。已经证明，不存在能够进行此类一般性分析的算法。尽管存在可以正确判断某些程序是否终止的算法，但不存在能判断任意程序是否会终止的算法。

4 技术要求是该语言为 *Turing complete*或 *computationally universal*，意味着它可以用来模拟任何图灵机。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

D Neural Representations

神经网络是非线性函数的参数化表示。[1] 函数
由神经网络表示的函是可微的，允许诸如随机梯度下降等基于梯度的优化算法优化其参数，以更好地逼近期望的输入-输出关系。[2] 神经

表示在决策相关的各种情境中都能发挥作用，比如表示概率模型、效用函数和决策策略。本附录概述了几种相关的架构。

D.1 Neural Networks

一个 *neural network* 是一个可微函数 $y = f_\theta(x)$，它将输入 x 映射以产生输出 y，并由 θ 参数化。现代神经网络可能拥有数百万个参数，并可用于将高维图像或视频形式的输入转换为高维输出，如多维分类或语音。

网络θ的参数通常经过调整，以最小化标量*lossfunction* $\ell(f_\theta(x), y)$，该标量与网络输出与期望输出的差距有关。损失函数和神经网络都是可微的，这使我们能够利用损失函数相对于参数化$\nabla_\theta \ell$的梯度，迭代地改进参数化。这一过程通常被称为神经网络*training*或*parameter tuning*。示例D.1中对此进行了演示。

神经网络通常是在输入-输出对的数据集D上进行训练的。在这种情况下，我们调整参数以最小化数据集上的总损失：

$$\arg\min_\theta \sum_{(x,y)\in D} \ell(f_\theta(x), y) \tag{D.1}$$

1 该名称源于对生物大脑中神经元网络的松散启发。我们不讨论这些生物学联系，但B. Müller、J. Reinhardt和M. T. Strickland在*Neural Networks*中提供了概述和历史视角。Springer，1995年。

2 当应用于具有许多层的神经网络时，这个优化过程，正如我们稍后将讨论的，通常被称为*deep learning* 有几本教科书完全致力于这些技术，包括I. Goodfellow、Y. Bengio和A. Courville，*Deep Learning*。MIT Press，2016年。Julia包`Flux.jl`提供了各种学习算法的高效实现。

考虑一个非常简单的神经网络，$f_\theta(x) = \theta_1 + \theta_2 x$。我们希望我们的神经网络能够接收房屋的平方英尺数 x 并预测其价格 y_{pred}。我们希望通过损失函数 $\ell(y_{\text{pred}}, y_{\text{true}}) = (y_{\text{pred}} - y_{\text{true}})^2$ 来最小化预测房价与真实房价之间的平方偏差。给定一个训练对，我们可以计算梯度：

$$\begin{aligned}\nabla_\theta \ell(f(x), y_{\text{true}}) &= \nabla_\theta (\theta_1 + \theta_2 x - y_{\text{true}})^2 \\ &= \begin{bmatrix} 2(\theta_1 + \theta_2 x - y_{\text{true}}) \\ 2(\theta_1 + \theta_2 x - y_{\text{true}})x \end{bmatrix}\end{aligned}$$

如果我们的初始参数化为$\theta = [10{,}000, 123]$，且我们有一对输入-输出($x = 2{,}500, y_{\text{true}} = 360{,}000$)，那么损失梯度将是$\nabla_\theta \ell = [-85{,}000, -2.125 \times 10^8]$。我们将朝着相反的方向迈出一小步，以改进我们的函数近似。

示例 D.1. 神经网络的基础知识与参数调优。

现代问题的数据集往往非常庞大，导致方程（D.1）的梯度评估成本高昂。常见的做法是在每次迭代中随机抽取训练数据的子集，利用这些 *batches* 来计算损失梯度。除了减少计算量外，使用较小批量大小计算梯度还为梯度引入了一定的随机性，有助于训练避免陷入局部最小值。

D.2 Feedforward Networks

神经网络通常被构建为将输入通过一系列层传递。[3] 具有许多层的网络通常被称为 *deep*。在 *feedforward networks* 中，
每一层都应用了仿射变换，随后逐个元素地应用非线性*activation function*：[4]

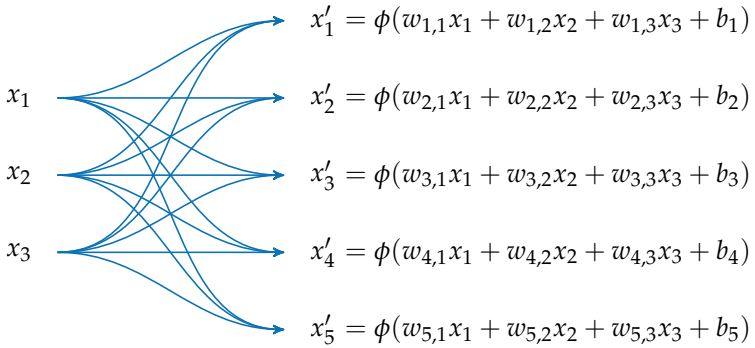
$$\mathbf{x}' = \phi.(\mathbf{W}\mathbf{x} + \mathbf{b}) \tag{D.2}$$

其中矩阵 W 和向量 b 是与该层相关的参数。全连接层如图 D.1 所示。当 W 不是方阵时，输出层的维度与输入层的维度不同。图 D.2 展示了同一网络的更紧凑描述。

3 一个足够大的单层神经网络理论上可以逼近任何函数。参见 A. Pinkus，“神经网络中 MLP 模型的逼近理论”，*Acta Numerica*，第 8 卷，第 143–195 页，1999 年。

4 激活函数引入的非线性提供了类似于生物神经元激活行为的特性，即输入积累最终导致神经元发放。A. L. Hodgkin和A. F. Huxley，“膜电流及其在神经传导与兴奋中应用的定量描述”，
Journal of Physiology，第117卷，第4期，第500–544页，1952年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

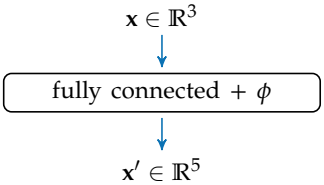


图D.1. 一个具有三组件输入和五组件输出的全连接层。

如果它们之间没有激活函数，多个连续的仿射变换可以合并为一个等价的仿射变换：

$$\mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = \mathbf{W}_2\mathbf{W}_1\mathbf{x} + (\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2) \tag{D.3}$$

这些非线性是必要的，以便神经网络能够适应并拟合任意目标函数。为了说明这一点，图D.3展示了一个训练用于近似非线性函数的神经网络的输出。



图D.2. 图D.1的更紧凑描述。神经网络层通常以块或切片的形式表示，以简化表示。

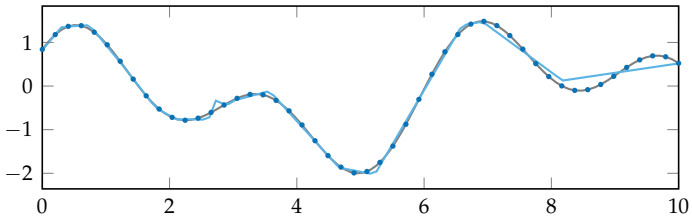
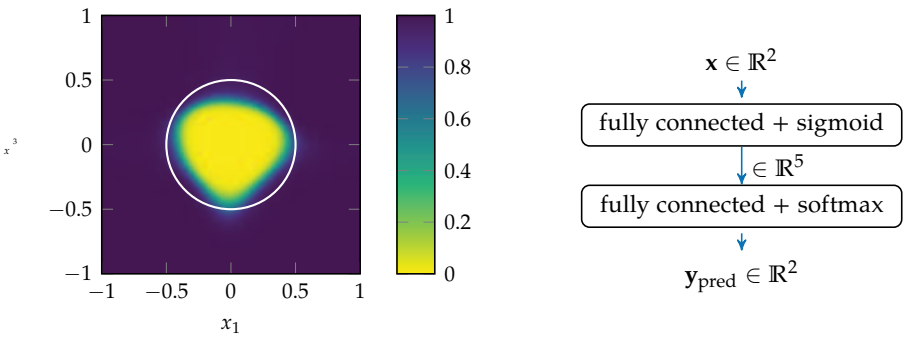


图 D.3. 一个深度神经网络拟合来自一个非线性函数的样本，以最小化平方误差。该神经网络具有四个仿射层，每个中间表示中有 10 个神经元。

有许多类型的激活函数是常用的。与它们的生物学灵感相似，当输入较低时，它们倾向于接近于零，而当输入较高时则较大。一些常见的激活函数显示在图D.5中。

有时会加入特殊层以实现某些效果。例如，在图D.4中，我们在末尾使用了一个*softmax*层，以强制输出表示一个二元类别分布。softmax函数应用

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com



图D.4。一个简单的、两层的全连接网络，经过训练以分类给定坐标是否位于一个圆内（以白色显示）。非线性特性使神经网络能够形成复杂的、非线性的决策边界。

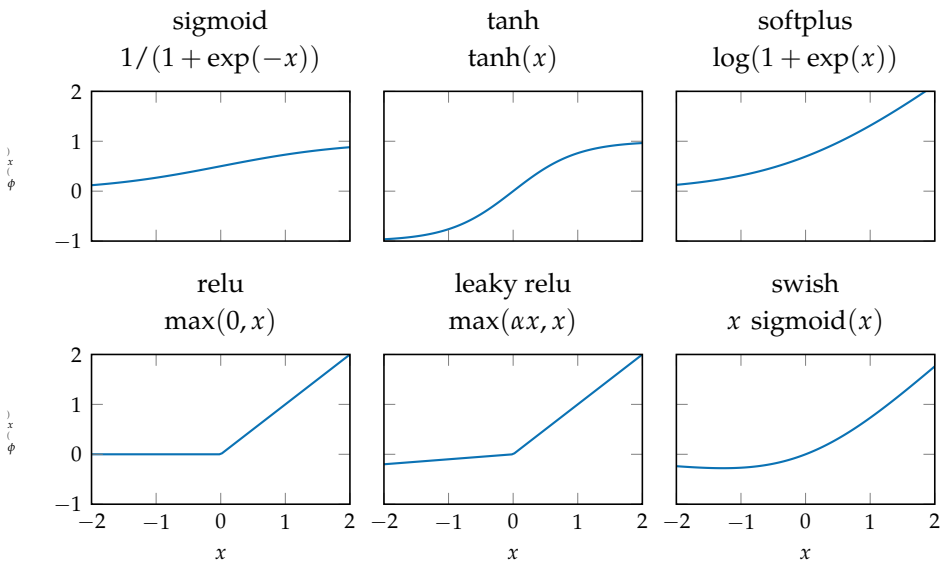


图 D.5。几种常见的激活函数。

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 b
ugs@algorithmsbook.com

对每个元素应用指数函数，确保它们为正，然后对结果值进行重新归一化：

$$\text{softmax}(\mathbf{x})_i = \frac{\exp(x_i)}{\sum_j \exp(x_j)} \tag{D.4}$$

神经网络的梯度通常使用 *reverse accumu- lation* 计算。[5] 该方法从正向步骤开始，其中神经网络
使用所有输入参数进行评估。在反向步骤中，从输出回溯到输入，计算每个感兴趣项的梯度。反向累积使用导数的链式法则：

$$\frac{\partial \mathbf{f}(\mathbf{g}(\mathbf{h}(\mathbf{x})))}{\partial \mathbf{x}} = \frac{\partial \mathbf{f}(\mathbf{g}(\mathbf{h}))}{\partial \mathbf{h}} \frac{\partial \mathbf{h}(\mathbf{x})}{\partial \mathbf{x}} = \left(\frac{\partial \mathbf{f}(\mathbf{g})}{\partial \mathbf{g}} \frac{\partial \mathbf{g}(\mathbf{h})}{\partial \mathbf{h}} \right) \frac{\partial \mathbf{h}(\mathbf{x})}{\partial \mathbf{x}} \tag{D.5}$$

示例D.2演示了这一过程。许多深度学习包使用此类自动微分技术来计算梯度。[6] 用户很少需要
提供它们自己的梯度。

5 这个过程通常被称为 *backpropagation*，特指应用于标量损失函数的反向累积。D. E. Rumel hart、G. E. Hinton 和 R. J. Williams，“通过反向传播误差学习表示”，*Nature*，第 323 卷，第 533–536 页，1986 年。

6 A. Griewank 和 A. Walther，*Eval-*，第2版，*uating Derivatives: Principles and Techniques of Algorithmic Differentiation*

D.3 Parameter Regularization

神经网络通常是 *underdetermined*，这意味着存在多种参数实例化可以导致相同的最优训练损失。[7] 它
通常使用 *parameter regularization*（也称为 *weight regularization*）在损失函数中引入一个额外项来惩罚大的参数值。正则化还有助于防止 *overfitting*，这种情况发生在网络过度专门适应训练数据但无法泛化到未见过的数据时。

7 例如，假设我们有一个具有最终 softmax 层的神经网络。该层的输入可以在产生相同输出以及因此相同的损失时进行缩放。

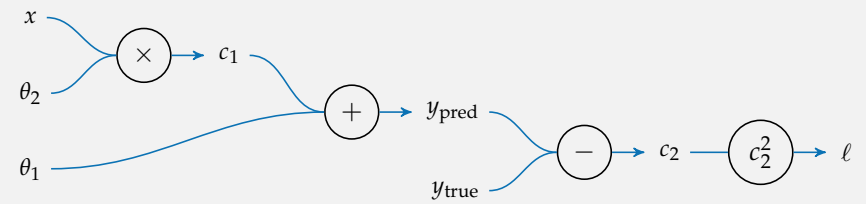
正则化通常采取参数向量的 L_2-范数形式：

$$\arg\min_{\theta} \sum_{(x,y)\in\mathbf{D}} \ell(f_\theta(x), y) + \beta\|\theta\|^2 \tag{D.6}$$

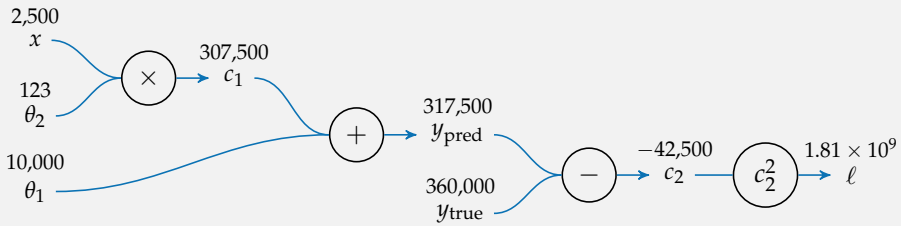
其中正标量β控制参数正则化的强度。该标量通常非常小，其值可低至10^{-6}，以尽量减少因引入正则化而牺牲对训练集的匹配程度。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

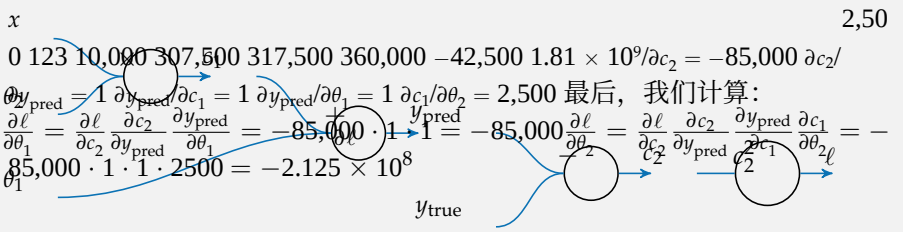
回顾示例 D.1 中的神经网络和损失函数。这里我们绘制了损失计算的计算图：



反向累积从正向传递开始，在此过程中计算图被评估。我们将再次使用 $\theta = [10{,}000, 123]$ 以及输入-输出对 $(x = 2{,}500, y_{\text{true}} = 360{,}000)$，如下所示：



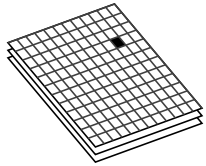
然后通过回溯树来计算梯度：



示例 D.2. 给定训练数据，如何使用反向累积来计算参数梯度。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

D.4 Convolutional Neural Networks



图D.6. 图像等多维输入将向量推广到张量。在这里，我们展示了一个三层的RGB图像。这样的输入可以拥有非常多的条目。

神经网络可能以图像或其他多维结构（如激光雷达扫描）作为输入。即使是一张相对较小的256 × 256 RGB图像（类似于图D.6），也有256 × 256 × 3 = 196,608个条目。任何以$m \times m \times 3$图像为输入并产生n个输出向量的全连接层，其权重矩阵将有$3m^2n$个值。需要学习的大量参数不仅在计算上昂贵，而且也是浪费的。图像中的信息通常是平移不变的；图像中向右移动1像素的物体应产生相似（如果不是完全相同）的输出。

Convolutional layers[8] 两者都显著减少了计算量，并且通过滑动一个较小的全连接窗口来产生输出，从而支持平移不变性。需要学习的参数显著减少。这些参数往往对局部纹理有感受性，就像视觉皮层中的神经元对其感受野内的刺激作出反应一样。

8 Y. LeCun, L. Bottou, Y. Bengio, 和 P. Haffner, “应用于文档识别的基于梯度的学习”，*Proceedings of the IEEE*，第86卷，第11期，第2278–2324页，1998年。

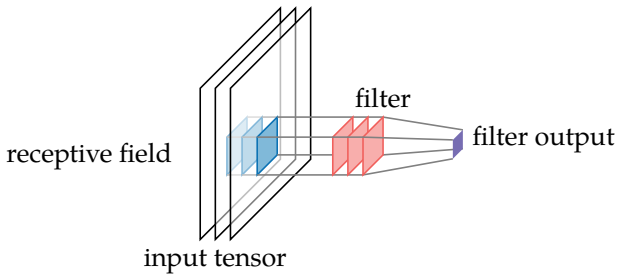


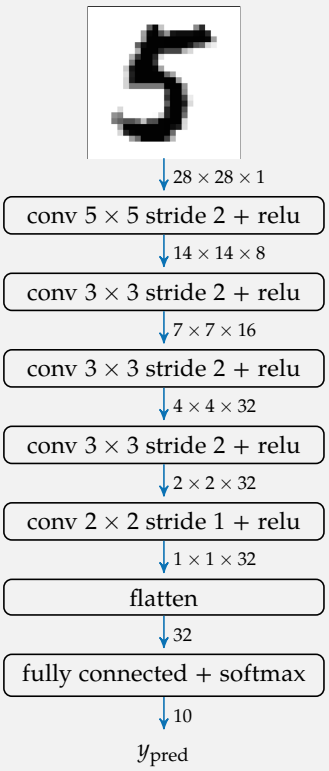
图 D.7. 卷积层反复将滤波器应用于输入张量（例如图像），以产生输出张量。该图展示了滤波器的每次应用如何像一个小型全连接层一样，作用于一个小感受野，以产生输出张量中的一个条目。每个滤波器根据预定的步幅在输入上移动。得到的输出具有与滤波器数量相同的层数。

卷积层由一组*features*或*kernels*组成，每个都相当于一个全连接层，可以向其中输入输入张量的较小区域。在图D.7中，单个核被应用一次。这些特征具有完整的深度，这意味着如果输入张量是$n \times m \times d$，特征也将具有第三维度d。这些特征通过在输入的第一维和第二维上滑动来多次应用。如果*stride*是1×1，那么所有k滤波器都应用于每个可能的位置，输出维度将是$n \times m \times k$。如果步幅是2×2，那么滤波器在每次应用时在第一维和第二维上移动2，导致输出大小为$n/2 \times m/2 \times k$。卷积神经网络通常随着每一层的增加而在第三维增加，并在前两个维度减少。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

卷积层具有平移不变性，因为无论滤波器应用于输入的哪个位置，其行为都是相同的。这一特性在空间处理中尤为有用，因为输入图像的平移可以得到相似的输出，从而使神经网络更容易提取共同特征。单个特征倾向于学习如何识别颜色和纹理等局部属性。

The MNIST data set contains handwritten digits in the form of 28×28 monochromatic images. It is a often used to test image classification networks. To the right, we have a sample convolutional neural network that takes an MNIST image as input and produces a categorical probability distribution over the 10 possible digits. Convolutional layers are used to efficiently extract features. The model shrinks in the first two dimensions and expands in the third dimension (the number of features) as the network depth increases. Eventually reaching a first and second dimension of 1 ensures that information from across the entire image can affect every feature. The flatten operation takes the $1 \times 1 \times 32$ input and flattens it into a 32-component output. Such operations are common when transitioning between convolutional and fully connected layers. This model has 19,722 parameters. The parameters can be tuned to maximize the likelihood of the training data.



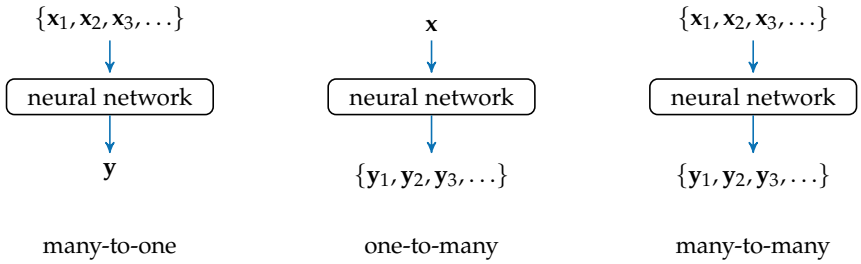
示例 D.3. 用于 MNIST 数据集的卷积神经网络。Y. LeCun, L. Bottou, Y. Bengio, 和 P. Haffner, “基于梯度的学习应用于文档识别，” *Proceedings of the IEEE*, 第 86 卷，第 11 期，第 2278–2324 页，1998 年。

D.5 *Recurrent Networks*

到目前为止讨论的神经网络架构不适用于时间或顺序输入。序列操作在处理图像时出现。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

来自视频、翻译单词序列或追踪时间序列数据时。在这些情况下，输出不仅仅依赖于最近的输入。此外，迄今为止讨论的神经网络架构自然不产生可变长度的输出。例如，一个撰写论文的神经网络将难以使用传统的全连接神经网络进行训练。



图D.8. 传统神经网络不能自然地接受可变长度输入或产生可变长度输出。

当神经网络具有序列输入、序列输出或两者兼有（图 D.8）时，我们可以使用 *recurrent neural network* 在多次迭代中起作用。这些神经网络维护一个循环状态 r，有时称为其 *memory*，以随时间保留信息。例如，在翻译中，句子早期使用的一个词可能与句子后面词语的正确翻译相关。图 D.9 展示了一个基本循环神经网络的结构，以及如何将同一个神经网络理解为一个在时间上展开的更大网络。

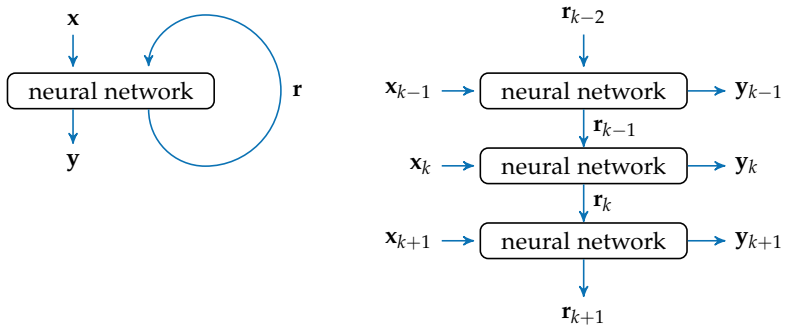
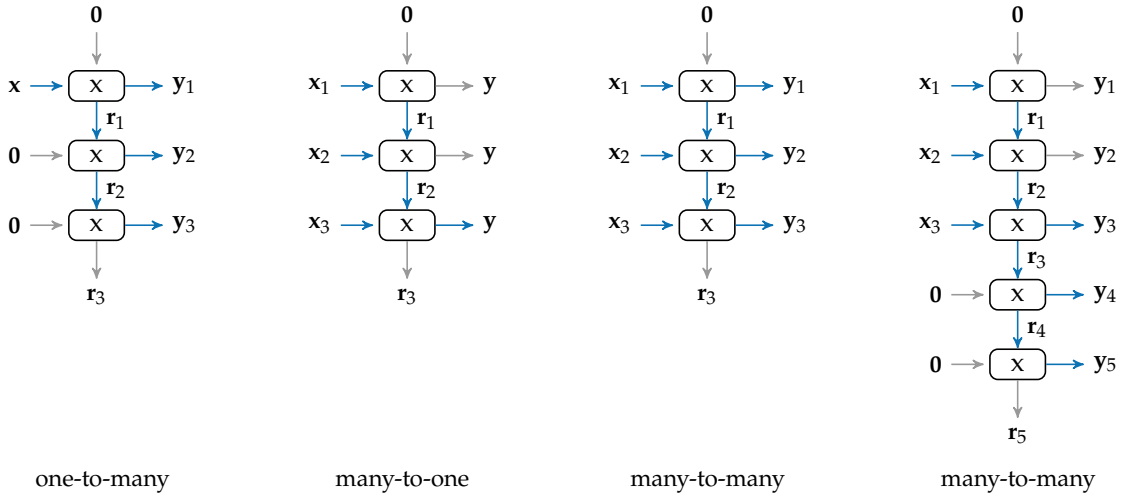


图 D.9. 循环神经网络（左）和同一循环神经网络按时间展开（右）。这些网络保持一个循环状态r，使网络能够形成一种记忆，跨迭代传递信息。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

这种展开结构可用于生成丰富多样的序列神经网络，如图D.10所示。多对多结构有多种形式。在一种形式中，输出序列以输入序列开始。在另一种形式中，输出序列不以输入序列开始。当使用变长输出时，神经网络输出本身通常指示序列何时开始或结束。循环状态通常初始化为零，输入序列传入后的额外输入也是如此，但情况并非必须如此。



图D.10。循环神经网络可以随时间展开以产生不同的关系。未使用或默认的输入和输出以灰色显示。

具有多层结构、在多个时间步上展开的循环神经网络，实际上产生了一个非常深的神经网络。在训练过程中，梯度是相对于损失函数计算的。距离损失函数较远的层对梯度的贡献往往小于距离损失函数较近的层。这导致了*vanishing gradient*问题，即深度神经网络在其上层中的梯度变得极其微小。这些微小的梯度会使训练变得缓慢。

非常深的神经网络也可能遭受*exploding gradients*的问题，其中通过各层的连续梯度贡献相结合会产生非常大的值。这样的大值会使学习变得不稳定。示例D.4展示了梯度爆炸和梯度消失两种情况。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

为了说明梯度消失和梯度爆炸，考虑一个由一维全连接层组成、使用relu激活函数的深度神经网络。例如，如果网络有三层，其输出为

$$f_\theta(x) = \text{relu}(w_3\ \text{relu}(w_2\ \text{relu}(w_1 x_1 + b_1) + b_2) + b_3)$$

损失函数相对于某个变量的梯度取决于f_θ的梯度。

我们可以在第一层的参数w_1和b_1中得到消失梯度，如果连续层中的梯度贡献小于1。例如，如果任何层的relu函数输入为负，其输入的梯度将为零，因此梯度完全消失。在一个不太极端的情况下，假设权重都是$\text{w} = 0.5\ 1$，偏置都是$\text{b} = 0$，输入x为正。在这种情况下，关于w_1的梯度是

$$\frac{\partial f}{\partial w_1} = x_1 \cdot w_2 \cdot w_3 \cdot w_4 \cdot w_5 \ldots$$

网络越深，梯度越小。

如果连续层中的梯度贡献大于1，我们会在第一层的参数中遇到梯度爆炸。如果我们仅仅将权重增加到$\text{w} = 2\ 1$，同样的梯度突然每层都会翻倍。

示例 D.4。演示深度神经网络中梯度消失和梯度爆炸是如何产生的。这个示例使用了一个非常简单的神经网络。在更大的全连接层中，同样的原理也适用。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

虽然爆炸梯度通常可以通过梯度裁剪、正则化和将参数初始化为小值来处理，但这些解决方案只是将问题转向了梯度消失的问题。循环神经网络通常使用专门构建的层来缓解梯度消失问题。它们通过选择性地决定是否保留记忆来发挥作用，而这些门控有助于调节记忆和梯度。两种常见的循环层是 *long short-term memory* (*LSTM*)[9]和*gated recurrent units* (*GRU*)。[10]

9 S. Hochreiter 和 J. Schmidhuber，“长短期记忆”，*Neural*，第9卷，第8期，第1735-1780页，1997年。

10 K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, 和 Y. Bengio，“使用RNN编码器-解码器学习短语表示用于统计机器翻译”，载于*Conference on Empirical Methods in Natural Language Processing* (*EMNLP*)，2014年。

D.6 Autoencoder Networks

神经网络通常用于处理高维输入，例如图像或点云。这些高维输入通常具有高度结构化的特点，其实际信息内容的维度远低于其呈现所在的高维空间。图像中的像素往往与其邻近像素高度相关，而点云通常包含许多连续区域。有时我们希望通过对数据集进行转换，将其表示为一个小得多的特征集合，或一个 *embedding*，从而建立对数据集信息内容的理解。这种压缩，或称 *representation learning*，具有许多优点。[11] 低维表示有助于促进

11 这种降维也可以通过传统机器学习技术完成，例如主成分分析。神经模型提供了更大的灵活性，并且能够处理非线性表示。

像贝叶斯网络这样的传统机器学习技术，到原本可能无法处理的情况。这些特征可以被检查，以增进对数据集信息内容的理解，并且这些特征可作为其他模型的输入。

一个 *autoencoder* 是一种神经网络，经过训练以发现更高层次输入的低维特征表示。自编码器网络接收高维输入 x，并产生具有相同维度的输出 x′。我们设计网络架构，使其通过称为 *bottleneck* 的低维中间表示。这个瓶颈处的激活 z 是我们的低维特征，它们存在于一个未被显式观察到的 *latent space* 中。这样的架构如图 D.11 所示。

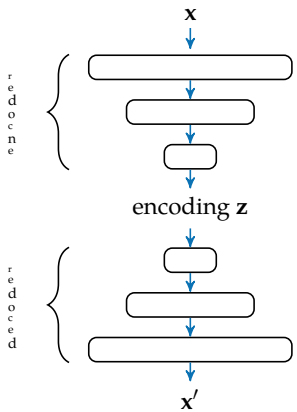


图 D.11。自编码器将高维输入通过低维瓶颈，然后重建原始输入。最小化重建损失可以得到高效的低维编码。

我们训练自编码器以重现其输入。例如，为了促使输出x′尽可能接近x，我们可以简单地最小化L_2-范数，

$$\underset{\theta}{\text{minimize}} \quad \mathop{\mathbb{E}}_{\mathbf{x}\in\mathbf{D}}[\|f_\theta(\mathbf{x}) - \mathbf{x}\|_2] \tag{D.7}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

噪声通常被添加到输入中，以生成更稳健的特征嵌入：

$$\underset{\theta}{\text{minimize}} \quad \underset{\mathbf{x}\in\mathbf{D}}{\mathbb{E}}[\|f_\theta(\mathbf{x}+\boldsymbol{\epsilon})-\mathbf{x}\|_2] \tag{D.8}$$

训练以最小化重建损失，迫使自编码器找到最有效的低维编码，足以精确重建原始输入。此外，训练是无监督的，因为我们无需引导训练至特定特征集。

训练完成后，自编码器瓶颈上方的部分可作为*encoder*，将输入转换为特征表示。自编码器下方的部分可作为*decoder*，将特征表示转换为输入表示。在训练神经网络生成图像或其他高维输出时，解码非常有用。示例D.5展示了为手写数字学习到的嵌入。

一个*variational autoencoder*，如图D.12所示，扩展了自编码器框架以学习概率编码器。[12] 它不输出确定性的

在采样中，编码器产生一个关于编码的分布，这允许模型对其编码赋予置信度。由于其数学上的便利性，通常使用具有对角协方差矩阵的多元高斯分布。在这种情况下，编码器同时输出编码均值和对角协方差矩阵。

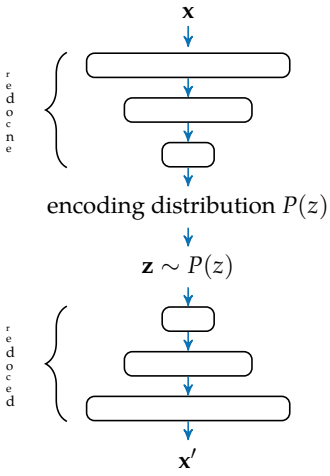


图 D.12. 变分自编码器将高维输入通过低维瓶颈，产生关于编码的概率分布。解码器从该编码中重构样本，以重建原始输入。因此，变分自编码器可以为每个编码特征赋予置信度。此后，解码器可用作生成模型。

变分自编码器的训练目标是在保持编码分量接近单位高斯分布的同时，最小化期望重构损失。前者通过每次传递时从编码分布中取单个样本 $\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}、\boldsymbol{\sigma}^\top \mathrm{I}\boldsymbol{\sigma})$ 来实现。为了使反向传播有效，我们通常将随机噪声 $\mathbf{w} \sim \mathcal{N}(0、\mathrm{I})$ 作为神经网络的额外输入，并根据 $\mathbf{z} = \boldsymbol{\mu} + \mathbf{w} \odot \boldsymbol{\sigma}$ 获得样本。

通过同时最小化KL散度（附录A.10），组件被保持接近单位高斯分布。[13] 该目标鼓励平滑的潜在空间

表示。网络因将潜在表示分散开（$\|\boldsymbol{\mu}\|$的值较大）以及将每个表示集中到非常小的编码空间中（$\|\boldsymbol{\sigma}\|$的值较小）而受到惩罚，从而确保更好地覆盖潜在空间。因此，解码器中的平滑变化可以产生平滑变化的输出。这一特性使得解码器可以用作*generative models*，其中来自单位多元高斯分布的样本可以输入到解码器以

12 D. Kingma 和 M. Welling，“自编码变分贝叶斯”，收录于 *International Conference on Learning Representations (ICLR)*，2013年。

13 两个单位高斯分布的KL散度为

$$\log\left(\frac{\sigma_2}{\sigma_1}\right) + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

在原始空间中生成逼真的样本。组合损失函数为

$$\begin{aligned} \underset{\theta}{\text{minimize}} \quad & \mathop{\mathbb{E}}_{\mathbf{x}\in\mathbf{D}}\left[\|\mathbf{x}' - \mathbf{x}\|_2 + c\sum_{i=1}^{|\boldsymbol{\mu}|} D_{\text{KL}}\left(\mathcal{N}\left(\mu_i, \sigma_i^2, \right) \middle|\middle| \mathcal{N}(0,1)\right)\right] \\ \text{subject to} \quad & \boldsymbol{\mu}, \boldsymbol{\sigma} = \text{encoder}(\mathbf{x} + \boldsymbol{\epsilon}) \\ & \mathbf{x}' = \text{decoder}(\boldsymbol{\mu} + \mathbf{w} \odot \boldsymbol{\sigma}) \end{aligned} \tag{D.9}$$

其中两个损失之间的权衡通过标量 $c > 0$ 进行调整。示例 D.6 在手写数字学习得到的潜在空间上演示了这一过程。

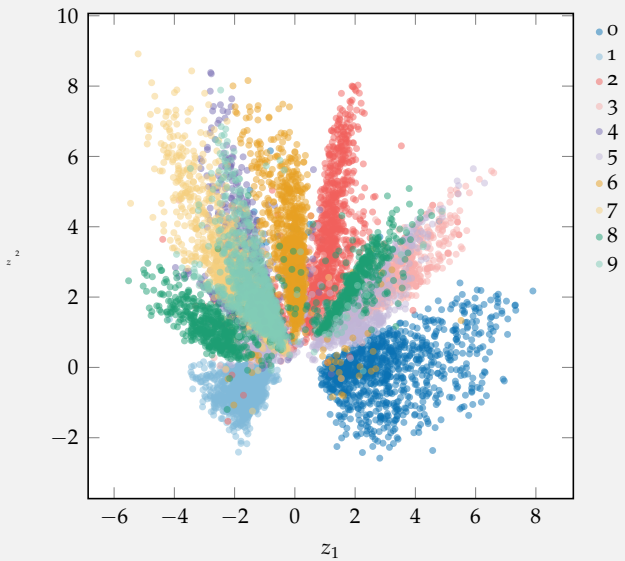
变分自编码器通过将编码器表示为条件分布 $q(\text{z} \mid \text{x})$ 来推导，其中 x 属于观测输入空间，z 位于未观测的嵌入空间。解码器则在相反方向上进行推断，表示 $p(\text{x} \mid \text{z})$，在这种情况下它同样输出一个概率分布。我们希望最小化 $q(\text{z} \mid \text{x})$ 与 $p(\text{z} \mid \text{x})$ 之间的KL散度，这等同于最小化 $\mathbb{E}[\log p(\text{x} \mid \text{z})] - D_{\text{KL}}(q(\text{z} \mid \text{x}) \mid\mid p(\text{z}))$，其中 $p(\text{z})$ 是我们的先验，即我们用来偏置编码分布的单位多元高斯分布。由此我们得到重建损失和KL散度。

D.7 Adversarial Networks

我们经常希望训练神经网络以生成高维输出，如图像或直升机控制输入序列。当输出空间很大时，训练数据可能仅覆盖状态空间的一小部分。因此，仅基于可用数据进行训练可能导致不切实际的结果或过拟合。我们通常希望神经网络能产生合理的输出。例如，当生成图像时，我们希望图像看起来真实。在模仿人类驾驶，如模仿学习（第18章）中，我们希望车辆通常能保持在车道内，并对其他车辆做出适当反应。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们可以使用自编码器来为 MNIST 数据集训练一个嵌入。在这个例子中，我们使用的编码器类似于示例 D.3 中的卷积网络，只是输出是二维的，并且没有 softmax 层。我们构建一个镜像编码器的解码器，并训练整个网络以最小化重构损失。以下是训练后来自 MNIST 数据集的 10,000 张图像的编码。每个编码根据对应的数字进行着色：

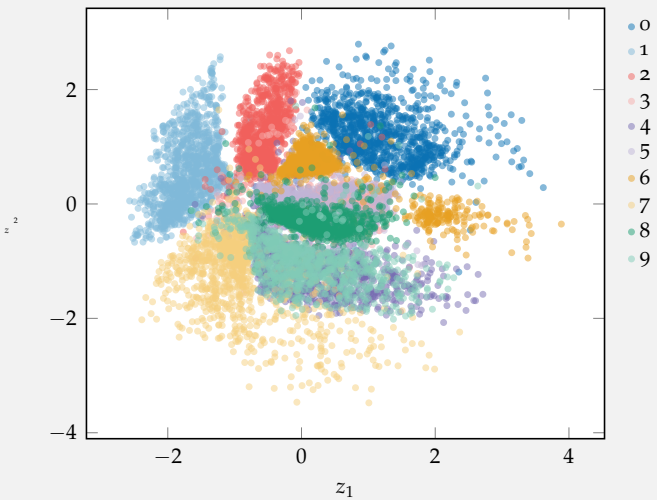


示例D.5。为MNIST数字学习的二维嵌入的可视化。

我们发现，数字倾向于聚集成从原点大致呈径向分布的区域。注意1和7的编码相似，因为这两个数字看起来很像。回想一下，训练是无监督的，网络没有获得任何关于数字值的信息。尽管如此，这些聚类还是产生了。

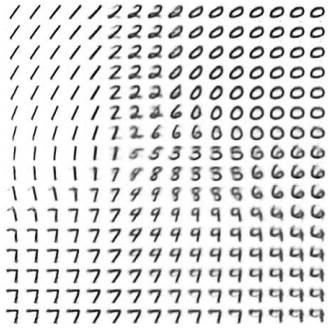
© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

在示例D.5中，我们在MNIST数据集上训练了一个自编码器。我们可以调整相同的网络，使其在瓶颈层产生二维均值向量和方差向量，而不是二维嵌入，然后训练它同时最小化重构损失和KL散度。这里，我们展示了MNIST数据集中相同的10,000张图像的均值编码。每个编码同样根据对应的数字着色：



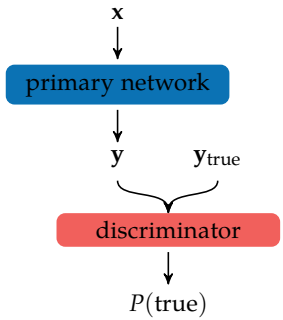
变分自编码器同样会在嵌入空间中为每个数字生成聚类，但这一次，它们大致按照零均值、单位方差的高斯分布进行分布。我们再次看到某些编码是如何相似的，例如4和9的显著重叠。

示例 D.6. 使用变分自编码器为 MNIST 数字学习的二维嵌入的可视化。在此，我们展示了在编码空间上平移输入后解码得到的输出：



© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

一种常见的惩罚偏离标称输出或行为的方法是使用*adversarial learning*，通过包含判别器来实现，如图D.13所示。[14] A *discriminator* 是一个神经网络，充当二元分类器，接收神经网络的输出，并学习区分训练集中的真实输出与主神经网络的输出。主神经网络，也称为*generator*，随后被训练来欺骗判别器，从而自然地产生更难以与数据集区分的输出。这种技术的主要优点在于，我们无需设计特殊特征来识别或量化输出如何未能匹配训练数据，而是可以让判别器自然地学习这些差异。



图D.13。生成对抗网络通过使用判别器迫使主网络产生更真实的输出，从而使主网络的输出更加真实。

学习是对抗性的，这意味着我们有两个神经网络：一个是希望产生逼真输出的主神经网络，另一个是区分主网络输出与真实样本的判别器网络。它们各自训练以超越对方。训练是一个迭代过程，其中每个网络轮流得到改进。有时平衡它们的相对表现可能颇具挑战；如果一个网络变得太强，另一个可能会停滞不前。

14 这些技术由I. Goodfellow、J. Pouget-Abadie、M. Mirza、B. Xu、D. Warde-Farley、S. Ozair、A. Courville和Y. Bengio在*Advances in Neural Information Processing Systems* (*NIPS*)，2014中引入，题为“生成对抗网络”。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

E Search Algorithms

一个 *search problem* 关注于寻找合适的动作序列，以在随后的确定性转移中最大化获得的奖励。搜索问题是具有确定性转移函数的马尔可夫决策过程（在第二部分中涵盖）。一些著名的搜索问题包括滑块拼图、魔方、推箱子和寻找到达目的地的最短路径。

E.1 Search Problems

在搜索问题中，我们在时间t根据观察到的状态s_t选择动作a_t，然后获得奖励r_t。*action space* $\mathcal{A}$是可能动作的集合，*state space* $\mathcal{S}$是可能状态的集合。一些算法假设这些集合是有限的，但通常这并不是必需的。状态确定性地演化，且仅依赖于当前状态和所采取的动作。我们使用$\mathcal{A}(s)$来表示状态s下的有效动作集合。当没有有效动作时，该状态被视为*absorbing*，并在所有未来时间步中产生零奖励。例如，目标状态通常是吸收态。

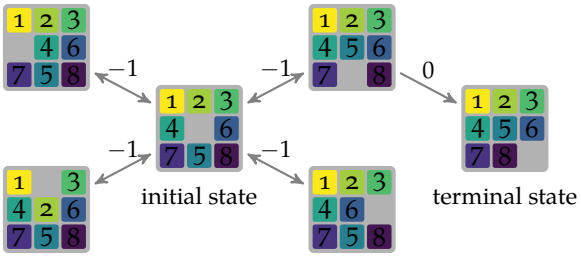
确定性状态转移函数 $T(s、a)$ 给出后继状态 s'。奖励函数 $R(s、a)$ 给出从状态 s 执行动作 a 时获得的奖励。搜索问题通常不包括对未来奖励进行惩罚的折扣因子 γ。目标是选择最大化奖励总和的动作序列，即 *return*。算法 E.1 提供了表示搜索问题的数据结构。

```
struct Search
    S  # state space
    A  # valid action function
    T  # transition function
    R  # reward function
end
```

算法E.1. 搜索问题数据结构。

E.2 Search Graphs

一个具有有限状态和动作空间的搜索问题可以表示为一个*search graph*。节点对应状态，边对应状态之间的转移。每条从源状态到目标状态的边都关联着一个导致该状态转移的动作，以及从源状态采取该动作时的期望奖励。图E.1描绘了3×3滑动拼图这类搜索图的一个子集。



图E.1. 滑动拼图游戏中的一些状态，以图的形式展示。从初始状态出发，可以采取两种转换以到达最终解决状态。边缘上的数字代表奖励。

许多图搜索算法从初始状态开始搜索，并从那里向外扩展。在此过程中，这些算法会描绘出一棵*search tree*。初始状态是根节点，每当我们在搜索过程中从s转移到s'时，都会向搜索树中添加一条从s到新节点s'的边。图E.2显示了同一个滑块拼图问题的搜索树。

E.3 Forward Search

也许最简单的图搜索算法是 *forward search* (算法 E.2)，它通过查看深度（或视界）d 内所有可能的动作-状态转移，来确定从初始状态 s 应采取的最佳动作。在深度 d 处，算法使用状态 $U(s)$ 价值的估计值。[1] 该算法调用

[1] 本章中的近似价值函数在处于没有可用动作的状态时预期会返回0。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

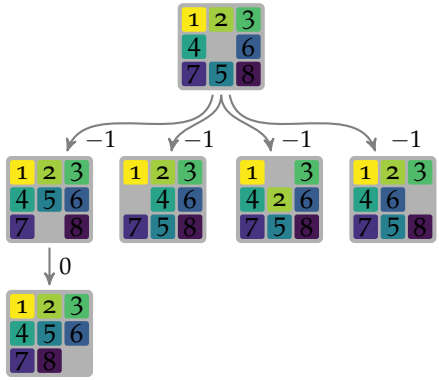


图 E.2。图 E.1 中的 3 × 3 滑动拼图问题可以表示为树搜索问题。搜索从根节点开始，并沿树向下进行。在这种情况下，可以遍历一条路径到达所需的目标状态。

本身以深度优先的方式递归，导致搜索树并返回一个包含最优动作a及其有限时间期望值u的元组。

```
function forward_search(𝒫::Search, s, d, U)
    𝒜, T, R = 𝒫.𝒜(s), 𝒫.T, 𝒫.R
    if isempty(𝒜) || d ≤ 0
        return (a=nothing, u=U(s))
    end
    best = (a=nothing, u=-Inf)
    for a in 𝒜
        s′ = T(s,a)
        u = R(s,a) + forward_search(𝒫, s′, d-1, U).u
        if u > best.u
            best = (a=a, u=u)
        end
    end
    return best
end
```

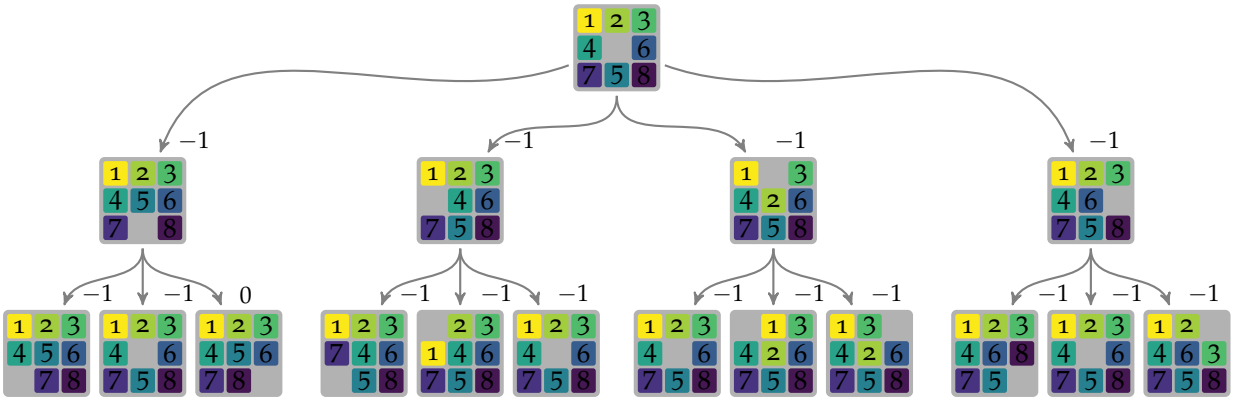
算法E.2. 用于从当前状态s出发，为离散搜索问题$\mathcal{P}$寻找近似最优动作的前向搜索算法。搜索进行至深度d，此时终端价值通过近似价值函数U估计。返回的命名元组包含最佳动作a及其有限视野期望值u。

图 E.3 显示了一个通过在滑动拼图游戏上运行前向搜索获得的搜索树示例。深度优先搜索可能造成浪费；给定深度下的所有可达状态都会被访问。对于有 $|\mathcal{A}|$ 个动作的问题，搜索到深度 d 将生成一个包含 $O(|\mathcal{A}|^d)$ 个节点的搜索树。

E.4 Branch and Bound

一种称为*branch and bound* (算法E.3)的通用方法，通过利用关于上下界的领域信息，可以显著减少计算量。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com



图E.3. 在滑动拼图问题中，进行深度为2的前向搜索所形成的搜索树。所有两步内可达的状态都被访问，且部分状态被多次访问。我们发现到达终端节点的路径仅有一条，该路径的回报为−1，而其他所有路径的回报均为−2。

期望奖励的界限。从状态 s 采取动作 a 的回报上界是 $Q(s、a)$。从状态 s 的回报下界是 $\underline{U}(s)$。分支定界遵循与深度优先搜索相同的流程，但它根据动作的上界遍历动作，并且仅当某一后继节点可能返回的最佳值高于先前动作已经发现的数值时，才继续前往该后继节点。分支定界搜索在示例 E.1 中与向前搜索进行了比较。

```
function branch_and_bound(𝒫::Search, s, d, Ulo, Qhi)
    𝒜, T, R = 𝒫.𝒜(s), 𝒫.T, 𝒫.R
    if isempty(𝒜) || d ≤ 0
        return (a=nothing, u=Ulo(s))
    end
    best = (a=nothing, u=-Inf)
    for a in sort(𝒜, by=a->Qhi(s,a), rev=true)
        if Qhi(s,a) ≤ best.u
            return best # safe to prune
        end
        u = R(s,a) + branch_and_bound(𝒫,T(s,a),d-1,Ulo,Qhi).u
        if u > best.u
            best = (a=a, u=u)
        end
    end
    return best
end
```

算法E.3. 用于从当前状态s寻找离散搜索问题$\mathcal{P}$的近似最优动作的分支定界搜索算法。搜索至深度d，采用值函数下界Ulo和动作值函数上界Qhi。返回的命名元组包含最佳动作a及其有限视界期望值u。

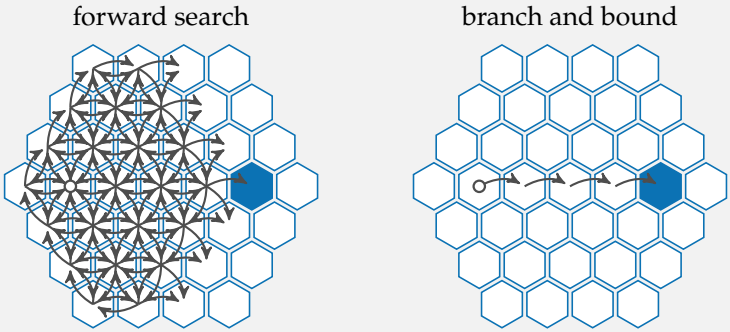
© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

考虑在一个六边形世界搜索问题上使用分支定界法。搜索问题中的动作导致确定性转移，因此与六边形世界的MDP不同，当采取相应的动作时，我们总是在相邻的瓦片之间正确转移。

圆圈表示起始状态。所有转移都会获得 -1 的奖励。蓝色方块是终止状态，进入时产生奖励 5。

这里，我们展示了前向搜索和分支定界到深度4的搜索树。对于分支定界，我们使用了下界$\underline{U}(s) = -6$和上界$Q(s, a) = 5 - \delta(T(s, a))$，其中函数$\delta(s)$是从给定状态到终端奖励状态的最小步数。分支定界的搜索树是前向搜索的子集，因为分支定界可以忽略它认为不是最优的部分。

由于上界的存在，分支定界法首先评估向右移动，并且因为恰好这是最优解，它能够立即确定最优动作序列并避免探索其他动作。如果起始状态和目标状态互换，搜索树会更大。在最坏的情况下，其规模可能与正向搜索一样大。



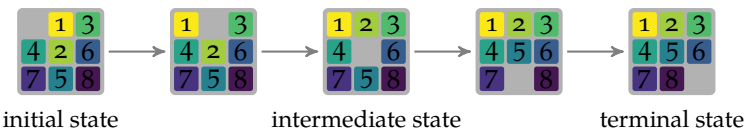
示例 E.1。分支定界相比于前向搜索所能节省的计算量比较。在合适的界限下，分支定界可以显著更高效。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

分支定界并不保证比前向搜索减少计算量。两种方法具有相同的最坏情况时间复杂度。算法的效率很大程度上取决于启发式方法。

E.5 Dynamic Programming

无论是前向搜索还是分支定界法，都没有记住某个状态是否曾被访问过；两者都通过多次评估这些状态来浪费计算资源。*Dynamic programming*（算法E.4）通过记住特定子问题何时已被解决来避免重复工作。动态规划可以应用于那些最优解可以从其子问题的最优解构造出来的问题，这一性质称为*optimal substructure*。例如，如果从s_1到s_3的最优动作序列经过s_2，那么从s_1到s_2以及从s_2到s_3的子路径也是最优的。这种子结构如图E.4所示。



图E.4。左侧的状态序列构成了从初始状态到终止状态的最优路径。最短路径问题具有最优子结构，意味着从初始状态到中间状态的序列也是最优的，从中间状态到终止状态的序列同样如此。

在图搜索的情况下，评估一个状态时，我们首先检查一个*transpositiontable*，看该状态之前是否已被访问过，如果是，则返回其存储的值。[2]否则，我们按常规评估该状态并存储结果

在置换表中。与正向搜索的比较如图E.5所示。

将昂贵计算的结果缓存起来，以便将来可以检索而无需重新计算，这被称为*memoization*。

E.6 Heuristic Search

Heuristic search[3]（算法E.5）通过排列其分支定界来改进
基于提供的启发式函数$U(s)$（收益的上界）采取行动。与动态规划类似，启发式搜索具有一种机制，可以缓存状态评估以避免冗余计算。此外，启发式搜索不需要分支定界所需的 lower bound 值函数。[4]

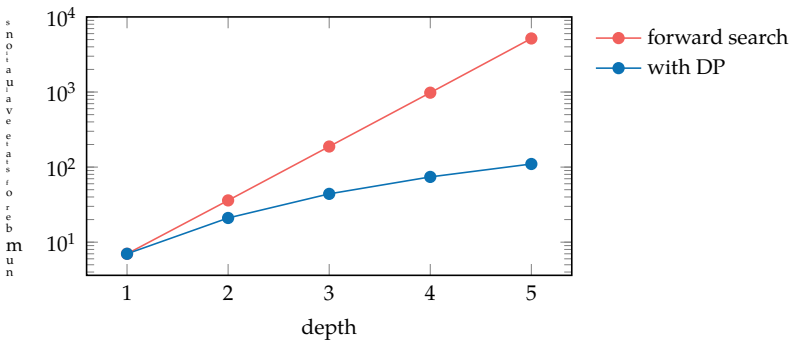
3 启发式搜索也被称为 *informed search* 或 *best-first search*。

4 我们的实现确实使用了两个价值函数：用于引导搜索的启发式函数和用于评估终止状态的近似价值函数。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
function dynamic_programming(𝒫::Search, s, d, U, M=Dict())
    if haskey(M, (d,s))
        return M[(d,s)]
    end
    𝒜, T, R = 𝒫.𝒜(s), 𝒫.T, 𝒫.R
    if isempty(𝒜) || d ≤ 0
        best = (a=nothing, u=U(s))
    else
        best = (a=first(𝒜), u=-Inf)
        for a in 𝒜
            s′ = T(s,a)
            u = R(s,a) + dynamic_programming(𝒫, s′, d-1, U, M).u
            if u > best.u
                best = (a=a, u=u)
            end
        end
    end
    M[(d,s)] = best
    return best
end
```

算法E.4. 应用于前向搜索的动态规划，其中包含一个转置表`M`。在这里，`M`是一个字典，存储了以往评估中的深度-状态元组，使得方法能够返回先前计算的结果。搜索进行到深度`d`，此时通过近似价值函数`U`估计终端价值。返回的命名元组包含最佳行动`a`及其有限视野期望值`u`。



图E.5. 在示例E.1的六边形世界搜索问题上，纯前向搜索与结合了动态规划的前向搜索的状态评估次数比较。动态规划通过缓存结果能够避免状态访问量的指数增长。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
function heuristic_search(𝒫::Search, s, d, Uhi, U, M)
    if haskey(M, (d,s))
        return M[(d,s)]
    end
    𝒜, T, R = 𝒫.𝒜(s), 𝒫.T, 𝒫.R
    if isempty(𝒜) || d ≤ 0
        best = (a=nothing, u=U(s))
    else
        best = (a=first(𝒜), u=-Inf)
        for a in sort(𝒜, by=a->R(s,a) + Uhi(T(s,a)), rev=true)
            if R(s,a) + Uhi(T(s,a)) ≤ best.u
                break
            end
            s′ = T(s,a)
            u = R(s,a) + heuristic_search(𝒫, s′, d-1, Uhi, U, M).u
            if u > best.u
                best = (a=a, u=u)
            end
        end
    end
    M[(d,s)] = best
    return best
end
```

算法E.5。启发式搜索算法，用于解决从状态`s`开始并搜索至最大深度`d`的搜索问题$\mathcal{P}$。启发式函数`Uhi`用于引导搜索，在终端状态评估近似价值函数`U`，而形式为包含深度-状态元组的字典的置换表`M`允许算法缓存先前探索过的状态的值。

行动根据即时奖励加上对未来回报的启发式估计进行排序：

$$R(s,a) + \overline{U}(T(s,a)) \tag{E.1}$$

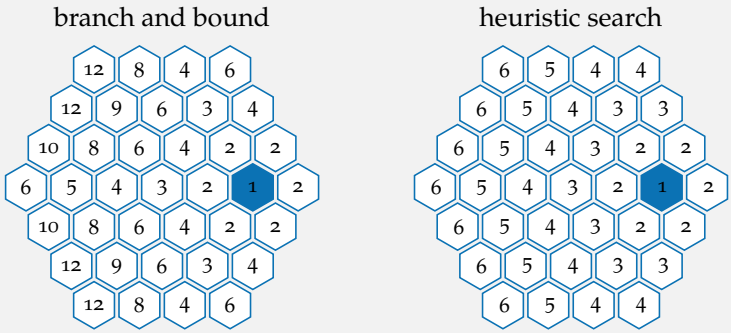
为了保证最优性，启发式函数必须同时满足*admissible*和*consistent*条件。一个可采纳的启发式函数是真实价值函数的上界。一个一致性的启发式函数永远不会小于转移到相邻状态所获得期望奖励的值：

$$\overline{U}(s) \geq R(s,a) + \overline{U}(T(s,a)) \tag{E.2}$$

该方法在示例E.2中与分支定界搜索进行了比较。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

我们可以将启发式搜索应用于与示例E.1中相同的六边形世界搜索问题。我们使用启发式$U(s) = 5 - \delta(s)$，其中$\delta(s)$是从给定状态到终端奖励状态的步数。在这里，我们展示了从每个起始状态运行分支定界（左侧）或启发式搜索（右侧）时访问的状态数量。分支定界在接近目标状态及其左侧的状态中同样高效，而启发式搜索能够从任何初始状态高效搜索。



示例 E.2. 启发式搜索相较于分支定界搜索所能节省的开销对比。启发式搜索会根据其前瞻启发式值自动对动作进行排序。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

F Problems

本节涵盖了本书中用到的一些决策问题。表 F.1 总结了这些问题的一些重要性质。

Problem	$\lvert\mathcal{I}\rvert$	$\lvert\mathcal{S}\rvert$	$\lvert\mathcal{A}\rvert$	$\lvert\mathcal{O}\rvert$	γ
Hex world	—	varies	6	—	0.9
2048	—	∞	4	—	1
Cart-pole	—	$(\subset \mathbb{R}^4)$	2	—	1
Mountain car	—	$(\subset \mathbb{R}^2)$	3	—	1
Simple regulator	—	$(\subset \mathbb{R})$	$(\subset \mathbb{R})$	—	1 or 0.9
Aircraft collision avoidance	—	$(\subset \mathbb{R}^3)$	3	—	1
Crying baby	—	2	3	2	0.9
Machine replacement	—	3	4	2	1
Catch	—	4	10	2	0.9
Prisoner's dilemma	2	—	2 per agent	—	1
Rock-paper-scissors	2	—	3 per agent	—	1
Traveler's dilemma	2	—	99 per agent	—	1
Predator-prey hex world	varies	varies	6 per agent	—	0.9
Multicaregiver crying baby	2	2	3 per agent	2 per agent	0.9
Collaborative predator-prey hex world	varies	varies	6 per agent	2 per agent	0.9

表 F.1. 问题概述。`DecisionMakingProblems.jl` 包实现了这些问题。

F.1 Hex World

*hex world problem*是一个简单的MDP，其中我们必须穿越瓦片地图以到达目标状态。瓦片地图中的每个单元格代表MDP中的一个状态。我们可以尝试向六个方向中的任一方向移动。这些动作的效果是随机的。如图F.1所示，我们以0.7的概率向指定方向移动1步，并以每个相邻方向0.15的概率向其中一个方向移动1步。如果碰到网格的外边界，则我们完全不动，代价为1.0。

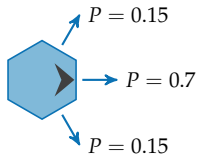


图 F.1。六边形世界问题中的动作具有概率效应。

在某些单元格中采取任何行动都会给予我们指定的奖励，然后将我们传送到终止状态。在终止状态下不会获得进一步的奖励。因此，六边形世界问题中的总状态数是瓦片数量加1，即终止状态。图F.2展示了本书中使用的两种六边形世界问题配置的最优策略。我们将较大的实例称为“六边形世界”，将较小、较简单的实例称为“直线六边形世界”。[1]

直线六边形世界公式用于说明奖励如何从其最右侧单元格上的单一奖励状态传播。

1 直线公式类似于*hall problem*，这是一种常见的基准MDP。例如，参见L. Baird，“残差算法：带函数逼近的强化学习”，载于*International Conference on Machine Learning (ICML)*，1995年。

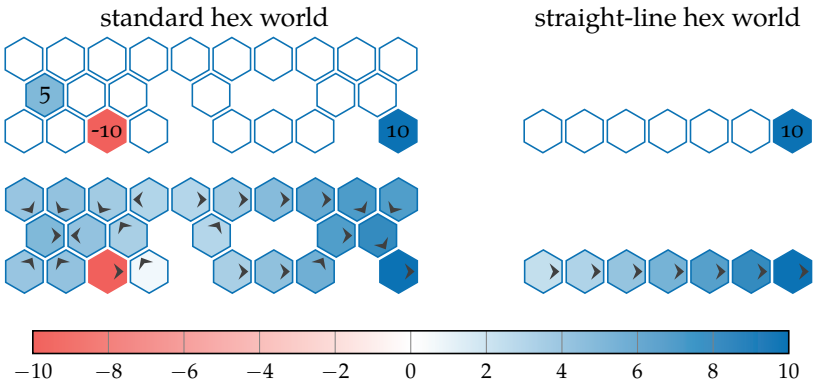


图 F.2. 标准六边形世界与直线六边形世界问题。顶行展示了基本问题设置，并用颜色标出了具有终端奖励的六边形。底行展示了针对每个问题的最优策略，根据期望值进行着色，并用箭头指示在每个状态下应采取的行动。

F.2 2048

2048 problem 基于一款在 4 × 4 棋盘上进行的流行瓷砖游戏。[2] 它具有离散状态和动作空间。棋盘最初除了两个方格为空，每个方格的值可以是2或4。图F.3显示了一个随机选择的初始状态。

2 这款游戏由 Gabriele Cirulli 于 2014 年开发。

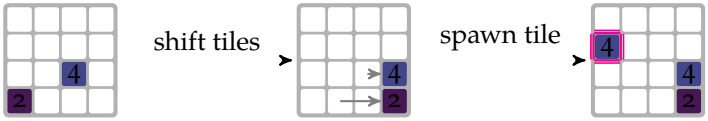


图 F.3. 2048 问题中的随机初始状态由两个瓦片组成，每个瓦片上的值为 2 或 4。

智能体可以移动所有瓦片 `left`、`down`、`right` 或 `up`。选择一个方向会将所有瓦片推向该方向。瓦片在撞到墙壁或另一个数值不同的瓦片时停止。撞到另一个数值相同的瓦片时，该瓦片会与其合并，形成数值相加的新瓦片。在移动和合并之后，会在一个随机空位生成一个数值为 2 或 4 的新瓦片。这一过程如图 F.4 所示。

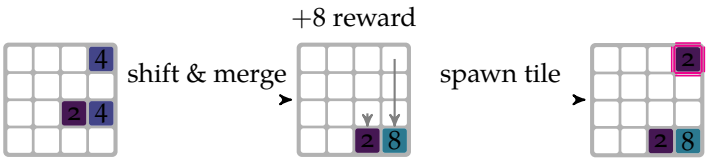
当无法再移动图块以产生空位时，游戏结束。只有在合并两个图块时才能获得奖励，且奖励等于

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com



图F.4。在2048中，一个动作会将所有瓷砖向选定方向移动，然后在空位生成一个新的瓷砖。

合并后的瓦片值。图F.5中展示了一个包含合并操作的状态-动作转换示例。



图F.5。在这里，使用`down`动作来移动所有图块，导致两个4图块合并产生一个8图块，并获得8的奖励。

一个常见的策略是选择一个角落，并在导向该方向的两个动作之间交替进行。这往往会使方块分层，使得数值较大的方块位于角落，而新生成的方块则位于边缘。

F.3 Cart-Pole

cart-pole problem，[3]有时也称为*pole balancing problem*，具有智能体来回移动小车。如图 F.6 所示，该小车上通过一个旋转接头连接了一根刚性杆，因此当小车来回移动时，杆开始旋转。目标是保持杆垂直平衡，同时使小车保持在允许的横向边界内。因此，在满足这些条件的每个时间步都会获得 1 奖励，而每当这些条件不满足时，就会转移到终止性的零奖励状态。

3 A. G. Barto、R. S. Sutton 和 C. W. Anderson，“能解决困难学习控制问题的类神经元自适应元件”，*IEEE Transactions on Systems, Man, and Cybernetics*，第 SMC-13 卷，第 5 期，第 834–846 页，1983 年。

动作是对小车施加向左或向右的力 F。状态空间由四个连续变量定义：小车的横向位置 x、其横向速度 v、杆的角度 θ 以及杆的角速度 ω。该问题涉及多种参数，包括小车的质量 m_{cart}、杆的质量 m_{pole}、杆的长度 ℓ、力的大小 $|F|$、重力加速度 g、时间步长 Δt、最大 x 偏差、最大角度偏差，以及小车与杆之间或小车与其轨道之间的摩擦损失。[4]

4 我们使用OpenAI Gym中实现的参数。G. Brockman，V. Cheung，L. Pettersson，J. Schneider，J. Schulman，J. Tang和W. Zaremba，“OpenAI Gym”，2016年。arXiv: 1606.01540v1。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

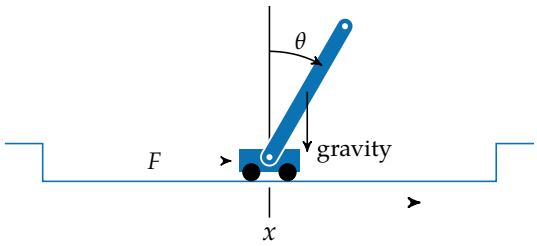


图 F.6. 在小车-摆杆问题中，车辆必须在向左和向右加速之间交替，以保持摆杆平衡。摆杆不允许超过给定角度下落，小车也不允许超出给定的移动范围。

给定输入力 F，杆的角加速度为

$$\alpha = \frac{g\sin(\theta) - \tau\cos(\theta)}{\frac{\ell}{2}\left(\frac{4}{3} - \frac{m_{\text{pole}}}{m_{\text{cart}}+m_{\text{pole}}}\cos(\theta)^2\right)} \tag{F.1}$$

其中

$$\tau = \frac{F + \omega^2 \ell \sin\theta/2}{m_{\text{cart}} + m_{\text{pole}}} \tag{F.2}$$

而侧向小车加速度为

$$a = \tau - \frac{\ell}{2}\alpha\cos(\theta)\frac{m_{\text{pole}}}{m_{\text{cart}} + m_{\text{pole}}} \tag{F.3}$$

状态通过欧拉积分更新：

$$\begin{aligned} x &\leftarrow x + v\Delta t \\ v &\leftarrow v + a\Delta t \\ \theta &\leftarrow \theta + \omega\Delta t \\ \omega &\leftarrow \omega + \alpha\Delta t \end{aligned} \tag{F.4}$$

小车倒立摆问题通常初始化为每个随机值从 $U(-0.05, 0.05)$ 中抽取。运行滚动展开直到横向或角度偏差被超出。

F.4 Mountain Car

在*mountain car problem*、[5]中，车辆必须向右驶出山谷。
谷壁非常陡峭，若速度不足就盲目向目标加速，车辆会停下并滑落。智能体必须学会先向左加速，以便在返回时获得足够的动量，从而冲上山坡。

5 该问题由 A. Moore 在“基于高效内存的机器人控制学习”，剑桥大学博士论文，1990年中提出。其流行的、采用离散动作空间的简化形式，最早由 S. P. Singh 和 R. S. Sutton 在“带有替换资格迹的强化学习”，*Machine Learning*，第22卷，第123–158页，1996年中给出。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

状态是车辆的横向位置 $x \in [-1.2, 0.6]$ 和速度 $v \in [-0.07, 0.07]$。在任何给定的时间步，车辆可以向左加速（$a = -1$）、向右加速（$a = 1$）或滑行（$a = 0$）。我们每回合获得 -1 奖励，并在车辆越过山谷右侧 $x = 0.6$ 时终止。该问题的可视化见图 F.7。

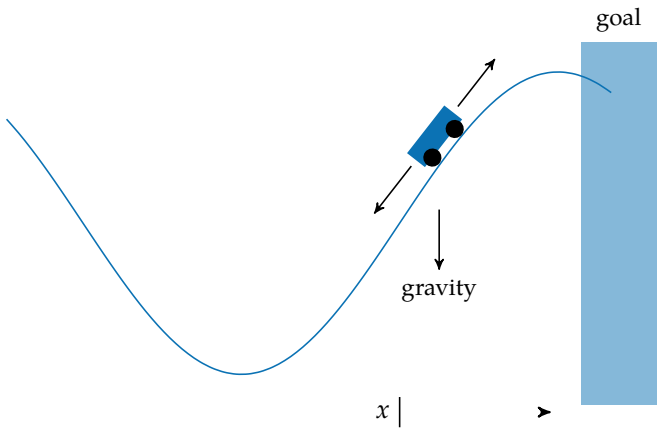


图 F.7. 在山地车问题中，车辆必须在向左和向右加速之间交替，以便使自己爬上山顶。目标区域以蓝色显示。

山地车问题中的状态转移是确定性的：

$$v' \leftarrow v + 0.001a - 0.0025\cos(3x)$$
$$x' \leftarrow x + v'$$

速度更新中的引力项正是将动力不足的车辆推回谷底的因素。状态转移被限制在状态空间的边界内。

山地车问题是一个具有延迟回报问题的好例子。达到目标状态需要许多动作，这使得未经训练的智能体很难收到除了一致的单位惩罚之外的任何东西。最好的学习算法能够有效地将从到达目标的轨迹中获得的知识传播回其余的状态空间。

F.5 Simple Regulator

simple regulator problem 是一个简单的线性二次调节器问题，具有单一状态。它是一个具有单一实值状态和单一实值的马尔可夫决策过程

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

动作。转移动态为线性高斯分布，后继状态s'从高斯分布$\mathcal{N}(s + a, 0.1^2)$中采样得到。奖励函数为二次型，$R(s, a) = -s^2$，且不依赖于动作。本书中的示例使用初始状态分布$\mathcal{N}(0.3, 0.1^2)$。

最优有限时域策略无法使用第7.8节中的方法推导。在这种情况下，$\mathrm{T}_s = [1]$，$\mathrm{T}_a = [1]$，$\mathrm{R}_s = [-1]$，$\mathrm{R}_a = [0]$，且w是从$\mathcal{N}(0, 0.1^2)$中抽取的。Riccati方程的应用要求R_a为负定，而它并非如此。

最优策略是$\pi(s) = -s$，导致后继状态分布以原点为中心。在策略梯度章节中，我们学习了形式为$\pi_\theta(s) = \mathcal{N}(\theta_1 s, \theta_2^2)$的参数化策略。在这种情况下，简单调节器问题的最优参数化为$\theta_1 = -1$，而θ_2渐近接近于零。

简单调节器问题的最优值函数也以原点为中心，奖励呈二次方递减：

$$\begin{aligned} U(s) &= -s^2 + \frac{\gamma}{1-\gamma}\,\mathbb{E}_{s\sim\mathcal{N}(0,0.1^2)}\left[-s^2\right] \\ &\approx -s^2 - 0.010\frac{\gamma}{1-\gamma} \end{aligned}$$

F.6 Aircraft Collision Avoidance

*aircraft collision avoidance problem*涉及决定何时向飞机发出爬升或下降 advisory 以避免闯入飞机。[6] 有三种
对应于无咨询、指令以5米/秒下降、以及指令以5米/秒爬升的动作。入侵者正以恒定的水平接近速度正面逼近我们。状态由我们飞机相对于入侵者飞机的高度 h、我们的垂直速率$\dot{h}$、先前动作 a_{prev} 以及潜在碰撞时间 t_{col} 指定。图F.8展示了问题场景。

6 这个公式是 M. J. Kochenderfer 和 J. P. Chryssanthacopoulos 在“通过动态规划实现鲁棒的空中碰撞避免”中描述问题的高度简化版本，麻省理工学院林肯实验室，项目报告 ATC-371，2011 年。

给定动作 a，状态变量更新如下：

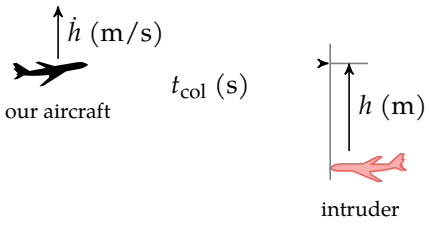
$$h \leftarrow h + \dot{h}\Delta t \tag{F.5}$$

$$\dot{h} \leftarrow \dot{h} + (\ddot{h} + \nu)\Delta t \tag{F.6}$$

$$a_{\text{prev}} \leftarrow a \tag{F.7}$$

$$t_{\text{col}} \leftarrow t_{\text{col}} - \Delta t \tag{F.8}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com



图F.8. 飞机防撞问题的状态变量。

其中 $\Delta t = 1$ 秒，ν 从 -2、0 或 2 m/s^2 上的离散分布中选择，相关概率分别为 0.25、0.5 和 0.25。$\ddot{h}$ 的值由下式给出：

$$\ddot{h} = \begin{cases} 0 & \text{if } a = \text{no advisory} \\ a/\Delta t & \text{if } |a - \dot{h}|/\Delta t < \ddot{h}_{\text{limit}} \\ \text{sign}(a - \dot{h})\ddot{h}_{\text{limit}} & \text{otherwise} \end{cases} \tag{F.9}$$

其中 $\ddot{h}_{\text{limit}} = 1$ m/s^2。

当采取动作且 $t_{\text{col}} < 0$ 时，回合终止。当入侵者在 $t_{\text{col}} = 0$ 时进入 50 米范围内，会有 1 的惩罚，而当 $a \neq a_{\text{prev}}$ 时，会有 0.01 的惩罚。

飞机防撞问题可以通过在离散化网格上使用反向归纳值迭代（第7.6节）高效解决，因为动态特性确定性地减少了t_{col}。最优价值函数和策略的切片如图F.9所示。

F.7 Crying Baby

crying baby problem[7] 是一个简单的 POMDP，具有两个状态、三个动作，以及两个观察结果。我们的目标是照顾婴儿，我们通过在每一步选择喂养婴儿、给婴儿唱歌或忽略婴儿来实现这一目标。

随着时间的推移，婴儿会变得饥饿。我们并不直接观察到婴儿是否饥饿；相反，我们通过婴儿是否哭泣这一带有噪声的观察来获取信息。状态、动作和观察空间如下所述：

$$\mathcal{S} = \{\text{sated}, \text{hungry}\}$$
$$\mathcal{A} = \{\text{feed}, \text{sing}, \text{ignore}\}$$
$$\mathcal{O} = \{\text{crying}, \text{quiet}\}$$

7 本文所呈现的哭闹婴儿问题版本是对M. J. Kochenderfer所著《*Decision Making Under Uncertainty: Theory and Application*》中原始较简单哭闹婴儿问题的扩展。MIT Press，2015年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

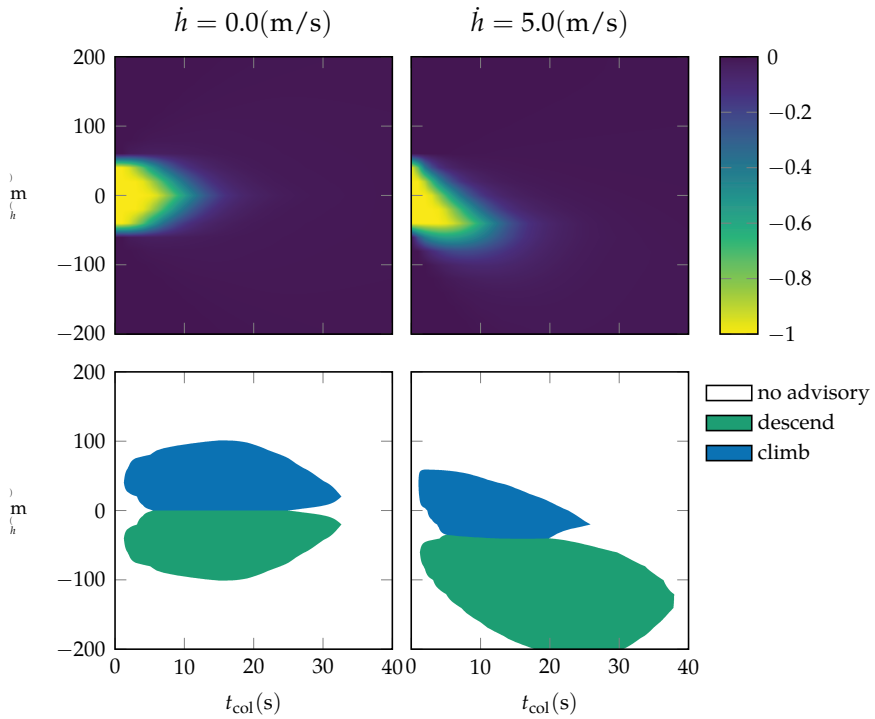


图 F.9. 飞机碰撞避免问题的最优价值函数切片（顶部）和政策切片（底部）。当垂直分离率为0时，价值函数和政策关于0对称，但当垂直分离率非零时，则呈现偏斜。总体而言，我们的飞机在入侵飞机接近之前无需采取行动。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

喂食总是能让婴儿满足。忽视婴儿的风险是，已满足的婴儿可能会变得饥饿，并确保饥饿的婴儿保持饥饿状态。对婴儿唱歌是一种信息收集行为，其状态转移动态与忽视相同，但当婴儿满足（不饥饿）时不会哭闹，而在饥饿时哭闹的概率增加。

转换动态如下：

$$
\begin{aligned}
T(\text{sated} \mid \text{hungry}, \text{feed}) &= 100\,\% \\
T(\text{hungry} \mid \text{hungry}, \text{sing}) &= 100\,\% \\
T(\text{hungry} \mid \text{hungry}, \text{ignore}) &= 100\,\% \\
T(\text{sated} \mid \text{sated}, \text{feed}) &= 100\,\% \\
T(\text{hungry} \mid \text{sated}, \text{sing}) &= 10\,\% \\
T(\text{hungry} \mid \text{sated}, \text{ignore}) &= 10\,\%
\end{aligned}
$$

观测动态如下：

$$
\begin{aligned}
O(\text{cry} \mid \text{feed}, \text{hungry}) &= 80\,\% \\
O(\text{cry} \mid \text{sing}, \text{hungry}) &= 90\,\% \\
O(\text{cry} \mid \text{ignore}, \text{hungry}) &= 80\,\% \\
O(\text{cry} \mid \text{feed}, \text{sated}) &= 10\,\% \\
O(\text{cry} \mid \text{sing}, \text{sated}) &= 0\,\% \\
O(\text{cry} \mid \text{ignore}, \text{sated}) &= 10\,\%
\end{aligned}
$$

奖励函数在婴儿饥饿时分配 -10 的奖励，与所采取的行动无关。喂养婴儿的努力额外增加 -5 的奖励，而唱歌则增加 -0.5 的奖励。作为婴儿看护者，我们寻求具有折扣因子 $\gamma = 0.9$ 的最优无限时域策略。图 F.10 展示了最优价值函数及相关的策略。

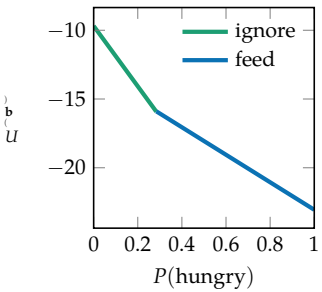


图 F.10. 哭泣婴儿问题的最优策略。该无限 horizon 解在任何信念状态下都不建议唱歌。如图 20.3 所示，在此问题的一些有限 horizon 版本中，唱歌是最优的。

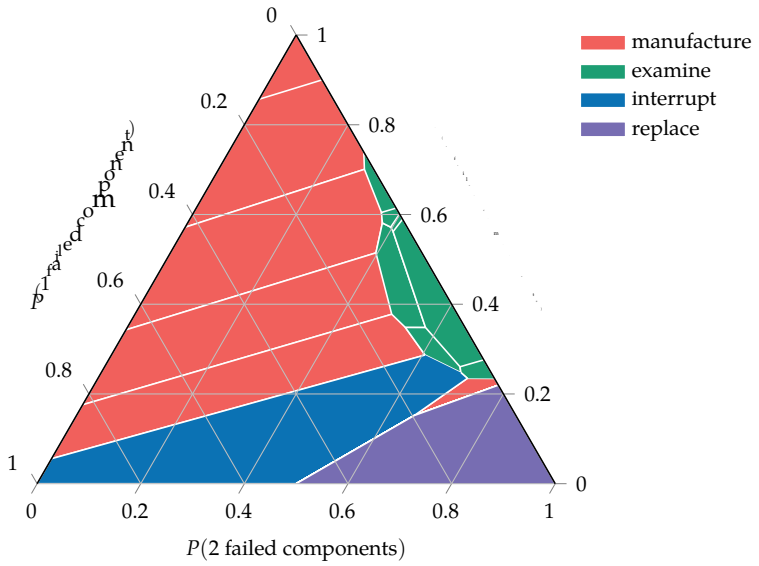
F.8 Machine Replacement

minal rewards

*machine replacement problem*是一个离散的POMDP，其中我们维护一台生产产品的机器。[8] 这个问题因其相对简单而被使用
以及最优策略区域的大小和形状各异。对于某些视界，最优策略甚至存在互不相交的区域，在这些区域中同一动作是最优的，如图F.11所示。

8 R. D. Smallwood 和 E. J. Sondik，“有限时域上部分可观测马尔可夫过程的最优控制”，*Operations Research*，第21卷，第5期，第1071–1088页，1973年。原始问题公式中包含*salvage values*，或*ter-*，等于工作部件的数量。本书中我们未单独建模终端奖励。通过将时域显式包含在问题状态中，终端奖励可以纳入我们的框架。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com



图F.11. 机器更换问题的14步最优策略中，存在多个互不相交的区域，在这些区域内制造是最优的。每个多边形对应一个特定α向量占优的区间。

当机器正常工作时，它为我们生产产品。随着时间推移，机器中的两个主要组件可能会一起或单独发生故障，导致产品有缺陷。我们可以通过检查产品，或直接检查机器内的组件，间接观察机器是否有故障。

该问题具有状态$\mathcal{S} = \{0、1、2\}$，对应于内部故障组件的数量。在生产周期之前，有四种行动可供采用：

1. *manufacture*，制造产品但不检验产品，2. *examine*，制造产品并检验产品，3. *interrupt*，中断生产、检查并更换故障组件，以及4. *replace*，在中断生产后更换两个组件。

当我们检查产品时，我们可以观察它是否有缺陷。所有其他操作只观察非缺陷产品。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

机器中的组件在每个生产周期内独立地有10 %的概率发生故障。每个故障组件有50 %的概率导致产品有缺陷。无缺陷产品净得1奖励，而有缺陷产品净得0奖励。转移动态假设组件故障是在产品制造之前确定的，因此在一台功能完好的机器上执行制造动作并不具有100 %的概率产生1奖励。

制造操作不会产生惩罚。检查产品的成本为0.25。中断生产线检查机器的成本为0.5，不生产任何产品，并且每个损坏组件产生1的成本。仅更换两个组件总是产生2的成本，但没有检查成本。

转移函数、观测函数和奖励函数如表F.2所示。不同增长视界下的最优策略如图F.12所示。

表F.2。机器更换问题的转移、观测和奖励函数。

Action	$T(s' \mid s, a)$	$O(o \mid a, s')$	$R(s, a)$
	s'	o	
manufacture	s $\begin{bmatrix} 0.81 & 0.18 & 0.01 \\ 0 & 0.9 & 0.1 \\ 0 & 0 & 1 \end{bmatrix}$	s' $\begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 1 & 0 \end{bmatrix}$	s $\begin{bmatrix} 0.9025 \\ 0.475 \\ 0.25 \end{bmatrix}$
examine	s $\begin{bmatrix} 0.81 & 0.18 & 0.01 \\ 0 & 0.9 & 0.1 \\ 0 & 0 & 1 \end{bmatrix}$	s' $\begin{bmatrix} 1 & 0 \\ 0.5 & 0.5 \\ 0.25 & 0.75 \end{bmatrix}$	s $\begin{bmatrix} 0.6525 \\ 0.225 \\ 0 \end{bmatrix}$
interrupt	s $\begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}$	s' $\begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 1 & 0 \end{bmatrix}$	s $\begin{bmatrix} -0.5 \\ -1.5 \\ -2.5 \end{bmatrix}$
replace	s $\begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}$	s' $\begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 1 & 0 \end{bmatrix}$	s $\begin{bmatrix} -2 \\ -2 \\ -2 \end{bmatrix}$

F.9 Catch

在*catch problem*中，Johnny希望成功接住父亲的投掷，并且他更倾向于接住远距离的投掷。然而，他不确定投掷距离与成功接住概率之间的关系。他知道，无论他是投掷还是接住，成功接住的概率都是相同的；并且在他不得不回家之前，他有有限次数的尝试接住机会来最大化他的期望效用。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

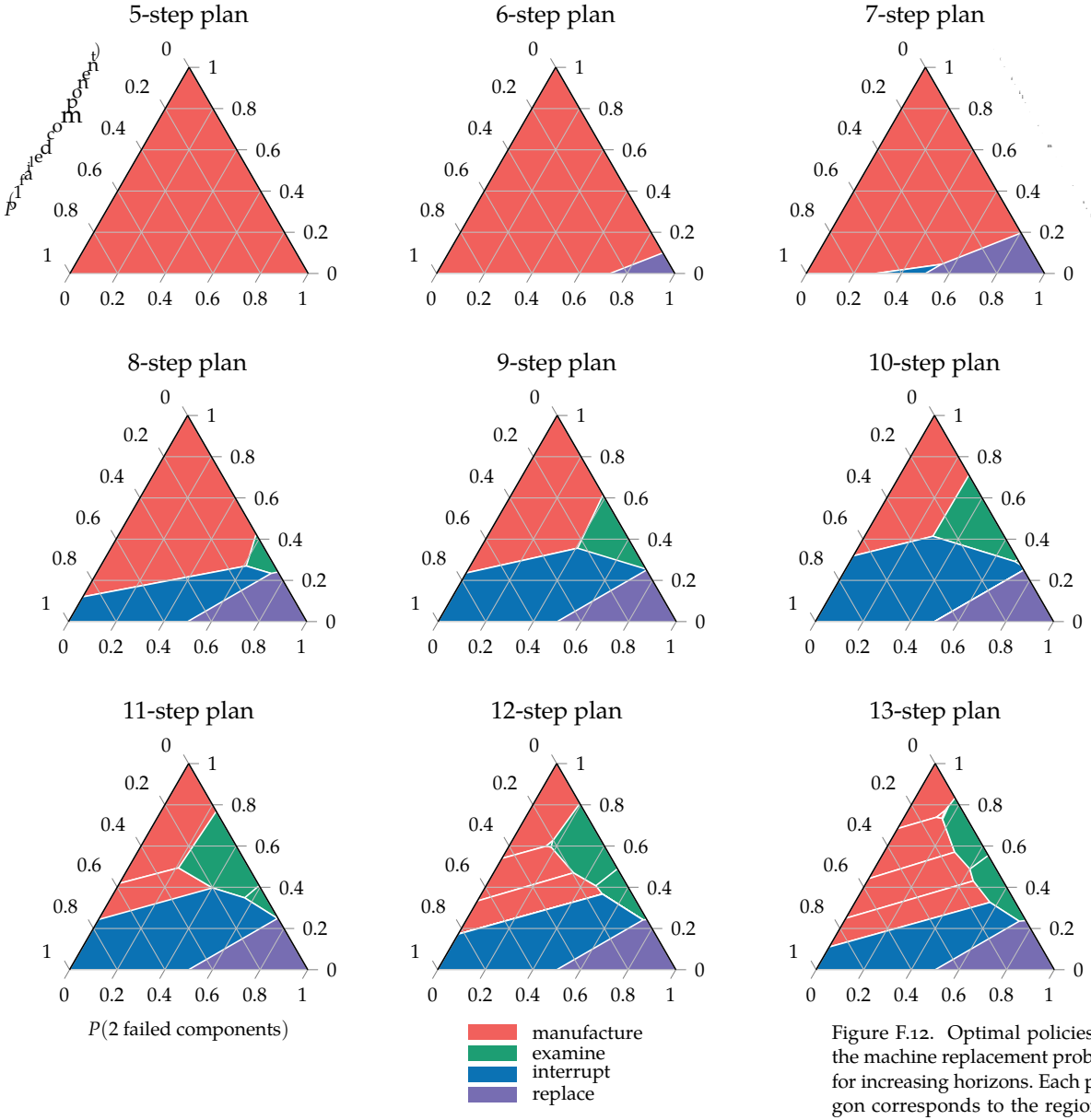


Figure F.12. Optimal policies for the machine replacement problem for increasing horizons. Each polygon corresponds to the region in which a particular alpha vector 占主导地位。

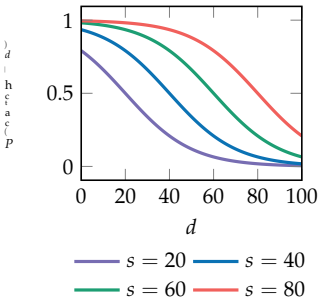
© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 b
ugs@algorithmsbook.com

如图F.13所示，Johnny将成功接住投掷距离为d的球的概率建模为

$$P(\text{catch} \mid d) = 1 - \frac{1}{1 + \exp\left(-\frac{d-s}{15}\right)} \tag{F.10}$$

其中熟练度 s 是未知的，并且不随时间变化。为了使事情易于管理，他假设 s 属于离散集合 $\mathcal{S} = \{20, 40, 60, 80\}$。

成功接住的奖励等于距离。如果接住不成功，则奖励为零。Johnny 希望在有限次数的投掷尝试中最大化奖励。每次投掷时，Johnny 从离散集合 $\mathcal{A} = \{10, 20, \ldots, 100\}$ 中选择一个距离。他开始时在 $\mathcal{S}$ 上具有均匀分布。



图F.13。在$\mathcal{S}$中，四种熟练度对应的接球概率随投掷距离d的变化。

F.10 Prisoner's Dilemma

*prisoner's dilemma*是博弈论中的一个经典问题，涉及目标冲突的代理人。有两名囚犯正在接受审判。他们可以选择合作，对共同犯罪保持沉默，或者背叛，将罪责归咎于对方。如果两人都合作，则两人都服刑一年。如果代理人i合作而另一名代理人$-i$背叛，那么i服刑四年，而$-i$不服刑。如果两人都背叛，那么两人都服刑三年。[9]

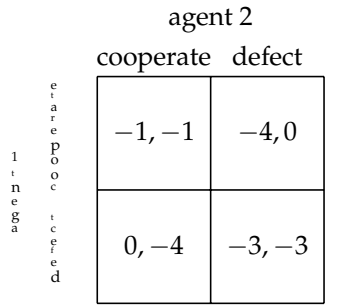
该博弈包含两个智能体，$\mathcal{I} = \{1、2\}$ 和 $\mathcal{A} = \mathcal{A}^1 \times \mathcal{A}^2$，每个智能体可选择 $\mathcal{A}^i = \{$合作、背叛$\}$。图F.14中的表格展示了各自的奖励。行表示智能体1的动作。列表示智能体2的动作。每个单元格中显示了智能体1和2的奖励：$R^1(a^1、a^2)$、$R^2(a^1、a^2)$。该博弈可以玩一次或重复任意次数。在无限期界的情况下，我们使用折现因子 $\gamma = 0.9$。

9 A. W. Tucker 为这个游戏命名并构思了故事。它基于 Merrill Flood 和 Melvin Dresher 在 1950 年于 RAND 提出的原始问题表述。历史背景由 W. Poundstone 提供，*Prisoner's Dilemma*。Doubleday，1992 年。

F.11 Rock-Paper-Scissors

世界各地常见的一种游戏是*rock-paper-scissors*。有两名智能体，各自可以选择石头、剪刀或布。石头胜剪刀，使选择石头的智能体获得单位奖励，选择剪刀的智能体获得单位惩罚。剪刀胜布，使选择剪刀的智能体获得单位奖励，选择布的智能体获得单位惩罚。最后，

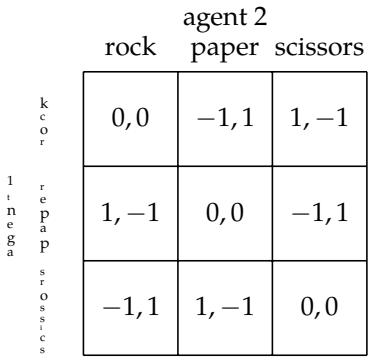
© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com



图F.14。与囚徒困境相关的奖励。

布包石，导致出布的智能体获得单位奖励，而出石的智能体受到单位惩罚。

我们有 $\mathcal{I} = \{1、2\}$ 和 $\mathcal{A} = \mathcal{A}^1 \times \mathcal{A}^2$，每个 $\mathcal{A}^i = \{石头、剪刀、布\}$。图 F.15 显示了与游戏相关的奖励，每个单元格表示 $R^1(a^1、a^2)$、$R^2(a^1、a^2)$。游戏可以玩一次或重复任意次数。在无限期情况下，我们使用折扣因子 $\gamma = 0.9$。



图F.15。与石头剪刀布游戏相关的奖励。

F.12 *Traveler's Dilemma*

*traveler's dilemma*是一个游戏，其中航空公司丢失了两名乘客的两个相同行李箱。[10]航空公司要求乘客写下他们的手提箱，价格在$2到$100之间（含）。如果双方都放下

10 K. Basu，“旅行者困境：博弈论中的理性悖论”，*American Economic Review*，第84卷，第2期，第391-395页，1994年。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

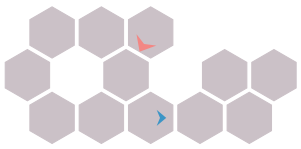
值相同，那么他们都得到那个值。否则，值较低的旅行者得到他们的值加上$2，值较高的旅行者得到较低的值减去$2。换句话说，奖励函数如下：

$$R_i(a_i, a_{-i}) = \begin{cases} a_i & \text{if } a_i = a_{-i} \\ a_i + 2 & \text{if } a_i < a_{-i} \\ a_{-i} - 2 & \text{otherwise} \end{cases} \tag{F.11}$$

大多数人倾向于出价在97到100美元之间。然而，有些反直觉的是，唯一的纳什均衡点仅为2美元。

F.13 Predator-Prey Hex World

predator-prey hex world problem 将六边形世界动力学扩展为包含多个由捕食者和猎物组成的智能体。捕食者试图尽快捕获猎物，而猎物则试图尽可能长时间地逃脱捕食者。六边形世界的初始状态如图 F.16 所示。该博弈中不存在终止状态。



图F.16. 捕食者-猎物六边形世界的初始状态。捕食者为红色，猎物为蓝色。箭头表示个体代理从初始细胞可能采取的行动。

有一组捕食者$\mathcal{I}_{\text{pred}}$和一组猎物$\mathcal{I}_{\text{prey}}$，其中$\mathcal{I} = \mathcal{I}_{\text{pred}} \cup \mathcal{I}_{\text{prey}}$。状态包含每个智能体的位置：$\mathcal{S} = \mathcal{S}^1 \times \cdots \times \mathcal{S}^{|\mathcal{I}|}$，每个$\mathcal{S}^i$等于所有六边形位置。联合动作空间为$\mathcal{A} = \mathcal{A}^1 \times \cdots \times \mathcal{A}^{|\mathcal{I}|}$，其中每个$\mathcal{A}^i$包含所有六个六边形移动方向。

如果捕食者 $i \in \mathcal{I}_{\text{pred}}$ 和猎物 $j \in \mathcal{I}_{\text{prey}}$ 与 $s_i = s_j$ 共处同一六边形，则猎物被吞噬。随后，猎物 j 被传送至随机的六边形单元格，表示其后代出现在世界中。否则，状态转换是独立的，并且如原始六边形世界所述。

如果所有捕食者恰好位于同一个网格单元中，一个或多个捕食者可以捕获一个或多个猎物。如果 n 个捕食者和 m 个猎物都位于同一个网格单元中，捕食者获得 m/n 的奖励。例如，如果两个捕食者共同捕获一个猎物，它们各自获得 1/2 的奖励。如果三个捕食者共同捕获五个猎物，它们各自获得 5/3 的奖励。移动的捕食者会获得单位惩罚。猎物可以不受惩罚地移动，但被吞噬时会受到 100 的惩罚。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

F.14 Multicaregiver Crying Baby

multicaregiver crying baby problem 是哭泣婴儿问题的多智能体扩展。对于每个看护者 $i \in \mathcal{I} = \{1, 2\}$，状态、动作和观察如下：

$$\mathcal{S} = \{\text{hungry}, \text{sated}\} \tag{F.12}$$
$$\mathcal{A}^i = \{\text{feed}, \text{sing}, \text{ignore}\} \tag{F.13}$$
$$\mathcal{O}^i = \{\text{crying}, \text{quiet}\} \tag{F.14}$$

转移动态与原始的哭闹婴儿问题相似，只是任一照护者都可以喂食来满足婴儿：

$$T(\text{sated} \mid \text{hungry}, (\text{feed}, \star)) = T(\text{sated} \mid \text{hungry}, (\star, \text{feed})) = 100\,\% \tag{F.15}$$

其中 $\star$ 表示所有可能的其他变量赋值。否则，如果动作不是喂食，那么婴儿像之前一样在饱腹和饥饿之间转换：

$$T(\text{hungry} \mid \text{hungry}, (\star, \star)) = 100\,\% \tag{F.16}$$
$$T(\text{sated} \mid \text{sated}, (\star, \star)) = 50\,\% \tag{F.17}$$

观测动力学也与单智能体版本类似，但该模型确保两位照料者对婴儿的观测相同，但不一定对彼此的照料动作选择有相同的观测：

$$O((\text{cry}, \text{cry}) \mid (\text{sing}, \star), \text{hungry}) = O((\text{cry}, \text{cry}) \mid (\star, \text{sing}), \text{hungry}) = 90\,\% \tag{F.18}$$
$$O((\text{quiet}, \text{quiet}) \mid (\text{sing}, \star), \text{hungry}) = O((\text{quiet}, \text{quiet}) \mid (\star, \text{sing}), \text{hungry}) = 10\,\% \tag{F.19}$$
$$O((\text{cry}, \text{cry}) \mid (\text{sing}, \star), \text{sated}) = O((\text{cry}, \text{cry}) \mid (\star, \text{sing}), \text{sated}) = 0\,\% \tag{F.20}$$

如果动作不是唱歌，那么观察结果如下：

$$O((\text{cry}, \text{cry}) \mid (\star, \star), \text{hungry}) = O((\text{cry}, \text{cry}) \mid (\star, \star), \text{hungry}) = 90\,\% \tag{F.21}$$
$$O((\text{quiet}, \text{quiet}) \mid (\star, \star), \text{hungry}) = O((\text{quiet}, \text{quiet}) \mid (\star, \star), \text{hungry}) = 10\,\% \tag{F.22}$$
$$O((\text{cry}, \text{cry}) \mid (\star, \star), \text{sated}) = O((\text{cry}, \text{cry}) \mid (\star, \star), \text{sated}) = 0\,\% \tag{F.23}$$
$$O((\text{quiet}, \text{quiet}) \mid (\star, \star), \text{sated}) = O((\text{quiet}, \text{quiet}) \mid (\star, \star), \text{sated}) = 100\,\% \tag{F.24}$$

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

两位照料者都想在婴儿饥饿时帮助婴儿，双方都给予相同的−10.0惩罚。然而，第一位照料者偏好喂食，第二位照料者偏好唱歌。对于喂食，第一位照料者仅额外受到−2.5的惩罚，而第二位照料者额外受到−5.0的惩罚。对于唱歌，第一位照料者受到−0.5的惩罚，而第二位照料者仅受到−0.25的惩罚。

F.15 Collaborative Predator-Prey Hex World

collaborative predator-prey hex world 是捕食者-猎物六边形世界的一个变体，其中一队捕食者追逐一只移动的猎物。捕食者必须合作才能捕获猎物。猎物随机移动到一个没有被捕食者占据的相邻单元格。

捕食者也只对环境进行嘈杂的局部观察。每个捕食者i检测邻近单元格$\mathcal{O}^i = \{\text{prey, nothing}\}$内是否有猎物。捕食者因移动而受到−1奖励的惩罚。如果一个或多个捕食者捕获了猎物，即它们与猎物处于同一单元格，它们将获得10的奖励。此时，猎物被随机分配到一个新的单元格，标志着新猎物的到来，捕食者可以重新开始狩猎。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

G Julia

Julia 是一门免费且开源的科學程式語言。[1] 它是一
相对较新的语言，它从Python、MATLAB和R等语言中汲取灵感。本书选用它，是因为它的抽象层次足够高[2]，从而算法可以被紧凑地表达且易于阅读，同时

同时也很快。本书与 Julia 1.7 版本兼容。本附录介绍了理解所提供代码所需的概念，并省略了该语言的许多高级特性。

1 Julia 可以从 http://julialang.org 获取。

2 与 C++ 等语言不同，Julia 不要求程序员操心内存管理和其他底层细节，但在需要时也允许进行底层控制。

G.1 Types

Julia有多种基本类型，能够表示以真值、数字、字符串、数组、元组和字典形式给出的数据。用户还可以自定义类型。本节将解释如何使用一些基本类型以及如何定义新类型。

G.1.1 Booleans

Julia 中的 *Boolean* 类型，写作 `Bool`，包括值 **`true`** 和 **`false`**。我们可以将这些值赋给变量。变量名可以是任意字符组成的字符串，包括 Unicode，但有一些限制。

```
α = true
done = false
```

变量名出现在等号左侧；要赋给该变量的值出现在右侧。

我们可以在 Julia 控制台中进行赋值。控制台，或称 REPL（即读取、求值、打印、循环），会对正在求值的表达式返回响应。 # 符号表示该行的其余部分是注释。

```
julia> x = true
true
julia> y = false; # semicolon suppresses the console output
julia> typeof(x)
Bool
julia> x == y # test for equality
false
```

支持标准布尔运算符：

```
julia> !x     # not
false
julia> x && y # and
false
julia> x || y # or
true
```

G.1.2 Numbers

朱莉娅支持整数和浮点数，如下所示：

```
julia> typeof(42)
Int64
julia> typeof(42.0)
Float64
```

这里，`Int64`表示一个64位整数，而`Float64`表示一个64位浮点值。[3] 我们可以执行标准的数学运算：

3 在32位机器上，像`42`这样的整数字面量被解释为`Int32`。

```
julia> x = 4
4
julia> y = 2
2
julia> x + y
6
julia> x - y
2
julia> x * y
8
julia> x / y
2.0
```

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

```
julia> x ^ y # exponentiation
16
julia> x % y # remainder from division
0
julia> div(x, y) # truncated division returns an integer
2
```

请注意，`x / y` 的结果是一个 `Float64`，即使 `x` 和 `y` 是整数。我们也可以在执行赋值的同时进行这些操作。例如，`x += 1` 是 `x = x + 1` 的简写。

我们也可以进行比较：

```
julia> 3 > 4
false
julia> 3 >= 4
false
julia> 3 ≥ 4   # unicode also works, use \ge[tab] in console
false
julia> 3 < 4
true
julia> 3 <= 4
true
julia> 3 ≤ 4   # unicode also works, use \le[tab] in console
true
julia> 3 == 4
false
julia> 3 < 4 < 5
true
```

G.1.3 Strings

string 是字符数组。在本教材中，字符串除用于报告某些错误外，使用不多。可以使用 `"` 个字符构造一个类型为 `String` 的对象。例如：

```
julia> x = "optimal"
"optimal"
julia> typeof(x)
String
```

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

G.1.4 Symbols

symbol 表示标识符。可以使用 **:** 运算符编写，或从字符串构造：

```
julia> :A
:A
julia> :Battery
:Battery
julia> Symbol("Failure")
:Failure
```

G.1.5 Vectors

vector 是存储一系列值的一维数组。我们可以使用方括号构造向量，元素之间用逗号分隔：

```
julia> x = [];                      # empty vector
julia> x = trues(3);                # Boolean vector containing three trues
julia> x = ones(3);                 # vector of three ones
julia> x = zeros(3);                # vector of three zeros
julia> x = rand(3);                 # vector of three random numbers between 0 and 1
julia> x = [3, 1, 4];               # vector of integers
julia> x = [3.1415, 1.618, 2.7182]; # vector of floats
```

可以使用 *array comprehension* 创建向量：

```
julia> [sin(x) for x in 1:5]
5-element Vector{Float64}:
  0.8414709848078965
  0.9092974268256817
  0.1411200080598672
 -0.7568024953079282
 -0.9589242746631385
```

我们可以检查向量的类型：

```
julia> typeof([3, 1, 4])              # 1-dimensional array of Int64s
Vector{Int64} (alias for Array{Int64, 1})
julia> typeof([3.1415, 1.618, 2.7182]) # 1-dimensional array of Float64s
Vector{Float64} (alias for Array{Float64, 1})
julia> Vector{Float64} # alias for a 1-dimensional array
Vector{Float64} (alias for Array{Float64, 1})
```

我们使用方括号对向量进行索引：

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
julia> x[1]       # first element is indexed by 1
3.1415
julia> x[3]       # third element
2.7182
julia> x[end]     # use end to reference the end of the array
2.7182
julia> x[end-1]   # this returns the second to last element
1.618
```

我们可以从数组中提取一组元素。范围使用冒号表示法指定：

```
julia> x =  [1, 2, 5, 3, 1]
5-element Vector{Int64}:
 1
 2
 5
 3
 1
julia> x[1:3]      # pull out the first three elements
3-element Vector{Int64}:
 1
 2
 5
julia> x[1:2:end]  # pull out every other element
3-element Vector{Int64}:
 1
 5
 1
julia> x[end:-1:1] # pull out all the elements in reverse order
5-element Vector{Int64}:
 1
 3
 5
 2
 1
```

我们可以对数组执行各种操作。函数名末尾的感叹号用于表示该函数*mutates* (即，更改)输入：

```
julia> length(x)
5
julia> [x, x]             # concatenation
2-element Vector{Vector{Int64}}:
```

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
 [1, 2, 5, 3, 1]
 [1, 2, 5, 3, 1]
julia> push!(x, -1)       # add an element to the end
6-element Vector{Int64}:
  1
  2
  5
  3
  1
 -1
julia> pop!(x)            # remove an element from the end
-1
julia> append!(x, [2, 3]) # append [2, 3] to the end of x
7-element Vector{Int64}:
 1
 2
 5
 3
 1
 2
 3
julia> sort!(x)           # sort the elements, altering the same vector
7-element Vector{Int64}:
 1
 1
 2
 2
 3
 3
 5
julia> sort(x);           # sort the elements as a new vector
julia> x[1] = 2; print(x) # change the first element to 2
[2, 1, 2, 2, 3, 3, 5]
julia> x = [1, 2];
julia> y = [3, 4];
julia> x + y              # add vectors
2-element Vector{Int64}:
 4
 6
julia> 3x - [1, 2]        # multiply by a scalar and subtract
2-element Vector{Int64}:
 2
 4
julia> using LinearAlgebra
```

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
julia> dot(x, y)          # dot product available after using LinearAlgebra
11
julia> x⋅y                # dot product using unicode character, use \cdot[tab] in console
11
julia> prod(y)            # product of all the elements in y
12
```

将各种函数逐元素应用于向量通常很有用。这是*broadcasting*的一种形式。对于中缀运算符（例如`+`、`*`和`^`），会在前面加一个点来表示逐元素广播。对于像`sqrt`和`sin`这样的函数，点则加在后面：

```
julia> x .* y   # elementwise multiplication
2-element Vector{Int64}:
 3
 8
julia> x .^ 2   # elementwise squaring
2-element Vector{Int64}:
 1
 4
julia> sin.(x)  # elementwise application of sin
2-element Vector{Float64}:
 0.8414709848078965
 0.9092974268256817
julia> sqrt.(x) # elementwise application of sqrt
2-element Vector{Float64}:
 1.0
 1.4142135623730951
```

G.1.6 Matrices

matrix 是一个二维数组。与向量类似，它使用方括号构造。我们用空格分隔同一行中的元素，用分号分隔各行。我们还可以对矩阵进行索引，并使用范围输出子矩阵：

```
julia> X = [1 2 3; 4 5 6; 7 8 9; 10 11 12];
julia> typeof(X)  # a 2-dimensional array of Int64s
Matrix{Int64} (alias for Array{Int64, 2})
julia> X[2]       # second element using column-major ordering
4
julia> X[3,2]     # element in third row and second column
8
```

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
julia> X[1,:]     # extract the first row
3-element Vector{Int64}:
 1
 2
 3
julia> X[:,2]     # extract the second column
4-element Vector{Int64}:
  2
  5
  8
 11
julia> X[:,1:2]   # extract the first two columns
4×2 Matrix{Int64}:
  1   2
  4   5
  7   8
 10  11
julia> X[1:2,1:2] # extract a 2x2 submatrix from the top left of x
2×2 Matrix{Int64}:
 1  2
 4  5
julia> Matrix{Float64} # alias for a 2-dimensional array
Matrix{Float64} (alias for Array{Float64, 2})
```

我们还可以构造各种特殊矩阵，并使用数组推导式：

```
julia> Matrix(1.0I, 3, 3)                 # 3x3 identity matrix
3×3 Matrix{Float64}:
 1.0  0.0  0.0
 0.0  1.0  0.0
 0.0  0.0  1.0
julia> Matrix(Diagonal([3, 2, 1]))        # 3x3 diagonal matrix with 3, 2, 1 on diagonal
3×3 Matrix{Int64}:
 3  0  0
 0  2  0
 0  0  1
julia> zeros(3,2)                         # 3x2 matrix of zeros
3×2 Matrix{Float64}:
 0.0  0.0
 0.0  0.0
 0.0  0.0
julia> rand(3,2)                          # 3x2 random matrix
3×2 Matrix{Float64}:
 0.637127  0.839181
```

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

```
 0.850811  0.691
 0.777713  0.556548
julia> [sin(x + y) for x in 1:3, y in 1:2] # array comprehension
3×2 Matrix{Float64}:
  0.909297   0.14112
  0.14112   -0.756802
 -0.756802  -0.958924
```

矩阵运算包括以下内容:

```
julia> X'              # complex conjugate transpose
3×4 adjoint(::Matrix{Int64}) with eltype Int64:
 1  4  7  10
 2  5  8  11
 3  6  9  12
julia> 3X .+ 2         # multiplying by scalar and adding scalar
4×3 Matrix{Int64}:
  5   8  11
 14  17  20
 23  26  29
 32  35  38
julia> X = [1 3; 3 1]; # create an invertible matrix
julia> inv(X)          # inversion
2×2 Matrix{Float64}:
 -0.125   0.375
  0.375  -0.125
julia> pinv(X)         # pseudoinverse (requires LinearAlgebra)
2×2 Matrix{Float64}:
 -0.125   0.375
  0.375  -0.125
julia> det(X)          # determinant (requires LinearAlgebra)
-8.0
julia> [X X]           # horizontal concatenation, same as hcat(X, X)
2×4 Matrix{Int64}:
 1  3  1  3
 3  1  3  1
julia> [X; X]          # vertical concatenation, same as vcat(X, X)
4×2 Matrix{Int64}:
 1  3
 3  1
 1  3
 3  1
julia> sin.(X)         # elementwise application of sin
2×2 Matrix{Float64}:
 0.841471  0.14112
```

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
 0.14112   0.841471
julia> map(sin, X)     # elementwise application of sin
2×2 Matrix{Float64}:
 0.841471  0.14112
 0.14112   0.841471
julia> vec(X)          # reshape an array as a vector
4-element Vector{Int64}:
 1
 3
 3
 1
```

G.1.7 Tuples

一个 *tuple* 是一个有序的值列表，可能包含不同类型的值。它们用括号构造。它们与向量类似，但不能被修改：

```
julia> x = ()   # the empty tuple
()
julia> isempty(x)
true
julia> x = (1,) # tuples of one element need the trailing comma
(1,)
julia> typeof(x)
Tuple{Int64}
julia> x = (1, 0, [1, 2],  2.5029, 4.6692) # third element is a vector
(1, 0, [1, 2], 2.5029, 4.6692)
julia> typeof(x)
Tuple{Int64, Int64, Vector{Int64}, Float64, Float64}
julia> x[2]
0
julia> x[end]
4.6692
julia> x[4:end]
(2.5029, 4.6692)
julia> length(x)
5
julia> x = (1, 2)
(1, 2)
julia> a, b = x;
julia> a
1
julia> b
2
```

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

G.1.8 Named Tuples

一个 *named tuple* 类似于元组，但每个条目都有自己的名称：

```
julia> x = (a=1, b=-Inf)
(a = 1, b = -Inf)
julia> x isa NamedTuple
true
julia> x.a
1
julia> a, b = x;
julia> a
1
julia> (; :a⇒10)
(a = 10,)
julia> (; :a⇒10, :b⇒11)
(a = 10, b = 11)
julia> merge(x, (d=3, e=10))  # merge two named tuples
(a = 1, b = -Inf, d = 3, e = 10)
```

G.1.9 Dictionaries

dictionary 是键值对的集合。键值对用双箭头操作符 `=>` 表示。我们可以像数组和元组一样，使用方括号来索引字典：

```
julia> x = Dict(); # empty dictionary
julia> x[3] = 4 # associate key 3 with value 4
4
julia> x = Dict(3⇒4, 5⇒1) # create a dictionary with two key-value pairs
Dict{Int64, Int64} with 2 entries:
  5 ⇒ 1
  3 ⇒ 4
julia> x[5]         # return the value associated with key 5
1
julia> haskey(x, 3) # check whether dictionary has key 3
true
julia> haskey(x, 4) # check whether dictionary has key 4
false
```

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

G.1.10 Composite Types

composite type 是一组命名字段的集合。默认情况下，复合类型的实例是不可变的（即，它不能改变）。我们使用 **struct** 关键字，然后给新类型一个名称并列出字段的名称：

```
struct A
    a
    b
end
```

添加关键字 **mutable** 使得实例可以改变：

```
mutable struct B
    a
    b
end
```

复合类型使用括号构造，在括号之间我们为每个字段传入值：

```
x = A(1.414, 1.732)
```

双冒号运算符可用于为任意字段指定类型：

```
struct A
    a::Int64
    b::Float64
end
```

这些类型注解要求我们为第一个字段传入 `Int64`，为第二个字段传入 `Float64`。为了简洁，本书不使用类型注解，但这是以性能为代价的。类型注解允许 Julia 提升运行时性能，因为编译器可以针对特定类型优化底层代码。

G.1.11 Abstract Types

到目前为止，我们已经讨论了*concrete types*，这是我们能够构造的类型。然而，具体类型只是类型层次的一部分。还存在*abstracttypes types*，它们是具体类型和其他抽象类型的超类型。

我们可以使用 `supertype` 和 `subtypes` 函数来探索图 G.1 中显示的 `Float64` 类型的类型层次结构：

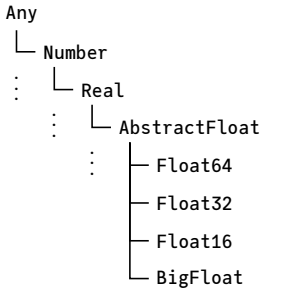


图 G.1. `Float64` 类型的类型层次结构。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
julia> supertype(Float64)
AbstractFloat
julia> supertype(AbstractFloat)
Real
julia> supertype(Real)
Number
julia> supertype(Number)
Any
julia> supertype(Any)          # Any is at the top of the hierarchy
Any
julia> using InteractiveUtils  # required for using subtypes in scripts
julia> subtypes(AbstractFloat) # different types of AbstractFloats
4-element Vector{Any}:
 BigFloat
 Float16
 Float32
 Float64
julia> subtypes(Float64)       # Float64 does not have any subtypes
Type[]
```

我们可以定义自己的抽象类型：

```
abstract type C end
abstract type D <: C end # D is an abstract subtype of C
struct E <: D # E is a composite type that is a subtype of D
    a
end
```

G.1.12 Parametric Types

Julia 支持 *parametric types*，这类类型可以接受参数。参数化类型的参数置于花括号中，并由逗号分隔。我们在字典示例中已经见过参数化类型：

```
julia> x = Dict(3⇒1.4, 1⇒5.9)
Dict{Int64, Float64} with 2 entries:
  3 ⇒ 1.4
  1 ⇒ 5.9
```

对于字典，第一个参数指定键类型，第二个参数指定值类型。示例中有`Int64`个键和`Float64`个值，使得字典类型为`Dict{Int64,Float64}`。Julia能够根据输入推断这些类型，但我们也可以显式指定：

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
julia> x = Dict{Int64,Float64}(3⇒1.4, 1⇒5.9);
```

虽然可以定义我们自己的参数化类型，但本文无需这样做。

G.2 Functions

一个 *function* 将其参数（以元组形式给出）映射到返回的结果。

G.2.1 Named Functions

定义 *named function* 的一种方法是使用 **function** 关键字，其后跟函数名称以及参数名称的元组：

```
function f(x, y)
    return x + y
end
```

我们也可以使用赋值形式来简洁地定义函数：

```
julia> f(x, y) = x + y;
julia> f(3, 0.1415)
3.1415
```

G.2.2 Anonymous Functions

一个 *anonymous function* 没有被赋予名称，尽管它可以被赋值给一个已命名的变量。定义匿名函数的一种方式是使用箭头运算符：

```
julia> h = x -> x^2 + 1 # assign anonymous function with input x to a variable h
#1 (generic function with 1 method)
julia> h(3)
10
julia> g(f, a, b) = [f(a), f(b)]; # applies function f to a and b and returns array
julia> g(h, 5, 10)
2-element Vector{Int64}:
  26
 101
julia> g(x->sin(x)+1, 10, 20)
2-element Vector{Float64}:
 0.4559788891106302
 1.9129452507276277
```

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

G.2.3 Callable Objects

我们可以定义一个类型并将函数与之关联，使得该类型的对象能够被 *callable*：

```
julia> (x::A)() = x.a + x.b    # adding a zero-argument function to the type A defined earlier
julia> (x::A)(y) = y*x.a + x.b # adding a single-argument function
julia> x = A(22, 8);
julia> x()
30
julia> x(2)
52
```

G.2.4 Optional Arguments

我们可以为参数指定默认值，使该参数的指定变为可选：

```
julia> f(x=10) = x^2;
julia> f()
100
julia> f(3)
9
julia> f(x, y, z=1) = x*y + z;
julia> f(1, 2, 3)
5
julia> f(1, 2)
3
```

G.2.5 Keyword Arguments

函数可以使用*keyword arguments*，这些是在调用函数时命名的参数。关键字参数位于所有位置参数之后。分号置于所有关键字之前，将它们与其他参数分隔开：

```
julia> f(; x = 0) = x + 1;
julia> f()
1
julia> f(x = 10)
11
julia> f(x, y = 10; z = 2) = (x + y)*z;
julia> f(1)
```

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
22
julia> f(2, z = 3)
36
julia> f(2, 3)
10
julia> f(2, 3, z = 1)
5
```

G.2.6 Dispatch

可以使用双冒号运算符来指定传递给函数的参数类型。如果提供了同一函数的多个方法，Julia 将执行适当的方法。选择执行哪个方法的机制称为 *dispatch*：

```
julia> f(x::Int64) = x + 10;
julia> f(x::Float64) = x + 3.1415;
julia> f(1)
11
julia> f(1.0)
4.141500000000001
julia> f(1.3)
4.4415000000000004
```

将使用类型签名与所给参数类型最匹配的方法：

```
julia> f(x) = 5;
julia> f(x::Float64) = 3.1415;
julia> f([3, 2, 1])
5
julia> f(0.00787499699)
3.1415
```

G.2.7 Splatting

使用 `...` 运算符将向量或元组的元素 *splat* 到函数参数中通常很有用：

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
julia> f(x,y,z) = x + y - z;
julia> a = [3, 1, 2];
julia> f(a...)
2
julia> b = (2, 2, 0);
julia> f(b...)
4
julia> c = ([0,0],[1,1]);
julia> f([2,2], c...)
2-element Vector{Int64}:
 1
 1
```

G.3 Control Flow

我们可以使用条件求值和循环来控制程序的流程。本节提供了书中使用的一些语法。

G.3.1 Conditional Evaluation

条件求值将检查布尔表达式的值，然后求值相应的代码块。最常见的方法之一是使用一个**if**语句：

```
if x < y
    # run this if x < y
elseif x > y
    # run this if x > y
else
    # run this if x == y
end
```

我们还可以使用带有问号和冒号语法的 *ternary operator*。它会先检查问号前的布尔表达式。如果表达式求值为真，则返回冒号之前的内容；否则，返回冒号之后的内容：

```
julia> f(x) = x > 0 ? x : 0;
julia> f(-10)
0
julia> f(10)
10
```

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

G.3.2 Loops

loop 允许重复求值表达式。一种循环是 while 循环，它重复求值一个表达式块，直到满足 **while** 关键字后指定的条件。以下示例对数组 X 中的值求和：

```
X = [1, 2, 3, 4, 6, 8, 11, 13, 16, 18]
s = 0
while !isempty(X)
    s += pop!(X)
end
```

另一种循环类型是 for 循环，它使用 **for** 关键字。以下示例也将对数组 X 中的值求和，但不会修改 X：

```
X = [1, 2, 3, 4, 6, 8, 11, 13, 16, 18]
s = 0
for y in X
    s += y
end
```

in 关键字可以替换为 = 或 ∈。以下代码块是等效的：

```
X = [1, 2, 3, 4, 6, 8, 11, 13, 16, 18]
s = 0
for i = 1:length(X)
    s += X[i]
end
```

G.3.3 Iterators

我们可以在诸如for循环和数组推导式等上下文中遍历集合。为了演示各种迭代器，我们将使用collect函数，它返回由迭代器生成的所有项组成的数组：

```
julia> X = ["feed", "sing", "ignore"];
julia> collect(enumerate(X)) # return the count and the element
3-element Vector{Tuple{Int64, String}}:
 (1, "feed")
 (2, "sing")
 (3, "ignore")
julia> collect(eachindex(X)) # equivalent to 1:length(X)
3-element Vector{Int64}:
```

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
 1
 2
 3
julia> Y = [-5, -0.5, 0];
julia> collect(zip(X, Y))    # iterate over multiple iterators simultaneously
3-element Vector{Tuple{String, Float64}}:
 ("feed", -5.0)
 ("sing", -0.5)
 ("ignore", 0.0)
julia> import IterTools: subsets
julia> collect(subsets(X))   # iterate over all subsets
8-element Vector{Vector{String}}:
 []
 ["feed"]
 ["sing"]
 ["feed", "sing"]
 ["ignore"]
 ["feed", "ignore"]
 ["sing", "ignore"]
 ["feed", "sing", "ignore"]
julia> collect(eachindex(X)) # iterate over indices into a collection
3-element Vector{Int64}:
 1
 2
 3
julia> Z = [1 2; 3 4; 5 6];
julia> import Base.Iterators: product
julia> collect(product(X,Y)) # iterate over Cartesian product of multiple iterators
3×3 Matrix{Tuple{String, Float64}}:
 ("feed", -5.0)    ("feed", -0.5)    ("feed", 0.0)
 ("sing", -5.0)    ("sing", -0.5)    ("sing", 0.0)
 ("ignore", -5.0)  ("ignore", -0.5)  ("ignore", 0.0)
```

G.4 Packages

package 是一个 Julia 代码和可能其他外部库的集合，可以导入以提供额外功能。本节简要回顾了我们在本书中所依赖的几个关键包。要添加像 `Distributions.jl` 这样的注册包，我们可以运行

```
using Pkg
Pkg.add("Distributions")
```

要更新软件包，我们使用

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
Pkg.update()
```

要使用一个包，我们按如下方式使用关键字**using**：

```
using Distributions
```

G.4.1 Graphs.jl

我们使用 `Graphs.jl` 包（版本 1.4）来表示图并对其进行操作：

```
julia> using Graphs
julia> G = SimpleDiGraph(3); # create a directed graph with three nodes
julia> add_edge!(G, 1, 3);   # add edge from node 1 to 3
julia> add_edge!(G, 1, 2);   # add edge from node 1 to 2
julia> rem_edge!(G, 1, 3);   # remove edge from node 1 to 3
julia> add_edge!(G, 2, 3);   # add edge from node 2 to 3
julia> typeof(G)
Graphs.SimpleGraphs.SimpleDiGraph{Int64}
julia> nv(G)                 # number of nodes (also called vertices)
3
julia> outneighbors(G, 1)    # list of outgoing neighbors for node 1
1-element Vector{Int64}:
 2
julia> inneighbors(G, 1)     # list of incoming neighbors for node 1
Int64[]
```

G.4.2 Distributions.jl

我们使用 `Distributions.jl` 包（版本0.24）来表示、拟合概率分布并从中抽样
：

```
julia> using Distributions
julia> dist = Categorical([0.3, 0.5, 0.2]) # create a categorical distribution
Distributions.Categorical{Float64, Vector{Float64}}(support=Base.OneTo(3), p=[0.3, 0.5, 0.2])
julia> data = rand(dist)               # generate a sample
1
julia> data = rand(dist, 2)            # generate two samples
2-element Vector{Int64}:
 1
 3
julia> μ, σ = 5.0, 2.5;                # define parameters of a normal distribution
julia> dist = Normal(μ, σ)             # create a normal distribution
Distributions.Normal{Float64}(μ=5.0, σ=2.5)
```

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 b
ugs@algorithmsbook.com

```
julia> rand(dist)                       # sample from the distribution
4.376087913615021
julia> data = rand(dist, 3)             # generate three samples
3-element Vector{Float64}:
 2.7998404479350363
 4.428211389440531
 7.86857737850412
julia> data = rand(dist, 1000);         # generate many samples
julia> Distributions.fit(Normal, data) # fit a normal distribution to the samples
Distributions.Normal{Float64}(μ=5.161545315532672, σ=2.53067978547563)
julia> μ = [1.0, 2.0];
julia> Σ = [1.0 0.5; 0.5 2.0];
julia> dist = MvNormal(μ, Σ)            # create a multivariate normal distribution
FullNormal(
dim: 2
μ: [1.0, 2.0]
Σ: [1.0 0.5; 0.5 2.0]
)
julia> rand(dist, 3)                    # generate three samples
2×3 Matrix{Float64}:
 0.246293  1.15997  3.65139
 1.31659   2.45144  2.67241
julia> dist = Dirichlet(ones(3))        # create a Dirichlet distribution Dir(1,1,1)
Distributions.Dirichlet{Float64, Vector{Float64}, Float64}(alpha=[1.0, 1.0, 1.0])
julia> rand(dist)                       # sample from the distribution
3-element Vector{Float64}:
 0.6788340679291495
 0.2725086391073811
 0.04865729296346948
```

G.4.3 JuMP.jl

我们使用 `JuMP.jl` 包（版本0.21）来指定优化问题，随后可以使用多种求解器求解，例如包含在 `GLPK.jl` 和 `Ipopt.jl` 中的那些求解器：

```
julia> using JuMP
julia> using GLPK
julia> model = Model(GLPK.Optimizer)         # create model and use GLPK as solver
A JuMP Model
Feasibility problem with:
Variables: 0
Model mode: AUTOMATIC
CachingOptimizer state: EMPTY_OPTIMIZER
```

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
Solver name: GLPK
julia> @variable(model, x[1:3])              # define variables x[1], x[2], and x[3]
3-element Vector{JuMP.VariableRef}:
 x[1]
 x[2]
 x[3]
julia> @objective(model, Max, sum(x) - x[2]) # define maximization objective
x[1] + 0 x[2] + x[3]
julia> @constraint(model, x[1] + x[2] ≤ 3)   # add constraint
x[1] + x[2] <= 3
julia> @constraint(model, x[2] + x[3] ≤ 2)   # add another constraint
x[2] + x[3] <= 2
julia> @constraint(model, x[2] ≥ 0)          # add another constraint
x[2] >= 0
julia> optimize!(model)                      # solve
julia> value.(x)                             # extract optimal values for elements in x
3-element Vector{Float64}:
 3.0
 0.0
 2.0
```

G.5 Convenience Functions

有一些函数可以让我们更紧凑地指定本书中的算法。以下函数在处理字典和命名元组时非常有用：

```
Base.Dict{Symbol,V}(a::NamedTuple) where V =
    Dict{Symbol,V}(n⇒v for (n,v) in zip(keys(a), values(a)))
Base.convert(::Type{Dict{Symbol,V}}, a::NamedTuple) where V =
    Dict{Symbol,V}(a)
Base.isequal(a::Dict{Symbol,<:Any}, nt::NamedTuple) =
    length(a) == length(nt) &&
    all(a[n] == v for (n,v) in zip(keys(nt), values(nt)))
```

```
julia> a = Dict{Symbol,Integer}((a=1, b=2, c=3))
Dict{Symbol, Integer} with 3 entries:
  :a ⇒ 1
  :b ⇒ 2
  :c ⇒ 3
julia> isequal(a, (a=1, b=2, c=3))
true
julia> isequal(a, (a=1, c=3, b=2))
true
```

© 2022 麻省理工学院，根据知识共享 CC-BY-NC-ND 许可协议共享。2024-02-06 20:54:49-08:00，评论至 bugs@algorithmsbook.com

```
julia> Dict{Dict{Symbol,Integer},Float64}((a=1, b=1)⇒0.2, (a=1, b=2)⇒0.8)
Dict{Dict{Symbol, Integer}, Float64} with 2 entries:
  Dict(:a⇒1, :b⇒1) ⇒ 0.2
  Dict(:a⇒1, :b⇒2) ⇒ 0.8
```

我们定义SetCategorical来表示离散集合上的分布：

```
struct SetCategorical{S}
    elements::Vector{S} # Set elements (could be repeated)
    distr::Categorical # Categorical distribution over set elements

    function SetCategorical(elements::AbstractVector{S}) where S
        weights = ones(length(elements))
        return new{S}(elements, Categorical(normalize(weights, 1)))
    end

    function SetCategorical(
            elements::AbstractVector{S},
            weights::AbstractVector{Float64}
        ) where S

        ℓ₁ = norm(weights,1)
        if ℓ₁ < 1e-6 || isinf(ℓ₁)
            return SetCategorical(elements)
        end
        distr = Categorical(normalize(weights, 1))
        return new{S}(elements, distr)
    end
end

Distributions.rand(D::SetCategorical) = D.elements[rand(D.distr)]
Distributions.rand(D::SetCategorical, n::Int) = D.elements[rand(D.distr, n)]
function Distributions.pdf(D::SetCategorical, x)
    sum(e == x ? w : 0.0 for (e,w) in zip(D.elements, D.distr.p))
end
```

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

```
julia> D = SetCategorical(["up", "down", "left", "right"],[0.4, 0.2, 0.3, 0.1]);
julia> rand(D)
"up"
julia> rand(D, 5)
5-element Vector{String}:
 "left"
 "left"
 "down"
 "down"
 "left"
julia> pdf(D, "up")
0.3999999999999999
```

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

References

1. P. Abbeel 和 A. Y. Ng, “通过逆向强化学习进行学徒学习”，载于 *International Conference on Machine Learning* (*ICML*)，2004 年（引自第 361 页）。2. J. Agar，*Science in the 20th Century and Beyond*。Polity，2012 年（引自第 10 页）。3. S. Amari，“自然梯度在学习中高效工作”，*Neural Computation*，第 10 卷，第 2 期，第 251–276 页，1998 年（引自第 253 页）。4. C. Amato、D. S. Bernstein 和 S. Zilberstein，“为 POMDP 和分散式 POMDP 优化固定大小随机控制器”，*Autonomous Agents and Multi-Agent Systems*，第 21 卷，第 3 期，第 293–320 页，2010 年（引自第 478、551 页）。5. D. Amodei、C. Olah、J. Steinhardt、P. Christiano、J. Schulman 和 D. Mané，“AI 安全中的具体问题”，2016 年。arXiv: 1606.06565v2（引自第 13 页）。6. P. Anand，“偏好公理真的理性吗？”*Theory and Decision*，第 23 卷，第 2 期，第 189–214 页，1987 年（引自第 112 页）。7. D. Ariely，*Predictably Irrational: The Hidden Forces That Shape Our Decisions*。Harper，2008 年（引自第 122 页）。8. S. Arnborg、D. G. Corneil 和 A. Proskurowski，“在 *k*-树中寻找嵌入的复杂性”，*SIAM Journal on Algebraic Discrete Methods*，第 8 卷，第 2 期，第 277–284 页，1987 年（引自第 52 页）。9. M. S. Arulampalam、S. Maskell、N. Gordon 和 T. Clapp，“在线非线性/非高斯贝叶斯跟踪的粒子滤波教程”，*IEEE Transactions on Signal Processing*，第 50 卷，第 2 期，第 174–188 页，2002 年（引自第 390 页）。10. K. J. Åström，“状态信息不完全的马尔可夫过程的最优控制”，*Journal of Mathematical Analysis and Applications*，第 10 卷，第 1 期，第 174–205 页，1965 年（引自第 407 页）。11. P. Auer、N. Cesa-Bianchi 和 P. Fischer，“多臂老虎机问题的有限时间分析”，*Machine Learning*，第 47 卷，第 2–3 期，第 235–256 页，2002 年（引自第 187 页）。

12. T. Ayer, O. Alagoz, and N. K. Stout, "A POMDP Approach to Personalize Mam- mograph y Screening Decisions," *Operations Research*, vol. 60, no. 5, pp. 1019–1034, 2012 (cit. on p. 4) . 13. H. Bai, D. Hsu, W. S. Lee, and V. A. Ngo, "Monte Carlo Value Iteration for Continuous-State POMDPs," in *International Workshop on the Algorithmic Foundations of Robotics (WAFR)*, 2011 (cit. on p. 475). 14. L. Baird, "Residual Algorithms: Reinforcement Learning w ith Function Approxima- tion," in *International Conference on Machine Learning (ICML)*, 199 5 (cit. on p. 610). 15. Y. Bar-Shalom, X. R. Li, and T. Kirubarajan, *Estimation with Applications to Tracking and Navigation*. Wiley, 2001 (cit. on p. 383). 16. D. B arber, *Bayesian Reasoning and Machine Learning*. Cambridge University Press, 2012 (cit. on p. 53). 17. A. G. Barto, R. S. Sutton, and C. W. Anderson, "Neuronlike Adaptive Elements That C an Solve Difficult Learning Control Problems," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-13, no. 5, pp. 834–846, 1983 (cit. on p. 611). 18. A. G. Barto, S. J. B radtke, and S. P. Singh, "Learning to Act Using Real-Time Dynamic Programming," *Artificial Intelligence*, vol. 72, no. 1–2, pp. 81–138, 1995 (cit. on p. 197). 19. K. Basu, "The Tra veler's Dilemma: Paradoxes of Rationality in Game Theory," *American Economic Review*, vo l. 84, no. 2, pp. 391–395, 1994 (cit. on p. 622). 20. R. Bellman, "Minimization Problem," *Bulletin of the American Mathematical Society*, vol. 62, no. 3, p. 270, 1956 (cit. on p. 399). 21. R. Bellman, *Eye of the Hurricane: An Autobiography*. World Scientific, 1984 (cit. on p. 136). 2 2. R. E. Bellman, *Dynamic Programming*. Princeton University Press, 1957 (cit. on pp. 133, 13 8). 23. A. Bemporad and M. Morari, "Robust Model Predictive Control: A Survey," in *Robustness in Identification and Control*, A. Garulli, A. Tesi, and A. Vicino, eds., Springer, 19 99, pp. 207–226 (cit. on p. 204). 24. J. Bentham, *Theory of Legislation*. Trübner & Company, 18 87 (cit. on p. 8). 25. U. Berger, "Brown's Original Fictitious Play," *Journal of Economic Theory*, vol. 135, no. 1, pp. 572–578, 2007 (cit. on p. 507). 26. D. Bernoull i, "Exposition of a New Theory on the Measurement of Risk," *Econo- metrica*, vol. 22, no. 1, pp. 23–36, 1954 (cit. on p. 112). 27. D. S. Bernstein, R. Givan, N. Immerman, and S. Zilberstein, "The Complexity of Decentralized Control of Markov Decision Processes," *Mathematics of Operation Research*, vol. 27, no. 4, pp. 819–840, 2002 (cit. on pp. 545, 546).

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

28. D. P. Bertsekas，*Dynamic Programming and Optimal Control*。Athena Scientific，2007（引自第148页）。 29. D. P. Bertsekas，*Reinforcement Learning and Optimal Control*。Athena Scientific，2019（引自第335页）。 30. D. P. Bertsekas 和 J. N. Tsitsiklis，*Introduction to Probability*。Athena Scientific，2002（引自第20页）。 31. M. Besançon、T. Papamarkou、D. Anthoff、A. Arslan、S. Byrne、D. Lin 和 J. Pearson，“Distributions.jl：JuliaStats生态系统中概率分布的定义与建模”，2019年。arXiv：1907.08611v1（引自第573页）。 32. W. M. Bolstad 和 J. M. Curran，*Introduction to Bayesian Statistics*。Wiley，2016（引自第11页）。 33. B. Bonet 和 H. Geffner，“Labeled RTDP：改进实时动态规划的收敛性”，载于*International Conference on Automated Planning and Scheduling* (*ICAPS*)，2003年（引自第197页）。 34. F. Borrelli、A. Bemporad 和 M. Morari，*Predictive Control for Linear and Hybrid Systems*。Cambridge University Press，2019（引自第200页）。 35. M. Bouton、A. Nakhaei、K. Fujimura 和 M. J. Kochenderfer，“用于复杂城市环境导航的场景分解安全强化学习”，载于*IEEE Intelligent Vehicles Symposium* (*IV*)，2019年（引自第3页）。 36. M. Bouton、J. Tumova 和 M. J. Kochenderfer，“部分可观测马尔可夫决策过程中基于点的模型检测方法”，载于*AAAI Conference on Artificial Intelligence* (*AAAI*)，2020年（引自第293页）。 37. M. Bowling，“多智能体学习中的收敛性与无遗憾”，载于*Advances in Neural Information Processing Systems* (*NIPS*)，2005年（引自第509页）。 38. M. Bowling 和 M. Veloso，“多智能体强化学习的随机博弈论分析”，卡内基梅隆大学，技术报告 CMU-CS-00-165，2000年（引自第521页）。 39. S. Boyd 和 L. Vandenberghe，*Convex Optimization*。Cambridge University Press，2004（引自第200、565页）。 40. R. I. Brafman 和 M. Tennenholtz，“R-MAX——一种用于近最优强化学习的通用多项式时间算法”，*Journal of Machine Learning Re- search*，第3卷，第213–231页，2002年（引自第323页）。 41. D. Brockhoff、A. Auger、N. Hansen、D. Arnold 和 T. Hohm，“进化策略的镜像采样与序列选择”，载于*International Conference on Parallel Problem Solving from Nature*，2010年（引自第224页）。 42. G. Brockman、V. Cheung、L. Pettersson、J. Schneider、J. Schulman、J. Tang 和 W. Zaremba，“OpenAI Gym”，2016年。arXiv：1606.01540v1（引自第611页）。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

43. G. W. Brown, “通过虚拟对局的迭代博弈求解”，*Activity Analysis of Production and Allocation*，第13卷，第1期，第374–376页，1951年（引自第505页）。44. C. B. Browne、E. Powley、D. Whitehouse、S. M. Lucas、P. I. Cowling、P. Rohlfshagen、S. Tavener、D. Perez、S. Samothrakis 和 S. Colton，“蒙特卡洛树搜索方法综述”，*IEEE Transactions on Computational Intelligence and AI in Games*，第4卷，第1期，第1–43页，2012年（引自第187、276页）。45. J. A. Bucklew，*Introduction to Rare Event Simulation*。Springer，2004年（引自第287页）。46. W. L. Buntine，“贝叶斯网络上的理论精化”，载于 *Conference on Uncertainty in Artificial Intelligence* (*UAI*)，1991年（引自第104页）。47. C. F. Camerer，*Behavioral Game Theory: Experiments in Strategic Interaction*。Princeton University Press，2003年（引自第504页）。48. A. R. Cassandra、M. L. Littman 和 N. L. Zhang，“增量剪枝：一种简单、快速、精确的部分可观测马尔可夫决策过程方法”，载于 *Conference on Uncertainty in Artificial Intelligence* (*UAI*)，1997年（引自第416页）。49. J. Chakravorty 和 A. Mahajan，“多臂老虎机、Gittins 指数及其计算”，载于 *Methods and Applications of Statistics in Clinical Trials*，N. Balakrishnan 编，第2卷，Wiley，2014年，第416–435页（引自第309页）。50. D. M. Chickering，“学习贝叶斯网络是NP完全的”，载于 *Learning from Data: Artificial Intelligence and Statistics V*，D. Fisher 和 H.-J. Lenz 编，Springer，1996年，第121–130页（引自第97页）。51. D. M. Chickering，“学习贝叶斯网络结构的等价类”，*Journal of Machine Learning Research*，第2卷，第445–498页，2002年（引自第106页）。52. D. M. Chickering、D. Heckerman 和 C. Meek，“贝叶斯网络的大样本学习是NP困难的”，*Journal of Machine Learning Research*，第5卷，第1287–1330页，2004年（引自第97页）。53. K. Cho、B. van Merriënboer、C. Gulcehre、D. Bahdanau、F. Bougares、H. Schwenk 和 Y. Bengio，“使用RNN编码器-解码器学习短语表示用于统计机器翻译”，载于 *Conference on Empirical Methods in Natural Language Processing* (*EMNLP*)，2014年（引自第592页）。54. E. K. P. Chong、R. L. Givan 和 H. S. Chang，“一种通过后见之明优化进行基于仿真的网络控制的框架”，载于 *IEEE Conference on Decision and Control* (*CDC*)，2000年（引自第207页）。55. B. Christian，*The Alignment Problem*。Norton & Company，2020年（引自第13页）。56. G. F. Cooper，“使用贝叶斯信念网络进行概率推断的计算复杂性”，*Artificial Intelligence*，第42卷，第2–3期，第393–405页，1990年（引自第53页）。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

57. G. F. Cooper 和 E. Herskovits，“一种从数据归纳概率网络的贝叶斯方法”，*Machine Learning*，第 4 卷，第 9 期，第 309–347 页，1992 年（引自第 97、100 页）。 58. T. H. Cormen、C. E. Leiserson、R. L. Rivest 和 C. Stein，*Introduction to Algorithms*，第 3 版。MIT Press，2009 年（引自第 136 页）。 59. A. Corso、R. J. Moss、M. Koren、R. Lee 和 M. J. Kochenderfer，“黑盒安全验证算法综述”，*Journal of Artificial Intelligence Research*，第 72 卷，第 377–428 页，2021 年（引自第 281 页）。 60. A. Couëtoux、J.-B. Hoock、N. Sokolovska、O. Teytaud 和 N. Bonnard，“连续上置信树”，载于 *Learning and Intelligent Optimization* (*LION*)，2011 年（引自第 197 页）。 61. F. Cuzzolin，*The Geometry of Uncertainty*。Springer，2021 年（引自第 19 页）。 62. G. B. Dantzig，“线性规划”，*Operations Research*，第 50 卷，第 1 期，第 42–47 页，2002 年（引自第 8 页）。 63. C. Daskalakis、P. W. Goldberg 和 C. H. Papadimitriou，“计算纳什均衡的复杂性”，*Communications of the ACM*，第 52 卷，第 2 期，第 89–97 页，2009 年（引自第 498 页）。 64. A. P. Dempster、N. M. Laird 和 D. B. Rubin，“通过 EM 算法从不完整数据中求最大似然”，*Journal of the Royal Statistical Society, Series B* (*Methodological*)，第 39 卷，第 1 期，第 1–38 页，1977 年（引自第 87 页）。 65. S. L. Dittmer 和 F. V. Jensen，“影响图中信息的短视价值”，载于 *Conference on Uncertainty in Artificial Intelligence* (*UAI*)，1997 年（引自第 119 页）。 66. J. Duchi、S. Shalev-Shwartz、Y. Singer 和 T. Chandra，“高维学习中 ℓ_1-球上的高效投影”，载于 *International Conference on Machine Learning* (*ICML*)，2008 年（引自第 485 页）。 67. M. J. Dupré 和 F. J. Tipler，“严格贝叶斯概率的新公理”，*Bayesian Analysis*，第 4 卷，第 3 期，第 599–606 页，2009 年（引自第 20 页）。 68. M. Egorov、Z. N. Sunberg、E. Balaban、T. A. Wheeler、J. K. Gupta 和 M. J. Kochenderfer，“POMDPs.jl：不确定性下序贯决策的框架”，*Journal of Machine Learning Research*，第 18 卷，第 26 期，第 1–5 页，2017 年（引自第 381 页）。 69. C. Elkan，“成本敏感学习的基础”，载于 *International Joint Con- ference on Artificial Intelligence* (*IJCAI*)，2001 年（引自第 373 页）。 70. P. H. Farquhar，“效用评估方法”，*Management Science*，第 30 卷，第 11 期，第 1283–1300 页，1984 年（引自第 114 页）。 71. J. A. Filar、T. A. Schultz、F. Thuijsman 和 O. Vrieze，“随机博弈中的非线性规划与平稳均衡”，*Mathematical Programming*，第 50 卷，第 1–3 期，第 227–237 页，1991 年（引自第 521 页）。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

72. A. M. Fink, “随机*n*人博弈中的均衡”, *Journal of Science of the Hiroshima University, Series A-I*, 第28卷, 第1期, 第89–93页, 1964年（引用自第520页）。73. C. Finn、S. Levine和P. Abbeel, “引导成本学习：通过策略优化的深度逆最优控制”, 载于*International Conference on Machine Learning* (*ICML*), 2016年（引用自第368页）。74. P. C. Fishburn, “效用理论”, *Management Science*, 第14卷, 第5期, 第335–378页, 1968年（引用自第111页）。75. P. C. Fishburn, “主观概率的公理”, *Statistical Science*, 第1卷, 第3期, 第335–345页, 1986年（引用自第20页）。76. H. Freudenthal, “有界平坦性的单纯剖分”, *Annals of Mathematics*, 第43卷, 第580–582页, 1942年（引用自第445页）。77. M. C. Fu, “梯度估计”, 载于*Simulation*, S. G. Henderson和B. L. Nelson编, Elsevier, 2006年, 第575–616页（引用自第231页）。78. D. Fudenberg和D. Levine, “一致性与谨慎虚拟博弈”, *Journal of Economic Dynamics and Control*, 第19卷, 第5–7期, 第1065–1089页, 1995年（引用自第509页）。79. D. Fudenberg和J. Tirole, *Game Theory*。MIT Press, 1991年（引用自第493页）。80. D. Gaines、G. Doran、M. Paton、B. Rothrock、J. Russino、R. Mackey、R. Anderson、R. Francis、C. Joswig、H. Justice、K. Kolcio、G. Rabideau、S. Schaffer、J. Sawoniewicz、A. Vasavada、V. Wong、K. Yu和A.-a. Agha-mohammadi, “自力更生的漫游车以提高任务生产率”, *Journal of Field Robotics*, 第37卷, 第7期, 第1171–1196页, 2020年（引用自第5页）。81. F. L. Gall, “张量的幂与快速矩阵乘法”, 载于*International Symposium on Symbolic and Algebraic Computation* (*ISSAC*), 2014年（引用自第490页）。82. S. Garatti和M. C. Campi, “控制设计中的鲁棒性调节：原理与算法”, *IEEE Control Systems Magazine*, 第33卷, 第2期, 第36–51页, 2013年（引用自第207页）。83. A. Garivier、T. Lattimore和E. Kaufmann, “关于先探索后承诺策略”, 载于*Advances in Neural Information Processing Systems* (*NIPS*), 2016年（引用自第303页）。84. A. Geramifard、T. J. Walsh、S. Tellex、G. Chowdhary、N. Roy和J. P. How, “动态规划与强化学习线性函数逼近器教程”, *Foundations and Trends in Machine Learning*, 第6卷, 第4期, 第375–451页, 2013年（引用自第162页）。85. M. Ghavamzadeh、S. Mannor、J. Pineau和A. Tamar, “贝叶斯强化学习：综述”, *Foundations and Trends in Machine Learning*, 第8卷, 第5–6期, 第359–483页, 2015年（引用自第326页）。86. S. B. Gillispie和M. D. Perlman, “无环有向图模型的马尔可夫等价类的大小分布”, *Artificial Intelligence*, 第141卷, 第1–2期, 第137–155页, 2002年（引用自第106页）。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

87. J. C. Gittins, “Bandit Processes and Dynamic Allocation Indices,” *Journal of the Royal Statistical Society. Series B (Methodological)*, 第41卷, 第2期, 第148–177页, 1979年（见第299、309页）。 88. J. Gittins、K. Glazebrook 和 R. Weber, *Multi-Armed Bandit Allocation Indices*, 第2版。Wiley, 2011年（见第309页）。 89. P. W. Glynn, “Likelihood Ratio Gradient Estimation for Stochastic Systems,” *Communications of the ACM*, 第33卷, 第10期, 第75–84页, 1990年（见第234页）。 90. P. J. Gmytrasiewicz 和 P. Doshi, “A Framework for Sequential Planning in Multi- Agent Settings,” *Journal of Artificial Intelligence Research*, 第24卷, 第1期, 第49–79页, 2005年（见第534页）。 91. D. E. Goldberg 和 J. Richardson, “An Experimental Comparison of Localization Methods,” 载于 *International Conference on Genetic Algorithms*, 1987年（见第394页）。 92. D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*。Addison-Wesley, 1989年（见第215页）。 93. O. Goldreich, *Computational Complexity: A Conceptual Perspective*。Cambridge Uni- versity Press, 2008年（见第575页）。 94. I. Goodfellow、Y. Bengio 和 A. Courville, *Deep Learning*。MIT Press, 2016年（见第581页）。 95. I. Goodfellow、J. Pouget-Abadie、M. Mirza、B. Xu、D. Warde-Farley、S. Ozair、A. Courville 和 Y. Bengio, “Generative Adversarial Nets,” 载于 *Advances in Neural Information Processing Systems (NIPS)*, 2014年（见第597页）。 96. L. Graesser 和 W. L. Keng, *Foundations of Deep Reinforcement Learning*。Addison Wesley, 2020年（见第344页）。 97. A. Greenwald 和 K. Hall, “Correlated Q-Learning,” 载于 *International Conference on Machine Learning (ICML)*, 2003年（见第503页）。 98. A. Griewank 和 A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 第2版。SIAM, 2008年（见第585页）。 99. E. A. Hansen, “Solving POMDPs by Searching in Policy Space,” 载于 *Conference on Uncertainty in Artificial Intelligence (UAI)*, 1998年（见第475页）。 100. E. A. Hansen、D. S. Bernstein 和 S. Zilberstein, “Dynamic Programming for Par- tially Observable Stochastic Games,” 载于 *AAAI Conference on Artificial Intelligence (AAAI)*, 2004年（见第533页）。 101. N. Hansen 和 A. Ostermeier, “Adapting Arbitrary Normal Mutation Distributions in Evolution Strategies: The Covariance Matrix Adaptation,” 载于 *IEEE International Conference on Evolutionary Computation*, 1996年（见第221页）。 102. A. Harutyunyan、M. G. Bellemare、T. Stepleton 和 R. Munos, “Q(λ) with Off- Policy Corrections,” 载于 *International Conference on Algorithmic Learning Theory (ALT)*, 2016年（见第343页）。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

103. T. Hastie、R. Tibshirani 和 J. Friedman，*The Elements of Statistical Learning: Data Mining, Inference, and Prediction*，第2版。Springer统计学系列，2001年（引用自第172、174页）。104. M. Hauskrecht，“部分可观测马尔可夫决策过程的价值函数近似”，*Journal of Artificial Intelligence Research*，第13卷，第33-94页，2000年（引用自第427、429、436页）。105. D. Heckerman、D. Geiger 和 D. M. Chickering，“学习贝叶斯网络：知识与统计数据的结合”，*Machine Learning*，第20卷，第3期，第197-243页，1995年（引用自第104页）。106. F. S. Hillier，*Introduction to Operations Research*。McGraw-Hill，2012年（引用自第12页）。107. J. Ho 和 S. Ermon，“生成对抗模仿学习”，载于 *Advances in Neural Information Processing Systems* (*NIPS*)，2016年（引用自第369页）。108. S. Hochreiter 和 J. Schmidhuber，“长短期记忆”，*Neural Computation*，第9卷，第8期，第1735-1780页，1997年（引用自第592页）。109. A. L. Hodgkin 和 A. F. Huxley，“膜电流的定量描述及其在神经传导与兴奋中的应用”，*Journal of Physiology*，第117卷，第4期，第500-544页，1952年（引用自第582页）。110. R. Hooke 和 T. A. Jeeves，“数值与统计问题的直接搜索解法”，*Journal of the ACM* (*JACM*)，第8卷，第2期，第212-229页，1961年（引用自第215页）。111. R. A. Howard，“信息价值理论”，*IEEE Transactions on Systems Science and Cybernetics*，第2卷，第1期，第22-26页，1966年（引用自第119页）。112. J. Hu 和 M. P. Wellman，“一般和随机博弈的纳什Q学习”，*Journal of Machine Learning Research*，第4卷，第1039-1069页，2003年（引用自第526页）。113. A. Hussein、M. M. Gaber、E. Elyan 和 C. Jayne，“模仿学习：学习方法综述”，*ACM Computing Surveys*，第50卷，第2期，第1-35页，2017年（引用自第355页）。114. IEEE历史中心工作人员，“IEEE会议录百年历程：2000-2009”，*Proceedings of the IEEE*，第100卷，第11期，第3131-3145页，2012年（引用自第387页）。115. J. E. Ingersoll，*Theory of Financial Decision Making*。Rowman and Littlefield出版社，1987年（引用自第115页）。116. G. N. Iyengar，“鲁棒动态规划”，*Mathematics of Operations Research*，第30卷，第2期，第257-280页，2005年（引用自第289页）。117. T. Jaakkola、M. I. Jordan 和 S. P. Singh，“关于随机迭代动态规划算法的收敛性”，*Neural Computation*，第6卷，第6期，第1185-1201页，1994年（引用自第336页）。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

118. E. T. Jaynes, “信息论与统计力学”, *Physical Review*, 第 106 卷, 第 4 期, 第 620–630 页, 1957 年（引自第 368 页）。119. E. T. Jaynes, *Probability Theory: The Logic of Science*。剑桥大学出版社, 2003 年（引自第 19、20 页）。120. F. V. Jensen 和 T. D. Nielsen, *Bayesian Networks and Decision Graphs*, 第 2 版。Springer, 2007 年（引自第 116 页）。121. I. L. Johansen 和 M. Rausand, “风险度量的基础与选择”, *Safety Science*, 第 62 卷, 第 386–399 页, 2014 年（引自第 281 页）。122. K. D. Julian 和 M. J. Kochenderfer, “使用深度强化学习的自主飞行器分布式野火监测”, *AIAA Journal of Guidance, Control, and Dynamics*, 第 42 卷, 第 8 期, 第 1768–1778 页, 2019 年（引自第 4 页）。123. S. J. Julier 和 J. K. Uhlmann, “无迹滤波与非线性估计”, *Proceedings of the IEEE*, 第 92 卷, 第 3 期, 第 401–422 页, 2004 年（引自第 387 页）。124. L. P. Kaelbling、M. L. Littman 和 A. R. Cassandra, “部分可观测随机域中的规划与行动”, *Artificial Intelligence*, 第 101 卷, 第 1–2 期, 第 99–134 页, 1998 年（引自第 407 页）。125. L. P. Kaelbling, *Learning in Embedded Systems*。MIT Press, 1993 年（引自第 305 页）。126. A. B. Kahn, “大型网络的拓扑排序”, *Communications of the ACM*, 第 5 卷, 第 11 期, 第 558–562 页, 1962 年（引自第 55 页）。127. D. Kahneman 和 A. Tversky, “前景理论：风险下决策的分析”, *Econometrica*, 第 47 卷, 第 2 期, 第 263–292 页, 1979 年（引自第 122 页）。128. S. M. Kakade, “自然策略梯度”, 载于 *Advances in Neural Information Pro- cessing Systems* (*NIPS*), 2001 年（引自第 254 页）。129. S. M. Kakade 和 J. Langford, “近似最优的近似强化学习”, 载于 *International Conference on Machine Learning* (*ICML*), 2002 年（引自第 256 页）。130. R. E. Kálmán, “线性滤波与预测问题的新方法”, *ASME Journal of Basic Engineering*, 第 82 卷, 第 35–45 页, 1960 年（引自第 383 页）。131. R. M. Karp, “组合问题之间的可归约性”, 载于 *Complexity of Com- puter Computations*, R. E. Miller 和 J. W. Thatcher 编, Plenum, 1972 年, 第 85–103 页（引自第 577 页）。132. E. Kaufmann, “关于序贯资源分配的贝叶斯索引策略”, *Annals of Statistics*, 第 46 卷, 第 2 期, 第 842–865 页, 2018 年（引自第 305 页）。133. M. Kearns 和 S. Singh, “多项式时间内的近乎最优强化学习”, *Machine Learning*, 第 49 卷, 第 2/3 期, 第 209–232 页, 2002 年（引自第 323 页）。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

134. M. J. Kearns、Y. Mansour 和 A. Y. Ng，“A Sparse Sampling Algorithm for Near-Optimal Planning in Large Markov Decision Processes，” *Machine Learning*，第 49 卷，第 2–3 期，第 193–208 页，2002 年（引自第 187 页）。135. L. G. Khachiyan，“Polynomial Algorithms in Linear Programming，” *USSR Compu- tational Mathematics and Mathematical Physics*，第 20 卷，第 1 期，第 53–72 页，1980 年（引自第 147 页）。136. D. Kingma 和 M. Welling，“Auto-Encoding Variational Bayes，” 载于 *International Conference on Learning Representations* (*ICLR*)，2013 年（引自第 593 页）。137. D. E. Kirk，*Optimal Control Theory: An Introduction*。Prentice-Hall，1970 年（引自第 2 页）。138. M. J. Kochenderfer，*Decision Making Under Uncertainty: Theory and Application*。MIT Press，2015 年（引自第 3、511、615 页）。139. M. J. Kochenderfer 和 T. A. Wheeler，*Algorithms for Optimization*。MIT Press，2019 年（引自第 102、172、213、250 页）。140. M. J. Kochenderfer 和 J. P. Chryssanthacopoulos，“Robust Airborne Collision Avoidance Through Dynamic Programming，” Massachusetts Institute of Technology，Lincoln Laboratory，Project Report ATC-371，2011 年（引自第 614 页）。141. M. J. Kochenderfer、J. P. Chryssanthacopoulos 和 P. Radecki，“Robustness of Optimized Collision Avoidance Logic to Modeling Errors，” 载于 *Digital Avionics Systems Conference* (*DASC*)，2010 年（引自第 289 页）。142. D. Koller 和 N. Friedman，*Probabilistic Graphical Models: Principles and Techniques*。MIT Press，2009 年（引自第 32、36、97 页）。143. A. Kolmogorov，*Foundations of the Theory of Probability*，第 2 版。Chelsea，1956 年（引自第 562 页）。144. H. Koontz，“The Management Theory Jungle，” *Academy of Management Journal*，第 4 卷，第 3 期，第 174–188 页，1961 年（引自第 12 页）。145. B. O. Koopman，*Search and Screening: General Principles with Historical Applications*。Pergamon Press，1980 年（引自第 11 页）。146. F. Kschischang、B. Frey 和 H.-A. Loeliger，“Factor Graphs and the Sum-Product Algorithm，” *IEEE Transactions on Information Theory*，第 47 卷，第 2 期，第 498–519 页，2001 年（引自第 53 页）。147. A. Kuefler、J. Morton、T. A. Wheeler 和 M. J. Kochenderfer，“Imitating Driver Behavior with Generative Adversarial Networks，” 载于 *IEEE Intelligent Vehicles Sym- posium* (*IV*)，2017 年（引自第 375 页）。148. H. Kuhn，“Extensive Games and the Problem of Information，” 载于 *Contributions to the Theory of Games II*，H. Kuhn 和 A. Tucker 编，Princeton University Press，1953 年，第 193–216 页（引自第 533 页）。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

149. S. Kullback 和 R. A. Leibler, “On Information and Sufficiency,” *Annals of Mathematical Statistics*，第 22 卷，第 1 期，第 79–86 页，1951 年（引自第 567 页）。150. S. Kullback，*Information Theory and Statistics*。Wiley，1959 年（引自第 567 页）。151. H. Kurniawati、D. Hsu 和 W. S. Lee，“SARSOP: Efficient Point-Based POMDP Planning by Approximating Optimally Reachable Belief Spaces,” 载于 *Robotics: Science and Systems*，2008 年（引自第 440、442 页）。152. Y. LeCun、L. Bottou、Y. Bengio 和 P. Haffner，“Gradient-Based Learning Applied to Document Recognition,” *Proceedings of the IEEE*，第 86 卷，第 11 期，第 2278–2324 页，1998 年（引自第 587、588 页）。153. R. Lee、M. J. Kochenderfer、O. J. Mengshoel、G. P. Brat 和 M. P. Owen，“Adaptive Stress Testing of Airborne Collision Avoidance Systems,” 载于 *Digital Avionics Systems Conference* (*DASC*)，2015 年（引自第 294 页）。154. J. Lehrer，*How We Decide*。Houghton Mifflin，2009 年（引自第 122 页）。155. T. P. Lillicrap、J. J. Hunt、A. Pritzel、N. Heess、T. Erez、Y. Tassa、D. Silver 和 D. Wierstra，“Continuous Control with Deep Reinforcement Learning,” 载于 *International Conference on Learning Representations* (*ICLR*)，2016 年。arXiv: 1509.02971v6（引自第 274 页）。156. L.-J. Lin，“Reinforcement Learning for Robots Using Neural Networks,” 博士论文，Carnegie Mellon University，1993 年（引自第 345 页）。157. R. J. A. Little 和 D. B. Rubin，*Statistical Analysis with Missing Data*，第 3 版。Wiley，2020 年（引自第 84 页）。158. M. L. Littman，“Markov Games as a Framework for Multi-Agent Reinforcement Learning,” 载于 *International Conference on Machine Learning* (*ICML*)，1994 年（引自第 517 页）。159. M. L. Littman、A. R. Cassandra 和 L. P. Kaelbling，“Learning Policies for Partially Observable Environments: Scaling Up,” 载于 *International Conference on Machine Learning* (*ICML*)，1995 年（引自第 427 页）。160. W. S. Lovejoy，“Computationally Feasible Bounds for Partially Observed Markov Decision Processes,” *Operations Research*，第 39 卷，第 1 期，第 162–175 页，1991 年（引自第 445 页）。161. O. Madani、S. Hanks 和 A. Condon，“On the Undecidability of Probabilistic Planning and Related Stochastic Optimization Problems,” *Artificial Intelligence*，第 147 卷，第 1–2 期，第 5–34 页，2003 年（引自第 427 页）。162. S. Mannor、R. Y. Rubinstein 和 Y. Gat，“The Cross Entropy Method for Fast Policy Search,” 载于 *International Conference on Machine Learning* (*ICML*)，2003 年（引自第 218 页）。163. H. Markowitz，“The Utility of Wealth,” *Journal of Political Economy*，第 60 卷，第 2 期，第 151–158 页，1952 年（引自第 114 页）。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

164. Mausam 和 A. Kolobov，*Planning with Markov Decision Processes: An AI Perspective*。Morgan & Claypool，2012（引自第197页）。165. S. B. McGrayne，*The Theory That Would Not Die*。耶鲁大学出版社，2011（引自第30页）。166. R. C. Merton，“连续时间模型中的最优消费与投资组合规则”，*Journal of Economic Theory*，第3卷，第4期，第373–413页，1971（引自第4页）。167. N. Meuleau、K.-E. Kim、L. P. Kaelbling 和 A. R. Cassandra，“通过搜索有限策略空间求解POMDP”，载于*Conference on Uncertainty in Artificial Intelligence* (*UAI*)，1999（引自第481页）。168. D. A. Mindell，*Between Human and Machine: Feedback, Control, and Computing Before Cybernetics*。JHU出版社，2002（引自第11页）。169. V. Mnih、K. Kavukcuoglu、D. Silver、A. Graves、I. Antonoglou、D. Wierstra 和 M. Riedmiller，“用深度强化学习玩Atari”，2013。arXiv: 1312.5602v1（引自第345页）。170. N. Moehle、E. Busseti、S. Boyd 和 M. Wytock，“动态能源管理”，载于*Large Scale Optimization in Supply Chains and Smart Manufacturing*，Springer，2019，第69–126页（引自第208页）。171. G. Molenberghs、G. Fitzmaurice、M. G. Kenward、A. Tsiatis 和 G. Verbeke 编，*Handbook of Missing Data Methodology*。CRC出版社，2014（引自第82页）。172. A. Moore，“用于机器人控制的高效基于记忆的学习”，博士论文，剑桥大学，1990（引自第612页）。173. A. W. Moore，“单纯形网格生成及其应用”，博士论文，康奈尔大学，1992（引自第168、170页）。174. A. W. Moore 和 C. G. Atkeson，“优先扫描：以更少数据和更少时间的强化学习”，*Machine Learning*，第13卷，第1期，第103–130页，1993（引自第321页）。175. G. E. Moore，“将更多元件塞进集成电路”，*Electronics*，第38卷，第8期，第114–117页，1965（引自第11页）。176. O. Morgenstern 和 J. von Neumann，*Theory of Games and Economic Behavior*。普林斯顿大学出版社，1953（引自第8页）。177. R. Motwani 和 P. Raghavan，*Randomized Algorithms*。剑桥大学出版社，1995（引自第54页）。178. B. Müller、J. Reinhardt 和 M. T. Strickland，*Neural Networks*。Springer，1995（引自第581页）。179. K. P. Murphy，*Probabilistic Machine Learning: An Introduction*。MIT出版社，2022（引自第71页）。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

180. R. B. Myerson, *Game Theory: Analysis of Conflict*。哈佛大学出版社，1997年（引用自第493页）。 181. R. Nair、M. Tambe、M. Yokoo、D. Pynadath和S. Marsella，“驯服去中心化POMDP：面向多智能体环境的高效策略计算”，载于 *International Joint Conference on Artificial Intelligence* (*IJCAI*)，2003年（引用自第550页）。 182. J. Nash，“非合作博弈”，*Annals of Mathematics*，第286–295页，1951年（引用自第498页）。 183. R. E. Neapolitan，*Learning Bayesian Networks*。普林斯顿霍尔出版社，2003年（引用自第97页）。 184. A. Y. Ng、D. Harada和S. Russell，“奖励变换下的策略不变性：理论及其在奖励塑形中的应用”，载于*International Conference on Machine Learning* (*ICML*)，1999年（引用自第343页）。 185. A. Y. Ng和M. Jordan，“一种用于大型MDP和POMDP的策略搜索方法”，载于 *Conference on Uncertainty in Artificial Intelligence* (*UAI*)，2000年（引用自第232页）。 186. N. J. Nilsson，*The Quest for Artificial Intelligence*。剑桥大学出版社，2009年（引用自第7、9页）。 187. N. Nisan、T. Roughgarden、É. Tardos和V. V. Vazirani编，*Algorithmic Game Theory*。剑桥大学出版社，2007年（引用自第503页）。 188. F. A. Oliehoek和C. Amato，*A Concise Introduction to Decentralized POMDPs*。斯普林格出版社，2016年（引用自第545页）。 189. C. Papadimitriou和J. Tsitsiklis，“马尔可夫决策过程的复杂性”，*Mathematics of Operation Research*，第12卷，第3期，第441–450页，1987年（引用自第427、549页）。 190. J. Pearl，*Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*。摩根考夫曼出版社，1988年（引用自第36页）。 191. J. Pearl，*Causality: Models, Reasoning, and Inference*，第2版。剑桥大学出版社，2009年（引用自第33页）。 192. J. Peng和R. J. Williams，“增量多步Q学习”，*Machine Learning*，第22卷，第1–3期，第283–290页，1996年（引用自第341页）。 193. J. Peters和S. Schaal，“基于策略梯度的运动技能强化学习”，*Neural Networks*，第21卷，第4期，第682–697页，2008年（引用自第234、243、253页）。 194. M. Peterson，*An Introduction to Decision Theory*。剑桥大学出版社，2009年（引用自第111页）。 195. A. Pinkus，“神经网络中MLP模型的逼近理论”，*Acta Numerica*，第8卷，第143–195页，1999年（引用自第582页）。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

196. R. Platt Jr.、R. Tedrake、L. P. Kaelbling 和 T. Lozano-Pérez，“假设最大似然观测的信念空间规划”，载于 *Robotics: Science and Systems*，2010年（引用见第454页）。197. D. A. Pomerleau，“用于自主导航的人工神经网络高效训练”，*Neural Computation*，第3卷，第1期，第88–97页，1991年（引用见第355页）。198. W. Poundstone，*Prisoner's Dilemma*。Doubleday，1992年（引用见第621页）。199. P. Poupart 和 C. Boutilier，“有界有限状态控制器”，载于 *Advances in Neural Information Processing Systems* (*NIPS*)，2003年（引用见第475页）。200. W. B. Powell，*Reinforcement Learning and Stochastic Optimization*。Wiley，2022年（引用见第161页）。201. W. B. Powell，*Approximate Dynamic Programming: Solving the Curses of Dimensionality*，第2版。Wiley，2011年（引用见第161页）。202. M. L. Puterman，*Markov Decision Processes: Discrete Stochastic Dynamic Programming*。Wiley，2005年（引用见第133页）。203. M. L. Puterman 和 M. C. Shin，“用于折扣马尔可夫决策问题的修正策略迭代算法”，*Management Science*，第24卷，第11期，第1127–1137页，1978年（引用见第141页）。204. J. Robinson，“一种求解博弈的迭代方法”，*Annals of Mathematics*，第296–301页，1951年（引用见第505页）。205. R. W. Robinson，“计数标记无环有向图”，载于 *Ann Arbor Conference on Graph Theory*，1973年（引用见第99页）。206. S. Ross 和 J. A. Bagnell，“模仿学习的高效归约”，载于 *International Conference on Artificial Intelligence and Statistics* (*AISTATS*)，2010年（引用见第358页）。207. S. Ross 和 B. Chaib-draa，“AEMS：一种用于大型POMDP中近似策略改进的随时在线搜索算法”，载于 *International Joint Conference on Artificial Intelligence* (*IJCAI*)，2007年（引用见第464页）。208. S. Ross、G. J. Gordon 和 J. A. Bagnell，“将模仿学习和结构化预测归约为无遗憾在线学习”，载于 *International Conference on Artificial Intelligence and Statistics* (*AISTATS*)，第15卷，2011年（引用见第358页）。209. S. Ross、J. Pineau、S. Paquet 和 B. Chaib-draa，“POMDP的在线规划算法”，*Journal of Artificial Intelligence Research*，第32卷，第663–704页，2008年（引用见第453页）。210. D. E. Rumelhart、G. E. Hinton 和 R. J. Williams，“通过反向传播误差学习表示”，*Nature*，第323卷，第533–536页，1986年（引用见第585页）。211. G. A. Rummery 和 M. Niranjan，“使用连接主义系统的在线Q学习”，剑桥大学，技术报告 CUED/F-INFENG/TR 166，1994年（引用见第338页）。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

212. S. Russell 和 P. Norvig，*Artificial Intelligence: A Modern Approach*，第4版。Pearson，2021年（引用自第2、116页）。213. D. Russo、B. V. Roy、A. Kazerouni、I. Osband 和 Z. Wen，“汤普森采样教程”，*Foundations and Trends in Machine Learning*，第11卷，第1期，第1-96页，2018年（引用自第306页）。214. A. Ruszczyński，“马尔可夫决策过程的风险规避动态规划”，*Mathematical Programming*，第125卷，第2期，第235-261页，2010年（引用自第282页）。215. T. Salimans、J. Ho、X. Chen、S. Sidor 和 I. Sutskever，“进化策略作为强化学习的可扩展替代方案”，2017年。arXiv: 1703.03864v2（引用自第224页）。216. T. Schaul、J. Quan、I. Antonoglou 和 D. Silver，“优先经验回放”，载于*International Conference on Learning Representations* (*ICLR*)，2016年（引用自第345页）。217. P. J. H. Schoemaker，“期望效用模型：其变体、目的、证据与局限”，*Journal of Economic Literature*，第20卷，第2期，第529-563页，1982年（引用自第111页）。218. J. Schulman、S. Levine、P. Moritz、M. Jordan 和 P. Abbeel，“信赖域策略优化”，载于*International Conference on Machine Learning* (*ICML*)，2015年（引用自第254页）。219. J. Schulman、P. Moritz、S. Levine、M. Jordan 和 P. Abbeel，“使用广义优势估计的高维连续控制”，载于*International Conference on Learning Representations* (*ICLR*)，2016年。arXiv: 1506.02438v6（引用自第269页）。220. J. Schulman、F. Wolski、P. Dhariwal、A. Radford 和 O. Klimov，“近端策略优化算法”，2017年。arXiv: 1707.06347v2（引用自第257页）。221. S. Seuken 和 S. Zilberstein，“用于Dec-POMDP的内存有限动态规划”，载于*International Joint Conference on Artificial Intelligence* (*IJCAI*)，2007年（引用自第550页）。222. S. Seuken 和 S. Zilberstein，“不确定性下分散决策的形式模型与算法”，*Autonomous Agents and Multi-Agent Systems*，第17卷，第2期，第190-250页，2008年（引用自第549页）。223. R. D. Shachter，“评估影响图”，*Operations Research*，第34卷，第6期，第871-882页，1986年（引用自第119页）。224. R. D. Shachter，“概率推理与影响图”，*Operations Re- search*，第36卷，第4期，第589-604页，1988年（引用自第119页）。225. R. D. Shachter，“信息价值的高效计算”，载于*Conference on Uncertainty in Artificial Intelligence* (*UAI*)，1999年（引用自第119页）。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

226. A. Shaiju 和 I. R. Petersen，“离散时间LQR、LQG、LEQG及极小极大LQG最优控制问题的公式”，*IFAC Proceedings Volumes*，第41卷，第2期，第8773-8778页，2008年（引用自第148、149页）。227. G. Shani、J. Pineau 和 R. Kaplow，“基于点的POMDP求解器综述”，*Autonomous Agents and Multi-Agent Systems*，第27卷，第1-51页，2012年（引用自第432页）。228. C. E. Shannon，“通信的数学理论”，*Bell System Technical Journal*，第27卷，第4期，第623-656页，1948年（引用自第565页）。229. L. S. Shapley，“随机博弈”，*Proceedings of the National Academy of Sciences*，第39卷，第10期，第1095-1100页，1953年（引用自第517页）。230. Z. R. Shi、C. Wang 和 F. Fang，“人工智能服务于社会公益：综述”，2020年。arXiv: 2001.01818v1（引用自第12页）。231. Y. Shoham 和 K. Leyton-Brown，*Multiagent Systems: Algorithmic, Game Theoretic, and Logical Foundations*。剑桥大学出版社，2009年（引用自第493、515页）。232. D. Silver、G. Lever、N. Heess、T. Degris、D. Wierstra 和 M. Riedmiller，“确定性策略梯度算法”，载于 *International Conference on Machine Learning* (*ICML*)，2014年（引用自第272页）。233. D. Silver、J. Schrittwieser、K. Simonyan、I. Antonoglou、A. Huang、A. Guez、T. Hubert、L. Baker、M. Lai、A. Bolton 等，“无需人类知识掌握围棋游戏”，*Nature*，第550卷，第354-359页，2017年（引用自第276页）。234. D. Silver 和 J. Veness，“大型POMDP中的蒙特卡洛规划”，载于 *Advances in Neural Information Processing Systems* (*NIPS*)，2010年（引用自第457页）。235. S. Singh、M. Kearns 和 Y. Mansour，“一般和博弈中梯度动力学的纳什收敛性”，载于 *Conference on Uncertainty in Artificial Intelligence* (*UAI*)，2000年（引用自第509页）。236. S. P. Singh 和 R. S. Sutton，“带替换资格迹的强化学习”，*Machine Learning*，第22卷，第123-158页，1996年（引用自第612页）。237. S. P. Singh 和 R. C. Yee，“近似最优值函数损失的上界”，*Machine Learning*，第16卷，第3期，第227-233页，1994年（引用自第142页）。238. R. D. Smallwood 和 E. J. Sondik，“有限时域上部分可观测马尔可夫过程的最优控制”，*Operations Research*，第21卷，第5期，第1071-1088页，1973年（引用自第617页）。239. T. Smith 和 R. G. Simmons，“POMDP的启发式搜索值迭代”，载于 *Conference on Uncertainty in Artificial Intelligence* (*UAI*)，2004年（引用自第442页）。240. E. Sonu、Y. Chen 和 P. Doshi，“代理群体匿名性下的决策论规划”，*Journal of Artificial Intelligence Research*，第59卷，第725-770页，2017年（引用自第534页）。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

241. M. T. J. Spaan 和 N. A. Vlassis，“Perseus：基于随机点的POMDP值迭代，” *Journal of Artificial Intelligence Research*，第24卷，第195–220页，2005年（引自第433页）。242. J. C. Spall，*Introduction to Stochastic Search and Optimization*。Wiley，2003年（引自第234页）。243. D. O. Stahl 和 P. W. Wilson，“关于玩家对其他玩家模型的实验证据，” *Journal of Economic Behavior & Organization*，第25卷，第3期，第309–327页，1994年（引自第504页）。244. G. J. Stigler，“效用理论的发展。I，” *Journal of Political Economy*，第58卷，第4期，第307–327页，1950年（引自第8页）。245. M. J. A. Strens，“强化学习的贝叶斯框架，” 在 *International Conference on Machine Learning* (*ICML*)，2000年（引自第330页）。246. F. P. Such，V. Madhavan，E. Conti，J. Lehman，K. O. Stanley，和 J. Clune，“深度神经进化：遗传算法是训练深度神经网络进行强化学习的有竞争力的替代方案，” 2017年。arXiv: 1712.06567v3（引自第215页）。247. Z. N. Sunberg 和 M. J. Kochenderfer，“具有连续状态、动作和观察空间的POMDP在线算法，” 在 *International Conference on Automated Planning and Scheduling* (*ICAPS*)，2018年（引自第457页）。248. R. Sutton，“通过时间差分方法学习预测，” *Machine Learning*，第3卷，第1期，第9–44页，1988年（引自第341页）。249. R. S. Sutton，“Dyna，一个集学习、规划和反应于一体的架构，” *SIGART Bulletin*，第2卷，第4期，第160–163页，1991年（引自第318页）。250. R. S. Sutton 和 A. G. Barto，*Reinforcement Learning: An Introduction*，第2版。MIT Press，2018年（引自第9、335页）。251. U. Syed 和 R. E. Schapire，“从学徒学习到分类的归约，” 在 *Advances in Neural Information Processing Systems* (*NIPS*)，2010年（引自第357页）。252. C. Szepesvári 和 T. Lattimore，*Bandit Algorithms*。剑桥大学出版社，2020年（引自第299页）。253. D. Szer，F. Charpillet，和 S. Zilberstein，“MAA*：解决去中心化POMDP的启发式搜索算法，” 在 *Conference on Uncertainty in Artificial Intelligence* (*UAI*)，2005年（引自第550页）。254. W. R. Thompson，“关于一个未知概率超过另一个的可能性，鉴于两个样本的证据，” *Biometrika*，第25卷，第3/4期，第285–294页，1933年（引自第306页）。255. S. Thrun，“概率机器人学，” *Communications of the ACM*，第45卷，第3期，第52–57页，2002年（引自第10页）。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

256. S. Thrun、W. Burgard 和 D. Fox，*Probabilistic Robotics*。MIT Press，2006年（引用见第379、385、394页）。257. K. S. Trivedi 和 A. Bobbio，*Reliability and Availability Engineering*。Cambridge University Press，2017年（引用见第92页）。258. A. M. Turing，“Intelligent Machinery”，National Physical Laboratory，Report，1948年（引用见第9页）。259. A. Tversky 和 D. Kahneman，“The Framing of Decisions and the Psychology of Choice”，*Science*，第211卷，第4481期，第453–458页，1981年（引用见第123、124页）。260. W. Uther 和 M. Veloso，“Adversarial Reinforcement Learning”，Carnegie Mellon University，Tech. Rep. CMU-CS-03-107，1997年（引用见第521页）。261. R. Vanderbei，*Linear Programming, Foundations and Extensions*，第4版。Springer，2014年（引用见第147页）。262. H. van Hasselt，“Double Q-Learning”，载于*Advances in Neural Information Processing Systems* (*NIPS*)，2010年（引用见第338页）。263. S. Vasileiadou、D. Kalligeropoulos 和 N. Karcanias，“Systems, Modelling and Control in Ancient Greece: Part 1: Mythical Automata”，*Measurement and Control*，第36卷，第3期，第76–80页，2003年（引用见第7页）。264. J. von Neumann 和 O. Morgenstern，*Theory of Games and Economic Behavior*。Princeton University Press，1944年（引用见第112页）。265. A. Wächter 和 L. T. Biegler，“On the Implementation of an Interior-Point Filter Line-Search Algorithm for Large-Scale Nonlinear Programming”，*Mathematical Programming*，第106卷，第1期，第25–57页，2005年（引用见第205页）。266. C. J. C. H. Watkins，“Learning from Delayed Rewards”，博士学位论文，University of Cambridge，1989年（引用见第336、341页）。267. D. J. White，“A Survey of Applications of Markov Decision Processes”，*Journal of the Operational Research Society*，第44卷，第11期，第1073–1096页，1993年（引用见第134页）。268. M. Wiering 和 M. van Otterlo 编，*Reinforcement Learning: State of the Art*。Springer，2012年（引用见第299页）。269. D. Wierstra、T. Schaul、T. Glasmachers、Y. Sun、J. Peters 和 J. Schmidhuber，“Natural Evolution Strategies”，*Journal of Machine Learning Research*，第15卷，第949–980页，2014年（引用见第219、222页）。270. R. J. Williams，“Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning”，*Machine Learning*，第8卷，第229–256页，1992年（引用见第245页）。271. B. Wong，“Points of View: Color Blindness”，*Nature Methods*，第8卷，第6期，第441–442页，2011年（引用见第xxii页）。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

272. J. R. Wright 和 K. Leyton-Brown, “超越均衡：预测标准形式博弈中的人类行为”，载于 *AAAI Conference on Artificial Intelligence* (*AAAI*)，2010年（引用自第505页）。273. J. R. Wright 和 K. Leyton-Brown, “行为博弈论模型：参数分析的贝叶斯框架”，载于 *International Conference on Autonomous Agents and Multiagent Systems* (*AAMAS*)，2012年（引用自第505页）。274. N. Ye、A. Somani、D. Hsu 和 W. S. Lee, “DESPOT：具有正则化的在线POMDP规划”，*Journal of Artificial Intelligence Research*，第58卷，第231-266页，2017年（引用自第459页）。275. B. D. Ziebart、A. Maas、J. A. Bagnell 和 A. K. Dey, “最大熵逆强化学习”，载于 *AAAI Conference on Artificial Intelligence* (*AAAI*)，2008年（引用自第365页）。276. M. Zinkevich, “在线凸规划与广义无穷小梯度上升”，载于 *International Conference on Machine Learning* (*ICML*)，2003年（引用自第509页）。

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

Index

2048问题，610个3
SAT，577

A/B测试，313 绝对齐次，562 吸收，599 抽象类型，638 动作分布，471 动作节点，116 动作轮廓，*see* 联合动作 动作空间，133，599 动作价值函数，139 激活函数，582 演员-评论家方法，267 无环，572 自适应注入，394 特别探索，301 可采纳，606 可采纳启发式，197 优势，245 优势函数，139 对抗分析，291 对抗学习，369，597 对手，291 智能体，1 飞机防撞问题，614 几乎石头剪刀布，515 AlphaGo Zero，276

alpha 向量，411 匿名函数，640 近似动态规划，161

数组推导式，630 人工智能，2 渐近记号，575 异步价值迭代，145 自编码器，592 平均回报，135 平均奖励，135 概率公理，20

反向传播, 585 备份, 432 反向归纳值迭代, 145 巴拿赫不动点定理, *see* 压缩映射定理 带宽, 82 基线, 241 基函数, 172 批次, 345, 582 批量强化学习, 299 贝叶斯法则, 30 贝叶斯自适应马尔可夫决策过程, 329 贝叶斯学习, 509 贝叶斯网络, 32 贝叶斯参数学习, 75

贝叶斯强化学习，326 贝叶斯评分，98 BDe，104 BDeu，104 行为克隆，6，355 行为博弈论，504 行为策略，530 信念，379 信念传播，53 信念单纯形，381 信念空间，381 信念状态，306 信念状态马尔可夫决策过程，407 信念向量，381 贝尔曼备份，141 贝尔曼期望方程，138 贝尔曼最优方程，142 贝尔曼残差，142 贝尔曼更新，*see* 贝尔曼备份 伯努利臂，*see* 二元臂 最佳动作最佳状态上界，429 最佳动作最差状态下界，431 最佳优先搜索，604 最佳响应，495 贝塔分布，78 偏差，239 大O符号，575

双线性插值，167 二元多臂老虎机，299 二元变量，20 二项多臂老虎机，*see* 二元老虎机位，566 盲下界，431 布尔，627 布尔可满足性，578 瓶颈，592 有界策略迭代，475 碗形，564 盒，27 分支定界，185，456，601 广播，633 预热期，61

可调用，641 倒立摆问题，611 级联错误，357 灾难性遗忘，345 接球问题，619 因果网络，33 删失数据，92 确定性效应，122 确定性等价，150，419 链式法则，33 机会节点，116 切比雪夫范数，563 棋盘范数，563 子节点，572 钳位，257 类条件分布，48 分类，48 子句，53 闭环规划，200 对补集封闭，561 对可数并集封闭，561 协作捕食者-猎物六边形世界，625 冒号表示法，20 组件策略，361

复合类型，638 计算复杂度，575 计算通用，579 凹的，565 具体类型，638 条件分布，29 条件边，118 条件高斯，31 条件独立，35 条件线性高斯，31 条件计划，408 条件概率，29 置信区间，285 共轭先验，398 联结主义，10 一致的，606 连续熵，*see* 微分熵 连续概率分布，21 压缩，*see* 压缩映射 压缩映射，136，570 压缩映射定理，570 收缩者，*see* 压缩映射 控制器，471 控制理论，2 凸组合，564 凸函数，564 凸包，168 凸集，564 卷积层，587 协调图，547 相关均衡，498，501 相关联合策略，498 成本敏感分类，373 可数可加性，561 协方差矩阵，28 Coxeter-Freudenthal-Kuhn三角剖分，168 交叉熵，219，566 交叉熵方法，218 哭泣婴儿问题，382，615

累积分布函数，21 个周期，572

d-分离，35 DAgger，*see* 数据集聚合 数据插补，84 数据集聚合，358 Dec-MDP，*see* 去中心化马尔可夫决策过程 Dec-POMDP，*see* 去中心化部分可观测马尔可夫决策过程 衰减因子，567 衰减步长因子，567 去中心化马尔可夫决策过程，546 去中心化部分可观测马尔可夫决策过程，16，545 决策网络，116 决策网络，14 决策理论，111 决策树，25 解码器，593 深度学习，581 深度神经网络，582 深度强化学习，344 深度优先搜索，183 理性深度，504 描述性理论，122 DESPOT，*see* 确定化稀疏部分可观测树 确定性最佳响应，495 确定性策略，135 确定性策略梯度，272 确定性变量，32 确定化信念树，459 确定化稀疏部分可观测树，459 确定化稀疏树搜索，459 确定化矩阵，460 诊断测试，121

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

字典, 637 微分熵, 566 边际效用递减, 115 有向无环图, 32 有向无环图模式, 104 有向探索, 303 有向图, 572 有向图搜索, 99 有向路径, 572 直接采样, 54 狄利克雷分布, 80 折现回报, 135 折现访问分布, 256 折现因子, 135 离散概率分布, 20 离散状态滤波器, 381 离散时间黎卡提方程, 150 判别器, 369, 597 调度, 642 距离度量, 163, 562 占优策略, 497 占优策略均衡, 497 支配, 291, 478 被支配, 412 双重渐进拓宽, 197 双重 Q-学习, 338 Dyna, 318 动态规划, 136, 549, 604

E步，*see*期望步边缘，572 ϵ-贪婪探索，303 EKF，*see*扩展卡尔曼滤波器资格迹，341 精英样本，218 EM，*see*期望最大化嵌入，592 编码器，593 熵，365，566 本质图，104 欧几里得范数，563

评估模型，289 事件空间，562 证据，29 证据变量，43 期望最大化，87 期望步，87 期望效用，116 经验回放，345 解释消除，36 梯度爆炸，590 利用，299 探索，299 探索奖励，189 探索性信念扩展，440 先探索后承诺探索，303 指数分布，38，92 指数加权平均，270 指数效用，115 扩展卡尔曼滤波，385

因子, 25 因子条件化, 44 因子化Dec-POMDP, 546 因子边缘化, 44 因子乘积, 44 快速信息界, 429 特征, 587 特征期望, 364 特征, 172 前馈网络, 582 虚拟博弈, 505, 521 有限差分, 231 有限视界, 134 有限状态控制器, 471 微积分第一基本定理,

568
Fisher信息矩阵，254 拟合，*see*学习前向差分，234 前向搜索，183，600 框架效应，122，124

Freudenthal 三角剖分，445 函数，640 函数边，118

GAIL，*see* 生成对抗模仿学习博弈论，493 伽马函数，78 间隙启发式搜索，460 门控循环单元，592 高斯-赛德尔值迭代，145 高斯分布，22 高斯核，165 高斯混合模型，23 广义优势估计，269 广义无穷小梯度上升，509 世代，218 生成对抗模仿学习，369 生成模型，183，593 生成器，597 遗传算法，103，215 遗传局部搜索，103 吉布斯采样，60 GIGA，*see* 广义无穷小梯度上升 Gittins分配指数，309 全局近似，163 梯度上升，249，509，526，567 梯度裁剪，250 梯度缩放，250 图，572 贪婪动作，301 贪婪包络，200 贪婪策略，139 GRU，*see* 门控循环单元

停机问题, 610 停机问题, 579 启发式, 197 启发式搜索, 197, 550, 604

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

启发式搜索值迭代，442 六边形世界问题，609 隐藏变量，43 分层 softmax，504 爬山法，100 事后优化，207 历史，135，457 历史树，459 Hooke-Jeeves 方法，215 HSVI，*see* 启发式搜索值迭代混合规划，185

I-POMDP，*see* 交互式POMDP 不可区分者的同一性，562 模仿学习，355 不道德v结构，104 重要性采样，256, 287, 570 不完全，82 增量估计，335 独立，25 独立同分布，71 独立参数，21 推理，43 无限视界，134 无穷小梯度上升，509 影响图，*see* 决策网络 信息边，118 信息内容，565 信息收集，429 知情搜索，604 初始状态分布，213, 282 交互不确定性，2 交互式POMDP，534 区间估计，*see* 分位数探索 区间探索，*see* 分位数探索 逆累积分布函数，*see* 分位数函数

逆强化学习, 361 各向同性高斯分布, 224 迭代上升, 249 迭代最佳响应, 503, 550

JESP，*see* 基于联合均衡的策略搜索 联合动作，493 联合动作空间，493 联合分布，24 基于联合均衡的策略搜索，550 联合完全可观测性，546 联合观测，533 联合观测空间，533 联合策略，494 联合奖励，493 联合奖励函数，493 连接树算法，53

k-nearest neighbors，163 K2，100 卡尔曼滤波，383 卡尔曼增益，385 核，587 核密度估计，82 核函数，82, 164 核平滑，164 关键字参数，641 柯尔莫哥洛夫公理，562 克罗内克δ函数，329 库尔贝克-莱布勒散度，253，*see* 相对熵

标记启发式搜索，197 地标，380 拉普拉斯分布，91 潜在空间，592 潜在变量，87 全概率公式，24 学习，71 学习曲线，337

学习率, 336, 567 似然比, 234 似然加权采样, 57 线性组合, 575 线性动力学, 148 线性函数逼近, 163 线性高斯, 31 线性插值, 167 期望的线性, 242 线性化, 386 线性规划, 147 线性二次高斯, 419 线性二次调节器, 148 线性回归, 172, 234 线搜索, 254 文字, 53 局部近似, 163 局部最优, 250 局部最优解, 102 局部搜索, 100, 215 对数效用, 115 对数导数技巧, 221 逻辑合取, 53 逻辑析取, 53 逻辑否定, 53 logit-level-*k*, *see* 层次softmax logit模型, 32 logit响应, *see* softmax响应 对数似然, 72 长短期记忆, 592 前瞻, 136 循环, 644 循环信念传播, 53 损失函数, 581 彩票, 112 LQG, *see* 线性二次高斯 LQR, *see* 线性二次调节器 LSTM, *see* 长短期记忆

M步，*see*最大化步骤机器学习，71

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

机器替换问题，617 边缘，365 边缘的，24 边缘化，44 边缘模式，85 马尔可夫假设，133 马尔可夫链蒙特卡洛，60 马尔可夫决策过程，14, 131, 133 马尔可夫等价类，104 马尔可夫等价，104 马尔可夫博弈，16, 517 矩阵，633 矩阵博弈，*see* 标准形博弈 最大化步骤，87 最大后验，77 最大熵逆强化学习，365 最大似然估计，71, 317 最大似然参数学习，71 最大间隔逆强化学习，361 最大范数，563 MBDP，*see* 记忆受限动态规划 MDP，*see* 马尔可夫决策过程，*see* 马尔可夫决策过程，*see* 马尔可夫决策过程 均值向量，28 可测集，561 测度，561 测度空间，561 模因算法，103 记忆化，604 记忆，589 记忆受限动态规划，550 度量，562 度量空间，562 MG，*see* 马尔可夫博弈 极小极大，204

镜像采样，224 缺失，82 随机缺失，84 缺失机制，84 混合策略，494 混合模型，23 MMA*，*see* 多智能体 A* MMDP，*see* 多智能体 MDP 模式，77 无模型强化学习，335 模型预测控制，200 模型不确定性，2 改进策略迭代，141 蒙特卡洛估计，569 蒙特卡洛方法，11，54 蒙特卡洛策略评估，214 蒙特卡洛树搜索，187，457 蒙特卡洛值迭代，475 最可能失败，293 山地车问题，612 MPO MDP，*see* 多智能体 POMDP 多智能体 A*，550 多智能体 MDP，548 多智能体 POMDP，548 多臂老虎机问题，299 多照顾者哭泣婴儿问题，624 多预测模型预测控制，207 多线性插值，168 多模态，23 多变量，573 多变量分布，24 多变量高斯分布，28 多变量高斯混合模型，28 多变量乘积分布，27 多变量均匀分布，27 变异，631

命名函数，640 命名元组，637 纳什均衡，498, 537 纳什 *Q*-学习，526 nat，566 自然，566 自然进化策略，219 自然梯度，253 ND-POMDP，*see* 网络分布式部分可观测马尔可夫决策过程 最近邻，163 最近邻插补，85 邻居，572 网络分布式部分可观测马尔可夫决策过程，547 神经网络，174, 581 神经网络回归，174 NEXP完全，546 自然语言处理，*see* 非线性规划 节点，471, 572 非线性规划，478, 551 非负性，561 非参数，82, 379 非平稳马尔可夫策略，530 正态分布，*see* 高斯分布 正规形式博弈，493 归一化常数，49 归一化效用函数，113 规范理论，122 赋范向量空间，562 NP完全，577 NP难，52, 577 NP，577

观测，1，379 观测独立性，547 观测空间，380 观测-行动循环，1

朴素贝叶斯，48

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

观察-行动循环，*see*观察-行动循环离策略，340 在策略，340 单臂老虎机，299 单步前瞻，412 在线规划，181 开环规划，200 运筹学，12 机会主义，102 最优策略，136 最优子结构，604 最优价值函数，136 不确定性下的乐观主义，305 订单，575 结果不确定性，2 过拟合，585

P，577 包，645 参数，21，161 参数化策略，213 参数正则化，585 参数调优，581 参数化，379 参数化表示，161 参数化类型，639 父节点，572 帕累托曲线，*see* 帕累托前沿 帕累托有效，*see* 帕累托最优 帕累托前沿，291 帕累托最优，291 部分有向图，104 部分可观测马尔可夫决策过程，15，377 部分可观测马尔可夫博弈，16，533 部分可观测蒙特卡洛规划，457 部分可观测随机博弈，533 粒子，390

粒子剥夺，390 粒子滤波，390 带拒绝的粒子滤波，390 粒子注入，394 路径，572 性能指标，281 分段均匀密度，24 规划，6 规划模型，289 基于点的值迭代，432 杆平衡问题，611 策略，135 策略评估，136 策略迭代，140 策略损失，142 策略搜索，213 POMCP，*see* 部分可观测蒙特卡洛规划 POMDP，*see* 部分可观测马尔可夫决策过程 POMG，*see* 部分可观测马尔可夫博弈种群，215 POSG，*see* 部分可观测随机博弈正仿射变换，113 正定，28，564 半正定，564 后验分布，43 后验采样，306，330 势博弈，503 幂效用，115 PPAD 完全，498 PPO，*see* 近端策略优化精度参数，305，497 捕食者-猎物六边形世界问题，623 预测步骤，383 偏好引出，114 最大熵原理，368 最大期望效用原理，116

先验，48 优先扫描，321 囚徒困境，494，621 概率公理，562 概率密度函数，21 概率分布，20 概率质量函数，20 概率测度，562 概率单纯形，381 概率空间，562 渐进拓宽，197 正比于，49 提议分布，287 前景理论，122 近端策略优化，257 剪枝，185，412 伪计数，79f 伪逆，174 PSPACE完全，427，577 PSPACE难，577 PSPACE，577 纯策略，494

Q-function、*see* 动作价值函数 *Q* 学习、336 QCLP、*see* 二次约束线性规划 QMDP、427 二次约束线性规划、481 二次奖励、148 二次效用、115 量子级别-*k*、*see* 分层 softmax 量子响应、*see* softmax 响应分位数、305 分位数探索、305 分位数函数、21 查询变量、43

R-MAX，323 随机信念扩展，440

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

随机化基于点的值迭代，433 随机化概率匹配，*see* 后验采样随机重启，102 排名塑造，221 理性学习，509 理性偏好，112 可达状态空间，181 滚动时域规划，181 循环神经网络，589 递归贝叶斯估计，381 REINFORCE，245 强化学习，7, 15, 297 相对熵，567 相对标准误差，285 重复博弈，493 经验回放，345 表示学习，592 响应，494 响应策略，519 限制步长，251 回报，134, 599 反向累积，585 奖励函数，133 奖励独立性，547 奖励塑造，343 未来奖励，240 鲁棒动态规划，289 鲁棒模型预测控制，204 石头剪刀布，495, 621 展开策略，183

锯齿上界，436 情景，459 分数等价，104 搜索分布，218 搜索图，600 搜索问题，599 搜索树，183，600 灵敏度，253 顺序交互演示，358 香农信息，565 塑造函数，343 sigma 点，387 sigmoid，32 简单决策，111 简单游戏，493 简单调节器问题，613 单纯形，168 单纯形插值，168 模拟退火，102 同时扰动随机逼近，234 奇异值分解，174 SMILe，*see* 随机混合迭代学习 平滑虚拟博弈，509 softmax，583 softmax 响应，497 softmax 响应策略，520 softmax 策略，303 软阈值，32 稀疏奖励，341 稀疏采样，187 球面高斯，*see* 各向同性高斯 splat，642 散布参数，388 标准基，232 标准差，22 标准误差，285

标准形式博弈，*see* 正规形式博弈状态空间，133，599状态转移模型，133状态不确定性，2平稳，133，520平稳马尔可夫完美均衡，520平稳策略，135步长因子，567随机博弈，*see* 马尔可夫博弈随机混合迭代学习，358随机策略，135压力测试，289严格凹，565严格凸，565步幅，587字符串，629最优策略下可达空间的逐次逼近，440后继分布，471和积算法，53和积变量消元，49监督学习，6支持，22f代理约束，256代理目标，256符号，630对称性，562

残值，617 样本空间，562 Sarsa，338 SARSOP，*see* Successive Approximations of the Reachable Space under Optimal Policies 可满足，578 锯齿形启发式搜索，442

禁忌搜索，103 目标参数化，274 出租车范数，563 泰勒近似，569 泰勒展开，568 泰勒级数，568 时间差分误差，336 时间差分残差，268 时序逻辑，293 终端奖励，617

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com

三元运算符，643 薄，61 汤普森采样，*see* 后验采样 拓扑排序，55 迹技巧，159 交易分析，291 权衡曲线，291 训练，581 轨迹，213 轨迹奖励，214 转移独立性，547 传递性，19 置换表，604 旅行者困境，622 三角不等式，562 TRPO，*see* 信赖域策略优化 截断高斯分布，23 信赖域，254 信赖域策略优化，254 元组，636 图灵完备，*see* 计算通用

图灵机，575

UCB1探索，305 UCB1探索启发式，187 UKF，*see*无迹卡尔曼滤波不确定性集，204 不可判定，579 欠定，585 无向探索，301 无向路径，572 均匀分布，21 单峰，23 单变量，573 单变量分布，24 普遍可比性，19 无迹卡尔曼滤波，387 无迹变换，387 无监督，593 更新步骤，385 置信上界，概率性，276 置信上界探索，*see*分位数探索效用，112

效用获取, 114 效用节点, 116 效用理论, 14, 111

价值函数, 136 价值函数近似, 161 值迭代, 141, 416 信息价值, 119 梯度消失, 590 方差, 239 变分自编码器, 593 向量, 630 向量空间, 562 顶点, *see* 节点 冯·诺伊曼-摩根斯坦公理, 112

权重正则化，585 个权重，174

零和博弈，494

© 2022 Massachusetts Institute of Technology, shared under a Creative Commons CC-BY-NC-ND license.
2024-02-06 20:54:49-08:00, comments to bugs@algorithmsbook.com